Алгоритмы и игры на Си

С абсолютного нуля — учимся программировать, разбирая каждую строку

Учебник

Содержание

Введение. Зачем учить Си через алгоритмы и игры **1**
Почему именно Си, если есть языки попроще . . . 1
Чего в этой книге не будет . . . 2
Что ты будешь уметь к концу . . . 3
Что понадобится . . . 4
Как читать эту книгу . . . 4
Сколько это займёт . . . 5
Что делать, если застрял . . . 5

Глава 1. Компилятор и первая программа **7**
Что мы сделаем . . . 7
Что такое компилятор . . . 7
Ставим компилятор . . . 8
macOS . . . 8
Linux . . . 9
Windows . . . 9
Шаг 1. Пишем первое, что приходит в голову . . . 10
Шаг 2. Делаем, как сказал компилятор . . . 11
Шаг 3. Возвращаем код завершения . . . 12
Полная программа . . . 12
Разбираем построчно . . . 13
`#include <stdio.h>` . . . 13
Пустая строка . . . 13
`int main(void)` . . . 13
`{` и `}` . . . 14
`printf("Привет, мир!\n");` . . . 14
`return 0;` . . . 15
Что происходит при компиляции: четыре фазы . . . 15
Фаза 1. Препроцессор — `cc -E` . . . 15
Фаза 2. Компилятор — `cc -S` . . . 16
Фаза 3. Ассемблер — `cc -c` . . . 18
Фаза 4. Компоновщик — линковка . . . 18
Включаем предупреждения — привычка на всю книгу . . . 20
Частые ошибки . . . 21
Забыл `;` в конце строки . . . 21
Опечатка в имени функции . . . 22
Забыл закрывающую `}` . . . 22
Забыл закрывающую кавычку . . . 22

Перепутал кавычки . . . 23
Запускаешь без `./` . . . 23
Компилятор не находит файл . . . 23
Файл не сохранён . . . 24
Упражнения . . . 24
Что мы получили . . . 25

Глава 2. Минимум Си: переменные, ввод, ветвления, циклы, функции 26
Что мы сделаем . . . 26
Разбираем задачу руками . . . 27
Шаг 1. Переменная — коробка для значения . . . 27
Шаг 2. Типы: сколько места и что туда влезает . . . 28
`double` считает приближённо . . . 28
Смотрим размеры своими глазами . . . 29
Что будет, если число не влезло . . . 31
`char` — это маленькое целое . . . 33
Шаг 3. `printf` — разбираем как функцию . . . 34
Если спецификатор не совпал с типом . . . 34
Ширина поля . . . 35
Шаг 4. `scanf` — спрашиваем у пользователя . . . 35
Что будет, если забыть `&` . . . 36
`scanf` возвращает число, и его надо проверять . . . 37
Буква вместо числа: почему программа зависает . . . 39
Настоящий фикс . . . 39
Шаг 5. Арифметика и её ловушки . . . 41
`i++` и `++i` — не одно и то же . . . 43
Шаг 6. Принимаем решения — `if` и `else` . . . 44
`=` вместо `==` — классический баг . . . 45
Сравнения и логика . . . 46
Шаг 7. Повторяем — циклы . . . 47
`while` . . . 47
`for` . . . 47
`break` и `continue` . . . 48
`do-while` . . . 49
Шаг 8. Свои функции . . . 49
Прототип: функция должна быть объявлена до вызова . . . 50
Локальные переменные и область видимости . . . 51
Аргументы передаются по значению . . . 52
`void` — функция, которая ничего не возвращает . . . 53
Полная программа: таблица умножения . . . 53
Как это работает . . . 55
Проверяем . . . 56
Частые ошибки . . . 57
Упражнения . . . 58
Что мы получили . . . 58

Глава 3. Массивы, строки, указатели и память 60
Что мы сделаем . . . 60

Часть 1. Массив — много коробок в ряд . . . 61
Разбираем задачу руками . . . 61
Первая программа с массивом . . . 61
Перебор циклом: сумма и среднее . . . 62
Выход за границу массива — покажем на живом примере . . . 63
Санитайзер — тот, кто ловит такое за руку . . . 65
`sizeof` — откуда взять длину массива . . . 66
Массив «распадается» в указатель . . . 67
Заполняем массив вводом . . . 69
Полная программа `code/03-arrays.c` . . . 70
Часть 2. Строки — это массивы символов . . . 72
Что будет без `'\0'` — проверим . . . 72
`sizeof` против `strlen` . . . 73
Функции из `<string.h>` . . . 74
Почему `==` не сравнивает строки . . . 75
Чтение строки от пользователя . . . 76
Почему `gets` запрещён . . . 78
Полная программа `code/03-strings.c` . . . 78
Часть 3. Указатели — переменные, хранящие адрес . . . 80
Адрес и разыменование . . . 81
Звёздочка в объявлении и звёздочка в выражении . . . 82
Зачем это нужно: функция, меняющая аргумент . . . 82
Арифметика указателей: `a[i]` — это `*(a + i)` . . . 83
`NULL` — указатель в никуда . . . 84
Полная программа `code/03-pointers.c` . . . 86
Часть 4. Динамическая память . . . 88
Полная программа `code/03-dynamic.c` . . . 89
Три способа испортить кучу . . . 91
Утечка памяти . . . 94
Замечание про русские буквы . . . 95
Проверяем . . . 96
Частые ошибки . . . 96
Упражнения . . . 98
Что мы получили . . . 99

Глава 4. Числа: чётность, сумма цифр, переворот числа, факториал, FizzBuzz 100
Что мы сделаем . . . 100
Мостик: чётность через остаток . . . 101
Часть 1. Сумма цифр и переворот числа . . . 101
Разбираем задачу руками . . . 101
Шаг 1. Наивная версия — считаем сумму . . . 102
Шаг 2. Проблема — где взять перевёрнутое число? . . . 103
Шаг 3. Проблема — последняя цифра тратится дважды . . . 103
Шаг 4. Краевые случаи — ноль и минус . . . 104
Шаг 5. Собираем наивную версию целиком и пробуем её сломать . . . 104
Шаг 6. Фикс — считаем в `long long` . . . 107
Полная программа `04-digits.c` . . . 107
Проверяем . . . 109

Сколько цифр в числе . . . 110
Часть 2. Факториал и ловушка переполнения . . . 112
Разбираем задачу руками . . . 112
Шаг 1. Наивная версия на `int` . . . 113
Шаг 2. Проблема — это неправильный ответ! . . . 114
Шаг 3. Фикс — берём `long long` . . . 115
Шаг 4. Где именно ломается каждый тип . . . 116
Шаг 5. Честная программа отказывается врать . . . 117
Полная программа `04-factorial.c` . . . 118
Проверяем . . . 118
Часть 3. FizzBuzz . . . 119
Разбираем задачу руками . . . 119
Шаг 1. Наивная версия — и её баг . . . 119
Шаг 2. Фикс — самую строгую проверку ставим первой . . . 120
Полная программа `04-fizzbuzz.c` . . . 120
Проверяем . . . 121
Частые ошибки . . . 121
Упражнения . . . 122
Что мы получили . . . 124

Глава 5. НОД Евклида, простые числа и решето Эратосфена 125
Что мы сделаем . . . 125
Часть 1. Наибольший общий делитель . . . 126
Разбираем задачу руками . . . 126
Шаг 1. Наивный перебор . . . 126
Шаг 2. Идея Евклида . . . 127
Шаг 3. Евклид через остаток . . . 128
Почему это вообще правильно . . . 129
Шаг 4. Ноль и минус — где обе версии расходятся . . . 130
Полная программа `05-gcd.c` . . . 132
Часть 2. Простое число . . . 135
Разбираем задачу руками . . . 135
Шаг 1. Наивная проверка . . . 136
Шаг 2. Почему достаточно до корня . . . 136
Шаг 3. Проблема — `i * i` само переполняет `int` . . . 138
Шаг 4. Фикс — переносим `i` в другую часть неравенства . . . 139
Полная программа `05-prime.c` . . . 140
Насколько быстрее — замеряем . . . 143
Часть 3. Решето Эратосфена . . . 144
Разбираем задачу руками . . . 144
Шаг 1. Массив-флаги . . . 144
Шаг 2. Вычёркиваем . . . 145
Полная программа `05-sieve.c` . . . 146
Краевые случаи решета . . . 147
Сколько работы делает решето . . . 148
Как это работает: трассировка решета . . . 149
Проверяем . . . 149
Частые ошибки . . . 150

Упражнения . . . 151
Что мы получили . . . 152

Глава 6. Алгоритмы над массивами: минимум/максимум, сумма, среднее, разворот, сдвиг 154
Что мы сделаем . . . 154
Часть 1. Минимум, максимум, сумма, среднее . . . 155
Разбираем задачу руками . . . 155
Шаг 1. Наивная версия — и её баг . . . 155
В чём проблема . . . 157
Шаг 2. Фикс — стартуем с первого элемента . . . 158
Пустой массив: почему `a[0]` — это не всегда безопасно . . . 159
Сумма: сто чисел — уже повод задуматься о типе . . . 159
Среднее: не наступи на целочисленное деление . . . 159
Полная программа `06-minmax.c` . . . 160
Проверяем . . . 162
Часть 2. Разворот массива на месте . . . 163
Разбираем задачу руками . . . 163
Шаг 1. Меняем местами через временную переменную . . . 163
Шаг 2. Сводим индексы навстречу . . . 164
Почему условие именно `i < j` . . . 164
Полная программа `06-reverse.c` . . . 165
Проверяем . . . 166
Часть 3. Подсчёт значения и сдвиг влево . . . 167
Задача А. Сколько раз встречается число . . . 167
Задача Б. Циклический сдвиг влево на 1 . . . 168
Шаг 1. Наивно — и ловушка с границей . . . 168
Шаг 2. Фикс — останавливаем цикл на один шаг раньше . . . 170
Полная программа `06-count-rotate.c` . . . 171
Проверяем . . . 172
Сдвиг на `k` позиций . . . 173
Частые ошибки . . . 176
Упражнения . . . 177
Что мы получили . . . 178

Глава 7. Поиск: линейный и бинарный 180
Что мы сделаем . . . 180
Разбираем задачу руками . . . 180
Шаг 1. Наивный линейный поиск прямо в main . . . 181
Шаг 2. Что не так с наивной версией . . . 182
Шаг 3. Линейный поиск как функция . . . 182
Полная программа: линейный поиск . . . 183
Линейный поиск медленный — и вот почему . . . 184
Шаг 4. Идея бинарного поиска — руками . . . 185
Шаг 5. Почему `mid = low + (high - low) / 2`, а не `(low + high) / 2` . . . 186
Полная программа: бинарный поиск . . . 188
Как это работает: трассировка по шагам . . . 190
Краевые случаи: проверяем функцию на прочность . . . 191

Во сколько раз бинарный быстрее . . . 193
Проверим на секундомере . . . 194
Проверяем . . . 196
Частые ошибки . . . 197
Упражнения . . . 198
Что мы получили . . . 199

Глава 8. Сортировки: пузырёк, выбором, вставками 201
Что мы сделаем . . . 201
Общая заготовка: печать массива . . . 201
Часть 1. Сортировка пузырьком . . . 202
Идея руками . . . 202
Шаг 1. Наивная версия . . . 203
Шаг 2. Проблема: лишняя работа . . . 204
Шаг 3. Фикс: укорачиваем хвост и ставим флаг . . . 204
Полная программа . . . 205
Проверяем . . . 206
Часть 2. Сортировка выбором . . . 207
Идея руками . . . 207
Код . . . 207
Проверяем . . . 209
Часть 3. Сортировка вставками . . . 209
Идея руками . . . 209
Шаг 1. Наивная мысль и сразу аккуратная версия . . . 209
Проверяем . . . 211
Как это работает: чем три сортировки отличаются . . . 211
Стабильность: что происходит с одинаковыми элементами . . . 212
Сложность: почему это медленно и что бывает быстрее . . . 214
Считаем сравнения руками . . . 214
Что значит $O(n^2)$ . . . 214
Проверяем подсчётом: лучший, средний и худший случай . . . 215
Замеряем по-настоящему . . . 217
Быстрее, чем n^2 . . . 218
Быстрая сортировка (quicksort) . . . 218
Сортировка слиянием (merge sort) . . . 219
Готовое решение: `qsort` из `<stdlib.h>` . . . 219
Что `qsort` обещает, а что нет . . . 221
Ловушка: `return a - b` . . . 221
Частые ошибки . . . 223
Упражнения . . . 224
Что мы получили . . . 225

Глава 9. Рекурсия: факториал, Фибоначчи, Ханойские башни 227
Что такое рекурсия . . . 227
Что будет без базы . . . 228
Стек вызовов простыми словами . . . 228
Часть 1. Факториал рекурсивно . . . 228
Пишем код . . . 229

Как это работает: раскрутка вызовов . . . 231
Проверяем . . . 231
Проверяем стек на прочность . . . 232
А теперь фокус: соберём то же самое с `-O2` . . . 234
Часть 2. Числа Фибоначчи и ловушка медленной рекурсии . . . 235
Наивная версия . . . 236
В чём проблема: дерево вызовов . . . 236
Фикс: один проход циклом . . . 239
Часть 3. Ханойские башни . . . 242
Разбираем руками . . . 242
Пишем код . . . 243
Проверяем . . . 245
Сколько получается ходов . . . 245
Частые ошибки . . . 246
Проверяем: все программы главы . . . 247
Упражнения . . . 247
Что мы получили . . . 249

Глава 10. Строковые алгоритмы: длина, переворот, палиндром, слова, анаграммы 250
Что мы сделаем . . . 250
Часть 1. Длина и переворот строки . . . 251
Разбираем задачу руками . . . 251
Шаг 1. Своя функция длины . . . 251
Шаг 2. Наивный переворот — и сюрприз при запуске . . . 252
Шаг 3. Виноват `'\n'` от fgets — убираем его . . . 254
Полная программа: `10-reverse.c` . . . 254
Проверяем . . . 256
Обещанная демонстрация: почему только латиница . . . 257
Часть 2. Палиндром . . . 258
Наивная версия: сравниваем в лоб . . . 258
Знакомимся: `<ctype.h>` . . . 259
Ловушка: почему `(unsigned char)` . . . 260
Идея: сначала чистим, потом сравниваем . . . 262
Полная программа: `10-palindrome.c` . . . 262
Проверяем . . . 264
Часть 3. Подсчёт слов . . . 266
Наивная версия: считаем пробелы . . . 266
Идея: ловим начало слова . . . 267
Полная программа: `10-words.c` . . . 268
Проверяем . . . 270
Часть 4. Анаграммы . . . 270
Идея: считаем частоты букв . . . 271
Наивная версия и две неудачи . . . 271
Выносим подсчёт в функцию . . . 272
Полная программа: `10-anagram.c` . . . 273
Проверяем . . . 275
Частые ошибки . . . 276
Упражнения . . . 278

Что мы получили . . . 279

Глава 11. Динамические структуры данных: растущий массив, стек, очередь, связный список 280

Знакомимся: `struct` — свой составной тип . . . 281
Указатель на структуру и стрелка `->` . . . 282
`typedef` — короткое имя для типа . . . 282
Часть 1. Растущий массив . . . 283
Разбираем руками . . . 283
Почему удвоение, а не «плюс один» . . . 283
Сколько стоит одна вставка: амортизированная O(1) . . . 284
`realloc` — изменить размер уже выделенной памяти . . . 285
Полная программа: `11-dynarray.c` . . . 287
Проверяем . . . 289
Часть 2. Стек (LIFO) . . . 290
Аналогия: стопка тарелок . . . 290
Строим на массиве . . . 290
Наивный pop и что бывает на пустом стеке . . . 290
Как вернуть «стек пуст»: статус плюс указатель на результат . . . 292
Полная программа: `11-stack.c` . . . 293
Проверяем . . . 296
Часть 3. Очередь (FIFO) . . . 297
Аналогия: очередь в магазине . . . 297
Наивная очередь: два индекса, которые ползут только вперёд . . . 297
Кольцевой буфер . . . 299
Полная программа: `11-queue.c` . . . 299
Проверяем . . . 302
Часть 4. Связный список . . . 303
Зачем он, если есть массив . . . 303
Описываем узел через `struct` . . . 303
Добавление в начало — по шагам . . . 304
Удаление головы . . . 304
Удаление последнего . . . 305
Освобождение всего списка и главная ошибка . . . 306
Полная программа: `11-list.c` . . . 308
Проверяем . . . 312
Сколько стоит каждая операция . . . 314
Частые ошибки . . . 314
Упражнения . . . 316
Что мы получили . . . 317

Глава 12. Игра «Угадай число» 318

Что мы сделаем . . . 318
Разбираем задачу руками . . . 319
Шаг 1. Откуда брать случайность . . . 320
Шаг 2. Почему `rand()` повторяется и что такое «засев» . . . 321
Шаг 3. Зерно из текущего времени — `time` . . . 322
Почему `srand` зовут ровно один раз . . . 323

Шаг 4. Сужаем диапазон до 1..100 . . . 324
Шаг 5. Одна попытка угадать . . . 325
Шаг 6. Игровой цикл — спрашиваем, пока не угадал . . . 326
Шаг 7. Считаем попытки . . . 328
Шаг 8. Защита от неправильного ввода . . . 328
Полная программа . . . 331
Как это работает . . . 333
Проверяем . . . 334
Частые ошибки . . . 336
Упражнения . . . 337
Что мы получили . . . 339

Глава 13. Игра «Камень, ножницы, бумага» 340
Что мы сделаем . . . 340
Разбираем задачу руками . . . 340
Шаг 1. Один раунд, выбор компьютера . . . 341
Шаг 2. Превращаем число в слово . . . 342
Шаг 3. Даём имена числам — `enum` . . . 343
Шаг 4. Кто победил — наивный способ . . . 343
Шаг 5. Красивый приём с остатком . . . 344
Не верь на слово — проверь машиной . . . 345
Шаг 6. Ввод игрока с проверкой . . . 347
Полная программа . . . 349
Как это работает . . . 351
Проверяем . . . 352
Честная ли игра: считаем шансы . . . 354
Частые ошибки . . . 356
Упражнения . . . 357
Что мы получили . . . 358

Глава 14. Игра «Виселица» 360
Что мы сделаем . . . 360
Разбираем задачу руками . . . 360
Шаг 1. Список слов и случайный выбор . . . 361
Шаг 2. Показываем слово прочерками . . . 363
Мина под этой функцией . . . 364
Шаг 3. Читаем букву — и ловим коварный перевод строки . . . 365
Приводим к нижнему регистру . . . 366
Проверим этот кусок отдельно . . . 366
Шаг 4. Запоминаем букву и открываем её . . . 368
Шаг 5. Жизни, победа и поражение . . . 369
Шаг 6. Рисуем виселицу . . . 370
Полная программа . . . 372
Как это работает . . . 376
Проверяем . . . 376
Частые ошибки . . . 380
Упражнения . . . 381
Что мы получили . . . 382

Глава 15. Игра «Крестики-нолики» 384
Что мы сделаем . . . 384
Разбираем задачу руками . . . 384
Двумерный массив — таблица в памяти . . . 385
Как это лежит в памяти . . . 386
Перебор двумерного массива — цикл в цикле . . . 386
Шаг 1. Рисуем пустое поле . . . 386
Шаг 2. Добавляем разделители и выносим печать в функцию . . . 387
Шаг 3. Один ход игрока . . . 389
Шаг 4. Проверяем ввод . . . 389
Шаг 5. Проверка победы — выносим в функцию . . . 391
Шаг 6. Проверка ничьей . . . 392
Полная программа . . . 393
Как это работает . . . 397
Проверяем . . . 397
Проверяем все восемь линий, а не три . . . 398
Победа последним ходом . . . 399
Что будет от кривого ввода . . . 400
Сколько всего бывает партий . . . 401
Частые ошибки . . . 404
Упражнения . . . 404
Решения упражнений 5 и 6 . . . 406
Что мы получили . . . 409

Глава 16. Игра «Сапёр» 410
Что мы сделаем . . . 410
Разбираем задачу руками . . . 411
Две карты, наложенные друг на друга . . . 411
Сколько мин вокруг клетки . . . 411
Заливка пустой области . . . 412
Шаг 1. Поле и его печать . . . 413
Шаг 2. Расставляем мины случайно . . . 414
Первый ход не должен убивать . . . 415
Честно ли раскиданы мины . . . 417
Шаг 3. Считаем соседей . . . 417
Шаг 4. Открываем клетку и запускаем заливку . . . 419
Насколько глубоко она уходит . . . 421
Шаг 5. Главный цикл: ход, проигрыш, победа . . . 422
Полная программа . . . 423
Как это работает . . . 428
Проверяем . . . 429
Пробуем сломать игру пальцами . . . 431
Частые ошибки . . . 433
Упражнения . . . 434
Решения упражнений 4 и 6 . . . 435
Что мы получили . . . 437

Глава 17. Игра «Змейка» 438

Что мы сделаем . . . 438
Разбираем задачу руками . . . 439
Как хранить змейку . . . 439
Почему обычный ввод не подходит . . . 439
Шаг 1. Поле и неподвижная змейка . . . 440
Шаг 2. Заставляем змейку ползти . . . 442
Шаг 3. Ловим клавиши без Enter — сырой режим . . . 445
Включаем сырой режим . . . 445
Страховка: терминал надо вернуть в любом случае . . . 449
Ctrl+Z — отдельная история . . . 450
Читаем клавишу . . . 451
Где ломается защита от разворота . . . 453
Шаг 4. Еда, рост и счёт . . . 455
Шаг 5. Столкновения и конец игры . . . 458
Полная программа . . . 460
Как это работает . . . 468
Проверяем . . . 469
Проверяем, что терминал возвращается . . . 470
Частые ошибки . . . 470
Упражнения . . . 472
Решение упражнения 3 . . . 473
Что мы получили . . . 474

Глава 18. Что дальше **475**
Что мы сделаем . . . 475
Что ты теперь умеешь . . . 475
Чего ты пока не умеешь . . . 476
Шаг 1. Разбиваем программу на файлы . . . 476
Заголовочный файл . . . 479
Раздельная компиляция . . . 480
Шаг 2. Makefile . . . 480
Шаг 3. Отладчик: ищем баг в `average` . . . 482
Шаг 4. Файлы: `fopen`, `fgets`, `fclose` . . . 485
Стандарты и платформы, коротко . . . 490
Частые ошибки . . . 491
Упражнения . . . 491
Что мы получили . . . 492

Как готовиться к собеседованию по Си **494**
Как проходит секция кодинга . . . 494
Что оценивают, кроме правильного ответа . . . 494
Что спрашивают на каждом уровне . . . 495
План подготовки . . . 496
Подводные камни именно в Си . . . 499
Что говорить, если не знаешь . . . 501
Что дальше . . . 501

Приложение А. Задачи с собеседований: Junior **502**

Как пользоваться . . . 502
Часть 1. Задачи с готовыми решениями . . . 503
Задача 1. Поменять местами две переменные . . . 503
Задача 2. Развернуть строку на месте . . . 505
Задача 3. Проверка палиндрома . . . 507
Задача 4. Факториал итеративно . . . 509
Задача 5. Второй по величине элемент массива . . . 511
Задача 6. Подсчёт гласных в строке . . . 513
Задача 7. Проверка на простое число . . . 515
Задача 8. Цифры числа: сколько их и какая сумма . . . 517
Часть 2. Три классики без отдельного файла . . . 519
Задача 9. Проверить чётность числа . . . 519
Задача 10. Максимум и минимум массива за один проход . . . 521
Задача 11. FizzBuzz . . . 522
Часть 3. Каверзы: «что выведет / в чём ошибка» . . . 524
Каверза 1. `sizeof('a')` — сколько это? . . . 524
Каверза 2. Целочисленное деление `5 / 2` . . . 525
Каверза 3. `printf("%d", c)` для `char` . . . 526
Каверза 4. `char` — знаковый или беззнаковый? . . . 526
Каверза 5. `=` вместо `==` в условии . . . 527
Каверза 6. Лишняя `;` после `for(...)` . . . 528
Каверза 7. `printf("%d %d", i++, ++i)` . . . 529
Каверза 8. Неинициализированная переменная . . . 530
Каверза 9. Остаток от отрицательного числа . . . 531
Ошибки, на которых сыплются в этих задачах . . . 532
Тренировочные задачи . . . 532
Мини-резюме . . . 533
Куда дальше . . . 534

Приложение Б. Задачи с собеседований: Middle 535
Как пользоваться этим приложением . . . 535
Часть 1. Задачи на код . . . 536
Задача 1. Свои `strlen`, `strcpy`, `strcmp` . . . 536
Задача 2. Развернуть односвязный список . . . 538
Задача 3. Два указателя: середина списка, цикл и его начало . . . 540
Задача 4. Биты: popcount и степень двойки . . . 545
Задача 5. Корректный бинарный поиск . . . 548
Задача 6. Слияние двух отсортированных массивов . . . 550
Задача 7. Удалить дубликаты из отсортированного массива на месте . . . 551
Задача 8. Число, встречающееся нечётное число раз (XOR) . . . 553
Задача 9. Поменять местами два числа через указатели . . . 554
Часть 2. Каверзы . . . 556
Каверза 1. `char *s = "abc"` против `char s[] = "abc"` . . . 556
Каверза 2. `sizeof` массива внутри функции . . . 558
Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)` . . . 559
Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++` . . . 560
Каверза 5. `const char *p` против `char * const p` . . . 561
Каверза 6. Найди баг: утечка памяти . . . 562

Каверза 7. Найди баг: use-after-free и двойной free . . . 563
Каверза 8. Возврат адреса локальной переменной . . . 564
Ошибки, на которых сыплются в этих задачах . . . 566
Тренировочные задачи . . . 566
Ответы к тренировочным задачам . . . 567
Мини-резюме . . . 574
Что дальше . . . 575

Приложение B. Задачи с собеседований: Senior 576
Часть 1. Задачи на код . . . 577
Задача 1. Быстрая сортировка (quicksort) . . . 577
Задача 2. Сортировка слиянием (merge sort) . . . 580
Задача 3. Обобщённый стек через `void *` . . . 583
Задача 4. Свои `atoi` и `itoa` с обработкой краёв . . . 586
Задача 5. Таблица обработчиков на указателях на функции . . . 590
Задача 6. Кольцевой буфер (ring buffer) . . . 592
Задача 7. Хеш-таблица «строка → счётчик» с цепочками . . . 595
Задача 8. Точка пересечения двух односвязных списков . . . 598
Задача 9. Набросок LRU-кэша . . . 600
Задача 10. Разворот числа с защитой от переполнения . . . 604
Часть 2. Каверзы и теория . . . 606
Каверза 1. Что такое UB и где оно прячется . . . 606
Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать . . . 608
Каверза 3. `volatile` — зачем он нужен . . . 610
Каверза 4. Две роли `static` . . . 610
Каверза 5. Stack vs heap и причины segfault . . . 611
Каверза 6. Выравнивание и padding: что выведет `sizeof` . . . 612
Каверза 7. Опасный макрос `#define SQR(x) x*x` . . . 614
Каверза 8. Идиома `do { ... } while(0)` в макросах . . . 614
Каверза 9. Порядок вычисления аргументов не определён . . . 615
Каверза 10. Endianness — порядок байтов . . . 616
Ошибки, на которых сыплются в этих задачах . . . 617
Тренировочные задачи . . . 618
Ответы к тренировочным задачам . . . 618
Итог практикума . . . 634
Напутствие . . . 634

Введение. Зачем учить Си через алгоритмы и игры

Эта книга для тех, кто **никогда не программировал**. Если ты умеешь открыть терминал и сохранить текстовый файл — этого достаточно. Всё остальное мы построим сами, с нуля, не пропуская ни одной мелочи.

Мы будем учить язык **Си** (по-английски — *C*). И учить не сухо, а на двух самых увлекательных вещах в программировании:

- **алгоритмы** — способы решать задачи: отсортировать числа, найти нужное среди миллиона за двадцать шагов, выписать все простые числа до тысячи;
- **игры** — настоящие, в которые можно играть прямо в терминале: «Угадай число», крестики-нолики, виселица, сапёр и даже змейка.

Каждая глава — это маленький проект, который мы соберём своими руками и запустим.

Почему именно Си, если есть языки попроще

Начинать с Си — выбор нетипичный. Обычно новичку советуют Python: там короче код, не надо думать про типы, программа запускается одной командой. Всё правда. И всё же Си даёт то, чего Python дать не может.

Си **ничего не прячет**. В большинстве современных языков между твоим кодом и машиной стоит толстый слой удобств: память выделяется и освобождается сама, тип подбирается по значению, список растёт, когда ему вздумается. Пока всё работает — прекрасно. Когда сломается, ты будешь смотреть на ошибку и не понимать, что произошло, потому что происходящее от тебя спрятали.

В Си всё видно. Пишешь `int n;` — знаешь: под число выделено несколько байтов (на обычном компьютере — четыре, в главе 2 проверим это своими глазами), и в них лежит мусор, пока ты сам туда что-нибудь не положишь. Просишь память под массив — просишь явно и обязан

вернуть. Передаёшь данные в функцию — сам решаешь, копию отдать или адрес. Утомительно ровно первую неделю, а потом превращается в понимание, как устроен компьютер.

Конкретика, ради которой всё затевается:

- **Память и указатели.** Переменная окажется адресом и размером, а не «именем, к которому привязано значение». А указатель — самое пугающее слово в программировании — окажется просто переменной, в которой лежит адрес другой переменной. Разницу между «положить в коробку» и «дать ссылку на коробку» ты будешь видеть глазами.
- **Стоимость операций.** В Си сразу видно, что скопировать массив — это реально скопировать тысячу чисел, а не «просто присвоить». Отсюда сам собой вырастает разговор, почему один алгоритм быстрее другого.
- **Ошибки, которые в других языках невозможны.** Выход за границу массива, забытый `\0` в строке, утечка памяти. Звучит как минус, а на деле отличная школа: приучает думать, что будет, когда всё пойдёт не по плану.

И практическая сторона. Си придумали в начале 1970-х, и он никуда не делся: ядро Linux, части macOS и Windows, SQLite, PostgreSQL, Redis, Git, Nginx, прошивки микроконтроллеров — всё это Си. Фигурные скобки, `if`, `for`, `while`, `==` и `++` из Си унаследовали C++, Java, C#, JavaScript, PHP, Go. После этой книги любой из них будет читаться почти без словаря.

Алгоритмы и игры — идеальный полигон. Алгоритм заставляет думать чётко: сделай это, потом это, повтори столько-то раз. Игра заставляет довести программу до конца: пока она не заработает, играть не получится.

Чего в этой книге не будет

Честно, чтобы не было разочарования на середине.

- **Графики, окон и мышки.** Всё в терминале: текст на экране, ввод с клавиатуры. Змейка в главе 17 — буквы на чёрном фоне, а не спрайты. SDL и raylib только упомянуты в конце книги.
- **Сетей и потоков.** Ни сокетов, ни многопоточности. Файлам (`fopen`, `fclose`) досталась одна небольшая программа в главе 18: ни одному алгоритму и ни одной игре из книги они не нужны.
- **Полного справочника по языку.** Берём тот Си, которого хватает для дела. `union`, битовые поля, сложный препроцессор, `goto`, тонкости `const` — этого нет или почти нет. Указатель на функцию появляется один раз — в главе 8, для `qsort`, а `struct` и `enum` — в главах 11–17, и все три разбираются ровно в том объёме, в каком там нужны.

- **Больших проектов из многих файлов.** Каждая программа книги — один `.c`-файл, который собирается одной командой. Разделение на модули, свои заголовки, `Makefile` — следующий шаг после книги.
- **C++.** Это другой язык: похож внешне, думают на нём иначе.

Зато то, что взято, разобрано до дна: каждая строка, каждый знак препинания, каждая функция стандартной библиотеки — что принимает, что возвращает, где подвох.

Что ты будешь уметь к концу

Не «познакомишься с», а именно уметь — потому что каждую из этих программ ты напишешь и запустишь сам.

Основы языка (главы 1–3): переменные и типы, ввод и вывод, ветвления, циклы, свои функции, массивы, строки, указатели, динамическая память через `malloc`/`free`.

Алгоритмы (главы 4–11):

- числовые задачи — чётность, сумма цифр, переворот числа, факториал с ловушкой переполнения `int`, FizzBuzz;
- алгоритм Евклида для НОД, проверка на простоту, решето Эратосфена;
- массивы — минимум, максимум, сумма, среднее, разворот на месте, подсчёт и сдвиг;
- поиск линейный и бинарный, с настоящим замером: во сколько раз бинарный быстрее на деле;
- три сортировки — пузырьком, выбором, вставками, — плюс `qsort` из стандартной библиотеки и разговор про сложность;
- рекурсия — факториал, Фибоначчи (и почему наивный вариант катастрофически медленный), Ханойские башни;
- строки — длина, переворот, палиндром, подсчёт слов, анаграммы;
- структуры данных своими руками — растущий массив, стек, очередь, односвязный список.

Шесть игр (главы 12–17), каждая собирается с нуля по шагам:

- «Угадай число» — случайность, `rand`, `srand`, игровой цикл, защита от кривого ввода;
- «Камень, ножницы, бумага» — `enum`, красивый приём с остатком от деления вместо кучи `if`;
- «Виселица» — массив слов, маска открытых букв, жизни, рисунок;
- «Крестики-нолики» — двумерный массив, проверка победы и ничьей;
- «Сапёр» — расстановка мин, подсчёт соседей, рекурсивная заливка пустых клеток;
- «Змейка» — движение в реальном времени, чтение клавиш без Enter через `termios`, рост, счёт, столкновения.

Подготовка к собеседованию (приложения А, Б, В): три набора задач уровней Junior, Middle и Senior с полным разбором — от «поменять две переменные местами» до хеш-таблицы с цепочками, кольцевого буфера и LRU-кэша, плюс каверзные вопросы вроде `sizeof('a')`, strict aliasing и порядка вычисления аргументов.

Всего по дороге — больше семидесяти работающих программ. Все они лежат в папке `code/` рядом с книгой: можно свериться, если что-то не сходится.

Что понадобится

- **Компьютер с macOS или Linux.** На Windows тоже можно — через WSL (Linux внутри Windows) или MinGW; в главе 1 расписано, как. Главное отличие всплывёт в главе про змейку: чтение клавиш без Enter на Windows устроено иначе.
- **Терминал** — тот самый чёрный экран, где набирают команды. Постоянно нужны будут три: перейти в папку, скомпилировать, запустить.
- **Текстовый редактор.** VS Code, Sublime, Vim, встроенный «Блокнот» — любой, лишь бы сохранял обычный текст. Среда разработки не нужна.
- **Компилятор Си.** Скорее всего он уже стоит; ставим и проверяем в главе 1.

Ничего платного и ничего тяжёлого — этого хватит до последней страницы.

Как читать эту книгу

Набирай код руками. Не копируй. Единственное серьёзное требование, и оно не про дисциплину, а про то, как работает голова. Копируешь листинг — глаза видят готовое решение, и мозг честно докладывает: «понятно». Набираешь — вынужден заметить каждую точку с запятой, каждую скобку, каждую звёздочку. Сделаешь опечатку, компилятор наорёт, найдёшь её — вот это и есть обучение. Скопированный код забывается за сутки, набранный остаётся.

Запускай после каждого шага. Листингов, падающих с неба целиком, в книге почти нет. Мы пишем наивную версию, запускаем, смотрим на результат, находим проблему, чиним, запускаем снова. Повторяй это у себя: показан вывод — добейся такого же на своей машине. Расхождение не повод расстроиться, а повод разобраться.

Ломай специально. Убери точку с запятой. Забудь `&` в `scanf`. Выйди за границу массива. Посмотри, что скажет компилятор и что сделает программа. Пять минут осознанной поломки объясняют больше страницы текста.

Не пропускай главы и делай упражнения. Книга идёт строго по порядку: разобранное в главе 2 в главе 8 уже не объясняется заново, только напоминается строкой со ссылкой. Упражнений в конце главы три-шесть, и к каждому указано, что должно получиться, — чтобы было с

чем сверить себя. Но сначала посиди и подумай сам, не подглядывая: в этом весь смысл. Хотя бы два из каждой главы, и разница будет ощутимой.

Сколько это займёт

Честная арифметика. Главы 1–3 — по одному-двум вечерам каждая, это самый плотный теоретический кусок. Алгоритмические главы 4–11 — по полтора-два часа с упражнениями. Игры 12–17 — от двух до четырёх часов каждая: они длинные, зато там уже почти всё знакомо. Приложения с задачами можно растянуть надолго.

В сумме — порядка семидесяти-восьмидесяти часов вдумчивой работы. При часе в день это два с половиной месяца, по выходным — до полугода. Быстрее не выйдет: книгу нельзя «прочитать», её можно только «написать».

Важнее темпа — регулярность. Полчаса каждый день дают больше, чем восемь часов раз в две недели: между занятиями знание не успевает выветриться.

Что делать, если застрял

Застревать ты будешь. Это часть процесса, а не признак того, что программирование не твоё. Порядок действий:

1. **Прочитай сообщение компилятора целиком**, а не первую строку. Там указан файл, номер строки, номер символа и часто прямая подсказка, что вписать. В главе 1 мы разберём, как эти сообщения устроены.
2. **Проверь строку выше указанной.** Классика: пропущенная `;` обнаруживается компилятором только на следующей строке.
3. **Сравни свой код с листингом посимвольно.** Скобка `{` вместо `(`, `=` вместо `==`, пропущенная кавычка. Девять раз из десяти дело в этом.
4. **Компилируется, но работает неправильно?** Вставь `printf` в подозрительные места и распечатай значения переменных. Самый простой отладчик на свете, и им пользуются все.
5. **Уменьши задачу.** Не работает сортировка на десяти числах — попробуй на трёх и проследи руками, что должно получаться на каждом шаге.
6. **Отойди.** Прогулка на двадцать минут решает задачи лучше, чем третий час втыкания в тот же экран.
7. **Сверься с файлом в `code/`.** Там рабочая версия каждой программы книги. Но сначала попробуй сам: заглянуть в ответ сразу — значит не научиться.

Непонятная ошибка в первый раз выглядит стеной, во второй — знакомой стеной, в пятый — просто строчкой текста, которую глаз читает автоматически. Умение спокойно читать ошибки нарабатывается только количеством.

Поехали. Следующая глава — ставим компилятор и запускаем первую программу.

Глава 1. Компилятор и первая программа

Сейчас мы поставим инструмент, который превращает наш текст в работающую программу, и напишем классическое «Привет, мир!». Причём напишем не набело: сначала попробуем как получится, посмотрим, что ответит компилятор, и починим по его подсказке. Так мы будем работать всю книгу.

Потом разберём получившуюся программу — **каждый символ**, от первой решётки до последней точки с запятой. И заглянем внутрь компиляции: покажем своими глазами, что там не одно действие, а четыре.

Что мы сделаем

- проверим, есть ли в системе компилятор, и поставим, если нет;
- напишем `hello.c`, сломаем его, прочитаем ошибку и починим;
- разберём готовую программу построчно;
- разложим компиляцию на четыре фазы и посмотрим результат каждой;
- заведём привычку компилировать с включёнными предупреждениями;
- специально наделаем типичных ошибок и научимся читать ответы компилятора.

Что такое компилятор

Процессор не понимает Си. Он понимает только **машинный код** — последовательность команд, записанных байтами. Каждая команда — это элементарное действие: «сложи два числа», «положи это в память», «прыгни на такой-то адрес»; у процессоров Intel одна команда занимает от 1 до 15 байтов. Писать в таких байтах человеку невозможно.

Поэтому мы пишем на Си — языке, понятном человеку, — а специальная программа переводит наш текст в машинный код. Эта программа-переводчик и называется **компилятор** (compiler, от англ. *compile* — собирать).

```
наш текст (hello.c)  →  [компилятор]  →  исполняемый файл  →  запускаем
```

Файл с нашим кодом — обычный текстовый файл с расширением `.c`. То, что выходит из компилятора, — **исполняемый файл**: готовая программа, которую операционная система умеет запускать.

Важное отличие от Python или JavaScript: там программу запускают прямо из исходника, а перевод происходит на лету при каждом запуске. В Си перевод делается один раз, заранее. Поэтому программа на Си стартует мгновенно и работает быстро — но и любую ошибку в синтаксисе ты узнаешь не при запуске, а сразу при компиляции. Это скорее плюс: компилятор ловит опечатки до того, как программа попадёт к пользователю.

Ставим компилятор

Нам подойдёт `clang` или `gcc` — это два разных компилятора Си, но для всего, что есть в книге, команды у них одинаковые.

macOS

Открой Терминал (Программы → Утилиты → Терминал, или Spotlight по слову «Терминал») и набери:

```
cc --version
```

`cc` — сокращение от *C Compiler*, стандартное имя компилятора в Unix-системах. На macOS за этим именем прячется `clang`. Вот что ответила эта команда на машине, где писалась книга:

```
Apple clang version 21.0.0 (clang-2100.1.1.101)
Target: x86_64-apple-darwin25.6.0
Thread model: posix
InstalledDir: /Applications/Xcode.app/Contents/Developer/
              Toolchains/XcodeDefault.xctoolchain/usr/bin
```

(Последняя строка в терминале одна и длинная — я перенёс её, чтобы влезла в страницу.)

У тебя номер версии и путь будут другими — это нормально. Важно одно: команда не сказала «command not found». Что показано:

- `Apple clang version 21.0.0` — название и версия компилятора.
- `Target: x86_64-apple-darwin25.6.0` — под какой процессор и систему он собирает по умолчанию. `x86_64` — это процессор Intel; на компьютерах Apple с чипами M1/M2/M3 там будет `arm64`. Дальше в главе это отличие всплывёт ещё раз, когда будем смотреть ассемблер.
- `InstalledDir` — где лежит сам компилятор.

Если вместо этого система предложит установить «Command Line Tools» — соглашайся, это и есть компилятор. Можно позвать установку и вручную:

```
xcode-select --install
```

Откроется окно, жми «Установить» и жди несколько минут. Полный Xcode (многогигабайтная среда разработки) для книги не нужен, хватает Command Line Tools.

Linux

Проверяем той же командой:

```
cc --version
```

Если ответа нет, на Ubuntu, Debian, Linux Mint ставим так:

```
sudo apt update
sudo apt install build-essential
```

`build-essential` — сборный пакет с инструментами для компиляции, включая `gcc`. На Fedora команда `sudo dnf install gcc`, на Arch — `sudo pacman -S base-devel`.

На Linux `cc` обычно указывает на `gcc`. Сообщения об ошибках у `gcc` формулируются немного иначе, чем у `clang`, но говорят о том же самом. Все сообщения, показанные в этой книге, сняты с `clang` на macOS — если у тебя `gcc`, текст будет похож, но не дословно совпадёт. Пугаться этого не надо: структура у всех сообщений одна (файл, строка, что не понравилось), и читать их мы сейчас научимся.

Windows

На Windows есть два разумных пути.

WSL (Windows Subsystem for Linux) — настоящий Linux внутри Windows. В PowerShell от администратора выполняется `wsl --install`, после перезагрузки появляется терминал Ubuntu, и дальше всё как в разделе про Linux: `sudo apt install build-essential`. Это самый удобный вариант: книга будет работать целиком, включая змейку из главы 17.

MinGW-w64 — порт `gcc` под Windows, ставится через MSYS2. Работает, но глава про змейку потребует переделки: там используется `termios` — часть стандарта POSIX, которого в чистой Windows нет. Об этом предупреждено прямо в той главе.

Проверок и точных выводов команд для Windows я тут не привожу намеренно: у меня под рукой только macOS и Linux, а выдумывать транскрипты, которых я не видел, — плохая практика. Если ты на Windows, доверяй официальным инструкциям WSL или MSYS2, а всё остальное в книге пойдёт без изменений.

Шаг 1. Пишем первое, что приходит в голову

Создай где-нибудь папку для книги, например `c-book`, зайди в неё и создай файл `hello.c`. Набери в нём вот это (руками, не копируя — помнишь уговор из введения):

```
int main(void)
{
    printf("Привет, мир!\n");
}
```

Что тут написано, разберём чуть позже, до последнего знака. Пока достаточно понимать общий смысл: «есть кусок кода по имени `main`, и он печатает строчку».

Сохрани файл. Теперь в терминале перейди в папку с ним и скомпилируй:

```
cc hello.c -o hello
```

Разберём команду:

- `cc` — зовём компилятор;
- `hello.c` — файл, который надо скомпилировать;
- `-o hello` — **флаг** `-o` (от *output* — вывод) задаёт имя результата. Получится исполняемый файл `hello`. Без `-o` компилятор назвал бы результат `a.out` — историческое имя по умолчанию, от *assembler output*.

И вот ответ:

```
hello.c:3:5: error: call to undeclared library function 'printf'
with type 'int (const char *, ...)'; ISO C99 and later do not support
implicit function declarations [-Wimplicit-function-declaration]
    3 |     printf("Привет, мир!\n");
      |     ^
hello.c:3:5: note: include the header <stdio.h> or explicitly provide
a declaration for 'printf'
1 error generated.
```

(В терминале первая строка и строка `note:` — каждая одна длинная строка; здесь я перенёс их, чтобы влезли в страницу. Так будет и дальше.)

Первое сообщение об ошибке в жизни выглядит устрашающе. Разберём его по косточкам — все сообщения устроены одинаково, и через пару глав ты будешь читать их мельком.

- `hello.c:3:5` — координаты: файл `hello.c`, строка 3, символ 5. Компилятор всегда показывает точное место.

- `error` — это ошибка, файл не собран, исполняемого файла нет. Бывает ещё `warning` (предупреждение) — тогда программа соберётся, но компилятору что-то не нравится.
- `call to undeclared library function 'printf'` — «вызов необъявленной библиотечной функции `printf`». Компилятор впервые видит имя `printf` и не знает, что это такое.
- `ISO C99 and later do not support implicit function declarations` — «стандарт C99 и более поздние не поддерживают неявные объявления функций». Раньше, в очень старом Си, компилятор в такой ситуации сам догадывался, что имелось в виду. Теперь — нет.
- `[-Wimplicit-function-declaration]` — в квадратных скобках имя «диагностики». По нему можно найти объяснение в интернете, и по нему же ошибку можно было бы отключить (не надо так делать).
- Ниже компилятор перепечатывает саму строку и ставит под ней курсор `^` — ровно под тем местом, которое ему не понравилось.
- `note:` — подсказка. Здесь она прямым текстом говорит, что делать: *include the header* `<stdio.h>`, подключи заголовок `<stdio.h>`.
- `1 error generated` — итог: одна ошибка.

Строки `note:` новички обычно пролистывают, а зря: компилятор часто сам пишет готовое решение.

Кстати, если в чужих старых текстах встретишь формулировку «implicit declaration of function» — это то же самое, только в прежней редакции. Много лет такой код собирался всего лишь с предупреждением; сегодняшний clang считает это ошибкой и отказывается собирать.

Шаг 2. Делаем, как сказал компилятор

Добавляем первой строкой то, что он просил:

```
#include <stdio.h>

int main(void)
{
    printf("Привет, мир!\n");
}
```

Компилируем снова:

```
cc hello.c -o hello
```

На этот раз команда отработала молча. Молчание компилятора — хороший знак: значит, всё собралось. Запускаем:

```
./hello
```

`./` означает «файл в текущей папке». Без этого система искала бы программу `hello` среди системных и не нашла бы. Результат:

```
Привет, мир!
```

Первая программа работает.

Шаг 3. Возвращаем код завершения

Программа готова, но одной детали не хватает. Каждая программа, завершаясь, сообщает системе число — **код возврата** (exit code). По соглашению `0` означает «всё прошло хорошо», любое другое число — «что-то пошло не так». Этим числом пользуются другие программы и скрипты, чтобы понять, удалось ли дело.

Посмотреть код возврата в терминале можно так — команда `echo $?` печатает результат последней запущенной программы:

```
./hello
echo $?
```

```
Привет, мир!
0
```

Ноль. Откуда он взялся, если мы ничего не возвращали? Тут особый случай: стандарт Си разрешает не писать возврат именно в `main` и в этом случае считает, что вернулся `0`. Но полагаться на такое послабление не стоит: во всех остальных функциях так нельзя, и читатель твоего кода не должен гадать. Пишем явно:

```
    return 0;
```

Полная программа

Итоговый файл главы — `code/01-hello.c`:

```
#include <stdio.h>

int main(void)
{
    printf("Привет, мир!\n");
    return 0;
}
```

Семь строк. Сейчас разберём каждую.

Разбираем построчно

`#include <stdio.h>`

Эта строка подключает к нашей программе чужой готовый код.

- `#include` — **директива препроцессора**. Препроцессор — помощник компилятора, который срабатывает первым и подготавливает текст. Всё, что начинается с решётки `#`, адресовано ему. `include` по-английски — «включить».
- `<stdio.h>` — имя подключаемого файла. Это **заголовочный файл** (header, расширение `.h`). `stdio` расшифровывается как *standard input/output* — стандартный ввод-вывод. Внутри описаны функции для работы с экраном, клавиатурой и файлами, в том числе `printf`.
- Угловые скобки `<...>` означают «ищи файл среди системных», то есть среди тех, что идут вместе с компилятором. Для своих файлов используют кавычки: `#include "мой_файл.h"`.
- В конце **нет точки с запятой**. Директивы препроцессора её не требуют — они заканчиваются концом строки. Это одно из немногих мест в Си, где точка с запятой не нужна.

Что именно даёт подключение, мы увидим своими глазами в разделе про фазы компиляции.

Пустая строка

Для читаемости. Пустые строки и лишние пробелы между словами Си игнорирует — они для людей, не для машины. Исключения два: внутри кавычек каждый пробел печатается как есть, а директива `#include` обязана стоять на отдельной строке. Всё, что после неё, уместилось бы и в одну строку, но читать это стало бы невозможно.

`int main(void)`

Объявление **функции** по имени `main`. Функция — это именованный кусок кода, который можно выполнить. `main` — особенная: с неё начинается выполнение **любой** программы на Си. Когда ты запускаешь программу, система находит `main` и выполняет то, что внутри.

По частям:

- `int` — **тип** значения, которое функция вернёт, когда закончит. `int` (от *integer* — целое) означает целое число. То есть `main` пообещала вернуть целое число.
- `main` — имя функции. Менять его нельзя, система ищет именно `main`. И регистр важен: `Main` — уже другое имя, Си различает большие и маленькие буквы.
- `(void)` — в круглых скобках перечисляют, что функция принимает на вход. `void` по-английски «пустота, ничто», значит, `main` не принимает ничего. Пустые скобки `()` до

стандарта C23 означали не «ничего», а «параметры не указаны» — компилятор не проверял, с чем функцию вызывают. Поэтому пишем именно `(void)`.

- Точки с запятой в конце нет: сразу за этой строкой идёт тело функции в фигурных скобках.

`{` и `}`

Фигурные скобки обрамляют **тело** функции — всё, что она делает. Открывающая `{` — «здесь начинается код функции», закрывающая `}` — «здесь он заканчивается». Всё между ними выполняется по порядку, сверху вниз.

Скобки всегда парные. Забудешь закрывающую — компилятор дойдёт до конца файла и пожалуется, что чего-то не хватает. Отступ в четыре пробела внутри скобок делается ради читаемости; компилятору он безразличен, а глазу помогает видеть, что вложено во что.

`printf("Привет, мир!\n");`

Вызов функции `printf` — той самой, что описана в `stdio.h`. `printf` (от *print formatted* — печать с форматированием) выводит текст на экран.

Сигнатура (описание того, что функция принимает и возвращает) у неё такая — расшифруем её постепенно, в главах 2 и 3:

```
int printf(const char *format, ...);
```

Принимает строку-шаблон, а после неё — сколько угодно дополнительных значений, которые надо в этот шаблон подставить. Возвращает количество напечатанных **байтов** (а при ошибке — отрицательное число); это значение обычно никого не интересует, и его спокойно игнорируют. Почему байтов, а не букв, — увидим ниже, в разделе про ассемблер, и проверим в главе 2.

По знакам:

- Круглые скобки после имени означают **вызов**: «выполни эту функцию». Внутри — **аргументы**, то, что мы ей передаём. Здесь аргумент один: текст `"Привет, мир!\n"`.
- Двойные кавычки `"..."` обрамляют **строку** — последовательность символов. То, что между кавычками, попадёт на экран.
- `\n` — это **один** специальный символ, перевод строки (*new line*). Обратный слеш `\` говорит: «следующий символ читай особым образом». `\n` означает «перейти на новую строку», как нажатие Enter. Такие пары называют *escape-последовательностями*; ещё бывают `\t` (табуляция) и `\\` (сам обратный слеш).

- `;` — **точка с запятой**. Ею заканчивается почти каждая команда (по-научному — *инструкция*, англ. *statement*). Это как точка в конце предложения: она говорит компилятору, где мысль закончилась. Забыть её — самая частая ошибка новичка.

Убери `\n` и запусти — строка останется незаконченной. В оболочке `bash` приглашение терминала прилипнет прямо к нашему тексту. А `zsh` (в macOS он стоит по умолчанию) допечатает после текста значок `%` в инверсном цвете и сам перенесёт приглашение на новую строку — так он показывает, что программа не закончила строку. Попробуй, это одно из упражнений.

```
return 0;
```

- `return` (англ. «вернуть») завершает функцию и возвращает значение туда, откуда её вызвали. `main` обещала вернуть `int` — вот мы и возвращаем целое `0`.
- `0` — тот самый код возврата: «программа отработала успешно».
- И снова точка с запятой в конце: `return` — такая же инструкция, как вызов `printf`.

Что происходит при компиляции: четыре фазы

Мы говорим «скомпилировать», но за одной командой `cc` прячутся четыре отдельных шага, и каждый можно посмотреть отдельно. Это не праздное любопытство: когда ошибка вылезет на одной из этих фаз, ты будешь понимать, где именно.

```
hello.c → [препроцессор] → [компилятор] → [ассемблер] → [компоновщик] → hello
              hello.i          hello.s         hello.o
```

Фаза 1. Препроцессор — `cc -E`

Препроцессор работает с текстом, ничего не понимая в языке. Он подставляет содержимое `#include`-файлов, раскрывает макросы, вырезает комментарии. Результат — тоже текст на Си, только раздутый.

Флаг `-E` (от *preprocess*) останавливает работу сразу после этой фазы и печатает результат. Направим его в файл, чтобы измерить:

```
cc -E hello.c > hello.i
wc -l hello.c hello.i
```

```
       7 hello.c
     572 hello.i
     579 total
```

Было 7 строк — стало 572. Это и есть содержимое `stdio.h` (и всех файлов, которые подключает уже он), вставленное в начало нашей программы. Найдём в этой куче то, ради чего всё затевалось:

```
grep -n "^int printf" hello.i
```

```
315:int printf(const char * restrict, ...)
    __attribute__((__format__ (__printf__, 1, 2)));
```

Вот оно — описание `printf`, приехавшее из заголовка (номер строки `315` у тебя будет свой). Именно этой строчки не хватало компилятору на шаге 1. Заметь: тут только описание («функция называется так, принимает такое, возвращает такое»), но не сам код `printf`. Код появится на четвёртой фазе.

А наша программа лежит в самом конце файла, нетронутая:

```
tail -5 hello.i
```

```
int main(void)
{
    printf("Привет, мир!\n");
    return 0;
}
```

Число 572 у тебя будет другим — оно зависит от версии системы и компилятора. Порядок величины тот же: несколько сотен строк.

Фаза 2. Компилятор — `cc -S`

Настоящая компиляция: текст на Си разбирается, проверяется на ошибки и переводится в **ассемблер** — язык, где каждая строка соответствует одной команде процессора. Это уже почти машинный код, только записанный буквами.

```
cc -S hello.c -o hello.s
```

Файл получился на 28 строк — покажу его целиком, только последнюю строку придётся перенести, иначе она не влезет в страницу:

```
    .build_version macos, 26, 0 sdk_version 26, 5
    .section    __TEXT,__text,regular,pure_instructions
    .globl  _main                           ## -- Begin function main
    .p2align    4
_main:                                  ## @main
    .cfi_startproc
```

```
## %bb.0:
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
    movl    $0, -4(%rbp)
    leaq    L_.str(%rip), %rdi
    movb    $0, %al
    callq   _printf
    xorl    %eax, %eax
    addq    $16, %rsp
    popq    %rbp
    retq
    .cfi_endproc
                                        ## -- End function
    .section    __TEXT,__cstring,cstring_literals
L_.str:                                 ## @.str
    .asciz  "\320\237\321\200\320\270\320\262\320\265\321\202,
             \320\274\320\270\321\200!\n"

.subsections_via_symbols
```

Разбирать ассемблер мы не будем — это отдельная большая тема. Но общую картину видно и без подготовки. Строки, начинающиеся с точки (`.section`, `.globl`, `.cfi_startproc`), — не команды процессора, а служебные пометки для ассемблера и системы. Настоящих команд от `_main:` до `retq` всего одиннадцать. Среди них:

- `callq _printf` — вызов `printf`;
- `leaq L_.str(%rip), %rdi` — «положи адрес нашей строки туда, откуда `printf` его заберёт»;
- `xorl %eax, %eax` — хитрый способ записать ноль: число, сложенное само с собой по правилу «исключающее или», всегда даёт ноль. Это и есть наш `return 0`;
- `retq` — возврат из функции.

Это команды процессора Intel. На Маке с чипом M1/M2/M3 набор команд другой (`arm64`), и файл выглядит иначе: вместо `callq _printf` там будет `bl _printf`. Смысл тот же — вызвать функцию и вернуть ноль.

Отдельно интересно, где спряталась наша строка, — в самом низу, под меткой `L_.str`. Латиница и знаки препинания видны как есть, а кириллица превратилась в числа: `\320\237` — это два байта, которыми в кодировке UTF-8 записывается буква «П». Одна русская буква занимает **два байта**, а не один. Пока это просто любопытный факт, но в главе 10, когда мы начнём разбирать строки по буквам, он окажется очень важным.

Фаза 3. Ассемблер — `cc -c`

Ассемблер (программа, а не язык) переводит текст с командами в настоящие байты машинного кода. Получается **объектный файл** `.o`.

```
cc -c hello.c -o hello.o
```

Это уже двоичный файл, в текстовом виде его не посмотришь. Зато можно посмотреть список имён, которые в нём есть, командой `nm`:

```
nm hello.o
```

```
0000000000000000 T _main
                 U _printf
```

Две строки, и они очень красноречивы:

- `T _main` — буква `T` значит «здесь определён код функции `main`». Наш код на месте.
- `U _printf` — буква `U` значит *undefined*, «не определён». Объектный файл знает, что надо вызвать `printf`, но самого кода `printf` у него нет. Дырка, которую предстоит заткнуть.

Фаза 4. Компоновщик — линковка

Компоновщик (linker, ещё говорят «линкер») собирает все объектные файлы вместе, добавляет к ним код из библиотек и подставляет настоящие адреса вместо дырок. Код `printf` живёт в стандартной библиотеке Си, которая подключается автоматически.

```
cc hello.o -o hello
./hello
```

```
Привет, мир!
```

Сравним размеры всех промежуточных файлов (`wc -c` печатает размер в байтах):

```
wc -c hello.c hello.i hello.s hello.o hello
```

```
      92 hello.c
   31582 hello.i
     773 hello.s
     784 hello.o
```

```
    8440 hello
   41671 total
```

92 байта текста — и 8440 байт готовой программы. Разница уходит на служебные заголовки исполняемого файла и таблицы, по которым система при запуске находит нужные библиотеки.

Когда компоновщик не может заткнуть дырку, он ругается совсем иначе, чем компилятор — без номера строки, потому что строк он уже не видит. Пообещаем функцию `greet` и не напишем её:

```c
#include <stdio.h>

void greet(void);

int main(void)
{
    greet();
    return 0;
}
```

```
Undefined symbols for architecture x86_64:
  "_greet", referenced from:
      _main in hello-0e6a22.o
ld: symbol(s) not found for architecture x86_64
clang: error: linker command failed with exit code 1 (use -v to see
invocation)
```

`ld` — это и есть компоновщик. Такое сообщение означает: «ты обещал функцию `greet`, я поверил, а кода её нигде нет». Обрати внимание: ни файла, ни номера строки тут нет, есть только имя временного объектного файла (`hello-0e6a22.o`, у тебя будет другой набор цифр). Если увидишь `Undefined symbols` — ищи не опечатку в строке, а забытый файл, неподключённую библиотеку или объявленную, но не написанную функцию.

Все четыре фазы `cc` выполняет за один заход, а промежуточные результаты не сохраняет: объектный файл создаётся во временной папке и удаляется (тот самый `hello-….o` из сообщения компоновщика), а текстовые `.i` и `.s` clang по дороге и вовсе не пишет на диск — препроцессор и ассемблер встроены в него самого. Мы просто попросили его останавливаться и показывать.

Включаем предупреждения — привычка на всю книгу

Компилятор умеет замечать подозрительные места, но по умолчанию молчит о многом. Заведи привычку компилировать так:

```
cc -std=c11 -Wall -Wextra -O2 hello.c -o hello
```

- `-std=c11` — использовать стандарт языка **C11** (редакция Си 2011 года). Без этого флага разные компиляторы возьмут разные версии по умолчанию, и поведение может отличаться.
- `-Wall` — от *warnings all*, включить основные предупреждения. Название врёт: это далеко не все, а самые полезные.
- `-Wextra` — добавить ещё пачку сверху.
- `-O2` — от *optimize level 2*, попросить сделать программу быстрее. Для учёбы не обязательно, но пусть будет: пригодится в главе 7, где мы замеряем скорость.

Разница не теоретическая. Возьмём программу с настоящей ошибкой. Тут есть две вещи из следующей главы — переменная `int n` («коробка для целого числа») и `%d` («подставь сюда число»); разберём их подробно в главе 2, а сейчас нам важно другое. Коробка заведена, но в неё ничего не положили — а печатать мы её всё равно печатаем:

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("n = %d\n", n);
    return 0;
}
```

Компилируем без флагов:

```
cc hello.c -o hello
./hello
```

Компилятор не сказал ни слова, программа собралась и напечатала:

```
n = 1165711240
```

Откуда число? Это мусор — то, что случайно лежало в памяти на этом месте. У тебя будет другое, и от запуска к запуску оно может меняться. Программа «работает», но результат бессмысленный. По стандарту Си чтение такой переменной — **неопределённое поведение** (undefined behavior): язык не обещает вообще ничего, и с другими флагами компилятор вправе напечатать что угодно. Подробнее про это — в главе 2. Теперь с флагами:

```
cc -std=c11 -Wall -Wextra hello.c -o hello
```

```
hello.c:6:24: warning: variable 'n' is uninitialized when used here
[-Wuninitialized]
    6 |     printf("n = %d\n", n);
      |                        ^
hello.c:5:10: note: initialize the variable 'n' to silence this warning
    5 |     int n;
      |          ^
      |           = 0
1 warning generated.
```

Компилятор нашёл ошибку сам и даже показал, куда дописать `= 0`. Бесплатно, за два флага.

Предупреждение (*warning*) — это не ошибка: программа соберётся и запустится. Но **в этой книге мы пишем код так, чтобы предупреждений не было ни одного**. Считай warning маленькой ошибкой, которую компилятор пока согласился простить.

Набирать длинную команду каждый раз лень. В большинстве оболочек помогает стрелка вверх — она возвращает предыдущие команды.

Частые ошибки

Здесь я не буду описывать ошибки словами — сломаю программу по-настоящему и покажу, что скажет компилятор. Все сообщения ниже сняты с Apple clang; у `gcc` формулировки чуть другие, но смысл тот же.

Забыл `;` в конце строки

Убираем точку с запятой после `printf`:

```
hello.c:5:38: error: expected ';' after expression
    5 |     printf("Привет, мир!\n")
      |                              ^
      |                              ;
1 error generated.
```

«Ожидал `;` после выражения». Компилятор не только показал место, но и нарисовал под курсором то, что надо вписать.

Тонкость, которая сэкономит много времени. Указано место, где **`;` не хватило**, — конец пятой строки. Но обнаружил пропажу компилятор только на шестой, дочитав до `return`: пока он не

увидел следующую инструкцию, он не мог знать, что предыдущая уже кончилась. В более сложном коде из-за этого ошибка иногда указывает на строку **ниже** настоящей опечатки. Правило: не видишь проблему в указанной строке — смотри строку выше.

Опечатка в имени функции

Пишем `pritnf` вместо `printf`:

```
hello.c:5:5: error: call to undeclared function 'pritnf'; ISO C99 and
later do not support implicit function declarations
[-Wimplicit-function-declaration]
    5 |     pritnf("Привет, мир!\n");
      |     ^
hello.c:5:5: note: did you mean 'printf'?
```

Дальше компилятор допечатывает ещё три строки — ещё один `note` с длинным путём к системному заголовку, где объявлен настоящий `printf`, и само это объявление, — а в конце привычное `1 error generated.`. Я их не привожу: пути там длиннее страницы, а нового они ничего не добавляют. А вот `did you mean 'printf'?` — «ты имел в виду `printf`?» — это главное. Компилятор сравнил незнакомое имя с известными и сам нашёл похожее.

Ровно то же самое будет от `Printf` с большой буквы: Си различает регистр, и `Printf` для него — другое, незнакомое имя.

Забыл закрывающую }

```
hello.c:6:14: error: expected '}'
    6 |     return 0;
      |              ^
hello.c:4:1: note: to match this '{'
    4 | {
      | ^
1 error generated.
```

«Ожидал }», и note показывает, какая именно открывающая скобка осталась без пары — та, что на строке 4. Когда скобок станет много, эта подсказка будет спасать.

Забыл закрывающую кавычку

```
hello.c:5:12: warning: missing terminating '"' character [-Winvalid-pp-token]
    5 |     printf("Привет, мир!\n);
```

```
      |           ^
hello.c:5:12: error: expected expression
1 warning and 1 error generated.
```

«Пропущена завершающая кавычка». Строка обязана закрываться на той же строке, где открылась.

Перепутал кавычки

Текст печатается в **двойных** кавычках `"..."`. Одинарные `'...'` в Си означают совсем другое — ровно один символ. Пишем `printf('Да')`:

```
hello.c:5:12: error: character too large for enclosing character literal type
    5 |     printf('Да');
      |            ^
hello.c:5:12: error: character too large for enclosing character literal type
hello.c:5:12: warning: multi-character character constant [-Wmultichar]
1 warning and 2 errors generated.
```

«Символ слишком велик для символьного литерала» — и ошибка повторена дважды, по разу на каждую букву: ни «Д», ни «а» не помещаются в место, рассчитанное на один символ (каждая занимает два байта, помнишь ассемблер?). А `multi-character character constant` — «многосимвольная символьная константа», то есть «в одинарных кавычках больше одного знака». Возьми слово подлиннее — получишь по ошибке на каждую букву.

Запомни: одинарные кавычки — это ровно один знак (`'A'`, `'?'`, `'\n'`), двойные — строка любой длины.

Запускаешь без `./`

Наберёшь `hello` — оболочка ответит:

```
zsh: command not found: hello
```

Она искала программу `hello` среди системных — в каталогах, перечисленных в переменной `PATH`, — а текущую папку туда не включают из соображений безопасности. Указывай путь явно: `./hello`. (В Linux обычно стоит `bash`, и он скажет то же самое своими словами: `bash: hello: command not found`.)

Компилятор не находит файл

```
clang: error: no such file or directory: 'hello.cc'
clang: error: no input files
```

Либо опечатка в имени, либо ты не в той папке. Проверь командой `ls`, что `hello.c` лежит там, где ты сейчас находишься.

Файл не сохранён

Изменил код, но забыл сохранить — компилируется старая версия, и ты смотришь на «ту же самую» ошибку, гадая, почему исправление не помогло. Сохраняй перед каждой компиляцией; в большинстве редакторов есть автосохранение, включи его.

Упражнения

1. Сделай так, чтобы программа печатала две строки: «Привет, мир!» и на следующей — «Я учу Си». Реши двумя способами: двумя вызовами `printf` и одним, с двумя `\n` внутри. *Ожидаемый результат* у обоих способов одинаковый:

   ```
   Привет, мир!
   Я учу Си
   ```

2. Убери `\n` из `printf` и запусти. Куда делось приглашение терминала? Верни `\n` на место. *Ожидаемый результат*: в `zsh` после текста появится значок `%` в инверсном цвете, в `bash` приглашение встанет прямо за восклицательным знаком.

3. Поставь в конец `return 5;` вместо `return 0;`, скомпилируй, запусти и посмотри `echo $?`. Убедись, что число дошло до системы. *Ожидаемый результат*: `echo $?` напечатает `5`. (Код возврата передаётся одним байтом, поэтому годятся числа от 0 до 255: `return 256;` система увидит как 0.)

4. Намеренно сломай программу тремя способами из раздела «Частые ошибки» и прочитай сообщения. Найди в каждом координаты, текст ошибки и подсказку. *Ожидаемый результат*: сообщения совпадут с приведёнными в разделе (у `gcc` — по смыслу).

5. Запусти `cc -E hello.c > hello.i` и открой `hello.i` в редакторе. Найди в нём строку с описанием `printf` и свою функцию `main` в самом конце. *Ожидаемый результат*: строка начинается с `int printf(const char * restrict, ...)`, `main` — последние пять строк файла.

6. Посмотри `cc -S hello.c -o hello.s` и найди в ассемблере строку `callq _printf` (на M1/M2/M3 она будет называться `bl _printf`). Убедись, что вызов действительно один. *Ожидаемый результат*: команда `grep -c _printf hello.s` напечатает `1`.

Что мы получили

Мы поставили компилятор, написали программу, сломали её, прочитали ошибку, починили по подсказке и запустили. Разобрали в ней каждый знак: `#include`, `main`, `int`, фигурные скобки, `printf`, `\n`, точку с запятой, `return 0`. Заглянули внутрь компиляции и увидели все четыре фазы своими глазами. И договорились всегда компилировать с `-std=c11 -Wall -Wextra`.

Это скелет любой программы на Си — дальше мы будем только наполнять его. В следующей главе программа оживёт: научимся хранить данные в переменных, печатать числа, спрашивать что-то у пользователя, принимать решения через `if` и повторять действия в цикле. Это весь минимум, которого хватит, чтобы взяться за первые алгоритмы.

Итоговый файл этой главы — `code/01-hello.c`.

Глава 2. Минимум Си: переменные, ввод, ветвления, циклы, функции

В первой главе программа умела ровно одно: напечатать заранее заготовленную строчку. Она ничего не помнила, ни о чём не спрашивала и всегда вела себя одинаково. Пора это исправить.

Здесь мы разберём всё, без чего не написать ни одного алгоритма: **переменные и типы**, **вывод чисел**, **ввод с клавиатуры**, **арифметику** с её ловушками, **ветвления** (`if`/`else`), **циклы** (`while`, `for`, `do-while`) и **свои функции**.

Глава длинная — самый плотный теоретический кусок в книге. Не пытайся заучить: набирай и запускай каждый пример руками, тогда уляжется само.

Что мы сделаем

К концу главы у нас будет маленькая интерактивная программа: она спрашивает число и печатает его таблицу умножения ровным столбиком, помечая чётные результаты:

```
Таблица умножения для какого числа? 3
 3 x  1 =   3
 3 x  2 =   6  (чётное)
 3 x  3 =   9
 3 x  4 =  12  (чётное)
 3 x  5 =  15
 3 x  6 =  18  (чётное)
 3 x  7 =  21
 3 x  8 =  24  (чётное)
 3 x  9 =  27
 3 x 10 =  30  (чётное)
```

Программка крошечная, но в ней уже есть переменная, ввод с проверкой, цикл, ветвление и собственная функция — весь минимум Си.

Разбираем задачу руками

Прежде чем писать код, сделай то же самое на бумаге. Пусть `n` равно 3. Что нужно напечатать?

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
...и так до 10
```

Видно повторение: меняется только второй множитель — 1, 2, 3, …, 10. Значит, нужна «коробка» под сам `n` (его вводит человек) и вторая коробка под текущий множитель, который мы увеличиваем на единицу десять раз подряд. Первая коробка — переменная, «увеличиваем десять раз» — цикл. С коробок и начнём.

Шаг 1. Переменная — коробка для значения

Переменная — это **именованная коробочка в памяти**, в которой лежит значение. У коробочки есть имя (чтобы к ней обращаться) и тип (чтобы знать, что в ней лежит и сколько места занимает).

```c
#include <stdio.h>

int main(void)
{
    int age = 25;
    printf("Возраст: %d\n", age);
    return 0;
}
```

Разбор новых вещей:

- `int age = 25;` — мы **объявили** переменную. По частям:
  - `int` — тип: целое число (мы его уже видели у `main`).
  - `age` — имя. Имена пишем латиницей: буквы, цифры (не первой) и `_`. Регистр важен, `age` и `Age` — разные переменные. Пробелов и дефисов быть не может, поэтому длинные имена пишут `max_value` или `maxValue`.
  - `=` — оператор **присваивания**: «положи в коробку `age` значение справа». Это **не** «равно» из математики, это команда «запиши». Запомни сразу: один `=` присваивает, два `==` сравнивают.
  - `25` — само значение.
  - `;` — конец инструкции.

- `printf("Возраст: %d\n", age);` — тут новое:
  - `%d` внутри строки — **спецификатор формата**: «сюда подставь целое число в десятичной записи» (`d` — *decimal*).
  - после строки, через запятую, мы передали `age`. Его значение и встанет на место `%d`.

Компилируем и запускаем — как в главе 1:

```
Возраст: 25
```

Значение можно менять сколько угодно раз: `age = 30;` затрёт старое содержимое коробки новым. Можно объявить переменную и без значения — `int age;` — но тогда до первой записи в ней лежит **мусор**: то, что случайно осталось в этом месте памяти. Читать такую переменную нельзя, сначала надо в неё что-то положить.

Шаг 2. Типы: сколько места и что туда влезает

Тип нужен компилятору, чтобы понять, сколько байт выделить под коробку и как трактовать лежащие там биты. Пока хватит четырёх типов:

- `int` — целое число (−7, 0, 42), спецификатор `%d`;
- `double` — дробное (3.14, −0.5), «двойная точность», `%f`;
- `char` — один символ (`'A'`, `'?'`), `%c`;
- `long long` — целое побольше, когда `int` мал, `%lld`.

```c
double y = 3.14159;
printf("%f\n", y);      // дробное как есть
printf("%.2f\n", y);    // два знака после точки
```

```
3.141590
3.14
```

`%f` печатает шесть знаков после точки — отсюда лишние нули. Точность задаётся через точку: `%.2f`. И заодно: `//` — это **комментарий**, всё от `//` до конца строки компилятор игнорирует. Есть и многострочный вид: `/* ... */`.

Одна ловушка сразу, чтобы потом не искать: `printf` печатает `double` через `%f`, а вот `scanf` (он появится в шаге 4) читает `double` только через `%lf` — с буквой `l`. С `%f` или `%d` в `scanf` компилятор выдаст предупреждение `-Wformat`, а в переменную попадёт чепуха.

`double` считает приближённо

Дробное число хранится в двоичном виде, а многие десятичные дроби в двоичной записи бесконечны, как 1/3 в десятичной (0,333…). Поэтому `0.1` в памяти — не ровно одна десятая, а

ближайшее к ней число, которое влезло в 8 байт. Обычно этого не видно, но стоит попросить побольше знаков:

```c
#include <stdio.h>

int main(void)
{
    double x = 0.1 + 0.2;
    printf("%.2f\n", x);
    printf("%.17f\n", x);
    printf("x == 0.3 -> %d\n", x == 0.3);
    return 0;
}
```

```
0.30
0.30000000000000004
x == 0.3 -> 0
```

С двумя знаками всё красиво, с семнадцатью вылезает хвост. И сравнение `x == 0.3` (про `==` подробно в шаге 6, пока читай его как «равно ли?») ответило 0 — «нет»: у суммы и у числа `0.3` разные хвосты. У `double` около 15–16 верных десятичных цифр, дальше — шум округления. Отсюда два правила: деньги и счётчики храни в целых (копейки, а не рубли с дробью), а дробные числа не сравнивай через `==`. Есть ещё тип `float` — 4 байта и всего около 7 верных цифр; в наших программах он не понадобится.

Смотрим размеры своими глазами

Сколько байт занимает `int`? В книгах пишут «обычно 4», но проверять надо на своей машине. Для этого есть оператор `sizeof`:

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("char      %zu байт\n", sizeof(char));
    printf("int       %zu байта\n", sizeof(int));
    printf("long      %zu байт\n", sizeof(long));
    printf("long long %zu байт\n", sizeof(long long));
    printf("double    %zu байт\n", sizeof(double));
    printf("char вмещает от %d до %d\n", CHAR_MIN, CHAR_MAX);
```

```
    printf("int  вмещает от %d до %d\n", INT_MIN, INT_MAX);
    return 0;
}
```

- `sizeof(int)` — размер типа в байтах. Это не функция, а оператор: для всех типов из этой книги он считается **на этапе компиляции**, программа во время работы ничего не измеряет.
- `%zu` — спецификатор для того типа, который возвращает `sizeof` (он называется `size_t`). Через `%d` его печатать неправильно.
- `#include <limits.h>` — ещё один заголовочный файл. Он не приносит функций, только готовые константы: `INT_MIN` и `INT_MAX` — самое маленькое и самое большое целое, влезающее в `int`, `CHAR_MIN` и `CHAR_MAX` — то же для `char`.

Запуск на моей машине (Apple clang 21, Mac с процессором Intel):

```
char      1 байт
int       4 байта
long      8 байт
long long 8 байт
double    8 байт
char вмещает от -128 до 127
int  вмещает от -2147483648 до 2147483647
```

Вот и ответ: `int` — четыре байта, то есть 32 бита. Двоичных комбинаций из 32 битов ровно 2^{32} = 4 294 967 296; половина отдана отрицательным числам, половина — нулю и положительным. Отсюда границы: от -2^{31} = −2 147 483 648 до $2^{31} - 1$ = 2 147 483 647, примерно ±2,1 миллиарда. Положительных на одно меньше, потому что одна комбинация досталась нулю.

Стандарт Си этих чисел не обещает. Он гарантирует только минимум: `int` вмещает хотя бы от −32 767 до 32 767 (на микроконтроллерах `int` и правда бывает двухбайтным), `sizeof(char)` всегда равен 1, а `long` не меньше `int`. Уже `long` выдаёт разницу: на Mac и Linux он 8 байт, а в Windows — 4. Поэтому аккуратные программы пишут `sizeof` и `INT_MAX`, а не «4» и «2 миллиарда».

`char` на этом Маке вмещает от −128 до 127: это самое маленькое целое, со знаком. Со знаком ли `char`, стандарт тоже оставил на выбор компилятора: на Linux с процессором ARM тот же `char` будет от 0 до 255. В главе 10 это различие нам ещё встретится.

Что будет, если число не влезло

Коробка конечного размера. Что будет, если прибавить единицу к самому большому `int`? Спросим число у пользователя (как работает `scanf`, разберём в шаге 4, пока просто пользуйся), чтобы компилятор не знал его заранее, и сравним `big + 1` с `big`:

```c
#include <stdio.h>

int main(void)
{
    int big;
    if (scanf("%d", &big) != 1) {
        return 1;
    }
    printf("big + 1     = %d\n", big + 1);
    printf("big + 1 > big: %d\n", big + 1 > big);
    return 0;
}
```

Вторая строка печатает результат сравнения: 1, если `big + 1` больше `big`, и 0, если нет (сравнения подробно — в шаге 6). Сохрани её как `over.c`, собери без оптимизации (`-O0` — «уровень оптимизации ноль») и введи 2147483647, то есть `INT_MAX`:

```
cc -std=c11 -Wall -Wextra -O0 over.c -o over
echo 2147483647 | ./over
```

(`echo 2147483647 |` подаёт число программе на вход, как будто ты набрал его сам и нажал Enter.)

```
big + 1     = -2147483648
big + 1 > big: 0
```

Прибавили единицу к самому большому числу — получили самое маленькое. Это **переполнение** (overflow). Почему именно такое число, видно по битам. Отрицательные числа процессор хранит в **дополнительном коде**: старший бит означает «минус 2^{31}», остальные 31 — обычные плюсы. `INT_MAX` — это ноль и за ним 31 единица. Прибавляем единицу, все единицы превращаются в нули с переносом, и включается старший бит: `10000…0`, а это ровно $-2^{31} = -2\,147\,483\,648$.

Теперь та же программа, собранная как всегда, с `-O2`:

```
big + 1     = -2147483648
big + 1 > big: 1
```

Первая строка та же, а вторая утверждает, что −2 147 483 648 больше 2 147 483 647. Это не глюк. Переполнение знакового целого в Си — **неопределённое поведение** (undefined behavior, UB): стандарт говорит, что после него программа вправе сделать что угодно. Оптимизатор этим пользуется: он рассуждает «переполнения не бывает, значит, `x + 1` всегда больше `x`» и заменяет сравнение готовой единицей, ничего не считая. В одной программе два противоречащих друг другу ответа.

Поймать такое помогает сам компилятор. Флаг `-fsanitize=undefined` вставляет в программу проверки, и при переполнении она сообщает о нём прямо во время работы:

```
cc -std=c11 -fsanitize=undefined over.c -o over
echo 2147483647 | ./over
```

```
over.c:9:38: runtime error: signed integer overflow: 2147483647 + 1
cannot be represented in type 'int'
```

«Строка 9, символ 38: знаковое переполнение, 2147483647 + 1 не представимо в типе `int`». В терминале это одна строка; за ней идёт строка `SUMMARY` с тем же местом, потом обычная строка вывода `big + 1     = -2147483648`, и то же самое повторяется для строки 10, где снова считается `big + 1`.

Запомни признак: сумма или произведение положительных чисел вдруг стала отрицательной — значит, `int` лопнул, и дальше программе верить нельзя. Лечится более широким типом, `long long`, а лучше — проверкой **до** операции; подробно займёмся этим в главе 4 на факториале.

У целых **без знака** (`unsigned int`, печатаются через `%u`) правило другое, и оно как раз определено стандартом: счёт идёт по кругу, по модулю 2^{32}. Самое большое `unsigned int` (константа `UINT_MAX`), увеличенное на единицу, честно превращается в 0:

```c
unsigned int u = UINT_MAX;
printf("u     = %u\n", u);
u = u + 1;
printf("u + 1 = %u\n", u);
```

```
u     = 4294967295
u + 1 = 0
```

Это не ошибка и не UB, а обещанное поведение — как у одометра, который после 999 999 показывает 000 000.

`char` — это маленькое целое

Тип `char` называют символьным, но внутри там обычное целое число — код символа. Никакой отдельной «буквы» в памяти нет. Убедимся: напечатаем одну переменную двумя разными спецификаторами.

```
char c = 'A';
printf("символ %c — это число %d\n", c, c);
printf("а %c — это %d\n", c + 1, c + 1);
```

```
символ A — это число 65
а B — это 66
```

- `'A'` в одинарных кавычках — символьная константа, ровно один знак. Не путай с `"A"` в двойных: это строка (о строках — глава 3).
- `%c` говорит `printf`: «трактуй число как код символа и напечатай сам символ». `%d` говорит: «напечатай число».
- `c + 1` — к символу можно прибавить единицу, потому что это число. 65 + 1 = 66, а 66 — это код `'B'`. Эти коды берутся из таблицы **ASCII**, где латинские буквы идут подряд, поэтому такой арифметикой удобно пользоваться: `'a' + 3` даст `'d'`. Стандарт Си подряд идущими обещает только цифры `'0'`…`'9'`, но на всех машинах, где ты будешь запускать примеры книги, латиница тоже идёт подряд.
- `c + 1` при печати через `%d` и `%c` передаётся в `printf` как обычный `int`: в арифметике `char` автоматически расширяется до `int`.

Размеры типов, беззнаковый круг и опыт с `char` собраны в один файл `code/02-types.c`. Целиком он печатает:

```
char      1 байт
int       4 байта
long      8 байт
long long 8 байт
double    8 байт
char вмещает от -128 до 127
int  вмещает от -2147483648 до 2147483647
u     = 4294967295
u + 1 = 0
символ A — это число 65
а B — это 66
```

Знакового переполнения в этом файле нет намеренно: программу книги, чей вывод зависит от настроения оптимизатора, нельзя проверить.

Шаг 3. `printf` — разбираем как функцию

Мы зовём `printf` с первой главы, но ни разу не посмотрели на неё как на функцию. Итак:

- **Живёт** в `<stdio.h>` (*standard input/output*). Без этого `#include` компилятор про неё не знает.
- **Принимает** первым аргументом строку формата, а дальше — сколько угодно аргументов, по одному на каждый спецификатор. Функции с переменным числом аргументов называют вариадическими; их в Си немного, и `printf` — главная.
- **Возвращает** `int` — сколько байт напечатала, а если печать не удалась — отрицательное число.
- Подводный камень: она не проверяет, что тип аргумента совпал со спецификатором.

Проверим, что возвращаемое значение не выдумка:

```c
#include <stdio.h>

int main(void)
{
    int n = printf("Привет, мир!\n");
    printf("printf напечатал %d байт\n", n);
    int m = printf("ABC\n");
    printf("а тут %d\n", m);
    return 0;
}
```

```
Привет, мир!
printf напечатал 22 байт
ABC
а тут 4
```

С `ABC\n` всё сходится: три буквы плюс перевод строки. А почему у «`Привет, мир!\n`» вышло 22, если знаков там 13? Потому что считаются **байты**: в кодировке UTF-8 каждая русская буква занимает два. Как раз поэтому все посимвольные алгоритмы в книге мы будем писать на латинице — с ней «один символ = один байт».

Если спецификатор не совпал с типом

Напишем заведомо неправильно: напечатаем `double` через `%d`.

```c
#include <stdio.h>

int main(void)
```

```
{
    double price = 19.99;
    printf("Цена: %d\n", price);
    return 0;
}
```

Компилятор с `-Wall` ругается сразу:

```
bad.c:6:30: warning: format specifies type 'int' but the argument has
      type 'double' [-Wformat]
    6 |     printf("Цена: %d\n", price);
      |                    ~~     ^~~~~
      |                    %f
1 warning generated.
```

«Формат обещает тип `int`, а аргумент имеет тип `double`», и внизу подсказка — надо `%f`. Но warning не ошибка, программа собралась. Запускаем:

```
Цена: -1112010808
```

Мусор. На этом процессоре целые аргументы и дробные передаются в разных местах (в разных регистрах — ячейках внутри процессора). `printf` послушно взял целое оттуда, где обычно лежат целые, а `double` туда не клали. У тебя число будет другим, и от запуска к запуску оно меняется: по стандарту несовпадение спецификатора с типом — неопределённое поведение, результат ничем не гарантирован.

Отсюда правило: **собирай всегда с `-Wall -Wextra` и не оставляй ни одного предупреждения**. Компилятор ловит такое бесплатно, а без него ты будешь искать это глазами.

Ширина поля

Спецификатор умеет не только «какой тип», но и «сколько места занять». `%3d` значит «напечатай целое, заняв не меньше трёх позиций, слева добей пробелами». Числа выравниваются по правому краю — ровно то, что нужно для таблиц. Если число длиннее ширины, оно не обрезается, а печатается целиком и просто сдвигает столбик: `%3d` для 1000 даст `1000`. Скоро пригодится.

Шаг 4. `scanf` — спрашиваем у пользователя

Считать число с клавиатуры умеет `scanf` (*scan formatted*), тоже из `<stdio.h>`.

```
#include <stdio.h>
```

```c
int main(void)
{
    int age;
    printf("Сколько тебе лет? ");
    scanf("%d", &age);
    printf("Через год будет %d\n", age + 1);
    return 0;
}
```

Запуск (после вопроса показано то, что ты набрал):

```
Сколько тебе лет? 25
Через год будет 26
```

Что нового:

- `int age;` — объявили переменную **без** значения, сейчас заполним её вводом.
- `"%d"` — формат: «ожидаю целое число». `scanf` пропустит пробелы и переводы строк, потом прочитает необязательный знак `+` или `-` и подряд идущие цифры. На первом символе, который в число не годится, она остановится и оставит его лежать во входе: `12abc` даст 12, `3.7` даст 3 (а `.7` останется непрочитанным).
- `&age` — знак `&` означает «**адрес** переменной `age`», то есть «где в памяти лежит коробка `age`». Разница принципиальная: `age` — это значение (25), `&age` — это место. `scanf` должна не прочитать переменную, а **записать** в неё, поэтому ей нужно место.
- `age + 1` — арифметика прямо в аргументе `printf`: сначала посчитается сумма, потом подставится в `%d`.

Что будет, если забыть `&`

Ошибка номер один у новичков, посмотрим на неё вживую. Файл `noamp.c` (вопрос `printf` убран, чтобы не мешал):

```c
#include <stdio.h>

int main(void)
{
    int age;
    scanf("%d", age);
    printf("Через год будет %d\n", age + 1);
    return 0;
}
```

Компилятор с `-Wall -Wextra` выдаёт два предупреждения:

```
noamp.c:6:17: warning: format specifies type 'int *' but the argument
      has type 'int' [-Wformat]
    6 |     scanf("%d", age);
      |            ~~   ^~~
noamp.c:6:17: warning: variable 'age' is uninitialized when used here
      [-Wuninitialized]
    6 |     scanf("%d", age);
      |                 ^~~
noamp.c:5:12: note: initialize the variable 'age' to silence this
      warning
    5 |     int age;
      |            ^
      |             = 0
2 warnings generated.
```

Первое: «формат обещает `int *` (адрес целого), а аргумент — `int`». Второе: «переменная `age` используется неинициализированной» — мы скормили `scanf` тот самый мусор, который в ней лежал, и она приняла его за адрес.

Дальше как повезёт. Собранная без `-O2`, программа на моём Маке после ввода `25` падает:

```
zsh: segmentation fault  ./noamp
```

Segmentation fault — «нарушение сегментации»: программа полезла в память, которая ей не принадлежит, и система её убила. Та же программа с `-O2` не упала, а напечатала `Через год будет 73896` — число, которое никто не вводил. Падение тут лучший исход: молчаливая порча памяти и чепуха в ответе опаснее, потому что их можно не заметить.

Правило: **в `scanf` перед обычной переменной всегда стоит `&`.**

`scanf` возвращает число, и его надо проверять

`scanf` возвращает, **сколько значений успешно прочитала**. Один `%d` в формате — при удаче вернёт 1. А если удачи нет?

```c
#include <stdio.h>

int main(void)
{
    int age;
    printf("Сколько тебе лет? ");
```

```
    int ok = scanf("%d", &age);
    printf("scanf вернул %d, age = %d\n", ok, age);
    return 0;
}
```

Три запуска: с числом, с буквами и вообще без ввода (ввод закрывается клавишами Ctrl+D, нажатыми в начале пустой строки).

```
Сколько тебе лет? 25
scanf вернул 1, age = 25
```

```
Сколько тебе лет? abc
scanf вернул 0, age = 32760
```

```
Сколько тебе лет? scanf вернул -1, age = 32760
```

- `1` — прочитано одно значение, всё хорошо.
- `0` — «данные на входе есть, но это не то, что я просила». `age` осталась нетронутой, там мусор (у тебя будет другое число).
- `-1` — это константа `EOF` (*end of file*, «конец файла»): читать больше нечего. Стандарт обещает только, что `EOF` отрицательная; на macOS и Linux она равна −1. Поэтому сравнивай с именем `EOF`, а не с числом.

Главное здесь: при `0` и `-1` переменная **не заполнилась**, и наша программа напечатала мусор. Если не проверить результат, дальше она так же спокойно будет считать по мусору. Поэтому в книге ты всюду будешь видеть такую форму:

```
if (scanf("%d", &n) != 1) {
    printf("Это не число.\n");
    return 1;
}
```

`return 1;` из `main` означает «завершились с ошибкой»: ноль — успех, любое другое число — код ошибки, так принято во всех командах терминала.

И одна дыра, которую проверка результата **не** закрывает. Если ввести число, которое не влезает в `int`, `scanf` всё равно вернёт 1:

```
Сколько тебе лет? 99999999999
scanf вернул 1, age = 1215752191
```

По стандарту это снова неопределённое поведение, и на практике в переменную попадает обрезок. Для учебных программ с этим можно жить; когда понадобится надёжный разбор чисел, есть функция `strtol`, которая умеет сообщать о переполнении.

Буква вместо числа: почему программа зависает

Логично попросить ввод заново, если он оказался плохим:

```
int n;
printf("Введи число: ");
while (scanf("%d", &n) != 1) {
    printf("Это не число. Ещё раз: \n");
}
printf("Принято: %d\n", n);
```

Выглядит разумно. Запускаем, вводим `abc`:

```
Введи число: Это не число. Ещё раз:
Это не число. Ещё раз:
Это не число. Ещё раз:
Это не число. Ещё раз:
Это не число. Ещё раз:
Это не число. Ещё раз:
```

И так до бесконечности, пока не нажмёшь Ctrl+C.

Причина вот в чём: наткнувшись на букву, `scanf` **не забирает** её из входного потока. Она возвращает 0 и оставляет `abc` лежать там же. На следующем витке цикла она видит те же самые буквы, снова возвращает 0 — и по кругу. Значит, плохой ввод надо выбросить самим.

Настоящий фикс

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Введи целое число: ");
    while (scanf("%d", &n) != 1) {
        int c;
        while ((c = getchar()) != '\n' && c != EOF) {
        }
        if (c == EOF) {
            printf("Ввод кончился, выхожу.\n");
            return 1;
```

```
        }
        printf("Это не число. Попробуй ещё раз: ");
    }

    long long twice = n;    /* копия в широкую коробку */
    printf("Принято: %d\n", n);
    printf("Удвоенное: %lld\n", twice * 2);
    return 0;
}
```

Новое здесь — `getchar`:

- **Живёт** в `<stdio.h>`, **принимает** ничего.
- **Возвращает** `int` — очередной байт со входа как число от 0 до 255, либо `EOF`, если читать больше нечего. Ровно 257 разных ответов, в `char` (256 значений) они не помещаются.
- Подводный камень: результат `getchar` всегда кладут в `int`. Положишь в `char` — на этом Маке, где `char` от −128 до 127, байт 255 превратится в −1 и программа примет его за конец ввода. А там, где `char` беззнаковый, наоборот, `EOF` превратится в 255 и конец ввода не распознается никогда.

Разбираем строку `while ((c = getchar()) != '\n' && c != EOF) { }`:

- `c = getchar()` — прочитали символ и **положили в `c`**. В Си присваивание само является выражением со значением — тем, которое присвоили. Поэтому его можно сравнивать прямо на месте.
- Внешние скобки вокруг `(c = getchar())` обязательны: у `!=` приоритет выше, чем у `=`, и без скобок получилось бы `c = (getchar() != '\n')` — в `c` легла бы не буква, а результат сравнения, 0 или 1.
- Тело цикла пустое: `{ }`. Делать с этими символами ничего не надо, их надо просто проглотить.
- После цикла — `if (c == EOF)`. Без этой проверки при закрытом вводе получился бы новый вечный цикл: `scanf` вечно возвращает −1, а чистить уже нечего.

Запускаем и вводим по очереди `abc`, `hex`, `7`:

```
Введи целое число: abc
Это не число. Попробуй ещё раз: hex
Это не число. Попробуй ещё раз: 7
Принято: 7
Удвоенное: 14
```

Осталась строка `long long twice = n;`. Зачем она? Если удваивать прямо в `int`, ввод `1073741824` (это 2^{30}) даст 2^{31}, которое в `int` уже не влезает, — то самое переполнение из шага 2, и программа напечатала бы `-2147483648`. Поэтому число сначала копируется в переменную `long long` (8 байт, до $9{,}2 \cdot 10^{18}$), и умножение идёт уже в ней. Печатаем через `%lld`. Проверим края:

```
Введи целое число: 2147483647
Принято: 2147483647
Удвоенное: 4294967294
```

И последнее: строку `12abc` программа примет как 12 — `scanf` прочитает цифры и остановится на букве, а это для неё успех. Буквы `abc` останутся лежать во входе до следующего `scanf`. Для наших учебных программ это не страшно, просто помни, что `scanf` проверяет начало ввода, а не всю строку.

Программа лежит в `code/02-input.c`. Приём «проверил результат `scanf` → вычистил строку → спросил заново» встретится ещё много раз, особенно в играх из второй половины книги.

Шаг 5. Арифметика и её ловушки

Операторы почти как в математике: `+`, `-`, `*`, `/` и `%` — остаток от деления. Но три вещи ведут себя не так, как ждёт человек, впервые увидевший Си.

```c
#include <stdio.h>

int main(void)
{
    printf("7 / 2   = %d\n", 7 / 2);
    printf("7 %% 2   = %d\n", 7 % 2);
    printf("7.0 / 2 = %f\n", 7.0 / 2);
    printf("-7 / 2  = %d\n", -7 / 2);
    printf("-7 %% 2  = %d\n", -7 % 2);
    printf("7 %% -2  = %d\n", 7 % -2);
    printf("2 + 3 * 4   = %d\n", 2 + 3 * 4);
    printf("(2 + 3) * 4 = %d\n", (2 + 3) * 4);
    printf("10 - 4 - 3  = %d\n", 10 - 4 - 3);
    return 0;
}
```

Сразу мелочь, которая смущает: `%%`. Знак процента для `printf` служебный, поэтому, чтобы напечатать сам процент, его удваивают — на экране получится один `%`.

```
7 / 2   = 3
7 % 2   = 1
7.0 / 2 = 3.500000
-7 / 2  = -3
-7 % 2  = -1
7 % -2  = 1
2 + 3 * 4   = 14
(2 + 3) * 4 = 20
10 - 4 - 3  = 3
```

Целочисленное деление. `7 / 2` дало **3**, а не 3.5. Если оба операнда (числа по бокам от знака операции) целые, деление тоже целое: дробная часть просто отбрасывается. Чтобы получить 3.5, хотя бы одно число должно быть дробным — `7.0 / 2`.

Посмотри и на `-7 / 2`: вышло **−3**, а не −4. Точный ответ −3,5, и Си отбрасывает дробную часть, то есть округляет **к нулю**: 3,5 → 3, −3,5 → −3. Так требует стандарт начиная с C99. Школьное «округление вниз» дало бы −4 — не путай.

Из целочисленного деления вырастает классический баг:

```c
int sum = 7, count = 2;
double avg = sum / count;      // avg станет 3.0, а не 3.5!
```

Деление произошло **до** записи в `double`, спасать уже нечего. Лечится приведением типа: `double avg = (double)sum / count;`. Запись `(double)x` называется **приведением типа** (cast) и означает «считай это значение дробным». Достаточно привести один операнд, второй подтянется сам.

И ещё про деление: на ноль в целых делить нельзя, это неопределённое поведение. На этом Маке программа, которой во время работы попался делитель 0 (скажем, его ввёл пользователь), просто падает с сообщением `zsh: floating point exception` (хотя числа целые — такое уж у этого сигнала историческое имя). Если делитель приходит от пользователя, проверяй его на ноль заранее.

Знак остатка. `%` даёт остаток: `7 % 2` равно 1. Штука невероятно полезная — `n % 2` равно нулю ровно для чётных, `n % 10` даёт последнюю цифру неотрицательного `n`, `i % 3` крутится по кругу 0, 1, 2, 0, 1, 2… Но с минусами внимательно: `-7 % 2` дало **−1**, а не 1.

Остаток в Си определён через деление: для целых `a` и `b` (где `b` не ноль, а частное помещается в `int`) выполняется равенство `a == (a / b) * b + a % b`. Подставим: `-7 / 2` равно −3, значит, `-7 % 2` = −7 − (−3)·2 = −7 + 6 = −1. Для `7 % -2`: `7 / -2` равно −3, остаток = 7 − (−3)·(−2) = 7 − 6 = 1. Вывод: ненулевой остаток в Си имеет знак **делимого** (левого числа), а знак делителя на него не влияет.

В математике «остаток» (его пишут `mod`) всегда неотрицательный: $-7 \bmod 2 = 1$, потому что $-7 = (-4) \cdot 2 + 1$. Оператор `%` в Си — **не** математический `mod`, для отрицательных чисел они расходятся.

Практический вывод: проверка на нечётность через `n % 2 == 1` **сломается** на отрицательных числах:

```
int n = -7;
printf("n %% 2       = %d\n", n % 2);
printf("n %% 2 == 1  -> %d\n", n % 2 == 1);
printf("n %% 2 != 0  -> %d\n", n % 2 != 0);
```

```
n % 2       = -1
n % 2 == 1  -> 0
n % 2 != 0  -> 1
```

Число −7 нечётное, но `n % 2 == 1` ответило «нет». Правильная проверка на нечётность — `n % 2 != 0`. А на чётность годится и `n % 2 == 0`: у нуля знака нет.

Приоритет. `2 + 3 * 4` дало 14: умножение и деление выполняются раньше сложения и вычитания, как в школе. Скобки этот порядок переопределяют. Операторы одного уровня считаются слева направо: `10 - 4 - 3` — это `(10 - 4) - 3 = 3`. Остаток `%` стоит на одном уровне с `*` и `/`, поэтому `1 + n % 2` — это `1 + (n % 2)`. Минус перед числом (унарный, у которого один операнд) связывает сильнее всех них: `-7 % 2` — это `(-7) % 2`, как мы и получили. Всю таблицу приоритетов заучивать не надо, надёжнее правило: сложнее двух операций — ставь скобки.

`i++` и `++i` — не одно и то же

`i = i + 1` пишут короче: `i++` или `++i`. Обе записи увеличивают `i` на единицу, но **значение самого выражения** у них разное. `i++` (постфиксный инкремент) возвращает старое значение и потом увеличивает; `++i` (префиксный) сначала увеличивает и возвращает уже новое. Разница видна, только если результат куда-то используется:

```
#include <stdio.h>

int main(void)
{
    int i = 5;
    int a = i++;
    printf("после a = i++:  a = %d, i = %d\n", a, i);

    int j = 5;
    int b = ++j;
```

```
    printf("после b = ++j:  b = %d, j = %d\n", b, j);
    return 0;
}
```

```
после a = i++:  a = 5, i = 6
после b = ++j:  b = 6, j = 6
```

`i` и `j` обе стали шестёрками — тут разницы нет. А `a` получила старую пятёрку, `b` — новую шестёрку.

Когда инкремент стоит отдельной строкой (`i++;`) или в третьей части `for`, разницы никакой, и пишут обычно `i++`. Не пихай `i++` внутрь сложных выражений: это верный способ получить код, который никто не может прочитать. А менять одну переменную дважды в одном выражении, как в `i = i++ + 1;`, вообще нельзя: это неопределённое поведение. Clang об этом предупреждает:

```
seq.c:6:10: warning: multiple unsequenced modifications to 'i'
[-Wunsequenced]
    6 |     i = i++ + 1;
      |       ~  ^
```

«Несколько неупорядоченных изменений `i`»: стандарт не говорит, какое из двух присваиваний случится последним. Похожие сокращения: `i--`, `i += 3`, `i -= 2`, `i *= 2`, `i /= 2` — краткая запись `i = i + 3` и так далее.

Шаг 6. Принимаем решения — `if` и `else`

Программа должна уметь выбирать путь: «если число чётное — сделай так, иначе — эдак». Для этого есть `if`.

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }
```

```c
    if (n % 2 == 0) {
        printf("%d — чётное\n", n);
    } else {
        printf("%d — нечётное\n", n);
    }
    return 0;
}
```

Два запуска:

```
Введи число: 7
7 — нечётное
```

```
Введи число: 10
10 — чётное
```

Разбор:

- `if (условие) { ... }` — если условие истинно, выполняется блок в фигурных скобках. Условие пишут в круглых скобках, и после `)` точка с запятой **не ставится**.
- `n % 2 == 0` — тут два оператора: `%` даёт остаток, а `==` **сравнивает**. Ещё раз: `==` сравнивает, `=` присваивает.
- `else { ... }` — «иначе», выполняется, если условие оказалось ложным. Не обязателен.
- `return 1;` в середине `main` мгновенно завершает программу — до второго `if` дело не дойдёт.

`=` вместо `==` — классический баг

Что будет, если в условии написать один знак равенства?

```c
int n = 3;
if (n = 5) {
    printf("n равно 5? n = %d\n", n);
}
```

Компилятор с `-Wall` предупреждает, и очень подробно:

```
asg.c:6:11: warning: using the result of an assignment as a condition
      without parentheses [-Wparentheses]
    6 |     if (n = 5) {
      |         ~~^~~
asg.c:6:11: note: place parentheses around the assignment to silence
```

```
      this warning
    6 |     if (n = 5) {
      |           ^
      |         (    )
asg.c:6:11: note: use '==' to turn this assignment into an equality
      comparison
    6 |     if (n = 5) {
      |           ^
      |           ==
1 warning generated.
```

А программа печатает:

```
n равно 5? n = 5
```

Условие оказалось истинным, да ещё и `n` испортилась: было 3, стало 5. Оба зла сразу. Компилятор прямым текстом советует `==` — слушайся.

Сравнения и логика

Сравнения: `==` равно, `!=` не равно, `<`, `>`, `<=`, `>=`.

Для условий Си обходится обычными числами: **ложь — это 0, истина — любое ненулевое число**. (Начиная с C99 есть и тип `bool` из заголовка `<stdbool.h>` со значениями `true` и `false`, но внутри это те же 1 и 0.) Сравнение возвращает ровно 1 или 0, это обычный `int`, его можно печатать через `%d` — мы так и делали, когда проверяли `n % 2 != 0`. Отсюда и коварство `if (n = 5)`: пятёрка не ноль, значит истина.

Условия соединяют:

- `&&` — «И», истина, когда истинны оба: `if (age >= 18 && age < 65);`
- `||` — «ИЛИ», истина, когда истинно хотя бы одно: `if (c == 'y' || c == 'Y');`
- `!` — «НЕ», переворачивает: `if (!found)` — «если не найдено».

Полезная деталь: `&&` и `||` **ленивые**. В `a && b` правая часть вообще не вычисляется, если левая ложна, а в `a || b` — если левая истинна: ответ в обоих случаях уже известен. Этим пользуются постоянно: `if (i < n && a[i] > 0)` сначала убедится, что индекс не вылез за границу, и только потом полезет в массив (массивы — глава 3).

Если веток несколько, их сцепляют через `else if`:

```c
if (n > 0) {
    printf("положительное\n");
```

```
} else if (n < 0) {
    printf("отрицательное\n");
} else {
    printf("ноль\n");
}
```

Проверки идут сверху вниз, и как только одна сработала, остальные пропускаются. Последний `else` ловит всё, что не подошло.

Шаг 7. Повторяем — циклы

`while`

Самый понятный цикл — `while` («пока»): пока условие истинно, повторяй тело. Напечатаем числа от 1 до 5:

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
printf("\n");
```

```
1 2 3 4 5
```

- `int i = 1;` — заводим **счётчик**.
- `while (i <= 5)` — условие проверяется **перед** каждым проходом.
- в `%d` после `d` стоит пробел, поэтому числа не слипаются.
- `i++;` — увеличиваем счётчик. **Без этой строки условие никогда не станет ложным и программа зациклится навсегда.** Случилось — Ctrl+C.
- отдельный `printf("\n")` после цикла переводит строку, чтобы приглашение терминала не приклеилось к нашим числам.

`for`

Когда у цикла есть счётчик с понятным началом, концом и шагом, удобнее `for`. Тот же вывод 1..5:

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
```

```
}
printf("\n");
```

```
1 2 3 4 5
```

В скобках `for` — три части через `;`:

1. `int i = 1` — **инициализация**, выполняется один раз в начале.
2. `i <= 5` — **условие**, проверяется перед каждым повтором.
3. `i++` — **шаг**, выполняется в конце каждого повтора, перед следующей проверкой условия.

Это почти тот же `while`, только всё про счётчик собрано в одном месте и его не забудешь. (Одно отличие увидим ниже, у `continue`.) Переменная `i`, объявленная прямо в `for`, живёт только внутри цикла — снаружи её нет.

Как выбирать: знаешь заранее, **сколько раз** повторять (десять строк таблицы, все элементы массива) — бери `for`. Повторяешь, **пока что-то не случится** (пока не введут ноль, пока не угадали число) — бери `while`. Технически они взаимозаменяемы, но по такому коду сразу видно намерение автора.

`break` и `continue`

Два слова, вмешивающиеся в ход цикла: `break;` немедленно **выходит** из цикла целиком, `continue;` бросает текущий проход и переходит к следующему. В `for` шаг `i++` при этом всё равно выполнится. А вот в `while`, где `i++;` стоит последней строкой тела, `continue` перепрыгнет и через него — счётчик застынет, и цикл станет вечным.

```
for (int i = 1; i <= 10; i++) {
    if (i % 3 == 0) {
        continue;
    }
    if (i == 8) {
        break;
    }
    printf("%d ", i);
}
printf("\n");
```

```
1 2 4 5 7
```

3 и 6 пропущены — на них сработал `continue`. На восьмёрке сработал `break` и вышиб нас из цикла, поэтому 8, 9 и 10 не напечатались вовсе.

do-while

Третий цикл проверяет условие **после** тела, поэтому тело выполняется хотя бы один раз. Разница видна на условии, ложном с самого начала:

```c
int i = 10;
while (i <= 5) {
    printf("while: %d\n", i);
    i++;
}

int j = 10;
do {
    printf("do-while: %d\n", j);
    j++;
} while (j <= 5);
```

```
do-while: 10
```

`while` не напечатал ничего: условие `10 <= 5` ложно, тело ни разу не запустилось. `do-while` напечатал строку — он сначала выполнил тело и только потом проверил. Заметь точку с запятой после `while (j <= 5);`: в `do-while` она обязательна.

Шаг 8. Свои функции

`printf`, `scanf`, `getchar` — чужие функции. Пишем свои: чтобы не повторять код и дать кускам логики понятные имена.

```c
#include <stdio.h>

int square(int x)
{
    return x * x;
}

int main(void)
{
    int n = 6;
    printf("%d в квадрате = %d\n", n, square(n));
    return 0;
}
```

Разбор `int square(int x)`:

- `int` (первый) — тип того, что функция **вернёт**.
- `square` — имя, придумали сами.
- `(int x)` — **параметр**: функция принимает одно целое и внутри называет его `x`. Параметров может быть несколько через запятую, у каждого свой тип: `int max2(int a, int b)`.
- `return x * x;` — посчитать и **вернуть** наружу. `return` заодно завершает функцию: код после него не выполнится.

В `main` запись `square(n)` — это **вызов**: передаём `n`, получаем результат, который тут же подставляется в `printf`.

Прототип: функция должна быть объявлена до вызова

Компилятор читает файл сверху вниз. Переставим `square` вниз, под `main` (файл `noproto.c`, вызов для краткости с готовым числом):

```c
#include <stdio.h>

int main(void)
{
    printf("%d\n", square(6));
    return 0;
}

int square(int x)
{
    return x * x;
}
```

```
noproto.c:5:20: error: call to undeclared function 'square'; ISO C99
      and later do not support implicit function declarations
      [-Wimplicit-function-declaration]
    5 |     printf("%d\n", square(6));
      |                    ^
1 error generated.
```

Это уже не warning, а **error** — сборки не будет. «Вызов необъявленной функции». Первый способ починить — держать функции выше `main`, как мы и делали. Второй, более гибкий, — **прототип**: объявление без тела, обещающее компилятору «такая функция есть, вот её вход и выход, тело будет ниже».

```c
#include <stdio.h>

int square(int x);          /* прототип: обещание компилятору */

int main(void)
{
    printf("square(6) = %d\n", square(6));
    return 0;
}

int square(int x)           /* тело — ниже */
{
    return x * x;
}
```

```
square(6) = 36
```

Прототип — это первая строка функции плюс точка с запятой вместо тела. Имя параметра в нём можно опустить (`int square(int);`), но с именем понятнее. Так устроены и заголовочные файлы: в `<stdio.h>` лежат прототипы `printf` и `scanf`, а тела — в готовой библиотеке. Потому `#include <stdio.h>` и обязателен.

Локальные переменные и область видимости

Переменные, объявленные внутри функции, называются **локальными**: они рождаются при входе в функцию и исчезают при выходе. Снаружи их не существует, и одноимённые переменные в разных функциях никак не связаны.

```c
#include <stdio.h>

int square(int x)
{
    int result = x * x;
    return result;
}

int main(void)
{
    int result = 100;
    printf("square(6) = %d\n", square(6));
```

```
    printf("result в main = %d\n", result);
    return 0;
}
```

```
square(6) = 36
result в main = 100
```

Две разные переменные с именем `result`, и они не мешают друг другу. Правило шире: переменная видна от места объявления до закрывающей фигурной скобки того блока, где объявлена. Поэтому `int i` внутри `for` не виден после цикла.

Аргументы передаются по значению

Главный факт про функции в Си: **функция получает копию аргумента, а не сам аргумент**. Что бы она с ним ни делала, снаружи ничего не изменится.

```
#include <stdio.h>

void add_ten(int x)
{
    x = x + 10;
    printf("внутри функции x = %d\n", x);
}

int main(void)
{
    int n = 5;
    printf("до вызова   n = %d\n", n);
    add_ten(n);
    printf("после вызова n = %d\n", n);
    return 0;
}
```

```
до вызова   n = 5
внутри функции x = 15
после вызова n = 5
```

Внутри `x` честно стал пятнадцатью, а `n` в `main` как был пятёркой, так и остался: при вызове значение `n` скопировалось в новую коробочку `x`, и функция всю дорогу возилась с копией.

Отсюда два следствия. Если функция должна что-то сообщить наружу — она это **возвращает** через `return`. А если нужно изменить саму переменную вызывающего, придётся передать её

адрес — тот самый `&`, который мы уже видели у `scanf`. Это указатели, ими займёмся в главе 3. Запомни этот запуск, мы к нему вернёмся.

`void` — функция, которая ничего не возвращает

У `add_ten` вместо типа результата стоит `void`:

```
void hello(void)
{
    printf("Привет!\n");
}
```

`void` как тип результата означает «ничего не возвращаю». Внутри такой функции можно написать `return;` без значения, чтобы выйти досрочно, а можно не писать вовсе. `void` в скобках, как в `main(void)`, означает другое — «не принимаю аргументов».

Полная программа: таблица умножения

Начнём с наивной версии — без ввода, без функций, только цикл и `printf`:

```
#include <stdio.h>

int main(void)
{
    int n = 7;

    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
```

```
7 x 9 = 63
7 x 10 = 70
```

Считает верно, но столбик съехал: в последней строке множитель двузначный, и знак = уехал вправо. Лечится шириной поля — той самой, про которую был разговор в шаге 3. Множителю дадим две позиции, результату три:

```
printf("%2d x %2d = %3d\n", n, i, n * i);
```

```
 7 x  1 =   7
 7 x  2 =  14
 7 x  3 =  21
 7 x  4 =  28
 7 x  5 =  35
 7 x  6 =  42
 7 x  7 =  49
 7 x  8 =  56
 7 x  9 =  63
 7 x 10 =  70
```

Всё выровнено по правому краю. Осталось спросить число у пользователя (с проверкой!) и пометить чётные результаты — для этого заведём свою функцию `is_even`.

```
#include <stdio.h>

int is_even(int x)
{
    return x % 2 == 0;
}

int main(void)
{
    int n;

    printf("Таблица умножения для какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }
    if (n < 1 || n > 99) {
        printf("Нужно число от 1 до 99.\n");
```

```
        return 1;
    }

    for (int i = 1; i <= 10; i++) {
        int result = n * i;
        if (is_even(result)) {
            printf("%2d x %2d = %3d  (чётное)\n", n, i, result);
        } else {
            printf("%2d x %2d = %3d\n", n, i, result);
        }
    }

    return 0;
}
```

Сохрани её как `code/02-multiplication.c`.

Вторая проверка, `if (n < 1 || n > 99)`, появилась не для красоты. Ширины полей `%2d` и `%3d` рассчитаны на множитель до двух цифр и результат до трёх: 99 · 10 = 990 ещё влезает, а 100 · 10 = 1000 уже сломает столбик. Вдобавок без ограничения ввод вроде `300000000` дал бы `300000000 * 8` = 2,4 миллиарда — переполнение `int` из шага 2. Проверяем ввод на входе — и внутри цикла ни о чём таком думать не надо. `||` здесь — «ИЛИ»: условие истинно, если число меньше 1 **или** больше 99.

Как это работает

Пройдём программу глазами, как будто мы компьютер, для ввода `3`.

1. `int n;` — заводим коробку под число, пока в ней мусор.
2. `printf(...)` — печатаем вопрос. Перевода строки в конце нет, и курсор остаётся на той же строке: приглашение и ответ окажутся рядом.
3. `scanf("%d", &n)` — программа замирает и ждёт Enter. Ты набрал `3`, `scanf` разобрала это как целое, положила в `n` тройку и вернула 1. Значит, `!= 1` ложно, в `if` мы не заходим. Следующая проверка `3 < 1 || 3 > 99` тоже ложна — число подходит.
4. `for (int i = 1; i <= 10; i++)` — заводим `i = 1`, проверка `1 <= 10` истинна, входим в тело.
5. `int result = n * i;` — **новая** переменная, объявленная внутри тела цикла: на каждом витке создаётся заново. `3 * 1 = 3`.
6. `if (is_even(result))` — вызываем свою функцию. Внутри `x` — копия тройки, считается `3 % 2 == 0`, то есть `1 == 0`, то есть ложь, то есть 0. `if` видит ноль и идёт в `else`.

7. Печатается `3 x  1 =  3`.
8. Выполняется шаг `i++`, `i` становится 2, снова проверка условия. `3 * 2 = 6`, `6 % 2 == 0` истинно, `is_even` возвращает 1 — и работает ветка `if`, со словом «(чётное)».
9. Так десять раз. Когда `i` станет 11, условие `11 <= 10` ложно, цикл заканчивается, `return 0;` завершает программу.

Отдельно про `is_even`: она возвращает результат сравнения `x % 2 == 0`, а сравнение даёт 1 или 0. Значит, `is_even` возвращает 1 для чётных и 0 для нечётных, и `if (is_even(result))` срабатывает ровно на чётных. Писать `if (is_even(result) == 1)` не нужно — то же самое, только длиннее.

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/02-multiplication.c -o mult
./mult
```

Вводим 4:

```
Таблица умножения для какого числа? 4
 4 x  1 =   4  (чётное)
 4 x  2 =   8  (чётное)
 4 x  3 =  12  (чётное)
 4 x  4 =  16  (чётное)
 4 x  5 =  20  (чётное)
 4 x  6 =  24  (чётное)
 4 x  7 =  28  (чётное)
 4 x  8 =  32  (чётное)
 4 x  9 =  36  (чётное)
 4 x 10 =  40  (чётное)
```

Все строки помечены — и это правильно: чётное число, умноженное на что угодно, остаётся чётным. Хорошая проверка здравого смысла. Введи 3 — пометки начнут чередоваться (вывод показан в начале главы).

Теперь проверим защиту от плохого ввода, введём `abc`:

```
Таблица умножения для какого числа? abc
Это не число.
```

Программа не полезла считать по мусору, а вежливо вышла — ровно то, ради чего мы проверяем результат `scanf`. И число вне диапазона:

```
Таблица умножения для какого числа? 100
Нужно число от 1 до 99.
```

Частые ошибки

`=` вместо `==` в условии. `if (n == 5)` сравнивает, `if (n = 5)` присваивает и всегда даёт истину, попутно портя `n`. Компилятор с `-Wall` предупреждает — мы видели это предупреждение выше.

Забытый `&` в `scanf`. `scanf("%d", n)` — либо падение, либо молчаливая порча памяти. Правильно `scanf("%d", &n)`.

Непроверенный `scanf`. Если ввод не подошёл, переменная останется с мусором, а программа пойдёт считать как ни в чём не бывало. Всегда `if (scanf(...) != 1) { ... }`.

Вечный цикл на плохом вводе. Плохие символы `scanf` не забирает из потока. Просто повторить `scanf` в цикле мало — надо вычистить строку через `getchar` и не забыть проверить `EOF`.

Неверный спецификатор. `double` через `%d` — мусор на экране, `int` через `%f` — тоже. Тип и спецификатор обязаны совпадать, `-Wall` это ловит.

Точка с запятой после `if` или `for`. `if (n > 0);` — пустое тело, и следующий блок выполнится всегда. `for (int i = 0; i < 10; i++);` — цикл, крутящийся вхолостую.

Целочисленное деление врасплох. `double avg = sum / count;`, где `sum` и `count` целые, делит нацело **до** записи в `double`. Приводи: `(double)sum / count`.

Проверка на нечётность через `% 2 == 1`. На отрицательных не работает: `-7 % 2` равно −1. Пиши `n % 2 != 0`.

Бесконечный цикл. Забыл менять счётчик в `while` — программа висит. Ctrl+C, и добавь `i++`.

Переполнение `int`. Результат больше 2 147 483 647 не влезает в `int`. Это неопределённое поведение: чаще всего число молча становится отрицательным, но оптимизатор может выдать и что похуже (мы видели «−2 147 483 648 больше 2 147 483 647»). Числа могут вырасти — бери `long long` или ограничивай ввод, как в таблице умножения.

Деление на ноль. `a / 0` и `a % 0` с целыми — неопределённое поведение, на этом Маке программа падает. Делитель от пользователя проверяй заранее.

`%f` в `scanf` для `double`. Читать `double` надо через `%lf`, печатать — через `%f`.

Переменная объявлена, но не заполнена. `int sum;` и сразу `sum += x;` — прибавление к мусору. Счётчики и накопители всегда инициализируй: `int sum = 0;`.

Упражнения

1. Программа спрашивает два числа и печатает их сумму, разность, произведение и частное. Частное — дробное, через `%.2f`. *Подсказка*: `scanf("%d %d", &a, &b)` читает сразу два числа и при удаче возвращает 2; частное считай как `(double)a / b`, и не забудь, что делить на ноль нельзя — проверь `b` заранее. *Ожидаемый результат* для ввода `7 2`: сумма 9, разность 5, произведение 14, частное 3.50; для `7 0` — первые три числа (7, 7, 0) и сообщение, что на ноль делить нельзя, без падения.
2. Программа спрашивает число и говорит, положительное оно, отрицательное или ноль. *Подсказка*: цепочка `if` / `else if` / `else`, три ветки, ни одной лишней проверки. *Ожидаемый результат*: `5` — «положительное», `-3` — «отрицательное», `0` — «ноль».
3. С помощью `for` напечатай все чётные числа от 2 до 20 в одну строку. *Подсказка*: можно идти с шагом 2 (`i += 2`), а можно идти по всем и пропускать нечётные через `continue`. Сделай обоими способами и реши, какой понятнее. *Ожидаемый результат* у обоих способов одинаковый: `2 4 6 8 10 12 14 16 18 20`.
4. Напиши функцию `int max2(int a, int b)`, возвращающую большее из двух чисел, и проверь её в `main` на трёх парах, включая пару равных чисел. *Подсказка*: тело — один `if` с `return` в каждой ветке; при равных числах любой ответ верен, но функция обязана что-то вернуть, а не провалиться до конца тела. *Ожидаемый результат*: `max2(3, 7)` = 7, `max2(-2, -5)` = −2, `max2(4, 4)` = 4.
5. Цикл `while`: спрашивай числа, пока пользователь не введёт 0, и в конце напечатай их сумму и количество. *Подсказка*: накопитель `int sum = 0;` заведи **до** цикла; выходить удобно через `break`, когда прочитан ноль; проверяй результат `scanf` — на `EOF` тоже надо выйти, иначе получишь вечный цикл. *Ожидаемый результат* для ввода `5 3 -2 0`: сумма 6, количество 3 (сам ноль не считается).
6. Возьми `code/02-multiplication.c` и сделай так, чтобы программа спрашивала ещё и до скольки печатать таблицу (`3` до `5` даёт пять строк). *Подсказка*: вторая переменная и второй `scanf` с проверкой, а в условии `for` вместо `10` подставь её. Посмотри, что выйдет при вводе `0` и при отрицательном числе — и добавь проверку, если результат тебе не понравится. *Ожидаемый результат*: для `3` и `5` — строки от `3 x 1 = 3` до `3 x 5 = 15`; без проверки `0` и `-2` не дают ни одной строки (условие `1 <= 0` ложно сразу), поэтому разумно требовать второе число от 1 до 99, как и первое. Тогда результат дорастёт до $99 \cdot 99 = 9801$, и поле придётся расширить до `%4d`.

Что мы получили

Самый плотный по теории кусок книги позади. Теперь у тебя есть всё для алгоритмов:

- переменные, типы и их реальные размеры на твоей машине, `sizeof`;

- приближённость `double`: `0.1 + 0.2` не равно `0.3`;
- переполнение `int`, увиденное своими глазами, и почему это неопределённое поведение, а `unsigned` честно идёт по кругу;
- `printf` как настоящая функция: формат, аргументы, возвращаемое значение, ширина поля, warning при несовпадении типа;
- `scanf` с проверкой результата и чисткой потока после плохого ввода — то, что отличает рабочую программу от «падает, если ввести букву»;
- арифметика с ловушками: целочисленное деление с округлением к нулю, знак остатка (не путать с математическим `mod`), приоритет, `i++` против `++i`;
- ветвления `if` / `else if` / `else`, логика `&&`, `||`, `!` и правило «ложь — это ноль»;
- циклы `while`, `for`, `do–while` плюс `break` и `continue`;
- свои функции: параметры, `return`, прототип, локальные переменные, `void` — и главное, передача аргументов **по значению**.

Если что-то осталось туманным — нормально: дальше мы применим это десятки раз, и оно уляжется на практике.

Файлы этой главы: `code/02-types.c`, `code/02-input.c`, `code/02-multiplication.c`.

В следующей главе разберём три вещи, без которых не обойтись ни в сортировках, ни в играх: **массивы** (много значений под одним именем), **строки** (текст как набор символов) и **указатели** (адреса в памяти). Там же выяснится, как всё-таки заставить функцию изменить переданную ей переменную — помнишь `add_ten`, который не сработал? После этого мы наконец возьмёмся за настоящие алгоритмы.

Глава 3. Массивы, строки, указатели и память

Один `int` хранит одно число. Но алгоритмы работают с **наборами**: отсортировать список оценок, найти слово в тексте, разложить колоду карт. Для этого нужны три кита, на которых стоит весь Си: **массивы** (много значений подряд), **строки** (текст) и **указатели** (адреса в памяти). Четвёртая тема — **динамическая память**: как попросить у системы место, когда заранее неизвестно, сколько его надо.

Глава большая, и это не случайно. Именно здесь Си показывает своё настоящее лицо: он даёт тебе прямой доступ к памяти и **не проверяет ни одного твоего движения**. Ошибся с индексом — программа не остановится, она молча испортит соседние данные. Поэтому мы будем не только писать правильный код, но и **специально ломать** программы, а потом смотреть, что говорят компилятор, операционная система и специальные инструменты-«ловцы ошибок».

Что мы сделаем

К концу главы у тебя будет четыре рабочие программы:

- `code/03-arrays.c` — массив оценок: индексация, перебор, сумма, среднее, передача массива в функцию;
- `code/03-strings.c` — строки: длина, копирование, сравнение, чтение строки с клавиатуры;
- `code/03-pointers.c` — адреса, разыменование, функция `swap`, которая по-настоящему меняет местами две переменные, и арифметика указателей;
- `code/03-dynamic.c` — `malloc`, `free` и `realloc`.

И, что не менее важно, у тебя будет привычка **проверять память инструментами**, а не надеждой.

Часть 1. Массив — много коробок в ряд

Разбираем задачу руками

Представь пять оценок: 4, 5, 3, 5, 2. Пока их пять, можно завести пять переменных: `m1`, `m2`, `m3`, `m4`, `m5`. Но посчитать их сумму циклом уже нельзя — у цикла нет способа сказать «возьми переменную с номером `i`». А если оценок сто? Двести имён?

Нужна другая идея: положить все пять чисел **в ряд**, дать ряду одно имя и обращаться к элементам **по номеру**. Так на полке стоят пять одинаковых коробок:

```
имя: marks
        +----+----+----+----+----+
        |  4 |  5 |  3 |  5 |  2 |
        +----+----+----+----+----+
индекс:    0    1    2    3    4
```

Обрати внимание на нижнюю строчку. Нумерация начинается **с нуля**. Это не прихоть: индекс в Си означает не «который по счёту», а **«на сколько шагов отступить от начала»**. Первый элемент лежит в самом начале — отступать не надо, значит 0. Третий — через два шага, значит 2. Такое соглашение позже окупится: когда мы дойдём до указателей, станет видно, что `marks[i]` буквально означает «взять адрес начала, прибавить `i` элементов, посмотреть, что там».

Первая программа с массивом

```c
#include <stdio.h>

int main(void)
{
    int marks[5] = {4, 5, 3, 5, 2};
    printf("Первая оценка: %d\n", marks[0]);
    printf("Третья оценка: %d\n", marks[2]);
    return 0;
}
```

Разбор по частям:

- `int marks[5]` — объявляем массив из **5** целых чисел. `int` — тип каждого элемента, `marks` — имя, `5` в квадратных скобках — **размер**, сколько элементов. Размер фиксированный: массив на 5 не станет массивом на 6.

- `= {4, 5, 3, 5, 2}` — сразу заполняем пятью значениями. Фигурные скобки со списком через запятую — это **инициализатор** массива. Работает он только при объявлении: написать `marks = {1, 2}` позже уже нельзя.
- `marks[0]` — обращение к элементу по индексу. Квадратные скобки здесь читаются как «элемент номер».

Собираем и запускаем:

```
cc -std=c11 -Wall -Wextra -O2 arrays.c -o arrays
./arrays
```

```
Первая оценка: 4
Третья оценка: 3
```

«Третья оценка» — это `marks[2]`, и напечаталось `3` — третье число из списка. Если бы мы написали `marks[3]`, получили бы `5`, четвёртый элемент. Привыкай сразу: последний элемент массива из пяти — это `marks[4]`, а не `marks[5]`.

Перебор циклом: сумма и среднее

Массив и цикл `for` (см. главу 2) созданы друг для друга:

```c
int marks[5] = {4, 5, 3, 5, 2};
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum = sum + marks[i];
}
printf("сумма: %d\n", sum);

double avg = sum / 5;
printf("среднее (без приведения): %.2f\n", avg);
printf("среднее (с приведением):  %.2f\n", (double)sum / 5);
```

Переменная `i` пробегает 0, 1, 2, 3, 4 — ровно все индексы. Условие `i < 5`, а не `i <= 5`: пятого индекса не существует, к нему мы вернёмся через минуту.

Запускаем:

```
сумма: 19
среднее (без приведения): 3.00
среднее (с приведением):  3.80
```

Проверь руками: 4 + 5 + 3 + 5 + 2 = 19, делим на 5 — получается 3.8. Значит, вторая строка вывода врёт, и вот почему.

Обе части выражения `sum / 5` — целые числа. Значит, и деление идёт **целочисленное**: 19 / 5 = 3, дробная часть выбрасывается **до** того, как результат попадёт в `double avg`. Тип переменной слева ни на что не влияет: сначала вычисляется правая часть, по своим правилам, потом уже результат кладут в переменную.

`(double)sum` — это **приведение типа** (cast). Запись `(тип)значение` означает «возьми это значение и посчитай его вот таким типом». Как только одна часть деления стала дробной, вторая подтягивается автоматически — и `19.0 / 5` даёт честные 3.8. Ошибка тихая и оттого противная: `3.00` выглядит правдоподобно, и никакого предупреждения при сборке не будет.

Выход за границу массива — покажем на живом примере

Теперь специально ошибёмся. Напишем `i <= 5` вместо `i < 5` — самая частая опечатка новичка. Файл `oob.c` (*out of bounds* — «за границами»):

```c
#include <stdio.h>

int main(void)
{
    int marks[5] = {4, 5, 3, 5, 2};
    int sum = 0;
    for (int i = 0; i <= 5; i++) {
        printf("marks[%d] = %d\n", i, marks[i]);
        sum = sum + marks[i];
    }
    printf("Сумма: %d\n", sum);
    return 0;
}
```

Собираем как обычно — и первое открытие:

```
cc -std=c11 -Wall -Wextra -O2 oob.c -o oob
```

Ни одного предупреждения. Компилятор промолчал. Сначала соберём без оптимизации — флаг `-O0` («не оптимизируй»), тогда программа делает буквально то, что написано в коде:

```
cc -std=c11 -Wall -Wextra -O0 oob.c -o oob0
./oob0
```

```
marks[0] = 4
marks[1] = 5
marks[2] = 3
marks[3] = 5
```

```
marks[4] = 2
marks[5] = 355
Сумма: 374
```

Программа не упала. Она прочитала **чужой кусок памяти** — четыре байта, которые лежали сразу за массивом, — и посчитала их шестой оценкой: 19 + 355 = 374. Число `355` у тебя будет другим: там мог оказаться остаток какой-то служебной переменной, кусок адреса, что угодно.

А теперь соберём с `-O2`, как обычно:

```
marks[0] = 4
marks[1] = 5
marks[2] = 3
marks[3] = 5
marks[4] = 2
marks[5] = 1250317928
Сумма: 73896
```

Посчитай: 19 + 1250317928 никак не даёт 73896. Одна и та же «шестая оценка» в `printf` оказалась одним числом, а в сумме — другим. Как так? Чтение за границей массива — это **неопределённое поведение** (undefined behavior, UB): стандарт Си не говорит, что должно получиться, и компилятор вправе считать, что в правильной программе такого не бывает. Здесь clang с `-O2` так и сделал: он заранее посчитал всё, что мог, а за `marks[5]` в память **вообще не полез** — в `printf` ушли случайные остатки из регистров процессора. Никакого «соседнего байта» там уже нет. Про UB мы ещё не раз поговорим; пока запомни: результат такой программы — не «мусор из соседней ячейки», а что угодно, и с другими флагами он другой.

Вот главное, что нужно вынести из этих запусков: **Си не проверяет границы массивов**. Не «иногда не проверяет» — вообще не проверяет. Проверка стоила бы времени на каждом обращении, а Си выбирает скорость. Следить за индексами — твоя работа.

Иногда компилятор всё-таки помогает. Если индекс — не переменная, а записанное прямо в коде число, clang видит ошибку. Файл `oob2.c` (сообщения компилятора и санитайзеров бывают длиннее книжной строки, поэтому мы их переносим; содержимое при этом не меняется):

```c
#include <stdio.h>

int main(void)
{
    int marks[5] = {4, 5, 3, 5, 2};
    printf("%d\n", marks[5]);
```

```
    return 0;
}
```

```
oob2.c:6:20: warning: array index 5 is past the end of the array
(that has type 'int[5]') [-Warray-bounds]
    6 |     printf("%d\n", marks[5]);
      |                     ^     ~
oob2.c:5:5: note: array 'marks' declared here
    5 |     int marks[5] = {4, 5, 3, 5, 2};
      |     ^
1 warning generated.
```

Формат сообщения `oob2.c:6:20` — это «файл, строка 6, столбец 20».

Но стоит индексу стать переменной цикла — и компилятор бессилен: во время компиляции он не знает, какие значения примет `i`.

Санитайзер — тот, кто ловит такое за руку

Есть инструмент, который всё-таки проверяет каждое обращение к памяти — **санитайзер адресов** (AddressSanitizer, коротко ASan). Компилятор по флагу встраивает в программу проверки: перед каждым чтением и записью смотрит, законен ли адрес. Программа становится медленнее (обычно примерно вдвое) и прожорливее по памяти — зато ошибка находится мгновенно и точно.

Собираем ту же `oob.c` с санитайзером. Флаг `-g` добавляет в программу отладочную информацию, чтобы в отчёте были номера строк, а `-O2` при отладке лучше не включать:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address oob.c -o oob_asan
./oob_asan
```

Программа печатает первые пять строк как раньше, а потом умирает вот таким отчётом. Отчёт длинный, приводим его начало — до итоговой строки `SUMMARY`; дальше идёт ещё карта служебной памяти, которая нам сейчас ни к чему:

```
=================================================================
==69592==ERROR: AddressSanitizer: stack-buffer-overflow on address
0x7ff7be25f814 at pc 0x000101c9f91f bp 0x7ff7be25f7d0 sp
0x7ff7be25f7c8
READ of size 4 at 0x7ff7be25f814 thread T0
    #0 0x000101c9f91e in main oob.c:8
    #1 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)
```

```
Address 0x7ff7be25f814 is located in stack of thread T0 at offset 52
in frame
    #0 0x000101c9f7df in main oob.c:4

  This frame has 1 object(s):
    [32, 52) 'marks' (line 5) <== Memory access at offset 52
overflows this variable
HINT: this may be a false positive if your program uses some custom
stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow oob.c:8 in main
```

Читаем отчёт как текст — он написан по-человечески, просто по-английски:

- `stack-buffer-overflow` — «выход за границу буфера на стеке». **Стек** — область памяти, где живут локальные переменные функции; наш `marks` лежит именно там. `READ of size 4` — читали 4 байта, то есть `int`.
- `#0 ... in main oob.c:8` — **точное место**: файл `oob.c`, строка 8. Это наш `printf` с `marks[i]`.
- `This frame has 1 object(s): [32, 52) 'marks' (line 5)` — в кадре один объект, массив `marks` со строки 5. Запись `[32, 52)` значит «с 32-го байта, не доходя до 52-го»: байты 32–51, всего 20 — пять `int`ов. А обращение было по смещению 52 — первый байт сразу за концом массива.
- `HINT` — оговорка про экзотические случаи, когда санитайзер может ошибиться; к нам не относится. `SUMMARY` — та же суть одной строкой.

Первое, что делаешь с любым отчётом санитайзера: ищешь строки с **именем своего файла**. Всё остальное — адреса, номера потоков, внутренности библиотек — можно пока не разглядывать. Числа в отчёте у тебя будут другими и поменяются при каждом запуске.

`sizeof` — откуда взять длину массива

Писать `5` в трёх местах программы плохо: поменяешь массив — забудешь поменять цикл. Длину можно вычислить. Оператор `sizeof` возвращает **размер в байтах**:

```c
int marks[5] = {4, 5, 3, 5, 2};
printf("sizeof marks    = %zu байт\n", sizeof marks);
printf("sizeof marks[0] = %zu байт\n", sizeof marks[0]);
printf("элементов       = %zu\n", sizeof marks / sizeof marks[0]);
```

```
sizeof marks    = 20 байт
sizeof marks[0] = 4 байт
элементов       = 5
```

Весь массив — 20 байт, один элемент — 4 байта, делим одно на другое, получаем 5. Пояснения:

- `sizeof` — это **оператор**, а не функция, поэтому скобки не обязательны: `sizeof marks` и `sizeof(marks)` одинаковы. Для типа скобки нужны: `sizeof(int)`.
- `%zu` — спецификатор для типа `size_t`, который возвращает `sizeof`. `size_t` — беззнаковый целый тип «размер чего-то в байтах». Печатать его через `%d` формально неверно, и компилятор поворчит.
- Приём `sizeof массив / sizeof массив[0]` работает **только там, где массив объявлен**. Почему — прямо сейчас и увидим.

Массив «распадается» в указатель

Передадим массив в функцию и попробуем посчитать длину внутри неё. Файл `decay.c`:

```c
#include <stdio.h>

void print_all(int a[])
{
    int n = sizeof a / sizeof a[0];
    printf("внутри функции: sizeof a = %zu, n = %d\n", sizeof a, n);
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int main(void)
{
    int marks[5] = {4, 5, 3, 5, 2};
    printf("в main: sizeof marks = %zu\n", sizeof marks);
    printf("в main: элементов    = %zu\n",
           sizeof marks / sizeof marks[0]);
    print_all(marks);
    return 0;
}
```

Компилятор сразу поднимает крик — и крик по делу. Всего он выдал три предупреждения: два одинаковых — на оба `sizeof a` (в строках 5 и 6) — и третье про деление `sizeof a / sizeof a[0]`: «`sizeof a` вернёт размер указателя, а не массива» (`-Wsizeof-pointer-div`). Вот первое:

```
decay.c:5:20: warning: sizeof on array function parameter will return
size of 'int *' instead of 'int[]' [-Wsizeof-array-argument]
    5 |     int n = sizeof a / sizeof a[0];
      |                    ^
```

«`sizeof` от параметра-массива вернёт размер `int *`, а не `int[]`». Запускаем и смотрим, чем это оборачивается:

```
в main: sizeof marks = 20
в main: элементов    = 5
внутри функции: sizeof a = 8, n = 2
4 5
```

Внутри функции `sizeof a` дал **8**, а не 20. И напечаталось два элемента вместо пяти.

Вот что произошло. В Си массив **нельзя** передать в функцию целиком. При передаче имя массива превращается в **адрес его первого элемента** — это называется «распад массива в указатель» (*array decay*). Функция получает не 20 байт данных, а один адрес. Адрес на 64-битной машине занимает 8 байт — отсюда и `sizeof a = 8`. Дальше арифметика честно считает 8 / 4 = 2, и цикл проходит по двум элементам.

Запись `int a[]` в списке параметров — обман зрения: компилятор молча переписывает её в `int *a`. Пустые квадратные скобки там разрешены только ради читаемости, размер в них (даже `int a[5]`) игнорируется.

Отсюда правило, которое ты увидишь в каждой сишной библиотеке: **длину массива передают отдельным аргументом**.

```c
int sum(const int a[], int n)
{
    int s = 0;
    for (int i = 0; i < n; i++) {
        s = s + a[i];
    }
    return s;
}
```

- `int n` — длина, которую вызывающий обязан сообщить. Никакого способа узнать её изнутри у функции нет.

- `const` перед типом — обещание «эта функция не будет менять содержимое массива». Компилятор проследит: попытка написать `a[0] = 7` внутри станет ошибкой. Читателю кода тоже полезно: сразу видно, что массив только читают.
- А раз функция получила адрес, а не копию, то без `const` она **и правда могла бы** менять исходный массив. Этим мы будем пользоваться постоянно в сортировках (глава 8).

Заполняем массив вводом

Пока динамической памяти у нас нет, массив объявляют «с запасом», а используют столько, сколько попросят. Файл `fill.c`:

```
#include <stdio.h>

int main(void)
{
    int a[100];
    int n;

    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > 100) {
        printf("Нужно число от 1 до 100\n");
        return 1;
    }
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        if (scanf("%d", &a[i]) != 1) {
            printf("Не число\n");
            return 1;
        }
    }

    printf("ввёл:");
    for (int i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");
    return 0;
}
```

- `int a[100];` — без инициализатора. У массива, объявленного внутри функции, значения **мусорные**, пока ты сам туда что-нибудь не запишешь. (Массив, объявленный вне всех функций, Си заполняет нулями, но на это мы пока не опираемся.)
- `&a[i]` — адрес конкретного элемента, куда `scanf` запишет число. Знак `&` тут ровно тот же, что и в `scanf("%d", &n)`.
- Проверка `n > 100` — не формальность. Без неё пользователь введёт 1000, и цикл вылезет далеко за массив, со всеми последствиями из предыдущего раздела.
- `scanf` возвращает число успешно прочитанных значений, поэтому `!= 1` ловит и буквы вместо цифр, и конец ввода.

Запуск (числа подаём через конвейер):

```
cc -std=c11 -Wall -Wextra -O2 fill.c -o fill
printf '3\n10\n20\n30\n' | ./fill
```

```
Сколько чисел? a[0] = a[1] = a[2] = ввёл: 10 20 30
```

Приглашения слиплись в одну строку — так бывает, когда ввод идёт из конвейера, а не с клавиатуры: `printf` не переводит строку, а пользовательских нажатий Enter тут нет. Запусти руками, и диалог будет нормальный.

Полная программа `code/03-arrays.c`

```c
#include <stdio.h>

/* Длину массива передаём отдельным аргументом: внутри функции
   имя массива превращается в указатель, и sizeof тут не поможет. */
int sum(const int a[], int n)
{
    int s = 0;
    for (int i = 0; i < n; i++) {
        s = s + a[i];
    }
    return s;
}

int main(void)
{
    int marks[5] = {4, 5, 3, 5, 2};
    int n = (int)(sizeof marks / sizeof marks[0]);
```

```
    printf("marks[0] (первая)    = %d\n", marks[0]);
    printf("marks[2] (третья)    = %d\n", marks[2]);
    printf("marks[%d] (последняя) = %d\n", n - 1, marks[n - 1]);

    printf("весь массив:");
    for (int i = 0; i < n; i++) {
        printf(" %d", marks[i]);
    }
    printf("\n");

    printf("sizeof marks    = %zu байт\n", sizeof marks);
    printf("sizeof marks[0] = %zu байт\n", sizeof marks[0]);
    printf("элементов       = %d\n", n);

    int s = sum(marks, n);
    printf("сумма   = %d\n", s);
    printf("среднее = %.2f\n", (double)s / n);
    return 0;
}
```

`(int)(sizeof ... / sizeof ...)` — приведение результата к `int`, чтобы дальше спокойно печатать его через `%d` и вычитать единицу.

```
cc -std=c11 -Wall -Wextra -O2 code/03-arrays.c -o arrays
./arrays
```

```
marks[0] (первая)    = 4
marks[2] (третья)    = 3
marks[4] (последняя) = 2
весь массив: 4 5 3 5 2
sizeof marks    = 20 байт
sizeof marks[0] = 4 байт
элементов       = 5
сумма   = 19
среднее = 3.80
```

Часть 2. Строки — это массивы символов

В Си **нет** отдельного типа «строка». Строка — это **массив `char`** (символов), в конце которого стоит специальный символ-метка `'\0'` (нулевой символ, код 0). Он говорит: «здесь текст закончился».

```
char name[] = "Anna";
```

- `char name[]` — массив символов. Размер не указали: компилятор посчитает его сам по инициализатору.
- `"Anna"` — компилятор разложит текст по буквам и **сам добавит** `'\0'`. В памяти окажется пять байт:

```
        +-----+-----+-----+-----+------+
        | 'A' | 'n' | 'n' | 'a' | '\0' |
        +-----+-----+-----+-----+------+
индекс:    0     1     2     3     4
```

Печатают строку через `%s` (*string*):

```
printf("Привет, %s!\n", name);
```

`printf` берёт байты от начала и печатает, **пока не встретит** `'\0'`. Вот зачем нужен нулевой символ: без него `printf` не знает, где остановиться.

Что будет без `'\0'` — проверим

Соберём два массива вручную. Один с меткой конца, второй без. Файл `noterm.c`:

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char good[5] = {'A', 'n', 'n', 'a', '\0'};
    char bad[4]  = {'A', 'n', 'n', 'a'};

    printf("good: [%s]\n", good);
    printf("strlen(good) = %zu\n", strlen(good));
    printf("bad:  [%s]\n", bad);
    printf("strlen(bad)  = %zu\n", strlen(bad));
    return 0;
}
```

Одиночные кавычки — это **символьный литерал**: `'A'` — один символ, `"A"` — уже строка из двух байт (`'A'` и `'\0'`). А `strlen` из `<string.h>` считает длину строки: идёт от начала и считает байты, пока не встретит `'\0'` (подробный разбор — чуть ниже). Запускаем:

```
good: [Anna]
strlen(good) = 4
bad:  [AnnaAnna]
strlen(bad)  = 8
```

`bad` содержит четыре буквы, а напечаталось восемь. `printf` дошёл до конца массива `bad`, метки не нашёл и поехал дальше по памяти — а там случайно лежал `good`. Так и вышло `AnnaAnna`. У тебя может получиться другой мусор: что окажется за массивом, зависит от того, как компилятор разложил переменные.

Санитайзер, конечно, такое видит. Собери эту программу с `-fsanitize=address` — получишь знакомый `stack-buffer-overflow` с указанием на строку 11 (тот самый `printf` с `bad`) и с пояснением, что `bad` занимает байты `[64, 68)`, а чтение ушло на 68-й.

Вывод простой: **массив `char` без `'\0'` — это не строка**. Если собираешь текст руками по символам, метку конца ставишь тоже руками.

`sizeof` против `strlen`

Два разных числа, которые новички постоянно путают:

```c
char name[] = "Anna";
printf("strlen(name) = %zu\n", strlen(name));
printf("sizeof name  = %zu\n", sizeof name);
printf("байт после последней буквы: код %d\n", name[strlen(name)]);
```

```
strlen(name) = 4
sizeof name  = 5
байт после последней буквы: код 0
```

- `strlen` считает байты **до** `'\0'`, саму метку не считая — 4.
- `sizeof` даёт размер всего массива в байтах, вместе с меткой — 5.
- `name[strlen(name)]` — тот самый байт с кодом 0. Он реально лежит в массиве и место занимает.

Правило: если размер массива компилятор посчитал сам по строке (`char name[] = "Anna";`), то `sizeof` ровно на единицу больше `strlen`. Если размер указан явно, связи нет: у `char buf[64] = "Anna";` `sizeof` равен 64, а `strlen` — всё те же 4. Для указателя на строку (о них — в части 3) `sizeof` даст размер адреса, 8, и никакой связи с длиной уже не будет.

Функции из `<string.h>`

Подключив `#include <string.h>`, получаешь готовые инструменты. Разберём три главные — с сигнатурами.

`size_t strlen(const char *s)` — длина строки. Принимает адрес начала, возвращает число байт до `'\0'`. Подводный камень: работает за время, пропорциональное длине — она каждый раз считается заново. Писать `for (i = 0; i < strlen(s); i++)` — значит пересчитывать длину на каждом шаге цикла: для строки из 1000 символов это 1000 проходов по 1000 символов. Иногда компилятор сам вынесет вызов из цикла, но рассчитывать на это не стоит — посчитай длину один раз в переменную.

`char *strcpy(char *dst, const char *src)` — копирует строку `src` в массив `dst` вместе с `'\0'`. Возвращает `dst` (этим редко пользуются). Подводный камень **огромный**: `strcpy` не знает, сколько места в `dst`, и копирует ровно столько, сколько есть в `src`. Не влезет — затрёт то, что лежит дальше.

```c
char dst[5];
strcpy(dst, "Konstantin");
printf("dst = %s\n", dst);
```

Здесь система защиты в стандартной библиотеке macOS ловит ошибку ещё при компиляции — длины-то обе известны:

```
cpy.c:7:5: warning: 'strcpy' will always overflow; destination buffer
has size 5, but the source string has length 11 (including NUL byte)
[-Wfortify-source]
    7 |     strcpy(dst, "Konstantin");
      |     ^
```

(Дальше компилятор показывает, во что разворачивается `strcpy` в заголовках macOS — эти строки для нас неинтересны.)

А если всё-таки запустить:

```
bash: line 1: 64142 Illegal instruction: 4  ./cpy
код возврата: 132
```

Программу расстреляли на месте. Заголовки macOS подменили `strcpy` на проверяющую версию, та увидела, что копия не влезет, и нарочно остановила программу сигналом номер 4 (`SIGILL`, «недопустимая инструкция»). Отсюда и код возврата: 132 = 128 + 4. Ниже, в части про указатели, эта арифметика встретится ещё раз.

Но так везёт только с очевидным случаем: если длина строки станет известна во время работы (пришла от пользователя), никакой проверки при компиляции уже не будет, и переполнение произойдёт тихо. Поэтому размер приёмника проверяй сам.

`int strcmp(const char *a, const char *b)` — сравнивает две строки посимвольно. Возвращает:

- `0`, если строки **равны** (да, именно ноль при равенстве — это сбивает с толку);
- отрицательное число, если `a` меньше `b` (идёт раньше по алфавиту);
- положительное, если `a` больше.

Стандарт обещает только **знак** результата, не конкретное число. Даже на одной машине число бывает разным: `strcmp("a", "z")` с двумя строками прямо в коде clang вычисляет сам ещё при сборке и подставляет `-1`, а если те же строки пришли во время работы (скажем, из командной строки), библиотека macOS вернёт `-25` — разницу кодов `'a'` (97) и `'z'` (122). Поэтому пишут `strcmp(a, b) < 0`, а не `strcmp(a, b) == -1`.

Почему == не сравнивает строки

Самая живучая ошибка новичка. Заведём два разных массива с одинаковым текстом (файл `eq.c`):

```
char a[] = "exit";
char b[] = "exit";

printf("адрес a = %p\n", (void *)a);
printf("адрес b = %p\n", (void *)b);
printf("a == b даёт %d\n", a == b);
printf("strcmp(a, b) даёт %d\n", strcmp(a, b));
printf("strcmp(\"abc\", \"abd\") = %d\n", strcmp("abc", "abd"));
printf("strcmp(\"abd\", \"abc\") = %d\n", strcmp("abd", "abc"));
```

Компилятор сразу подсказывает, что дело нечисто:

```
eq.c:11:38: warning: array comparison always evaluates to false
[-Wtautological-compare]
   11 |     printf("a == b даёт %d\n", a == b);
      |                                      ^
1 warning generated.
```

Запуск:

```
адрес a = 0x7ff7b3211900
адрес b = 0x7ff7b3211908
```

```
a == b даёт 0
strcmp(a, b) даёт 0
strcmp("abc", "abd") = -1
strcmp("abd", "abc") = 1
```

Тексты одинаковые, а `a == b` дало 0, то есть «не равны». Смотри на первые две строки: адреса разные, 0x...900 и 0x...908. Имя массива в выражении — это адрес его начала, значит `a == b` сравнило **два адреса**, а не два текста. Два разных массива лежат в разных местах — результат всегда «не равны», независимо от содержимого.

А `strcmp(a, b)` дал 0 — «различий нет», то самое равенство. Две последние строки показывают знаки: `"abc"` идёт раньше `"abd"`, потому что на третьем символе `c` меньше `d`, — отсюда минус; поменяли аргументы местами — знак перевернулся.

Правильно так:

```c
if (strcmp(word, "exit") == 0) {
    printf("Пока!\n");
}
```

Читается как «если различий нет». Многие для наглядности заводят себе привычку писать `if (strcmp(a, b) == 0)` полностью, а не `if (!strcmp(a, b))` — второе короче, но новичка путает.

Чтение строки от пользователя

Для одного слова без пробелов годится `%s` в `scanf`:

```c
char word[64];
scanf("%63s", word);
```

- Перед `word` **нет** `&`! Имя массива само по себе уже адрес его начала. Это то самое исключение из правила «в `scanf` всегда `&`».
- `%63s` с числом — ограничиваем ввод 63 символами, чтобы влезть в массив из 64 (последний байт под `'\0'`). Без ограничения длинный ввод затрёт соседнюю память — классическая уязвимость.

Для **целой строки с пробелами** `scanf("%s")` не годится: он читает до первого пробела. Берём `fgets`.

`char *fgets(char *s, int size, FILE *stream)` — читает строку. Принимает: куда класть, сколько максимум байт (включая `'\0'`), откуда читать. Возвращает `s` при успехе и `NULL`, если читать больше нечего (конец файла) или случилась ошибка. Проверять этот `NULL` нужно — иначе при пустом вводе будешь работать с мусором.

```
char line[64];
printf("Введи команду: ");
if (fgets(line, sizeof line, stdin) == NULL) {
    printf("\nВвод закончился\n");
    return 1;
}
printf("как есть:   [%s]\n", line);
printf("длина: %zu\n", strlen(line));
```

- `sizeof line` — размер массива в байтах (64). Так `fgets` не вылезет за границу, и при изменении размера массива менять код не придётся. Если строка длиннее 63 символов, `fgets` прочитает первые 63, поставит `'\0'` и вернётся; остаток строки (вместе с её `'\n'`) останется ждать следующего чтения.
- `stdin` — *standard input*, стандартный ввод: клавиатура, а если ввод перенаправлен (`printf ... | ./prog`), то конвейер или файл.

Подаём слово `exit` и смотрим:

```
Введи команду: как есть:   [exit
]
длина: 5
```

Квадратная скобка уехала на следующую строку, а длина — 5, хотя букв четыре. `fgets` **оставляет в строке символ перевода строки** `'\n'` — тот самый, что появился от нажатия Enter. Это фирменная ловушка: `strcmp(line, "exit")` вернёт «не равно», потому что в `line` лежит `"exit\n"`.

Чиним — отрезаем последний символ, если это перевод строки:

```
size_t len = strlen(line);
if (len > 0 && line[len - 1] == '\n') {
    line[len - 1] = '\0';
}
```

Записав `'\0'` на место `'\n'`, мы сдвинули метку конца строки на один символ влево. Байты дальше никуда не делись, но для всех строковых функций строка теперь короче. Проверка `len > 0` защищает от `line[-1]` на пустой строке.

```
без '\n':   [exit]
длина: 4
Это команда выхода. Пока!
```

Подробнее строковые алгоритмы разберём в главе 10, там эта обрезка `'\n'` встретится снова.

Почему `gets` запрещён

Была в Си функция `gets(s)` — читает строку, и всё. Без размера буфера. Совсем. Сколько пользователь введёт, столько она и запишет, хоть мегабайт в массив на 10 байт. Через эту дыру взламывали программы десятилетиями, и в стандарте C11 функцию **удалили из языка**.

Посмотрим, что скажет компилятор сегодня:

```
char buf[10];
gets(buf);
```

```
gets_test.c:6:5: warning: 'gets' is deprecated: This function is
provided for compatibility reasons only.  Due to security concerns
inherent in the design of gets(3), it is highly recommended that you
use fgets(3) instead. [-Wdeprecated-declarations]
    6 |     gets(buf);
      |     ^
```

(Дальше идут строки `note:` с указанием на место в системном заголовке, где `gets` помечена устаревшей.)

Заголовки macOS всё ещё объявляют `gets` ради совместимости со старым кодом, но помечают её как устаревшую и прямым текстом советуют `fgets`. Программа при этом **соберётся** — предупреждение не ошибка, а при запуске macOS ещё и напечатает в терминал `warning: this program uses gets(), which is unsafe.` Именно поэтому предупреждение нельзя пропускать мимо глаз: на других системах `gets` может не найтись вовсе, и там сборка просто упадёт. Вывод один: **только `fgets`**.

Полная программа `code/03-strings.c`

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char name[] = "Anna";

    printf("Привет, %s!\n", name);
    printf("strlen(name) = %zu\n", strlen(name));
    printf("sizeof name  = %zu\n", sizeof name);
    printf("байт после последней буквы: код %d\n", name[strlen(name)]);
```

```
    char copy[16];
    strcpy(copy, name);
    copy[0] = 'J';
    printf("копия после правки: %s, оригинал: %s\n", copy, name);

    /* Два разных массива с одинаковым текстом. */
    char a[] = "exit";
    char b[] = "exit";
    const char *pa = a;             /* через указатели — чтобы  */
    const char *pb = b;             /* компилятор не ругался    */
    printf("адреса равны? %d\n", pa == pb);
    printf("strcmp(a, b)  = %d\n", strcmp(a, b));
    printf("strcmp(\"abc\", \"abd\") = %d\n", strcmp("abc", "abd"));

    char line[64];
    printf("Введи команду: ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        printf("\nВвод закончился\n");
        return 1;
    }
    printf("как есть:   [%s]\n", line);
    printf("длина: %zu\n", strlen(line));

    size_t len = strlen(line);
    if (len > 0 && line[len - 1] == '\n') {
        line[len - 1] = '\0';
    }
    printf("без '\\n':   [%s]\n", line);
    printf("длина: %zu\n", strlen(line));

    if (strcmp(line, "exit") == 0) {
        printf("Это команда выхода. Пока!\n");
    } else {
        printf("Такой команды не знаю\n");
    }
    return 0;
}
```

Пара мелочей из листинга:

- `copy[0] = 'J';` — правим копию и печатаем обе строки. `strcpy` делает именно копию, а не второе имя для тех же данных, поэтому оригинал не пострадает.
- `const char *pa = a;` — переменная-указатель на символы, в неё положили адрес начала массива `a`. Сравнение `pa == pb` — то же самое сравнение адресов, что и `a == b`, но записанное так, чтобы компилятор не ругался предупреждением на очевидную бессмыслицу. Про указатели — прямо в следующей части.
- `"strcmp(\"abc\", \"abd\")"` — обратный слеш перед кавычкой говорит: это кавычка внутри текста, а не конец строки.
- `"без '\\n'"` — два обратных слеша дают один настоящий в выводе.

```
cc -std=c11 -Wall -Wextra -O2 code/03-strings.c -o strings
printf 'exit\n' | ./strings
```

```
Привет, Anna!
strlen(name) = 4
sizeof name  = 5
байт после последней буквы: код 0
копия после правки: Jnna, оригинал: Anna
адреса равны? 0
strcmp(a, b)  = 0
strcmp("abc", "abd") = -1
Введи команду: как есть:   [exit
]
длина: 5
без '\n':   [exit]
длина: 4
Это команда выхода. Пока!
```

Часть 3. Указатели — переменные, хранящие адрес

Мы уже встречали `&` в `scanf` и слово «адрес» — раз двадцать. Пришло время разобраться до конца.

У каждой переменной есть **адрес** — номер ячейки в памяти, где она лежит. Память компьютера — это длинная лента пронумерованных байт; адрес переменной и есть её номер на этой ленте. **Указатель** — это переменная, которая хранит такой номер.

Адрес и разыменование

```c
int x = 42;
int *p = &x;      /* p хранит адрес x */

printf("значение x:        %d\n", x);
printf("адрес x:           %p\n", (void *)&x);
printf("p хранит адрес:    %p\n", (void *)p);
printf("значение по p:     %d\n", *p);

*p = 100;         /* меняем x через указатель */
printf("теперь x = %d\n", x);
```

Разбор — ключевой момент всей главы:

- `int *p` — объявляем **указатель на int**. Звёздочка в объявлении говорит: «`p` — не число, а адрес, по которому лежит число». Тип переменной `p` целиком — «указатель на `int`».
- `= &x` — кладём в `p` **адрес** переменной `x`. Оператор `&` — «взять адрес».
- `*p` в выражении — оператор **разыменования** (*dereference*): «значение, лежащее **по** адресу `p`». Как «открыть коробку, на которую указывает стрелка».
- `*p = 100;` — записать 100 туда, куда указывает `p`. Раз `p` указывает на `x`, изменится именно `x`.
- `%p` — спецификатор для печати адреса; `(void *)` — приведение, которого требует `printf` для адресов. `void *` — «указатель на что-то, тип неизвестен».

Часть вывода программы `code/03-pointers.c`:

```
значение x:        42
адрес x:           0x7ff7bca36bdc
p хранит адрес:    0x7ff7bca36bdc
значение по p:     42
теперь x = 100
```

Смотри на две средние строки: адрес `x` и содержимое `p` — **одно и то же число**. Это и значит «указатель хранит адрес». А `*p` вернул 42 — то, что по этому адресу лежит.

Само число (`0x7ff7bca36bdc`) у тебя будет другим, и даже при повторных запусках оно меняется: система каждый раз кладёт программу в разное место памяти. Запоминать его бессмысленно — важно лишь то, что оба раза оно совпало. Начальное `0x` означает шестнадцатеричную запись, в ней адреса печатать удобнее.

Звёздочка в объявлении и звёздочка в выражении

Один знак, две разные роли — и это главный источник путаницы:

```
int *p = &x;   /* объявление: p — указатель       */
*p = 100;      /* выражение:  положить 100 по p   */
int y = *p;    /* выражение:  взять значение по p */
```

В объявлении `*` — часть типа, «указатель на». В выражении `*` — действие «пойти по адресу». Шпаргалка: `&x` — «**адрес** x», `int *p` — «p это **указатель**», `*p` — «**значение по** p».

Пишут звёздочку по-разному: `int *p`, `int* p`, `int * p` — для компилятора одно и то же. В книге держимся `int *p`: такая запись честнее при объявлении нескольких переменных, ведь в строке `int *a, b;` указатель только `a`, а `b` — обычный `int`.

Зачем это нужно: функция, меняющая аргумент

В Си аргументы передаются **по значению** — функция получает копию. Поэтому такая функция бесполезна:

```
void bad_swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
    printf("  внутри bad_swap: a=%d b=%d\n", a, b);
}
```

Чтобы функция поменяла **сами** переменные, передаём их **адреса**:

```
void swap(int *a, int *b)
{
    int t = *a;             /* запомнили значение по a  */
    *a = *b;                /* в a положили значение по b */
    *b = t;                 /* в b положили запомненное */
}
```

Позовём обе подряд:

```
int a = 1, b = 2;
bad_swap(a, b);
printf("после bad_swap: a=%d b=%d\n", a, b);
swap(&a, &b);
printf("после swap: a=%d b=%d\n", a, b);
```

```
  внутри bad_swap: a=2 b=1
после bad_swap: a=1 b=2
после swap: a=2 b=1
```

Три строки, и в них вся суть. Внутри `bad_swap` обмен **произошёл**: `a=2 b=1`. Но менялись копии, они умерли на выходе из функции, и снаружи осталось `a=1 b=2`. `swap` получила адреса, полезла по ним в оригиналы — и вот они действительно поменялись.

Это и есть причина, по которой `scanf` требует `&`: ему нужен адрес, чтобы записать прочитанное в **твою** переменную, а не в копию, которая тут же исчезнет.

Приём «вернуть значение через указатель-аргумент» пригодится всюду, где результатов больше одного: `return` умеет отдать только одно значение, а указателей можно передать сколько угодно. `swap` мы ещё раз напишем в главе 19 — это классический вопрос на собеседовании.

Арифметика указателей: `a[i]` — это `*(a + i)`

В начале главы мы обещали: `marks[i]` означает «взять адрес начала, прибавить `i` элементов, посмотреть, что там». Пора это увидеть. К указателю можно прибавить целое число, и получится другой указатель:

```c
int arr[4] = {10, 20, 30, 40};
int *first = arr;              /* то же, что &arr[0] */
int *third = first + 2;        /* на два int дальше */

printf("first     = %p\n", (void *)first);
printf("first + 1 = %p\n", (void *)(first + 1));
printf("*(first + 2) = %d, arr[2] = %d, *third = %d\n",
       *(first + 2), arr[2], *third);
printf("third - first = %td\n", third - first);
printf("байт между ними: %td\n", (char *)third - (char *)first);
```

```
first     = 0x7ff7bca36be0
first + 1 = 0x7ff7bca36be4
*(first + 2) = 30, arr[2] = 30, *third = 30
third - first = 2
байт между ними: 8
```

Разбираем:

- `int *first = arr;` — имя массива в выражении превращается в адрес первого элемента (тот самый распад), поэтому `&` не нужен.

- `first + 1` — адрес вырос не на 1, а на **4** (`...be0` → `...be4`, в шестнадцатеричной записи). Прибавить к указателю единицу значит шагнуть на **один элемент**, а элемент `int` занимает 4 байта. Для `char *` шаг был бы 1 байт, для `double *` — 8. Поэтому тип указателя важен: он задаёт длину шага.
- `*(first + 2)` — «шагни на два элемента и открой коробку». Это ровно `arr[2]`. Квадратные скобки в Си и есть сокращение: запись `a[i]` компилятор понимает как `*(a + i)`. Отсюда и нумерация с нуля: `a[0]` — это `*(a + 0)`, сам первый элемент, без шагов.
- `third - first` — указатели внутри одного массива можно вычитать. Ответ — число **элементов** между ними (2), а не байт. Тип у этой разности особый, `ptrdiff_t` (знаковое целое), и печатают его через `%td`.
- `(char *)third - (char *)first` — тот же трюк, но мы сначала посмотрели на оба адреса как на указатели на `char`. Шаг `char` — один байт, поэтому разность уже в байтах: 8 = 2 элемента по 4 байта.

Граница у арифметики та же, что у индексов. Указатель можно передвинуть в пределах массива и ещё на одну позицию **сразу за** последним элементом: `first + 4` — законный адрес, с ним можно сравнивать, но разыменовывать его нельзя. А `first + 5` — уже неопределённое поведение, даже если по этому адресу ничего не читать.

`NULL` — указатель в никуда

Иногда указателю нечего хранить: список пуст, поиск не нашёл элемента, память не выделилась. Для этого есть `NULL` — специальное значение «никуда не указываю». Оно определено в нескольких стандартных заголовках, в том числе в `<stdio.h>` и `<stdlib.h>`.

```
int *q = NULL;
printf("q = %p\n", (void *)q);
if (q == NULL) {
    printf("q никуда не указывает, *q писать нельзя\n");
}
```

```
q = 0x0
q никуда не указывает, *q писать нельзя
```

Адрес нулевой. Если запустишь это на Linux, вместо `0x0` увидишь `(nil)`: как печатать нулевой указатель через `%p`, стандарт не оговаривает, и библиотека GNU выбрала слово, а не число. Значение от этого не меняется — указатель пустой в обоих случаях.

А что будет, если всё-таки разыменовать? Файл `nullderef.c`:

```
#include <stdio.h>
```

```c
int main(void)
{
    int *p = NULL;
    printf("p = %p\n", (void *)p);
    printf("значение по p: %d\n", *p);
    return 0;
}
```

Собираем без оптимизации (`-O0`) и запускаем:

```
cc -std=c11 -Wall -Wextra -O0 nullderef.c -o nullderef0
./nullderef0
```

```
p = 0x0
bash: line 1: 63713 Segmentation fault: 11  ./nullderef0
код возврата: 139
```

Первая строка успела напечататься, а на второй программа умерла. **Segmentation fault** — «нарушение сегментации»: операционная система заметила обращение по запрещённому адресу и убила программу сигналом номер 11 (`SIGSEGV`). Код возврата 139 = 128 + 11: так оболочки bash и zsh сообщают, каким сигналом программу прибили (мы уже видели 132 = 128 + 4 у `strcpy`). Твоя оболочка может написать то же другими словами (`zsh: segmentation fault`), суть та же.

Нулевой адрес специально сделан запрещённым — чтобы такая ошибка падала сразу и громко, а не портила данные.

Сюрприз в том, что «громко» бывает не всегда. Собери ту же программу с `-O2`:

```
p = 0x0
значение по p: 73896
код возврата: 0
```

Не упала! Разыменование `NULL` — неопределённое поведение, как и выход за границу массива. Компилятор с `-O2` рассудил: «в правильной программе по `NULL` не читают» — и выбросил чтение `*p` целиком. В `printf` ушло то, что случайно лежало в регистре, отсюда `73896`. Не жди, что такая ошибка обязательно проявится падением.

Ловит это второй санитайзер, `-fsanitize=undefined` (он следит не за памятью, а за неопределённым поведением):

```
cc -std=c11 -Wall -Wextra -g -fsanitize=undefined nullderef.c \
   -o nullderef_ub
./nullderef_ub
```

```
nullderef.c:7:45: runtime error: load of null pointer of type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior
nullderef.c:7:45
```

Строка 7, столбец 45 — ровно `*p` внутри второго `printf`. Следом санитайзер печатает ещё отчёт о падении (`SEGV on unknown address 0x000000000000`): предупредив, он даёт программе выполнить запрещённое чтение, и та всё равно падает.

Оба санитайзера обычно включают вместе: `-fsanitize=address,undefined`.

Полная программа `code/03-pointers.c`

```c
#include <stdio.h>

/* Принимает КОПИИ — снаружи ничего не изменится. */
void bad_swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
    printf("  внутри bad_swap: a=%d b=%d\n", a, b);
}

/* Принимает АДРЕСА — меняет сами переменные. */
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

int main(void)
{
    int x = 42;
    int *p = &x;

    printf("значение x:        %d\n", x);
    printf("адрес x:           %p\n", (void *)&x);
    printf("p хранит адрес:    %p\n", (void *)p);
```

```
    printf("значение по p:     %d\n", *p);

    *p = 100;
    printf("теперь x = %d\n", x);

    int a = 1, b = 2;
    bad_swap(a, b);
    printf("после bad_swap: a=%d b=%d\n", a, b);
    swap(&a, &b);
    printf("после swap: a=%d b=%d\n", a, b);

    /* Арифметика указателей: шаг — один элемент, а не один байт. */
    int arr[4] = {10, 20, 30, 40};
    int *first = arr;              /* то же, что &arr[0] */
    int *third = first + 2;        /* на два int дальше */

    printf("first     = %p\n", (void *)first);
    printf("first + 1 = %p\n", (void *)(first + 1));
    printf("*(first + 2) = %d, arr[2] = %d, *third = %d\n",
           *(first + 2), arr[2], *third);
    printf("third - first = %td\n", third - first);
    printf("байт между ними: %td\n", (char *)third - (char *)first);

    int *q = NULL;
    printf("q = %p\n", (void *)q);
    if (q == NULL) {
        printf("q никуда не указывает, *q писать нельзя\n");
    }
    return 0;
}
```

```
cc -std=c11 -Wall -Wextra -O2 code/03-pointers.c -o pointers
./pointers
```

```
значение x:        42
адрес x:           0x7ff7bca36bdc
p хранит адрес:    0x7ff7bca36bdc
значение по p:     42
теперь x = 100
```

```
  внутри bad_swap: a=2 b=1
после bad_swap: a=1 b=2
после swap: a=2 b=1
first     = 0x7ff7bca36be0
first + 1 = 0x7ff7bca36be4
*(first + 2) = 30, arr[2] = 30, *third = 30
third - first = 2
байт между ними: 8
q = 0x0
q никуда не указывает, *q писать нельзя
```

Часть 4. Динамическая память

`int a[100]` требует знать размер при написании кода. Мало — не влезет, много — память тратится зря. А если число элементов выяснится только во время работы? Тогда память просят у системы **во время выполнения** — это **динамическая память**, она же **куча** (heap).

Нужен `#include <stdlib.h>` и две функции.

`void *malloc(size_t size)` (*memory allocate* — «выделить память») — просит у системы кусок в `size` **байт**. Возвращает адрес начала этого куска или `NULL`, если памяти не хватило. Тип `void *` — «указатель непонятно на что»; в Си он сам приводится к нужному типу, поэтому писать `(int *)malloc(...)` не требуется.

`void free(void *ptr)` — возвращает системе кусок, полученный от `malloc`. Ничего не возвращает. Сама переменная-указатель после `free` не меняется — в ней остаётся старый адрес, но пользоваться им уже нельзя.

```c
int n = 4;
int *a = malloc(n * sizeof(int));
if (a == NULL) {
    printf("Не хватило памяти\n");
    return 1;
}
for (int i = 0; i < n; i++) {
    a[i] = i * i;
}
free(a);
```

- `n * sizeof(int)` — размер в **байтах**, а не в элементах. Четыре числа по 4 байта — 16 байт. Забыть про `sizeof` и написать `malloc(n)` — классическая ошибка: получишь вчетверо меньше места.
- `if (a == NULL)` — проверка. На настольном компьютере `malloc` почти всегда успешен, но привычка проверять стоит одной строки, а спасает от падения без объяснений.
- `a[i]` — к динамической памяти обращаются **точно как к массиву**. Указатель и массив здесь неотличимы в использовании: `a[i]` для компилятора означает «элемент через `i` шагов от адреса `a`».
- `free(a)` — **обязательно**, когда память больше не нужна. Правило: на каждый `malloc` однажды должен прийтись ровно один `free`.

Полная программа `code/03-dynamic.c`

Заодно посмотрим на `realloc` — функцию «мне мало, дай больше».

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    /* Верхняя граница нужна не для красоты: ниже считаются i * i
       для i до 2n - 1, а 46341 * 46341 в int уже не влезает.
       Переполнение int — неопределённое поведение. */
    if (scanf("%d", &n) != 1 || n <= 0 || n > 1000) {
        printf("Нужно число от 1 до 1000\n");
        return 1;
    }

    int *a = malloc(n * sizeof(int));
    if (a == NULL) {
        printf("Не хватило памяти\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        a[i] = i * i;
    }
```

```
    printf("квадраты (%d шт.):", n);
    for (int i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");

    /* Мало места? Просим ещё — realloc сохраняет то, что уже лежит. */
    int bigger_n = n * 2;
    int *bigger = realloc(a, bigger_n * sizeof(int));
    if (bigger == NULL) {
        printf("Не хватило памяти на расширение\n");
        free(a);
        return 1;
    }
    a = bigger;

    for (int i = n; i < bigger_n; i++) {
        a[i] = i * i;
    }
    printf("после realloc (%d шт.):", bigger_n);
    for (int i = 0; i < bigger_n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");

    free(a);
    return 0;
}
```

`void *realloc(void *ptr, size_t new_size)` — меняет размер куска, ранее полученного от `malloc`. Возвращает адрес куска нового размера (возможно, **другой** — если на старом месте не хватило, система выделит новое место и **сама скопирует** туда старые данные) или `NULL` при неудаче.

Обрати внимание, что результат кладётся в **новую** переменную `bigger`, а не сразу в `a`. Если написать `a = realloc(a, ...)` и получить `NULL`, старый адрес будет потерян навсегда — и память не освободить, и данные не достать. Через отдельную переменную мы сохраняем шанс аккуратно всё прибрать. Этот приём — основа растущего массива, который мы построим в главе 11.

```
cc -std=c11 -Wall -Wextra -O2 code/03-dynamic.c -o dynamic
printf '4\n' | ./dynamic
```

```
Сколько чисел? квадраты (4 шт.): 0 1 4 9
после realloc (8 шт.): 0 1 4 9 16 25 36 49
```

Ввели 4 — программа попросила место ровно под четыре числа и заполнила их квадратами индексов: 0^2, 1^2, 2^2, 3^2. Потом попросила вдвое больше, и первые четыре числа остались на месте — их сохранил `realloc`. Введи 10 — получишь десять и двадцать чисел, и никакого `int a[100]` «на всякий случай» для этого не понадобилось.

Откуда потолок 1000? Самый большой индекс после расширения — `2n - 1`, и для него считается `i * i`. Квадрат растёт быстро: $46340^2 = 2\,147\,395\,600$ ещё помещается в `int` (потолок 2 147 483 647), а $46341^2 = 2\,147\,488\,281$ уже нет. Значит, при `n` больше 23170 программа дошла бы до переполнения — это неопределённое поведение, и санитайзер `-fsanitize=undefined` на входе 30000 так и пишет: `signed integer overflow: 46341 * 46341 cannot be represented in type 'int'`. С потолком 1000 самый большой квадрат — $1999^2 = 3\,996\,001$, запас огромный, да и две тысячи чисел на экране читать уже тяжело.

Три способа испортить кучу

Ошибки с динамической памятью коварны тем, что программа выглядит работающей и при этом сломана. Разберём три классические, и каждую покажем санитайзеру.

Выход за границу выделенного куска. Тот же `i <= n`, что и с обычным массивом. Файл `hoob.c` (*heap out of bounds*):

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n = 4;
    int *a = malloc(n * sizeof(int));
    for (int i = 0; i <= n; i++) {
        a[i] = i * i;
    }
    printf("a[3] = %d\n", a[3]);
    free(a);
    return 0;
}
```

Обычная сборка с `-O2` молча печатает `a[3] = 9` и выходит с кодом 0 — запись за границу никто не заметил. Санитайзер замечает:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address hoob.c -o hoob
./hoob
```

```
=================================================================
==69615==ERROR: AddressSanitizer: heap-buffer-overflow on address
0x6020000000e0 at pc 0x0001063425db bp 0x7ff7b9bbc8d0 sp
0x7ff7b9bbc8c8
WRITE of size 4 at 0x6020000000e0 thread T0
    #0 0x0001063425da in main hoob.c:9
    #1 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

0x6020000000e0 is located 0 bytes after 16-byte region
[0x6020000000d0,0x6020000000e0)
allocated by thread T0 here:
    #0 0x00010689f792 in malloc+0x82
(libclang_rt.asan_osx_dynamic.dylib:x86_64h+0xa6792)
    #1 0x000106342572 in main hoob.c:7
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

SUMMARY: AddressSanitizer: heap-buffer-overflow hoob.c:9 in main
```

Теперь `heap-`, а не `stack-`: испортили кучу. «0 байт после 16-байтного участка» — ровно на один `int` дальше, чем можно. И показано, где эта память была выделена: строка 7.

Обращение к освобождённой памяти (*use after free*). Указатель после `free` продолжает хранить старый адрес, и соблазн им воспользоваться велик. Файл `uaf.c`:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *a = malloc(4 * sizeof(int));
    a[0] = 7;
    free(a);
    printf("a[0] = %d\n", a[0]);
    return 0;
}
```

Без санитайзера программа не падает, а печатает что попало: у меня вышло `a[0] = -1135598768` вместо семёрки — библиотека уже записала в освобождённый кусок свои служебные данные. С `-fsanitize=address` первые строки отчёта такие:

```
=================================================================
==69619==ERROR: AddressSanitizer: heap-use-after-free on address
0x6020000000d0 at pc 0x0001067c187e bp 0x7ff7b973d8d0 sp
0x7ff7b973d8c8
READ of size 4 at 0x6020000000d0 thread T0
    #0 0x0001067c187d in main uaf.c:9
```

Дальше санитайзер показывает всю историю участка: где выделили (строка 6), где освободили (строка 8), где попытались читать (строка 9). Без него это может «работать» месяцами, пока освобождённый кусок не займёт кто-то другой. Простая профилактика: сразу после `free` писать `a = NULL;`. Тогда ошибочное чтение пойдёт по нулевому адресу — а это, как мы видели, обычно честное падение (хотя с `-O2` и не гарантированное), и санитайзер назовёт его прямо. А повторный `free(NULL)` по стандарту безопасен: он просто ничего не делает.

Двойное освобождение (*double free*) — `free` от одного и того же адреса дважды. Файл `dfree.c`:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *a = malloc(4 * sizeof(int));
    a[0] = 7;
    free(a);
    free(a);
    printf("ok\n");
    return 0;
}
```

```
=================================================================
==69623==ERROR: AddressSanitizer: attempting double-free on
0x6020000000d0 in thread T0:
    #0 0x00010e08789b in free+0x8b
(libclang_rt.asan_osx_dynamic.dylib:x86_64h+0xa689b)
    #1 0x00010db29848 in main dfree.c:9
```

Без санитайзера результат зависит от флагов. С `-O0` ошибку ловит сама библиотека macOS:

```
dfree0(45239,0x7ff84a85dc40) malloc: *** error for object
0x600001c68050: pointer being freed was not allocated
dfree0(45239,0x7ff84a85dc40) malloc: *** set a breakpoint in
malloc_error_break to debug
bash: line 1: 45239 Abort trap: 6           ./dfree0
```

А та же программа с `-O2` напечатала «ok» и вышла с кодом 0: компилятор увидел, что выделенная память толком не используется, и выбросил всю затею с `malloc` и `free` целиком. Ещё одно напоминание, что «не упало» и «правильно» — разные вещи.

Утечка памяти

Забыть `free` — значит **утечь**: кусок останется занятым до конца работы программы. Одна забытая строчка в цикле, который крутится часами, и программа съедает всю память машины.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *a = malloc(4 * sizeof(int));
    a[0] = 1;
    printf("a[0] = %d\n", a[0]);
    return 0;              /* free забыли */
}
```

Программа отработает как ни в чём не бывало: `a[0] = 1`, код возврата 0. Санитайзер адресов утечку тоже не покажет — ищет их отдельная часть, LeakSanitizer, а **на macOS она не поддерживается**:

```
ASAN_OPTIONS=detect_leaks=1 ./leak
```

```
==62643==AddressSanitizer: detect_leaks is not supported on this
platform.
```

На Linux эта же команда напечатала бы отчёт об утечке. На macOS для того же есть системная утилита `leaks`: `MallocStackLogging=1` просит систему запоминать, кто и откуда выделял память, а `--atExit` говорит «проверь в момент выхода».

```
cc -std=c11 -Wall -Wextra -g leak_demo.c -o leak_demo
MallocStackLogging=1 leaks --atExit -- ./leak_demo
```

Начинается отчёт с полутора десятков строк справки о процессе (путь, версия macOS, время запуска), а вот его содержательная часть:

```
leaks Report Version: 4.0, multi-line stacks
Process 70125: 188 nodes malloced for 28 KB
Process 70125: 1 leak for 32 total leaked bytes.

STACK OF 1 INSTANCE OF 'ROOT LEAK: <malloc in main>':
2   dyld                                0x7ff804aa4b28 start + 3240
1   leak_demo                             0x108e4d489 main + 25
leak_demo.c:6
0   libsystem_malloc.dylib              0x7ff804ca8219
_malloc_zone_malloc_instrumented_or_legacy + 115
```

«1 утечка, 32 байта, `malloc` в `main`, файл `leak_demo.c`, строка 6». Стек здесь напечатан снизу вверх: самая нижняя строка — сам `malloc` в системной библиотеке, над ней — наша строка 6, над ней — запуск программы. Просили мы 16 байт, а числится 32. Это не ошибка в нашем счёте: под `leaks` программа работает с отладочным распределителем памяти, и тот выдаёт куски с запасом (у меня `malloc(1)` числился как 16 байт, `malloc(16)` — как 32, `malloc(100)` — как 112). Смотри не на число байт, а на сам факт утечки и на строку, где память выделили.

Хорошая новость: пока `malloc` и `free` стоят парой в одной функции, утечь трудно. Плохая — как только память начинает жить дольше одной функции, за парами приходится следить самому. Этим мы вплотную займёмся в главе 11, когда будем строить списки.

Замечание про русские буквы

Современные системы хранят текст в кодировке **UTF-8**, где русская буква занимает **2 байта**, а не 1. Возьми те же четыре буквы, но русские:

```c
char ru[] = "Анна";
char en[] = "Anna";
printf("ru: strlen = %zu, sizeof = %zu\n", strlen(ru), sizeof ru);
printf("en: strlen = %zu, sizeof = %zu\n", strlen(en), sizeof en);
printf("strlen(\"Привет\") = %zu\n", strlen("Привет"));
```

```
ru: strlen = 8, sizeof = 9
en: strlen = 4, sizeof = 5
strlen("Привет") = 12
```

Четыре русские буквы дали 8 байт, четыре латинские — 4. `strlen` считает **байты**, а не буквы. Поэтому в этой книге везде, где важно считать символы (перевернуть строку, проверить палиндром, посчитать гласные), мы работаем с латиницей и цифрами — по 1 байту на символ. Для вывода сообщений русский текст годится полностью: `printf` печатает байты как есть, и всё выглядит правильно.

Проверяем

Собери и прогони все четыре программы главы:

```
cc -std=c11 -Wall -Wextra -O2 code/03-arrays.c -o arrays
./arrays
cc -std=c11 -Wall -Wextra -O2 code/03-pointers.c -o pointers
./pointers
cc -std=c11 -Wall -Wextra -O2 code/03-strings.c -o strings
printf 'exit\n' | ./strings
cc -std=c11 -Wall -Wextra -O2 code/03-dynamic.c -o dynamic
printf '4\n' | ./dynamic
```

Ни одна не должна выдать ни единого предупреждения при сборке. И контрольный прогон под санитайзерами — он тоже обязан промолчать:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address,undefined \
   code/03-dynamic.c -o dyn_san
printf '4\n' | ./dyn_san
```

Санитайзер молчит, а программа печатает то же самое — значит, ни выходов за границы, ни переполнений на этом входе нет. Утечки проверяем отдельно, через `leaks` (обычная сборка без санитайзера):

```
printf '4\n' > four.txt
leaks --atExit -- ./dynamic < four.txt
```

В конце отчёта должна стоять строка `0 leaks for 0 total leaked bytes.` — «ноль утечек».

Частые ошибки

Выход за границы массива. `for (i = 0; i <= n; i++)` вместо `i < n`. Си границы не проверяет: программа прочитает или испортит чужую память и, скорее всего, даже не упадёт. Собирай с `-fsanitize=address`, когда что-то ведёт себя странно.

Число элементов от пользователя — без проверки. Объявил `int a[100]`, спросил `n` и сразу пошёл циклом до `n`. Пользователь ввёл 1000 — и ты за границей. Проверяй `n` сразу после `scanf` или бери `malloc`.

Сравнение строк через `==`. Сравнивает адреса, а не текст, и для двух разных массивов всегда даёт «не равны». Нужен `strcmp`, причём `== 0` означает «равны».

Забытый `'\0'`. Собираешь строку вручную по символам — ставь метку конца сам, иначе `printf("%s")` и `strlen` уедут в чужую память.

`fgets` и оставшийся `'\n'`. Строка `"exit\n"` не равна `"exit"`. Обрезай перевод строки сразу после чтения.

`strcpy` в маленький буфер. Функция не знает размера приёмника. Проверяй, что влезет, — или бери `snprintf`, который принимает размер.

`scanf("%s", ...)` без ограничения длины. Пиши `%63s` для массива из 64 байт. И помни, что `&` перед именем массива не нужен.

`malloc(n)` вместо `malloc(n * sizeof(int))`. Получишь вчетверо меньше памяти, чем думаешь, и выход за границу на ровном месте.

`malloc` без проверки на `NULL` и **без `free`.** Первое — возможное падение, второе — утечка.

`a = realloc(a, ...)` напрямую. При неудаче потеряешь и память, и данные. Пиши в отдельную переменную, проверяй, потом присваивай.

Обращение к памяти после `free`. Указатель остаётся прежним, но пользоваться им нельзя. Привычка `a = NULL;` сразу после `free` превращает тихую порчу данных в обращение по `NULL`, которое обычно падает сразу, а повторный `free` делает безвредным.

Звёздочка путается в объявлении и в использовании. В `int *p` она объявляет указатель, в `*p = 5` — разыменовывает. Разные роли одного знака.

«Указатель + 1 — это следующий байт». Нет: следующий **элемент**. Для `int *` адрес вырастет на 4, для `double *` — на 8.

«Не упало — значит, работает». Выход за границу, чтение по `NULL`, двойной `free` с `-O2` у нас не упали ни разу. Неопределённое поведение может выглядеть как нормальная работа — проверяй санитайзерами.

Упражнения

1. Заполни массив из 10 чисел вводом и напечатай его в обратном порядке. *Подсказка*: второй цикл идёт от `i = 9` вниз до `i >= 0` с шагом `i--`. *Что должно получиться*: на ввод `1 2 3 4 5 6 7 8 9 10` — строка `10 9 8 7 6 5 4 3 2 1`.

2. Найди в массиве максимальный элемент. *Подсказка*: заведи `max` и положи в него **первый элемент**, а не ноль — иначе на массиве из одних отрицательных чисел ответ будет неверным. *Что должно получиться*: на `{-5, -2, -9}` правильный ответ −2; вариант со стартом из нуля выдаст 0 — числа, которого в массиве нет.

3. Напиши функцию `void fill(int *a, int n, int value)`, которая заполняет первые `n` элементов значением `value`. *Подсказка*: `int *a` и `int a[]` в параметрах — одно и то же; функция получает адрес, поэтому изменения увидит и вызывающий. Здесь `const` ставить нельзя — мы как раз меняем массив. *Что должно получиться*: массив `{1, 2, 3, 4, 5}` после `fill(a, 3, 7)` печатается как `7 7 7 4 5` — последние два элемента не тронуты.

4. Прочитай строку через `fgets`, обрежь `'\n'` и напечатай длину. Потом сравни строку с `"stop"` через `strcmp` и напиши «стоп» или «дальше». *Подсказка*: проверь, что программа не падает, если ввод пустой (сразу Enter) и если ввод закончился (`fgets` вернёт `NULL`). *Что должно получиться*: на `stop` — длина 4 и «стоп»; на пустую строку — длина 0 и «дальше»; на `go` — длина 2 и «дальше»; при конце ввода (`printf '' | ./prog` или Ctrl+D в терминале) — твоё сообщение о конце ввода, без падения.

5. Напиши функцию `void min_max(const int *a, int n, int *mn, int *mx)`, которая за один проход находит минимум и максимум и возвращает **оба** через указатели. *Подсказка*: внутри пиши `*mn = ...`, а вызывай как `min_max(a, n, &lo, &hi)`. Это тот же приём, что в `swap`. Начальные значения — снова первый элемент, `*mn = a[0]; *mx = a[0];` (значит, `n` должно быть хотя бы 1). *Что должно получиться*: на массиве оценок `{4, 5, 3, 5, 2}` — `min = 2, max = 5`.

6. Через `malloc` выдели массив на введённое число элементов, заполни кубами индексов (`i*i*i`), напечатай, потом расширь через `realloc` вдвое и заполни хвост. Не забудь `free`. *Подсказка*: собери решение с `-fsanitize=address,undefined` и убедись, что санитайзер молчит; потом специально убери `free` и проверь утечку через `leaks --atExit`. Ограничь ввод сверху, как в `03-dynamic.c`: 1290^3 = 2 146 689 000 ещё влезает в `int`, а 1291^3 = 2 151 685 171 уже нет, поэтому самый большой индекс `2n - 1` должен быть не больше 1290, то есть `n` не больше 645. *Что должно получиться*: на ввод 3 — `0 1 8 27 64 125`, и `leaks` пишет `0 leaks`, пока `free` на месте.

Что мы получили

Теперь у нас есть всё необходимое для работы с данными: массивы для наборов значений, строки как массивы символов с меткой `'\0'`, указатели как адреса (и понимание, зачем `scanf` хочет `&`, и почему `a[i]` — это `*(a + i)`), динамическая память с тройкой `malloc`/`realloc`/`free`. И, что не менее ценно, — умение поймать ошибку памяти инструментом, а не гаданием: `-Wall -Wextra` при сборке, `-fsanitize=address,undefined` при отладке, `leaks` для утечек.

Это последняя «теоретическая» глава. Дальше — чистая практика. Следующая глава открывает часть про алгоритмы: начнём с простых, но показательных задач над числами — чётность, сумма цифр, переворот числа, факториал и знаменитый FizzBuzz. Заодно встретим переполнение `int` — ещё одну тихую ошибку, которую видно только если знать, куда смотреть.

Итоговые программы главы — `code/03-arrays.c`, `code/03-strings.c`, `code/03-pointers.c` и `code/03-dynamic.c`.

Глава 4. Числа: чётность, сумма цифр, переворот числа, факториал, FizzBuzz

Хватит теории — займёмся первыми настоящими алгоритмами. Все они про числа, и все опираются на один скромный, но мощный инструмент: **остаток от деления** `%` и **целочисленное деление** `/`. С их помощью мы научимся разбирать число на цифры, считать их сумму, переворачивать число задом наперёд, вычислять факториал (и заодно наткнёмся на переполнение `int` — важнейшую ловушку Си) и напишем знаменитую задачку с собеседований — FizzBuzz.

Всё, что нужно из прошлых глав, у тебя уже есть: переменные и типы, `printf`/`scanf` с `&`, арифметика и `%`, `if`/`else`, циклы `while` и `for`, функции (см. главы 2 и 3). Новое тут — только пара приёмов и тип `long long`. Их разберём подробно.

Что мы сделаем

К концу главы у тебя будут четыре рабочие программы:

- `04-digits.c` — вводит число, печатает сумму его цифр и само число задом наперёд;
- `04-factorial.c` — считает факториал, корректно справляясь с переполнением и отрицательным вводом;
- `04-overflow.c` — маленькая демонстрация: таблица, из которой видно, на каком именно `n` факториал перестаёт влезать в `int` и в `long long`;
- `04-fizzbuzz.c` — печатает числа от 1 до 100, заменяя кратные 3 на `Fizz`, кратные 5 — на `Buzz`, кратные 15 — на `FizzBuzz`.

А главное — ты прочувствуешь два оператора, `%` и `/`, на которых держится половина числовых алгоритмов.

Мостик: чётность через остаток

Вспомним из главы 2: `%` — это **остаток от деления**. `7 % 2` равно 1, потому что 7 = 2 · 3 + **1**. А `8 % 2` равно 0, потому что 8 делится на 2 без остатка.

Отсюда главный трюк:

> `n % 2 == 0` — число чётное; `n % 2 != 0` — нечётное.

И шире: `n % k == 0` означает «`n` делится на `k` нацело». Это правило мы будем гонять всю главу. Кратно 3? `n % 3 == 0`. Кратно 10? `n % 10 == 0`. Запомни эту формулу — она ключ ко всему дальнейшему.

Слово «делится» здесь в строгом математическом смысле: `n` делится на `k`, если найдётся **целое** `q`, для которого `n = k · q`. Например, 12 делится на 3 (12 = 3 · 4), а 13 — нет. Из этого определения следуют три вещи, которые полезно держать в голове:

- **0 делится на любое ненулевое число** (0 = k · 0), значит, 0 — чётное. Программа это подтверждает: `0 % 2` равно 0.
- **Отрицательные тоже бывают чётными**: −8 = 2 · (−4). И проверка `n % 2 == 0` для них работает: `-8 % 2` равно 0.
- **На ноль делить нельзя** ни в математике, ни в Си: `n % 0` и `n / 0` — неопределённое поведение (о нём ниже). Поэтому `k` в проверке `n % k == 0` должно быть ненулевым.

И одна тонкость из главы 2, которая здесь сразу пригодится: в Си деление целых **отбрасывает дробную часть**, то есть округляет к нулю (`7 / 2` = 3, `-7 / 2` = −3), а остаток `%` получает **знак делимого**: `-7 % 2` равно −1, а не 1. Это не математический остаток, который всегда неотрицателен. Поэтому нечётность проверяют как `n % 2 != 0`, а не `n % 2 == 1`: на −7 второй вариант ошибётся.

Часть 1. Сумма цифр и переворот числа

Разбираем задачу руками

Возьмём число 12345. Как достать из него отдельные цифры, не зная заранее, сколько их? Тут и пригодятся `%` и `/`.

Посмотри, что делает остаток от деления на 10:

```
12345 % 10 = 5   ← последняя цифра!
```

Деление на 10 нацело отбрасывает эту последнюю цифру:

```
12345 / 10 = 1234   ← число без последней цифры
```

Дальше повторяем то же самое с 1234, потом с 123, и так пока число не станет нулём. Распишем по шагам:

```
n = 12345   n % 10 = 5   n / 10 = 1234
n = 1234    n % 10 = 4   n / 10 = 123
n = 123     n % 10 = 3   n / 10 = 12
n = 12      n % 10 = 2   n / 10 = 1
n = 1       n % 10 = 1   n / 10 = 0   ← стоп, n стал 0
```

Мы «откусили» цифры по одной с конца: 5, 4, 3, 2, 1. Сложи их — вот и сумма цифр (15). А если приклеивать их в новое число в том порядке, в каком достаём, — получится перевёрнутое число (54321). Осталось оформить это кодом.

Шаг 1. Наивная версия — считаем сумму

Начнём с малого: только сумма цифр.

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    scanf("%d", &n);

    int sum = 0;
    while (n > 0) {
        sum = sum + n % 10;   /* добавили последнюю цифру */
        n = n / 10;           /* отбросили её */
    }

    printf("Сумма цифр: %d\n", sum);
    return 0;
}
```

Разбор новых для главы мест (всё остальное — из глав 2–3):

- `int sum = 0;` — заводим «копилку» и обнуляем её. Очень важно стартовать с нуля: если оставить `sum` без значения, в ней будет мусор и сумма получится бессмысленной.

- `while (n > 0)` — крутим цикл, пока в числе ещё остались цифры. Как только `n` дошёл до 0 (все цифры откусили), условие станет ложным и цикл закончится.
- `sum = sum + n % 10;` — здесь важен **порядок операций**. `%` и `+`: сначала считается `n % 10` (последняя цифра), потом она прибавляется к `sum`. Скобки не нужны — `%` «сильнее» `+`, как умножение сильнее сложения в обычной математике.
- `n = n / 10;` — целочисленное деление (см. главу 2): дробная часть отбрасывается, число «укорачивается» на одну цифру справа.

Запустим с числом 12345:

```
Введи целое число: 12345
Сумма цифр: 15
```

Работает. 1+2+3+4+5 = 15 — верно.

Шаг 2. Проблема — где взять перевёрнутое число?

Нам нужна ещё и вторая величина — число задом наперёд. Заведём переменную `reversed` и будем «пристраивать» к ней цифры по мере того, как достаём.

Главный приём — вот эта строка:

```
reversed = reversed * 10 + digit;
```

Как она собирает число? Каждый шаг мы сдвигаем то, что уже накопили, на разряд влево (умножение на 10 дописывает справа ноль), а на освободившееся место ставим новую цифру. Распишем для 12345:

```
старт:           reversed = 0
цифра 5:  0*10 + 5  = 5
цифра 4:  5*10 + 4  = 54
цифра 3: 54*10 + 3  = 543
цифра 2: 543*10 + 2 = 5432
цифра 1: 5432*10 + 1 = 54321
```

Так из 0 постепенно вырастает 54321. Тот же приём (накопить число, дописывая цифры справа) встретится тебе ещё много раз.

Шаг 3. Проблема — последняя цифра тратится дважды

В наивной версии я писал `n % 10` дважды: один раз для суммы, другой раз понадобится для переворота. Но между ними нельзя менять `n`! Аккуратнее — посчитать `n % 10` **один раз**, положить в отдельную переменную `digit`, и пользоваться ею:

```
int digit = n % 10;          /* последняя цифра — один раз */
sum = sum + digit;
reversed = reversed * 10 + digit;
n = n / 10;                  /* и только теперь укорачиваем n */
```

Это и понятнее, и надёжнее: видно, что обе операции работают с одной и той же цифрой, а укорачивание `n` происходит **после** них.

Шаг 4. Краевые случаи — ноль и минус

Наивный цикл `while (n > 0)` спотыкается на двух входах.

Ноль. Если ввести 0, условие `n > 0` ложно сразу, цикл не сделает ни шагу, и `sum`/`reversed` останутся нулями. Для нуля это как раз правильный ответ (сумма цифр нуля — 0, переворот — 0), так что нам повезло. Но держать этот случай в голове полезно.

Отрицательное число. Введи −90 — и `n > 0` тоже ложно сразу, программа выдаст нули. Неверно: у −90 сумма цифр 9. Поменять условие на `n != 0` мало: остаток от отрицательного получает знак делимого (`-123 % 10` равно −3), и «цифры» пойдут с минусом, сумма у −123 выйдет −6. Напрашивается простой фикс — в начале взять модуль (отбросить знак):

```
if (n < 0) {
    n = -n;   /* унарный минус: меняет знак на противоположный */
}
```

`-n` — это **унарный минус**: один операнд, меняет знак. Для −90 он даст 90, и дальше всё работает как обычно. Знак мы при этом теряем — но для суммы цифр и переворота он и не нужен.

И ещё мелочь: в цикле `n` испортится (мы же его делим), а напечатать хочется исходное число. Поэтому до цикла сохраним копию в отдельную переменную `original`.

Шаг 5. Собираем наивную версию целиком и пробуем её сломать

Соберём всё, что придумали, в один файл. Назови его `naive.c` — этот вариант мы сейчас будем ломать, поэтому в `code/` он не пойдёт:

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
```

```
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        n = -n;
    }
    int original = n;

    int sum = 0;
    int reversed = 0;
    while (n > 0) {
        int digit = n % 10;
        sum = sum + digit;
        reversed = reversed * 10 + digit;
        n = n / 10;
    }

    printf("Число:           %d\n", original);
    printf("Сумма цифр:      %d\n", sum);
    printf("Задом наперёд:   %d\n", reversed);
    return 0;
}
```

На 12345, 100 и 0 она отвечает верно, на −90 печатает сумму 9 и переворот 9 (в строке «Число» показывает уже модуль, 90 — мелкая неаккуратность, её мы тоже поправим). А теперь два злых входа, и оба ломают именно `int`.

Злой вход первый: 1000000009. Само число в `int` влезает (меньше 2 147 483 647). А вот перевёрнутое — 9 000 000 001 — уже нет:

```
Введи целое число: 1000000009
Число:           1000000009
Сумма цифр:      10
Задом наперёд:   410065409
```

Сумма цифр верная, а переворот — чушь. Вход помещался, результат уже нет: перевёрнутое число бывает **больше** исходного, про это легко забыть.

Злой вход второй: −2147483648. Это самое маленькое значение `int`. У него внутри `int` нет модуля: 2147483648 там уже не помещается, потолок — 2147483647. Значит, строка `n = -n;` физически не может дать правильный ответ:

```
Введи целое число: -2147483648
Число:           -2147483648
Сумма цифр:      47
Задом наперёд:   -126087180
```

Программа даже не напечатала модуль — «Число» осталось отрицательным, хотя знак мы вроде бы поменяли.

Обе поломки — это **неопределённое поведение** (undefined behaviour, сокращённо UB): по стандарту Си переполнение знакового целого не имеет никакого предписанного результата. Компилятор вправе делать что угодно, и с разной оптимизацией результат разный. Убедимся: соберём тот же `naive.c` без оптимизации (`-O0`) и с ней (`-O2`) и сравним на входе −2147483648:

```
cc -std=c11 -Wall -Wextra -O0 naive.c -o naive-O0
cc -std=c11 -Wall -Wextra -O2 naive.c -o naive-O2
```

```
-O0:  Сумма цифр: 0    Задом наперёд: 0
-O2:  Сумма цифр: 47   Задом наперёд: -126087180
```

Один и тот же исходник, один и тот же компилятор — два разных ответа. Вот чем UB хуже обычного бага: он не воспроизводится стабильно, и на твоей машине программа может «работать» ровно до того дня, когда её пересоберут с другими флагами.

Ловить такие места глазами тяжело, поэтому есть инструмент — **санитайзер неопределённого поведения** (UndefinedBehaviorSanitizer). Он встроен в clang и gcc, включается одним флагом и во время работы пишет в терминал, если программа сделала что-то запрещённое:

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=undefined naive.c -o naive-ub
```

Запускаем на 1000000009 — и он не молчит:

```
naive.c:22:29: runtime error: signed integer overflow: 900000000 * 10
cannot be represented in type 'int'
```

А на −2147483648:

```
naive.c:13:13: runtime error: negation of -2147483648 cannot be
represented in type 'int'; cast to an unsigned type to negate this
value to itself
```

Санитайзер назвал файл, строку, столбец и суть проблемы: строка 22 — это `reversed = reversed * 10 + digit;`, строка 13 — `n = -n;`. Возьми за привычку: сомневаешься в программе — пере-

собери её с `-fsanitize=undefined` и прогони ещё раз. Замедление в разы, зато находятся вещи, которых не видно вообще никак.

Шаг 6. Фикс — считаем в `long long`

Обе беды лечит одна и та же мысль: **считать в коробке побольше**. Тип `long long` (мы подробно разберём его во второй части главы) — это целое на 8 байт, оно вмещает примерно до 9·10^18. В нём и 2147483648, и 9000000001 помещаются свободно.

План такой:

- само введённое `n` не трогаем — оно нам нужно, чтобы честно напечатать, что ввёл пользователь;
- заводим рабочую копию `long long m = n;` и модуль берём уже у неё;
- `reversed` тоже делаем `long long`;
- а если перевёрнутое всё-таки не влезло бы в `int` — прямо говорим об этом читателю, а не делаем вид, что всё хорошо.

Чтобы сравнить с границей `int`, нам нужна сама эта граница. Она уже записана в стандартном заголовке `<limits.h>` под именем `INT_MAX`. Подключаем заголовок строкой `#include <limits.h>` рядом с `<stdio.h>`, и дальше можно писать:

```
if (reversed > INT_MAX) {
    printf("  (в int уже не влезает!)");
}
```

`INT_MAX` — константа «самое большое значение типа `int`», на типичной машине 2147483647. Рядом живут `INT_MIN`, `LLONG_MAX` и прочие. Пользоваться ими лучше, чем вбивать 2147483647 руками: на другой машине размер типа может отличаться, а `INT_MAX` подстроится.

Полная программа `04-digits.c`

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
```

```
    }

    /* Работаем не с самим n, а с копией в long long. Две причины:
       1) -n для INT_MIN не влезает в int, а в long long влезает;
       2) перевёрнутое число бывает больше исходного и int переполняет. */
    long long m = n;
    if (m < 0) {
        m = -m;   /* берём модуль: знак цифрам не нужен */
    }

    int sum = 0;             /* сюда копим сумму цифр */
    long long reversed = 0;  /* сюда собираем перевёрнутое число */

    while (m > 0) {
        int digit = (int)(m % 10);         /* последняя цифра */
        sum = sum + digit;                 /* добавили к сумме */
        reversed = reversed * 10 + digit;  /* приклеили справа */
        m = m / 10;                        /* отбросили цифру */
    }

    printf("Число:           %d\n", n);
    printf("Сумма цифр:      %d\n", sum);
    printf("Задом наперёд:   %lld", reversed);
    if (reversed > INT_MAX) {
        printf("  (в int уже не влезает!)");
    }
    printf("\n");
    return 0;
}
```

Что тут стоит разобрать построчно:

- `if (scanf("%d", &n) != 1)` — `scanf` возвращает **число успешно прочитанных значений**. Мы просили одно (`%d`), значит при успехе он вернёт 1. Если пользователь ввёл буквы вместо цифр, вернётся не 1 — и мы вежливо выходим с `return 1`. По соглашению `return 0` из `main` значит «всё хорошо», а ненулевое — «была ошибка».
- `long long m = n;` — копируем `int` в `long long`. Такое присваивание всегда безопасно: маленькая коробка целиком влезает в большую.

- `int digit = (int)(m % 10);` — `m % 10` имеет тип `long long`, а нам нужна одна цифра, для неё `int` за глаза. Запись `(int)` — это **приведение типа** (cast): «возьми это значение и считай его `int`». Скобки обязательны. Без приведения компилятор с `-Wall -Wextra` промолчал бы, но написать явно — честнее: видно, что сужение осознанное и безопасное (цифра от 0 до 9).
- `%lld` в `printf` — спецификатор для `long long` (буквы `ll` от *long long*, `d` — десятичное). Для `int` был `%d`; перепутаешь — получишь мусор.
- последний `printf("\n")` отдельной строкой: мы печатаем перевод строки уже после того, как решили, добавлять примечание про `int` или нет.

Ноль отдельным `if` не обрабатываем: при `n = 0` цикл просто не выполнится ни разу, `sum` и `reversed` останутся нулями — а это и есть верный ответ.

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/04-digits.c -o digits
./digits
```

Прогоним на всех входах, что нам встретились:

```
Введи целое число: 12345
Число:           12345
Сумма цифр:      15
Задом наперёд:   54321
```

```
Введи целое число: 100
Число:           100
Сумма цифр:      1
Задом наперёд:   1
```

Обрати внимание на 100: задом наперёд получилось `1`, а не `001`. Ведущие нули в числе не хранятся — `001` это просто `1`. Так и должно быть, когда мы переворачиваем именно **число**, а не строку.

```
Введи целое число: -90
Число:           -90
Сумма цифр:      9
Задом наперёд:   9
```

Введённое число печатаем как есть, со знаком, а цифры считаем по модулю.

```
Введи целое число: 1000000009
Число:           1000000009
Сумма цифр:      10
Задом наперёд:   9000000001  (в int уже не влезает!)
```

Вот теперь честно: 9000000001 — настоящий переворот, и программа сама предупредила, что в `int` этот ответ не поместился бы.

```
Введи целое число: -2147483648
Число:           -2147483648
Сумма цифр:      47
Задом наперёд:   8463847412  (в int уже не влезает!)
```

Самый злой вход тоже посчитан правильно (2+1+4+7+4+8+3+6+4+8 = 47). И санитайзер на всех этих запусках теперь молчит — проверь сам:

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   code/04-digits.c -o digits-san
echo 1000000009 | ./digits-san
```

Ни одной строчки `runtime error` — значит, запрещённых операций не осталось.

Одна дыра всё же есть, и она не в нашем цикле, а во вводе. Введи число, которое само не влезает в `int`:

```
Введи целое число: 99999999999
Число:           1215752191
Сумма цифр:      34
Задом наперёд:   1912575121
```

`scanf` не сказал ни слова и вернул 1 («прочитал»), а в `n` оказалось совсем другое число. По стандарту Си это тоже неопределённое поведение: если прочитанное значение не помещается в тип, `scanf` ничего не обещает. На этом маке в `n` попадают младшие 32 бита, отсюда 1215752191. Санитайзер такое не ловит — ошибка внутри библиотеки, а не в нашем коде. Надёжно проверять слишком длинные числа умеет функция `strtol` из `<stdlib.h>`, но она требует разговора про указатели на указатели, поэтому здесь просто помни: вход `scanf("%d")` должен быть в пределах `int`.

Сколько цифр в числе

Тот же цикл «откусывания» умеет и считать цифры: сколько раз откусили — столько цифр. Но тут краевые случаи меняются. У нуля цифра **одна** (сам ноль), а цикл `while (m > 0)` для нуля не

сделает ни шагу и насчитает 0. Лечится циклом `do … while` из главы 2: его тело выполняется **хотя бы раз**, до первой проверки условия. Файл `count.c`:

```c
#include <stdio.h>
#include <math.h>
#include <limits.h>

/* Число цифр целого числа. Знак цифрой не считаем. */
int count_digits(int n)
{
    long long m = n;     /* в long long модуль INT_MIN помещается */
    if (m < 0) {
        m = -m;
    }
    int count = 0;
    do {
        count++;         /* одна цифра есть всегда, даже у нуля */
        m = m / 10;
    } while (m > 0);
    return count;
}

int main(void)
{
    int xs[] = {0, 7, -90, 12345, INT_MIN};
    int k = (int)(sizeof xs / sizeof xs[0]);
    for (int i = 0; i < k; i++) {
        printf("%d: цифр %d\n", xs[i], count_digits(xs[i]));
    }

    printf("12345 по формуле: %d\n", (int)floor(log10(12345.0)) + 1);

    long long big = 999999999999999LL;   /* 15 девяток */
    printf("%lld по формуле: %d\n", big,
           (int)floor(log10((double)big)) + 1);
    return 0;
}
```

```
cc -std=c11 -Wall -Wextra -O2 count.c -o count -lm
./count
```

```
0: цифр 1
7: цифр 1
-90: цифр 2
12345: цифр 5
-2147483648: цифр 10
12345 по формуле: 5
999999999999999 по формуле: 16
```

Цикл отвечает верно на всех входах: у нуля одна цифра, у −90 — две (минус — это знак, а не цифра), у `INT_MIN` — десять.

А что за «формула»? Число из `k` цифр лежит между `10^(k-1)` и `10^k`: пятизначное 12345 не меньше 10 000 = 10^4 и меньше 100 000 = 10^5. **Десятичный логарифм** $\log_{10} x$ — это степень, в которую надо возвести 10, чтобы получить `x`: $\log_{10} 10000 = 4$, $\log_{10} 100000 = 5$. Значит, $\log_{10} 12345$ лежит между 4 и 5 (≈ 4.09), и если отбросить дробную часть (это делает `floor` — «пол», округление вниз) и прибавить единицу, получится число цифр:

$$\text{число цифр } n = \lfloor \log_{10} n \rfloor + 1, \quad \text{только для } n \geq 1$$

Для нуля формула не работает вовсе (логарифма нуля не существует), для отрицательных — тоже, их сначала берут по модулю. Функции `log10` и `floor` живут в `<math.h>`; на Linux к такой программе нужно ещё дописать `-lm` в конце команды сборки — подключить математическую библиотеку, на macOS она подключена всегда.

Последняя строка вывода показывает, почему в программах цифры всё равно считают циклом. У числа из 15 девяток цифр 15, а формула дала 16. Настоящий $\log_{10} 999999999999999$ равен 14.99999999999999957…, но `log10` считает в типе `double`, который хранит только около 16 значащих цифр. Ближайшее к ответу число `double` — ровно 15.0, и после `floor` и `+ 1` выходит 16. Дробные числа приближённые, а целочисленный цикл точен.

Часть 2. Факториал и ловушка переполнения

Разбираем задачу руками

Факториал числа `n` (пишут `n!`) — это произведение всех целых положительных чисел от 1 до `n`:

$$5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$$

Смысл у него простой: `n!` — это число способов выстроить `n` разных предметов в ряд. Три карты A, B, C раскладываются 3! = 6 способами: ABC, ACB, BAC, BCA, CAB, CBA.

Отдельно: `0! = 1`. Это не прихоть, а следствие правила `n! = n · (n−1)!`: подставь `n = 1` — получится `1! = 1 · 0!`, и раз `1! = 1`, то `0!` обязан быть 1. Да и по смыслу: пустой набор можно «выстроить» ровно одним способом — ничего не делать. А для отрицательных `n` факториал не определён: правило `n! = n · (n−1)!` дальше вниз не продолжается (из `0! = 0 · (−1)!` пришлось бы получить 1 умножением на ноль).

Считать будем циклом: завести накопитель `fact = 1` и домножать его на 2, 3, …, `n`. Стартуем именно с 1, а не с 0 — ведь умножение на 0 всё обнулит.

Шаг 1. Наивная версия на `int`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    scanf("%d", &n);

    int fact = 1;
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %d\n", n, fact);
    return 0;
}
```

Разбор:

- `int fact = 1;` — накопитель произведения, стартует с 1.
- `for (int i = 2; i <= n; i++)` — идём от 2 до `n` включительно (`<=`). С 2, а не с 1: умножать на 1 бессмысленно, результат не меняется. Если `n` равно 0 или 1, цикл не выполнится ни разу, и `fact` останется 1 — как раз правильный ответ для `0!` и `1!`.
- `fact = fact * i;` — домножаем накопитель на очередное число.

Проверим на 5 и 12:

```
Факториал какого числа? 5
5! = 120
```

```
Факториал какого числа? 12
12! = 479001600
```

Отлично. А теперь введём 13:

```
Факториал какого числа? 13
13! = 1932053504
```

Шаг 2. Проблема — это неправильный ответ!

13! на самом деле равно **6 227 020 800**, а программа выдала 1 932 053 504. Откуда мусор?

Вспомни из главы 2: `int` вмещает примерно от −2 до +2 миллиардов (точнее, до 2 147 483 647). А 13! — больше шести миллиардов. Число **не помещается** в коробку `int`. Это и есть **переполнение целого** (integer overflow) — одна из самых коварных ошибок: программа выглядит рабочей, а считает чушь.

Формально, как мы уже видели в первой части главы, переполнение знакового целого — неопределённое поведение: стандарт не обещает никакого конкретного числа. 1 932 053 504 — это то, что выдал вот этот компилятор при вот этих флагах, а не «правило Си». На практике чаще всего получается «обрезок» — младшие 32 бита настоящего произведения: 6 227 020 800 − 4 294 967 296 = 1 932 053 504, где 4 294 967 296 = 2^{32} — столько разных значений умещается в 32 бита. Но полагаться на это нельзя.

(У **беззнаковых** типов, `unsigned int` и других, всё наоборот: там переход через край стандартом разрешён и определён — результат берётся по модулю 2^{32}, то есть ровно «младшие 32 бита». Неопределённо только знаковое переполнение.)

Доведём до абсурда — попросим 20!:

```
Факториал какого числа? 20
20! = -2102132736
```

Факториал стал **отрицательным**! Произведение положительных чисел — отрицательное. Так бывает, когда младшие 32 бита произведения начинаются с единицы: в `int` старший бит означает знак. Отрицательное произведение положительных — верный признак, что `int` лопнул. Обратное неверно: 13! тоже переполнился, а остался положительным.

Шаг 3. Фикс — берём `long long`

Нужна коробка побольше. В Си для этого есть тип `long long` — **целое увеличенной разрядности**, обычно 8 байт. Он вмещает примерно до 9.2 · 10^18 (девять квинтиллионов) — на порядки больше, чем `int`.

```
long long fact = 1;
```

Печатать `long long` нужно специальным спецификатором — `%lld` (буквы `ll` — *long long*, `d` — десятичное). Если оставить `%d`, компилятор с `-Wall` тут же на это укажет. Проверим на крошечной программе (файл `lld.c`):

```
#include <stdio.h>
int main(void)
{
    long long fact = 6227020800;
    printf("%d\n", fact);
    return 0;
}
```

```
lld.c:5:20: warning: format specifies type 'int' but the argument has
type 'long long' [-Wformat]
    5 |     printf("%d\n", fact);
      |             ~~     ^~~~
      |             %lld
1 warning generated.
```

Компилятор даже подсказал внизу, чем `%d` заменить. И это не придирка: несовпадение типа и спецификатора — снова неопределённое поведение. Если предупреждение проигнорировать, на этом маке программа напечатает `1932053504` вместо `6227020800` — `printf` взял только младшие 32 бита. Пишем как надо:

```
printf("%d! = %lld\n", n, fact);
```

С `long long` 13! и 20! считаются правильно:

```
Факториал какого числа? 13
13! = 6227020800
```

```
Факториал какого числа? 20
20! = 2432902008176640000
```

Шаг 4. Где именно ломается каждый тип

«13 много для `int`, 21 много для `long long`» — откуда эти числа? Давай не поверим на слово, а посмотрим таблицу. Программа `code/04-overflow.c` считает факториал в обоих типах, но перед каждым умножением спрашивает себя: «а результат-то влезет?»

```
if (int_ok && fact_int > INT_MAX / i) {
    int_ok = 0;   /* дальше в int считать нельзя */
}
```

Приём стоит запомнить. Мы хотим узнать, влезет ли `fact_int * i`, но **не имеем права** это произведение вычислить: оно и будет тем самым UB. Поэтому переносим `i` в другую часть неравенства: `fact * i` больше `INT_MAX` ровно тогда, когда `fact` больше `INT_MAX / i`. Деление тут целочисленное, с отброшенной дробной частью, но для положительных чисел равносильность от этого не ломается. Проверим на числах: при `i = 13` получаем `INT_MAX / 13` = 165 191 049, а `fact` = 12! = 479 001 600 больше — значит, 13! не влезет, и это правда.

Само деление здесь безопасно: `INT_MAX` и `i` положительные, и частное не больше делимого. (Вообще-то целое деление тоже умеет переполняться, но ровно в одном случае: `INT_MIN / -1` — ответ 2 147 483 648 в `int` не влезает, и на этом маке программа падает с сообщением «Floating point exception».) Отсюда правило: переполнение проверяют **до** операции, а не после.

Запускаем:

```
cc -std=c11 -Wall -Wextra -O2 code/04-overflow.c -o overflow
./overflow
```

```
 n |          n! в int |         n! в long long
---+--------------------+-----------------------
 1 |                  1 |                      1
 2 |                  2 |                      2
 3 |                  6 |                      6
 4 |                 24 |                     24
 5 |                120 |                    120
 6 |                720 |                    720
 7 |               5040 |                   5040
 8 |              40320 |                  40320
 9 |             362880 |                 362880
10 |            3628800 |                3628800
11 |           39916800 |               39916800
12 |          479001600 |              479001600
13 |       переполнение |             6227020800
```

```
14 |       переполнение |             87178291200
15 |       переполнение |           1307674368000
16 |       переполнение |          20922789888000
17 |       переполнение |         355687428096000
18 |       переполнение |        6402373705728000
19 |       переполнение |      121645100408832000
20 |       переполнение |     2432902008176640000
21 |       переполнение |            переполнение
22 |       переполнение |            переполнение

INT_MAX   = 2147483647
LLONG_MAX = 9223372036854775807
```

Границы видно точно: **последний факториал, влезающий в `int`, — это 12! = 479 001 600**, а в `long long` — **20! = 2 432 902 008 176 640 000**. Начиная с 13 и с 21 соответственно считать в этих типах нельзя.

Сверься с потолками внизу таблицы: 13! = 6 227 020 800 действительно больше `INT_MAX` (2 147 483 647), а 21! ≈ 5.1 · 10^19 больше `LLONG_MAX` (≈ 9.2 · 10^18).

Шаг 5. Честная программа отказывается врать

Бесконечно растить тип не выйдет (для совсем больших чисел нужна «длинная арифметика», это отдельная большая тема). Раз мы теперь знаем границу точно — 20, — программа должна **сама** отказаться считать дальше:

```c
if (n > 20) {
    printf("Слишком большое: даже long long переполнится при n > 20.\n");
    return 1;
}
```

И заодно отсечём отрицательный ввод, для которого факториал не определён:

```c
if (n < 0) {
    printf("Факториал определён только для n >= 0.\n");
    return 1;
}
```

Полная программа `04-factorial.c`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }

    if (n > 20) {
        printf("Слишком большое: даже long long переполнится при n > 20.\n");
        return 1;
    }

    long long fact = 1;   /* 1! = 1, и 0! = 1 — стартуем с единицы */
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %lld\n", n, fact);
    return 0;
}
```

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/04-factorial.c -o factorial
```

```
Факториал какого числа? 5
5! = 120
```

```
Факториал какого числа? 21
Слишком большое: даже long long переполнится при n > 20.
```

```
Факториал какого числа? -3
Факториал определён только для n >= 0.
```

Запомни вывод из этой части: **прежде чем выбрать тип, прикинь, какие числа в нём окажутся.** Маленький `int` для счётчика — нормально. Для факториала, суммы больших величин, произведений — сразу думай про `long long`.

Часть 3. FizzBuzz

Разбираем задачу руками

Классическая задачка с собеседований. Печатаем числа от 1 до 100, но:

- если число кратно 3 — вместо него пишем `Fizz`;
- если кратно 5 — пишем `Buzz`;
- если кратно **и** 3, и 5 (то есть кратно 15) — пишем `FizzBuzz`;
- иначе — само число.

Кратность мы уже умеем проверять: `i % 3 == 0`, `i % 5 == 0`. Вся хитрость — в **порядке** проверок.

Шаг 1. Наивная версия — и её баг

Напишем «в лоб», как читается условие:

```c
for (int i = 1; i <= 100; i++) {
    if (i % 3 == 0) {
        printf("Fizz\n");
    } else if (i % 5 == 0) {
        printf("Buzz\n");
    } else {
        printf("%d\n", i);
    }
}
```

Что выведет для 15? 15 кратно 3, значит сработает **первая** ветка — напечатается `Fizz`. А мы хотели `FizzBuzz`! Случай «кратно и 3, и 5» мы вообще не предусмотрели. Это и есть главная ловушка FizzBuzz, на которой спотыкаются на собеседованиях.

Шаг 2. Фикс — самую строгую проверку ставим первой

В цепочке `if / else if` ветки проверяются **сверху вниз**, и срабатывает **первая** подходящая, остальные пропускаются. Значит самое частное условие — «кратно 15» — должно идти **раньше** более общих «кратно 3» и «кратно 5». Тогда 15 поймается до того, как дойдёт до них:

```
if (i % 15 == 0) {
    printf("FizzBuzz\n");
} else if (i % 3 == 0) {
    printf("Fizz\n");
} else if (i % 5 == 0) {
    printf("Buzz\n");
} else {
    printf("%d\n", i);
}
```

Почему `i % 15`, а не `i % 3 == 0 && i % 5 == 0`? Для 3 и 5 это одно и то же: число делится и на 3, и на 5 ровно тогда, когда делится на 15. Но работает это не потому, что 15 — произведение, а потому, что у 3 и 5 нет общих делителей, кроме 1 (такие числа называют **взаимно простыми**, подробнее — в главе 5). Для 2 и 4 правило «перемножь» ломается: 4 делится и на 2, и на 4, но на 8 — нет. В общем случае вместо произведения нужно **наименьшее общее кратное** (НОК) — самое маленькое положительное число, которое делится на оба: НОК(3, 5) = 15, а НОК(2, 4) = 4. Самый надёжный способ записи — две проверки через `&&`; `% 15` для классического FizzBuzz просто короче.

Полная программа `04-fizzbuzz.c`

```
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
```

```
        }
    }
    return 0;
}
```

Эта программа ничего не вводит — поэтому `scanf` тут не нужен.

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/04-fizzbuzz.c -o fizzbuzz
./fizzbuzz
```

Всего программа печатает 100 строк. Первые 16:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
```

Видишь: 3, 6, 9, 12 → `Fizz`; 5, 10 → `Buzz`; а 15 → `FizzBuzz`, как и задумано. Порядок проверок решил всё.

Частые ошибки

Накопитель не обнулён/не выставлен в 1. Для суммы стартуй с `sum = 0`, для произведения — с `fact = 1`. Перепутаешь — получишь мусор или вечный ноль.

`while (n > 0)` и отрицательный ввод. Цикл откусывания цифр не работает с минусом — сначала возьми модуль.

`n = -n` для самого маленького `int`. У `INT_MIN` (−2147483648) модуля внутри `int` не существует, и унарный минус тут — неопределённое поведение. Либо считай модуль в `long long`, как мы, либо явно отсекай этот вход. Ловушка редкая, но именно из-за редкости она и доживает до продакшена.

Переполнение `int`. Факториалы, произведения, суммы больших чисел легко вылетают за 2 миллиарда. Признак — внезапно отрицательный или явно неправильный результат. Лечится переходом на `long long` (и не забудь `%lld` в `printf`).

Результат больше входа. Перевёрнутое число, произведение, сумма — всё это может не влезть в тип, в который прекрасно влезали исходные данные. Прикидывай размер **результата**, а не аргументов.

Проверка переполнения «постфактум». Написать `if (fact * i < 0)` после умножения — плохая идея: само умножение уже совершило UB, и компилятор вправе выкинуть проверку целиком (он ведь «знает», что произведение положительных не бывает отрицательным). Проверяй до: `if (fact > INT_MAX / i)`.

`%d` вместо `%lld` для `long long`. Спецификатор и тип должны совпадать, иначе мусор. Компилятор с `-Wall` предупредит — не игнорируй.

Неверный порядок в FizzBuzz. Если «кратно 3» проверяется раньше «кратно 15», числа вроде 15 и 30 получат `Fizz` вместо `FizzBuzz`. Самое строгое условие — первым.

«Делится на a и на b — значит, делится на a·b». Верно только для взаимно простых `a` и `b`. Для 2 и 4 контрпример — число 4.

Деление и остаток от нуля. `n % 0` и `n / 0` — неопределённое поведение. На этом маке (процессор x86_64) программа падает с сообщением «Floating point exception: 8» и кодом возврата 136, хотя никаких дробей в ней нет, — так называется этот сигнал. Но стандарт падения не обещает. В наших программах делитель всегда 10, 3, 5, 15 — не ноль, так что мы в безопасности. Если делитель приходит от пользователя — проверяй его на ноль.

Слишком большое число на входе `scanf("%d")`. Если введённое не помещается в `int`, `scanf` не сообщает об ошибке, а в переменной оказывается что-то другое (у нас 99999999999 превратилось в 1215752191). Проверять такой ввод надёжно умеет `strtol`.

Нечётность через `n % 2 == 1`. На отрицательных ломается: `-7 % 2` равно −1. Пиши `n % 2 != 0`.

Упражнения

1. Двоичные цифры. Переделай `count_digits` так, чтобы она считала цифры числа в **двоичной** записи: делить надо не на 10, а на 2. Подсказка: это та же формула, только с лариф-

мом по основанию 2 — $\lfloor \log_2 n \rfloor + 1$ для $n \geq 1$. *Что должно получиться*: 0 → 1, 1 → 1, 5 → 3 (5 = 101 в двоичной), 255 → 8, 256 → 9, 1024 → 11, `INT_MAX` → 31.

2. Палиндром. Число-палиндром читается одинаково в обе стороны (121, 1331). Используя готовый переворот, проверь: если `reversed` равен модулю введённого числа, печатай «палиндром». Подсказка: сравнивать надо `long long` с `long long`, поэтому модуль тоже придётся где-то сохранить до цикла — сам `m` цикл испортит. *Что должно получиться*: 121, 1331, 7 и 0 — палиндромы; 123 и 10 — нет (перевёрнутое 10 — это 1); −121 при сравнении по модулю — палиндром.
3. Сумма от 1 до n. Напиши функцию `long long sum_to(int n)`, возвращающую сумму всех чисел от 1 до `n` циклом. Прогони её на `n = 100000` и посчитай ту же сумму формулой `n * (n + 1) / 2` в `int` — сравни результаты и объясни разницу. *Что должно получиться*: цикл даёт 5000050000. Формула в `int` на этом маке напечатала 705082704: произведение 100000 · 100001 = 10 000 100 000 не влезает в `int`, это UB, и санитайзер `-fsanitize=undefined` пишет `signed integer overflow: 100000 * 100001`. Исправление — считать формулу в `long long`: результат `(long long)n * (n + 1) / 2` равен 5000050000.
4. Степень. Напиши `long long power(int base, int exp)`, считающую `base` в степени `exp` циклом (накопитель стартует с 1, домножаем `exp` раз). Добавь проверку «до умножения» из шага 4 (с `LLONG_MAX`) и найди, при каком `exp` ломается `power(2, exp)` и при каком — `power(3, exp)`. *Что должно получиться*: 2^{62} = 4611686018427387904 ещё влезает, 2^{63} — уже нет (`LLONG_MAX` = $2^{63} - 1$); 3^{39} = 4052555153018976267 влезает, 3^{40} — нет.
5. FizzBuzz по-своему. Спроси у пользователя два делителя (вместо жёстких 3 и 5) и два слова, и сделай настраиваемый FizzBuzz от 1 до
   100. Не забудь проверить, что делители не равны нулю. Подсказка: случай «делится на оба» проверяй через `&&`, а не через произведение. *Что должно получиться*: для делителей 2 и 4 и слов `Foo` и `Bar` начало такое: `1 Foo 3 FooBar 5 Foo 7 FooBar` — 4 и 8 делятся на оба. С проверкой `i % (a * b)` вышло бы `1 Foo 3 Foo 5 Foo 7 FooBar`, и 4 потеряло бы `Bar`.
6. Сумма чётных цифр. Переделай сумму цифр так, чтобы она складывала только **чётные** цифры числа (подсказка: внутри цикла добавь `if (digit % 2 == 0)`). *Что должно получиться*: 12345 → 6 (2 + 4), −2468 → 20, 13579 → 0, 0 → 0.
7. Санитайзер в деле. Возьми `naive.c` из шага 5, собери его с `-fsanitize=undefined` и подбери ещё один вход, на котором санитайзер ругнётся. Подсказка: переворот девятизначного числа не больше 999 999 999 и влезает всегда, так что нужно десятизначное, у которого переворот больше 2 147 483 647 — например, с последней цифрой 3 и больше. *Что должно получиться*: на 1000000003 санитайзер пишет `naive.c:22:29: runtime error: signed integer overflow: 300000000 * 10 cannot be represented in type 'int'`, а на 2000000001 молчит (переворот 1000000002 влезает).

Что мы получили

Четыре рабочие программы — и, что важнее, чувство двух операторов, `%` и `/`, которыми разбирают числа на части. Ты увидел, как «откусывать» цифры с конца (`n % 10` и `n / 10`), как собирать число обратно (`r = r*10 + digit`), как копить сумму и произведение, и как порядок веток `if / else if` меняет результат (FizzBuzz).

А ещё ты трижды столкнулся с переполнением вживую — в перевороте, в модуле `INT_MIN` и в факториале — и получил к нему три инструмента: таблицу границ (`04-overflow.c`), проверку «до операции» (`fact > INT_MAX / i`) и санитайзер `-fsanitize=undefined`, который находит то, что глазами не видно. Этот урок будет возвращаться всю книгу.

В следующей главе останемся в мире чисел, но копнём в теорию чисел: найдём **наибольший общий делитель** двух чисел алгоритмом Евклида (тем самым `%`, только хитрее) и научимся проверять, **простое** ли число. Это первые алгоритмы, у которых есть имя и история.

Итоговые программы главы — `code/04-digits.c`, `code/04-factorial.c`, `code/04-overflow.c` и `code/04-fizzbuzz.c`.

Глава 5. НОД Евклида, простые числа и решето Эратосфена

Числа умеют больше, чем складываться и делиться. У них есть скрытая структура: общие делители, простота, разложение на множители. В этой главе мы напишем три маленькие, но знаменитые программы — и каждую доведём от наивной, медленной версии до быстрой и красивой.

Сначала найдём **наибольший общий делитель** двух чисел и познакомимся с алгоритмом Евклида — ему больше двух тысяч лет, и он до сих пор в работе. Потом научимся проверять, **простое** ли число, и поймём, почему достаточно проверок «до корня». В конце соберём **решето Эратосфена** — оно одним махом находит все простые до заданной границы.

Что мы сделаем

К концу главы у тебя будут три программы:

- `05-gcd.c` — НОД двух чисел: сперва тупым перебором делителей, потом алгоритмом Евклида.
- `05-prime.c` — проверка числа на простоту, наивно и с оптимизацией «до корня».
- `05-sieve.c` — решето Эратосфена, печатающее все простые до 100.

Плюс маленький `05-bench.c` — им мы измерим, во сколько раз умная проверка простоты обгоняет наивную, чтобы «в разы быстрее» было не словами, а числом.

Ничего нового из синтаксиса почти не понадобится: всё держится на циклах, остатке `%`, массивах и функциях из глав 2–4. Главное здесь — **идеи алгоритмов**. Их и будем разбирать руками — а потом ломать на краевых случаях, как уже привыкли в главе 4.

Часть 1. Наибольший общий делитель

Разбираем задачу руками

Наибольший общий делитель (НОД) двух чисел — это самое большое число, на которое делятся **оба** без остатка.

Возьмём 48 и 36. Выпишем делители каждого:

- делители 48: 1, 2, 3, 4, 6, 8, 12, 16, 24, 48;
- делители 36: 1, 2, 3, 4, 6, 9, 12, 18, 36.

Общие у них: 1, 2, 3, 4, 6, 12. Самый большой — **12**. Значит, НОД(48, 36) = 12.

Так мы посчитали вручную: перебрали делители и выбрали наибольший общий. Эту же мысль можно записать кодом напрямую.

Шаг 1. Наивный перебор

Сначала договоримся о слове. «`d` **делит** `a`» (или «`a` делится на `d`») значит: `a` равно `d`, умноженному на какое-то целое число. 12 делит 48, потому что $48 = 12 \cdot 4$. А 5 не делит 48: $48 = 5 \cdot 9 + 3$, и остаток 3 не ноль. В Си это проверяют остатком: `a % d == 0`.

Теперь перебор. Делитель положительного числа не может быть больше самого числа, так что общий делитель `a` и `b` не больше меньшего из них. Начнём с этого меньшего числа и пойдём **вниз**: первый же `d`, который делит оба числа, и есть наибольший общий делитель, — все кандидаты крупнее мы уже проверили и отбросили.

```c
int gcd_slow(int a, int b)
{
    int smaller = a;
    if (b < smaller) {
        smaller = b;
    }
    for (int d = smaller; d > 1; d--) {
        if (a % d == 0 && b % d == 0) {
            return d;
        }
    }
    return 1;   /* общего делителя больше 1 не нашлось */
}
```

Разбор:

- `int smaller = a; if (b < smaller) smaller = b;` — кладём в `smaller` меньшее из двух чисел. С него начнём проверку.
- `for (int d = smaller; d > 1; d--)` — счётчик `d` идёт **вниз**: `smaller`, `smaller - 1`, …, 2. `d--` уменьшает `d` на 1.
- `a % d == 0` — остаток от деления `a` на `d` равен нулю, то есть `d` делит `a` нацело. Помнишь из главы 2: `%` — это остаток.
- `a % d == 0 && b % d == 0` — `d` делит оба числа, значит, он общий делитель. Связка `&&` — логическое «И» (см. главу 2): истинно, только если истинны обе половины.
- `return d;` — нашли — сразу выходим. Этот `d` — первый подошедший при движении сверху вниз, значит, самый большой.
- `return 1;` — до 1 цикл не доходит (условие `d > 1`): 1 делит любое число, проверять её незачем. Если никто крупнее не подошёл, ответ — 1.

Запустим (НОД(48, 36)):

```
Перебором: НОД(48, 36) = 12
```

Верно: цикл проверил 36, 35, …, 13 и на 12 вернул ответ, всего 25 проверок. Но в чём проблема? Если общих делителей, кроме 1, нет, цикл пройдёт **весь путь от меньшего числа до 1**. Для 48 и 36 — пустяк. А для пары вроде 2 000 000 011 и 2 000 000 012 это два миллиарда проверок, несколько секунд работы. Медленно: мы перебираем всё подряд, хотя ответ можно получить за несколько шагов.

Шаг 2. Идея Евклида

Евклид заметил простую вещь. Пусть `a` больше `b`. Тогда **любой общий делитель `a` и `b` делит и их разность `a - b`**.

Почему? Если `d` делит `a` и делит `b`, то `a = d · x` и `b = d · y` для каких-то целых `x` и `y`. Тогда `a - b = d · (x - y)` — тоже кратное `d`. Например, 6 делит 48 и 36, и 6 делит 48 − 36 = 12. И наоборот: если `d` делит `a - b` и `b`, то делит и их сумму `(a - b) + b = a`. Значит, у пары `(a, b)` и у пары `(a - b, b)` **одни и те же** общие делители — а раз так, то и наибольший общий делитель один и тот же.

Отсюда рецепт: **большее число заменяем на разность большего и меньшего** и повторяем, пока числа не сравняются. Это сравнявшееся число и есть НОД.

Проследим на 48 и 36 руками:

```
(48, 36) -> 48 - 36 = 12 -> (36, 12)
(36, 12) -> 36 - 12 = 24 -> (24, 12)
(24, 12) -> 24 - 12 = 12 -> (12, 12)
числа сравнялись -> НОД = 12
```

Получили те же 12, но всего за три вычитания вместо перебора всех делителей. Запишем версию через вычитание:

```
int gcd_sub(int a, int b)
{
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}
```

- `while (a != b)` — крутимся, пока числа не равны (`!=` — «не равно»).
- из большего вычитаем меньшее, меньшее не трогаем.
- когда `a` и `b` сравнялись — выходим и возвращаем `a` (он же равен `b`).

У этой версии есть условие: оба числа должны быть **положительными**. Подай ей `(0, 5)` — и из пятёрки будет вечно вычитаться ноль: пара `(0, 5)` никогда не изменится, и программа зависнет. С отрицательными числами тоже беда: для `(-4, 6)` разность только растёт. Для `gcd_sub` это надо помнить; обработку нуля и минуса мы сделаем чуть позже, в шаге 4.

Уже хорошо. Но и тут есть слабое место. Если числа сильно разные, например 1000000 и 1, вычитание будет отнимать по единице миллион раз. А ведь «отнять `b` из `a` столько раз, сколько влезет» — это ровно **взять остаток** `a % b`.

Шаг 3. Евклид через остаток

Заменим череду вычитаний одним остатком. Пока `b` не ноль, делаем `a` и `b` новой парой: `(b, a % b)`. Когда `b` станет нулём, ответ — в `a`.

```
int gcd(int a, int b)
{
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

Разбор по строкам — здесь важен порядок:

- `while (b != 0)` — работаем, пока второе число не обнулилось.
- `int t = b;` — заводим временную переменную и прячем в неё текущее `b`. Зачем? Сейчас мы испортим `b`, а его старое значение нужно будет положить в `a`.
- `b = a % b;` — новое `b` — это остаток от деления `a` на `b`.
- `a = t;` — `a` в `a` кладём то, что раньше было в `b` (мы сохранили это в `t`).

Эти три строки — классический **обмен с пересчётом**. Без `t` мы бы сначала затёрли `b` и потеряли нужное значение. (Похожий приём с временной переменной ты видел в `swap` из главы 3.)

Проследим на 48 и 36:

```
a=48 b=36 -> t=36, b=48%36=12, a=36 -> (36, 12)
a=36 b=12 -> t=12, b=36%12=0,  a=12 -> (12, 0)
b == 0 -> возвращаем a = 12
```

Два шага — и ответ. Для огромных чисел Евклид тоже укладывается в считаные десятки шагов, и это можно сказать точно.

Остаток «съедает» числа очень быстро: можно доказать, что за каждые **два** шага первое число пары уменьшается как минимум вдвое. Самые неудобные для Евклида пары — соседние **числа Фибоначчи** (0, 1, 1, 2, 3, 5, 8, 13, 21, …: каждое, начиная с третьего, — сумма двух предыдущих; подробно о них — в главе 9). У них каждое деление даёт частное 1, и остаток уменьшается медленнее всего. Самая большая такая пара, что влезает в `int`, — 1 836 311 903 и 1 134 903 170, и на ней наш цикл делает 44 шага (45, если подать числа в обратном порядке: первый шаг просто поменяет их местами). Это худший случай для любых двух чисел `int`: **больше 45 шагов Евклиду не понадобится никогда**. А перебору на тех же числах пришлось бы проверить больше миллиарда кандидатов.

Почему это вообще правильно

Три строчки, а результат верный — почему? Разберём на пальцах, тут всего две мысли.

Мысль первая: остаток не теряет общих делителей. Что такое `a % b`? Это `a` минус `b`, взятое столько раз, сколько влезло:

```
a % b = a − k ∗ b     (где k = a / b, целое деление)
```

Например, `48 % 36 = 48 − 1·36 = 12`, а `100 % 7 = 100 − 14·7 = 2`.

Теперь пусть `d` делит и `a`, и `b`. Тогда `a` — это сколько-то целых `d`, и `k ∗ b` — тоже сколько-то целых `d`. Разность двух величин, кратных `d`, снова кратна `d`. Значит, `d` делит и `a % b`.

Верно и в обратную сторону: если `d` делит `b` и делит `a % b`, то он делит и `a = (a % b) + k * b` — опять сумма двух кратных `d`.

Вывод: у пары `(a, b)` и у пары `(b, a % b)` **множества общих делителей совпадают**. Раз совпадают множества — совпадают и их наибольшие элементы. Замена пары не меняет ответ. Ровно это мы и делаем в цикле, шаг за шагом.

Мысль вторая: это когда-нибудь закончится. Пока оба числа неотрицательные (а в полной программе мы этого добьёмся, см. шаг 4), остаток от деления на положительное `b` лежит от 0 до `b - 1`, то есть строго меньше `b`. Значит, новое `b` меньше старого `b`, и так на каждом шаге. Убывающая последовательность неотрицательных целых не может убывать вечно — рано или поздно упрётся в 0. Цикл завершится обязательно.

А когда `b` стал нулём, в `a` лежит ответ. Почему? НОД(`a`, 0) = `a` для положительного `a`: `a` делит сам себя, и `a` делит ноль ($0 = a \cdot 0$), так что `a` — общий делитель. А больше `a` делителей у самого `a` не бывает.

Обе мысли вместе и дают доказательство: пара меняется, ответ не меняется, процесс конечен, а в конце ответ виден невооружённым глазом.

Шаг 4. Ноль и минус — где обе версии расходятся

Программа собрана и на 48 и 36 работает. Но у нас в `main` стоит `scanf("%d %d", ...)`, а `%d` спокойно читает и ноль, и отрицательные. Проверим, что тогда получится. Вот настоящие запуски первой версии программы — функции `gcd_slow` из шага 1 и `gcd` из шага 3 плюс `main`, как в полной программе ниже, только без проверки на `INT_MIN`:

```
Введи два числа через пробел: 0 5
Перебором: НОД(0, 5) = 1
Евклидом:  НОД(0, 5) = 5
```

```
Введи два числа через пробел: 4 -6
Перебором: НОД(4, -6) = 1
Евклидом:  НОД(4, -6) = -2
```

Две функции, которые обязаны согласовываться, дают **разные** ответы — верный признак, что как минимум одна из них врёт. Разберём обе.

Перебор и ноль. При `a = 0` меньшее из чисел — ноль, цикл стартует с `d = 0`, условие `0 > 1` ложно, и цикл не выполняется ни разу. Возвращается запасная единица. А правильный ответ — 5: пятёрка делит пятёрку, и пятёрка делит ноль ($0 = 5 \cdot 0$).

Перебор и минус. При `a = 4, b = −6` меньшее — это −6, условие `−6 > 1` тоже сразу ложно. Снова возвращается единица, хотя общие делители 4 и −6 — это 1, 2 и их «минусовые двойники» −1, −2 (4 = (−2) · (−2), тоже нацело). Наибольший из них — 2, это и есть правильный ответ.

Евклид и минус. Тут интереснее. В математике остаток от деления на 4 всегда от 0 до 3: −6 = 4 · (−2) + 2, остаток 2. А Си делит целые с **отбрасыванием дробной части в сторону нуля**: `−6 / 4` — это −1.5, отбрасываем дробь, получаем `−1`. Остаток тогда `−6 − 4 * (−1)`, то есть `−2`. Отсюда правило: в Си знак у `%` **такой же, как у делимого** (левого числа): `−6 % 4` равно `−2`, `6 % −4` равно `2`. Мы уже встречали это правило в главе 2, а здесь оно впервые ломает алгоритм.

Поэтому в пару затекают отрицательные числа, и `a` в конце может оказаться со знаком минус. Проследи `gcd(4, −6)` сам: пары идут `(4, −6) → (−6, 4) → (4, −2) → (−2, 0)`, ответ −2. А `gcd(−4, 6)` даёт 2. По модулю ответ правильный, а знак зависит от того, какое из чисел было отрицательным. Но НОД — наибольший из общих делителей, а 2 больше −2, так что −2 — это ошибка.

Фикс один на всех: в самом начале берём модуль. У чисел `a` и `−a` делители одни и те же, так что знак смело выбрасываем:

```
if (a < 0) {
    a = -a;
}
if (b < 0) {
    b = -b;
}
```

А перебору вдобавок пропишем нулевые случаи явно:

```
if (a == 0) {
    return b;   /* НОД(0, b) = b */
}
if (b == 0) {
    return a;
}
```

Теперь и перебор, и Евклид на `(0, 5)` дают 5, а на `(0, 0)` — 0.

С парой нулей особая история. Ноль делится на любое число (0 = 7 · 0, 0 = 1000 · 0), поэтому общие делители у 0 и 0 — вообще все целые числа, и **наибольшего среди них нет**. По буквальному определению НОД(0, 0) не существует. Математики и программисты договорились считать его равным 0: так остаётся верным правило НОД(a, 0) = |a| (модуль `a`) для любого `a`, включая ноль, и так же отвечает, например, функция `std::gcd` в C++. Наши обе функции следуют этой договорённости.

Остался один хвост — тот самый `INT_MIN` из главы 4. Строка `a = -a;` для −2147483648 снова упрётся в неопределённое поведение. Здесь нам хватит честного отказа в `main`:

```
if (a == INT_MIN || b == INT_MIN) {
    printf("Число %d слишком маленькое: у него нет модуля в int.\n",
           INT_MIN);
    return 1;
}
```

`INT_MIN` живёт в том же `<limits.h>`, что и знакомый `INT_MAX`.

Полная программа `05-gcd.c`

```
#include <stdio.h>
#include <limits.h>

/* Наибольший общий делитель двумя способами:
   1) наивный перебор делителей;
   2) алгоритм Евклида через остаток.
   Обе функции сначала берут модуль: у −12 и 12 делители одни и те же,
   а вот знак результата иначе получается случайным. */

int gcd_slow(int a, int b)
{
    if (a < 0) {
        a = -a;
    }
    if (b < 0) {
        b = -b;
    }
    if (a == 0) {
        return b;   /* НОД(0, b) = b: на b делится и b, и ноль */
    }
    if (b == 0) {
        return a;
    }

    int smaller = a;
    if (b < smaller) {
```

```
        smaller = b;
    }
    for (int d = smaller; d > 1; d--) {
        if (a % d == 0 && b % d == 0) {
            return d;
        }
    }
    return 1;   /* общего делителя больше 1 не нашлось */
}

int gcd(int a, int b)
{
    if (a < 0) {
        a = -a;
    }
    if (b < 0) {
        b = -b;
    }

    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int main(void)
{
    int a, b;
    printf("Введи два числа через пробел: ");
    if (scanf("%d %d", &a, &b) != 2) {
        printf("Нужно два целых числа.\n");
        return 1;
    }

    /* У самого маленького int модуля в int не существует: -INT_MIN
       не влезает обратно. Честнее отказаться, чем соврать. */
```

```
    if (a == INT_MIN || b == INT_MIN) {
        printf("Число %d слишком маленькое: у него нет модуля в int.\n",
               INT_MIN);
        return 1;
    }

    printf("Перебором: НОД(%d, %d) = %d\n", a, b, gcd_slow(a, b));
    printf("Евклидом:  НОД(%d, %d) = %d\n", a, b, gcd(a, b));
    return 0;
}
```

Новое в `main`:

- `scanf("%d %d", &a, &b)` — читаем **два** числа за один вызов. Пробел в формате означает «тут может быть пробел или перевод строки» между числами. Перед каждой переменной — `&` (адрес, куда писать), как обычно для `scanf`.
- `if (scanf(...) != 2)` — `scanf` **возвращает**, сколько значений ему удалось прочитать. Мы ждём 2. Если пришло меньше (ввели буквы), ругаемся и выходим с кодом 1. `return 1;` из `main` — сигнал «программа завершилась с ошибкой» (0 — всё хорошо, не ноль — что-то не так).
- обрати внимание: аргументы функций (`a` и `b` внутри `gcd`) — это **копии**. Мы меняем их сколько хотим — `a` и `b` в `main` от этого не портятся, и вторая строка печатает те же исходные числа, что и первая. Про копии-аргументы была история с `bad_swap` в главе 3.

Запуск:

```
Введи два числа через пробел: 48 36
Перебором: НОД(48, 36) = 12
Евклидом:  НОД(48, 36) = 12
```

И ещё несколько примеров — обычные пары, а за ними те, на которых первая версия ломалась:

```
Введи два числа через пробел: 1071 462
Перебором: НОД(1071, 462) = 21
Евклидом:  НОД(1071, 462) = 21
```

```
Введи два числа через пробел: 17 5
Перебором: НОД(17, 5) = 1
Евклидом:  НОД(17, 5) = 1
```

17 и 5 не имеют общих делителей, кроме 1, — такие числа называют **взаимно простыми**.

```
Введи два числа через пробел: 0 5
Перебором: НОД(0, 5) = 5
Евклидом:  НОД(0, 5) = 5
```

```
Введи два числа через пробел: 4 -6
Перебором: НОД(4, -6) = 2
Евклидом:  НОД(4, -6) = 2
```

```
Введи два числа через пробел: 0 0
Перебором: НОД(0, 0) = 0
Евклидом:  НОД(0, 0) = 0
```

```
Введи два числа через пробел: -2147483648 4
Число -2147483648 слишком маленькое: у него нет модуля в int.
```

Обе функции сошлись на всех входах. Такая сверка двух независимых реализаций — дешёвый и очень действенный способ ловить баги: пока они согласны, обе, скорее всего, правы; разошлись — иди разбираться.

Часть 2. Простое число

Разбираем задачу руками

Простое число — это натуральное число (целое больше нуля), у которого ровно два натуральных делителя: 1 и оно само. 7 — простое (делится только на 1 и 7). 12 — нет: кроме 1 и 12, оно делится на 2, 3, 4, 6. Натуральные числа больше 1, которые не простые, называют **составными**: их можно разложить в произведение двух меньших натуральных чисел ($12 = 3 \cdot 4$).

Кто остаётся за бортом:

- **1** — ни простое, ни составное. У неё один делитель, а не два. Так договорились не случайно: иначе разложение на простые множители перестало бы быть единственным ($6 = 2 \cdot 3 = 1 \cdot 2 \cdot 3 = 1 \cdot 1 \cdot 2 \cdot 3$…).
- **0** — тоже ни то ни другое: на него делится вообще всё.
- **отрицательные** числа простыми и составными в школьном смысле не называют — эти слова придуманы для натуральных чисел.

Так что у любого целого числа три возможных ответа: «простое», «составное» или «ни то ни другое» (всё, что меньше 2). Программа должна различать все три.

Как проверить вручную? Берём число и пробуем поделить его на 2, 3, 4 и так далее. Если хоть на что-то поделилось без остатка — оно составное. Если ни на что не поделилось — простое.

Шаг 1. Наивная проверка

Переберём все возможные делители от 2 до `n - 1`. Нашёлся делитель — число составное, сразу возвращаем «нет».

```
int is_prime_slow(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}
```

Разбор:

- `if (n < 2) return 0;` — отсекаем 0, 1 и отрицательные: они не простые. Помни из главы 2: 0 — это «ложь/нет», 1 — «истина/да». Здесь функция возвращает именно такой ответ-флаг. Функция отвечает только на вопрос «простое ли?»; различать «составное» и «ни то ни другое» будем в `main`.
- `for (int i = 2; i < n; i++)` — пробуем делители от 2 до `n - 1`.
- `if (n % i == 0) return 0;` — нашёлся делитель → число составное, немедленно выходим из функции с ответом 0. `return` прекращает функцию сразу, остаток цикла не выполняется.
- `return 1;` — если цикл закончился и ни одного делителя не нашлось, число простое.

Работает. Но снова медленно: для большого **простого** `n` мы делаем `n - 2` проверок — делитель так и не найдётся, и цикл пройдёт до конца. (Составное число может выйти раньше: чётное — уже на `i = 2`.) Можно ли меньше?

Шаг 2. Почему достаточно до корня

Ключевое наблюдение. Если `n` делится на какое-то `i`, то есть и парный делитель `n / i`. Например, у 36: 2 и 18, 3 и 12, 4 и 9, 6 и 6.

Квадратный корень из `n` (пишут $\sqrt{n}$) — это такое число, которое в квадрате даёт `n`: $\sqrt{36} = 6$, потому что $6 \cdot 6 = 36$; $\sqrt{50} \approx 7.07$.

В каждой паре **один из множителей не больше $\sqrt{n}$**, а другой не меньше. Не может быть так, чтобы оба были больше корня — тогда их произведение было бы больше $\sqrt{n} \cdot \sqrt{n} = n$. Значит, если у числа вообще есть делитель, кроме 1 и самого `n`, найдётся такой делитель **не больше $\sqrt{n}$**.

Вывод: достаточно проверять `i` только до корня из `n`. Для 36 это проверки до 6 — а не до 35. Для миллиона — до тысячи вместо миллиона. Если до корня делителей нет, их нет вообще, и число простое.

Как написать «до корня» без подключения `math.h` и функции `sqrt`? Напрашивается: вместо `i <= sqrt(n)` пишем `i * i <= n`. Вроде то же самое условие, только без дробей и без библиотеки — сравниваем квадрат `i` с самим `n`.

```c
int is_prime(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}
```

Изменилась одна деталь — условие цикла: `i * i <= n` вместо `i < n`. Цикл обрывается, как только `i` переваливает за корень. Для простого `n` около миллиона это тысяча проверок вместо миллиона. Проверим на знакомых числах:

```
Введи число: 7
7 — простое число
```

```
Введи число: 97
97 — простое число
```

```
Введи число: 12
12 — составное число
```

Всё сходится. Но давай, как в главе 4, попробуем сломать.

Шаг 3. Проблема — `i * i` само переполняет `int`

Возьмём самое большое, что вообще влезает в `int`: 2 147 483 647. Это число известное — простое (математики называют его числом Мерсенна, 2^31 − 1). Что скажет наша проверка?

```
Введи число: 2147483647
2147483647 — составное число
```

Ответ неверный. И заметь: ошиблись мы не в математике, а в типе.

Смотри, где ломается. Корень из 2 147 483 647 — примерно 46 340,9. Значит, цикл дойдёт до `i = 46341` и посчитает `i * i`:

```
46341 · 46341 = 2 147 488 281
```

А потолок `int` — 2 147 483 647. Произведение не влезло, и мы получили знакомое по главе 4 неопределённое поведение. На этой машине переполнение дало отрицательное число, отрицательное меньше `n`, и условие `i * i <= n` осталось истинным. Дальше хуже: какое бы значение ни получилось у «сломанного» `i * i`, оно лежит в `int`, а значит, не больше 2 147 483 647 — то есть не больше нашего `n`. Условие не может стать ложным вообще. Цикл, который должен был остановиться на 46 340, покатился дальше, до самого `i = n`, где `n % n == 0`. Отсюда и вердикт «составное» — после нескольких секунд раздумий.

Санитайзер неопределённого поведения из главы 4 всё это видит. Сохрани наивный вариант отдельным файлом `prime-naive.c`:

```c
#include <stdio.h>

int is_prime(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

int main(void)
{
```

```
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое число\n", n);
    } else if (n < 2) {
        printf("%d — ни простое, ни составное\n", n);
    } else {
        printf("%d — составное число\n", n);
    }
    return 0;
}
```

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=undefined \
   prime-naive.c -o prime-naive
echo 2147483647 | ./prime-naive
```

```
prime-naive.c:8:23: runtime error: signed integer overflow:
46341 * 46341 cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior
prime-naive.c:8:23
Введи число: 2147483647 — составное число
```

(Длинные строки санитайзера здесь перенесены, чтобы влезли в страницу; в терминале каждая из первых двух — одна строка.)

Строка 8 — это как раз `for (int i = 2; i * i <= n; i++)`, позиция 23 в ней — знак `*`. Санитайзер не останавливает программу, а только сообщает о переполнении, поэтому ниже она всё равно печатает свой неверный ответ. Догадка подтвердилась.

Шаг 4. Фикс — переносим `i` в другую часть неравенства

Тот же приём, что и в таблице факториалов: **не вычислять то, что может не влезть**. Условие `i * i <= n` при положительных `i` и `n` равносильно условию

```
i <= n / i
```

Здесь `n` не меньше 2, а `i` положительное, поэтому `n / i` всегда помещается в `int` и не может переполниться. (Переполнить деление в Си можно ровно одним способом — `INT_MIN / -1`, ответ 2 147 483 648 не влезает; и ещё делить на ноль нельзя вовсе. У нас нет ни того, ни другого.) Так что новая проверка безопасна на любом `n`. Убедимся, что это действительно то же самое: если `i * i <= n`, то `n / i` (целое деление) не меньше `i`; если же `i * i > n`, то `n / i` строго меньше `i`. Граница ровно та же.

```
int is_prime(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}
```

Один символ разницы в условии — и:

```
Введи число: 2147483647
2147483647 — простое число
```

Санитайзер на этом входе молчит. Ответ правильный на всём диапазоне `int`. Цена фикса — деление `n / i` на каждом шаге вместо умножения `i * i`, а деление процессор выполняет заметно дольше умножения. Замер ниже покажет, что на общем выигрыше это почти не сказывается.

Полная программа `05-prime.c`

```
#include <stdio.h>

/* Проверка, простое ли число.
   is_prime_slow — делим на всё подряд до n-1.
   is_prime      — делим только до корня из n.

   Границу «до корня» пишем как i <= n / i, а НЕ как i * i <= n:
   при n около двух миллиардов произведение i * i уже не влезает в
   int, и проверка начинает врать (см. главу 4 про переполнение). */
```

```c
int is_prime_slow(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

int is_prime(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

int main(void)
{
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое число\n", n);
    } else if (n < 2) {
```

```
        printf("%d — ни простое, ни составное\n", n);
    } else {
        printf("%d — составное число\n", n);
    }
    return 0;
}
```

Новое в `main` — цепочка `if … else if … else`: сначала спрашиваем «простое?», если нет — «меньше 2?», и только если и это нет, число составное. Проверки идут сверху вниз, срабатывает первая истинная.

В файле лежат обе функции, чтобы их можно было сравнить, но `main` зовёт быструю `is_prime`. Компилятор при этом не ругается на неиспользуемую `is_prime_slow` — и вот почему: она обычная, «глобальная», а значит, её теоретически может позвать код из другого файла, который соберут вместе с нашим. Компилятор видит только один файл и делать выводы не берётся. А вот если приписать ей `static` («эта функция видна только внутри своего файла»), он сразу поймёт, что звать её больше неоткуда:

```
05-prime.c:11:12: warning: unused function 'is_prime_slow'
[-Wunused-function]
   11 | static int is_prime_slow(int n)
      |            ^~~~~~~~~~~~~
1 warning generated.
```

Запуски:

```
Введи число: 7
7 — простое число
```

```
Введи число: 12
12 — составное число
```

```
Введи число: 1
1 — ни простое, ни составное
```

```
Введи число: 0
0 — ни простое, ни составное
```

```
Введи число: -5
-5 — ни простое, ни составное
```

```
Введи число: 2
2 — простое число
```

Двойка — единственное чётное простое, и наша проверка её не теряет: цикл при `n = 2` не выполняется ни разу (`2 <= 2 / 2` — это `2 <= 1`, ложь), поэтому сразу возвращается 1.

```
Введи число: 97
97 — простое число
```

Насколько быстрее — замеряем

«В разы меньше работы» — звучит убедительно, но давай измерим. В `code/05-bench.c` лежат обе проверки, каждая ещё и считает, сколько раз проверила «делится ли `n` на `i`», а функция `clock()` из `<time.h>` засекает, сколько процессорного времени ушло на каждую. Гоняем их на худшем случае — большом простом числе, где цикл придётся прокрутить целиком:

```
cc -std=c11 -Wall -Wextra -O2 code/05-bench.c -o bench
./bench
```

```
проверяем n = 2147483647
наивно   (i < n):    2147483645 проверок,  2364.852 мс, ответ 1
до корня (i <= n/i):      46339 проверок,     0.183 мс, ответ 1
проверок меньше в 46342 раз(а)
```

(мс — миллисекунды, тысячные доли секунды.) Две с лишним секунды против двух десятых миллисекунды. Время у тебя выйдет другое — и процессор другой, и от запуска к запуску оно гуляет на несколько процентов (эти цифры сняты на Intel Core i9 2,3 ГГц). А вот количество проверок одинаково на любой машине.

Посчитаем их руками. Наивная версия пробует `i` от 2 до `n - 1`, это `n - 2` = 2 147 483 645 проверок. Умная — от 2 до $\lfloor\sqrt{n}\rfloor$ (скобки $\lfloor\ \rfloor$ значат «округлить вниз до целого»). $\sqrt{2\,147\,483\,647} \approx 46\,340.95$, то есть до 46 340, и проверок 46 340 − 1 = 46 339. Отношение 2 147 483 645 / 46 339 ≈ 46 342 — примерно $\sqrt{n}$: ведь `n / √n = √n`. То есть выигрыш растёт вместе с числом: чем больше `n`, тем сильнее отрыв.

По времени выигрыш у меня вышел меньше — около 13 тысяч раз, а не 46. Причина в том, что обороты циклов не одинаково дорогие: в быстрой версии на каждом шаге два деления (`n / i` в условии и `n % i` в теле), в медленной — одно. Да и таймер на отрезке в десятую долю миллисекунды меряет грубо. Но порядок разницы от этого не меняется: это первое в книге место, где выбор алгоритма меняет время не на проценты, а в тысячи раз.

Часть 3. Решето Эратосфена

Разбираем задачу руками

А если нужны не одно число, а **все простые до 100** сразу? Можно вызвать `is_prime` для каждого, но есть способ изящнее и быстрее, его придумал Эратосфен больше двух тысяч лет назад.

Идея — **вычёркивать**. Выпишем все числа от 2 до 100. Берём первое невычеркнутое — это 2, оно простое. Теперь вычеркнем всех его «родню»: 4, 6, 8, 10… — все числа, кратные 2 (они точно составные, раз делятся на 2).

Идём дальше. Следующее невычеркнутое — 3, тоже простое. Вычёркиваем кратные 3: 6, 9, 12, 15… (6 уже была вычеркнута — ничего, вычеркнем повторно, не страшно).

Дальше 4 — но она уже вычеркнута (как кратная 2), пропускаем. Берём 5, вычёркиваем 10, 15, 20… И так до конца. В конце все **невычеркнутые** числа — простые.

Почему невычеркнутое число обязательно простое? Если бы оно было составным, у него был бы простой делитель меньше него самого, и при проходе этого делителя число вычеркнули бы как его кратное.

Маленькая хитрость про экономию. Когда мы дошли до простого `i` и хотим вычёркивать его кратные, начинать можно не с `2*i`, а сразу с `i*i`. Почему? Любое кратное `i`, меньшее `i*i`, — это `k*i`, где `k` меньше `i`. У числа `k` есть простой делитель, не больший самого `k`, то есть меньший `i`, и `k*i` уже вычеркнули, когда проходили этот делитель. Например, для `i = 5` числа 10, 15, 20 уже убрали двойка (10, 20) и тройка (15). Так что начинаем с 25.

И ещё: запускать вычёркивание имеет смысл, только пока `i*i` не больше границы. Любое составное число до 100 раскладывается как `a * b`, где меньший множитель не больше $\sqrt{100} = 10$ (та же мысль «до корня», что и в проверке на простоту). Значит, у каждого составного до 100 есть простой делитель не больше 10, и к моменту, когда `i*i` перевалит за 100, все составные уже вычеркнуты.

Шаг 1. Массив-флаги

Нам нужно где-то отмечать, вычеркнуто число или нет. Это идеальная работа для массива: заведём массив флагов, где индекс — само число.

```
int is_composite[N + 1] = {0};
```

- `is_composite` — массив флагов. `is_composite[k]` отвечает на вопрос «число `k` составное?»: 0 — пока считаем простым, 1 — вычеркнули как составное.

- `[N + 1]` — почему плюс один? Мы хотим обращаться по индексу вплоть до самого `N` (то есть `is_composite[100]`). Индексы массива идут с нуля, поэтому, чтобы существовал элемент с индексом 100, нужен массив длиной 101. Индексы 0 и 1 просто не используем.
- `= {0}` — короткая, но важная запись: **заполнить весь массив нулями**. Если в фигурных скобках указать хотя бы один элемент, остальные С обнулит сам. Без этой инициализации в массиве был бы мусор (см. главу 3), и «невычеркнутость» нельзя было бы отличить от случайных значений.
- `N` — у нас это не переменная, а **именованная константа**. Её задаёт строка `#define N 100` в начале файла. `#define ИМЯ значение` — указание препроцессору: «перед компиляцией замени везде в тексте `N` на `100`». Удобно: поменяешь одно число — изменится везде. Имена констант принято писать заглавными.

Шаг 2. Вычёркиваем

```
for (int i = 2; i * i <= N; i++) {
    if (is_composite[i] == 0) {
        for (int j = i * i; j <= N; j += i) {
            is_composite[j] = 1;
        }
    }
}
```

Разбор — здесь два вложенных цикла:

- внешний `for (int i = 2; i * i <= N; i++)` — перебираем кандидатов в «вычёркиватели». Условие `i * i <= N` — та самая граница «до корня».
- `if (is_composite[i] == 0)` — берёмся вычёркивать только от **ещё не вычеркнутого** `i`. Если `i` сам составной, его кратные уже убраны его делителями.
- внутренний `for (int j = i * i; j <= N; j += i)` — идём по кратным `i`, начиная с `i * i`. Шаг `j += i` (прибавить `i`) перескакивает сразу к следующему кратному: `i*i`, `i*i + i`, `i*i + 2i`, …
- `is_composite[j] = 1;` — отмечаем `j` как составное (вычёркиваем).

Вложенный цикл — новинка по сравнению с прошлыми главами, но ничего магического: это просто цикл внутри цикла. На каждом шаге внешнего цикла внутренний пробегает свои кратные целиком.

Полная программа `05-sieve.c`

```c
#include <stdio.h>

/* Решето Эратосфена: печатает все простые числа до N. */

#define N 100

int main(void)
{
    int is_composite[N + 1] = {0};   /* 0 = считаем простым, 1 = вычеркнуто */

    for (int i = 2; i * i <= N; i++) {
        if (is_composite[i] == 0) {
            for (int j = i * i; j <= N; j += i) {
                is_composite[j] = 1;
            }
        }
    }

    printf("Простые числа до %d:\n", N);
    int count = 0;
    for (int i = 2; i <= N; i++) {
        if (is_composite[i] == 0) {
            printf("%d ", i);
            count++;
        }
    }
    printf("\nВсего: %d\n", count);
    return 0;
}
```

Вторая половина — печать результата:

- проходим числа от 2 до `N` и печатаем те, что **не** вычеркнуты (`is_composite[i] == 0`).
- `count` считает, сколько простых нашли, — приятно увидеть итог.

Запуск:

```
Простые числа до 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Всего: 25
```

25 простых чисел до сотни — можешь сверить со справочником, всё на месте. (Пробел после 97 перед переводом строки не опечатка: `"%d "` печатает пробел после каждого числа, и после последнего тоже.) Хочешь до 1000? Поменяй одну строку `#define N 100` на `#define N 1000`, перекомпилируй — и решето справится мгновенно (последней строкой напечатается «Всего: 168»; сам список сюда не поместится).

Краевые случаи решета

Проверим границы, как привыкли.

`N = 1` (или 0). Массив выходит длиной 2, внешний цикл не выполняется (`2 * 2 <= 1` ложно), цикл печати тоже (`2 <= 1` ложно):

```
Простые числа до 1:

Всего: 0
```

Пустая строка — это тот самый `printf("\n")` перед итогом. Ответ верный: простых до 1 нет.

Верхняя граница массива. Мы обращаемся к `is_composite[N]`, и именно ради этого объявили массив длиной `N + 1`. Прогони решето под адресным санитайзером — он проверяет каждое обращение к памяти и молча пропускает только корректные:

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   code/05-sieve.c -o sieve-san
./sieve-san
```

Санитайзер ничего не сказал — за границу не вылезаем.

А если N сделать очень большим? Тут ждёт ловушка, о которой в книгах про решето обычно не пишут. Массив `int is_composite[N + 1]` лежит на **стеке** — это небольшая область памяти, которую система выдаёт функции под её локальные переменные. На macOS и типичном Linux её размер по умолчанию 8 мегабайт (посмотреть можно командой `ulimit -s`, она печатает килобайты).

Посчитаем. Один `int` — 4 байта. При `N = 1 000 000` массиву нужно 4 000 004 байта, около 3.8 МБ, — влезает. А при `N = 10 000 000` уже больше 38 МБ, и:

```
Segmentation fault: 11
код возврата: 139
```

(Так пишет `bash`; `zsh` скажет `zsh: segmentation fault`, но смысл тот же. Код возврата 139 = 128 + 11, где 11 — номер сигнала SIGSEGV, «обращение к чужой памяти».) Программа падает мгновенно, ещё до первого `printf`. Это **переполнение стека** (stack overflow), и никакого предупреждения при компиляции не будет.

Лечится тем же `malloc` из главы 3: память, выданная через `malloc`, лежит не на стеке, а в куче, и её размер ограничен уже оперативкой, а не восемью мегабайтами. Плюс `malloc` умеет брать размер из переменной, а не из `#define`, — поэтому `N` можно будет спросить у пользователя. Ровно это в упражнении 5.

Заодно можно ужать саму память: под флаг «вычеркнуто» хватает одного байта, а не четырёх. Замени `int is_composite[]` на `char is_composite[]` — и потребление упадёт вчетверо, а остальной код не изменится ни на строчку. Правда, для `N = 10 000 000` это всё равно 10 миллионов байт — больше восьми мегабайт стека, и программа падает точно так же. Так что `char` отодвигает потолок (у меня `N = 2 000 000` на `char` работает, а на `int` уже `N = 3 000 000` падает), но не снимает его — снимает только `malloc`.

Сколько работы делает решето

Основная работа решета — вычёркивания во внутреннем цикле. Прикинем, сколько их. Простое `p` вычёркивает свои кратные с шагом `p`, то есть примерно каждое `p`-е число до `N`: это около `N / p` вычёркиваний. Складываем по всем простым:

```
N/2 + N/3 + N/5 + N/7 + ... = N · (1/2 + 1/3 + 1/5 + 1/7 + ...)
```

Сумма в скобках (обратные величины простых чисел до `N`) растёт невероятно медленно — примерно как ln ln N, «логарифм от логарифма» `N`. **Натуральный логарифм** ln x отвечает на вопрос «в какую степень надо возвести число e ≈ 2.718, чтобы получить x»: ln 1 000 000 ≈ 13.8, а ln 13.8 ≈ 2.6. Даже для миллиарда ln ln N меньше 3.1. Поэтому говорят, что решето делает порядка N · ln ln N операций — почти линейно по `N`, как будто на каждое число уходит пара-тройка действий. (В главе 8 такие оценки научимся записывать коротко: O(N log log N).)

Я посчитал вычёркивания нашим решетом (со стартом от `i*i` они чуть реже, чем по грубой оценке выше):

```
N             вычёркиваний   на одно число
100                    104            1.04
1 000 000        2 122 048            2.12
100 000 000    242 570 204            2.43
```

`N` выросло в миллион раз, а работа на одно число — чуть больше чем вдвое. Сравни с проверкой каждого числа по отдельности через `is_prime`: чтобы найти все простые до 10 000 000, ей

понадобилось 1 746 210 133 проверки делимости и 6.7 секунды на этом маке, а решету — 22 850 051 вычёркивание и 0.03 секунды. Посчитать вычёркивания сам можно в упражнении 8.

Как это работает: трассировка решета

Чтобы решето уложилось окончательно, прокрутим его для маленького `N = 10` руками. Массив (индексы 2..10), все нули:

```
индекс:  2 3 4 5 6 7 8 9 10
старт:   0 0 0 0 0 0 0 0 0
```

- `i = 2`, `2*2=4 <= 10`. Двойка не вычеркнута → вычёркиваем 4, 6, 8, 10:

```
индекс:  2 3 4 5 6 7 8 9 10
i=2:     0 0 1 0 1 0 1 0 1
```

- `i = 3`, `3*3=9 <= 10`. Тройка не вычеркнута → вычёркиваем 9 (с `i*i=9`, дальше `12 > 10` — стоп):

```
индекс:  2 3 4 5 6 7 8 9 10
i=3:     0 0 1 0 1 0 1 1 1
```

- `i = 4`: `4*4=16 > 10` → внешний цикл останавливается.

Невычеркнутые (нули): 2, 3, 5, 7. Это и есть простые до 10. Сошлось.

Проверяем

Скомпилируй и запусти каждую программу:

```
cc -std=c11 -Wall -Wextra -O2 code/05-gcd.c   -o gcd
cc -std=c11 -Wall -Wextra -O2 code/05-prime.c -o prime
cc -std=c11 -Wall -Wextra -O2 code/05-sieve.c -o sieve
./gcd
./prime
./sieve
```

Ни одного предупреждения быть не должно. Ожидаемый вывод решета — 25 простых чисел, как выше. НОД проверь хотя бы на `48 36`, `0 5`, `4 -6` и `0 0` — на трёх последних первая версия ломалась. Простоту прогони на `2`, `1`, `97` и `2147483647`.

И ещё раз, теперь все три под санитайзерами:

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   code/05-gcd.c -o gcd-san
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   code/05-prime.c -o prime-san
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   code/05-sieve.c -o sieve-san
echo "2147483647 2147483647" | ./gcd-san
echo 2147483647 | ./prime-san
./sieve-san
```

Первая команда `echo` подаёт НОД-у самые большие числа `int` — на них перебор стартует с 2 147 483 647 и сразу находит ответ. Ни строчки `runtime error` — значит, на этих входах ни переполнений, ни выходов за границу массива нет. (Санитайзер проверяет только то, что реально выполнилось, поэтому прогоняй его на краевых входах, а не на одном удобном.)

Частые ошибки

Перепутанный порядок строк в Евклиде. Если написать сначала `b = a % b;`, а потом `int t = b; a = t;`, старое `b` потеряется: в `a` попадёт уже новый остаток, и на следующем шаге `b % b` даст 0. Цикл закончится, но ответ будет 0 при любом ненулевом `b` — например, для `(48, 36)` пары идут `(48, 36) → (12, 12) → (0, 0)`. Сохраняй `b` в `t` **до** того, как его испортишь.

Забыл про модуль в НОД. Отрицательный вход даёт отрицательный ответ (потому что в Си `-6 % 4` равно `-2`: знак остатка такой же, как у делимого). Лечит строчка `if (a < 0) a = -a;` (и такая же для `b`) в начале каждой функции. А ноль ломает наивный перебор независимо от знака — ему нужны отдельные `if (a == 0) return b;` и `if (b == 0) return a;`.

`gcd_sub` на нуле. Евклид через вычитание на паре `(0, 5)` вечно вычитает ноль и виснет. Прежде чем звать его, убери знаки и ноль.

`<= n` вместо `< n` в наивной проверке простоты. С `i <= n` число поделится само на себя (`n % n == 0`), и любое число объявится составным. Делители проверяем строго до `n` (а лучше — до корня).

Забыл отсечь `n < 2`. Без проверки `if (n < 2)` единица, ноль и любое отрицательное число проскочат пустой цикл и ошибочно объявятся «простыми».

«Не простое» — значит «составное». Нет: 1, 0 и отрицательные не простые и не составные. Если программа печатает «1 — составное», это ошибка в математике, даже когда код работает.

`i * i <= n` на больших числах. Само произведение переполняет `int` раньше, чем цикл успевает остановиться, и проверка врёт — мы это видели на 2147483647. Пиши `i <= n / i`.

`i <= sqrt(n)` через дроби. Не тащи `math.h` ради `sqrt`. Для `int` такое сравнение ещё работает (`double` хранит все целые до 2^53 точно, а `sqrt` возвращает правильно округлённый результат), но стоит перейти на 64-битные числа, и `double` уже не различает соседние целые — граница может съехать на единицу. Да и `sqrt` придётся пересчитывать или заранее сохранить в переменную. Целочисленное `i <= n / i` и надёжнее, и проще.

Массив на `N`, а не на `N + 1`. Объявишь `is_composite[N]` и обратишься к `is_composite[N]` — это выход за границу (см. главу 3). Под индекс `N` нужен размер `N + 1`.

Забыл `= {0}`. Неинициализированный массив полон мусора, и решето выдаст случайную чепуху. Обнуляй флаги при объявлении.

Огромный массив на стеке. `int is_composite[10000000]` — это падение с `Segmentation fault` сразу на старте. Всё, что больше пары мегабайт, проси через `malloc`.

Упражнения

1. В `05-gcd.c` добавь функцию `gcd_sub` (Евклид через вычитание из шага 2) и печатай её результат третьей строкой. Не забудь, что на нуле и минусах она зависает: сначала возьми модули и обработай нули, как в `gcd_slow`. Ожидаемо: на `48 36`, `0 5`, `4 -6`, `0 0` и `1071 462` все три строки дают 12, 5, 2, 0 и 21. А на `2147483647 1` вычитание будет крутиться около двух миллиардов раз — сравни с мгновенным ответом `gcd`.

2. **Наименьшее общее кратное** (НОК) — самое маленькое положительное число, которое делится и на `a`, и на `b`: НОК(4, 6) = 12. Оно связано с НОД формулой `НОК(a, b) = a / gcd(a, b) * b` (для положительных `a` и `b`). Допиши функцию `lcm`. Подсказки: дели **до** умножения — `a / gcd(a, b)` делится нацело и меньше `a`; но даже так результат может не влезть в `int`: НОК(65536, 65537) = 4 295 032 832. Поэтому возвращай `long long` и считай в нём (НОК двух `int` всегда влезает в `long long`). Знак убери модулем, а если одно из чисел ноль, ответ по договорённости 0 (и заодно не придётся делить на НОД(0, 0) = 0). Ожидаемо: НОК(4, 6) = 12, НОК(−4, 6) = 12, НОК(0, 5) = 0, НОК(65536, 65537) = 4295032832.

3. Сделай так, чтобы `05-prime.c` печатал не только «простое/нет», но и **наименьший делитель** составного числа (первое `i`, на которое разделилось). Он всегда простой — подумай, почему. Ожидаемо: 91 → 7, 49 → 7, 15 → 3, 1000000 → 2.

4. На основе `is_prime` напечатай первые 20 простых чисел (заведи счётчик, перебирай числа подряд, пока не наберёшь 20). Ожидаемо: `2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71`.

5. Преврати решето в подсчёт: пусть `05-sieve.c` спрашивает `N` у пользователя и печатает только **количество** простых до него. (Подсказка: массив фиксированного размера на сте-

ке здесь не годится — вспомни `malloc` из главы 3 и не забудь `free`. Проверь на `N = 10000000`: без `malloc` программа падала.) Ожидаемо: до 100 — 25, до 1000 — 168, до 1 000 000 — 78 498, до 10 000 000 — 664 579; до 0 и до 1 — 0. Ограничь `N` сверху, например, сотней миллионов: у самого потолка `int` уже `j += i` и `i++` в циклах решета переполнятся.

6. Для каждого простого числа до 50 напечатай его «пару-близнеца», если она есть: простые-близнецы отличаются на 2 (например, 11 и 13, 17 и 19). Ожидаемо — шесть пар: 3 и 5, 5 и 7, 11 и 13, 17 и 19, 29 и 31, 41 и 43.

7. Сверь две реализации сами с собой: напиши цикл, который для всех `n` от 0 до 100000 вызывает `is_prime_slow` и `is_prime` и печатает `n`, если ответы разошлись. Ничего печататься не должно. А теперь верни в `is_prime` старое условие `i * i <= n` и найди вход, на котором расхождение появится. (Подсказка: искать придётся не в первой сотне тысяч, а у самого потолка `int`; `is_prime_slow` там слишком медленная, сравнивай с правильной `is_prime`.) Ответ: `i * i` переполняется на всех простых от 2 147 395 609 до 2 147 483 647 (у них цикл доходит до `i = 46341`), и санитайзер ругнётся на каждом. Но на этом маке неверный **ответ** получился только на 2 147 483 647: на остальных «сломанное» произведение случайно оказывалось больше `n`, и цикл выходил. Неопределённое поведение коварно именно этим — оно может долго выглядеть как правильное.

8. Добавь в решето счётчик вычёркиваний (типа `long long`) и сверь с таблицей из раздела «Сколько работы делает решето»: для `N = 100` должно выйти 104, для `N = 1 000 000` — 2 122 048. Потом начни внутренний цикл не с `i * i`, а с `2 * i` и посмотри, насколько выросло число (для миллиона — 2 197 839: на треть больше).

Что мы получили

Три классических алгоритма над числами — и в каждом мы прошли путь от наивной версии к умной. Алгоритм Евклида показал, как замена операции (вычитание → остаток) превращает миллиард шагов в несколько десятков (для `int` — не больше 45), и мы не поверили ему на слово, а разобрали, почему он даёт верный ответ и почему обязан завершиться. Проверка простоты научила приёму «достаточно до корня» — он встречается постоянно — и заодно ещё раз ткнула нас носом в переполнение: `i * i <= n` пришлось переписать как `i <= n / i`. А решето Эратосфена впервые свело нас с массивом-флагами и вложенными циклами, работающими над набором данных сразу, — и оказалось почти линейным: пара-тройка вычёркиваний на число.

Отдельно запомни привычку, которую мы тут завели: **писать вторую, заведомо тупую реализацию и сверять с ней первую**. На нулях и минусах две наши функции НОД разошлись — и только поэтому баг нашёлся.

Решето заодно подводит к следующей главе: оно обрабатывало **весь массив целиком**, и в главе 6 мы возьмёмся за такие алгоритмы над массивами вплотную: минимум и максимум, сумма и среднее, разворот на месте, подсчёт значений и циклический сдвиг. Массивы из фона выйдут на первый план, а поиском — линейным и двоичным — займёмся сразу после, в главе 7.

Итоговые программы главы — `code/05-gcd.c`, `code/05-prime.c` и `code/05-sieve.c`; замер скорости — `code/05-bench.c`.

Глава 6. Алгоритмы над массивами: минимум/максимум, сумма, среднее, разворот, сдвиг

Массив у нас уже есть: мы умеем его объявить, заполнить вводом через цикл и пройтись по элементам. Пора заставить массив работать. В этой главе мы разберём пять задач, которые встречаются буквально везде — от обработки оценок до игровой логики:

- найти **минимум** и **максимум**;
- посчитать **сумму** и **среднее**;
- **развернуть** массив задом наперёд, не заводя второй массив;
- **посчитать**, сколько раз встречается заданное число;
- **сдвинуть** массив циклически влево — сначала на одну позицию, потом на любое число позиций `k`, в том числе больше длины массива и отрицательное.

Все они — про один проход циклом и пару переменных. Никакой новой тяжёлой теории: только приёмы, которые ты будешь повторять до конца книги.

Что мы сделаем

К концу главы у тебя будут три рабочие программы:

- `06-minmax.c` — вводит числа и печатает минимум, максимум, сумму и среднее.
- `06-reverse.c` — переворачивает массив **на месте**.
- `06-count-rotate.c` — считает, сколько раз встретилось число, и сдвигает массив влево на одну позицию.

Плюс небольшой `rotate-k.c`, который ты наберёшь сам по ходу главы: он сдвигает массив на `k` позиций за один проход.

И, что важнее, ты поймёшь три вещи, на которых спотыкается почти каждый новичок: **почему минимум/максимум нельзя начинать с нуля**, **почему разворот «двумя индексами» останавливается ровно посередине** и **почему выход за границу массива — самый опасный баг в Си** (подсказка: потому что программа при этом печатает правильный ответ).

Часть 1. Минимум, максимум, сумма, среднее

Разбираем задачу руками

Пусть ввели пять чисел: `7  -3  10  4  10`.

Сумму ты бы считал так: ведёшь «копилку», начинаешь с нуля, идёшь по числам и каждое добавляешь. `0 → 7 → 4 → 14 → 18 → 28`. Сумма 28. Среднее (точнее, **среднее арифметическое**) — сумма, делённая на количество: `28 / 5 = 5.6`. Оно не обязано совпадать ни с одним из чисел набора и не обязано быть целым.

А как ты ищешь минимум глазами? Смотришь на первое число и держишь его в голове как «пока самое маленькое». Потом смотришь на каждое следующее: если оно меньше того, что в голове, — запоминаешь новое. Дошёл до конца — в голове лежит минимум. Максимум — то же самое, только «больше».

Главное здесь — **с чего начать**. «В голове» сначала лежит **первое число массива**, а не ноль. Запомни этот момент, сейчас увидим, почему он критичен.

Шаг 1. Наивная версия — и её баг

Сделаем «в лоб»: заведём `min` и `max`, поставим их в 0 (кажется логичным — «пусто»), сумму тоже в 0, и пройдёмся. Вот весь файл, назовём его `naive.c`:

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > 100) {
        printf("Нужно от 1 до 100 чисел.\n");
        return 1;
    }

    int a[100];
```

```
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        if (scanf("%d", &a[i]) != 1) {
            printf("\nЖдал число, получил что-то другое.\n");
            return 1;
        }
    }

    int min = 0;
    int max = 0;
    int sum = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] < min) min = a[i];
        if (a[i] > max) max = a[i];
        sum += a[i];
    }

    printf("Минимум: %d\n", min);
    printf("Максимум: %d\n", max);
    printf("Сумма:   %d\n", sum);
    return 0;
}
```

Разберём то, что тут нового:

- `if (scanf("%d", &n) != 1 || n <= 0 || n > 100)` — три проверки в одной. `scanf` возвращает, **сколько значений** он успешно прочитал; мы просили одно, значит ждём `1`. Дальше `n <= 0` отсекает пустой и отрицательный размер, а `n > 100` — размер больше нашего массива. Связка `||` — логическое «ИЛИ»: достаточно, чтобы сработало любое из трёх условий (см. главу 2).
- `int a[100];` — массив ровно на 100 чисел. Больше в него не влезет, поэтому такую же цифру мы и поставили в проверку выше. (Как выделить массив под точное `n`, разберём в упражнениях — там понадобится `malloc` из главы 3.)
- `if (scanf("%d", &a[i]) != 1)` — проверяем **каждое** введённое число, а не только первое. Пропустишь эту проверку — и на любом «плохом» вводе (буквы вместо цифр, конец файла) элемент `a[i]` останется **неинициализированным**, то есть в нём будет мусор, оставшийся в памяти от прошлых дел. Дальше программа честно посчитает минимум и сумму по мусору и уверенно напечатает бессмыслицу.

- `&a[i]` — адрес `i`-го элемента. `scanf` всегда хочет адрес, куда писать; для элемента массива он берётся так же, как для обычной переменной.
- `sum += a[i];` — короткая запись `sum = sum + a[i]` (видели в главе 2). К копилке прибавляем очередной элемент.
- `if (a[i] < min) min = a[i];` — если текущий элемент меньше нашего «рекордсмена», обновляем рекорд. Без фигурных скобок: когда в теле `if` одна инструкция, скобки можно опустить (но будь аккуратен).

Запускаем на числах `7 −3 10 4 10`:

```
Сколько чисел? 5
a[0] = 7
a[1] = −3
a[2] = 10
a[3] = 4
a[4] = 10
Минимум: −3
Максимум: 10
Сумма:    28
```

Минимум и максимум совпали с ожиданием. Случайно. Теперь введём числа, которые **все положительные**: `7 3 10 4 9`.

```
Сколько чисел? 5
a[0] = 7
a[1] = 3
a[2] = 10
a[3] = 4
a[4] = 9
Минимум: 0
Максимум: 10
Сумма:    33
```

Максимум верный, а минимум — **0**, хотя нуля в данных вообще нет! Вот он, баг.

В чём проблема

Мы стартовали `min` с нуля. Дальше условие `if (a[i] < min)` спрашивает «меньше ли элемент, чем 0?». Все наши числа положительные — значит, **ни одно** не меньше нуля, `min` так и остаётся 0. Программа уверенно врёт.

Если бы все числа были, наоборот, **отрицательными**, сломался бы `max`: он остался бы нулём, потому что ничего не больше нуля. Проверим на `-7 -3 -10`:

```
Сколько чисел? 3
a[0] = -7
a[1] = -3
a[2] = -10
Минимум: -10
Максимум: 0
Сумма:    -20
```

Так и есть — максимум 0, хотя самое большое здесь −3.

Ноль — это не «нейтральное пустое значение». Это конкретное число, которое влезает в сравнение и портит результат. Для суммы ноль правильный старт (прибавить ноль — ничего не меняет). А для минимума и максимума нужен старт, который **точно есть среди данных**.

Шаг 2. Фикс — стартуем с первого элемента

Правильное начало для `min` и `max` — **первый элемент массива**, `a[0]`. Он гарантированно из наших данных, поэтому ничего не исказит: в худшем случае минимум так и останется первым элементом, и это будет правдой.

```
int min = a[0];
int max = a[0];
int sum = 0;
for (int i = 0; i < n; i++) {
    if (a[i] < min) min = a[i];
    if (a[i] > max) max = a[i];
    sum += a[i];
}
```

Цикл по-прежнему идёт с `i = 0`. Да, на первом шаге мы сравниваем `a[0]` сам с собой — это не ошибка, просто `min` и `max` не изменятся. Лишнего тут нет, зато код проще.

Теперь на `7 3 10 4 9`:

```
Сколько чисел? 5
a[0] = 7
a[1] = 3
a[2] = 10
a[3] = 4
a[4] = 9
```

```
Минимум: 3
Максимум: 10
Сумма:    33
```

Минимум правильный. Баг побеждён одной строкой.

Пустой массив: почему `a[0]` — это не всегда безопасно

У фикса «стартуем с `a[0]`» есть условие: элемент `a[0]` должен **существовать**. Если `n` равно нулю, никакого `a[0]` в массиве нет, и строка `int min = a[0];` прочитает из памяти неизвестно что.

Именно поэтому в самом начале мы отсекаем `n <= 0` и выходим с ошибкой. Строчка `n <= 0` в проверке `scanf` — не формальность, а защита следующей за ней строки:

```
Сколько чисел? 0
Нужно от 1 до 100 чисел.
```

Кстати, а что вообще считать минимумом пустого набора? Ответа нет — минимума пустого множества не существует. Поэтому отказаться считать здесь честнее, чем выдумать какое-нибудь число.

Сумма: сто чисел — уже повод задуматься о типе

Мы держим сумму в `int`. Наши пять чисел дают 28, всё хорошо. А если пользователь введёт сто чисел, каждое около двух миллиардов? Сумма будет под 2 · 10^11, а потолок `int` — 2 147 483 647. Знакомое по главе 4 переполнение, и снова неопределённое поведение.

Проверка простая: **максимум входа × количество**. 100 × 2 · 10^9 = 2 · 10^11, в `int` не влезает, в `long long` (до 9 · 10^18) влезает с огромным запасом. Значит:

объявляем `long long sum = 0;` вместо `int sum = 0;`, а при печати берём `%lld` вместо `%d`:

```
printf("Сумма:    %lld\n", sum);
```

Два слова в объявлении, две буквы в формате — и целый класс багов закрыт.

А вот `min` и `max` менять не надо: они всегда равны какому-то элементу массива, а элементы у нас `int`. Тип расширяют там, где значение **вычисляется**, а не там, где оно просто копируется.

Среднее: не наступи на целочисленное деление

Количество у нас целое, сумма — тоже (пусть и `long long`). Если написать `sum / n`, Си поделит **нацело** и отбросит дробную часть — `28 / 5` даст 5, а не 5.6 (мы разбирали эту ловушку в главе 2).

Дробь отбрасывается **в сторону нуля**: `-28 / 5` даст −5, а не −6. Чтобы получить дробь, приводим сумму к `double` **до** деления:

```
double avg = (double)sum / n;
```

Разница не теоретическая, вот три варианта и что каждый печатает при сумме 28 и `n = 5`:

```
сумма:            28
sum / n:          5.00
(double)(sum / n): 5.00
(double)sum / n:  5.60
```

Первый вариант неправильный: `sum / n` — деление двух целых, дробь отброшена ещё до печати. Второй — самая частая ошибка: кажется, что раз мы приводим к `double`, всё хорошо. Но скобки поставлены не там: сначала внутри них выполняется **целое** деление `28 / 5 = 5`, дробная часть уже потеряна, и `double` получает готовую пятёрку. Приводить нужно **операнд**, а не результат.

Как только один из операндов стал `double`, второй тоже автоматически превращается в `double` на время операции (это правило называется «обычные арифметические преобразования»), и деление идёт дробное. Результат — настоящие 5.6.

`%.2f` в `printf` — печать дробного числа с двумя знаками после запятой. Просто `%f` дал бы шесть знаков: `5.600000`.

Полная программа `06-minmax.c`

```
#include <stdio.h>

#define MAXN 100

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAXN) {
        printf("Нужно от 1 до %d чисел.\n", MAXN);
        return 1;
    }

    int a[MAXN];
    for (int i = 0; i < n; i++) {
```

```
        printf("a[%d] = ", i);
        if (scanf("%d", &a[i]) != 1) {
            printf("\nЖдал число, а получил что-то другое.\n");
            return 1;
        }
    }

    int min = a[0];
    int max = a[0];
    long long sum = 0;   /* сто чисел по два миллиарда int не выдержит */
    for (int i = 0; i < n; i++) {
        if (a[i] < min) {
            min = a[i];
        }
        if (a[i] > max) {
            max = a[i];
        }
        sum += a[i];
    }

    double avg = (double)sum / n;

    printf("Минимум: %d\n", min);
    printf("Максимум: %d\n", max);
    printf("Сумма:    %lld\n", sum);
    printf("Среднее:  %.2f\n", avg);
    return 0;
}
```

Что изменилось по сравнению с наивной версией, кроме старта `min` и `max` от `a[0]`:

- `#define MAXN 100` — вместимость массива вынесена в **именованную константу** (мы завели такую же `N` в решете, глава 5). Теперь число 100 записано в программе один раз, а не в трёх местах, и поменять потолок можно одной строкой. Заглавные буквы в имени — общепринятое соглашение для констант.
- `int a[MAXN];` и `n > MAXN` в проверке — обе цифры теперь берутся из одного источника и не могут разъехаться.
- `long long sum` и `%lld` — та самая защита от переполнения суммы.

- `return 1;` — выход из `main` с кодом, отличным от нуля. По традиции `0` значит «всё хорошо», а ненулевое — «была ошибка».

Заметь ещё, чего здесь **нет**: соблазнительной строки `if (n > 100) n = 100;` — «попросили больше, молча посчитаем сто». Это плохая идея. Программа, которая тихо подменяет то, что у неё попросили, врёт пользователю: он ввёл 150 чисел, а обработали 100, и никто ему об этом не сказал. Лучше честный отказ. (Как снять потолок совсем — через `malloc` из главы 3 — в упражнении 6.)

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/06-minmax.c -o minmax
./minmax
```

Вводим пять чисел `7 -3 10 4 10` (каждое и количество — с новой строки):

```
Сколько чисел? 5
a[0] = 7
a[1] = -3
a[2] = 10
a[3] = 4
a[4] = 10
Минимум: -3
Максимум: 10
Сумма:    28
Среднее:  5.60
```

Минимум `-3`, максимум `10`, сумма `28`, среднее `5.60`. Всё сходится.

Один элемент — тоже законный случай, проверим и его:

```
Сколько чисел? 1
a[0] = 42
Минимум: 42
Максимум: 42
Сумма:    42
Среднее:  42.00
```

Минимум равен максимуму, и это правильно: в наборе из одного числа оно и самое маленькое, и самое большое.

А вот что бывает, если ввод оборвался (например, нажали Ctrl+D или подали файл, где чисел меньше обещанного):

```
Сколько чисел? 3
a[0] = 1
a[1] =
Ждал число, а получил что-то другое.
```

Программа не стала считать по мусору, а честно остановилась. Без проверки `scanf` на этом же вводе она посчитала бы минимум и сумму с участием `a[1]` и `a[2]`, в которых лежит неизвестно что. Я убрал проверку и попробовал: вышло «Минимум: 0 … Среднее: 0.33», хотя нуля никто не вводил. В другой раз, на другой машине или с другими флагами там может оказаться и «Минимум: -1167682112» — предсказать нельзя, это зависит от того, что лежало в памяти до нас.

Часть 2. Разворот массива на месте

Разбираем задачу руками

Есть массив `1 2 3 4 5`, нужно получить `5 4 3 2 1`. Условие — **без второго массива**: переставляем элементы внутри того же самого. Такой алгоритм называют работающим **на месте** (по-английски in-place): кроме самого массива, ему нужна лишь пара переменных, и их число не растёт с длиной массива — хоть 5 элементов, хоть миллион.

Возьми пять карточек в ряд. Как перевернуть порядок руками? Меняешь местами **крайние**: первую с последней. Получилось `5 2 3 4 1`. Потом вторую с предпоследней: `5 4 3 2 1`. Средняя карточка (третья) остаётся на месте — ей и так быть в центре. Готово за два обмена.

Идея: два «пальца». Левый `i` стоит на первом элементе, правый `j` — на последнем. Меняем местами то, на что они показывают, и сдвигаем пальцы навстречу друг другу: `i` вправо, `j` влево. Останавливаемся, когда они встретились или прошли друг друга.

Шаг 1. Меняем местами через временную переменную

Обмен двух элементов массива — то же самое, что обмен двух переменных из главы 3, только теперь это элементы:

```c
int t = a[i];
a[i] = a[j];
a[j] = t;
```

Почему нельзя короче, `a[i] = a[j]; a[j] = a[i];`? Потому что первая строка **затирает** `a[i]` значением `a[j]`, и старое `a[i]` теряется навсегда — вторая строка положит обратно то же `a[j]`.

Поэтому нужна временная коробка `t`, куда мы прячем `a[i]` до перезаписи. (В главе 3 мы делали ровно это через указатели в функции `swap`; здесь обмен прямо на элементах массива.)

Шаг 2. Сводим индексы навстречу

```
int i = 0;
int j = n - 1;
while (i < j) {
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
    i++;
    j--;
}
```

- `int j = n - 1;` — правый индекс ставим на **последний** элемент. Помни: индексы идут `0 .. n-1`, поэтому последний — `n - 1`, а не `n`. Написать `a[n]` значило бы выйти за границу массива.
- `while (i < j)` — повторяем, **пока левый палец левее правого**.
- внутри — обмен через `t`.
- `i++; j--;` — двигаем пальцы навстречу: левый на шаг вправо, правый на шаг влево.

Почему условие именно `i < j`

Это сердце алгоритма, разберём по случаям.

Почему не `i <= j`? Когда `i == j`, оба пальца стоят на **одном и том же** элементе (это центр массива нечётной длины). С `i <= j` цикл сделает ещё один оборот и поменяет центр сам с собой. Вреда нет, но и смысла нет — лишняя работа. Сразу после этого `i` станет больше `j`, и цикл всё равно остановится.

Почему нельзя идти до конца массива? Частая ошибка — пройти `i` по всему массиву, от 0 до `n - 1`, считая пару как `j = n - 1 - i`:

```
for (int i = 0; i < n; i++) {
    int j = n - 1 - i;
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
}
```

Первая половина оборотов развернёт массив, а вторая пройдётся по тем же парам ещё раз (`i = 3, j = 1` — та же пара, что `i = 1, j = 3`) и **развернёт его обратно**. На `1 2 3 4 5` такой цикл печатает `1 2 3 4 5` — как будто ничего не произошло. Условие `i < j` останавливает нас ровно посередине.

Посмотрим на `1 2 3 4 5` (нечётная длина, n = 5):

```
старт:  i=0 j=4   меняем a[0] и a[4] -> 5 2 3 4 1
шаг:    i=1 j=3   меняем a[1] и a[3] -> 5 4 3 2 1
шаг:    i=2 j=2   i < j ложно -> стоп (центр 3 не трогаем)
```

А на `1 2 3 4` (чётная длина, n = 4):

```
старт:  i=0 j=3   меняем a[0] и a[3] -> 4 2 3 1
шаг:    i=1 j=2   меняем a[1] и a[2] -> 4 3 2 1
шаг:    i=2 j=1   i < j ложно -> стоп
```

В обоих случаях каждый элемент, кроме центрального, переставлен ровно один раз, а обменов ровно `n / 2` в целочисленном смысле: 5 / 2 = 2, 4 / 2 = 2 — половина длины с отброшенной дробью. Быстро и без лишней памяти.

Полная программа `06-reverse.c`

```c
#include <stdio.h>

#define MAXN 100

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAXN) {
        printf("Нужно от 1 до %d чисел.\n", MAXN);
        return 1;
    }

    int a[MAXN];
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        if (scanf("%d", &a[i]) != 1) {
            printf("\nЖдал число, а получил что-то другое.\n");
```

```
            return 1;
        }
    }

    int i = 0;
    int j = n - 1;
    while (i < j) {
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
        i++;
        j--;
    }

    printf("Развёрнутый массив:");
    for (int k = 0; k < n; k++) {
        printf(" %d", a[k]);
    }
    printf("\n");
    return 0;
}
```

Ввод здесь ровно такой же, как в `06-minmax.c`: константа `MAXN`, отказ при `n` вне диапазона, проверка `scanf` на каждом числе. Дальше — разворот и печать.

При печати мы взяли отдельный счётчик `k`, потому что `i` и `j` после цикла уже «израсходованы» (они встретились в центре). И обрати внимание на формат `" %d"` — пробел **перед** числом: так числа красиво разделяются пробелами, а лишнего пробела в начале строки нет (он идёт перед первым числом, сразу после двоеточия в `printf` выше).

Проверяем

Нечётная длина — тот случай, где в центре остаётся неподвижный элемент:

```
Сколько чисел? 5
a[0] = 1
a[1] = 2
a[2] = 3
a[3] = 4
```

```
a[4] = 5
Развёрнутый массив: 5 4 3 2 1
```

Чётная — тут неподвижных нет, все элементы разбились на пары:

```
Сколько чисел? 4
a[0] = 1
a[1] = 2
a[2] = 3
a[3] = 4
Развёрнутый массив: 4 3 2 1
```

И два вырожденных случая. Два элемента — ровно один обмен:

```
Сколько чисел? 2
a[0] = 7
a[1] = 8
Развёрнутый массив: 8 7
```

Один элемент — ноль обменов: `i` и `j` сразу равны нулю, `i < j` ложно, цикл не выполняется:

```
Сколько чисел? 1
a[0] = 7
Развёрнутый массив: 7
```

Отдельно проверять «а не сломается ли на массиве из одного элемента» не пришлось — условие `i < j` разобралось само. Хороший признак: если краевой случай не требует отдельной ветки `if`, алгоритм, скорее всего, записан правильно.

Часть 3. Подсчёт значения и сдвиг влево

Две короткие, но полезные задачи в одной программе.

Задача A. Сколько раз встречается число

Идея простая: заводим счётчик `count = 0`, идём по массиву, и каждый раз, когда элемент равен искомому, прибавляем 1.

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (a[i] == target) {
        count++;
```

```
    }
}
```

- `target` — то значение, которое ищем (спросим у пользователя).
- `a[i] == target` — сравнение **на равенство**, двойное == (один = был бы присваиванием — классический баг из главы 2).
- `count++` — увеличить счётчик на 1 при каждом совпадении.

Всё. Один проход, одна переменная.

Задача Б. Циклический сдвиг влево на 1

«Сдвинуть влево на 1» значит: каждый элемент переезжает на одну позицию левее, а самый первый, которому уже некуда деваться слева, **уходит в самый конец**. Это и есть «циклический» (по кругу) сдвиг.

Руками на `10 20 30 40`:

```
было:   10 20 30 40
стало:  20 30 40 10
```

`20` встало на место `10`, `30` на место `20`, `40` на место `30`, а `10` уехало в хвост.

Как это запрограммировать? Первый элемент мы вот-вот затрём — значит, сначала **сохраним его** в отдельную переменную. Потом сдвинем все остальные: на позицию `i` положим то, что было на `i+1`. И в самый конец вернём сохранённый первый элемент.

Шаг 1. Наивно — и ловушка с границей

Напрашивается такой цикл:

```
int first = a[0];
for (int i = 0; i < n; i++) {
    a[i] = a[i + 1];   // опасно на последнем шаге!
}
a[n - 1] = first;
```

Логика «двигаем каждый влево» вроде верная, но посмотри на последний оборот цикла: при `i = n - 1` строка обращается к `a[i + 1]`, то есть к `a[n]`. А элемента `a[n]` **не существует** — индексы только до `n - 1`. Это выход за границу массива: Си не проверяет границы и молча читает чужой кусок памяти.

Самое неприятное — что программа при этом **выглядит рабочей**. Проверим на маленьком файле `naive-rotate.c`:

```c
#include <stdio.h>

int main(void)
{
    int a[4] = {10, 20, 30, 40};
    int n = 4;

    int first = a[0];
    for (int i = 0; i < n; i++) {
        a[i] = a[i + 1];
    }
    a[n - 1] = first;

    printf("После сдвига:");
    for (int i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");
    return 0;
}
```

```
cc -std=c11 -Wall -Wextra -O2 naive-rotate.c -o naive-rotate
./naive-rotate
```

```
После сдвига: 20 30 40 10
```

Ответ **правильный**! Компилятор не сказал ни слова, программа не упала. Прочитанный за границей мусор попал в `a[3]`, а следующей же строкой мы этот `a[3]` перезаписали сохранённым `first` — вот баг и не проявился. Ровно так и живут годами ошибки с границами массива: пока не повезёт, их не видно.

Позовём адресный санитайзер (AddressSanitizer) — он проверяет каждое обращение к памяти:

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   naive-rotate.c -o naive-rotate-san
./naive-rotate-san
```

Первые же строки, которые он печатает:

```
naive-rotate.c:10:16: runtime error: index 4 out of bounds for type
'int[4]'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior
```

```
naive-rotate.c:10:16
=================================================================
==47818==ERROR: AddressSanitizer: stack-buffer-overflow on address
0x7ff7b1214bb0 at pc 0x00010ecea969 bp 0x7ff7b1214b70 sp 0x7ff7b1214b68
READ of size 4 at 0x7ff7b1214bb0 thread T0
```

(Длинные строки перенесены под ширину страницы.)

Вот и приговор: `index 4 out of bounds for type 'int[4]'` — читаем элемент номер 4 у массива, где есть только 0, 1, 2, 3. Строка 10 — это как раз `a[i] = a[i + 1];`. Дальше санитайзер печатает ещё десяток строк с адресами и картой памяти; там, в частности, написано `'a' <== Memory access at offset 48 overflows this variable` — мы полезли ровно за правый край массива `a`.

Номер процесса (`==47818==`) и адреса (`0x7ff7...`) у тебя будут другие, они меняются от запуска к запуску. Важны строка с `runtime error` (её печатает санитайзер неопределённого поведения, `undefined`) и строка с `ERROR: AddressSanitizer` (это уже адресный). Первый только сообщает и даёт программе идти дальше, второй останавливает её на месте — поэтому `После сдвига:` в этот раз не напечатается.

Шаг 2. Фикс — останавливаем цикл на один шаг раньше

Нам нужно сдвигать только до предпоследнего элемента включительно: последний (`a[n-1]`) мы всё равно перезапишем сохранённым `first` вручную. Поэтому условие — `i < n - 1`:

```c
int first = a[0];
for (int i = 0; i < n - 1; i++) {
    a[i] = a[i + 1];
}
a[n - 1] = first;
```

Теперь максимальное `i` равно `n - 2`, и `a[i + 1]` — это `a[n - 1]`, последний существующий элемент. За границу не вылезаем.

Проследим на `10 20 30 40` (n = 4, `first = 10`):

```
i=0:  a[0] = a[1]  ->  20 20 30 40
i=1:  a[1] = a[2]  ->  20 30 30 40
i=2:  a[2] = a[3]  ->  20 30 40 40
конец цикла, ставим a[3] = first(10) -> 20 30 40 10
```

Именно то, что мы насчитали руками.

Полная программа `06-count-rotate.c`

```c
#include <stdio.h>

#define MAXN 100

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAXN) {
        printf("Нужно от 1 до %d чисел.\n", MAXN);
        return 1;
    }

    int a[MAXN];
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        if (scanf("%d", &a[i]) != 1) {
            printf("\nЖдал число, а получил что-то другое.\n");
            return 1;
        }
    }

    int target;
    printf("Какое значение посчитать? ");
    if (scanf("%d", &target) != 1) {
        printf("\nЖдал число, а получил что-то другое.\n");
        return 1;
    }

    int count = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            count++;
        }
    }
    printf("Значение %d встречается %d раз(а)\n", target, count);
```

```
    int first = a[0];
    for (int i = 0; i < n - 1; i++) {
        a[i] = a[i + 1];
    }
    a[n - 1] = first;

    printf("После сдвига влево:");
    for (int i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");
    return 0;
}
```

Проверку `scanf` мы поставили и на `target` тоже — иначе на оборванном вводе программа искала бы в массиве случайное число из мусора и печатала бы что-нибудь вроде «Значение 1167885744 встречается 0 раз(а)». Правило простое: **любое** значение, пришедшее от пользователя, проверяется на месте.

Проверяем

```
Сколько чисел? 6
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 20
a[4] = 50
a[5] = 20
Какое значение посчитать? 20
Значение 20 встречается 3 раз(а)
После сдвига влево: 20 30 20 50 20 10
```

Число `20` встретилось 3 раза — верно. И массив сдвинулся: первый `10` уехал в хвост, остальные сместились влево. Ничего не потерялось: было шесть чисел, стало шесть тех же самых, просто в другом порядке — в этом и смысл слова «циклический».

Массив из одного элемента сдвиг тоже переживает. Цикл `for (int i = 0; i < n - 1; i++)` при `n = 1` не выполняется ни разу (`0 < 0` ложно), а строка `a[n - 1] = first;` кладёт `a[0]` обратно в `a[0]`:

```
Сколько чисел? 1
a[0] = 5
Какое значение посчитать? 5
Значение 5 встречается 1 раз(а)
После сдвига влево: 5
```

Прогоним обе программы главы под санитайзерами — они проверяют и границы массива, и переполнения:

```
cc -std=c11 -Wall -Wextra -O1 -fsanitize=address,undefined \
   code/06-count-rotate.c -o rotate-san
printf '6\n10\n20\n30\n20\n50\n20\n20\n' | ./rotate-san
```

Ни одной строчки `runtime error` — в отличие от наивной версии со сдвигом до `i < n`.

Сдвиг на k позиций

Сдвиг на одну позицию умеем. А если надо на `k`? Руками на `10 20 30 40 50` при `k = 2`: два первых числа уезжают в хвост, в том же порядке:

```
было:   10 20 30 40 50
стало:  30 40 50 10 20
```

Наивно — повторить сдвиг на 1 ровно `k` раз. Работать будет, но посчитаем цену. Один сдвиг на 1 — это `n` присваиваний (`n - 1` в цикле плюс последнее `a[n - 1] = first`) и ещё одно для `first`. `k` сдвигов — примерно `k · n` присваиваний. Для массива в миллион элементов и `k = 500 000` это полтриллиона присваиваний — минуты работы там, где хватило бы миллиона.

Прежде чем ускорять, разберёмся со странными `k`.

`k` больше `n`. Сдвинь массив из 5 элементов на 5 — каждый элемент обойдёт полный круг и встанет на своё место. Значит, сдвиг на 7 — то же самое, что сдвиг на 7 − 5 = 2, а вообще важен только остаток `k % n`. Сдвиг на 1 000 000 007 позиций массива из 5 элементов — это сдвиг на 2 (1 000 000 007 = 5 · 200 000 001 + 2).

Отрицательный `k`. Естественно понимать его как сдвиг **вправо**: `k = -1` — последний элемент уезжает в начало. А сдвиг вправо на 1 — то же самое, что сдвиг влево на `n - 1` (проверь на пальцах: `10 20 30 40 50` → `50 10 20 30 40` в обоих случаях). Осторожно с остатком: в Си знак `%` такой же, как у делимого (см. главы 2 и 5), так что `-1 % 5` равно −1, а не 4, и `-6 % 5` тоже −1. Поэтому после `k % n` отрицательный результат поправляем прибавлением `n`: −1 + 5 = 4.

`n` равно 0. Тогда `k % n` — деление на ноль, это неопределённое поведение (на этом маке программа падает). Массив из нуля или одного элемента сдвигать нечего, так что такие `n` отсекаем до всякой арифметики.

Быстро: три разворота. Есть красивый приём, который делает сдвиг на любое `k` за один проход, **на месте** и без всяких повторов. Разрежем массив на две части: `A` — первые `k` элементов, `B` — остальные. Сдвиг влево на `k` превращает `A B` в `B A`. И вот как это получить разворотами из части 2:

```
было:              10 20 | 30 40 50      (A = 10 20, B = 30 40 50)
развернули A:      20 10 | 30 40 50
развернули B:      20 10 | 50 40 30
развернули всё:    30 40 50 | 10 20      (это B A)
```

Почему работает? Разворот всего массива меняет местами блоки **и** переворачивает каждый блок внутри. Мы заранее перевернули блоки по отдельности, и второй разворот вернул их содержимое в правильный порядок — а сами блоки поменялись местами.

Сколько это стоит? Разворот куска длины `m` делает `m / 2` обменов. Всего `k / 2 + (n - k) / 2 + n / 2`, то есть не больше `n` обменов при любом `k`. Для миллиона элементов — миллион обменов вместо полутриллиона присваиваний.

Собери файл `rotate-k.c`:

```c
#include <stdio.h>

/* Разворачивает кусок массива a[from..to] включительно. */
void reverse_range(int *a, int from, int to)
{
    while (from < to) {
        int t = a[from];
        a[from] = a[to];
        a[to] = t;
        from++;
        to--;
    }
}

/* Циклический сдвиг влево на k позиций за три разворота. */
void rotate_left(int *a, int n, int k)
{
    if (n <= 1) {
        return;                 /* сдвигать нечего */
    }
    k = k % n;                  /* лишние полные круги долой */
```

```
    if (k < 0) {
        k = k + n;              /* вправо на 1 = влево на n - 1 */
    }
    reverse_range(a, 0, k - 1);
    reverse_range(a, k, n - 1);
    reverse_range(a, 0, n - 1);
}

int main(void)
{
    int ks[] = {0, 1, 2, 5, 7, -1, -6};
    int count = (int)(sizeof ks / sizeof ks[0]);

    for (int t = 0; t < count; t++) {
        int a[5] = {10, 20, 30, 40, 50};
        rotate_left(a, 5, ks[t]);
        printf("k = %2d:", ks[t]);
        for (int i = 0; i < 5; i++) {
            printf(" %d", a[i]);
        }
        printf("\n");
    }
    return 0;
}
```

Разбор:

- `void reverse_range(int *a, int from, int to)` — тот же разворот двумя пальцами, только не всего массива, а куска с `a[from]` по `a[to]` включительно. Массив приходит в функцию как адрес начала (`int *a`, см. главу 3), поэтому функция меняет настоящий массив `main`, а не копию. `void` — функция ничего не возвращает.
- `if (n <= 1) return;` — защита от деления на ноль в `k % n` и от бессмысленной работы на одном элементе.
- `k = k % n;` — отбрасываем полные круги. После этого `k` лежит от `-(n - 1)` до `n - 1`. (Даже для самого маленького `int`, −2147483648, это безопасно: переполнить `%` может только делитель −1, а у нас `n` не меньше 2.)
- `if (k < 0) k = k + n;` — отрицательный остаток превращаем в равносильный сдвиг влево.

- три вызова `reverse_range` — ровно три разворота из таблички выше. При `k = 0` первый вызов получит кусок `a[0..-1]`, где `from > to`, и цикл в нём не выполнится ни разу; второй и третий развернут весь массив туда и обратно. Лишняя работа, но ответ верный.
- `int ks[] = {0, 1, 2, 5, 7, -1, -6};` — набор проверочных `k`. Квадратные скобки без числа: размер компилятор посчитает сам по списку.
- `(int)(sizeof ks / sizeof ks[0])` — число элементов массива, приём из главы 3.
- `int a[5] = {10, 20, 30, 40, 50};` внутри цикла — на каждом обороте массив заводится заново, поэтому каждый `k` сдвигает исходный набор, а не результат прошлого сдвига.
- `%2d` — печать числа в поле ширины 2, чтобы столбец выровнялся.

```
cc -std=c11 -Wall -Wextra -O2 rotate_k.c -o rotate_k
./rotate_k
```

```
k =  0: 10 20 30 40 50
k =  1: 20 30 40 50 10
k =  2: 30 40 50 10 20
k =  5: 10 20 30 40 50
k =  7: 30 40 50 10 20
k = -1: 50 10 20 30 40
k = -6: 50 10 20 30 40
```

`k = 5` вернул массив на место (полный круг), `k = 7` совпал с `k = 2`, а `k = -1` и `k = -6` — со сдвигом вправо на 1. Я сверил `rotate_left` с наивным «`k` раз по одному» на всех `n` от 1 до 9 и всех `k` от −30 до 30 — расхождений нет, санитайзеры молчат.

Частые ошибки

Инициализация min/max нулём. Самая коварная: на положительных данных врёт минимум, на отрицательных — максимум, а на смешанных может случайно «повезти». Всегда стартуй с `a[0]`, а не с 0 (и не с «огромного числа наугад» — это тоже хрупко).

Минимум/максимум по пустому массиву. Если `n` равно 0, то `a[0]` не существует, и старт `min = a[0]` уже ошибка. Поэтому мы заранее отсекаем `n <= 0`. У пустого набора минимума просто нет — правильный ответ тут «отказаться считать», а не выдумать число.

`scanf` без проверки в цикле ввода. Проверить первое число и забыть про остальные — типичная невнимательность. Если ввод оборвётся на середине, `a[i]` останется неинициализированным, и дальше программа посчитает минимум, сумму и среднее по случайному мусору из памяти — причём без единого предупреждения. Проверяй **каждый** `scanf`.

Молчаливое урезание `n`. `if (n > 100) n = 100;` выглядит аккуратно, а на деле подменяет задачу: попросили 150, посчитали 100 и не сказали. Либо честный отказ, либо `malloc` под точный размер.

Выход за границу при сдвиге. `a[i + 1]` в цикле до `i < n` лезет в `a[n]`. Останавливай цикл на `i < n - 1`. То же правило — при любом обращении к «соседу справа». И помни: такой баг легко **не проявляется** — собери программу с `-fsanitize=address`, если есть хоть малейшее сомнение.

`a[n]` вместо `a[n - 1]`. Последний элемент — `n - 1`. Постоянный источник ошибок и в развороте (`j = n - 1`), и в сдвиге.

`=` вместо `==` при подсчёте. `if (a[i] = target)` присвоит элементу `target` (испортит массив!), а условием станет само присвоенное значение. Если `target` не ноль, условие истинно на **каждом** элементе и счётчик выйдет равным `n`; если `target` равен нулю — ложно всегда, счётчик 0, а массив весь в нулях. Нужно `==`. Компилятор с `-Wall` предупреждает («using the result of an assignment as a condition without parentheses») — не игнорируй предупреждения.

Среднее нацело. `double avg = sum / n;` посчитает нацело **до** записи в `double`. `(double)(sum / n)` — та же ошибка в маскировке: скобки не там. Приводи операнд: `(double)sum / n`.

Сумма в `int`. Сами элементы влезают, а их сумма — уже нет. Прикинь максимум входа, умножь на количество и, если результат за двумя миллиардами, бери `long long` (и `%lld` в `printf`).

Обмен без временной переменной. `a[i] = a[j]; a[j] = a[i];` теряет одно из значений. Нужна `t`.

Потеря первого элемента при сдвиге. Если не сохранить `a[0]` в `first` до цикла, самый первый шаг его затрёт, и в хвост уезжать будет уже нечему. Забудешь последнюю строку совсем — на `10 20 30 40` получишь `20 30 40 40` (последний элемент задвоился). Напишешь в конце `a[n - 1] = a[0];` вместо `first` — получишь `20 30 40 20`: в `a[0]` к этому моменту уже лежит 20, а не 10.

Сдвиг на `k` без `k % n`. Повторять сдвиг на 1 миллиард раз при `n = 5` — впустую крутить полные круги. А отрицательный `k` без поправки `k + n` даст отрицательный индекс. И помни, что при `n = 0` выражение `k % n` — деление на ноль.

Упражнения

1. Добавь в `06-minmax.c` вывод **позиции** (индекса) минимума и максимума, а не только их значений. Подсказка: заведи `imin`, `imax` и обновляй их рядом с `min`, `max`. Отдельно подумай, что печатать, если минимум встречается несколько раз, — первую позицию или последнюю? Проверь на `3 1 5 1`. Ожидаемо: максимум 5 на позиции 2; минимум 1 — на позиции

1, если обновлять по строгому < (первое вхождение), и на позиции 3, если по <= (последнее).

2. Второй по величине элемент. Найди за **один** проход не максимум, а второй по величине (для `7 3 10 4 9` это 9). Подсказка: держи две переменные, `max` и `second`, и обновляй их аккуратно. Сначала договорись об определении: «второе по величине **различное** значение» или «второй элемент, если отсортировать по убыванию»? Для `10 10 3` первое определение даёт 3, второе — 10. А на `5 5 5` по первому определению ответа нет вовсе — программа должна так и сказать, а не печатать мусор или `max`.
3. Сделай разворот **функцией** `void reverse(int *a, int n)` и вызови её из `main`. Вспомни из главы 3: имя массива — это адрес начала, поэтому функция изменит настоящий массив. Проверь, что она корректна при `n = 0` и `n = 1`. Ожидаемо: при `n = 0` будет `j = -1`, условие `0 < -1` ложно, и функция ничего не трогает; при `n = 1` — тоже ноль обменов.
4. Циклический сдвиг **вправо** на 1: последний элемент уходит в начало, остальные сдвигаются вправо. Подумай, в какую сторону теперь должен идти цикл, чтобы не затереть нужное значение раньше времени. Подсказка: сохранить надо `a[n - 1]`, а цикл пойдёт с конца (`a[i] = a[i - 1]` для `i` от `n - 1` вниз до 1). Ожидаемо: `10 20 30 40` → `40 10 20 30`. Сверь с `rotate_left(a, n, -1)` из `rotate-k.c`.
5. Сдвиг влево на `k` «в лоб»: повтори сдвиг на 1 ровно `k % n` раз (с поправкой для отрицательного `k`) и сверь результат с `rotate_left` для всех `k` от −20 до 20 на массиве из 5 элементов. Ожидаемо: ни одного расхождения. Потом добавь счётчик присваиваний в обе версии и сравни их для `n = 1000`, `k = 500`: у наивной выйдет около полумиллиона (500 сдвигов по ~1000), у трёх разворотов — 1000 обменов по 3 присваивания, то есть 3000.
6. Перепиши любую из программ так, чтобы массив выделялся через `malloc(n * sizeof(int))` (глава 3) под точное `n`, без потолка в 100 элементов. Не забудь `free` в конце и проверку `malloc` на `NULL`. `n` по-прежнему проверяй: хотя бы `n >= 1` (иначе снова нет `a[0]`) и разумный потолок сверху, чтобы `n * sizeof(int)` не попросил невозможного. Прогони результат под `-fsanitize=address` — он поймает выход за границу кучи. А вот утечки на macOS он не ловит (см. главу 3): их проверяй `leaks --atExit -- ./программа`. Забыл `free` — увидишь строку вида `1 leak for 416 total leaked bytes` (так выходит для 100 `int`: 400 байт, округлённые менеджером памяти до 416); не забыл — `0 leaks for 0 total leaked bytes`.

Что мы получили

Теперь ты умеешь главные «однопроходные» приёмы над массивом: накопить сумму, удержать рекорд (мин/макс) с правильным стартом от `a[0]`, переставить элементы на месте двумя индексами навстречу и аккуратно сдвинуть массив, не вылезая за его границу, — а тремя разворотами сдвинуть и на любое `k`, хоть больше длины, хоть отрицательное, за `n` обменов вместо

`k · n` присваиваний. Это не просто упражнения — накопление и «бегунок навстречу» лежат в основе сортировок и поиска, которыми мы займёмся дальше.

И три привычки, которые теперь стоит носить с собой: проверять **каждый** `scanf`, а не только первый; отдельно думать про пустой массив и массив из одного элемента; и при любом сомнении в границах пересобирать программу с `-fsanitize=address,undefined`. Наивный сдвиг из части 3 напечатал абсолютно правильный ответ и всё равно был сломан — глазами такое не ловится.

В следующей главе возьмёмся за **поиск**: как найти нужный элемент в массиве. Сначала линейный поиск (тот же проход циклом, что и здесь), потом — двоичный, который находит число в отсортированном массиве за считанные шаги даже среди миллионов элементов.

Итоговые программы главы — `code/06-minmax.c`, `code/06-reverse.c` и `code/06-count-rotate.c`; `rotate-k.c` целиком напечатан в части 3.

Глава 7. Поиск: линейный и бинарный

Самая частая работа с массивом — что-то в нём **найти**. Есть ли число 15 в списке? На каком оно месте? В этой главе мы решим эту задачу двумя способами. Сначала в лоб — пройдём весь массив подряд (линейный поиск). Потом — хитро, вдвое сужая зону поиска на каждом шаге (бинарный поиск). Заодно увидим, почему второй способ на больших данных в тысячи раз быстрее, и какую цену за это приходится платить.

Что мы сделаем

Напишем две функции поиска. Обе получают массив, его длину и искомое число, а возвращают **индекс** найденного элемента или `-1`, если его нет. Линейный поиск работает на любом массиве. Бинарный — только на **отсортированном**, зато молниеносно. К концу главы ты будешь понимать, когда какой брать и почему.

Разбираем задачу руками

Представь список номеров шкафчиков, выданных на руки:

```
индекс:  0   1   2   3   4   5   6
число:  17   4  42   8  15  23  16
```

Вопрос: на каком месте стоит число 15? Глазами ты просто пробегаешь слева направо: 17 — не то, 4 — не то, 42 — не то, 8 — не то, 15 — есть! Это позиция (индекс) 4. Именно так и работает линейный поиск: проверяй элементы по очереди, пока не наткнёшься на нужный.

А если числа нет — например, ищем 99? Тогда придётся дойти до самого конца, проверить все семь штук и только тогда сказать «нету». Запомни этот момент: когда элемента нет, линейный поиск всегда проверяет **весь** массив.

Шаг 1. Наивный линейный поиск прямо в main

Сначала без функции, чтобы увидеть саму идею. Пройдём массив циклом и, как только встретим нужное число, напечатаем его позицию.

```c
#include <stdio.h>

int main(void)
{
    int a[] = {17, 4, 42, 8, 15, 23, 16};
    int n = (int)(sizeof a / sizeof a[0]);
    int target = 15;

    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            printf("Найдено на позиции %d\n", i);
        }
    }
    return 0;
}
```

Здесь почти всё знакомо, но есть одна новая полезная конструкция.

- `int a[] = {17, 4, ...};` — массив без явного размера в скобках. Компилятор сам посчитает, что элементов семь (см. главу 3).
- `sizeof a / sizeof a[0]` — так узнают **длину массива**. `sizeof a` — размер всего массива в байтах (у нас `int` занимает 4 байта, значит, 7 чисел по 4 байта = 28). `sizeof a[0]` — размер одного элемента (4 байта). Поделили — получили **число элементов**: 28 / 4 = 7. Это надёжнее, чем писать 7 руками: поменяешь список — длина пересчитается сама.
- `(int)(...)` — `sizeof` выдаёт значение специального беззнакового типа `size_t`. Чтобы хранить длину в обычном `int` без предупреждений компилятора, мы приводим результат к `int` (приведение типа из главы 3).
- `target` — англ. «цель», искомое число.
- Внутри цикла сравниваем каждый `a[i]` с `target` через `==` и при совпадении печатаем индекс.

Собираем и запускаем:

```
Найдено на позиции 4
```

Работает. Но у этой наивной версии есть три проблемы, и сейчас мы их вытащим на свет.

Шаг 2. Что не так с наивной версией

Проблема 1: цикл не останавливается. Мы нашли число на позиции 4, но цикл всё равно добегает до конца массива. На семи элементах это пустяк, а на миллионе — впустую потраченное время. Нашёл — выходи.

Проблема 2: нет ответа «не найдено». Поищем число 99 — программа просто промолчит, ничего не напечатает. Это плохо: «ничего не вывели» и «не нашли» — разные вещи, но выглядят одинаково.

Проблема 3: поиск зашит в `main`. Захочешь искать в другом массиве — придётся копировать весь цикл. Логику поиска надо вынести в **функцию**, чтобы вызывать её сколько угодно раз.

Чинить будем все три разом: завернём поиск в функцию, которая **возвращает** индекс найденного элемента, а если не нашла — возвращает `-1`. Почему именно `-1`? Потому что настоящий индекс всегда `0` или больше. Значит, `-1` не спутать с настоящей позицией — это удобный знак «такого нет».

Шаг 3. Линейный поиск как функция

```c
int linear_search(const int a[], int n, int target)
{
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;       /* нашли — сразу отдаём индекс */
        }
    }
    return -1;              /* дошли до конца и не встретили target */
}
```

Разбор сигнатуры `int linear_search(const int a[], int n, int target)`:

- первый `int` — тип возвращаемого значения. Функция вернёт индекс (целое число) или `-1`, тоже целое.
- `linear_search` — имя, придумали сами. От англ. *linear* — «линейный», то есть идём по прямой, элемент за элементом.
- `const int a[]` — параметр-массив. Когда передаёшь массив в функцию, туда уходит не копия элементов, а **адрес его начала** (помнишь из главы 3: имя массива превращается в адрес). Запись `int a[]` и означает «сюда придёт массив целых». А слово **`const`** — обещание: «внутри функции я этот массив **менять не буду**, только читаю». Это новое для нас. `const` (от *constant* — постоянный) защищает данные: если ты случайно напишешь `a[i] =`

5, компилятор остановит тебя с ошибкой. Для поиска это в самый раз — искать значит читать, а не портить.

- `int n` — сколько элементов в массиве. Массив сам по себе **не знает** свою длину (внутри функции трюк с `sizeof` уже не сработает — там `a` это адрес, а не весь массив), поэтому длину передают отдельным параметром. Это стандартный приём в Си.
- `int target` — что ищем.

Теперь самое важное — **`return` внутри цикла**:

- `return i;` — как только нашли совпадение, функция немедленно заканчивается и **отдаёт наружу** `i`. Цикл при этом обрывается на месте — `return` выходит сразу из всей функции, а не только из цикла. Это решает проблему 1: лишних шагов нет.
- `return -1;` — эта строка стоит **после** цикла. До неё дойдёт выполнение, только если цикл отработал целиком и ни разу не сделал `return i`. То есть мы проверили все элементы и не нашли. Значит — `-1`. Это решает проблему 2.

Полная программа: линейный поиск

Файл `code/07-linear.c`:

```c
#include <stdio.h>

/* Линейный поиск: ищем значение target в первых n элементах массива a.
   Возвращаем индекс первого совпадения или -1, если ничего не нашли. */
int linear_search(const int a[], int n, int target)
{
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;       /* нашли — сразу отдаём индекс */
        }
    }
    return -1;              /* дошли до конца и не встретили target */
}

int main(void)
{
    int a[] = {17, 4, 42, 8, 15, 23, 16};
    int n = (int)(sizeof a / sizeof a[0]);

    int target = 15;
```

```
    int index = linear_search(a, n, target);

    if (index != -1) {
        printf("Число %d найдено на позиции %d\n", target, index);
    } else {
        printf("Число %d в массиве не найдено\n", target);
    }

    int missing = 99;
    int index2 = linear_search(a, n, missing);
    if (index2 != -1) {
        printf("Число %d найдено на позиции %d\n", missing, index2);
    } else {
        printf("Число %d в массиве не найдено\n", missing);
    }

    return 0;
}
```

В `main` мы вызываем функцию и **сохраняем** её ответ в переменную `index`. Дальше одной проверкой `if (index != -1)` решаем, что печатать: позицию или «не найдено». Так результат функции используется по-взрослому — сначала получили число, потом разобрались, что оно значит.

Запуск:

```
Число 15 найдено на позиции 4
Число 99 в массиве не найдено
```

Оба случая обработаны: и когда число есть, и когда его нет.

Линейный поиск медленный — и вот почему

Линейный поиск прост и работает на **любом** массиве, хоть как перемешанном. Но у него есть слабое место. Прикинем, сколько сравнений `a[i] == target` он делает. Число это зависит от того, где лежит искомое, поэтому про скорость алгоритма говорят тремя словами:

- **лучший случай** — самый удачный вход. Искомое стоит первым, хватает одного сравнения;
- **худший случай** — самый неудачный вход. Искомого нет или оно стоит последним, и придётся проверить **все** `n` элементов;

- **средний случай** — сколько выходит в среднем, если искомое с равным шансом может оказаться на любом из `n` мест. На первом месте — одно сравнение, на втором — два, …, на последнем — `n`. Среднее `(1 + 2 + ... + n) / n = (n + 1) / 2`, то есть примерно половина массива. Для наших семи чисел: `(1 + 2 + ... + 7) / 7 = 28 / 7 = 4`.

Лучший случай почти ничего не говорит: повезти может любому алгоритму. Обычно смотрят на худший — это гарантия «хуже не будет»:

- массив из 10 элементов — до 10 проверок;
- из 1000 — до 1000 проверок;
- из 1 000 000 — до миллиона проверок.

Сколько элементов — столько и работы. Растёт массив в тысячу раз — растёт и время поиска в тысячу раз (и в худшем, и в среднем случае). Для небольших списков это незаметно. Но представь телефонный справочник на миллионы записей, где ищут постоянно — перебирать всё подряд каждый раз слишком расточительно.

Хорошая новость: если массив **отсортирован**, искать можно несравнимо быстрее. Этим и займёмся.

Шаг 4. Идея бинарного поиска — руками

Ты наверняка так искал слово в бумажном словаре. Не листаешь с первой страницы подряд! Открываешь примерно посередине, смотришь, в какую половину попадает твоё слово, и **отбрасываешь вторую половину целиком**. Потом повторяешь с оставшейся половиной. Каждый раз книга «худеет» вдвое — и нужная страница находится за считанные шаги.

Это и есть **бинарный поиск** (binary — «двоичный», «на две части»). Но у него жёсткое условие: он работает **только на отсортированном массиве**. Иначе «отбросить половину» нельзя — без порядка непонятно, в какой стороне искать.

Возьмём отсортированный по возрастанию массив и поищем в нём число 7:

```
индекс:  0   1   2   3   4   5
число:   1   3   5   7   9  11
```

Заведём две границы зоны поиска: `low` (левая, начало) и `high` (правая, конец). В начале зона — весь массив: `low = 0`, `high = 5`.

Шаг A. Середина: `mid = (0 + 5) / 2 = 2`. В середине стоит `a[2] = 5`. Это **меньше** семёрки. Раз массив отсортирован, всё, что левее середины, тоже меньше 7 — значит, левая половина и сама середина нам не нужны. Сдвигаем левую границу за середину: `low = mid + 1 = 3`. Зона сузилась до индексов 3..5.

Шаг B. Середина новой зоны: `mid = (3 + 5) / 2 = 4`. Там `a[4] = 9`. Это **больше** семёрки. Значит, всё, что правее (и сама девятка), нам не подходит. Сдвигаем правую границу: `high = mid - 1 = 3`. Зона — только индекс 3.

Шаг C. `mid = (3 + 3) / 2 = 3`. Там `a[3] = 7`. **Совпало!** Ответ — позиция 3.

Три шага вместо потенциальных шести. На маленьком массиве выигрыш скромный, но он растёт стремительно с размером — скоро увидим, насколько.

Шаг 5. Почему `mid = low + (high - low) / 2`, а не `(low + high) / 2`

Казалось бы, середину проще считать как `(low + high) / 2` — и математически это то же самое. Но в реальном коде есть ловушка.

`low` и `high` — переменные типа `int`. У `int` есть **потолок**: 2 147 483 647 (об этом упоминали в главе 2). Если обе границы заехали близко к потолку, их **сумма** `low + high` его перепрыгнет — это **переполнение** (overflow). В Си переполнение знакового целого — не «число просто заворачивается по кругу», а **неопределённое поведение**: стандарт не обещает ничего, компилятор вправе выдать любой результат. На практике обычно получается отрицательное число, и `mid` указывает в никуда.

Не будем верить на слово — посчитаем середину обоими способами на числах, близких к потолку. Файл `overflow.c`:

```c
#include <stdio.h>

int main(void)
{
    int low  = 1500000000;
    int high = 2000000000;

    int safe  = low + (high - low) / 2;   /* так правильно */
    int naive = (low + high) / 2;         /* а вот тут беда */

    printf("low + (high - low) / 2 = %d\n", safe);
    printf("(low + high) / 2       = %d\n", naive);
    return 0;
}
```

Собираем обычным образом и запускаем:

```
cc -std=c11 -Wall -Wextra -O2 overflow.c -o overflow
./overflow
low + (high - low) / 2 = 1750000000
(low + high) / 2       = -397483648
```

Правильная середина — 1 750 000 000. А «наивная» формула выдала **минус** 397 миллионов. Индексом такое число быть не может: `a[-397483648]` — это обращение к чужой памяти и почти наверняка падение программы.

Компилятор с обычными флагами промолчал — ни одного предупреждения. Зато не промолчит санитайзер неопределённого поведения, знакомый по главе 4. Пересобираем с `-fsanitize=undefined`:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=undefined overflow.c -o ovf-san
./ovf-san
overflow.c:9:22: runtime error: signed integer overflow: 1500000000 +
2000000000 cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior overflow.c:9:22
low + (high - low) / 2 = 1750000000
(low + high) / 2       = -397483648
```

(Первая строка отчёта на экране длинная, здесь она перенесена.) Программа сама показала пальцем: строка 9, столбец 22 — там, где стоит `+` в `(low + high)`. «Signed integer overflow: сумма 1500000000 + 2000000000 не представима в типе int». Строка `SUMMARY` — итог: сколько и где нашлось неопределённого поведения. Отчёт санитайзер печатает в момент ошибки, а программа после этого работает дальше, поэтому оба наших `printf` идут следом.

А теперь про масштаб: **когда это случается в жизни?** Переполнение наступает, только если `low + high` превысит 2 147 483 647. Обе границы не больше `n - 1`, значит, их сумма не больше `(n - 1) + (n - 1) = 2n - 2`. Чтобы она перевалила за 2 147 483 647, нужно `n` не меньше 1 073 741 825 (два в тридцатой степени плюс один) — чуть больше миллиарда элементов. Массив `int` такого размера занимает около **четырёх гигабайт** памяти. На учебных примерах ты этого не увидишь никогда.

Так зачем возиться? Потому что такие массивы бывают — в базах данных, в поисковых индексах, в обработке логов. Ровно на этом когда-то обожглись в стандартной библиотеке Java: её `Arrays.binarySearch` считала середину через `(low + high) / 2`, ошибку заметили только в 2006 году, спустя годы после выхода. Безопасная форма не стоит ни строчки лишнего кода, а привычка однажды спасёт.

Запись `low + (high - low) / 2` даёт **то же самое** значение середины, но устроена безопасно:

- `high - low` — это ширина зоны. Внутри цикла обе границы лежат между `0` и `n - 1`, и `low <= high`, поэтому разность не бывает меньше нуля и больше `n - 1`, переполниться ей негде;
- делим её пополам — `(high - low) / 2`, это половина ширины;
- прибавляем к `low` — получаем точку ровно посередине между границами.

Сумма двух больших чисел нигде не возникает, потолок не пробивается. Привыкай писать середину именно так — это стандартная защита, её ждут от тебя в любом нормальном коде.

Полная программа: бинарный поиск

Файл `code/07-binary.c`:

```c
#include <stdio.h>

/* Бинарный поиск. ВАЖНО: массив a ДОЛЖЕН быть отсортирован по
   возрастанию. Ищем target, возвращаем его индекс или -1.
   Если target встречается несколько раз — вернётся индекс одного
   из совпадений, не обязательно первого. */
int binary_search(const int a[], int n, int target)
{
    int low = 0;            /* левая граница диапазона поиска */
    int high = n - 1;       /* правая граница диапазона поиска */

    while (low <= high) {
        int mid = low + (high - low) / 2;   /* середина без переполнения */

        if (a[mid] == target) {
            return mid;                     /* попали точно в середину */
        } else if (a[mid] < target) {
            low = mid + 1;                  /* target правее середины */
        } else {
            high = mid - 1;                 /* target левее середины */
        }
    }
    return -1;                              /* диапазон схлопнулся — нет */
}

int main(void)
{
```

```
    /* Массив уже отсортирован по возрастанию. */
    int a[] = {1, 3, 5, 7, 9, 11};
    int n = (int)(sizeof a / sizeof a[0]);

    int target = 7;
    int index = binary_search(a, n, target);

    if (index != -1) {
        printf("Число %d найдено на позиции %d\n", target, index);
    } else {
        printf("Число %d в массиве не найдено\n", target);
    }

    int missing = 8;
    int index2 = binary_search(a, n, missing);
    if (index2 != -1) {
        printf("Число %d найдено на позиции %d\n", missing, index2);
    } else {
        printf("Число %d в массиве не найдено\n", missing);
    }

    return 0;
}
```

Разбор тела функции:

- `int low = 0;` и `int high = n - 1;` — границы зоны поиска. В начале она охватывает весь массив: от первого индекса `0` до последнего `n - 1`. (Последний индекс именно `n - 1`, а не `n` — нумерация с нуля, см. главу 3.)
- `while (low <= high)` — повторяем, **пока зона не пуста**. Условие `low <= high` означает «в зоне ещё есть хотя бы один элемент». Как только границы перехлестнутся (`low` станет больше `high`), искать негде — выходим. Знак `<=`, а не `<`, важен: при `low == high` в зоне остался ровно один элемент, и его тоже надо проверить.
- `int mid = low + (high - low) / 2;` — середина зоны, посчитанная безопасно (только что разобрали почему).
- `if (a[mid] == target) return mid;` — повезло, в середине ровно то, что ищем. Отдаём индекс `mid`.
- `else if (a[mid] < target)` — в середине число **меньше** искомого. Раз массив отсортирован по возрастанию, искомое (если оно есть) лежит **правее**. Отрезаем левую половину:

`low = mid + 1`. Прибавляем 1, потому что саму середину уже проверили — повторять её незачем.

- `else` — остаётся случай «в середине число **больше** искомого». Тогда искомое лежит **левее**. Отрезаем правую половину: `high = mid - 1`.
- `return -1;` — если цикл завершился сам (границы сошлись), числа нет.

Обрати внимание: `mid + 1` и `mid - 1` — не прихоть. Если бы мы писали `low = mid` (без +1), то в зоне из двух элементов середина не двигалась бы, и цикл крутился бы **вечно**. Сдвиг через середину гарантирует, что зона на каждом шаге реально уменьшается.

Запуск:

```
Число 7 найдено на позиции 3
Число 8 в массиве не найдено
```

Семёрку нашли на позиции 3 — ровно как считали руками. А восьмёрки в массиве нет (там только нечётные), и функция честно вернула `-1`.

Как это работает: трассировка по шагам

Пройдём `binary_search(a, 6, 7)` по строчкам, выписывая значения границ. Массив: `1 3 5 7 9 11`, индексы `0..5`.

```
Старт:   low=0, high=5
Шаг 1:   mid = 0 + (5-0)/2 = 2   a[2]=5   5 < 7  -> low = 3
Шаг 2:   mid = 3 + (5-3)/2 = 4   a[4]=9   9 > 7  -> high = 3
Шаг 3:   mid = 3 + (3-3)/2 = 3   a[3]=7   7 == 7 -> return 3
```

Зона поиска ужималась так: сначала весь массив (6 элементов), потом индексы 3..5 (3 элемента), потом только индекс 3 (1 элемент) — и попадание. Каждый шаг выбрасывал примерно половину оставшихся вариантов.

А теперь трассировка для отсутствующего числа `8`:

```
Старт:   low=0, high=5
Шаг 1:   mid = 2   a[2]=5   5 < 8  -> low = 3
Шаг 2:   mid = 4   a[4]=9   9 > 8  -> high = 3
Шаг 3:   mid = 3   a[3]=7   7 < 8  -> low = 4
Теперь low=4, high=3  ->  low > high  ->  цикл кончился  ->  return -1
```

Границы перехлестнулись (`low` обогнал `high`) — это и есть сигнал «зона пуста, числа нет».

Краевые случаи: проверяем функцию на прочность

Алгоритм на красивом примере работает почти у всех. Ломается он на **краях**: пустой массив, массив из одного элемента, искомое в самом начале или в самом конце, повторяющиеся числа. Именно там живут баги, и именно это спрашивают на собеседовании. Прогоним нашу функцию через все такие случаи разом.

Файл `code/07-binary-edge.c` — та же `binary_search`, но `main` в нём устраивает ей допрос:

```
int main(void)
{
    int empty[1] = {0};        /* массив есть, но искать будем в нуле */
    int one[] = {5};
    int a[] = {1, 3, 5, 7, 9, 11};
    int dup[] = {2, 4, 4, 4, 4, 7, 9};

    printf("пустой массив, ищем 5:        %d\n",
           binary_search(empty, 0, 5));
    printf("один элемент {5}, ищем 5:     %d\n",
           binary_search(one, 1, 5));
    printf("один элемент {5}, ищем 4:     %d\n",
           binary_search(one, 1, 4));
    printf("1 3 5 7 9 11, ищем 1:         %d\n",
           binary_search(a, 6, 1));
    printf("1 3 5 7 9 11, ищем 11:        %d\n",
           binary_search(a, 6, 11));
    printf("1 3 5 7 9 11, ищем 0:         %d\n",
           binary_search(a, 6, 0));
    printf("1 3 5 7 9 11, ищем 12:        %d\n",
           binary_search(a, 6, 12));
    printf("2 4 4 4 4 7 9, ищем 4:        %d\n",
           binary_search(dup, 7, 4));

    /* А теперь запретный приём: неотсортированный массив.
       Каждое число в нём ЕСТЬ, но бинарный поиск это не спасёт. */
    int mess[] = {17, 4, 42, 8, 15, 23, 16};
    int m = (int)(sizeof mess / sizeof mess[0]);

    printf("\nнеотсортированный 17 4 42 8 15 23 16:\n");
    for (int i = 0; i < m; i++) {
```

```
        printf("  ищем %2d (лежит на %d) -> %d\n",
               mess[i], i, binary_search(mess, m, mess[i]));
    }

    return 0;
}
```

Пара слов про `empty`. Массив нулевой длины в Си объявить нельзя, поэтому мы завели массив из одного элемента, а функции передали `n = 0` — «элементов ноль». Функция обязана честно ответить «не нашла», ни разу не заглянув в память.

Запускаем:

```
cc -std=c11 -Wall -Wextra -O2 07-binary-edge.c -o 07-binary-edge
./07-binary-edge
пустой массив, ищем 5:        -1
один элемент {5}, ищем 5:     0
один элемент {5}, ищем 4:     -1
1 3 5 7 9 11, ищем 1:         0
1 3 5 7 9 11, ищем 11:        5
1 3 5 7 9 11, ищем 0:         -1
1 3 5 7 9 11, ищем 12:        -1
2 4 4 4 4 7 9, ищем 4:        3

неотсортированный 17 4 42 8 15 23 16:
  ищем 17 (лежит на 0) -> -1
  ищем  4 (лежит на 1) -> 1
  ищем 42 (лежит на 2) -> -1
  ищем  8 (лежит на 3) -> 3
  ищем 15 (лежит на 4) -> 4
  ищем 23 (лежит на 5) -> 5
  ищем 16 (лежит на 6) -> -1
```

Разберём каждую строчку.

- **Пустой массив.** `n = 0`, значит, `high = n - 1 = -1`. Условие `low <= high` — это `0 <= -1`, ложь. Цикл не выполняется **ни разу**, функция сразу возвращает `-1`. Никакого обращения к `a[mid]` не происходит — именно поэтому важно, что `high` считается как `n - 1`, а проверка стоит **до** тела цикла.
- **Один элемент.** `low = high = 0`, условие `0 <= 0` истинно, `mid = 0 + (0 - 0) / 2 = 0`. Проверяем единственный элемент. Если это наше число — вернули `0`; если нет — двинули границу,

зона схлопнулась, вернули `-1`. Тот самый случай, ради которого в условии стоит `<=`, а не `<`.

- **Первый и последний элементы** находятся нормально: `0` и `5`. Здесь часто ошибаются в границах — заводят `high = n` вместо `n - 1` и выходят за массив.
- **Число вне диапазона** (`0` меньше всех, `12` больше всех) даёт `-1`. Границы сходятся и перехлёстываются, лишних обращений к памяти нет.
- **Повторы.** Вот здесь надо остановиться. В массиве `2 4 4 4 4 7 9` четвёрка лежит на позициях 1, 2, 3 и 4. Функция вернула `3`. Не первую и не последнюю — **какую попалось**, ту, куда угодила середина.

Это не баг, а свойство алгоритма: бинарный поиск обещает найти **одно из совпадений**, и не обещает, какое именно. Пока ты просто спрашиваешь «есть или нет» — разницы никакой. Но если тебе нужна **первая** позиция (или последняя, или сколько всего повторов) — обычный бинарный поиск не подходит, его надо дорабатывать. Обещание функции стоит записать прямо в комментарии над ней, чтобы через полгода не гадать.

- **Неотсортированный массив.** Последний блок — тот самый массив из начала главы, `17 4 42 8 15 23 16`. Все семь чисел в нём есть. Бинарный поиск нашёл только четыре: 17, 42 и 16 он объявил отсутствующими. Проследи за семнадцаткой: `low = 0`, `high = 6`, `mid = 3`, там `8`. Восьмёрка меньше семнадцати, значит (по логике алгоритма) искать надо правее — `low = 4`. А семнадцать лежит на позиции 0, слева. Всё, потеряли. И заметь: программа не упала, не заругалась, а спокойно ответила `-1`. Это самый опасный сорт ошибки — тихий.

Чтобы убедиться, что за границы массива мы нигде не залезли, соберём эту же программу с двумя санитайзерами: `undefined` ловит запрещённую арифметику, а `address` — обращения к чужой памяти:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address,undefined \
   07-binary-edge.c -o 07-binary-edge-san
./07-binary-edge-san
```

Вывод получается ровно тот же, ни одной жалобы. Значит, чтения из чужой памяти не было — в том числе на пустом массиве.

Во сколько раз бинарный быстрее

Линейный поиск в худшем случае делает `n` сравнений. Бинарный на каждом шаге (одном проходе цикла `while`) **делит** зону пополам. Сколько шагов нужно в худшем случае? Столько, сколько раз зону можно поделить пополам, пока в ней не останется один элемент, — и ещё один шаг, чтобы проверить этот последний элемент.

«Сколько раз число делится пополам» — это **двоичный логарифм**. Пишут его $\log_2 n$ (в коде и таблицах — `log2 n`). Определение такое: $\log_2 n$ — это степень, в которую надо возвести 2, чтобы получить n. Два в третьей степени — восемь ($2 \cdot 2 \cdot 2 = 8$), значит, $\log_2 8 = 3$: восьмёрку можно поделить пополам ровно три раза ($8 \to 4 \to 2 \to 1$). Два в десятой — 1024, значит, $\log_2 1024 = 10$. Если n не степень двойки, логарифм выходит дробным: $\log_2 1000 \approx 9.97$. Нам дробная часть не нужна — половинку элемента не проверишь, — поэтому округляем вниз. Округление вниз записывают скобками $\lfloor\ \rfloor$: $\lfloor 9.97 \rfloor = 9$. Итого для нашей функции:

$$\text{шагов в худшем случае} = \lfloor \log_2 n \rfloor + 1$$

Проверим на массиве из шести, где мы искали руками: $\log_2 6 \approx 2.58$, вниз — 2, плюс один — 3. Ровно три шага мы и сделали, когда искали 7 и 8.

А теперь почувствуй разницу на больших числах:

размер массива n	линейный (до n)	бинарный ($\lfloor \log_2 n \rfloor + 1$)
10	10	4
100	100	7
1 000	1 000	10
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

Числа в правом столбце не на глаз: я перебрал программой все возможные искомые (и те, что есть в массиве, и те, что попадают между ними) и взял самый долгий поиск. Для 10, 100, 1000 и миллиона вышло ровно 4, 7, 10 и 20 шагов, а миллиард — просто по формуле.

В массиве из тысячи элементов линейный поиск может сделать до 1000 проверок, а бинарный — не больше 10 шагов. Это в сотню раз меньше работы. А на миллиарде элементов разница уже в десятки миллионов раз: 30 шагов против миллиарда. Вот почему отсортированные данные так ценят: по ним можно искать почти мгновенно.

Одна оговорка про слово «шаг». За один шаг наша функция сравнивает `a[mid]` с `target` один раз (если попала) или два (==, потом <). Так что сравнений у бинарного поиска до $2 \cdot (\lfloor \log_2 n \rfloor + 1)$ — для тысячи элементов до 20. Против тысячи у линейного это всё равно ничто.

Удвой массив — линейному добавится столько же работы, сколько было. Бинарному добавится всего **один** шаг: $\log_2 (2n) = \log_2 n + 1$. В этом вся магия деления пополам.

Проверим на секундомере

Таблица — это теория. Замерим настоящим временем: возьмём массив на миллион отсортированных чисел и по 10 000 раз поищем в нём последний элемент (худший случай для линей-

ного поиска). Секундомер — `clock()` из `<time.h>`; таким же замером в главе 5 мы сравнивали две проверки на простоту, но саму функцию тогда не разбирали. Пора:

```
int n = 1000000;
int *a = malloc((size_t)n * sizeof(int));
for (int i = 0; i < n; i++) {
    a[i] = i * 2;                    /* уже отсортирован */
}
int queries = 10000;
int last = (n - 1) * 2;
long long sink = 0;

clock_t t0 = clock();
for (int q = 0; q < queries; q++) {
    sink += linear_search(a, n, last);
}
double lin = (double)(clock() - t0) / CLOCKS_PER_SEC;

t0 = clock();
for (int q = 0; q < queries; q++) {
    sink += binary_search(a, n, last);
}
double bin = (double)(clock() - t0) / CLOCKS_PER_SEC;
```

Разбор новых мест:

- `clock()` возвращает, сколько процессорного времени истратила программа с начала работы, в условных единицах — «тиках» часов. Тип результата — `clock_t`. Само число ни о чём не говорит: сколько тиков в секунде, стандарт Си оставляет на усмотрение системы, и лежит это число в константе `CLOCKS_PER_SEC` (на macOS и Linux — миллион). Поэтому разницу делим на неё и получаем секунды.
- `(double)` перед разностью обязательно: обе величины целые, и без приведения деление ушло бы нацело (глава 2) — дробная часть пропала бы. Три с лишним секунды линейного поиска превратились бы в ровно 3, а тысячные доли секунды у бинарного — в 0.
- `sink` — «копилка»: складываем туда все найденные индексы. Зачем? Если результат поиска никуда не девать, компилятор с `-O2` вправе заметить, что вызов бесполезен, и выбросить весь цикл. Тогда мы замерим пустоту и получим восхитительный, но враньём добытый ноль. Копилка делает результат «нужным».

Полная программа лежит в `code/07-bench.c`. Она меряет ещё и третий случай — бинарный поиск, который каждый раз ищет **другое** число. Вот свежий запуск на моей машине:

```
cc -std=c11 -Wall -Wextra -O2 07-bench.c -o 07-bench
./07-bench
массив: 1000000 элементов, по 10000 поисков
линейный, всегда последний:   3.3934 с  ( 339.34 мкс на поиск)
бинарный, всегда последний:   0.0002 с  (   0.02 мкс на поиск)
бинарный, каждый раз новый:   0.0012 с  (   0.12 мкс на поиск)
линейный медленнее бинарного примерно в 2807 раз
(контрольная сумма индексов: 24990385000)
```

Три с лишним секунды против одной тысячной. На один поиск: 339 микросекунд у линейного и 0.12 микросекунды у бинарного — разница почти в три тысячи раз. (Три запуска подряд на одной машине дали 2441, 2766 и 2807 раз: секундомер немного плавает, порядок — нет.)

Сверим с теорией. Линейный поиск последнего элемента делает миллион сравнений, бинарный — не больше 20 шагов. Отношение — 50 000, а секундомер показал около 3000. Расхождение не ошибка: шаг бинарного поиска дороже одного сравнения линейного (посчитать середину, прыгнуть в другое место памяти), а линейный идёт по памяти подряд — это самый удобный для процессора порядок. Теория говорит, **как растёт** работа, а не сколько секунд займёт каждый шаг.

Почему две строки про бинарный? Когда мы 10 000 раз подряд ищем **одно и то же** число, поиск каждый раз идёт по одному и тому же маршруту: процессор держит нужные ячейки под рукой и заранее угадывает все повороты. Выходит 0.02 микросекунды — красиво, но нечестно: так в жизни не бывает. Когда число каждый раз новое, маршрут новый, данные приходится тянуть из памяти — и получается 0.12 микросекунды. В шесть раз хуже и всё равно в тысячи раз быстрее линейного. Мерить надо на том, что похоже на настоящую нагрузку, иначе замерльстит.

Абсолютные секунды у тебя будут свои — они зависят от машины. Но порядок разницы воспроизведётся: тысячи раз.

Обрати внимание: сам массив мы отсортированным **сделали** — просто заполнили по возрастанию. В реальной задаче за сортировку ещё нужно заплатить, и об этой цене — следующая глава.

Проверяем

Собери и запусти обе программы:

```
cc -std=c11 -Wall -Wextra -O2 07-linear.c -o 07-linear
./07-linear
```

```
Число 15 найдено на позиции 4
Число 99 в массиве не найдено

cc -std=c11 -Wall -Wextra -O2 07-binary.c -o 07-binary
./07-binary
Число 7 найдено на позиции 3
Число 8 в массиве не найдено
```

Ни одного предупреждения от компилятора, вывод совпал с ожидаемым — значит, обе функции работают.

Всего в главе четыре программы, и все собираются одной и той же командой:

- `07-linear.c` — линейный поиск;
- `07-binary.c` — бинарный поиск;
- `07-binary-edge.c` — тот же бинарный поиск на краевых случаях;
- `07-bench.c` — замер скорости на миллионе элементов.

Первые три полезно ещё раз прогнать с санитайзерами (`-fsanitize=address,undefined`): если функция где-то читает за границей массива, ты узнаешь об этом сразу, а не через полгода.

Частые ошибки

Бинарный поиск по неотсортированному массиву. Ошибка номер один. Если массив не отсортирован, «отбросить половину» бессмысленно: мы только что видели, как на массиве `17 4 42 8 15 23 16` поиск не нашёл три числа из семи, хотя они там лежали. Функция **молча** даёт неверный ответ, не падает и не ругается. Прежде чем искать бинарно — убедись, что массив отсортирован.

Бесконечный цикл из-за неверного сдвига границ. Если написать `low = mid` вместо `low = mid + 1` (или `high = mid` вместо `high = mid - 1`), зона может перестать уменьшаться, и `while` закрутится навсегда. Всегда двигай границу **за** середину: `mid + 1` или `mid - 1`. Если программа зависла — Ctrl+C, и проверь эти две строки.

`<` вместо `<=` в условии цикла. С `while (low < high)` ты пропустишь случай, когда в зоне остался последний единственный элемент (`low == high`), и не найдёшь число, стоящее именно там. Нужно `while (low <= high)`.

Середина через `(low + high) / 2`. На учебных массивах работает безупречно — переполнение начинается только около миллиарда элементов. Но привычка важнее: пиши `low + (high - low) / 2` всегда, чтобы однажды не искать этот баг неделю.

`high = n` вместо `high = n - 1`. Последний индекс массива из `n` элементов — это `n - 1`. Напишешь `n` — и на первом же шаге функция прочитает `a[n]`, элемент за границей. Программа при этом может не упасть и вернуть правдоподобную чушь; санитайзер `-fsanitize=address` такое ловит сразу.

Ждать от поиска первое совпадение. Если чисел-дубликатов несколько, бинарный поиск вернёт **какое-то** из них, а не самое левое (мы это видели: в `2 4 4 4 4 7 9` для четвёрки вышел индекс 3). Нужна именно первая позиция — алгоритм придётся дописать.

Пустой массив забыли проверить. Хорошая новость: наша версия справляется сама. `high = n - 1 = -1`, условие `low <= high` сразу ложно, цикл не запускается, возвращается `-1`. Но это работает только потому, что проверка стоит **до** тела цикла. Переделаешь на `do { ... } while` — получишь чтение из `a[0]` в пустом массиве.

Перепутаны направления. Если массив отсортирован по возрастанию и `a[mid] < target`, цель **правее** — двигаем `low`. Перепутаешь `low` и `high` местами — поиск будет уходить не туда и ничего не найдёт.

Линейный поиск без выхода (`return` в цикле). Если забыть `return i` и просто печатать совпадения, цикл прогонит весь массив зря, а функция не сможет вернуть индекс. Нашёл — возвращай сразу.

Упражнения

1. Добавь в линейный поиск **счётчик сравнений**: пусть функция через указатель-параметр (вспомни `&` и `*` из главы 3) сообщает в `main`, сколько раз сравнила элементы. Поищи число, стоящее последним, и убедись, что счётчик равен длине массива. *Ответ:* на массиве из главы `17 4 42 8 15 23 16` поиск шестнадцати вернёт индекс 6, а счётчик покажет 7. Подсказка по сигнатуре: `int linear_search(const int a[], int n, int target, int *cmps)`, в начале `*cmps = 0;`, в цикле перед сравнением `(*cmps)++;`, в `main` — вызов с `&count`.

2. Сделай то же для бинарного поиска (считай шаги — проходы цикла `while`) и сравни счётчики на одном и том же отсортированном массиве из 15 элементов. Во сколько раз меньше шагов? *Ответ:* возьми `1 3 5 ... 29` и ищи последнее число, 29. Линейный сделает 15 сравнений, бинарный — 4 шага (середины 7, 11, 13, 14), почти вчетверо меньше. Хуже 4 шагов бинарный на 15 элементах не бывает: $\lfloor \log_2 15 \rfloor + 1 = 3 + 1 = 4$.

3. Напиши `int count_occurrences(const int a[], int n, int target)` — функцию, которая считает, **сколько раз** число встречается в массиве (линейным проходом, без `return` на первом совпадении). *Ответ:* на `2 4 4 4 4 7 9` для четвёрки — 4, для пятёрки — 0.

4. Измени бинарный поиск так, чтобы он искал в массиве, отсортированном **по убыванию** (например, `11 9 7 5 3 1`). Что нужно поменять в двух ветках со сдвигом границ? *Ответ:* только знак в проверке: при `a[mid] > target` искомое лежит **правее** (`low = mid + 1`), иначе — левее (`high = mid - 1`). На `11 9 7 5 3 1` поиск 7 даст 2, поиск 11 — 0, поиск 1 — 5, поиск 8 — `-1`.

5. Дай пользователю ввести искомое число с клавиатуры (`scanf`), а потом найди его бинарным поиском в захардкоженном отсортированном массиве. Не забудь проверить результат `scanf` (см. главу 6). *Ожидаемо:* на массиве `1 3 5 7 9 11` ввод `9` печатает позицию 4, ввод `4` — «не найдено», ввод `abc` — сообщение об ошибке ввода, а не поиск по мусору.

6. Подумай и проверь на бумаге: сколько шагов сделает бинарный поиск на массиве из 16 элементов в худшем случае? А из 17? Совпало ли с твоей догадкой про `log2`? *Ответ:* 5 и 5. $\lfloor \log_2 16 \rfloor + 1 = 4 + 1 = 5$ ровно, плюс последний шаг — 5. У 17 логарифм ≈ 4.09, вниз — снова 4, плюс один — 5. Шестой шаг понадобится только с 32 элементов.

7. **Первое совпадение.** Доработай бинарный поиск так, чтобы на массиве `2 4 4 4 4 7 9` он для четвёрки возвращал `1`, а не `3`. Подсказка: нашёл совпадение — не возвращайся сразу, а запомни индекс в отдельную переменную и продолжай искать **левее** (`high = mid - 1`). Когда зона схлопнется, в переменной останется самый левый найденный индекс. *Ответ:* для 4 — `1`, для 2 — `0`, для 5 — `-1`. Шагов по-прежнему не больше $\lfloor \log_2 n \rfloor + 1$: мы не перебираем повторы, а продолжаем делить зону пополам.

8. **Сломай нарочно.** Возьми `07-binary.c`, замени `high = n - 1` на `high = n`, а искомое число сделай **больше всех** в массиве (например, 12). Собери с `-fsanitize=address` и запусти. Санитайзер напечатает `stack-buffer-overflow` и покажет строку с `a[mid]` — это чтение элемента `a[6]`, которого нет. Потом верни как было. Заметь, что без санитайзера программа спокойно печатает `-1` и не подаёт виду.

Что мы получили

Теперь у тебя два инструмента поиска. **Линейный** — простой и всеядный: работает на любом массиве, но в худшем случае проверяет все `n` элементов. **Бинарный** — стремительный (не больше $\lfloor \log_2 n \rfloor + 1$ шагов вместо `n`), но требует **отсортированного** массива и аккуратной работы с границами `low`/`high`/`mid`. Ты увидел, как функция возвращает индекс или `-1`, почему середину считают через `low + (high - low) / 2`, и насколько бешеной становится разница в скорости на больших данных.

Заодно ты приобрёл две привычки, которые пригодятся до конца книги. Первая — **гонять алгоритм по краям**: пустой вход, один элемент, самое начало, самый конец, повторы, «нет такого». Красивый пример работает у всех, ломается всё на краях. Вторая — **не верить замерам**

и заявлениям на слово: собрать, запустить, посмотреть на настоящие секунды, включить санитайзер и дать программе самой рассказать, где она врёт.

И тут всплывает честный вопрос: бинарный поиск великолепен, но он держится на одном условии — массив должен быть **отсортирован**. А что, если данные пришли вперемешку? Значит, перед поиском их надо упорядочить. Как расставить элементы по возрастанию — большая и интересная тема. Ею мы и займёмся в следующей главе: разберём **сортировки**, после чего бинарный поиск станет применим к любым данным, а не только к заранее аккуратным спискам.

Глава 8. Сортировки: пузырёк, выбором, вставками

Отсортировать — значит расставить числа по порядку: от меньшего к большему. Звучит просто, но именно на сортировках удобно почувствовать, что такое **алгоритм**: одну и ту же задачу можно решить по-разному, и способы отличаются скоростью и идеей.

В этой главе мы напишем три классические сортировки **с нуля**: пузырёк, выбором и вставками. Для каждой сначала разложим всё руками на маленьком массиве `[5, 2, 4, 1]`, потом превратим в код, потом запустим. В конце поговорим, почему эти три — медленные на больших данных, какие есть быстрые (быстрая сортировка и слияние) и как пользоваться готовой `qsort` из стандартной библиотеки.

Что мы сделаем

К концу главы у тебя будут три рабочие программы-сортировки и понимание, чем они отличаются. Ты научишься:

- расставлять элементы массива по возрастанию;
- видеть, как массив меняется проход за проходом;
- оценивать, быстрый алгоритм или медленный;
- вызывать стандартную `qsort` со своей функцией сравнения.

Всё, что нужно, ты уже знаешь из глав 1–7: массивы, циклы, обмен двух переменных через временную (swap), функции. Новым будет только устройство самих алгоритмов.

Общая заготовка: печать массива

Во всех трёх программах нам надо смотреть на массив до и после. Чтобы не повторять цикл печати, вынесем его в маленькую функцию:

```c
void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}
```

Разбор:

- `void` — функция ничего не возвращает, она просто печатает.
- `const int a[]` — принимаем массив целых. `const`, как ты помнишь по главе 7, — обещание «массив только читаю, не меняю»; запись в него компилятор не пропустит. Печати менять нечего — значит, `const` здесь к месту.
- `int n` — сколько элементов печатать. Массив сам не знает свою длину, её всегда передают отдельным числом (см. главу 3).
- внутри — обычный перебор и `printf("%d ", a[i])` с пробелом после каждого числа, в конце один перевод строки.

Эту функцию мы вставим в каждую из трёх программ.

Часть 1. Сортировка пузырьком

Идея руками

Пузырёк сравнивает **соседей** и меняет их местами, если они стоят не по порядку. Идём слева направо по всему массиву. За один такой проход самый большой элемент «всплывает» в самый конец — как пузырёк воздуха поднимается в воде. Отсюда название.

Возьмём `[5, 2, 4, 1]` и сделаем **первый проход**. Сравниваем пары соседей слева направо:

```
[5 2 4 1]  5 > 2 ? да  -> меняем -> [2 5 4 1]
[2 5 4 1]  5 > 4 ? да  -> меняем -> [2 4 5 1]
[2 4 5 1]  5 > 1 ? да  -> меняем -> [2 4 1 5]
```

После первого прохода: `[2 4 1 5]`. Заметь — **5 уехала в самый конец** и больше двигаться не будет. Это и есть всплывший максимум.

Второй проход (последний элемент уже на месте, его не трогаем):

```
[2 4 1 5]  2 > 4 ? нет
[2 4 1 5]  4 > 1 ? да  -> меняем -> [2 1 4 5]
```

После второго прохода: `[2 1 4 5]`. Теперь на местах два хвостовых элемента — `4` и `5`.

Третий проход:

```
[2 1 4 5]  2 > 1 ? да  -> меняем -> [1 2 4 5]
```

Получили `[1 2 4 5]` — отсортировано. Каждый следующий проход можно делать чуть короче: хвост уже разобран.

Шаг 1. Наивная версия

Запишем идею в лоб: внешний цикл повторяет проходы, внутренний идёт по соседям и меняет их местами.

```
void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++) {
        for (int i = 0; i < n - 1; i++) {
            if (a[i] > a[i + 1]) {
                int t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
            }
        }
    }
}
```

Разбор:

- `int a[]` — здесь **без** `const`: эту функцию массив именно меняет.
- внешний `for (pass ...)` — счётчик проходов. Проходов хватает `n - 1`: за каждый проход как минимум один элемент встаёт на место, а последний оставшийся окажется правильным сам собой.
- внутренний `for (i ...)` сравнивает соседей `a[i]` и `a[i + 1]`. Условие `i < n - 1`, а не `i < n`: иначе `a[i + 1]` при `i = n - 1` залезет за границу массива (см. главу 3, выход за границы — частый баг).
- `if (a[i] > a[i + 1])` — соседи стоят «неправильно» (левый больше правого), надо поменять.
- три строки с `t` — это **swap** через временную переменную, ровно как в главе 6: запомнили левый в `t`, на его место положили правый, а правому отдали `t`.

Эта версия уже сортирует правильно. Но в ней есть слабое место.

Шаг 2. Проблема: лишняя работа

Внутренний цикл всегда доходит до конца (`i < n - 1`), хотя хвост массива после каждого прохода уже отсортирован. И ещё хуже: даже если массив **уже** упорядочен, мы всё равно сделаем все `n - 1` проходов впустую.

Дай этой версии почти отсортированный `[1, 2, 3, 5, 4]` — она переставит `5` и `4` за первый же проход, а потом сделает ещё кучу проходов, ничего не меняя. Обидно.

Шаг 3. Фикс: укорачиваем хвост и ставим флаг

Две доработки:

1. Внутренний цикл доводим только до `n - 1 - pass`: всё, что правее, уже всплыло на свои места, проверять там нечего.
2. Заводим **флаг** `swapped`. Перед проходом ставим его в 0. Если за проход был хоть один обмен — ставим 1. Если после прохода флаг так и остался 0, значит, все соседи уже стояли по порядку — массив отсортирован, и можно выйти досрочно через `break`.

```c
void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;
        for (int i = 0; i < n - 1 - pass; i++) {
            if (a[i] > a[i + 1]) {
                int t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
                swapped = 1;
            }
        }
        if (swapped == 0) {
            break;
        }
    }
}
```

Разбор добавленного:

- `int swapped = 0;` — флаг «были ли обмены». 0 — «нет», 1 — «да». Это обычное использование числа как «да/нет»: помнишь из главы 2, ложь это 0, истина это любое ненулевое.

- `i < n - 1 - pass` — на проходе номер `pass` последние `pass` элементов уже на местах, до них не идём.
- `swapped = 1;` — отмечаем, что обмен случился.
- `if (swapped == 0) break;` — за весь проход ничего не поменяли, значит, дальше делать нечего. `break` немедленно прекращает внешний цикл.

На уже отсортированном массиве теперь хватит **одного** прохода: он пройдёт без обменов, флаг останется 0, и мы выйдем.

Полная программа

`code/08-bubble.c`:

```c
#include <stdio.h>

/* Печать массива в одну строку — пригодится во всех трёх программах. */
void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Сортировка пузырьком по возрастанию.
   За каждый проход самый большой из оставшихся «всплывает» вправо.
   Оптимизация: если за проход не было ни одного обмена — массив
   уже отсортирован, выходим досрочно. */
void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;              /* были ли обмены в этом проходе */

        /* На каждом следующем проходе хвост уже отсортирован,
           поэтому не доходим до конца: -1 за i+1, -pass за хвост. */
        for (int i = 0; i < n - 1 - pass; i++) {
            if (a[i] > a[i + 1]) {    /* соседи стоят не по порядку */
                int t = a[i];         /* классический swap через temp */
                a[i] = a[i + 1];
                a[i + 1] = t;
```

```
                swapped = 1;
            }
        }

        if (swapped == 0) {            /* за проход ничего не меняли */
            break;                     /* значит, всё уже на местах */
        }
    }
}

int main(void)
{
    int a[] = {5, 2, 4, 1};
    int n = (int)(sizeof a / sizeof a[0]);

    printf("До:    ");
    print_array(a, n);

    bubble_sort(a, n);

    printf("После: ");
    print_array(a, n);
    return 0;
}
```

Про `int n = (int)(sizeof a / sizeof a[0]);` — это привычный приём из главы 3: `sizeof a` — размер всего массива в байтах, `sizeof a[0]` — размер одного элемента; их частное даёт число элементов. Работает только там, где `a` — настоящий массив, а не пришедший в функцию указатель.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-bubble.c -o bubble
$ ./bubble
До:    5 2 4 1
После: 1 2 4 5
```

Совпадает с тем, что мы получили руками.

Часть 2. Сортировка выбором

Идея руками

Здесь логика другая. Мысленно делим массив на две части: слева — уже **отсортированная** (поначалу пустая), справа — ещё нет. На каждом шаге находим **минимум** во всей правой части и ставим его в самое её начало, меняя местами с тем, что там стояло. Отсортированная часть растёт на один элемент за шаг.

`[5, 2, 4, 1]`, отсортированная часть отмечена `|`:

```
| 5 2 4 1   минимум справа = 1 (поз. 3), меняем с поз. 0
1 | 2 4 5   минимум справа = 2 (поз. 1), уже на месте
1 2 | 4 5   минимум справа = 4 (поз. 2), уже на месте
1 2 4 | 5   остался один элемент — он на месте
```

Готово: `[1 2 4 5]`. Обрати внимание: обменов делается мало (максимум по одному на шаг), но сравнений много — каждый раз заново просматриваем весь остаток.

Код

Наивной и «починенной» версий тут не будет — алгоритм сразу пишется понятно. Важно лишь не менять элемент сам с собой, если минимум и так на месте.

`code/08-selection.c`:

```c
#include <stdio.h>

void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Сортировка выбором по возрастанию.
   Делим массив на две части: слева — уже отсортированная,
   справа — ещё нет. На каждом шаге находим минимум в правой части
   и ставим его в начало этой части. */
void selection_sort(int a[], int n)
{
```

```
    for (int i = 0; i < n - 1; i++) {
        int min_pos = i;              /* пока считаем минимумом a[i] */

        /* ищем настоящий минимум среди a[i+1 .. n-1] */
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min_pos]) {
                min_pos = j;          /* запомнили позицию меньшего */
            }
        }

        if (min_pos != i) {           /* минимум не на своём месте */
            int t = a[i];             /* меняем его с началом части */
            a[i] = a[min_pos];
            a[min_pos] = t;
        }
    }
}

int main(void)
{
    int a[] = {5, 2, 4, 1};
    int n = (int)(sizeof a / sizeof a[0]);

    printf("До:    ");
    print_array(a, n);

    selection_sort(a, n);

    printf("После: ");
    print_array(a, n);
    return 0;
}
```

Разбор `selection_sort`:

- внешний `for (i ...)` — `i` это граница: позиция, куда сейчас поставим очередной минимум. Хватает до `n - 1`: последний элемент окажется правильным сам.
- `int min_pos = i;` — храним не само минимальное значение, а его **позицию** (индекс). Позиция нужна, чтобы потом сделать обмен.

- внутренний `for (j = i + 1; ...)` просматривает неотсортированный остаток, начиная сразу за `i`. Если нашли элемент меньше текущего минимума — обновляем `min_pos`.
- `if (min_pos != i)` — если минимум уже стоит на позиции `i`, обмен не нужен (это не баг, а просто экономия). Иначе — обычный swap через `t`.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-selection.c -o selection
$ ./selection
До:    5 2 4 1
После: 1 2 4 5
```

Часть 3. Сортировка вставками

Идея руками

Лучшая аналогия — как ты держишь карты в руке. Берёшь карты по одной из стопки и **вставляешь** каждую новую на правильное место среди уже разобранных, сдвигая остальные. Слева у тебя всегда упорядоченный веер, справа — то, что ещё не разобрал.

В массиве то же самое. Считаем, что первый элемент уже «отсортирован» (один элемент всегда упорядочен). Берём следующий и продвигаем его влево, пока слева есть элементы больше него.

`[5, 2, 4, 1]`. Отсортированную левую часть отметим `|`:

```
5 | 2 4 1   берём 2; 5 > 2 -> сдвигаем 5 вправо, 2 встаёт в начало
2 5 | 4 1   берём 4; 5 > 4 -> сдвигаем 5; 2 < 4 -> стоп, 4 за 2
2 4 5 | 1   берём 1; 5,4,2 все > 1 -> сдвигаем все, 1 в начало
1 2 4 5 |   разобрали все карты
```

Получили `[1 2 4 5]`. Главное действие — **сдвиг**: чтобы освободить место для вставляемого элемента, большие соседи едут на одну позицию вправо.

Шаг 1. Наивная мысль и сразу аккуратная версия

Новичок часто пытается «искать место, потом вставлять» в два прохода. Но сдвиг и поиск места удобно совместить в одном цикле `while`: пока слева элемент больше вставляемого — двигаем его вправо. Где цикл остановился, там и место.

Есть тонкость: вставляемый элемент надо **сначала запомнить** в отдельную переменную (`key`). Если этого не сделать, сдвиг затрёт его первым же шагом.

code/08-insertion.c:

```c
#include <stdio.h>

void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Сортировка вставками по возрастанию.
   Слева растёт отсортированная часть. Берём очередной элемент key
   и вставляем его в нужное место этой части, сдвигая большие
   элементы вправо. Так раскладывают карты в руке. */
void insertion_sort(int a[], int n)
{
    for (int i = 1; i < n; i++) {
        int key = a[i];                /* элемент, который вставляем */
        int j = i - 1;                 /* идём по отсортированной части */

        /* сдвигаем вправо всё, что больше key */
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;                /* нашлось место — кладём key */
    }
}

int main(void)
{
    int a[] = {5, 2, 4, 1};
    int n = (int)(sizeof a / sizeof a[0]);

    printf("До:    ");
    print_array(a, n);
```

```
    insertion_sort(a, n);

    printf("После: ");
    print_array(a, n);
    return 0;
}
```

Разбор `insertion_sort`:

- внешний `for (i = 1; ...)` начинается с **1**, а не с 0: элемент с индексом 0 считаем уже отсортированной частью из одного элемента.
- `int key = a[i];` — запоминаем вставляемый элемент. Дальше место `a[i]` будет затёрто сдвигом, поэтому значение нужно сохранить заранее.
- `int j = i - 1;` — `j` будет двигаться по отсортированной части справа налево.
- `while (j >= 0 && a[j] > key)` — два условия через `&&` (логическое «И», см. главу 2):
  - `j >= 0` — не вышли за левый край массива;
  - `a[j] > key` — текущий элемент больше вставляемого, значит, его надо подвинуть.

  Порядок важен! Сначала проверяется `j >= 0`. Если `j` стал −1, `&&` дальше не смотрит на `a[j]` (это называется «короткое замыкание»), и обращения к `a[-1]` не происходит. Поменяй условия местами — получишь выход за границу.
- `a[j + 1] = a[j];` — сдвигаем элемент на одну позицию вправо.
- `j--;` — шаг влево.
- после цикла `a[j + 1] = key;` — `j` указывает на ближайший слева элемент, не превышающий `key` (или на −1, если `key` меньше всех в отсортированной части). Значит, место для `key` — следующая клетка, `j + 1`.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-insertion.c -o insertion
$ ./insertion
До:    5 2 4 1
После: 1 2 4 5
```

Маленький бонус вставок: на почти отсортированных данных она очень быстра — для каждого элемента `while` почти сразу останавливается.

Как это работает: чем три сортировки отличаются

Все три дают один результат, но «характер» у них разный:

- **Пузырёк** много **меняет** соседей и медленно «протаскивает» большие числа в конец. С флагом досрочно ловит уже готовый массив.
- **Выбор** делает мало обменов (по одному на шаг), но всегда честно просматривает весь остаток в поисках минимума — даже если массив уже отсортирован.
- **Вставки** аккуратно «дораскладывают» массив и особенно хороши, когда данные почти упорядочены.

И одно общее свойство. Все три сортируют **на месте** (по-английски *in-place*): переставляют элементы внутри того же массива и берут сверху лишь несколько переменных (`t`, `key`, `min_pos`, счётчики циклов). Сортируешь хоть десять чисел, хоть миллион — дополнительной памяти нужно одинаково мало. Второй массив под копию данных им не нужен. Запомни это слово: чуть ниже встретится сортировка, которая на месте работать не умеет.

Стабильность: что происходит с одинаковыми элементами

Есть у сортировок ещё одно свойство, о котором новичкам обычно не рассказывают, а на работе оно всплывает сразу же. Называется **устойчивость**, или **стабильность** (stability): в русских книгах встречаются оба слова, значат они одно и то же. Мы будем говорить «стабильность».

Представь список учеников, уже отсортированный по фамилии. Теперь ты сортируешь его по оценке. У троих одна и та же пятёрка — в каком порядке они окажутся между собой? Стабильная сортировка сохранит их прежний порядок (по фамилии). Нестабильная перемешает как попало.

Определение: сортировка **стабильная**, если элементы с одинаковым ключом остаются в том же порядке, в каком лежали до сортировки.

Проверим это на наших трёх. Массив из обычных `int` не годится: две одинаковые пятёрки друг от друга не отличишь, поменялись они местами или нет — не видно. Поэтому зашьём в каждое число сразу две вещи: **десятки** — ключ, по которому сортируем, **единицы** — метку исходного места. Число `42` читается как «ключ 4, метка 2».

Достать половинки помогут две крошечные функции:

```
static int key_of(int x)
{
    return x / 10;              /* десятки — то, по чему сортируем */
}

static int tag_of(int x)
{
```

```
    return x % 10;              /* единицы — откуда элемент пришёл */
}
```

Здесь работают знакомые из главы 4 приёмы: `/ 10` отрезает последнюю цифру, `% 10` её достаёт. Слово `static` перед функцией означает «эта функция нужна только внутри своего файла, наружу её не показывать» — для таких мелких помощников это хороший тон.

Дальше во всех трёх сортировках меняется ровно одно: сравниваем не `a[i] > a[i + 1]`, а `key_of(a[i]) > key_of(a[i + 1])`. Метки в сравнении не участвуют вообще — сортировка про них не знает. Полностью программа лежит в `code/08-stable.c`. Берём массив `41 42 43 24` (три элемента с ключом 4 и один с ключом 2) и смотрим:

```
cc -std=c11 -Wall -Wextra -O2 08-stable.c -o 08-stable
./08-stable
до:        4/1  4/2  4/3  2/4
пузырёк:   2/4  4/1  4/2  4/3
вставки:   2/4  4/1  4/2  4/3
выбор:     2/4  4/2  4/3  4/1
```

Ключи везде расставлены правильно: сначала 2, потом три четвёрки. А вот метки:

- **пузырёк** — `1 2 3`, порядок сохранён. Стабилен;
- **вставки** — `1 2 3`, тоже сохранён. Стабильны;
- **выбор** — `2 3 1`. Порядок сломан. **Нестабилен**.

Откуда взялась разница? У пузырька и вставок сравнение **строгое**: `a[i] > a[i + 1]` и `a[j] > key`, а не `>=`. Равные элементы не считаются «стоящими не по порядку», значит, местами их никто не меняет, а двигаются элементы только к соседу, через головы не прыгают. Поставь там `>=` — и обе сортировки мгновенно потеряют стабильность.

А выбор ломает порядок самим устройством. Проследи за первым шагом: `i` = 0, минимум — это `2/4` на позиции 3. Обмен `a[0]` и `a[3]` **швыряет** `4/1` с начала массива в самый конец, через головы `4/2` и `4/3`. Один дальний перелёт — и относительный порядок четвёрок сломан навсегда. Так устроен любой обмен «издалека», и вылечить это, не переписав алгоритм, нельзя.

Запомни короткий вывод: **пузырёк и вставки стабильны, выбор — нет**. Пока сортируешь голые числа, это не важно. Как только у элемента появляются «довески» (фамилия рядом с оценкой, дата рядом с суммой), стабильность становится настоящим требованием.

Сложность: почему это медленно и что бывает быстрее

У всех трёх есть общая беда: **два вложенных цикла**. Посчитаем точно, сколько сравнений это даёт, а не «примерно».

Считаем сравнения руками

Возьмём пузырёк без флага. На первом проходе он сравнивает `n - 1` пару соседей, на втором — `n - 2` (хвост уже на месте), и так до последнего прохода с одной парой. Всего

```
(n - 1) + (n - 2) + ... + 2 + 1 = n(n - 1) / 2
```

Откуда формула? Запиши эту сумму дважды: один раз как есть, второй — задом наперёд, одну под другой, и сложи столбиками. В каждом столбике выходит ровно `n` (`(n - 1) + 1`, `(n - 2) + 2`, …), столбиков `n - 1`. Две суммы вместе дают `n(n - 1)`, одна — половину. Проверка на нашем массиве из четырёх: `3 + 2 + 1 = 6`, и `4 · 3 / 2 = 6`. Для ста элементов — 4950 сравнений.

Выбор делает столько же: при `i = 0` внутренний цикл сравнивает `n - 1` раз, при `i = 1` — `n - 2`, и так далее. Вставки в худшем случае (массив задом наперёд) — тоже: каждый новый элемент протаскивается через всю левую часть, `1 + 2 + ... + (n - 1)`.

При больших `n` в формуле главное — квадрат: `n(n - 1) / 2 = n²/2 - n/2`, и вторая часть рядом с первой крошечная. При `n = 1000` это 500 000 против 500.

Что значит O(n²)

Для такого «главного в формуле» есть короткая запись — **O-нотация** (читают «о большое», от английского *order* — порядок). Пишут **O(n²)** («о от эн квадрат») и имеют в виду: начиная с какого-то `n`, работа не превышает `n²`, умноженного на какое-то постоянное число. У наших сортировок это число — одна вторая: `n(n - 1) / 2` никогда не больше `n² / 2`.

Три тонкости, о которых часто забывают:

- **O — это потолок**, оценка сверху. Формально про линейный поиск из главы 7 тоже верно, что он O(n²): `n` не больше `n²`. Верно, но бесполезно. Когда хотят сказать «растёт ровно как `n²`, не быстрее и не медленнее», математики пишут `Θ(n²)` (буква «тета»). Программисты в разговоре обычно говорят «O», подразумевая именно точный рост, — и мы в книге так делаем.
- **Постоянный множитель O не видит.** `n²/2` и `5n²` — оба O(n²), хотя второй в десять раз медленнее. Поэтому два O(n²)-алгоритма запросто различаются по скорости в разы — скоро увидим это на секундомере.
- **Оценка относится к конкретному случаю** — лучшему, худшему или среднему (эти слова мы разобрали в главе 7). У одной и той же сортировки они бывают очень разными.

Практический смысл $O(n^2)$ такой: увеличишь массив в 10 раз — работы станет примерно в 100 раз больше; удвоишь — вчетверо.

Проверяем подсчётом: лучший, средний и худший случай

Формулы — на бумаге. Пусть сортировки сами посчитают свою работу. Программа `code/08-count.c` — те же три функции, только в каждую добавлены два счётчика: `cmps` растёт перед каждым сравнением двух элементов, `moves` — на каждый обмен (у пузырька и выбора) и на каждый сдвиг (у вставок). Счётчики объявлены вне всех функций, в самом начале файла (`long cmps = 0;`), поэтому их видят все три сортировки. Такие переменные называются глобальными, подробнее о них — в главе 9.

Пузырёк и выбор получили по строчке `cmps++;` перед своим `if` и `moves++;` рядом с обменом. Во вставках сравнение спрятано в условии `while`, поэтому цикл пришлось разложить на части:

```
while (j >= 0) {
    cmps++;             /* сравниваем a[j] с key */
    if (a[j] <= key) {
        break;          /* нашли место — дальше не идём */
    }
    a[j + 1] = a[j];
    moves++;            /* один сдвиг вправо */
    j--;
}
```

Это тот же цикл `while (j >= 0 && a[j] > key)`, только вторая половина условия переехала внутрь: если `a[j]` не больше `key`, место найдено, и `break` выходит из `while`. Зато теперь каждое сравнение посчитано.

Входов три, по 100 чисел: уже отсортированный (`0 1 2 ... 99`), задом наперёд (`99 98 ... 0`) и перемешанный. Перемешиваем без `rand`, чтобы вывод у всех был одинаковым: `a[i] = (i * 37) % 100`. Так получаются все числа от 0 до 99 по одному разу, но вразброс: 0, 37, 74, 11, 48, …

```
cc -std=c11 -Wall -Wextra -O2 08-count.c -o 08-count
./08-count
n = 100, n(n-1)/2 = 4950

сравнений перемещений   сортировка, вход
        99           0   пузырёк, отсортирован
      4950        4950   пузырёк, задом наперёд
      4895        2448   пузырёк, перемешан
      4950           0   выбором, отсортирован
```

```
4950          50   выбором, задом наперёд
4950          88   выбором, перемешан
  99           0   вставками, отсортирован
4950        4950   вставками, задом наперёд
2547        2448   вставками, перемешан
```

Читаем по сортировкам.

- **Пузырёк.** На отсортированном входе — 99 сравнений, то есть `n - 1`: один проход без обменов, флаг `swapped` остался нулём, `break`. Это **лучший случай**, и он растёт как `n`, а не n^2. На входе задом наперёд — **худший**: ровно 4950 сравнений и 4950 обменов. На перемешанном флаг сэкономил совсем немного: 4895 сравнений.
- **Выбор.** 4950 сравнений **на любом входе**, даже на уже отсортированном: он не умеет замечать, что всё и так на месте, и каждый раз честно ищет минимум в остатке. Зато обменов не больше `n - 1`: 0, 50 и 88.
- **Вставки.** Лучший и худший случаи — как у пузырька (99 и 4950), но на перемешанном сравнений почти вдвое меньше: 2547.

Почему у пузырька и вставок на перемешанном входе перемещений **ровно поровну**, 2448 и 2448? Тут помогает ещё одно понятие. **Инверсия** — это пара элементов, стоящих не по порядку: левый больше правого. В `[5, 2, 4, 1]` инверсий пять: 5 и 2, 5 и 4, 5 и 1, 2 и 1, 4 и 1. Обмен двух **соседей**, стоящих не по порядку, убирает ровно одну инверсию — ту, что между ними, а порядок остальных пар не меняет. Сортировка кончается, когда инверсий не осталось. Значит, пузырёк делает ровно столько обменов, сколько инверсий было в массиве. Сдвиг во вставках — то же самое: `key` перепрыгивает через один больший элемент, одна инверсия исчезает. Проверь на `[5, 2, 4, 1]`: пять инверсий — пять обменов у пузырька (мы их делали руками в начале главы: три, потом один, потом ещё один).

В отсортированном массиве инверсий ноль. В обратном — каждая пара стоит не по порядку, их `n(n - 1) / 2` = 4950. В случайном массиве каждая пара с равным шансом стоит так или наоборот, поэтому инверсий в среднем половина от всех пар: `n(n - 1) / 4`, для ста чисел — 2475. У нашего перемешанного 2448 — близко. Выходит, и в **среднем** случае пузырёк и вставки делают порядка n^2 работы.

Итог в одну строку на каждую сортировку:

- **пузырёк с флагом**: лучший случай O(n), средний и худший — $O(n^2)$;
- **выбор**: $O(n^2)$ сравнений всегда, даже в лучшем случае, но не больше `n - 1` обменов;
- **вставки**: лучший случай O(n), средний и худший — $O(n^2)$; на случайных данных сравнений вдвое меньше, чем у пузырька.

Прикинь масштаб квадратичного роста:

```
n = 10         ->     ~100 операций      мгновенно
n = 1 000      -> ~1 000 000 операций    ещё терпимо
n = 1 000 000 -> ~10^12 операций         минут двадцать
```

Запись `10^12` значит «десять в двенадцатой степени», то есть единица с двенадцатью нулями. Откуда «минут двадцать» — сейчас увидим.

Замеряем по-настоящему

Подсчёт сравнений — это теория, а секундомер — жизнь. Программа `code/08-bench.c` заполняет массив случайными числами, сортирует его всеми тремя способами и засекает время `clock()` — тем самым секундомером из главы 7. Размер каждый раз удваивается:

```
cc -std=c11 -Wall -Wextra -O2 08-bench.c -o 08-bench
./08-bench
       n   пузырёк   выбором  вставками
    5000     0.010     0.005      0.003
   10000     0.055     0.017      0.010
   20000     0.327     0.069      0.039
   40000     1.654     0.285      0.151

пузырёк по уже отсортированным 40000: 0.000015 с
qsort по 1000000 элементам: 0.1260 с
```

Читаем таблицу столбиком сверху вниз. Удваиваем `n` — и время растёт не вдвое, а примерно вчетверо: у выбора 0.017 → 0.069 → 0.285, у вставок 0.010 → 0.039 → 0.151. Вот это «в четыре раза при удвоении» и есть $O(n^2)$ вживую: удвоили `n`, значит, `n × n` выросло в `2 × 2 = 4` раза.

У пузырька скачки крупнее: 0.055 → 0.327 → 1.654, в 5–6 раз. Число сравнений у него растёт ровно вчетверо (это мы только что посчитали), значит, дороже становится **каждое** сравнение. Скорее всего, дело в процессоре: он заранее угадывает, куда пойдёт `if`, а на случайных данных у пузырька эти догадки часто мимо, и каждый промах стоит времени. О-нотация такие вещи не описывает: она говорит, как растёт число шагов, а не сколько наносекунд стоит шаг.

Заодно видно, что три «одинаково квадратичных» алгоритма всё же не равны: на 40 000 элементах вставки примерно в 11 раз быстрее пузырька на одних и тех же данных. Порядок роста общий, а постоянные множители разные — ровно та вторая тонкость О-нотации. Сравнений у вставок на случайных данных вдвое меньше ($n^2/4$ против почти $n^2/2$), каждое перемещение — одно присваивание вместо трёх в обмене, а остальное, видимо, добирают промахи процессора у пузырька.

Предпоследняя строка — проверка того самого флага `swapped`. По уже отсортированным 40 000 элементам пузырёк отработал за 15 микросекунд: один проход без единого обмена, флаг остался нулём, `break`. Сравни с 1.654 секунды на тех же 40 000 вперемешку — разница примерно в сто тысяч раз. Это и есть «лучший случай O(n) против среднего O(n^2)».

Последняя строка — гвоздь программы. Библиотечная `qsort` разобрала **миллион** элементов за 0.13 секунды. Это в 25 раз больше данных, чем в строке про 40 000, — и примерно в 13 раз быстрее по времени. Если бы мы взялись за тот же миллион пузырьком, по нашей же таблице вышло бы примерно `1.654 × 25 × 25 ≈ 1030` секунд, то есть около семнадцати минут (и это оптимистично: мы видели, что у пузырька время растёт даже быстрее, чем вчетверо). Разница между O(n^2) и O(n log n) на глаз.

Быстрее, чем n^2

На миллионе элементов O(n^2) уже неприемлемо. К счастью, есть сортировки посложнее, но куда быстрее — со сложностью **O(n log n)**. Тут `log n` — это двоичный логарифм из главы 7, $\log_2 n$: сколько раз можно поделить `n` пополам, пока не дойдём до единицы (для миллиона это около 20). То есть вместо `n × n` получается порядка `n × 20` — огромная разница. Идею двух главных таких алгоритмов разберём словами, без полного кода. Обе построены на рекурсии — ей посвящена глава 9, — а целиком, с кодом и разбором, мы их соберём в приложении B (задачи 1 и 2).

Быстрая сортировка (quicksort)

Выбираем из массива один элемент — **опорный** (pivot). Переставляем остальные так, чтобы слева от опорного оказались все меньшие, а справа — все большие (равные — в любую сторону). Теперь опорный стоит на своём окончательном месте, а массив разбит на две части. Дальше **точно так же** сортируем левую часть и правую — каждую как отдельный массив поменьше. Деление продолжается, пока в частях не останется по одному элементу или ни одного.

Если опорный каждый раз делит массив примерно пополам, уровней деления выходит около $\log_2 n$, а на каждом уровне все элементы вместе просматриваются один раз — отсюда `n log n`. На случайных данных так и бывает в среднем: **средний случай — O(n log n)**. Но если опорный раз за разом оказывается самым маленьким или самым большим (например, всегда берём первый элемент, а массив уже отсортирован), части выходят размером `n - 1` и `0`, и получается тот же `n(n - 1) / 2`: **худший случай — O(n^2)**. Хорошие реализации выбирают опорный хитрее, чтобы худший случай почти не встречался. Сортирует quicksort на месте, но стабильным не является: при перестановках элементы прыгают далеко, через головы равных.

Сортировка слиянием (merge sort)

Делим массив пополам, каждую половину тоже пополам, и так до кусочков по одному элементу (один элемент уже «отсортирован»). Потом начинаем **сливать** кусочки обратно попарно, и при слиянии сразу располагаем элементы по порядку: берём меньший из первых элементов двух кусков, потом следующий меньший, и так пока оба не закончатся. Слить два упорядоченных куска общей длиной `k` можно за `k` шагов, а уровней деления всего около $\log_2 n$. Получаем O(n log n) — **в любом случае**, без неудачных входов, как у quicksort. Если при равенстве брать элемент из левого куска, сортировка ещё и **стабильна**. Платим за это памятью: сливать приходится во временный массив длиной `n`, так что на месте, как наши три, merge sort не работает.

Готовое решение: `qsort` из `<stdlib.h>`

В реальном коде свою сортировку обычно не пишут — берут библиотечную `qsort`. Она универсальная: сортирует массив **чего угодно** (числа, строки, структуры), потому что не знает заранее, что и как сравнивать. Эти знания ты даёшь ей сам — через **функцию сравнения** (компаратор).

`code/08-qsort.c`:

```c
#include <stdio.h>
#include <stdlib.h>     /* здесь живёт qsort */

void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Компаратор: говорит qsort, какой из двух элементов «раньше».
   Получает адреса двух элементов как const void *.
   Возвращает < 0, если первый меньше; 0, если равны; > 0, если больше. */
int cmp_int(const void *pa, const void *pb)
{
    int a = *(const int *)pa;   /* приводим адрес к int* и разыменовываем */
    int b = *(const int *)pb;
    if (a < b) return -1;
    if (a > b) return 1;
```

```
    return 0;
}

int main(void)
{
    int a[] = {5, 2, 4, 1, 9, 3, 7};
    int n = (int)(sizeof a / sizeof a[0]);

    /* qsort(база, число_элементов, размер_элемента, компаратор) */
    qsort(a, (size_t)n, sizeof a[0], cmp_int);

    print_array(a, n);
    return 0;
}
```

Разбор — здесь несколько новых деталей:

- `qsort(a, (size_t)n, sizeof a[0], cmp_int)` — четыре аргумента:
  - `a` — массив (его адрес начала);
  - `(size_t)n` — сколько элементов сортировать. `size_t` — целочисленный тип «для размеров и количеств», именно его ждёт `qsort`. Он беззнаковый: отрицательных значений в нём не бывает. Без приведения программа тоже соберётся, и с нашими флагами молча: `int` сам превратится в `size_t`. Пишем `(size_t)n`, чтобы переход был виден глазами (строгий флаг `-Wsign-conversion` о таком неявном переходе предупреждает);
  - `sizeof a[0]` — размер одного элемента в байтах. По нему `qsort` понимает, какими «шагами» ходить по массиву;
  - `cmp_int` — имя нашей функции сравнения. Передаём **саму функцию** (без скобок!) — `qsort` будет вызывать её, когда надо сравнить два элемента. Передача функции как аргумента — новая для нас вещь.
- `int cmp_int(const void *pa, const void *pb)` — компаратор обязан иметь именно такую форму, её требует `qsort`:
  - `const void *` — «адрес чего-то, тип заранее неизвестен». `void *` — «указатель на что угодно». `qsort` универсальна и не знает, что внутри, поэтому передаёт элементы безликими адресами;
  - `const` — обещаем не менять элементы при сравнении.
- `*(const int *)pa` — два действия сразу:
  - `(const int *)pa` — **приведение** безликого адреса к «адресу на `int`»: мы-то знаем, что сравниваем целые;
  - `*` перед этим — **разыменование** (см. главу 3): достаём само число по адресу.

- возврат: компаратор должен вернуть **отрицательное** число, если первый элемент должен идти раньше второго, **0** — если они равны, и **положительное** — если позже. По этим знакам `qsort` и расставляет всё по порядку. Хочешь сортировку по убыванию — просто поменяй местами `a` и `b` в сравнениях.

Запуск:

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-qsort.c -o qsort_demo
$ ./qsort_demo
1 2 3 4 5 7 9
```

Одна строка `qsort(...)` — и массив отсортирован быстрым алгоритмом. Понимать, что у неё внутри, всё равно полезно: иначе компаратор кажется магией.

Что `qsort` обещает, а что нет

Три вещи стоит знать заранее, чтобы потом не удивляться.

Название обманывает. `qsort` — сокращение от quicksort, но стандарт Си нигде не требует, чтобы внутри была именно быстрая сортировка. Библиотека вправе взять что угодно, лишь бы результат был упорядочен. Разные системы так и делают.

Стабильность не обещана. Помнишь предыдущий раздел? Так вот, про `qsort` стандарт прямо говорит: если два элемента при сравнении равны, их порядок в результате не уточняется — какой выйдет, такой и будет. Нужна стабильность — либо пиши свою сортировку (вставки стабильны), либо добавляй в компаратор второй ключ: «при равных оценках сравни ещё и номер в исходном списке».

Компаратор должен быть честным. `qsort` строит свою работу на том, что твоя функция ведёт себя как нормальное сравнение: если `cmp(a, b)` отрицательное, то `cmp(b, a)` обязано быть положительным; если `a` раньше `b`, а `b` раньше `c`, то `a` обязан быть раньше `c`; и на одних и тех же аргументах ответ всегда один. Нарушишь это — по стандарту поведение `qsort` не определено. Компаратор, который отвечает по настроению (скажем, зовёт `rand()`), в какой-нибудь библиотеке может завести `qsort` за границы массива — и это уже не «неверный порядок», а падение.

Ловушка: `return a - b`

В интернете и в чужом коде ты обязательно встретишь короткий вариант компаратора:

```
int cmp_bad(const void *pa, const void *pb)
{
    int a = *(const int *)pa;
```

```
    int b = *(const int *)pb;
    return a - b;               /* коротко, красиво и неверно */
}
```

Логика на первый взгляд безупречна: если `a` меньше `b`, разность отрицательна; если больше — положительна; если равны — ноль. Ровно то, что просит `qsort`.

Ломается это на больших числах. Разность двух `int` сама должна влезть в `int`, а она не обязана: `2000000000 - (-2000000000)` — это четыре миллиарда, потолок `int` (2 147 483 647) почти вдвое ниже. Получаем то самое переполнение из главы 4 и главы 7 — неопределённое поведение. На практике разность «заворачивается» и выходит **отрицательной**: −294 967 296. Компаратор сообщает `qsort`, что два миллиарда **меньше** минус двух миллиардов, и та ставит их не в том порядке.

Проверим. Программа `code/08-cmp-trap.c` сортирует один и тот же набор чисел дважды: плохим компаратором и хорошим:

```
int bad[] = {2000000000, -2000000000, 7, -7};
int n = (int)(sizeof bad / sizeof bad[0]);
int good[4];

for (int i = 0; i < n; i++) {
    good[i] = bad[i];       /* второй экземпляр тех же чисел */
}

qsort(bad,  (size_t)n, sizeof bad[0],  cmp_bad);
qsort(good, (size_t)n, sizeof good[0], cmp_good);
```

Запускаем:

```
cc -std=c11 -Wall -Wextra -O2 08-cmp-trap.c -o 08-cmp-trap
./08-cmp-trap
до сортировки:    2000000000 -2000000000 7 -7
через a - b:      2000000000 -2000000000 -7 7
через сравнения:  -2000000000 -7 7 2000000000
```

Хороший компаратор дал правильный ответ. Плохой оставил два самых больших по модулю числа **на своих местах** — то есть двухмиллиардное положительное стоит перед двухмиллиардным отрицательным. Массив не отсортирован, а программа об этом даже не намекнула.

Причину покажет санитайзер из главы 4:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=undefined 08-cmp-trap.c \
   -o 08-cmp-trap-san
./08-cmp-trap-san
08-cmp-trap.c:24:14: runtime error: signed integer overflow: 2000000000
- -2000000000 cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior 08-cmp-trap.c:24:14
до сортировки:    2000000000 -2000000000 7 -7
через a - b:      2000000000 -2000000000 -7 7
через сравнения:  -2000000000 -7 7 2000000000
```

Первая строка отчёта на экране длинная, здесь она перенесена. Строка 24 — это `return a - b;`. Вот и весь фокус. Отчёт появляется один раз, хотя плохих вычитаний могло быть несколько: санитайзер сообщает о каждом месте в коде только при первом срабатывании.

Живёт эта ловушка тихо: пока числа маленькие, `a - b` работает годами. Стрельнет она в тот день, когда в данные попадёт что-нибудь вроде идентификатора на два миллиарда. Пиши три сравнения — они чуть длиннее, зато не ломаются никогда. Есть и короткая надёжная запись в одну строку: `return (a > b) - (a < b);`. Каждое сравнение даёт 1 или 0 (глава 2), и разность этих двух чисел — ровно −1, 0 или 1, без всякого переполнения.

Частые ошибки

Выход за границу в условии цикла. В пузырьке сравниваем `a[i]` и `a[i + 1]`, поэтому внутренний цикл идёт до `i < n - 1`, а не `i < n`. Иначе при `i = n - 1` обратишься к несуществующему `a[n]`.

Swap без временной переменной. Если написать `a[i] = a[j]; a[j] = a[i];`, ты сначала затрёшь `a[i]`, и оба элемента станут одинаковыми. Нужна третья переменная `t` (см. главу 6).

В выборе хранят значение вместо позиции. Для обмена нужен **индекс** минимума (`min_pos`), а не само минимальное число. По числу непонятно, какой элемент с каким менять.

Во вставках забыли сохранить `key`. Сдвиг `a[j + 1] = a[j]` затрёт вставляемый элемент, если ты не запомнил его заранее в `key`.

Перепутан порядок в `&&` у вставок. Условие должно быть `j >= 0 && a[j] > key`, а не наоборот. Сначала проверка границы, потом обращение к `a[j]`, иначе при `j == -1` залезешь за край.

Компаратор возвращает `a - b`. Разность двух `int` не обязана влезать в `int`. Переполнение — неопределённое поведение, на практике знак выходит неверным, и `qsort` ставит пару элементов не в том порядке — мы это только что видели своими глазами. Надёжнее три сравнения или `(a > b) - (a < b)`.

Ждут от `qsort` стабильности. Стандарт её не обещает. Если у элементов есть «довесок» и порядок равных важен — сортируй вставками или дописывай в компаратор второй ключ.

Забыли `(size_t)` и `sizeof` в вызове `qsort`. Второй аргумент — **число элементов**, третий — **размер одного элемента в байтах**. Перепутаешь местами или подставишь размер всего массива — `qsort` пойдёт по памяти неверными шагами. Компилятор здесь не помощник: оба параметра одного типа `size_t`, любая пара чисел ему подходит.

Передали компаратор со скобками. Нужно `cmp_int`, а не `cmp_int()`. Первое — сама функция, второе — попытка вызвать её прямо здесь. Со скобками программа не соберётся:

```
08-qsort.c:30:46: error: too few arguments to function call, expected 2,
have 0
```

Боятся пустого массива и одного элемента. Все три наши сортировки справляются с ними сами, без единой дополнительной проверки. При `n = 0` и `n = 1` внешний цикл пузырька и выбора (`pass < n - 1`, `i < n - 1`) не делает ни одного шага, а у вставок цикл начинается с `i = 1` и при `n <= 1` тоже не запускается. Массив из нуля или одного элемента уже отсортирован — так и должно быть. Отрицательные числа и массив, где все элементы равны, тоже проходят нормально: проверь сам под `-fsanitize=address,undefined`.

Ставят `>=` вместо `>` в сравнении. На результат сортировки это не влияет (числа всё равно встанут по порядку), зато пузырёк и вставки теряют стабильность и вдобавок делают кучу бессмысленных обменов равных элементов.

Упражнения

1. Добавь в `bubble_sort` счётчик обменов (`long swaps`) и печатай его в конце. Сравни число обменов для `[5, 2, 4, 1]` и для уже отсортированного `[1, 2, 3, 4]`. *Ответ:* 5 и 0 — ровно число инверсий в каждом массиве.

2. Сделай так, чтобы все три сортировки печатали массив **после каждого прохода** (для пузырька/выбора) или **после каждой вставки**. Понаблюдай, как массив приходит в порядок. *Ожидаемо* на `[5, 2, 4, 1]`: пузырёк — `2 4 1 5`, `2 1 4 5`, `1 2 4 5`; выбор — `1 2 4 5` уже после первого шага (единица поменялась с пятёркой), дальше без изменений; вставки — `2 5 4 1`, `2 4 5 1`, `1 2 4 5`. Сверь с раскладками руками из этой главы.

3. Переделай `selection_sort` на поиск **максимума** и установку его в **конец** неотсортированной части. Результат должен быть тем же. *Подсказка:* внешний цикл `for (int end = n - 1; end > 0; end--)`, максимум ищем среди `a[0..end]` и меняем с `a[end]`. На массиве из `08-qsort.c` получится `1 2 3 4 5 7 9`.

4. Напиши функцию `int is_sorted(const int a[], int n)`, возвращающую 1, если массив отсортирован по возрастанию, и 0 иначе. Проверяй ею массив до и после сортировки. *Ответ:* для `[5, 2, 4, 1]` до — 0, после — 1. Пустой массив и массив из одного элемента — 1: они уже упорядочены. Сравнивай соседей `a[i] > a[i + 1]` при `i + 1 < n`, чтобы не выйти за край.

5. Заполни массив из 20 чисел вводом (или случайными, если уже умеешь `rand`) и отсортируй его любой из трёх сортировок. *Проверка:* `is_sorted` из упражнения 4 после сортировки возвращает 1.

6. Возьми `08-qsort.c` и сделай вторую версию компаратора `cmp_desc`, сортирующую по **убыванию**. Отсортируй один и тот же массив сначала по возрастанию, потом по убыванию. *Ответ:* в `cmp_desc` при `a > b` возвращаем −1, при `a < b` — 1. Вывод: `1 2 3 4 5 7 9`, затем `9 7 5 4 3 2 1`.

7. **Проверь стабильность руками.** Возьми `08-stable.c`, замени в пузырьке и вставках `>` на `>=` и запусти снова. Метки у четвёрок встанут в обратном порядке `3 2 1` — обе сортировки перестали быть стабильными от одного знака. Объясни себе, почему. *Ответ:* теперь равные соседи считаются «стоящими не по порядку» и меняются местами, так что каждая следующая четвёрка проталкивается левее предыдущих.

8. **Свой замер.** Добавь в `08-bench.c` строку с `n = 80000` и предскажи время **до** запуска: раз при удвоении время растёт вчетверо, пузырёк должен уложиться примерно в 7 секунд (`1.65 × 4 ≈ 6.6`). Запусти и сравни с предсказанием. *У меня* вышло 7.02 и 7.15 секунды на двух запусках — чуть больше прогноза, как и в таблице: пузырёк растёт немного быстрее, чем вчетверо. У выбора — около 1.1 секунды, у вставок — 0.6–0.7.

9. **Сломай компаратор.** Напиши компаратор, который всегда возвращает `0` («все элементы равны»), и посмотри, что `qsort` сделает с массивом. Потом такой, что всегда возвращает `1`. Второй нечестен: он утверждает, что `a` больше `b` **и** что `b` больше `a` одновременно. Результат окажется каким угодно, и полагаться на него нельзя, даже если у тебя он выглядит осмысленно. *Что вышло у меня на macOS* с массивом `5 2 4 1 9 3 7`: с нулём массив остался как был (компаратор честный, просто все равны, а порядок равных `qsort` не обещает), с единицей — `7 3 9 1 4 2 5`, то есть задом наперёд. На массиве из 40 чисел «единичный» компаратор дал уже полную мешанину. В другой библиотеке (например, на Linux) результат может быть другим — в этом и смысл упражнения.

Что мы получили

Ты написал три сортировки с нуля и понял, чем они отличаются: пузырёк «всплывает» обменами соседей (с флагом для досрочного выхода), выбор ищет минимум и ставит его в начало,

вставки раскладывают элементы как карты в руке. Все три сортируют на месте и в среднем и худшем случае делают $O(n^2)$ работы: до `n(n – 1) / 2` сравнений. В лучшем случае пузырёк с флагом и вставки обходятся `n – 1` сравнением, а выбор — нет. И это не отвлечённые буквы: счётчики в `08–count.c` показали эти числа точно, а секундомер — как удвоение `n` примерно учетверяет время и как `qsort` разбирает миллион элементов быстрее, чем пузырёк — сорок тысяч.

Попутно набралось несколько вещей, которые отличают «работает» от «работает правильно»: пузырёк и вставки **стабильны**, а выбор нет; `qsort` не обещает ни конкретного алгоритма, ни стабильности; компаратор `return a – b` тихо ломается на больших числах.

Все программы главы: `08–bubble.c`, `08–selection.c`, `08–insertion.c`, `08–qsort.c`, а к ним четыре вспомогательные — `08–count.c` (счётчики сравнений и перемещений), `08–stable.c` (стабильность), `08–cmp–trap.c` (ловушка компаратора) и `08–bench.c` (замер скорости). Собираются все одинаково: `cc –std=c11 –Wall –Wextra –O2 файл.c –o имя`.

И quicksort, и merge sort устроены на одном приёме: **разбить задачу на части поменьше и решить их тем же способом**. Это называется **рекурсия** — функция, которая вызывает сама себя. Без неё эти быстрые сортировки не написать красиво. Поэтому следующая глава целиком про рекурсию: разберём её на простых примерах. А сами quicksort и merge sort, уже с кодом, ждут тебя в приложении В (задачи 1 и 2).

Глава 9. Рекурсия: факториал, Фибоначчи, Ханойские башни

До сих пор, когда нужно было что-то повторить, мы брали цикл. Но есть второй способ повторения — когда функция **вызывает сама себя**. Это называется **рекурсия** (recursion). Звучит как фокус «зеркало в зеркале», но идея простая и очень мощная: большую задачу мы сводим к такой же задаче, только поменьше, и так — пока не упрёмся в самый маленький, очевидный случай.

В этой главе мы разберём рекурсию с нуля на трёх примерах:

- **факториал** — сравним рекурсию с циклом из главы 4;
- **числа Фибоначчи** — увидим, как наивная рекурсия становится чудовищно медленной, и починим это;
- **Ханойские башни** — задача, которую циклом писать больно, а рекурсией она решается в три строки.

Что такое рекурсия

Возьмём бытовой пример. Ты стоишь в очереди и хочешь узнать, какой ты по счёту. Можно посчитать всех впереди самому. А можно так: спроси у того, кто стоит прямо перед тобой, какой он по счёту, и прибавь 1. Но ему, чтобы ответить, надо спросить у того, кто перед ним. И так далее — **до самого первого человека**, который точно знает: «я первый». Его ответ покатится назад по очереди, и каждый прибавит свою единицу.

Вот это и есть рекурсия. У неё всегда две обязательные части:

1. **База** (base case) — самый простой случай, ответ на который мы знаем сразу, без всяких вызовов. В очереди это «я первый».
2. **Шаг** (recursive step) — сведение задачи к **меньшей** такой же задаче плюс немного работы. В очереди это «спроси соседа и прибавь 1».

Если есть и база, и шаг, и каждый шаг приближает нас к базе — рекурсия доберётся до базы и остановится.

Что будет без базы

Убери базу — и катастрофа. Функция будет вызывать себя, потом снова себя, и снова, и так **бесконечно**. Это **бесконечная рекурсия**. Обычно она кончается не зависанием, как бесконечный цикл, а аварией: программа падает с ошибкой **stack overflow** («переполнение стека вызовов»). Сейчас разберём, что это за стек.

Стек вызовов простыми словами

Когда одна функция вызывает другую, компьютер должен **запомнить**, куда вернуться и какие у вызвавшей функции были локальные переменные. Для этого есть **стек вызовов** (call stack) — особая область памяти, устроенная как стопка тарелок.

- Вызвали функцию — на стопку **легла новая тарелка** (её локальные переменные, параметры, точка возврата). По-научному такая тарелка называется **кадром стека** (stack frame) — это слово встретится в отчётах санитайзера.
- Функция доработала и сделала `return` — её **тарелку сняли**, и управление вернулось к той, что лежит ниже.

Снимать и класть можно только сверху — как с настоящей стопкой тарелок.

При рекурсии тарелки кладутся одна на другую: `factorial(4)` ещё не закончил — он ждёт `factorial(3)`, а тот ждёт `factorial(2)`, и так далее. Стопка растёт. Когда дойдём до базы, тарелки начнут сниматься сверху вниз, и ответы поедут обратно.

Стопка не бесконечная — память под неё ограничена (обычно несколько мегабайт). Если базы нет или до неё слишком далеко, тарелки переполнят стек, и программа рухнет. Это и есть **stack overflow**. Запомни как правило номер один: **у каждой рекурсии должна быть база, к которой шаги реально приближаются**.

Часть 1. Факториал рекурсивно

Факториал числа `n` (пишется `n!`) — это произведение всех целых от 1 до `n`. Например, `5! = 1·2·3·4·5 = 120`. И отдельно договорённость: `0! = 1`.

В главе 4 мы считали факториал циклом — просто перемножали числа от 2 до `n`:

```
long long fact = 1;
for (int i = 2; i <= n; i++) {
```

```
    fact = fact * i;
}
```

Теперь посмотрим на факториал **другими глазами**. Заметь:

```
5! = 5 · 4 · 3 · 2 · 1
4! =     4 · 3 · 2 · 1
```

То есть `5! = 5 · 4!`. А `4! = 4 · 3!`. В общем виде:

```
n! = n · (n−1)!
```

Это и есть **шаг**: факториал `n` выражается через факториал меньшего числа `n−1`. А **база** — `0! = 1` (и `1! = 1`), дальше уменьшать некуда.

Пишем код

```
long long factorial(int n)
{
    if (n <= 1) {          /* база: дальше не спускаемся */
        return 1;
    }
    return n * factorial(n - 1);   /* шаг: сводим к меньшей задаче */
}
```

Разбор:

- `long long factorial(int n)` — функция принимает целое `n` и возвращает `long long`. Тип `long long` — это целое **увеличенной вместимости** (обычно 8 байт против 4 у `int`). Факториалы растут стремительно, и обычного `int` хватает лишь до `12!`; `long long` тянет до `20!`. Печатают его спецификатором `%lld` (на одну `l` больше, чем у `int`).
- `if (n <= 1) return 1;` — **база**. Для `0` и `1` ответ известен сразу: `1`. Никаких вызовов себя — функция просто возвращает число. Это та самая «остановка», без которой рекурсия не закончится.
- `return n * factorial(n - 1);` — **шаг**. Чтобы получить `n!`, мы умножаем `n` на результат **вызова себя же** с аргументом `n - 1`. Аргумент стал меньше — значит, рано или поздно дойдём до `1` и сработает база.

Вся собранная программа — `code/09-factorial.c`:

```
#include <stdio.h>

/* Рекурсивный факториал.
```

```c
   База: 0! = 1 и 1! = 1.
   Шаг: n! = n * (n-1)!. */
long long factorial(int n)
{
    if (n <= 1) {          /* база: дальше не спускаемся */
        return 1;
    }
    return n * factorial(n - 1);   /* шаг: сводим к меньшей задаче */
}

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }

    if (n > 20) {
        printf("Слишком большое: long long переполнится при n > 20.\n");
        return 1;
    }

    printf("%d! = %lld\n", n, factorial(n));
    return 0;
}
```

Проверки в `main` те же, что и в главе 4: `scanf` может вернуть не 1 (ввели не число), факториал не определён для отрицательных, а при `n > 20` даже `long long` переполнится (`21!` — это около 5.1 · 10^19, а потолок `long long` — около 9.2 · 10^18). Переполнение знакового целого, как ты помнишь по главе 4, — неопределённое поведение: на практике число обычно «заворачивает-

ся» и становится мусором без всякого предупреждения, а по стандарту может случиться что угодно. Поэтому мы отсекаем такие `n` заранее.

Как это работает: раскрутка вызовов

Самое важное в рекурсии — увидеть, как вызовы **раскручиваются**. Посчитаем `factorial(4)`. Сначала вызовы идут вглубь, каждый ждёт младшего:

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1            <- база! вот она, остановка
```

Дошли до базы — `factorial(1)` вернул `1` без вызовов. Теперь ответы едут **обратно** (тарелки снимаются со стека сверху вниз):

```
factorial(1) = 1
factorial(2) = 2 * 1  = 2
factorial(3) = 3 * 2  = 6
factorial(4) = 4 * 6  = 24
```

Готово: `4! = 24`. Обрати внимание на форму — вниз мы **спускаемся**, дробя задачу, а вверх **поднимаемся**, собирая ответ. В этот момент на стеке одновременно лежали четыре «тарелки» `factorial`. Для `n = 20` их будет двадцать — пустяк для стека. А вот для миллиона рекурсия уже переполнила бы стек, и тут цикл надёжнее.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/09-factorial.c -o factorial
$ ./factorial
Факториал какого числа? 5
5! = 120
$ ./factorial
Факториал какого числа? 0
0! = 1
$ ./factorial
Факториал какого числа? 20
20! = 2432902008176640000
```

Результат тот же, что у цикла из главы 4. Какой способ выбрать? Для факториала — дело вкуса. Умножений оба делают одинаково, `n - 1` штук. Разница в памяти: рекурсия держит на стеке до `n` кадров сразу, цикл — ни одного лишнего. Рекурсия читается ближе к математическому

определению `n! = n·(n−1)!`, цикл не тратит стек. Но дальше будет задача, где рекурсия — это ловушка.

Проверяем стек на прочность

Мы сказали, что стопка тарелок не бесконечная. Насколько не бесконечная — можно выяснить опытным путём. Факториал для этого не годится: он упирается в потолок `long long` уже на 21. Возьмём задачу, где `n` может быть каким угодно, — сумму `1 + 2 + ... + n`:

```
long depth = 0;                 /* текущая глубина вложенности */

long long sum_to(int n)
{
    depth++;                    /* легла ещё одна «тарелка» на стек */
    if (depth % 10000 == 0) {
        printf("глубина %ld\n", depth);
        fflush(stdout);         /* печатаем сразу, а не в буфер:
                                   после падения буфер пропадёт */
    }
    if (n == 0) {               /* база */
        return 0;
    }
    return n + sum_to(n - 1);   /* шаг */
}
```

Здесь одна новая функция — **`fflush`** из `<stdio.h>`. Сигнатура: `int fflush(FILE *stream)`. Дело в том, что `printf` не пишет на экран сразу: он копит текст в **буфере** и вываливает его пачкой, когда буфер заполнится или программа нормально завершится. Но наша программа завершится **не** нормально — она рухнет. Всё, что успело осесть в буфере, пропадёт вместе с ней, и мы не увидим последние строки. `fflush` приказывает вытолкнуть буфер немедленно. `stdout` — стандартный вывод, тот самый экран.

Программа целиком — `code/09-depth.c`, `main` в ней просит посчитать сумму до миллиона. Собираем **без оптимизации**, ключом `-O0` (почему — через минуту), и запускаем:

```
cc -std=c11 -Wall -Wextra -O0 09-depth.c -o 09-depth
./09-depth
считаем 1+2+...+1000000 рекурсией
глубина 10000
глубина 20000
глубина 30000
```

```
глубина 40000
глубина 50000
глубина 60000
глубина 70000
глубина 80000
глубина 90000
глубина 100000
глубина 110000
глубина 120000
глубина 130000
глубина 140000
глубина 150000
глубина 160000
глубина 170000
```

И всё. Никакой строки `сумма = ...` — программа оборвалась чуть дальше 170 000 вызовов. Спросим у оболочки код возврата последней команды:

```
echo $?
139
```

Когда программу убивает сигнал (так система сообщает процессу о беде), оболочка показывает код возврата `128 + номер сигнала`. `139` — это `128 + 11`, а сигнал номер 11 называется `SIGSEGV` (segmentation fault, «нарушение сегментации»): программа полезла в память, которая ей не принадлежит. Именно это и есть переполнение стека: тарелки кончились, а функция положила ещё одну.

Если хочется точную границу — подбери её вручную, меняя число в `main`. На моей машине `sum_to(174500)` считается, а `sum_to(175000)` уже падает.

Хочешь услышать диагноз словами, а не кодом возврата, — собери с санитайзером адресов:

```
cc -std=c11 -Wall -Wextra -O0 -g -fsanitize=address 09-depth.c -o 09-d-san
./09-d-san
```

Первые строки «глубина» те же, что и раньше, а хвост вывода такой (длинную строку с `ERROR` я перенёс, адреса у тебя будут другие):

```
глубина 160000
глубина 170000
AddressSanitizer:DEADLYSIGNAL
=================================================================
```

```
==57326==ERROR: AddressSanitizer: stack-overflow on address 0x7ff7b7624ff4
(pc 0x0001080db7d8 bp 0x7ff7b7625000 sp 0x7ff7b7624fe0 T0)
    #0 0x0001080db7d8 in sum_to 09-depth.c:23

SUMMARY: AddressSanitizer: stack-overflow 09-depth.c:23 in sum_to
==57326==ABORTING
```

Прямым текстом: `stack-overflow`. Строка `#0 ... in sum_to 09-depth.c:23` говорит, где это случилось: в функции `sum_to`, строка 23 файла. Это открывающая скобка `{` сразу после заголовка `long long sum_to(int n)` — то самое место, где функция при входе кладёт на стек свою тарелку. Очередная тарелка уже не влезла. `ABORTING` — санитайзер сам завершает программу, поэтому код возврата здесь будет другой (134, сигнал 6 — `SIGABRT`, «аварийная остановка»).

Сойдётся ли это с теорией? Размер стека главной функции на macOS и Linux по умолчанию 8 мегабайт — проверь командой `ulimit -s`, она печатает размер в килобайтах:

```
ulimit -s
8192
```

Восемь мегабайт — это `8 · 1024 · 1024 = 8 388 608` байт. Делим на 175 000 вызовов — примерно 48 байт на один вызов. Ровно столько и уходит на «тарелку» у `sum_to` без оптимизации: 8 байт адреса возврата, 8 байт на служебный указатель кадра и 32 байта под параметр `n` и промежуточные значения (компилятор округляет кадр до кратного 16 — так положено на x86-64). Я проверил это, заглянув в машинный код командой `objdump -d`. А `8 388 608 / 48 = 174 762` — граница, которая попадает точно между 174 500 и 175 000. Сходится. (На другом процессоре или компиляторе кадр может быть другого размера, и граница сдвинется.)

А теперь фокус: соберём то же самое с `-O2`

Строк «глубина» теперь будет сто штук, поэтому попросим показать только последние три — для этого есть команда `tail -3`, а вертикальная черта `|` передаёт ей вывод программы:

```
cc -std=c11 -Wall -Wextra -O2 09-depth.c -o 09-depth-fast
./09-depth-fast | tail -3
глубина 990000
глубина 1000000
сумма = 500000500000
```

Миллион вызовов — и никакого падения. Что произошло?

Компилятор посмотрел на `sum_to` и переписал её в цикл. Настоящих рекурсивных вызовов в машинном коде не осталось ни одного (это видно через `objdump -d`), стек не растёт.

Тут пригодится ещё один термин. Рекурсия называется **хвостовой** (tail recursion), если вызов себя — самое последнее действие функции и его результат сразу возвращается, без всякой доработки. Например:

```
long long sum_acc(int n, long long acc)
{
    if (n == 0) {
        return acc;
    }
    return sum_acc(n - 1, acc + n);   /* вызов — последнее действие */
}
```

Здесь сумма копится во втором параметре `acc` (от *accumulator* — накопитель): `sum_acc(3, 0)` зовёт `sum_acc(2, 3)`, та — `sum_acc(1, 5)`, та — `sum_acc(0, 6)`, и ответ 6 возвращается наверх без изменений. Такую рекурсию превратить в цикл проще всего: вызову незачем возвращаться в старый кадр, в нём больше нечего делать, так что можно просто переиспользовать его.

Наша `sum_to` формально **не** хвостовая: после вызова ещё выполняется сложение `n + ...`. Но clang (компилятор на macOS) достаточно умён, чтобы сам завести такой накопитель: сложение можно выполнять в любом порядке, и он этим пользуется.

Из этого два вывода. Первый: не всякая рекурсия действительно ест стек — умный компилятор часть случаев спрямляет. Второй, куда важнее: **полагаться на это нельзя**. Стандарт Си такого превращения не обещает вовсе. Даже честная хвостовая `sum_acc` на миллионе падает, если собрать её с `-O0` (проверь сам — код 139), а другой компилятор или другая форма функции могут не спрямиться и на `-O2`. Пиши так, будто каждый вызов честно ложится на стек, и держи глубину разумной. Наивный `fib` из следующей части, например, никакой оптимизацией не спасёшь: там два вызова себя, и хотя бы один из них — не последнее действие, после него ещё нужен второй вызов и сложение.

Часть 2. Числа Фибоначчи и ловушка медленной рекурсии

Последовательность Фибоначчи: каждое число — сумма двух предыдущих. Начинается с 0 и 1:

`0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...`

Определение прямо просится в рекурсию:

- база: `fib(0) = 0`, `fib(1) = 1`;
- шаг: `fib(n) = fib(n-1) + fib(n-2)`.

Наивная версия

```
long long fib_slow(int n)
{
    if (n < 2) {                /* база: fib(0)=0, fib(1)=1 */
        return n;
    }
    return fib_slow(n - 1) + fib_slow(n - 2);
}
```

Разбор:

- `if (n < 2) return n;` — хитрый, но честный приём для **базы**. При `n == 0` вернётся `0`, при `n == 1` вернётся `1` — ровно то, что нужно. Два базовых случая закрыты одной строкой.
- `return fib_slow(n - 1) + fib_slow(n - 2);` — **шаг**. Здесь функция вызывает себя **дважды**. Вот эта пара вызовов и будет источником беды.

Запустим — ответы правильные:

```
Какое число Фибоначчи посчитать? 10
Рекурсия:  fib(10) = 55
Цикл:      fib(10) = 55
```

В чём проблема: дерево вызовов

Посмотрим, что происходит при вычислении `fib(5)`. Каждый вызов порождает два новых, те — ещё по два, и получается **дерево**:

```
                    fib(5)
                  /        \
            fib(4)          fib(3)
            /     \         /     \
       fib(3)   fib(2)   fib(2)   fib(1)
       /    \    /   \    /   \
  fib(2) fib(1) f(1) f(0) f(1) f(0)
   /  \
 f(1) f(0)
```

Вглядись в дерево. `fib(3)` считается **дважды**. `fib(2)` — **трижды**. `fib(1)` и `fib(0)` — много-много раз. Рекурсия пересчитывает одни и те же значения снова и снова, не помня прошлых ответов. Посчитай узлы дерева: всего 15 вызовов ради `fib(5) = 5`. Чем больше `n`, тем чудовищнее размножаются повторы: число вызовов растёт **экспоненциально** — на каждый шаг `n` оно умно-

жается примерно на 1.6 (откуда именно 1.6 — разберём ниже, когда посмотрим на числа). **Экспоненциальный рост** — это когда прибавка к `n` не прибавляет, а **умножает** работу: `n` больше на единицу — работы в 1.6 раза больше, на десять — в 1.6 в десятой степени, то есть больше чем в сто раз.

«Экспоненциально» — это не фигура речи, это можно замерить. Секундомер — `clock()` из `<time.h>`, мы подробно разобрали его в главе 7:

```
#include <time.h>

clock_t t0 = clock();                /* засекли */
long long v = fib_slow(n);
double sec = (double)(clock() - t0) / CLOCKS_PER_SEC;
```

Напомню коротко: `clock()` отдаёт потраченное процессорное время в тиках, `CLOCKS_PER_SEC` — тиков в секунде, а `(double)` нужен, чтобы деление не ушло нацело и не отрезало дробную часть: иначе 0.26 секунды превратились бы в 0.

А заодно посчитаем, **сколько раз функция вызвалась**. Для этого заведём рядом её копию со счётчиком:

```
long long calls = 0;                 /* объявлена ВНЕ всех функций */

long long fib_counted(int n)
{
    calls++;                         /* нас вызвали — отмечаем */
    if (n < 2) {
        return n;
    }
    return fib_counted(n - 1) + fib_counted(n - 2);
}
```

Переменная `calls` объявлена вне любой функции — такая называется **глобальной**. Её видят все функции файла, и живёт она всё время работы программы, а не до конца одного вызова. Обычной локальной переменной тут не обойтись: каждый новый вызов заводил бы свою, и счёт сбрасывался бы. Злоупотреблять глобальными не стоит — их может испортить кто угодно откуда угодно, — но для счётчика в учебной программе это ровно то, что нужно.

Полностью замер лежит в `code/09-fib-bench.c`. Запускаем:

```
cc -std=c11 -Wall -Wextra -O2 09-fib-bench.c -o 09-fib-bench
./09-fib-bench
  n        fib(n)      секунды        вызовов
```

```
 25         75025       0.0002         242785
 30        832040       0.0022        2692537
 35       9227465       0.0240       29860703
 40     102334155       0.2634      331160281
 45    1134903170       2.8793     3672623805

цикл: fib(45) = 1134903170 и fib(90) = 2880067194370816120 за 0.000002 с
```

Всмотрись в последний столбец. Чтобы получить `fib(45)` — число, которое ты выпишешь на бумажке за минуту, — функция вызвала сама себя **три с половиной миллиарда раз**. Не сложений, не сравнений: именно вызовов.

Абсолютные секунды у тебя будут свои, они зависят от машины. А вот **отношения** между строками воспроизведутся у всех:

- каждые +5 к `n` умножают и время, и число вызовов примерно на 11: 0.0022, 0.0240, 0.2634, 2.8793 — каждое следующее больше предыдущего в 10.9–11 раз;
- каждый +1 к `n` — умножение примерно на 1.6. Это и есть экспоненциальный рост: не «прибавляется», а «умножается».

Откуда 1.6? Посмотри на последний столбец внимательно: 242785 — это `2 · 121393 − 1`, а 121393 — это `fib(26)`. 2692537 — это `2 · fib(31) − 1`. Закономерность такая: **вызовов ровно `2 · fib(n+1) − 1`**. Доказать несложно. Обозначим число вызовов `calls(n)`. Вызов `fib_slow(n)` — это он сам плюс все вызовы внутри `fib_slow(n−1)` и `fib_slow(n−2)`:

```
calls(n) = 1 + calls(n−1) + calls(n−2),   calls(0) = calls(1) = 1
```

Прибавь к обеим частям по единице: `calls(n) + 1 = (calls(n−1) + 1) + (calls(n−2) + 1)`. Получилось то же правило, что у самих чисел Фибоначчи, — «каждое равно сумме двух предыдущих». А начало у `calls + 1` такое: 2 и 2, то есть ровно `2 · fib(1)` и `2 · fib(2)`. Раз правило и начало совпадают с удвоенными числами Фибоначчи (со сдвигом на один), то и дальше совпадёт всё: `calls(n) + 1 = 2 · fib(n+1)`. Для `fib(5)`: `2 · fib(6) − 1 = 2 · 8 − 1 = 15` — ровно столько узлов в нашем дереве.

А сами числа Фибоначчи растут так, что каждое следующее больше предыдущего почти ровно в `φ = (1 + √5) / 2 ≈ 1.618` раз (буква «фи»; это число называют **золотым сечением**). Проверь: `55 / 34 ≈ 1.618`, `89 / 55 ≈ 1.618`. Значит, и вызовы растут в 1.618 раза на каждый шаг `n`, а за пять шагов — в `1.618` в пятой степени, это `≈ 11.09`. Вот и наше «умножается на 11».

Прикинь дальше сам, умножая на 11 каждые пять шагов. `fib(50)` — уже полминуты. `fib(60)` — больше часа. `fib(80)` — года два. `fib(100)` — около тридцати тысяч лет. И всё это ради числа,

которое цикл выдаёт мгновенно: последняя строка замера показывает, что `fib(45)` и `fib(90)` вместе посчитались за две миллионные доли секунды.

Фикс: один проход циклом

Беда наивной рекурсии — в повторном счёте. Лекарство простое: считать числа **по порядку, снизу вверх**, и помнить два последних. Никаких повторов, один проход:

```
long long fib_fast(int n)
{
    long long a = 0;             /* fib(0) */
    long long b = 1;             /* fib(1) */
    for (int i = 0; i < n; i++) {
        long long next = a + b;  /* следующее число */
        a = b;                   /* сдвигаем окно вправо */
        b = next;
    }
    return a;
}
```

Разбор:

- `a` и `b` — два соседних числа Фибоначчи. Стартуем с `a = fib(0) = 0` и `b = fib(1) = 1`.
- На каждом шаге считаем `next = a + b` — следующее число. Затем **сдвигаем окно**: то, что было `b`, становится новым `a`, а `next` — новым `b`. Так пара `(a, b)` ползёт по последовательности вправо.
- После `n` шагов в `a` лежит ровно `fib(n)`. Возвращаем его.

Здесь каждое число считается **ровно один раз**. Для `fib(90)` цикл сделает 90 сложений — доли микросекунды. Работа растёт как `n`, а не как `1.6` в степени `n`: **линейная** сложность, O(n), вместо экспоненциальной. Сравни: наивная рекурсия на таком `n` не закончилась бы и за всю твою жизнь.

Полная программа — `code/09-fib.c` — считает оба способа, чтобы видеть, что ответы совпадают:

```
#include <stdio.h>

/* Наивная рекурсия: красиво, но медленно.
   База: fib(0) = 0, fib(1) = 1.
   Шаг: fib(n) = fib(n-1) + fib(n-2). */
long long fib_slow(int n)
```

```
{
    if (n < 2) {                /* база: fib(0)=0, fib(1)=1 */
        return n;
    }
    return fib_slow(n - 1) + fib_slow(n - 2);
}

/* Быстрый вариант: один проход циклом, две переменные. */
long long fib_fast(int n)
{
    long long a = 0;            /* fib(0) */
    long long b = 1;            /* fib(1) */
    for (int i = 0; i < n; i++) {
        long long next = a + b; /* следующее число */
        a = b;                  /* сдвигаем окно вправо */
        b = next;
    }
    return a;
}

int main(void)
{
    int n;
    printf("Какое число Фибоначчи посчитать? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }
    if (n < 0) {
        printf("Нужно n >= 0.\n");
        return 1;
    }
    if (n > 90) {
        printf("Слишком большое: long long переполнится при n > 90.\n");
        return 1;
    }

    /* Наивную рекурсию зовём только на малых n: на n = 50 она уже
```

```
       считает полминуты, а на n = 60 — больше часа. Не даём программе
       зависнуть у читателя. */
    if (n <= 35) {
        printf("Рекурсия:  fib(%d) = %lld\n", n, fib_slow(n));
    } else {
        printf("Рекурсия:  пропущена, при n > 35 она слишком "
               "медленная\n");
    }
    printf("Цикл:      fib(%d) = %lld\n", n, fib_fast(n));
    return 0;
}
```

Два места в `main` стоит разобрать отдельно.

Почему потолок 90, а не 92. Последнее число Фибоначчи, влезающее в `long long`, — это `fib(92) = 7540113804746346429`. Уже `fib(93)` (12200160415121876738) переваливает за потолок `long long` (9223372036854775807) — это переполнение, то есть неопределённое поведение, а на практике мусор. Но наш `fib_fast` на каждом шаге считает `next = a + b`, то есть **на одно число вперёд**: чтобы вернуть `fib(92)`, он на последнем шаге сложит `fib(93)`. Поэтому самое большое безопасное `n` для него — 91, и мы берём 90 с запасом в один шаг.

Почему `fib_slow` вызывается только до 35. Если бы мы звали её на любом допустимом `n`, программа на вводе `50` замерла бы на полминуты, а на `60` — на час, без единого слова на экране. Пользователь решил бы, что она зависла. Поэтому выше 35 мы честно говорим, что пропустили медленный вариант. Строчка `printf` тут разбита на две части в кавычках подряд:

```
printf("Рекурсия:  пропущена, при n > 35 она слишком "
       "медленная\n");
```

Две строковые константы, написанные рядом, компилятор **склеивает** в одну ещё до сборки. Приём удобный: длинный текст переносится на следующую строку, а никакого `+` для склейки, как в других языках, в Си не нужно.

Обе функции в одной программе оставлены нарочно — чтобы ты видел, что ответы совпадают, и мог сам почувствовать разницу на небольших числах.

Проверим на маленьком `n`, где обе версии быстрые:

```
$ cc -std=c11 -Wall -Wextra -O2 code/09-fib.c -o fib
$ ./fib
Какое число Фибоначчи посчитать? 10
```

```
Рекурсия:  fib(10) = 55
Цикл:      fib(10) = 55
```

А теперь большое `n` — то самое, на котором наивная рекурсия сдалась бы:

```
$ ./fib
Какое число Фибоначчи посчитать? 90
Рекурсия:  пропущена, при n > 35 она слишком медленная
Цикл:      fib(90) = 2880067194370816120
```

Цикл выдал девяностое число Фибоначчи мгновенно. Оно длиной в девятнадцать цифр, и до потолка `long long` осталось всего три шага: `fib(92)` ещё влезает, `fib(93)` — уже нет.

Вывод: рекурсия — красивый инструмент, но не всегда правильный. Если подзадачи **перекрываются** (как ветки дерева Фибоначчи), наивная рекурсия катастрофически медленна. Иногда её спасают, **запоминая** уже посчитанные ответы (это называется мемоизация), но часто, как здесь, проще и честнее обычный цикл.

Часть 3. Ханойские башни

Теперь задача, ради которой рекурсию и любят. Есть три стержня — `A`, `B`, `C`. На стержне `A` надета пирамидка из `n` дисков: внизу самый большой, выше — всё меньше. Нужно перенести всю башню на стержень `C`. Правила:

- за один ход переносим **только один** диск (верхний с какого-то стержня);
- **больший диск нельзя класть на меньший** — никогда.

Циклом это писать мучительно. А рекурсией — почти само собой.

Разбираем руками

Допустим, мы умеем переносить `n–1` дисков куда угодно (пока поверим в это на слово — это и есть рекурсивная вера). Тогда перенести `n` дисков с `A` на `C` можно так:

1. Перенести верхние `n–1` дисков с `A` на `B` (стержень `C` пока свободен — используем его как промежуточный).
2. Перенести самый большой, оставшийся диск с `A` на `C`. Один ход.
3. Перенести `n–1` дисков с `B` на `C` (теперь промежуточный — `A`).

Всё. Большая задача («перенести `n`») свелась к двум таким же, но меньшим («перенести `n–1`») плюс один ход в середине. Это **шаг**. А **база** — `n == 0`: переносить нечего, ничего не делаем.

Заметь, как ловко меняются роли стержней. В пункте 1 промежуточный — `C`, в пункте 3 — `A`. Поэтому функции удобно передавать три стержня: откуда (`from`), куда (`to`) и через какой (`via`).

Пишем код

```c
void hanoi(int n, char from, char to, char via)
{
    if (n == 0) {                /* база: переносить нечего */
        return;
    }
    hanoi(n - 1, from, via, to);  /* верхние n-1 уводим на промежуточный */
    printf("Переложить диск %d с %c на %c\n", n, from, to);
    hanoi(n - 1, via, to, from);  /* возвращаем n-1 на цель */
}
```

Разбор:

- `void hanoi(...)` — функция ничего не возвращает (`void`), она просто печатает ходы. Параметры — это **имена стержней** типа `char`: `from` (откуда), `to` (куда), `via` (промежуточный).
- `if (n == 0) return;` — **база**. Ноль дисков — пустой ход, выходим сразу. `return;` без значения, потому что тип `void`.
- `hanoi(n - 1, from, via, to);` — шаг 1. Уводим `n-1` дисков с `from` на `via`. Заметь: на месте «куда» теперь стоит `via`, а «через» — старая цель `to`. Роли поменялись.
- `printf("Переложить диск %d с %c на %c\n", n, from, to);` — шаг 2. Печатаем единственный ход для самого большого диска: с `from` на `to`. Спецификатор `%c` печатает один символ — имя стержня.
- `hanoi(n - 1, via, to, from);` — шаг 3. Переносим `n-1` дисков с `via` на `to`, используя `from` как промежуточный.

Полная программа — `code/09-hanoi.c`:

```c
#include <stdio.h>

/* Перенести n дисков со стержня from на стержень to,
   используя via как промежуточный.
   База: 0 дисков — ничего не делаем.
   Шаг: n-1 на via, большой на to, n-1 с via на to. */
void hanoi(int n, char from, char to, char via)
{
    if (n == 0) {                /* база: переносить нечего */
        return;
```

```
    }
    hanoi(n - 1, from, via, to);  /* верхние n-1 уводим на промежуточный */
    printf("Переложить диск %d с %c на %c\n", n, from, to);
    hanoi(n - 1, via, to, from);  /* возвращаем n-1 на цель */
}

int main(void)
{
    int n;
    printf("Сколько дисков? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }
    if (n < 0) {
        printf("Нужно n >= 0.\n");
        return 1;
    }
    if (n > 30) {
        printf("Больше 30 не надо: это уже миллиард ходов.\n");
        return 1;
    }

    hanoi(n, 'A', 'C', 'B');      /* с A на C, B — запасной */
    return 0;
}
```

В `main` мы вызываем `hanoi(n, 'A', 'C', 'B')` — перенести всё с `A` на `C`, а `B` пусть будет запасным. Буквы стержней пишем в одинарных кавычках: `'A'` — это символьная константа типа `char` (см. главу 2).

Проверка `n > 30` — защита от слишком большого ввода. Сколько ходов нужно для `n` дисков, посчитаем чуть ниже, а пока поверь: для 30 дисков это уже миллиард строк на экране. А на совсем больших `n` программа даже не начала бы печатать: чтобы сделать первый ход, `hanoi` спускается вглубь на `n` уровней, и на вводе вроде `1000000` стек кончается сразу — я проверил, падение с кодом 139, как у `sum_to`.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/09-hanoi.c -o hanoi
$ ./hanoi
Сколько дисков? 3
Переложить диск 1 с A на C
Переложить диск 2 с A на B
Переложить диск 1 с C на B
Переложить диск 3 с A на C
Переложить диск 1 с B на A
Переложить диск 2 с B на C
Переложить диск 1 с A на C
```

Семь ходов — и башня из трёх дисков на месте. Пройдись по списку диском в руке (или монетками разного размера): ни разу больший не ляжет на меньший. Магия в том, что мы **не расписывали** эти семь ходов — мы лишь объяснили компьютеру правило «`n` = `n-1` + один + `n-1`», а рекурсия развернула его сама.

Сколько получается ходов

Число ходов для `n` дисков — это $2^n - 1$ (два в степени `n` минус один). Докажем это словами, без формул сверх школьных.

Наша программа, чтобы перенести `n` дисков, дважды переносит `n-1` и делает один ход между ними. Значит, `ходов(n) = 2 · ходов(n-1) + 1`, а `ходов(0) = 0`. Разворачиваем: 0, 1, 3, 7, 15, 31 — каждый раз на единицу меньше степени двойки. Почему так будет всегда? Прибавь к обеим частям равенства единицу: `ходов(n) + 1 = 2 · (ходов(n-1) + 1)`. Выходит, что величина `ходов + 1` при каждом новом диске ровно **удваивается**. Начинается она с `ходов(0) + 1 = 1`, после `n` удвоений становится 2^n. Значит, `ходов(n) =` $2^n - 1$.

А можно ли быстрее, если хитрить? Нельзя, и вот почему. Самый большой диск обязан хотя бы раз сдвинуться с `A`. В этот момент на `A` над ним не должно быть ничего, а стержень, куда он едет, должен быть пуст (больший диск ни на кого класть нельзя). Значит, все остальные `n-1` дисков в этот момент лежат на третьем стержне — их туда надо было сначала перенести, а это не меньше `ходов(n-1)`. Теперь посмотри на **последний** переезд большого диска — на `C`. В этот момент остальные `n-1` дисков снова все на другом стержне, и после него их надо перенести на `C` — снова не меньше `ходов(n-1)`. Вместе с самим переездом итого не меньше `2 · ходов(n-1) + 1` — ровно столько, сколько делает наша программа. Так что $2^n - 1$ — не просто число ходов нашего решения, а наименьшее возможное.

Проверять это глазами не нужно — пусть посчитает терминал. Программа печатает по одной строке на ход, а команда `wc -l` считает строки в том, что ей передали. Вертикальная черта `|` соединяет одно с другим:

```
echo 3 | ./09-hanoi | wc -l
       7
echo 4 | ./09-hanoi | wc -l
      15
echo 10 | ./09-hanoi | wc -l
    1023
echo 20 | ./09-hanoi | wc -l
 1048575
```

`echo 3` подставляет тройку вместо ввода с клавиатуры, поэтому вопрос «Сколько дисков?» отвечается сам. Числа сходятся: 7, 15, 1023, 1048575 — это 2^3-1, 2^4-1, $2^{10}-1$, $2^{20}-1$.

Рост тут злой. Двадцать дисков — миллион ходов, шестьдесят четыре (как в легенде про монахов, перекладывающих башню Брахмы) — `2^64 - 1`, примерно 18 миллиардов миллиардов. Даже по миллиону ходов в секунду это больше полумиллиона лет. Так что для проверки бери небольшие `n`.

Заметь важное: **глубина** рекурсии в Ханое маленькая — одновременно на стеке лежит не больше `n + 1` кадров `hanoi` (от `n` дисков до нуля). А вот **число вызовов** огромное: каждый вызов с `n > 0` порождает два вызова с `n-1`, и всего их выходит `2^(n+1) - 1` — вдвое больше, чем ходов (половина вызовов — с `n == 0`, они сразу возвращаются). Для 20 дисков это 21 уровень стека и два миллиона вызовов. Стек при разумных `n` не переполнится, а вот времени уйдёт сколько угодно.

Частые ошибки

Нет базы (или до неё не дойти). Если забыть условие остановки или сделать так, что аргумент не уменьшается, рекурсия будет вызывать себя вечно и упадёт со **stack overflow**. Всегда проверяй: есть ли база и **приближает** ли её каждый шаг? `factorial(n-1)` приближает (аргумент падает), а вот `factorial(n)` внутри `factorial(n)` — нет, это бесконечность.

Рекурсия там, где подзадачи повторяются. Наивный `fib` — учебный пример катастрофы: экспоненциальное число повторных вычислений. Прежде чем писать рекурсию «в лоб», спроси себя: не будут ли ветки считать одно и то же? Если да — цикл или мемоизация.

Неверный спецификатор для `long long`. Печатать `long long` нужно через `%lld`, а не `%d`. С `%d` получишь мусор или предупреждение компилятора.

Переполнение результата. Факториалы и числа Фибоначчи растут взрывообразно. `20!` — предел `long long` для факториала, `fib(92)` — для Фибоначчи. Дальше — переполнение знакового числа, неопределённое поведение; на практике число молча «заворачивается» в мусор. Мы отсекаем такие `n` заранее.

Путаница со стержнями в Ханое. Порядок аргументов важен: `hanoi(n-1, from, via, to)` и `hanoi(n-1, via, to, from)`. Перепутаешь `via` и `to` — и алгоритм начнёт класть большие диски на маленькие. Держи в голове: первый под-вызов освобождает цель, второй — её заполняет.

Путают глубину и число вызовов. Это разные вещи. У факториала глубина `n` и вызовов `n` — совпало. У Ханоя глубина всего `n + 1`, а вызовов `2^(n+1) - 1`: стек в порядке, время — нет. У наивного `fib` глубина `n`, а вызовов `2 · fib(n+1) - 1`, и растут они как 1.6^n. Переполнение стека грозит от **глубины**, бесконечное ожидание — от **числа вызовов**.

Рассчитывают, что компилятор всё спрямит. Мы видели: с `-O2` наш `sum_to` на миллион уровней отработал без падения, потому что компилятор превратил рекурсию в цикл. Но это подарок, а не гарантия: стандарт Си такого не обещает даже для хвостовой рекурсии. Чуть другая форма функции, другой уровень оптимизации, другой компилятор — и подарок испарится. Пиши так, будто каждый вызов честно ложится на стек.

Печатают перед падением и ничего не видят. Если программа рухнет, всё, что `printf` успел сложить в буфер, пропадёт вместе с ней. Отлаживая падение выводом на экран, ставь после `printf` вызов `fflush(stdout)` — иначе последние (самые интересные) строки до тебя не доедут.

Проверяем: все программы главы

В главе пять файлов, и собираются они одной командой:

```
cc -std=c11 -Wall -Wextra -O2 файл.c -o имя
```

- `09-factorial.c` — рекурсивный факториал;
- `09-fib.c` — Фибоначчи двумя способами;
- `09-hanoi.c` — Ханойские башни;
- `09-fib-bench.c` — замер, во что обходится наивная рекурсия;
- `09-depth.c` — переполнение стека. Единственное исключение: его надо собирать с `-O0`, иначе падения не будет.

Упражнения

1. **Сумма цифр рекурсивно.** Напиши `int digit_sum(int n)`, которая возвращает сумму цифр числа `n >= 0`. Подсказка: последняя цифра — это `n % 10`, а всё остальное число

— `n / 10`. База — однозначное число (`n < 10`), его сумма цифр — оно само. Шаг: `n % 10 + digit_sum(n / 10)`. *Ответ:* `digit_sum(12345)` = 15, `digit_sum(7)` = 7, `digit_sum(0)` = 0. Для отрицательных `n` эта функция не годится: в Си `-123 % 10` равно `-3` (см. главу 4).

2. **Степень рекурсивно.** Напиши `long long power(int base, int exp)`, возводящую `base` в степень `exp >= 0`. База: любое число в степени 0 равно 1 (и `0` в нулевой тоже договоримся считать единицей). Шаг: `base * power(base, exp - 1)`. *Ответ:* `power(2, 10)` = 1024, `power(3, 0)` = 1, `power(2, 62)` = 4611686018427387904 — а `power(2, 63)` уже не влезет в `long long`.

3. **Обратный отсчёт.** Функция `void countdown(int n)` печатает числа от `n` до 1, а потом слово «Пуск!». Сделай это рекурсией без цикла. Где здесь база? *Ответ:* база — `n == 0`: печатаем «Пуск!» и выходим. Шаг — напечатать `n` и вызвать `countdown(n - 1)`. Для `countdown(3)` на экране по строке: `3`, `2`, `1`, `Пуск!`.

4. **Сумма от 1 до n.** Напиши `long long sum_to(int n)`, возвращающую `1 + 2 + ... + n` через рекурсию. Сравни с формулой `n*(n+1)/2`. *Ответ:* для `n = 100` обе дают 5050. Формулу выводят тем же приёмом, что в главе 8: сложи сумму саму с собой задом наперёд — `n` столбиков по `n + 1`.

5. **Фибоначчи: посчитай вызовы сам.** Не подглядывая в `09-fib-bench.c`, заведи глобальный счётчик и увеличивай его в начале `fib_slow`. Запусти для `n = 10`, `20`, `30`. Должно получиться 177, 21891 и 2692537 — сверься с формулой `2 · fib(n+1) - 1`, которую мы вывели в этой главе: `fib(11) = 89`, `fib(21) = 10946`, `fib(31) = 1346269`.

6. **Ханой с подсчётом ходов.** Доработай `hanoi` так, чтобы она ещё и считала общее число ходов, и сравни результат с $2^n - 1$. *Ответ:* заведи глобальный `long long moves = 0;` и делай `moves++` там, где сейчас `printf`. Для 5 дисков — 31, для 10 — 1023, для 0 — 0.

7. **Найди свою границу стека.** Возьми `09-depth.c`, собери с `-O0` и подбирай число в `main`, пока программа не начнёт падать. Раздели 8 мегабайт на найденную глубину — сколько байт уходит на один вызов? Потом добавь в `sum_to` большой локальный массив (`int buf[100]; buf[0] = n;`) и посмотри, как граница обвалится. *У меня:* без массива — 48 байт на вызов и граница около 174 700. С массивом кадр вырос до 464 байт (400 байт массива плюс служебное), и граница упала почти в десять раз: 18 000 вызовов проходят, 19 000 — уже падение.

8. **Мемоизация.** Заведи массив `long long memo[95]`, заполненный нулями. В начале `fib_slow` проверяй: если `memo[n]` не ноль — сразу возвращай его, не считая. Перед `return` записывай ответ в `memo[n]`. Запусти для `n = 45` и сравни со временем из замера. Это тот самый приём, о котором мы говорили, — рекурсия, которая помнит. *Ответ:* `fib(45) = 1134903170` за пару микросекунд вместо трёх секунд, а вызовов вместо трёх с половиной миллиардов — 89: каждое `fib(k)` считается один раз, остальные вызовы сразу отдают готовое из `memo`.

Массив глобальный (объявлен вне функций) — такие обнуляются сами, ещё до запуска `main`.

Что мы получили

Теперь ты владеешь рекурсией — вторым после цикла способом повторять действия. Главное, что нужно унести:

- у любой рекурсии есть **база** (где останавливаемся) и **шаг** (где сводим задачу к меньшей); без базы — `stack overflow`;
- вызовы складываются на **стек**, как стопка тарелок, и снимаются в обратном порядке — на этом строится «раскрутка» ответа;
- стек не бесконечен, и мы это увидели своими глазами: около 175 000 вызовов на 8 мегабайтах, дальше — падение с сигналом `SIGSEGV` и код возврата 139;
- рекурсия бывает **изящной** (факториал, Ханой), а бывает **ловушкой** (наивный Фибоначчи: три с половиной миллиарда вызовов ради `fib(45)`) — и тогда выручает цикл;
- **глубина** и **число вызовов** — разные величины: первая грозит стеком, вторая временем;
- некоторые задачи (Ханойские башни) рекурсией решаются почти даром, а циклом — мучительно.

И ещё одно, не про рекурсию, а про ремесло. В этой главе ты трижды не поверил на слово и полез проверять: замерил секундомером, что наивный `fib` растёт экспоненциально; довёл программу до настоящего падения, чтобы увидеть границу стека; посчитал ходы Ханоя через `wc -l` и сверил с формулой. Так и надо относиться к любому утверждению про скорость — включая те, что написаны в этой книге.

В следующей главе мы вернёмся к строкам и займёмся **строковыми алгоритмами**: измерим длину без `strlen`, перевернём строку, проверим её на палиндром и научимся убирать тот самый `'\n'`, который оставляет `fgets`.

Глава 10. Строковые алгоритмы: длина, переворот, палиндром, слова, анаграммы

Строка в Си — это массив символов с меткой конца `'\0'` (глава 3). А раз это массив, над ним можно гонять циклы и считать что угодно: длину, число слов, симметрию, состав букв. В этой главе мы напишем четыре маленьких, но настоящих строковых алгоритма — не вызывая готовых функций, а собирая логику руками.

Заодно познакомимся с `<ctype.h>` — набором функций «а это буква? а цифра? а пробел?» — и с одной ловушкой, которая ждёт там всех новичков и половину не-новичков.

Что мы сделаем

Четыре программы, каждая в своём файле:

- `10-reverse.c` — своя функция длины строки и переворот строки на месте;
- `10-palindrome.c` — читается ли фраза одинаково с двух концов, если не обращать внимания на регистр и знаки препинания;
- `10-words.c` — подсчёт слов во фразе и длина самого длинного слова;
- `10-anagram.c` — состоят ли две фразы из одних и тех же букв.

Нового синтаксиса тут почти нет — массивы, циклы, `if` и `char`. Вся соль в **идеях алгоритмов** и в краевых случаях: пустая строка, строка из одного символа, три пробела подряд, заглавные буквы.

Сразу предупреждение, к которому мы вернёмся с живым примером: **вводим только латиницу, цифры, пробелы и знаки препинания**. Почему русские буквы ломают посимвольные алгоритмы — покажу через пару страниц, когда у нас будет работающая программа, на которой это видно.

Часть 1. Длина и переворот строки

Разбираем задачу руками

Возьмём слово `hello`. Перевернуть его — значит получить `olleh`. Как это сделать «на бумаге»? Поставим палец слева (на `h`) и палец справа (на `o`) и **поменяем их местами**:

```
h e l l o     меняем края: h и o
^       ^


o e l l h     сдвинули пальцы к центру, меняем e и l
  ^   ^


o l l e h     пальцы встретились на середине — готово
    ^
```

Когда пальцы встретились (или перекрестились) — всё, строка перевёрнута. Средняя буква (если длина нечётная) остаётся на месте, и правильно: двигать её некуда. Этот приём — **два индекса с концов** — мы будем использовать постоянно: и здесь, и в палиндроме, и потом в сортировках.

Но прежде чем переворачивать, надо знать **длину** строки: где правый край. Длину можно взять из `strlen`, но давай напишем свою — так ты раз и навсегда поймёшь, как Си «видит» конец строки.

Шаг 1. Своя функция длины

Строка заканчивается невидимым символом `'\0'` (нулевой байт). Значит, длина — это сколько символов лежит **до** него. Идём циклом, пока не упрёмся в `'\0'`, и считаем шаги:

```c
int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}
```

Разбор:

- `const char *s` — параметр-строка. `char *` — указатель на символы, то есть адрес начала строки (вспомни главу 3: имя массива — это адрес его первого элемента). Слово `const` —

обещание «я эту строку менять не буду, только читаю». Компилятор за этим проследит: попытка написать `s[0] = 'x'` внутри такой функции не скомпилируется.

- `s[len]` — `len`-й символ строки. К указателю на символы можно обращаться по индексу точно так же, как к массиву.
- `while (s[len] != '\0')` — крутимся, **пока** текущий символ не нулевой. `!=` — «не равно». `'\0'` в одинарных кавычках — это один символ с кодом 0, метка конца строки.
- На каждом шаге `len++` — продвигаемся дальше. Когда `s[len]` окажется `'\0'`, цикл встанет, а в `len` будет ровно число символов до него. Его и возвращаем.

Сам `'\0'` в длину **не входит** — мы останавливаемся на нём, не посчитав. Так же работает и настоящий `strlen` из `<string.h>`. Одно отличие: `strlen` возвращает не `int`, а `size_t` — беззнаковый целый тип, специально придуманный под размеры (его печатают через `%zu`, см. главу 3). Нам в маленьких примерах хватит `int`, но знай, что «настоящая» подпись такая:

```
size_t strlen(const char *s);
```

Шаг 2. Наивный переворот — и сюрприз при запуске

Прочитаем строку через `fgets` и сразу перевернём. Сам приём с двумя индексами мы уже придумали, так что напишем в лоб и запустим:

```
#include <stdio.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

int main(void)
{
    char line[100];
    printf("Введи строку (латиницей): ");
    fgets(line, sizeof line, stdin);

    int len = my_strlen(line);
    printf("Длина: %d\n", len);
```

```
    int i = 0, j = len - 1;
    while (i < j) {
        char tmp = line[i];
        line[i] = line[j];
        line[j] = tmp;
        i++;
        j--;
    }

    printf("Перевёрнутая: %s\n", line);
    return 0;
}
```

- `char line[100]` — буфер под строку: сто ячеек типа `char`.
- `fgets(line, sizeof line, stdin)` — читает строку с клавиатуры. Три аргумента: куда класть, сколько байт максимум можно занять, откуда читать (`stdin` — стандартный ввод, клавиатура). `sizeof line` — это 100, размер массива в байтах; так `fgets` физически не сможет вылезти за границу. Именно поэтому `fgets`, а не `gets`: у `gets` размера нет, и любая длинная строка затирает соседнюю память. `gets` настолько плох, что в стандарте C11 его убрали совсем.
- `char tmp` — временная коробка для обмена. Чтобы поменять два значения местами, одно надо где-то придержать, иначе затрёшь (тот же приём `swap`, что в главе 3, только для символов).

Собираем и вводим `hello`:

```
$ cc -std=c11 -Wall -Wextra -O2 naive-reverse.c -o naive-reverse
$ ./naive-reverse
Введи строку (латиницей): hello
Длина: 6
Перевёрнутая:
olleh
```

Два странных факта разом. Во-первых, длина слова `hello` почему-то **6**, а не 5. Во-вторых, результат `olleh` уехал на новую строку, а перед ним пустая.

Шаг 3. Виноват `'\n'` от fgets — убираем его

Когда ты нажимаешь Enter, в строку попадает символ перевода строки `'\n'`, и `fgets` его **не выбрасывает**, а честно кладёт в буфер. То есть в `line` лежит не `hello`, а `hello\n` — шесть символов. Вот и «Длина: 6».

Дальше понятно: переворот шести символов поставил `'\n'` в самое начало, получилось `\nolleh`. `printf` напечатал перевод строки, потом `olleh`. Отсюда и пустая строка.

Лечится это всегда одинаково: после `fgets` смотрим на последний символ, и если это `'\n'` — затираем его на `'\0'`. Вынесем в функцию, она нам понадобится во всех четырёх программах:

```
/* Срезает хвостовой '\n', который оставляет после себя fgets. */
void chomp(char *s)
{
    int len = my_strlen(s);
    if (len > 0 && s[len - 1] == '\n') {
        s[len - 1] = '\0';
    }
}
```

- `void` — функция ничего не возвращает. Она меняет саму строку: ей передают адрес, поэтому правки видны снаружи (глава 3).
- `char *s` — здесь без `const`: мы как раз собираемся строку менять.
- `len > 0` — защита от пустой строки. Если в `s` сразу `'\0'`, длина 0, и без этой проверки мы полезли бы в `s[-1]` — чтение и запись за левой границей массива.
- `s[len - 1]` — последний реальный символ (индексы с нуля, поэтому последний — это `len - 1`).
- `s[len - 1] = '\0';` — ставим метку конца на место `'\n'`. Теперь для всех функций строка кончается раньше, а `'\n'` как бы исчез. Байт-то на месте, просто до него никто уже не дойдёт.

А почему `&&`, а не два вложенных `if`? `&&` — это «и»: условие верно, только если верны обе половины. И у него есть важное свойство — **короткое замыкание**: если левая половина ложна, правую Си даже не считает. Поэтому при `len == 0` выражение `s[len - 1]` вообще не вычисляется. Порядок половин здесь не случаен, менять их местами нельзя.

Этот кусочек тебе понадобится после **каждого** `fgets`. Запомни его как рефлекс.

Полная программа: `10-reverse.c`

```c
#include <stdio.h>

/* Своя длина строки: считаем символы до '\0'. */
int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

/* Срезает хвостовой '\n', который оставляет после себя fgets. */
void chomp(char *s)
{
    int len = my_strlen(s);
    if (len > 0 && s[len - 1] == '\n') {
        s[len - 1] = '\0';
    }
}

int main(void)
{
    char line[100];

    printf("Введи строку (латиницей): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;                       /* ввода не было */
    }
    chomp(line);

    int len = my_strlen(line);

    /* Переворот на месте: меняем местами края, идём к центру. */
    int i = 0;
    int j = len - 1;
    while (i < j) {
```

```
        char tmp = line[i];
        line[i] = line[j];
        line[j] = tmp;
        i++;
        j--;
    }

    printf("Длина: %d\n", len);
    printf("Перевёрнутая: %s\n", line);
    return 0;
}
```

Что изменилось против наивной версии:

- `if (fgets(...) == NULL)` — `fgets` возвращает адрес буфера при успехе и `NULL`, если читать было нечего: файл кончился или пользователь нажал Ctrl+D. Тогда в `line` остаётся мусор, и работать с ним нельзя. Проверять результат ввода — хорошая привычка, мы её заводим тут.
- `chomp(line);` сразу после чтения, и только потом считаем длину.
- `while (i < j)` — крутимся, пока левый индекс левее правого. Как только они встретились в центре (`i == j`) или разошлись — конец.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 10-reverse.c -o 10-reverse
$ ./10-reverse
Введи строку (латиницей): hello
Длина: 5
Перевёрнутая: olleh
```

Длина стала 5, лишнего переноса нет. Теперь краевые случаи — их надо проверять всегда, а не «когда-нибудь потом».

Пустая строка (просто Enter):

```
$ ./10-reverse
Введи строку (латиницей):
Длина: 0
Перевёрнутая:
```

Один символ:

```
$ ./10-reverse
Введи строку (латиницей): x
Длина: 1
Перевёрнутая: x
```

Два символа:

```
$ ./10-reverse
Введи строку (латиницей): ab
Длина: 2
Перевёрнутая: ba
```

При длине 0 у нас `i = 0`, `j = -1`, условие `0 < -1` ложно — цикл не крутится ни разу, в `line[-1]` мы не лезем. При длине 1: `i = 0`, `j = 0`, условие `0 < 0` тоже ложно. Оба вырожденных случая обработались сами, без единого дополнительного `if`, — просто потому, что условие цикла написано через `<`, а не через `<=`. Напиши мы `while (i <= j)` — при `i == j` символ поменялся бы сам с собой (не страшно), а вот при длине 0 мы бы шагнули в `line[-1]`.

Обещанная демонстрация: почему только латиница

Теперь у нас есть рабочая программа, и на ней видно, что происходит с русским текстом. Вводим `привет` — шесть букв:

```
$ ./10-reverse
Введи строку (латиницей): привет
Длина: 12
```

Третью строку, «Перевёрнутая: …», я сюда не переносил: вместо слова там кракозябры, и как именно они выглядят, зависит от терминала. Чуть ниже мы посмотрим на неё побайтно.

Длина 12, а не 6. Дело в кодировке. Современные системы хранят текст в **UTF-8**, где латинская буква, цифра, пробел и знак препинания занимают ровно **1 байт**, а русская буква — **2 байта**. Посмотрим на байты слова `привет`:

```
$ printf 'привет' | od -An -tx1
 d0  bf  d1  80  d0  b8  d0  b2  d0  b5  d1  82
```

`od` печатает байты числами в шестнадцатеричном виде (`-tx1` — «по одному байту в шестнадцатеричном»), `-An` убирает столбик со смещениями. Двенадцать чисел — это шесть пар. `d0 bf` — буква «п», `d1 80` — «р», и так далее. Для программы на Си строка — просто байты, никаких «букв» она не знает. `my_strlen` честно насчитал 12.

Посмотрим теперь на байты, которые программа напечатала (берём хвост вывода — 14 последних байт):

```
$ printf 'привет\n' | ./10-reverse | tail -c 14 | od -An -tx1
           20  82  d1  b5  d0  b2  d0  b8  d0  80  d1  bf  d0  0a
```

Первый байт `20` — это пробел после двоеточия в «Перевёрнутая: », последний `0a` — перевод строки. Между ними наши двенадцать байт, только в обратном порядке. Пары развалились: было `d0 bf`, стало `bf d0`. Таких пар в UTF-8 не бывает, поэтому терминал печатает вместо букв вопросительные знаки и случайные закорючки — то, что в народе зовут кракозябрами.

Вывод простой: **в этой главе вводим только латиницу**. Русский текст годится для сообщений (`printf` печатает байты как есть, ничего не разбирая), но не для посимвольной обработки. Полноценная работа с Юникодом — отдельная большая тема, она за рамками книги.

Часть 2. Палиндром

Палиндром — строка, которая читается одинаково слева направо и справа налево: `level`, `radar`, `noon`. А `world` — не палиндром. Строго говоря, это строка, которая совпадает со своим переворотом символ в символ. Для фраз обычно договариваются мягче: смотрят только на буквы и не различают заглавные и строчные. Сначала напишем строгую проверку, потом увидим, почему для фраз её мало.

Наивная версия: сравниваем в лоб

Приём тот же, что в перевороте — два индекса с концов, — только теперь мы ничего не меняем, а **сравниваем**. Левый символ должен совпадать с правым, второй слева — со вторым справа, и так до центра. Нашли хоть одно несовпадение — не палиндром, дальше можно не проверять.

```
    int i = 0;
    int j = my_strlen(line) - 1;
    int is_palindrome = 1;              /* пока считаем, что да */
    while (i < j) {
        if (line[i] != line[j]) {
            is_palindrome = 0;          /* нашли несовпадение */
            break;
        }
        i++;
        j--;
    }
```

- `int is_palindrome = 1;` — **флаг**: переменная-да/нет. `1` — «истина», `0` — «ложь». В Си есть тип `_Bool`, а привычные имена `bool`, `true` и `false` даёт заголовок `<stdbool.h>`. Нам хватит обычного `int`: условие `if` в Си всё равно понимает ноль как «ложь», а всё остальное как «истину». Начинаем с оптимизма: считаем фразу палиндромом, пока не доказано обратное.
- `break;` — досрочный выход из цикла. Раз нашли несовпадение, дальше проверять незачем, ответ уже известен. `break` обрывает ближайший цикл, а выполнение продолжается со строки после него.

Обвязка вокруг этого куска — та же, что в `10-reverse.c`: `fgets`, `chomp`, а в конце печать ответа, как в полной программе ниже. Собираем и гоняем на четырёх примерах:

```
$ ./naive-palindrome
Введи строку (латиницей): level
"level" — палиндром
$ ./naive-palindrome
Введи строку (латиницей): Level
"Level" — не палиндром
$ ./naive-palindrome
Введи строку (латиницей): never odd or even
"never odd or even" — не палиндром
$ ./naive-palindrome
Введи строку (латиницей): A man, a plan, a canal: Panama
"A man, a plan, a canal: Panama" — не палиндром
```

Первый ответ верный, остальные три — обидные. `Level` не палиндром только потому, что `'L'` и `'l'` — **разные символы** с разными кодами (76 и 108). `never odd or even` спотыкается на пробелах: слева первый символ `n`, справа последний `n`, но дальше пробелы стоят не симметрично. А знаменитая фраза про Панаму валится ещё и на запятых с двоеточием.

Люди-то читают палиндром иначе: они смотрят только на **буквы** и не различают заглавные со строчными. Значит, и программа должна сначала выкинуть всё лишнее, а уже потом сравнивать.

Знакомимся: `<ctype.h>`

Чтобы «выкинуть всё, что не буква», нужно уметь спрашивать: а этот символ — буква? Написать проверку руками можно:

```
if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) { ... }
```

Но в стандартной библиотеке это уже есть. Заголовок `<ctype.h>` (*character type* — тип символа) даёт целый набор:

```
int isalpha(int c);   /* буква?            'a', 'Z' -> не 0 */
int isdigit(int c);   /* цифра?            '0'..'9' -> не 0 */
int isalnum(int c);   /* буква или цифра? */
int isspace(int c);   /* пробельный?       ' ', '\t', '\n', '\r' -> не 0 */
int isupper(int c);   /* заглавная буква? */
int islower(int c);   /* строчная буква? */
int ispunct(int c);   /* знак препинания? */

int tolower(int c);   /* 'A' -> 'a', остальное возвращает как есть */
int toupper(int c);   /* 'a' -> 'A', остальное возвращает как есть */
```

Все они возвращают `int`. Функции семейства `is...` возвращают **не ноль**, если ответ «да», и ноль, если «нет». Не обязательно 1 — просто что-то ненулевое, поэтому пиши `if (isalpha(c))`, а не `if (isalpha(c) == 1)`. `tolower`/`toupper` возвращают уже сам символ (его код), причём символ, не подходящий под переделку, возвращают нетронутым: `tolower('5')` — это `'5'`.

Ловушка: почему `(unsigned char)`

А теперь главное про `<ctype.h>`. Посмотри на подпись: аргумент — **`int`**, а не `char`. Не случайно. Стандарт требует, чтобы значение аргумента было либо кодом символа в диапазоне `unsigned char` (0..255), либо `EOF` (обычно −1). Всё остальное — **неопределённое поведение**.

Где тут подвох? В том, что тип `char` в Си **может быть знаковым**, и на твоём маке он именно такой: диапазон −128..127. Так на x86 и на Apple Silicon. А вот на Linux для ARM `char` беззнаковый (0..255): стандарт оставляет выбор компилятору. Байт `0xD0` (первый байт русской «п») в такой `char` попадает как **−48**. Передашь его в `isalpha` — и передашь отрицательное число, которого там быть не должно.

`isalpha` и `tolower` внутри обычно устроены как таблица: берём код символа, лезем по этому индексу в массив признаков, читаем ответ. Отрицательный аргумент — это обращение к массиву **до его начала**. Некоторые библиотеки специально держат под таблицей запасные ячейки, и у них такой вызов «прокатывает»; другие — нет. Гадать, повезёт тебе или нет, не надо: стандарт этого не обещает.

Проверим на своей собственной таблице — ровно такой же, какую держит библиотека:

```
#include <stdio.h>
#include <ctype.h>
```

```c
int main(void)
{
    char word[] = "\xd0\xbf";   /* два байта русской буквы "п" в UTF-8 */
    int count[256] = {0};

    for (int i = 0; word[i] != '\0'; i++) {
        printf("байт %d: char = %d\n", i, word[i]);
        count[word[i]]++;       /* БЕЗ приведения к unsigned char */
    }
    printf("готово\n");
    return 0;
}
```

Компилятор ругается ещё до запуска — и правильно делает:

```
$ cc -std=c11 -Wall -Wextra -O1 -g -fsanitize=address,undefined \
     badcast.c -o badcast
badcast.c:11:14: warning: array subscript is of type 'char' [-Wchar-subscripts]
   11 |         count[word[i]]++;       /* БЕЗ приведения к unsigned char */
      |              ^~~~~~~
1 warning generated.
```

`-Wchar-subscripts` — «индекс массива имеет тип `char`». Это ровно наш случай. Запускаем — и санитайзер ловит промах (в терминале вывод программы и сообщения санитайзера перемешаны именно так):

```
$ ./badcast
байт 0: char = -48
badcast.c:11:9: runtime error: index -48 out of bounds for type 'int[256]'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior badcast.c:11:9
байт 1: char = -65
готово
```

`index -48 out of bounds` — мы прибавили единицу к памяти, лежащей за 192 байта левее нашего массива (48 ячеек по 4 байта в каждом `int`). Про второй байт (−65) санитайзер уже промолчал: об одном и том же месте в коде он сообщает один раз. Что там было? Может, другая переменная. Может, ничего важного. Может, служебные данные функции. Программа не упала — и это худший вариант: такие ошибки живут годами и всплывают случайно.

Лечение простое и универсальное: **приводи символ к `unsigned char`** перед тем, как использовать его как индекс или отдавать в `<ctype.h>`.

```
unsigned char c = (unsigned char)line[k];
if (isalpha(c)) { ... }
```

`(unsigned char)` — это **приведение типа**: «посмотри на этот байт как на беззнаковое число». Байт `0xD0` из −48 превращается в 208, и всё попадает в разрешённый диапазон 0..255.

Запомни правило целиком: **любой `char`, который идёт в `<ctype.h>` или в индекс массива-таблицы, сначала приводим к `unsigned char`**. Забыть эту скобку — самая частая необнаруженная ошибка в коде со строками.

Идея: сначала чистим, потом сравниваем

Раз сравнивать надо только буквы и без учёта регистра, разобьём задачу на два шага:

1. **Нормализация.** Проходим по строке и переписываем в новый массив только буквы, каждую — в нижнем регистре. Фраза `A man, a plan, a canal: Panama` превращается в `amanaplanacanalpanama`.
2. **Сравнение.** К чистой строке применяем ровно тот же приём двух индексов, что и раньше, — теперь ему ничто не мешает.

Так каждый шаг делает одну понятную вещь. И бонусом мы сможем напечатать промежуточный результат — читателю сразу видно, что программа сравнивала на самом деле.

Насколько большой нужен второй массив? Мы кладём в него не больше символов, чем есть в исходной строке, значит, буфера того же размера хватит с запасом. Возьмём оба по 200.

Полная программа: `10-palindrome.c`

```
#include <stdio.h>
#include <ctype.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

/* Срезает хвостовой '\n', который оставляет после себя fgets. */
```

```
void chomp(char *s)
{
    int len = my_strlen(s);
    if (len > 0 && s[len - 1] == '\n') {
        s[len - 1] = '\0';
    }
}

int main(void)
{
    char line[200];

    printf("Введи строку (латиницей): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;
    }
    chomp(line);

    /* Шаг 1: оставляем только буквы, приводя их к нижнему регистру.
     * clean не меньше line, значит переполнить его нечем. */
    char clean[200];
    int m = 0;
    for (int k = 0; line[k] != '\0'; k++) {
        unsigned char c = (unsigned char)line[k];
        if (isalpha(c)) {
            clean[m] = (char)tolower(c);
            m++;
        }
    }
    clean[m] = '\0';

    /* Шаг 2: два индекса идут навстречу друг другу. */
    int i = 0;
    int j = m - 1;
    int is_palindrome = 1;              /* пока считаем, что да */
    while (i < j) {
        if (clean[i] != clean[j]) {
            is_palindrome = 0;          /* нашли несовпадение */
```

```
            break;
        }
        i++;
        j--;
    }

    printf("Только буквы: \"%s\" (%d шт.)\n", clean, m);
    if (is_palindrome) {
        printf("\"%s\" — палиндром\n", line);
    } else {
        printf("\"%s\" — не палиндром\n", line);
    }
    return 0;
}
```

Разбор новых мест:

- `for (int k = 0; line[k] != '\0'; k++)` — обход строки без предварительного подсчёта длины: идём, пока не встретим `'\0'`. Так в Си ходят по строкам чаще всего.
- `unsigned char c = (unsigned char)line[k];` — то самое приведение. Дальше `c` можно смело отдавать и в `isalpha`, и в `tolower`.
- `clean[m] = (char)tolower(c);` — `tolower` возвращает `int`, а кладём мы в `char`. Приведение `(char)` не обязательно, компилятор сузил бы сам, — но оно говорит читателю «да, я знаю, что сужаю тип, это не описка». А если добавить к сборке флаг `-Wconversion`, без приведения вылезет `warning: implicit conversion loses integer precision: 'int' to 'char'`. Лучше сказать явно.
- `m` — счётчик букв в чистой строке. Он же индекс, куда класть следующую букву, он же в конце — длина `clean`.
- `clean[m] = '\0';` — **обязательная** строчка. Мы собрали строку руками, и метку конца тоже надо поставить руками. Забудешь — `printf` с `%s` пойдёт печатать мусор из соседней памяти, пока случайно не наткнётся на нулевой байт.
- `\"%s\"` — обратный слэш перед кавычкой говорит `printf`: «это обычная кавычка, печатай её», а не конец строки формата. Так фраза в выводе берётся в кавычки.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 10-palindrome.c -o 10-palindrome
$ ./10-palindrome
Введи строку (латиницей): level
```

```
Только буквы: "level" (5 шт.)
"level" — палиндром
```

Те три случая, на которых спотыкалась наивная версия:

```
$ ./10-palindrome
Введи строку (латиницей): Level
Только буквы: "level" (5 шт.)
"Level" — палиндром
$ ./10-palindrome
Введи строку (латиницей): never odd or even
Только буквы: "neveroddoreven" (14 шт.)
"never odd or even" — палиндром
$ ./10-palindrome
Введи строку (латиницей): A man, a plan, a canal: Panama
Только буквы: "amanaplanacanalpanama" (21 шт.)
"A man, a plan, a canal: Panama" — палиндром
```

Строка «Только буквы» отлично показывает, что именно программа сравнивала. И контрольный не-палиндром:

```
$ ./10-palindrome
Введи строку (латиницей): world
Только буквы: "world" (5 шт.)
"world" — не палиндром
```

Краевые случаи. Пустой ввод и один символ:

```
$ ./10-palindrome
Введи строку (латиницей):
Только буквы: "" (0 шт.)
"" — палиндром
$ ./10-palindrome
Введи строку (латиницей): x
Только буквы: "x" (1 шт.)
"x" — палиндром
```

И то и другое — палиндромы по определению: читать в обе стороны нечего или нечему различаться. Программа это учитывает даром: при `m <= 1` цикл `while (i < j)` не выполняется ни разу, флаг остаётся `1`.

А что со строкой, в которой букв нет вовсе?

```
$ ./10-palindrome
Введи строку (латиницей): !!!
Только буквы: "" (0 шт.)
"!!!" — палиндром
```

После чистки осталась пустая строка — значит, палиндром. Ответ спорный, но он хотя бы предсказуемый и объяснимый. Такие решения надо принимать осознанно и записывать в комментарий к функции, а не оставлять «как само получилось»: через месяц ты сам не вспомнишь, был это замысел или недосмотр.

Часть 3. Подсчёт слов

Посчитаем, сколько слов во фразе.

Наивная версия: считаем пробелы

Первая мысль почти у всех одна: слов на один больше, чем пробелов.

```
    int spaces = 0;
    for (int i = 0; line[i] != '\0'; i++) {
        if (line[i] == ' ') {
            spaces++;
        }
    }
    printf("Слов: %d\n", spaces + 1);
```

На аккуратной фразе работает:

```
$ ./naive-words
Введи фразу (латиницей): the quick brown fox
Слов: 4
```

А теперь то же самое, но с лишними пробелами — по краям и между словами:

```
$ ./naive-words
Введи фразу (латиницей):   the   quick brown   fox
Слов: 12
```

Двенадцать слов вместо четырёх. И на пустой строке тоже мимо:

```
$ ./naive-words
Введи фразу (латиницей):
Слов: 1
```

Ноль пробелов плюс один — получилось одно слово, хотя фраза пустая. Идея «пробелов плюс один» верна только для идеально причёсанного текста, а такой текст пользователь вводит примерно никогда.

Идея: ловим начало слова

Считать надо не разделители, а **начала слов**. Слово начинается там, где после разделителя (или в самом начале строки) идёт не-разделитель.

Заведём флаг `in_word` — «мы сейчас внутри слова или нет» — и пройдём по строке символ за символом:

- встретили пробельный символ → мы вне слова (`in_word = 0`);
- встретили не-пробельный и до этого были вне слова → это **начало нового слова**: прибавляем счётчик и ставим `in_word = 1`;
- встретили не-пробельный, но уже были внутри слова → ничего не делаем, просто продолжаем то же слово.

Так каждое слово считается ровно один раз — в момент своей первой буквы. Сколько бы пробелов ни стояло вокруг, лишних единиц не набежит. Пустая строка даст ноль: цикл просто не выполнится.

Пройдём руками по фразе `ab  c` (два пробела, `ab`, два пробела, `c`):

```
символ:   ' ' ' ' 'a' 'b' ' ' ' ' ' 'c'
in_word:   0   0   1   1   0   0   1
words:     0   0   1   1   1   1   2
```

Счётчик подскочил только на `a` и на `c` — ровно на первых буквах слов.

Что считать разделителем? Пробел — точно. Табуляция `'\t'` — тоже: её легко вставить, нажав Tab или скопировав текст из таблицы. Ещё бывает `'\r'` (возврат каретки) — он приезжает в файлах, сделанных на Windows. Перечислять всё это руками через `||` можно, но именно для этого в `<ctype.h>` есть `isspace`: она отвечает «да» на пробел, `\t`, `\n`, `\v`, `\f` и `\r`. Одна проверка вместо шести. (Это список для стандартной настройки языка — локали `"C"`, в которой работает любая программа, пока сама не вызовет `setlocale`. У нас так и есть.)

Заметь, что «слово» у нас — любая непрерывная цепочка не-пробельных символов. Знаки препинания в слово входят: `fox.` — одно слово из четырёх символов, `a,b` — тоже одно слово. Для подсчёта слов в обычном тексте этого хватает, а более хитрые правила — уже дело вкуса.

Раз уж мы идём по словам, посчитаем заодно **длину самого длинного слова** — это почти бесплатно. Держим `current` (длина слова, которое сейчас идём) и `longest` (рекорд). На каждой букве `current++`, и если он побил рекорд — обновляем `longest`. На разделителе `current = 0`.

Полная программа: `10-words.c`

```c
#include <stdio.h>
#include <ctype.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

/* Срезает хвостовой '\n', который оставляет после себя fgets. */
void chomp(char *s)
{
    int len = my_strlen(s);
    if (len > 0 && s[len - 1] == '\n') {
        s[len - 1] = '\0';
    }
}

int main(void)
{
    char line[200];

    printf("Введи фразу (латиницей): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;
    }
    chomp(line);

    int words = 0;
    int longest = 0;                    /* длина самого длинного слова */
    int current = 0;                    /* длина слова, которое идём */
    int in_word = 0;                    /* сейчас мы внутри слова? */
```

```
    for (int i = 0; line[i] != '\0'; i++) {
        unsigned char c = (unsigned char)line[i];
        if (isspace(c)) {
            in_word = 0;                /* пробел — слово кончилось */
            current = 0;
        } else {
            if (in_word == 0) {         /* переход «пробел -> буква» */
                words++;                /* начало нового слова */
                in_word = 1;
            }
            current++;
            if (current > longest) {
                longest = current;
            }
        }
    }

    printf("Слов: %d\n", words);
    printf("Самое длинное слово: %d символов\n", longest);
    return 0;
}
```

Разбор:

- `unsigned char c = (unsigned char)line[i];` — то же правило, что и в палиндроме: перед `<ctype.h>` приводим к `unsigned char`.
- `if (isspace(c))` — разделитель ли это. Одна проверка покрывает пробел, табуляцию и возврат каретки.
- В ветке-разделителе сбрасываем оба счётчика: `in_word = 0` («мы вышли из слова») и `current = 0` («новое слово начнётся с нуля»).
- В другой ветке смотрим `if (in_word == 0)` — если мы только что были вне слова, значит это его первая буква. Прибавляем `words++` и ставим `in_word = 1`. Пока буквы идут подряд, `in_word` остаётся `1`, и счётчик слов не растёт.
- `if (current > longest) longest = current;` — обновление рекорда. Мы делаем его на каждой букве, а не в конце слова: так не нужен отдельный код «слово кончилось» и не забудется последнее слово строки, если она обрывается без пробела в конце. Классическая ошибка «последнее слово не посчиталось» тут просто не может случиться.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 10-words.c -o 10-words
$ ./10-words
Введи фразу (латиницей): the quick brown fox
Слов: 4
Самое длинное слово: 5 символов
```

Теперь тот же текст, но с двумя пробелами в начале, тремя между словами, табуляцией вместо пробела перед `brown` и двумя пробелами в конце. Набрать такое в терминале руками неудобно (Tab там часто перехватывает оболочка), поэтому подадим ввод через `printf` — `\t` в нём означает табуляцию, `\n` — Enter:

```
$ printf '  the   quick\tbrown   fox  \n' | ./10-words
Введи фразу (латиницей): Слов: 4
Самое длинное слово: 5 символов
```

Четыре — как и должно быть. Приглашение и ответ оказались на одной строке просто потому, что вводить было нечего: `printf` подал текст сразу, эхо от клавиатуры не появилось.

Пустая строка и строка из одних пробелов:

```
$ printf '\n' | ./10-words
Введи фразу (латиницей): Слов: 0
Самое длинное слово: 0 символов
$ printf '     \n' | ./10-words
Введи фразу (латиницей): Слов: 0
Самое длинное слово: 0 символов
```

Ноль и ноль. Флаг `in_word` ни разу не включился, цикл честно отработал вхолостую.

Часть 4. Анаграммы

Анаграммы — две фразы, в которых каждая буква встречается одинаковое число раз, а порядок букв может быть любым: `listen` и `silent`, `anagram` и `nagaram`. А `hello` и `world` — нет. Заметь: важно именно «одинаковое число раз», а не просто «те же буквы». В `aab` и `ab` набор букв один и тот же (`a` и `b`), но `a` в первом слове две, а во втором одна — это не анаграммы. И ещё одна мелочь из определения: слово — анаграмма самого себя (порядок «может быть любым», в том числе прежним), и наша программа на `listen` и `listen` скажет «анаграммы».

Идея: считаем частоты букв

Как проверить руками? Можно отсортировать буквы каждого слова и сравнить: `listen` → `eilnst`, `silent` → `eilnst`, совпало. Но есть приём проще и быстрее — **подсчёт частот**. Сортировка тратит порядка `n log n` действий (глава 8), а подсчёт — один проход по каждой фразе плюс проверка 256 счётчиков: время растёт просто пропорционально длине.

Заведём на каждую возможную букву свой счётчик. Пройдём по первой фразе и для каждой буквы **прибавим** её счётчик. Потом пройдём по второй и для каждой буквы **вычтем**. Если фразы — анаграммы, то каждую букву мы столько же раз прибавили, сколько вычли, и все счётчики в конце окажутся **нулевыми**. Остался где-то не ноль — состав не совпал.

Пройдём руками по `abc` и `cab`, показывая только ненулевые счётчики:

```
после "abc":   a: +1   b: +1   c: +1
после "cab":   a:  0   b:  0   c:  0   -> анаграммы
```

А теперь `aab` и `ab`:

```
после "aab":   a: +2   b: +1
после "ab":    a: +1   b:  0   -> у 'a' остался 1, не анаграммы
```

Заметь приятный побочный эффект: разная длина отсекается сама собой. Если в первой фразе букв больше, какой-то счётчик обязательно останется положительным. Отдельная проверка длин не нужна (хотя как быстрая отсечка она была бы не лишней — см. упражнения).

Под счётчики возьмём массив `int count[256]`. Почему 256? Один байт принимает значения от 0 до 255, и массив на 256 ячеек даёт отдельный счётчик под **любой** возможный байт. Сам код символа служит индексом: для буквы `a` (код 97) растёт `count[97]`.

Наивная версия и две неудачи

Напишем как есть — считаем все символы подряд, регистр не трогаем:

```
    int count[256] = {0};
    for (int i = 0; a[i] != '\0'; i++) {
        count[(unsigned char)a[i]]++;   /* в первой фразе — прибавляем */
    }
    for (int i = 0; b[i] != '\0'; i++) {
        count[(unsigned char)b[i]]--;   /* во второй — вычитаем */
    }
```

Проверяем:

```
$ ./naive-anagram
Первая фраза (латиницей): listen
Вторая фраза (латиницей): silent
"listen" и "silent" — анаграммы
$ ./naive-anagram
Первая фраза (латиницей): Listen
Вторая фраза (латиницей): Silent
"Listen" и "Silent" — не анаграммы
$ ./naive-anagram
Первая фраза (латиницей): Dirty Room
Вторая фраза (латиницей): dormitory
"Dirty Room" и "dormitory" — не анаграммы
```

Строчные слова разобраны верно. Стоит появиться заглавным — ломается: `'L'` (код 76) и `'l'` (код 108) попадают в разные ячейки массива. А знаменитая пара `Dirty Room` / `dormitory` не проходит ещё и из-за пробела: в первой фразе он есть, во второй нет, и `count[' ']` остаётся единицей.

Лечение то же, что и в палиндроме: считать только буквы и только в одном регистре.

Выносим подсчёт в функцию

Два цикла отличаются одним знаком: в первом `++`, во втором `--`. Это просьба вынести общее в функцию и передать разницу параметром:

```c
/* Проходит по строке и меняет счётчики букв на delta:
 * +1 для первой фразы, -1 для второй. Не-буквы пропускает. */
void tally(const char *s, int count[], int delta)
{
    for (int i = 0; s[i] != '\0'; i++) {
        unsigned char c = (unsigned char)s[i];
        if (isalpha(c)) {
            count[tolower(c)] += delta;
        }
    }
}
```

- `int count[]` в параметрах — массив передаётся по адресу, а не копируется (глава 3). Значит, изменения внутри функции видны снаружи — именно это нам и нужно. Квадратные скобки пустые: длина в параметре всё равно не хранится, запись `int count[]` и `int *count` тут означают одно и то же.

- `delta` — на сколько менять счётчик: +1 или −1.
- `count[tolower(c)] += delta;` — индекс это уже приведённый к нижнему регистру код буквы, так что `A` и `a` попадают в одну ячейку. `x += y` — сокращение для `x = x + y`.
- `if (isalpha(c))` — не-буквы (пробелы, запятые, цифры) просто пропускаем, они на ответ не влияют.

Полная программа: `10-anagram.c`

```c
#include <stdio.h>
#include <ctype.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

/* Срезает хвостовой '\n', который оставляет после себя fgets. */
void chomp(char *s)
{
    int len = my_strlen(s);
    if (len > 0 && s[len - 1] == '\n') {
        s[len - 1] = '\0';
    }
}

/* Проходит по строке и меняет счётчики букв на delta:
 * +1 для первой фразы, -1 для второй. Не-буквы пропускает. */
void tally(const char *s, int count[], int delta)
{
    for (int i = 0; s[i] != '\0'; i++) {
        unsigned char c = (unsigned char)s[i];
        if (isalpha(c)) {
            count[tolower(c)] += delta;
        }
```

```
    }
}

int main(void)
{
    char a[200];
    char b[200];

    printf("Первая фраза (латиницей): ");
    if (fgets(a, sizeof a, stdin) == NULL) {
        return 0;
    }
    printf("Вторая фраза (латиницей): ");
    if (fgets(b, sizeof b, stdin) == NULL) {
        return 0;
    }

    chomp(a);
    chomp(b);

    /* По счётчику на каждый из 256 возможных байтов. */
    int count[256] = {0};
    tally(a, count, +1);        /* в первой фразе — прибавляем */
    tally(b, count, -1);        /* во второй — вычитаем */

    /* Если все счётчики нулевые — буквы совпали по составу. */
    int anagram = 1;
    for (int k = 0; k < 256; k++) {
        if (count[k] != 0) {
            anagram = 0;
            break;
        }
    }

    if (anagram) {
        printf("\"%s\" и \"%s\" — анаграммы\n", a, b);
    } else {
        printf("\"%s\" и \"%s\" — не анаграммы\n", a, b);
```

```
    }
    return 0;
}
```

Ещё пара мест, которые стоит проговорить:

- `int count[256] = {0};` — массив из 256 счётчиков. Запись `= {0}` заполняет **все** ячейки нулями (не только первую: остальные доинициализируются нулями по правилам языка). Без этого в массиве лежал бы мусор — что угодно, что осталось в этой памяти от прошлых вычислений, — и счёт был бы бессмысленным.
- Финальный цикл проверяет, что **все** счётчики обнулились. Нашли ненулевой — `anagram = 0` и `break`.
- Два буфера по 200 и два `fgets` подряд. Тут есть подводный камень: `fgets` кладёт в буфер не больше 199 символов (200-й байт уходит под `'\0'`). Если в первой фразе 199 символов или больше, в буфер не влезет `'\n'`, а то и хвост фразы. Они останутся в потоке ввода и приедут во второй `fgets` вместо второй фразы. Об этом — в «Частых ошибках».

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 10-anagram.c -o 10-anagram
$ ./10-anagram
Первая фраза (латиницей): listen
Вторая фраза (латиницей): silent
"listen" и "silent" — анаграммы
```

Случаи, на которых валилась наивная версия:

```
$ ./10-anagram
Первая фраза (латиницей): Listen
Вторая фраза (латиницей): Silent
"Listen" и "Silent" — анаграммы
$ ./10-anagram
Первая фраза (латиницей): Dirty Room
Вторая фраза (латиницей): dormitory
"Dirty Room" и "dormitory" — анаграммы
```

Разная длина и разный состав:

```
$ ./10-anagram
Первая фраза (латиницей): aab
Вторая фраза (латиницей): ab
```

```
"aab" и "ab" — не анаграммы
$ ./10-anagram
Первая фраза (латиницей): hello
Вторая фраза (латиницей): world
"hello" и "world" — не анаграммы
```

Приём «массив-счётчик, где индекс — сам символ» очень мощный. Им же считают, сколько раз встречается каждая буква, ищут первый неповторяющийся символ, проверяют, можно ли из букв одной фразы собрать другую. Запомни его — на собеседованиях он выручает постоянно.

Частые ошибки

Забыли обрезать `'\n'` после `fgets`. Главная беда этой главы. Перевод строки портит и переворот (улетает в начало), и сравнение (`"level\n"` уже не палиндром), и длину. После каждого `fgets` — `chomp`. Рефлекс.

Забыли `(unsigned char)` перед `<ctype.h>` или перед индексом. Тип `char` на маке знаковый, байты выше 127 становятся отрицательными, и ты читаешь память до начала таблицы. Компилятор предупредит только про индекс массива (`-Wchar-subscripts`), а про `isalpha(c)` промолчит — там формально всё в порядке по типам. Проверять придётся самому.

Не поставили `'\0'`, собирая строку вручную. Скопировал символы в новый массив — поставь в конце `'\0'` сам. Иначе `printf("%s", ...)` пойдёт печатать всё подряд, пока не наткнётся на случайный нулевой байт.

Не обнулили массив-счётчик. `int count[256];` без `= {0}` содержит мусор, и подсчёт частот соврёт — причём правдоподобно.

Сравнение строк через `==`. `a == b` для двух `char *` сравнивает **адреса**, а не текст (глава 3). Для содержимого нужен `strcmp` из `<string.h>`. А вот отдельные символы (`line[i] != line[j]`) сравнивать через `==`/`!=` можно и нужно: это числа.

`line[len - 1]` без проверки `len > 0`. На пустой строке это `line[-1]` — чтение за левой границей массива. Всегда пиши `if (len > 0 && ...)`, и именно в таком порядке.

Строка длиннее буфера. `fgets` возьмёт `sizeof line - 1` символов, а остаток оставит в потоке — и следующий `fgets` прочитает **его**, а не то, что ты наберёшь. Возьмём нарочно крошечные буферы:

```
#include <stdio.h>


int main(void)
```

```
{
    char first[10];
    char second[10];

    printf("Первое слово: ");
    if (fgets(first, sizeof first, stdin) == NULL) {
        return 0;
    }
    printf("Второе слово: ");
    if (fgets(second, sizeof second, stdin) == NULL) {
        return 0;
    }

    printf("first  = [%s]\n", first);
    printf("second = [%s]\n", second);
    return 0;
}
```

Подаём одно длинное слово и одно короткое:

```
$ cc -std=c11 -Wall -Wextra -O2 trunc.c -o trunc
$ printf 'extraordinary\nsecond\n' | ./trunc
Первое слово: Второе слово: first  = [extraordi]
second = [nary
]
```

Слово `second` не попало никуда: во второй буфер приехал хвост первого (`nary` вместе с переводом строки). В `first` легло девять символов — `sizeof first - 1`, десятое место ушло под `'\0'`.

Признак обрезки — в буфере **нет** `'\n'`. Так и проверяй: если `'\n'` не нашёлся, строка вместе с Enter не влезла в буфер, и остаток надо дочитать `getchar()`-ом до `'\n'` или до `EOF`, иначе он испортит следующий ввод. Одно исключение: последняя строка файла, после которой нет перевода строки, тоже придёт без `'\n'`. Тогда дочитывать нечего, `getchar()` сразу вернёт `EOF`.

Кириллица в посимвольном алгоритме. Мы это видели: UTF-8 разложит русское слово на пары байтов, и переворот с палиндромом дадут бессмыслицу. В этой главе — только латиница.

Упражнения

1. Добавь в `10-reverse.c` подсчёт **гласных** (`a, e, i, o, u`) и выведи их число. Регистр учитывать не надо — сначала `tolower`. Подсказка: одну проверку удобно написать через `strchr("aeiou", c)` из `<string.h>`, но можно и просто цепочкой `||`. Осторожно со `strchr`: она находит в строке и завершающий `'\0'`, так что `strchr("aeiou", '\0')` вернёт не `NULL`. Внутри цикла по строке `'\0'` тебе не попадётся, но знать об этом стоит. Ожидаемый результат: на `hello` — `Гласных: 2`, на `Programming In C` — `Гласных: 4` (o, a, i и заглавная I).

2. Напиши свой `int my_tolower(int c)` — без `<ctype.h>`, руками. Буквы `'A'..'Z'` идут подряд, и `'a'..'z'` тоже, поэтому разница между заглавной и строчной постоянна и равна `'a' - 'A'`. Все остальные символы функция должна возвращать нетронутыми. Проверь, что `10-palindrome.c` с твоей функцией работает так же. Тонкость: подряд идущие буквы — свойство таблицы ASCII, на которой живут все современные системы, а стандарт Си этого не обещает. Он обещает другое: подряд идут **цифры** `'0'..'9'`, поэтому `'7' - '0' == 7` на любой машине. Ожидаемый результат: `my_tolower('Q')` — `'q'`, `my_tolower('q')` — `'q'`, `my_tolower('5')` — `'5'`; на всех четырёх примерах из главы палиндром отвечает так же, как с библиотечной `tolower`.

3. Перепиши палиндром **без второго массива**: два индекса идут навстречу прямо по исходной строке, а не-буквы каждый из них пропускает сам (`while (i < j && !isalpha(...)) i++;`). Так делают на собеседованиях, когда просят «без дополнительной памяти». Ожидаемый результат: ответы совпадают с `10-palindrome.c` до буквы — `level`, `Level`, `never odd or even` и фраза про Панаму — палиндромы, `world` — нет; пустая строка, `x` и `!!!` — палиндромы.

4. Научи `10-words.c` печатать не только длину самого длинного слова, но и само слово. Подсказка: запоминай индекс начала рекордного слова, а в конце выведи его через `%.*s` — этот формат печатает ровно столько символов, сколько указано: `printf("%.*s\n", len, line + start);`. Ожидаемый результат: на `the quick brown fox` — `quick`. Слова `quick` и `brown` одной длины, но рекорд обновляется только при строгом `current > longest`, поэтому побеждает первое.

5. Напиши программу, которая печатает, **сколько раз** встречается каждая латинская буква во введённой фразе, без учёта регистра. Массив-счётчик из части 4, в конце пройдись по `'a'..'z'` и выведи только буквы с ненулевым счётчиком. Ожидаемый результат на `Hello, World`:

```
d: 1
e: 1
h: 1
l: 3
```

```
o: 2
r: 1
w: 1
```

6. Добавь в `10-anagram.c` быструю отсечку: если число букв в фразах разное, ответ «не анаграммы» известен сразу, без проверки 256 счётчиков. Подсказка: пусть `tally` заодно возвращает число посчитанных букв. Ожидаемый результат: ответы те же, что в главе; на `aab` и `ab` (3 буквы против 2) ответ «не анаграммы» даёт уже отсечка.

7. Напиши функцию `int my_strcmp(const char *a, const char *b)`, которая возвращает 0 при равных строках, отрицательное число, если `a` идёт раньше `b`, и положительное иначе, — своими руками, сравнивая символы в цикле, без `<string.h>`. «Раньше» здесь значит **лексикографически** (как в словаре), но по кодам символов, а не по алфавиту: `"Zebra"` окажется раньше `"apple"`, потому что код `'Z'`

   (90) меньше кода `'a'` (97). Подсказка: иди по обеим строкам одновременно, пока символы равны и не кончились; в конце верни разность кодов, приведя оба символа к `unsigned char`. Ожидаемый результат (знак ответа): `"abc"` и `"abc"` — 0; `"abc"` и `"abd"` — минус; `"b"` и `"a"` — плюс; `"ab"` и `"abc"` — минус (кончившаяся строка меньше); `"Zebra"` и `"apple"` — минус. Знаки совпадают с библиотечным `strcmp`.

Что мы получили

Ты умеешь обрабатывать строки **изнутри**, не полагаясь на готовые функции: считать длину циклом до `'\0'`, ходить двумя индексами с концов (переворот, палиндром), нормализовать текст перед сравнением, вести флаг состояния (`in_word` при подсчёте слов) и применять массив-счётчик по коду символа (анаграммы, частоты).

Заодно ты познакомился с `<ctype.h>` и с правилом `(unsigned char)`, которое отличает работающий код от кода, который «пока не упал». И ты знаешь главную бытовую ловушку — хвостовой `'\n'` от `fgets`, — и срезаешь его на автомате.

Пока все наши данные жили в массивах фиксированного размера: `char line[100]`, `int count[256]`. Но что, если заранее неизвестно, сколько элементов понадобится, и они должны то добавляться, то удаляться? Тогда массива мало. В следующей главе — **динамические структуры данных**: растущий массив, стек, очередь и связный список, где элементы связаны указателями и разбросаны по памяти. Там наконец пригодятся `malloc`/`free` из главы 3 и появится `struct` — способ склеить несколько значений в один объект.

Итоговые программы главы — `code/10-reverse.c`, `code/10-palindrome.c`, `code/10-words.c`, `code/10-anagram.c`.

Глава 11. Динамические структуры данных: растущий массив, стек, очередь, связный список

До сих пор мы работали с обычным массивом: завёл `int a[100]` — и живи в этих рамках. Но в настоящих программах данные приходят и уходят: игрок делает ходы, в очередь встают задачи, в историю добавляются записи. Заранее размер не угадать.

В этой главе соберём четыре **структуры данных** — четыре способа хранить набор значений, у каждого свой характер:

- **растущий массив** — обычный массив, который сам увеличивается, когда заполнится;
- **стек** — «последним пришёл, первым ушёл»;
- **очередь** — «первым пришёл, первым ушёл»;
- **связный список** — цепочка узлов, разбросанных по памяти.

А заодно познакомимся с `realloc` (расширить уже выделенную память) и с **структурами** (`struct`) — способом склеить несколько значений в один объект. Без `struct` ни списка, ни аккуратного стека не построить, поэтому начнём с него.

Эта глава — про память. Здесь легко наделать ошибок, которые не видны глазом: выход за границу массива, чтение освобождённого узла, утечка. Поэтому мы будем регулярно запускать программы под **санитайзерами** — специальным режимом компилятора, который ловит такие вещи на лету:

```
cc -std=c11 -Wall -Wextra -O1 -g -fsanitize=address,undefined prog.c -o prog
```

`-fsanitize=address` следит за обращениями к памяти, `undefined` — за неопределённым поведением вроде выхода индекса за границы. Программа работает медленнее, зато при первой же ошибке печатает, где именно она случилась. Правильный код под санитайзерами молчит. Одна оговорка из главы 3: утечки памяти санитайзер на macOS не ищет (эта его часть,

LeakSanitizer, на маке не работает). Для утечек у нас системная утилита `leaks`, её мы тоже будем запускать.

Знакомимся: `struct` — свой составной тип

`int` хранит одно число. Массив — много чисел **одного** типа. А как описать узел списка, в котором лежит и число, и адрес следующего узла? Нужен тип, который объединяет **разные** поля под одним именем. Это и есть **структура** (`struct`, от *structure*).

Представь карточку игрока: имя, очки, уровень. Три разные вещи, но логически это один объект. Опишем его:

```
struct Player {
    char name[32];
    int score;
    int level;
};
```

Разбор:

- `struct Player { ... };` — мы **определили** новый тип с именем `struct Player`. Само по себе это не создаёт ни одной переменной — это чертёж, описание «из чего состоит игрок». Памяти он не занимает.
- внутри фигурных скобок — **поля** (fields): `name`, `score`, `level`. Каждое со своим типом, как обычные переменные. Поля идут через `;`.
- после закрывающей `}` обязательно стоит `;` — про неё забывают чаще всего, и компилятор выдаёт странную ошибку на следующей строке. Не забудь точку с запятой.

Теперь заведём переменную этого типа и заполним поля:

```
struct Player hero;
hero.score = 0;
hero.level = 1;
```

- `struct Player hero;` — создали **переменную** `hero` типа `struct Player`. Вот теперь в памяти лежит реальный игрок: место под имя, под очки, под уровень.
- `hero.score = 0;` — обращение к полю через **точку** (`.`): «у объекта `hero` поле `score` сделай равным 0».

Полей у структуры может быть сколько угодно, и они лежат в памяти подряд, одно за другим. Размер структуры узнаётся привычным `sizeof`: `sizeof(struct Player)`. Он не всегда равен точной сумме размеров полей — компилятор имеет право вставлять между ними пустые байты

для скорости (это называется выравнивание). Полагаться на конкретное число не надо, `sizeof` всегда скажет правду.

Указатель на структуру и стрелка ->

Часто структуру создают в динамической памяти (`malloc`) или передают в функцию — и тогда у нас на руках не сама структура, а **указатель** на неё. К полям через точку уже не обратишься, нужен другой способ.

```
struct Player *p = &hero;   /* p — адрес нашего игрока */

(*p).score = 50;            /* длинная запись */
p->score = 50;              /* короткая, то же самое */
```

- `struct Player *p` — указатель на структуру (как `int *p`, только тип другой).
- `(*p).score` — сначала `*p` разыменовывает указатель («сама структура, на которую смотрит `p`»), потом `.score` берёт поле. Скобки нужны: без них `*p.score` компилятор поймёт как `*(p.score)`, потому что точка «прилипает» сильнее звёздочки.
- `p->score` — **стрелка** `->`. Это просто **сокращение** для `(*p).score`. Запомни намертво:

  > `p->field` — это в точности `(*p).field`. Стрелка = «разыменуй и возьми поле».

  Когда у тебя указатель на структуру — пиши через стрелку, так короче и так пишут все.

Важное следствие: если функция получает `struct Player *p`, она работает с **оригиналом**, а не с копией. Изменит `p->score` — изменение увидит и тот, кто вызвал. Это тот же механизм, что у `swap` из главы 3, просто со структурой. А если передать структуру по значению (`struct Player p`), она **скопируется** целиком, вместе с 32-байтным массивом имени. Поэтому большие структуры почти всегда передают по указателю: копировать один адрес дешевле, чем сорок байт.

`typedef` — короткое имя для типа

Писать `struct Player` каждый раз длинно. `typedef` даёт типу псевдоним:

```
typedef struct Player Player;   /* теперь "Player" = "struct Player" */

Player hero;                    /* можно без слова struct */
```

Часто это совмещают с определением:

```
typedef struct {
    int value;
```

```
} Box;

Box b;          /* удобно */
```

В этой главе я оставлю полную запись `struct Node`, `struct Stack` — так нагляднее видно, что это структура. Но знай: `typedef` ровно для того, чтобы избавиться от повторов слова `struct`.

Этого про структуры хватит, чтобы строить дальше.

Часть 1. Растущий массив

Разбираем руками

Проблема обычного массива: размер фиксирован. `int a[100]` — а если понадобится 101? А если выделишь сразу миллион «про запас» — впустую займёшь память.

Идея растущего массива простая, как у блокнота: начни с маленького, а когда страницы кончатся — **доклей** ещё. Конкретно:

1. Выделяем место под несколько элементов (пусть под 2).
2. Заводим счётчик `count` — сколько реально занято.
3. Кладём элемент — увеличиваем `count`.
4. Как только `count` упёрся во вместимость — **расширяемся**.

Вопрос на миллион: расширяться **на сколько**? Прибавить одну ячейку? Удвоить? Разница выглядит мелочью, а на деле это разница между «работает» и «висит».

Почему удвоение, а не «плюс один»

Расширить массив — это в худшем случае **переехать**: выделить новый блок, перенести туда все старые элементы, освободить старый. Перенос стоит столько, сколько в массиве элементов.

Посчитаем на бумаге для 100 000 вставок.

Стратегия «плюс один»: переезжаем на **каждой** вставке. Первый переезд копирует 0 элементов, второй — 1, третий — 2, и так до 99 999. Сумма `0 + 1 + 2 + ... + 99999` — это примерно `100000 * 100000 / 2`, то есть **пять миллиардов** копирований. Если точно: сумма всех чисел от 0 до `n − 1` равна `n × (n − 1) / 2`, подставим `n = 100000` и получим `100000 × 99999 / 2 = 4 999 950 000`.

Стратегия «удвоение»: переезжаем, только когда место кончилось. Вместимость идёт 1, 2, 4, 8, 16 … 131072 — всего **восемнадцать** выделений памяти (при самом первом копировать ещё

нечего), а копирований суммарно `0 + 1 + 2 + 4 + ... + 65536 = 131071` — около **131 тысячи**. Это даже меньше, чем 2 × 100 000: при удвоении на один элемент приходится меньше двух копирований, сколько бы их ни было. Почему так при любом числе элементов — разберём чуть ниже. Тот редкий случай, когда «делать реже, но крупнее» выигрывает не на проценты, а в десятки тысяч раз.

Проверим руками — программа `code/11-growth.c` делает обе стратегии, копируя элементы явным циклом, и считает копирования:

```
$ cc -std=c11 -Wall -Wextra -O2 11-growth.c -o 11-growth
$ ./11-growth
вставляем 100000 элементов
рост на +1:   скопировано 4999950000 элементов, 0.850 с
удвоение:     скопировано 131071 элементов, 0.001 с
копирований меньше в 38147 раз
```

Числа копирований — ровно те, что мы насчитали на бумаге. Секунды на твоей машине будут другие, а вот число копирований — нет, оно чисто математическое. Поэтому готовые растущие массивы в библиотеках разных языков увеличивают вместимость **умножением** — кто в 2 раза, кто в 1,5, кто ещё как, — а не прибавлением.

Сколько стоит одна вставка: амортизированная O(1)

В главе 8 мы оценивали сортировки через «О большое». Здесь понадобятся ещё две оценки попроще. **O(1)** — число действий не зависит от `n`: сколько бы элементов ни лежало, операция укладывается в одно и то же небольшое число шагов. **O(n)** — число действий растёт не быстрее, чем пропорционально `n`: вдвое больше элементов — примерно вдвое больше работы, но не хуже. Напомню, что «О» — это оценка **сверху**: «не хуже, чем».

Теперь честно посмотрим на одну вставку при удвоении. Обычно она дешёвая: записали число в свободную ячейку, и всё. Но иногда она очень дорогая: вставка номер 65 537 застаёт массив полным (65 536 из 65 536) и тащит за собой переезд всех 65 536 элементов. Так что сказать «каждая вставка стоит O(1)» нельзя — отдельная вставка бывает O(n).

Зато можно посчитать цену **всех** вставок вместе. На `n` вставок уходит `n` записей новых элементов и копирования при переездах: `1 + 2 + 4 + ... + 2^(k-1) = 2^k - 1`, где `2^(k-1)` — последняя вместимость, из которой пришлось переезжать. Переезд случается, только когда пришёл ещё один элемент, а места нет, значит, эта вместимость меньше `n`, и копирований меньше `2 × 2^(k-1)`, то есть меньше `2n`. Проверим на наших числах: последний переезд был из вместимости 65 536, копирований 131 071 — это меньше, чем `2 × 100000`. Итого на `n` вставок меньше `3n` действий, то есть меньше трёх действий на вставку, при любом `n`.

Это и называется **амортизированная сложность O(1)**: дорогие вставки редки, и их цену можно «размазать» по дешёвым, которые были до них. Это гарантия на всю последовательность операций, а не на каждую отдельную. Не путай со словами «в среднем» из главы 8 про быструю сортировку: там усреднение по случайным входным данным, и на неудачных данных можно проиграть. Здесь никакой случайности нет — оценка верна для **любой** последовательности вставок.

А у «плюс одного» на `n` вставок уходит `n × (n − 1) / 2` копирований. Это $O(n^2)$ на все вставки, то есть в среднем по O(n) на каждую.

Годится ли другой коэффициент роста? Да, **любое умножение больше единицы** даёт амортизированную O(1), меняется только константа. Я посчитал программой для тех же 100 000 вставок: при росте в 2 раза — 131 071 копирование (1,3 на элемент), при росте в 1,5 раза — 276 521 (2,8 на элемент), зато пустующих ячеек в среднем меньше. А вот **прибавление** постоянного числа — хоть +1, хоть +1000 — даёт квадрат: при +1000 переездов всё равно около `n / 1000`, и каждый копирует уже накопленное.

`realloc` — изменить размер уже выделенной памяти

Расширять память умеет `realloc` (*reallocate* — перевыделить), из `<stdlib.h>`:

```
void *realloc(void *ptr, size_t size);
```

Что делает `realloc(старый_указатель, новый_размер_в_байтах)`:

- пытается сделать блок, на который указывает `ptr`, нужного размера;
- **старые данные сохраняются** — если блок пришлось перенести на новое место, `realloc` сам их туда скопирует;
- возвращает **адрес** блока нужного размера. Этот адрес может оказаться **другим** — поэтому результат обязательно надо куда-то положить.

Иногда `realloc` расширяет блок прямо на месте, ничего не копируя, — но рассчитывать на это нельзя, это зависит от того, что лежит рядом в памяти. Считай, что каждый рост — это переезд.

Ловушка, в которую падают почти все:

```
data = realloc(data, new_size);   /* ОПАСНО */
```

Если `realloc` не сможет выполнить просьбу, он вернёт `NULL`, а **старый блок оставит нетронутым**. И вот тут запись `data = realloc(data, ...)` затирает `data` этим `NULL` — единственный адрес старого блока потерян. Данные недоступны, память не освободить: утечка.

Убедимся, что старый блок правда остаётся целым. Попросим заведомо невозможный размер:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *data = malloc(2 * sizeof(int));
    if (data == NULL) {
        return 1;
    }
    data[0] = 42;
    data[1] = 7;

    /* Просим заведомо невозможный размер — половину адресного
     * пространства. Столько памяти нам никто не даст. */
    size_t huge = (size_t)-1 / 2;
    int *bigger = realloc(data, huge);
    if (bigger == NULL) {
        printf("realloc вернул NULL\n");
        printf("а старый блок цел: data[0] = %d, data[1] = %d\n",
               data[0], data[1]);
        free(data);
        return 0;
    }

    data = bigger;
    free(data);
    return 0;
}
```

```
$ cc -std=c11 -Wall -Wextra -O2 realloc-fail.c -o realloc-fail
$ ./realloc-fail
realloc вернул NULL
а старый блок цел: data[0] = 42, data[1] = 7
```

Числа на месте, и `free(data)` отработал нормально. Кстати, этот пример нарочно просит невозможное, поэтому запускай его **обычной** сборкой: санитайзер на такую заявку ругается сам и прерывает программу, не давая `realloc` вернуть `NULL`.

- `(size_t)-1` — хитрый, но общепринятый способ получить самое большое возможное значение типа `size_t`. Тип беззнаковый, поэтому −1 в нём «заворачивается» в максимум: беззнаковая арифметика в Си идёт по модулю `2^N`. Пополам делим просто для наглядности: это всё ещё заведомо невыполнимая просьба (на 64-битном маке — около 8 эксабайт), но уже «половина всей памяти», а не странное число-максимум.

Если всё же хочется прогнать этот пример под санитайзером, попроси его не прерывать программу, а возвращать `NULL`, как обычный `malloc`: `ASAN_OPTIONS=allocator_may_return_null=1 ./realloc-fail`. Тогда вывод будет тот же, что выше.

Правильный шаблон, который надо выучить:

```
int *bigger = realloc(data, new_size);
if (bigger == NULL) {
    /* старый data ещё цел — можно спасти данные или аккуратно выйти */
}
data = bigger;
```

Сначала во **временный** указатель, потом проверка, и только потом присваивание.

Полная программа: `11-dynarray.c`

Складываем в растущий массив числа от 1 до 10, начав всего с двух ячеек.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int capacity = 2;                       /* сколько влезает сейчас */
    int count = 0;                          /* сколько реально лежит */
    int *data = malloc((size_t)capacity * sizeof(int));
    if (data == NULL) {
        printf("Не хватило памяти\n");
        return 1;
    }

    for (int value = 1; value <= 10; value++) {
        if (count == capacity) {            /* заполнили — растём */
            capacity = capacity * 2;
            int *bigger = realloc(data,
                                  (size_t)capacity * sizeof(int));
```

```
            if (bigger == NULL) {           /* не дали — старое цело */
                printf("Не хватило памяти при росте\n");
                free(data);
                return 1;
            }
            data = bigger;                  /* теперь работаем с новым */
            printf("[рост: вместимость стала %d]\n", capacity);
        }
        data[count] = value;                /* кладём в свободную ячейку */
        count = count + 1;                  /* занятых стало больше */
    }

    printf("Всего элементов: %d, вместимость: %d\n", count, capacity);
    for (int i = 0; i < count; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");

    free(data);
    return 0;
}
```

Разбор:

- `int capacity = 2;` и `int count = 0;` — два разных числа! `capacity` — сколько ячеек **выделено**, `count` — сколько из них **занято**. Путать их — типичная ошибка, из-за неё пишут за границу массива.
- `malloc((size_t)capacity * sizeof(int))` — просим память под 2 целых (глава 3). Зачем `(size_t)`? `capacity` — это `int`, и на большом массиве произведение `capacity * sizeof(int)` могло бы переполнить `int` ещё до того, как результат превратится в размер. Приведение к `size_t` считает умножение уже в широком беззнаковом типе. На наших десяти элементах это неважно, но привычка правильная.
- `if (count == capacity)` — проверка «свободного места не осталось». Делаем её **перед** записью, чтобы было куда писать.
- блок с `realloc` — тот самый безопасный шаблон.
- `data[count] = value;` — к памяти от `malloc`/`realloc` обращаемся как к обычному массиву, по индексу.
- `free(data);` — один `free` на один блок, в конце. `realloc` не требует отдельного `free` для старого блока: он сам его освободил, когда переехал.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 11-dynarray.c -o 11-dynarray
$ ./11-dynarray
[рост: вместимость стала 4]
[рост: вместимость стала 8]
[рост: вместимость стала 16]
Всего элементов: 10, вместимость: 16
1 2 3 4 5 6 7 8 9 10
```

Видно, как массив рос: 2 → 4 → 8 → 16. Три переезда на десять вставок. Все числа на месте — `realloc` сохранил их при каждом переезде. Заметь и цену удобства: под 10 чисел выделено место на 16, шесть ячеек простаивают. Это нормальная плата за скорость.

И обязательный шаг для любой программы с `malloc` — прогон под санитайзерами:

```
$ cc -std=c11 -Wall -Wextra -O1 -g -fsanitize=address,undefined \
     11-dynarray.c -o 11-dynarray-san
$ ./11-dynarray-san
[рост: вместимость стала 4]
[рост: вместимость стала 8]
[рост: вместимость стала 16]
Всего элементов: 10, вместимость: 16
1 2 3 4 5 6 7 8 9 10
```

Вывод тот же, ни одного лишнего слова — значит, за границы блока мы не вылезали и освобождённую память не трогали. Молчание санитайзера и есть хороший результат; если бы он что-то нашёл, ты бы это ни с чем не спутал — сейчас увидишь.

А вот про утечки молчание санитайзера на маке ничего не говорит — их проверяем утилитой `leaks` (глава 3):

```
$ cc -std=c11 -Wall -Wextra -g 11-dynarray.c -o 11-dynarray-g
$ leaks --atExit -- ./11-dynarray-g | grep "leaks for"
Process 51187: 0 leaks for 0 total leaked bytes.
```

`grep "leaks for"` оставляет из длинного отчёта одну итоговую строку (номер процесса у тебя будет свой). `MallocStackLogging=1` из главы 3 тут не нужен: он помогает найти, **откуда** утечка, а нам пока надо только убедиться, что её нет. Ноль утечек: единственный блок мы вернули через `free`.

Часть 2. Стек (LIFO)

Аналогия: стопка тарелок

Представь стопку чистых тарелок. Новую кладёшь **сверху**. Берёшь — тоже **сверху**. До нижней так просто не добраться: сначала снимешь все, что лежат на ней.

Это и есть **стек** (stack — стопка). Правило работы: **LIFO** — *Last In, First Out*, «последним пришёл — первым ушёл».

Три операции:

- **push** — положить элемент на вершину;
- **pop** — снять элемент с вершины (и вернуть его);
- **peek** — подсмотреть, что на вершине, **не снимая**.

Где встречается: кнопка «отменить» (Ctrl+Z) — последнее действие отменяется первым; вызовы функций в программе (тот самый стек вызовов из главы 9); проверка парности скобок в выражении.

Строим на массиве

Стек прекрасно ложится на массив плюс одно число — `top` (вершина). Договоримся: `top` — это **индекс первой свободной ячейки**, он же заодно равен количеству элементов в стеке.

- push: положить в `data[top]`, потом `top++`;
- pop: сначала `top--`, потом взять `data[top]`;
- стек пуст, если `top == 0`; переполнен, если `top == CAPACITY`.

Порядок в pop строгий: **сначала** `top--`, потом читаем. Наоборот — прочитаешь свободную ячейку.

Наивный pop и что бывает на пустом стеке

Напишем pop в лоб, без проверок:

```
#include <stdio.h>

#define CAPACITY 5

/* Наивный pop: снимает с вершины и возвращает значение. */
int pop(int data[], int *top)
{
    *top = *top - 1;
```

```
    return data[*top];
}

int main(void)
{
    int data[CAPACITY];
    int top = 0;                    /* стек пуст */

    int value = pop(data, &top);    /* снимаем из пустого стека */
    printf("сняли %d, top стал %d\n", value, top);
    return 0;
}
```

- `#define CAPACITY 5` — **директива препроцессора**: перед компиляцией все слова `CAPACITY` в тексте программы заменяются на `5`. Удобно задавать константу в одном месте. Точки с запятой в конце `#define` **нет** — это не обычная инструкция, а команда препроцессору.
- `int data[]` и `int *top` в параметрах: массив передаётся по адресу сам собой, а вот `top` — обычное число, поэтому его передаём указателем, чтобы функция могла его изменить (глава 3).

Запускаем обычной сборкой три раза подряд:

```
$ cc -std=c11 -Wall -Wextra -O2 naive-stack.c -o naive-stack
$ ./naive-stack
сняли -1214647560, top стал -1
$ ./naive-stack
сняли -1235148040, top стал -1
$ ./naive-stack
сняли -1285528840, top стал -1
```

Каждый раз новое число — потому что мы прочитали `data[-1]`, ячейку **перед** массивом, а там лежит что попало. Программа не упала, ничего не сообщила, просто вернула мусор. Такие баги — самые злые.

Теперь то же самое под санитайзером:

```
$ cc -std=c11 -Wall -Wextra -O1 -g -fsanitize=address,undefined \
     naive-stack.c -o naive-stack
$ ./naive-stack 2>&1 | head -16
=================================================================
==46054==ERROR: AddressSanitizer: stack-buffer-underflow on address
```

```
  0x7ff7b723ab5c at pc 0x000108cc49bc bp 0x7ff7b723aaf0 sp 0x7ff7b723aae8
READ of size 4 at 0x7ff7b723ab5c thread T0
    #0 0x000108cc49bb in pop naive-stack.c:9
    #1 0x000108cc4b2a in main naive-stack.c:17
    #2 0x7ff809b681c7 in start+0xca7 (dyld:x86_64+0x131c7)

Address 0x7ff7b723ab5c is located in stack of thread T0 at offset 28 in
  frame
    #0 0x000108cc4a67 in main naive-stack.c:13

  This frame has 2 object(s):
    [32, 52) 'data' (line 14) <== Memory access at offset 28
      underflows this variable
    [96, 100) 'top' (line 15)
HINT: this may be a false positive if your program uses some custom
      stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-underflow naive-stack.c:9 in pop
```

Читается это так: `stack-buffer-underflow` — чтение **перед** началом буфера; `READ of size 4` — читали 4 байта, то есть один `int`; `in pop naive-stack.c:9` — вот строка, где это случилось; `'data' (line 14) <== Memory access ... underflows this variable` — а вот переменная, границу которой мы нарушили. Дальше идёт ещё три десятка строк с картой памяти — их я обрезал через `head -16`, для нас интересна первая часть.

Пара строк отчёта не влезала в ширину страницы, я их перенёс. И учти: адреса, номер процесса и номера строк библиотеки у тебя будут свои, они меняются от запуска к запуску. Постоянны только имена ошибок и строки твоего исходника — а это ровно то, что нужно.

Санитайзер нашёл ошибку за секунду и ткнул пальцем в строку. Без него мы бы гадали неделю.

Как вернуть «стек пуст»: статус плюс указатель на результат

Проверку добавить легко. Сложнее решить, **что вернуть**, если стек пуст. Первая мысль — какое-нибудь особое число:

```c
int pop(int data[], int *top)
{
    if (*top == 0) {
        return -1;              /* «стек пуст» */
    }
```

```
    *top = *top - 1;
    return data[*top];
}
```

Работает ровно до того момента, пока кто-нибудь не положит в стек `-1`. Тогда честно снятое значение `-1` не отличить от сообщения об ошибке. И так с любым «особым» числом: стек-то хранит **произвольные** `int`, свободных значений в нём нет.

Выход — разделить два ответа: «получилось или нет» возвращаем из функции, а само значение отдаём **через указатель**:

```
int pop(struct Stack *s, int *out);
```

Функция возвращает `1` (сняли) или `0` (пусто), а снятое число кладёт по адресу `out`. Такой параметр называют **выходным** (out-parameter). Мы уже видели этот приём в главе 3, когда `swap` менял значения через адреса; здесь он же, только с одним значением.

Вызов выглядит так:

```
int x;
if (pop(&s, &x)) {
    /* тут x — снятое значение */
} else {
    /* стек был пуст, x не тронут */
}
```

Обрати внимание: если `pop` вернул 0, он в `*out` **ничего не писал**, и `x` остался таким, каким был. Это тоже часть договора, и его надо соблюдать.

Заодно упакуем массив и `top` в одну структуру — вот и пригодился `struct`. Так стек становится **одним объектом**, который удобно передавать в функции: не два параметра, а один указатель.

Полная программа: `11-stack.c`

```
#include <stdio.h>

#define CAPACITY 5      /* сколько тарелок влезает в стопку */

struct Stack {
    int data[CAPACITY];
    int top;            /* индекс первой СВОБОДНОЙ ячейки = размер */
};
```

```c
/* Кладём на вершину. 1 — положили, 0 — стек полон. */
int push(struct Stack *s, int value)
{
    if (s->top == CAPACITY) {
        return 0;
    }
    s->data[s->top] = value;
    s->top++;
    return 1;
}

/* Снимаем с вершины в *out. 1 — сняли, 0 — стек пуст. */
int pop(struct Stack *s, int *out)
{
    if (s->top == 0) {
        return 0;
    }
    s->top--;
    *out = s->data[s->top];
    return 1;
}

/* Подсматриваем вершину, не снимая. 1 — есть что смотреть, 0 — пусто. */
int peek(const struct Stack *s, int *out)
{
    if (s->top == 0) {
        return 0;
    }
    *out = s->data[s->top - 1];
    return 1;
}

int main(void)
{
    struct Stack s;
    s.top = 0;                  /* пустой стек */
```

```
    /* Кладём шесть значений в стопку на пять — последнее не влезет. */
    for (int value = 10; value <= 60; value += 10) {
        if (push(&s, value)) {
            printf("push %d (в стеке %d)\n", value, s.top);
        } else {
            printf("push %d: стек полон, не кладу\n", value);
        }
    }

    int x;
    if (peek(&s, &x)) {
        printf("peek: наверху %d (в стеке всё те же %d)\n", x, s.top);
    }

    /* Снимаем, пока pop возвращает 1. */
    while (pop(&s, &x)) {
        printf("pop %d (осталось %d)\n", x, s.top);
    }

    /* Ещё один pop на пустом стеке: не падаем, а честно отказываем. */
    if (pop(&s, &x) == 0) {
        printf("pop на пустом стеке: вернул 0, x не тронут (там %d)\n", x);
    }

    return 0;
}
```

Разбор:

- `struct Stack { int data[CAPACITY]; int top; };` — массив и вершина теперь одно целое. Заметь: массив лежит **внутри** структуры, никакого `malloc` тут нет, память под него берётся там же, где под саму переменную `s`.
- `struct Stack s; s.top = 0;` — создали стек и объявили его пустым. Ячейки `data` при этом заполнены мусором, и это нормально: мы никогда не читаем выше `top`, а туда всегда сначала пишем.
- `s->top == CAPACITY` в `push` — проверка на переполнение. Без неё шестая тарелка легла бы в `data[5]`, а в массиве всего пять ячеек с индексами 0..4 — это запись за границу. Санитайзер такое ловит, а без него получится тихо испорченная соседняя память: скорее всего, поле `top`, которое лежит в структуре сразу за массивом.

- `const struct Stack *s` в `peek` — `const` обещает, что функция стек не изменит. Компилятор проследит: попытка написать `s->top = 0` внутри `peek` не соберётся. Отличный способ выразить намерение прямо в коде.
- `value += 10` в цикле — шаг 10, чтобы класть 10, 20, …, 60. Шесть значений при вместимости пять — специально, чтобы увидеть отказ.
- `while (pop(&s, &x))` — идиома «работаем, пока получается». `pop` возвращает 1, пока в стеке что-то есть, и 0, когда опустел, — тогда цикл сам и закончится. Никаких отдельных проверок на пустоту.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 11-stack.c -o 11-stack
$ ./11-stack
push 10 (в стеке 1)
push 20 (в стеке 2)
push 30 (в стеке 3)
push 40 (в стеке 4)
push 50 (в стеке 5)
push 60: стек полон, не кладу
peek: наверху 50 (в стеке всё те же 5)
pop 50 (осталось 4)
pop 40 (осталось 3)
pop 30 (осталось 2)
pop 20 (осталось 1)
pop 10 (осталось 0)
pop на пустом стеке: вернул 0, x не тронут (там 10)
```

Читаем по строчкам. Пять первых `push` прошли, шестой честно отказал — за границу массива мы не полезли. `peek` показал `50` и оставил стек в покое: размер как был 5, так и остался.

Дальше порядок при pop: вышло **50, 40, 30, 20, 10** — ровно наоборот тому, как клали. Это и есть LIFO в действии.

Последняя строка — то, ради чего мы городили выходной параметр. `pop` на пустом стеке вернул 0 и **ничего не записал**, поэтому в `x` так и лежит десятка, снятая предыдущим вызовом. Никакого мусора, никакого выхода за границу, никакого «волшебного» числа-ошибки.

Санитайзеры на этой программе молчат — можешь проверить сам той же командой, что и в первой части.

Часть 3. Очередь (FIFO)

Аналогия: очередь в магазине

Очередь у кассы работает честно: кто встал раньше, того обслужат раньше. Новый покупатель встаёт **в хвост**, кассир берёт из **головы**.

Это **очередь** (queue). Правило: **FIFO** — *First In, First Out*, «первым пришёл — первым ушёл». Полная противоположность стеку.

Операции:

- **enqueue** — встать в хвост;
- **dequeue** — забрать из головы.

Где встречается: задачи на печать (документы печатаются в порядке отправки), сообщения в мессенджере, обход в ширину по графу.

Наивная очередь: два индекса, которые ползут только вперёд

Наивно: храним `head` (откуда забираем) и `tail` (куда кладём). enqueue двигает `tail` вправо, dequeue двигает `head` вправо. Напишем и посмотрим, где сломается:

```c
#include <stdio.h>

#define CAPACITY 5

int main(void)
{
    int data[CAPACITY];
    int head = 0;           /* откуда забираем */
    int tail = 0;           /* куда кладём */

    for (int value = 1; value <= 5; value++) {
        data[tail] = value;
        tail++;
        printf("enqueue %d (head=%d, tail=%d)\n", value, head, tail);
    }

    for (int k = 0; k < 3; k++) {
        int value = data[head];
        head++;
```

```
        printf("dequeue %d (head=%d, tail=%d)\n", value, head, tail);
    }

    /* в массиве заняты только ячейки 3 и 4, свободно три штуки */
    for (int value = 6; value <= 7; value++) {
        if (tail == CAPACITY) {
            printf("enqueue %d: места нет (tail=%d)\n", value, tail);
            continue;
        }
        data[tail] = value;
        tail++;
        printf("enqueue %d (head=%d, tail=%d)\n", value, head, tail);
    }
    return 0;
}
```

```
$ cc -std=c11 -Wall -Wextra -O2 naive-queue.c -o naive-queue
$ ./naive-queue
enqueue 1 (head=0, tail=1)
enqueue 2 (head=0, tail=2)
enqueue 3 (head=0, tail=3)
enqueue 4 (head=0, tail=4)
enqueue 5 (head=0, tail=5)
dequeue 1 (head=1, tail=5)
dequeue 2 (head=2, tail=5)
dequeue 3 (head=3, tail=5)
enqueue 6: места нет (tail=5)
enqueue 7: места нет (tail=5)
```

Смотри на последние строки. В очереди стоят всего **двое** (значения 4 и 5), ячейки 0, 1, 2 свободны — а программа говорит «места нет». Оба индекса ползут только вперёд, `tail` упёрся в конец массива и встал. Массив «кончился», хотя он больше чем наполовину пуст.

Можно было бы после каждого dequeue сдвигать все элементы влево — но это копирование на каждой выдаче, дорого и глупо.

Кольцевой буфер

Лечение — **кольцевой буфер**: считаем, что после последней ячейки снова идёт нулевая, будто массив свёрнут в кольцо. Чтобы индекс «перепрыгивал» с конца в начало, берём остаток от деления на размер:

```
tail = (tail + 1) % CAPACITY;
```

Когда `tail` дорастает до `CAPACITY`, остаток делает его снова 0 — мы вернулись в начало кольца и используем освободившиеся ячейки. Вот зачем оператор `%` (остаток), который мы разбирали в главе 2.

Но теперь возникает новая беда. Пусть `head == tail`. Что это значит? Очередь **пуста** (никто не стоит) — или очередь **полна** (мы прошли полный круг и вернулись в ту же точку)? По двум индексам не различить, картинка одинаковая. Классические решения — держать одну ячейку всегда пустой (тогда полная очередь это `(tail + 1) % CAPACITY == head`) или завести отдельный счётчик.

Мы возьмём счётчик — он проще и не тратит ячейку впустую. Храним `head` и `count` (сколько элементов сейчас лежит), а хвост **вычисляем**:

```
tail = (head + count) % CAPACITY;
```

Теперь всё однозначно: пуста при `count == 0`, полна при `count == CAPACITY`. И `tail` как отдельную переменную хранить не надо — одной причиной для рассинхронизации меньше.

Полная программа: `11-queue.c`

```c
#include <stdio.h>

#define CAPACITY 5      /* длина кольцевого буфера */

struct Queue {
    int data[CAPACITY];
    int head;           /* индекс первого в очереди */
    int count;          /* сколько элементов сейчас лежит */
};

/* Индекс первой свободной ячейки — туда встанет следующий. */
int queue_tail(const struct Queue *q)
{
    return (q->head + q->count) % CAPACITY;
```

```
}

/* Встаём в хвост. 1 — встали, 0 — очередь полна. */
int enqueue(struct Queue *q, int value)
{
    if (q->count == CAPACITY) {
        return 0;
    }
    q->data[queue_tail(q)] = value;
    q->count++;
    return 1;
}

/* Забираем из головы в *out. 1 — забрали, 0 — очередь пуста. */
int dequeue(struct Queue *q, int *out)
{
    if (q->count == 0) {
        return 0;
    }
    *out = q->data[q->head];
    q->head = (q->head + 1) % CAPACITY;
    q->count--;
    return 1;
}

/* Ставит value в очередь и печатает, что получилось. */
void try_enqueue(struct Queue *q, int value)
{
    int cell = queue_tail(q);           /* куда встанет — до вставки */
    if (enqueue(q, value)) {
        printf("enqueue %d -> ячейка %d (в очереди %d)\n",
               value, cell, q->count);
    } else {
        printf("enqueue %d: очередь полна\n", value);
    }
}

int main(void)
```

```
{
    struct Queue q;
    q.head = 0;
    q.count = 0;

    /* Шесть человек на пять мест — последний не влезет. */
    for (int value = 1; value <= 6; value++) {
        try_enqueue(&q, value);
    }

    /* Обслуживаем троих: голова уезжает вправо, слева пусто. */
    int x;
    for (int k = 0; k < 3; k++) {
        if (dequeue(&q, &x)) {
            printf("dequeue %d (осталось %d, head=%d)\n", x, q.count, q.head);
        }
    }

    /* Кольцо в деле: новые встают на освободившиеся ячейки в начале. */
    for (int value = 6; value <= 8; value++) {
        try_enqueue(&q, value);
    }

    while (dequeue(&q, &x)) {
        printf("dequeue %d (осталось %d, head=%d)\n", x, q.count, q.head);
    }

    /* Ещё один dequeue на пустой очереди: отказ, а не падение. */
    if (dequeue(&q, &x) == 0) {
        printf("dequeue на пустой очереди: вернул 0\n");
    }

    return 0;
}
```

Разбор:

- `queue_tail` — вся арифметика кольца в одном месте. Если голова на 3, а стоят двое, следующий встанет на `(3 + 2) % 5 = 0`, то есть в самое начало массива. Именно это нам и нужно.
- `enqueue` и `dequeue` устроены как `push`/`pop`: статус наружу, значение через `*out`.
- `void try_enqueue(...)` — вспомогательная функция ради печати. Она спрашивает `queue_tail` **до** вставки: после `enqueue` счётчик уже изменится, и ответ будет про следующую ячейку, а не про эту. Мелочь, о которую легко споткнуться.
- `q->head = (q->head + 1) % CAPACITY;` — голова тоже умеет заворачиваться в начало кольца.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 11-queue.c -o 11-queue
$ ./11-queue
enqueue 1 -> ячейка 0 (в очереди 1)
enqueue 2 -> ячейка 1 (в очереди 2)
enqueue 3 -> ячейка 2 (в очереди 3)
enqueue 4 -> ячейка 3 (в очереди 4)
enqueue 5 -> ячейка 4 (в очереди 5)
enqueue 6: очередь полна
dequeue 1 (осталось 4, head=1)
dequeue 2 (осталось 3, head=2)
dequeue 3 (осталось 2, head=3)
enqueue 6 -> ячейка 0 (в очереди 3)
enqueue 7 -> ячейка 1 (в очереди 4)
enqueue 8 -> ячейка 2 (в очереди 5)
dequeue 4 (осталось 4, head=4)
dequeue 5 (осталось 3, head=0)
dequeue 6 (осталось 2, head=1)
dequeue 7 (осталось 1, head=2)
dequeue 8 (осталось 0, head=3)
dequeue на пустой очереди: вернул 0
```

Вот ради этих строк всё и затевалось:

```
enqueue 6 -> ячейка 0 (в очереди 3)
```

Шестёрка встала в **нулевую** ячейку — ту самую, которую освободила единица. Кольцо замкнулось, и наивная версия тут сказала бы «места нет». Дальше семёрка и восьмёрка заняли ячейки 1 и 2, очередь снова полная, все пять мест в деле.

А теперь порядок выдачи: **4, 5, 6, 7, 8** — точно в том порядке, в каком вставали, хотя лежат они в массиве вперемешку (4 и 5 в конце, 6, 7, 8 в начале). Это FIFO. Сравни со стеком из прошлой части: там был обратный порядок.

Смотри и на `head`: он доходит до 4, потом становится 0 — тоже заворачивается. Последняя строка — отказ на пустой очереди, без падений.

Часть 4. Связный список

Зачем он, если есть массив

У массива (даже растущего) есть слабое место: чтобы вставить элемент в **середину** или в **начало**, надо сдвинуть всё, что правее, — это копирование, O(n). Связный список вставляет в начало за O(1): за одни и те же три шага, сколько бы узлов ни было. И не требует непрерывного куска памяти: узлы могут лежать где угодно, лишь бы каждый знал, **где следующий**.

Плата за это — потеря быстрого доступа по индексу. В массиве 500-й элемент `a[499]` достаётся за одно действие. В списке, чтобы добраться до 500-го узла, придётся начать с головы и пройти по `next` 499 раз. Каждая структура данных за что-то платит.

Связный список (linked list) — цепочка **узлов** (nodes). Каждый узел хранит своё значение и адрес (указатель) на следующий узел. Последний узел указывает в `NULL` — «дальше никого».

Картинка цепочки из трёх узлов:

```
head
 |
 ▼
[3 | ·]──▶[2 | ·]──▶[1 | ·]──▶ NULL
```

В каждой клетке слева значение, справа стрелка `next`. `head` — наш единственный вход в список: указатель на первый узел. Потеряешь `head` — потеряешь весь список.

Описываем узел через `struct`

Вот ради чего мы изучали структуры:

```c
struct Node {
    int value;              /* полезные данные узла */
    struct Node *next;      /* адрес следующего узла (или NULL) */
};
```

- `int value;` — то, что узел хранит.

- `struct Node *next;` — указатель на **такой же** узел. Заметь: внутри определения `struct Node` мы уже ссылаемся на `struct Node *`. Так можно — это указатель (адрес), его размер компилятору известен заранее, даже пока сам тип ещё описывается. А вот поле `struct Node next;` (без звёздочки) написать нельзя: получилась бы структура, содержащая саму себя, бесконечного размера. Компилятор такое отвергнет.

Узел, ссылающийся на узел того же типа, — и делает цепочку цепочкой.

Добавление в начало — по шагам

Пусть в списке уже `2 -> 1 -> NULL`, и мы хотим добавить `3`. Три шага:

1. Создаём новый узел через `malloc`, кладём в него `value = 3`.
2. Говорим новому узлу смотреть туда, куда сейчас смотрит `head`: `node->next = head;` — то есть на старый первый узел (`2`).
3. Делаем новый узел головой: `head = node;`.

До вставки:

```
head ──▶ [2|·] ──▶ [1|·] ──▶ NULL
```

Шаг 1–2: создали `[3|·]` и направили его `next` на текущую голову:

```
        [3|·]
          │ next
          ▼
head ──▶ [2|·] ──▶ [1|·] ──▶ NULL
```

Шаг 3: переставили `head` на новый узел:

```
head ──▶ [3|·] ──▶ [2|·] ──▶ [1|·] ──▶ NULL
```

Порядок шагов 2 и 3 **нельзя менять**. Если сначала сделать `head = node`, то адрес старой головы пропадёт: `head` уже указывает на новый узел, а `node->next` ещё не выставлен. Это называется **потеря головы списка** — вся остальная цепочка станет недостижимой и утечёт. Сначала `node->next = head`, потом `head = node`.

Пустой список (`head == NULL`) обрабатывается тем же кодом сам собой: `node->next` получит `NULL`, и новый узел окажется единственным и последним. Отдельный `if` не нужен.

Удаление головы

Убрать первый узел — три строчки, но порядок опять решает всё:

```
struct Node *pop_front(struct Node *head)
{
    if (head == NULL) {
        return NULL;                    /* удалять нечего */
    }
    struct Node *rest = head->next;     /* СНАЧАЛА запомнили хвост */
    free(head);                         /* потом освободили голову */
    return rest;
}
```

Если сделать наоборот — сначала `free(head)`, потом `head->next` — мы прочитаем поле из уже уничтоженного узла. Разберём это подробно чуть ниже, на живом примере.

Функция возвращает новую голову, а вызывающий её запоминает: `head = pop_front(head);`. Так принято, когда операция может изменить голову: возвращаем актуальный `head` наружу.

Удаление последнего

С последним узлом сложнее: чтобы отцепить его, нужен **предпоследний** — именно у него надо обнулить `next`. А назад по односвязному списку не пройдёшь, стрелки только вперёд. Значит, идём с головы и ищем узел, у которого `next->next == NULL`:

```
struct Node *pop_back(struct Node *head)
{
    if (head == NULL) {
        return NULL;                    /* удалять нечего */
    }
    if (head->next == NULL) {           /* в списке всего один узел */
        free(head);
        return NULL;                    /* список опустел */
    }
    struct Node *cur = head;
    while (cur->next->next != NULL) {   /* ищем ПРЕДпоследний узел */
        cur = cur->next;
    }
    free(cur->next);                    /* убрали последний */
    cur->next = NULL;                   /* предпоследний стал последним */
    return head;
}
```

Три случая, и все три обязательны:

- **список пуст** — делать нечего, возвращаем `NULL`. Без этой проверки `head->next` разыменовал бы `NULL` и программа упала бы;
- **ровно один узел** — предпоследнего не существует, поэтому просто освобождаем голову и возвращаем `NULL` (список опустел). Без этой ветки условие `cur->next->next` полезло бы в `NULL->next`;
- **два и больше** — идём до предпоследнего и отцепляем хвост.

`cur->next->next` читается по стрелкам слева направо: «у узла `cur` поле `next`; у того узла, в свою очередь, поле `next`». Цикл встаёт, когда эта двойная стрелка упирается в `NULL`, — значит, `cur->next` и есть последний узел.

И `cur->next = NULL;` не забудь: иначе предпоследний продолжит указывать на освобождённую память, и следующий же обход прочитает мёртвый узел.

Заметь цену: чтобы удалить последний узел, пришлось пройти весь список от головы. Удаление с конца у односвязного списка — O(n), а удаление головы — O(1).

Освобождение всего списка и главная ошибка

Обход: заводим «бегунок» `cur`, ставим на `head`, и пока он не `NULL`, работаем с `cur->value` и прыгаем дальше: `cur = cur->next`.

Освобождение выглядит так же, но есть нюанс. Вот как **нельзя**:

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;
    struct Node *next;
};

int main(void)
{
    struct Node *head = malloc(sizeof(struct Node));
    if (head == NULL) {
        return 1;
    }
    head->value = 1;
    head->next = NULL;

    struct Node *cur = head;
```

```
    while (cur != NULL) {
        free(cur);              /* сначала освободили... */
        cur = cur->next;        /* ...а потом читаем из мёртвого узла */
    }
    printf("список освобождён\n");
    return 0;
}
```

После `free(cur)` узел больше не наш: система вправе отдать эту память кому угодно и перезаписать. А мы лезем в него за `next`. Запускаем обычной сборкой:

```
$ cc -std=c11 -Wall -Wextra -O2 bad-free.c -o bad-free
$ ./bad-free
bad-free(46301,0x7ff84a85dc40) malloc: *** error for object 0x7fb:
  pointer being freed was not allocated
bad-free(46301,0x7ff84a85dc40) malloc: *** set a breakpoint in
  malloc_error_break to debug
zsh: abort      ./bad-free
$ echo $?
134
```

(Длинные строки я снова перенёс.) Что случилось: после `free` система записала в освобождённый узел свои служебные данные, и `cur->next` прочитал оттуда не `NULL`, а какое-то число `0x7fb`. Цикл решил, что узел есть, и на втором круге вызвал `free(0x7fb)`. Такой адрес `malloc` никогда не выдавал — библиотека это заметила и аварийно остановила программу (сигнал `abort`, отсюда код возврата 128 + 6 = 134). До строчки «список освобождён» дело так и не дошло.

Заметь, что в сообщении нет ни имени файла, ни номера строки: только «кто-то освобождает то, что не выделялось». Где именно — ищи сам. И это ещё удачный исход: при другом мусоре программа могла бы молча отработать и испортить чужие данные. А теперь под санитайзером:

```
$ cc -std=c11 -Wall -Wextra -O1 -g -fsanitize=address bad-free.c -o bad-free
$ ./bad-free 2>&1 | head -17
=================================================================
==70556==ERROR: AddressSanitizer: heap-use-after-free on address
  0x6020000000d8 at pc 0x00010cbd388b bp 0x7ff7b332b8d0 sp 0x7ff7b332b8c8
READ of size 8 at 0x6020000000d8 thread T0
    #0 0x00010cbd388a in main bad-free.c:21
    #1 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

0x6020000000d8 is located 8 bytes inside of 16-byte region
```

```
  [0x6020000000d0,0x6020000000e0)
freed by thread T0 here:
    #0 0x00010d13189b in free+0x8b
      (libclang_rt.asan_osx_dynamic.dylib:x86_64h+0xa689b)
    #1 0x00010cbd3837 in main bad-free.c:20
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

previously allocated by thread T0 here:
    #0 0x00010d131792 in malloc+0x82
      (libclang_rt.asan_osx_dynamic.dylib:x86_64h+0xa6792)
    #1 0x00010cbd37e0 in main bad-free.c:11
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)
```

Отчёт читается как протокол: `heap-use-after-free` — использование освобождённой памяти в куче; читали 8 байт (это размер указателя) в строке 21; освобождена память была строкой 20; выделена — строкой 11. Три строки исходника, три момента истории одного байта. Дальше идёт карта памяти, её я обрезал через `head -17`.

Правильный порядок — **сначала** запомнить адрес следующего узла во временную переменную, **потом** освобождать текущий:

```c
void free_list(struct Node *head)
{
    struct Node *cur = head;
    while (cur != NULL) {
        struct Node *next = cur->next;  /* СНАЧАЛА запомнили следующий */
        free(cur);                      /* потом удалили текущий */
        cur = next;                     /* перешли к запомненному */
    }
}
```

Переменная `next` объявлена **внутри** цикла — она нужна ровно на одну итерацию, и заводить её снаружи незачем.

Полная программа: `11-list.c`

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
```

```
    int value;              /* полезные данные узла */
    struct Node *next;      /* адрес следующего узла (или NULL) */
};

/* Создаёт узел со значением value, который смотрит на next.
 * Возвращает NULL, если памяти не дали. */
struct Node *make_node(int value, struct Node *next)
{
    struct Node *node = malloc(sizeof(struct Node));
    if (node == NULL) {
        return NULL;
    }
    node->value = value;
    node->next = next;
    return node;
}

/* Печатает список: "список: 3 -> 2 -> 1 -> NULL". */
void print_list(const struct Node *head)
{
    printf("список: ");
    for (const struct Node *cur = head; cur != NULL; cur = cur->next) {
        printf("%d -> ", cur->value);
    }
    printf("NULL\n");
}

/* Удаляет первый узел. Возвращает новую голову. */
struct Node *pop_front(struct Node *head)
{
    if (head == NULL) {
        return NULL;                    /* удалять нечего */
    }
    struct Node *rest = head->next;     /* СНАЧАЛА запомнили хвост */
    free(head);                         /* потом освободили голову */
    return rest;
}
```

```c
/* Удаляет последний узел. Возвращает голову (возможно, новую). */
struct Node *pop_back(struct Node *head)
{
    if (head == NULL) {
        return NULL;                    /* удалять нечего */
    }
    if (head->next == NULL) {           /* в списке всего один узел */
        free(head);
        return NULL;                    /* список опустел */
    }
    struct Node *cur = head;
    while (cur->next->next != NULL) {   /* ищем ПРЕДпоследний узел */
        cur = cur->next;
    }
    free(cur->next);                    /* убрали последний */
    cur->next = NULL;                   /* предпоследний стал последним */
    return head;
}

/* Освобождает весь список: на каждый malloc свой free. */
void free_list(struct Node *head)
{
    struct Node *cur = head;
    while (cur != NULL) {
        struct Node *next = cur->next;  /* СНАЧАЛА запомнили следующий */
        free(cur);                      /* потом удалили текущий */
        cur = next;                     /* перешли к запомненному */
    }
}

int main(void)
{
    struct Node *head = NULL;           /* пустой список */

    /* Добавляем 1..5 в начало списка. */
    for (int value = 1; value <= 5; value++) {
        struct Node *node = make_node(value, head);
        if (node == NULL) {
```

```
            printf("Не хватило памяти\n");
            free_list(head);            /* уже выделенное — вернуть */
            return 1;
        }
        head = node;                    /* новый узел стал головой */
        printf("добавили %d\n", value);
    }
    print_list(head);

    head = pop_front(head);
    printf("удалили голову:\n");
    print_list(head);

    printf("удаляем с конца, пока не опустеет:\n");
    while (head != NULL) {
        head = pop_back(head);
        print_list(head);
    }

    head = pop_back(head);              /* удаление из пустого списка */
    if (head == NULL) {
        printf("pop_back на пустом списке: голова так и осталась NULL\n");
    }

    /* Соберём список заново и освободим его одним махом. */
    for (int value = 10; value <= 30; value += 10) {
        struct Node *node = make_node(value, head);
        if (node == NULL) {
            printf("Не хватило памяти\n");
            free_list(head);
            return 1;
        }
        head = node;
    }
    print_list(head);

    free_list(head);
    head = NULL;                        /* адреса больше не действуют */
```

```
    printf("free_list: весь список освобождён\n");
    print_list(head);

    return 0;
}
```

Разбор оставшихся мест:

- `struct Node *head = NULL;` — пустой список. `NULL` здесь значит «узлов ещё нет». Всегда инициализируй голову в `NULL`: без этого в ней мусор, и первый же обход улетит по случайному адресу.
- `make_node` — вся возня с `malloc` в одном месте. `malloc(sizeof(struct Node))` просит память под **один** узел; `sizeof` сам посчитает нужный размер. Функция возвращает `NULL`, если памяти не дали, и это однозначный сигнал: `NULL` — не узел, тут спутать не с чем (в отличие от истории с `pop` и `-1`).
- проверка `if (node == NULL)` в `main` — а перед выходом `free_list`, чтобы не бросить уже выделенные узлы. В маленькой программе система и так всё вернёт при выходе, но привычка отпускать своё — правильная.
- `const struct Node *head` у `print_list` — печать список не меняет, и `const` это фиксирует.
- цикл обхода `for (cur = head; cur != NULL; cur = cur->next)` — пока бегунок указывает на узел, печатаем и сдвигаемся по `next`. У последнего узла `next` равен `NULL`, условие станет ложным, цикл закончится. На пустом списке цикл не выполнится ни разу и напечатается просто `список: NULL`.
- `head = NULL;` после `free_list` — сам `free_list` голову обнулить не может, она у него копия параметра. Не обнулишь — в `head` останется адрес освобождённого узла, и любое обращение по нему будет тем самым use-after-free. Такой указатель называют **висячим** (dangling).

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 11-list.c -o 11-list
$ ./11-list
добавили 1
добавили 2
добавили 3
добавили 4
добавили 5
список: 5 -> 4 -> 3 -> 2 -> 1 -> NULL
удалили голову:
```

```
список: 4 -> 3 -> 2 -> 1 -> NULL
удаляем с конца, пока не опустеет:
список: 4 -> 3 -> 2 -> NULL
список: 4 -> 3 -> NULL
список: 4 -> NULL
список: NULL
pop_back на пустом списке: голова так и осталась NULL
список: 30 -> 20 -> 10 -> NULL
free_list: весь список освобождён
список: NULL
```

Разберём по кускам. Добавляли 1, 2, 3, 4, 5, а список вышел `5 -> 4 -> 3 -> 2 -> 1`. Логично: каждый новый узел вставал **в начало**, оттесняя предыдущие. Вставка в начало вместе с `pop_front` — это готовый стек: последним пришёл — первым ушёл.

`pop_front` снял пятёрку — голова сменилась мгновенно, ничего искать не пришлось. Дальше четыре `pop_back` подряд съели список с конца: 1, 2, 3 и, наконец, одинокая четвёрка — тот самый особый случай «в списке всего один узел». Пятый `pop_back` уже на пустом списке спокойно вернул `NULL`.

Потом список собрался заново и был убит целиком одним `free_list`. Финальный `список: NULL` доказывает, что голова обнулена.

Теперь то, ради чего затевалась вся аккуратность, — санитайзеры. Собери и запусти так:

```
$ cc -std=c11 -Wall -Wextra -O1 -g -fsanitize=address,undefined \
     11-list.c -o 11-list-san
$ ./11-list-san
```

Вывод будет в точности тот же, что и выше, — ни одной лишней строки, ни единой жалобы: ни один узел не прочитали после смерти, за границы не вылезли. А утечки, как договорились, проверяем `leaks`:

```
$ cc -std=c11 -Wall -Wextra -g 11-list.c -o 11-list-g
$ leaks --atExit -- ./11-list-g | grep "leaks for"
Process 51190: 0 leaks for 0 total leaked bytes.
```

Восемь `malloc` (пять узлов, потом ещё три) и восемь `free`: каждый выданный узел вернули. Именно так и должна выглядеть программа с ручным управлением памятью.

Сколько стоит каждая операция

Сведём вместе, что мы узнали по дороге. `n` — сколько элементов сейчас лежит в структуре.

```
растущий массив
  доступ к элементу по индексу       O(1)
  добавить в конец                   O(1) амортизированно,
                                     отдельная вставка до O(n)
  вставить в начало или в середину   O(n): сдвиг всех правее
  найти значение                     O(n): смотрим по очереди

стек на массиве
  push, pop, peek                    O(1)

очередь на кольцевом буфере
  enqueue, dequeue                   O(1)
  (наивная, со сдвигом влево         dequeue O(n))

односвязный список
  вставить в начало, pop_front       O(1)
  pop_back, вставить в конец         O(n): ищем последний узел
  добраться до k-го узла             O(k)
  найти значение                     O(n)
```

Стек и очередь у нас фиксированной вместимости, поэтому переездов у них нет вовсе и все операции — честные O(1), без оговорок. Вставку в конец списка можно тоже сделать O(1), если хранить рядом с `head` ещё и указатель на последний узел, — но его придётся аккуратно обновлять в каждой операции.

Частые ошибки

Забытая `;` после `struct`. После закрывающей `}` определения структуры точка с запятой обязательна: `struct Node { ... };`. Забудешь — clang скажет прямо:

```
box2.c:5:2: error: expected ';' after struct
    5 | }
      |  ^
      |  ;
```

Точка вместо стрелки (и наоборот). Указатель — `p->field`, сама структура — `s.field`. Тут компилятор тоже помогает и даже подсказывает замену:

```
box.c:7:6: error: member reference type 'struct Box *' is a pointer;
           did you mean to use '->'?
    7 |     p.v = 1;
      |     ~^
      |      ->
```

Это тот приятный случай, когда сообщение об ошибке можно просто выполнить буквально.

`data = realloc(data, ...)` напрямую. При неудаче `realloc` вернёт `NULL`, ты затрёшь единственный адрес старого блока — и потеряешь и данные, и возможность освободить память. Пиши во временный указатель.

Путать `count` и `capacity`. «Сколько лежит» и «сколько влезает» — разные числа. Записать в `data[capacity]` — выход за границу; печатать `capacity` элементов вместо `count` — печать мусора.

Не проверять `malloc` на `NULL`. `malloc` может отказать. Обращение `node->value` при `node == NULL` — падение сразу.

Читать/писать за границей массива. `data[top]` при `top == CAPACITY` и `data[top]` при `top == -1` — обе беды из этой главы. Проверки на пустоту и на переполнение обязательны, и делать их надо **до** обращения, а не после.

Возвращать «особое» значение вместо статуса. `pop`, отдающий `-1` при пустом стеке, ломается, как только в стеке появится `-1`. Отдавай успех/неуспех отдельно, а значение — через указатель.

Потеря головы списка. Сначала `node->next = head;`, только потом `head = node;`. Перепутаешь порядок — вся цепочка станет недостижимой.

`free` перед чтением `next`. В цикле освобождения сначала запомни `cur->next` во временную переменную, потом `free(cur)`. Мы видели, чем это кончается: `heap-use-after-free`.

Висячий указатель после `free`. Освободил — обнули: `head = NULL`. Иначе в переменной остаётся адрес чужой теперь памяти, и однажды по нему обязательно кто-нибудь пройдётся.

Утечки памяти. На каждый `malloc`/`realloc` однажды должен прийтись `free`. У растущего массива — один `free` в конце; у списка — по одному `free` на каждый узел. Стек и очередь у нас на массиве внутри структуры, там `free` не нужен вовсе: память освободится сама, когда переменная выйдет из области видимости.

Не запускать под санитайзерами. Половина ошибок этой главы не проявляется на маленьких примерах: программа «работает», а память тихо портится. `-fsanitize=address,undefined`

находит их за секунду, а утечки на маке — `leaks --atExit`. Возьми за правило гонять так любой код с указателями.

Упражнения

1. Заведи в растущем массиве счётчик переездов и после ста вставок напечатай, сколько раз массив рос и до какой вместимости дорос. Проверь себя до запуска: при старте с двух ячеек вместимость идёт 2, 4, 8, 16… — какая из них первой перевалит за 100 и сколько удвоений для этого понадобится? Ожидаемый результат: `элементов: 100, переездов: 6, вместимость: 128` (2 → 4 → 8 → 16 → 32 → 64 → 128, шесть удвоений).
2. Сделай стек символьным: `char data[CAPACITY]` вместо `int`. Положи в него слово по буквам, потом снимай по одной и печатай — на экране слово развернётся задом наперёд. Сравни с переворотом из главы 10: стек делает то же самое, но без индексов. Не забудь взять `CAPACITY` не меньше длины слова, иначе `push` откажет на хвосте. Ожидаемый результат: для `hello` — `olleh`.
3. Напиши с помощью стека проверку **парности скобок**: строка вида `([]{})` правильная, а `([)]` и `((` — нет. Открывающую скобку кладём в стек, на закрывающей снимаем и сверяем пару. В конце стек должен оказаться пустым. Если на закрывающей скобке стек уже пуст — строка тоже неправильная. Ожидаемый результат: `([]{})` — правильная; `([)]` — неправильная (к `)` на вершине лежит `[`); `((` — неправильная (стек не опустел); `)(` — неправильная (закрывать нечего); пустая строка и `a(b)c` — правильные.
4. Добавь в очередь счётчик `total` — сколько элементов прошло через неё за всё время. Он только растёт при удачном `enqueue` и никогда не уменьшается. Ожидаемый результат: с тем же `main`, что в `11-queue.c`, `total` в конце равен 8 — пять первых встали, шестой получил отказ, потом встали ещё трое.
5. Напиши для списка функцию `int length(const struct Node *head)`, возвращающую число узлов, и `int contains(const struct Node *head, int x)`, отвечающую 1, если значение `x` в списке есть. Обе — обычным обходом, обе O(n). Ожидаемый результат: для списка `5 -> 4 -> 3 -> 2 -> 1` — длина 5, `contains(head, 3)` равно 1, `contains(head, 7)` равно 0; для пустого списка длина 0.
6. Добавь в список вставку **в конец**, чтобы порядок сохранялся: `1 -> 2 -> 3 -> NULL`. Подсказка: если список пуст, новый узел становится головой; иначе дойди обходом до узла, у которого `next == NULL`, и подвесь новый к нему. Не забудь проверить санитайзером и `leaks`. Ожидаемый результат: после вставок 1, 2, 3 — `список: 1 -> 2 -> 3 -> NULL`, санитайзер молчит, `leaks` показывает 0 утечек. Каждая такая вставка — O(n), потому что идём до конца.
7. (Посложнее.) Напиши `struct Node *reverse(struct Node *head)` — разворот списка на месте, без выделения новых узлов. Подсказка: иди по списку с тремя указателями (`prev`,

`cur`, `next`) и на каждом шаге переставляй `cur->next` на `prev`. В конце головой станет `prev`. Ожидаемый результат: `5 -> 4 -> 3 -> 2 -> 1 -> NULL` превращается в `1 -> 2 -> 3 -> 4 -> 5 -> NULL`; пустой список остаётся пустым, список из одного узла — тем же самым.

Что мы получили

Теперь в твоём арсенале четыре структуры данных и понимание, когда какая нужна.

Растущий массив — когда нужен быстрый доступ по индексу, а размер заранее неизвестен: `realloc` с удвоением даёт амортизированную O(1) на вставку, а «плюс один» стоит квадрат от числа вставок. **Стек** — когда важен обратный порядок, LIFO. **Очередь** — когда важен прямой порядок, FIFO, на кольцевом буфере, где `count` избавляет от путаницы «пусто или полно». **Связный список** — гибкая цепочка узлов на `struct` и указателях, с дешёвой вставкой в начало и ручным управлением памятью.

Плюс мы освоили `struct`, точку, стрелку `->`, `typedef` и приём «статус из функции, значение через указатель» — без всего этого в Си дальше никуда. И завели привычку гонять код с указателями под `-fsanitize=address,undefined`.

На этом часть про базовые алгоритмы и структуры данных закончена. Дальше начинается самое весёлое — **игры**. В следующей главе напишем первую: классическую **«Угадай число»**. Компьютер загадает число, ты будешь гадать, а программа — подсказывать «больше» или «меньше». Там пригодится почти всё, что мы прошли: ввод, цикл, ветвления, а ещё появятся случайные числа. Идём играть.

Итоговые программы главы — `code/11-dynarray.c`, `code/11-stack.c`, `code/11-queue.c`, `code/11-list.c`; замер стратегий роста — `code/11-growth.c`.

Глава 12. Игра «Угадай число»

Часть про алгоритмы позади. Теперь начинается самое весёлое — **игры**. Мы напишем настоящие маленькие программы, в которые можно играть, и по дороге выучим то, чего ещё не касались.

Первая игра — классика всех времён. Компьютер загадывает число от 1 до 100, а ты пытаешься его угадать. После каждой попытки программа подсказывает: «больше» или «меньше». В конце она поздравляет тебя и сообщает, за сколько попыток ты справился.

Звучит просто, но по пути мы разберём важную новую тему — **случайные числа**. Без них почти ни одна игра не обходится: бросок кубика, перемешивание колоды, появление врага в неожиданном месте — всё это случайность.

Что мы сделаем

К концу главы у тебя будет программа, которая:

- загадывает **случайное** число от 1 до 100 (от запуска к запуску разное);
- в цикле спрашивает твою догадку;
- отвечает «Больше» или «Меньше», подводя тебя к ответу;
- считает попытки;
- поздравляет, когда ты угадал, и не падает, если ввести ерунду.

Новое в этой главе:

- генератор псевдослучайных чисел: `rand`, `srand`, `time`;
- как из `rand()` получить число в нужном диапазоне (`% 100 + 1`);
- игровой цикл `while`, который крутится, **пока не угадал**;
- защита от неправильного ввода, включая слишком длинные числа;
- почему «бей в середину» угадывает число от 1 до 100 не больше чем за 7 попыток.

Разбираем задачу руками

Сыграем партию на бумаге. Пусть компьютер загадал **45** (мы-то этого не знаем, диапазон 1..100).

Самая глупая стратегия — называть числа подряд: 1, 2, 3… В худшем случае это 100 попыток. Долго.

Умнее — каждый раз бить в **середину** оставшегося диапазона. Это та самая идея, что и в бинарном поиске из главы 7, только теперь её выполняешь ты, а «массив» — это числа от 1 до 100.

```
диапазон 1..100, говорю 50  → «Меньше» (значит, секрет < 50) → 1..49
диапазон 1..49,  говорю 25  → «Больше» (значит, секрет > 25) → 26..49
диапазон 26..49, говорю 37  → «Больше» → 38..49
диапазон 38..49, говорю 43  → «Больше» → 44..49
диапазон 44..49, говорю 46  → «Меньше» → 44..45
диапазон 44..45, говорю 44  → «Больше» → 45..45
говорю 45 → «Угадал!»
```

Семь попыток вместо сотни. Середину я каждый раз брал как `(левый + правый) / 2` с отбрасыванием дробной части: (1 + 49) / 2 = 25, (44 + 49) / 2 = 46 (точно — 46,5).

Почему для 100 чисел семи догадок хватает **всегда**, а шести — не всегда? Посчитаем, сколько чисел можно «покрыть» за k вопросов. Первая догадка угадывает одно число — ту самую середину. Если мимо, остаётся одна из двух половин, и вторая догадка угадывает одну из двух новых середин. Третья — одну из четырёх, и так далее: за k догадок можно наверняка угадать не больше чем

```
1 + 2 + 4 + ... + 2^(k−1) = 2^k − 1  чисел
```

(`2^k` здесь — «2 в степени k», произведение k двоек: 2^3 = 2 × 2 × 2 = 8). Подставим: за 6 догадок — 2^6 − 1 = 63 числа, меньше сотни. За 7 — 2^7 − 1 = 127, больше сотни. Значит, 7 — это минимум, который гарантирует победу на 100 числах, и ни одна стратегия не обещает меньше.

В общем виде для чисел от 1 до n нужно наименьшее k, при котором 2^k − 1 ≥ n, то есть 2^k ≥ n + 1. Это $\log_2(n + 1)$, округлённый **вверх** до целого. `log₂` — двоичный логарифм из главы 7: «в какую степень надо возвести двойку, чтобы получить число» ($\log_2 8 = 3$, потому что 2^3 = 8). Для n = 100: $\log_2(101) \approx 6{,}66$, вверх — 7. Я проверил это программой, перебрав все сто загадок: худший случай — ровно 7 догадок (столько нужно для 37 загадок из ста), а в среднем выходит 5,8. Эту стратегию мы потом подарим компьютеру в упражнениях — пусть он сам угадывает твоё число.

А пока загадывать будет машина. Значит, сперва надо научить её придумывать случайное число.

Шаг 1. Откуда брать случайность

В Си генератор случайных чисел живёт в заголовке `<stdlib.h>` (*standard library* — «стандартная библиотека»). Главная функция — `rand` (от *random* — «случайный»). Сигнатура у неё простейшая:

```
int rand(void);
```

Читается так: «`rand` не принимает аргументов (`void` в скобках) и возвращает `int`». Возвращённое число лежит в диапазоне от 0 до константы `RAND_MAX` — она объявлена в том же `<stdlib.h>`. Стандарт обещает только, что `RAND_MAX` не меньше 32767; на практике обычно больше. Напечатай `printf("RAND_MAX = %d\n", RAND_MAX);` — у меня вышло `RAND_MAX = 2147483647`, ровно максимальное значение `int` (2^31 − 1). У тебя может быть другое число: `RAND_MAX` зависит от системы, потому его и не зашивают в код, а берут по имени.

Больше стандарт про `rand` почти ничего не обещает: ни формулу, ни качество, ни то, что все числа от 0 до `RAND_MAX` будут выпадать. Это не придирка. На моём маке (macOS, процессор Intel) `rand()` никогда не возвращает ни 0, ни сам `RAND_MAX`: я прогнал его по полному кругу, и все значения легли в диапазон от 1 до 2147483646. Разных значений получается 2147483646. Это число нам ещё пригодится.

Теперь напечатаем три «случайных» числа:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("%d\n", rand());
    printf("%d\n", rand());
    printf("%d\n", rand());
    return 0;
}
```

Новое:

- `#include <stdlib.h>` — подключаем библиотеку, где лежит `rand`.
- `rand()` — вызов функции без аргументов (скобки пустые). Возвращает целое число, которое мы тут же печатаем через `%d`.

Запустим:

```
16807
282475249
1622650073
```

Числа и правда выглядят случайными. Но запусти программу **ещё раз**:

```
16807
282475249
1622650073
```

Те же самые! И в третий раз будут те же. Это и есть проблема.

(Конкретные три числа у тебя могут отличаться: разные системы считают `rand` по разным формулам. А вот **повторяемость** будет ровно такая же — в этом и суть.)

Шаг 2. Почему `rand()` повторяется и что такое «засев»

`rand()` не достаёт числа из воздуха. Внутри он считает их по формуле из предыдущего числа. Самое первое число он берёт от стартового значения — **зерна** (англ. *seed*). По умолчанию зерно всегда одно и то же (стандарт Си прямо говорит: пока ты не засеял генератор сам, он ведёт себя так, будто засеян единицей), поэтому и вся цепочка чисел каждый запуск одинаковая.

Такие числа называют **псевдослучайными** («как будто случайными»): они выглядят вразнобой, но целиком определяются зерном. Одно и то же зерно — одна и та же цепочка. На моём маке формулу видно прямо по выводу: 16807 — это 16807 × 1, следующее число — 16807 × 16807 = 282475249, и дальше каждое число умножается на 16807 и берётся остаток от деления на 2147483647.

Для игры это никуда не годится: компьютер будет загадывать одно и то же число при каждом старте, и угадать его станет скучно.

Решение — **засеять** генератор разным зерном перед игрой. За это отвечает функция `srand` (*seed random* — «засеять случайность»), тоже из `<stdlib.h>`:

```
void srand(unsigned int seed);
```

Она принимает одно беззнаковое целое — зерно — и ничего не возвращает (`void`). Пользуются ей так:

```
srand(42);   /* засеяли зерном 42 */
```

Но если всегда писать `srand(42)`, зерно опять постоянное — толку ноль. Нам нужно зерно, которое **само меняется** от запуска к запуску. Идеальный кандидат — текущее время: оно не стоит на месте.

Шаг 3. Зерно из текущего времени — `time`

Узнать текущее время умеет функция `time` из заголовка `<time.h>` (*time* — «время»). На macOS и Linux вызов `time(NULL)` возвращает число секунд, прошедших с 1 января 1970 года (стандарт Си сам единицы не фиксирует, это договорённость POSIX — стандарта Unix-систем). Каждую секунду это число другое — годный источник свежего зерна.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    printf("%d\n", rand());
    printf("%d\n", rand());
    printf("%d\n", rand());
    return 0;
}
```

Разбор новой строки `srand((unsigned int)time(NULL));`:

- `#include <time.h>` — подключили заголовок ради `time`.
- `time(NULL)` — спрашиваем текущее время. Аргумент `NULL` означает «результат никуда дополнительно не записывай, просто верни его» (`time` умеет ещё писать ответ по переданному адресу, но нам это не нужно).
- `(unsigned int)` — приведение типа (см. главу 2). `time` возвращает специальный тип `time_t` (на моём маке это 8-байтовое целое), а `srand` ждёт `unsigned int` («беззнаковое целое» — целое без минуса, 4 байта). Лишние старшие байты при переходе отбрасываются, для зерна это не страшно. С флагами книги компилятор и без приведения промолчит, но с добавочным `-Wconversion` clang предупреждает: «implicit conversion loses integer precision». Приведение говорит ему и читателю: «да, я знаю, что обрезаю, так и задумано».
- `srand(...)` — засеваем генератор полученным зерном.

Теперь каждый запуск (если между ними прошла хотя бы секунда) даёт новую тройку чисел. Для игры этого хватает. Но настоящей случайностью это не стало: зерно — всего лишь текущая секунда, и её легко угадать. На моём маке видно даже больше. Первое число после `srand(t)` равно 16807 × t (с остатком от деления на 2147483647), поэтому через секунду оно вырастает ровно на 16807, а секрет `rand() % 100 + 1` из следующего шага — на 7: запустил игру, получил 85, через секунду — 92. Для «Угадай число» это безобидно, а там, где на кону деньги или пароли, `rand` не годится вовсе. Сама справка `man 3 rand` на маке называет его «bad random number generator» и отсылает к `arc4random`.

Почему `srand` зовут ровно один раз

Правило звучит так: **`srand` вызывают один раз, в самом начале программы.** Дальше сколько угодно дёргают `rand()` — он сам продвигается по цепочке.

Новички часто «чинят» случайность наоборот: раз `srand` даёт свежесть, надо звать его почаще. Посмотрим, что из этого выходит:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    for (int i = 0; i < 5; i++) {
        srand((unsigned int)time(NULL));   /* так делать НЕ надо */
        printf("%d\n", rand() % 100 + 1);
    }
    return 0;
}
```

```
34
34
34
34
34
```

Пять одинаковых чисел. Причина простая: `time(NULL)` меняется раз в **секунду**, а пять витков цикла компьютер проскакивает за микросекунды. Значит, все пять раз зерно одно и то же, и `rand()` пять раз стартует цепочку заново с одного и того же места — выдавая одно и то же первое число. Запусти программу через секунду — снова получишь пять близнецов, только

других (у меня вышло пять `41`: плюс те самые 7 за секунду). Так что `srand` — один раз, и только один.

(Строго говоря, если цикл попадёт ровно на смену секунды, числа разделятся на две группы, например `34 34 34 41 41`. Сути это не меняет.)

Шаг 4. Сужаем диапазон до 1..100

`rand()` возвращает огромное число (до миллиардов), а нам нужно от 1 до 100. Тут на помощь приходит **остаток от деления** `%`, который мы разбирали в главе 2.

Вспомни: `a % b` — это остаток от деления `a` на `b`. Каким бы большим ни было неотрицательное `a`, остаток от деления на 100 всегда попадает в диапазон **0..99** (остаток не может быть равен делителю или больше него). Слово «неотрицательное» тут важно: в главе 2 мы видели, что у отрицательного `a` остаток в Си получается со знаком минус (`-7 % 2` равно −1). Нас это не касается — `rand()` отрицательных чисел не возвращает.

```
rand() % 100        // число от 0 до 99
```

Нам нужно 1..100, а не 0..99. Просто прибавим 1 — весь диапазон сдвинется на единицу вверх:

```
rand() % 100 + 1    // число от 1 до 100
```

Разберём по шагам на примере. Пусть `rand()` вернул 16807 — то самое первое число из шага 1.

- `16807 % 100` = 7 (последние две цифры — это и есть остаток от деления на 100).
- `7 + 1` = 8.

Получили 8 — годное число из диапазона 1..100. Если бы остаток вышел 0, после `+ 1` стало бы 1 (нижняя граница); если остаток 99 — после `+ 1` станет 100 (верхняя). Концы диапазона на месте, ни 1, ни 100 не потеряны.

Общая формула на будущее: чтобы получить число от `min` до `max` включительно, пиши `rand() % (max - min + 1) + min`. Для 1..100 это `rand() % 100 + 1`.

Проверим, что числа правда попадают в диапазон:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    for (int i = 0; i < 5; i++) {
```

```
        printf("%d\n", rand() % 100 + 1);
    }
    return 0;
}
```

```
88
83
74
23
14
```

Все пять — в пределах 1..100. Генератор загадок готов.

Шаг 5. Одна попытка угадать

Сложим первую наивную версию игры: загадываем число, спрашиваем **одну** догадку, сравниваем.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    int secret = rand() % 100 + 1;

    int guess;
    printf("Угадай число от 1 до 100: ");
    scanf("%d", &guess);

    if (guess < secret) {
        printf("Больше.\n");
    } else if (guess > secret) {
        printf("Меньше.\n");
    } else {
        printf("Угадал!\n");
    }
```

```
    return 0;
}
```

Разбор:

- `int secret = rand() % 100 + 1;` — загадали число и положили в переменную `secret` («секрет»). Игрок его не видит.
- `scanf("%d", &guess);` — читаем догадку. Не забываем `&` перед `guess` (см. главу 2): `scanf` нужен **адрес**, куда записать ввод.
- сравнение в три ветки:
  - `guess < secret` — игрок назвал **меньше**, чем загадано, значит, нужное число **больше**. Печатаем «Больше».
  - `guess > secret` — наоборот, «Меньше».
  - иначе числа равны — «Угадал!».

Запуск:

```
Угадай число от 1 до 100: 50
Меньше.
```

Работает, но играть невозможно: программа задаёт **один** вопрос и завершается. Чтобы реально угадать, придётся запускать её снова и снова — и каждый раз будет новое число. Нужен **цикл**, который повторяет вопрос, пока не угадаешь.

Шаг 6. Игровой цикл — спрашиваем, пока не угадал

Обернём вопрос и сравнение в цикл `while`. Условие цикла — «пока догадка не равна секрету»:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    int secret = rand() % 100 + 1;
    int guess = 0;

    printf("Я загадал число от 1 до 100. Угадай!\n");

    while (guess != secret) {
```

```
        printf("Твой вариант: ");
        scanf("%d", &guess);

        if (guess < secret) {
            printf("Больше.\n");
        } else if (guess > secret) {
            printf("Меньше.\n");
        } else {
            printf("Угадал!\n");
        }
    }
    return 0;
}
```

Что изменилось:

- `int guess = 0;` — теперь догадку **обязательно** задаём заранее значением 0. Почему именно 0? Потому что условие `while` проверяется **до** первого ввода, а секрет всегда в диапазоне 1..100. Значит, `0 != secret` гарантированно истинно, и цикл точно зайдёт внутрь хотя бы раз. Если бы мы оставили `guess` без значения, там лежал бы мусор — могло бы случайно совпасть с секретом (маловероятно, но оставлять переменную без значения нельзя).
- `while (guess != secret)` — «пока `guess` **не равно** `secret`, повторяй тело». Оператор `!=` — «не равно» (см. главу 2). Как только игрок введёт число, равное секрету, условие станет ложным, и цикл закончится.
- внутри — тот же вопрос и то же сравнение, что и раньше, только теперь они повторяются.

Запуск:

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: 50
Меньше.
Твой вариант: 25
Больше.
Твой вариант: 37
Больше.
Твой вариант: 43
Больше.
Твой вариант: 45
Угадал!
```

Вот это уже игра! Цикл крутится, подсказки ведут к ответу, на верной догадке всё завершается.

Чего не хватает? Игра не считает попытки. Добавим счётчик.

Шаг 7. Считаем попытки

Заведём переменную-счётчик и будем увеличивать её на каждой догадке. Приём знакомый — так мы считали в циклах в главе 2.

```
int attempts = 0;
```

Внутри цикла, сразу после ввода, добавим:

```
attempts++;
```

`attempts++` — увеличить `attempts` на 1 (краткая запись `attempts = attempts + 1`). Ставим её после `scanf`, чтобы каждая введённая догадка засчитывалась как попытка.

А в ветке «Угадал!» напечатаем итог:

```
printf("Угадал! Это было число %d.\n", secret);
printf("Тебе понадобилось попыток: %d.\n", attempts);
```

Теперь игра поздравляет и сообщает результат. Осталась одна неприятность — что будет, если вместо числа ввести буквы?

Шаг 8. Защита от неправильного ввода

Проверим версию из шага 6 на прочность: вместо числа введём `abc`. Вот что видно на экране в первые доли секунды:

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: abc
Больше.
Твой вариант: Больше.
Твой вариант: Больше.
Твой вариант: Больше.
Твой вариант: Больше.
Твой вариант: Больше.
Твой вариант: Больше.
```

Это не обрезанный вывод, а **бесконечный**: «Больше» летит сплошным потоком, и остановить его можно только через Ctrl+C. Выше — просто то, что успело напечататься, прежде чем я его нажал. Почему так?

`scanf("%d", &guess)` ждёт **целое число**. Когда на входе буквы, он не может ничего прочитать: переменную `guess` не меняет (там остаётся прежний 0), а буквы **оставляет во входном буфере** — они никуда не делись. На следующем витке `scanf` снова натыкается на те же буквы, снова ничего не читает, и так бесконечно. Это и есть зацикливание. Заодно понятно, откуда именно «Больше»: `guess` так и остался нулём, а 0 меньше любого секрета из 1..100.

Такая же беда будет и в конце ввода. Если запустить программу и сразу нажать Ctrl+D («ввод кончился»), `scanf` тоже ничего не прочитает — и цикл снова закрутится вхолостую.

Спасает то, что `scanf` **возвращает значение** — сколько значений ему удалось прочитать (мы это уже делали в главе 2). Если он успешно прочёл наше число, вернёт 1. Если не смог (буквы вместо цифр) — вернёт 0. А если ввод вообще закончился (пользователь нажал Ctrl+D или ввод идёт из файла, который кончился) — вернёт особое значение `EOF` (*end of file* — «конец ввода»). `EOF` — это константа из `<stdio.h>`, обычно равная −1; сравнивать надо именно с именем `EOF`, а не с −1 вручную.

Первая, простая версия проверки выглядит так:

```
int rc = scanf("%d", &guess);
if (rc == EOF) {
    printf("\nИгра прервана. Я загадывал %d.\n", secret);
    return 0;
}
if (rc != 1) {
    printf("Это не число. Введи целое от 1 до 100.\n");
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        /* выбрасываем мусорный ввод до конца строки */
    }
    continue;
}
```

Разбираем по кускам:

- `int rc = scanf(...)` — сохраняем результат `scanf` в переменную `rc` (от *return code* — «код возврата»), чтобы проверить его.
- `if (rc == EOF)` — ввод иссяк. Дальше спрашивать некого: вежливо прощаемся и выходим через `return 0`. Без этой проверки на конце ввода программа тоже зациклилась бы.
- `if (rc != 1)` — `scanf` не смог прочитать число (попались буквы). Сообщаем об ошибке и **чистим буфер** от застрявшего мусора:

- `getchar()` — функция из `<stdio.h>`, читает **один символ** из ввода и возвращает его (или `EOF`, если ввод кончился). Возвращает она `int`, а не `char`, чтобы `EOF` не спутался ни с одним настоящим символом (см. главу 2).
- `while ((c = getchar()) != '\n' && c != EOF)` — крутим, пока не дочитаем до конца строки (символ `'\n'` — перевод строки от Enter) или до конца ввода. Так мы выбрасываем всё, что осталось от неудачной попытки, и следующий `scanf` начнёт с чистого листа. Само тело цикла пустое — вся работа в условии.
- `continue;` — прерываем текущий виток и сразу идём к проверке условия `while` (см. главу 2). Мы не считаем такой ввод за попытку и не сравниваем мусор с секретом — просто спрашиваем заново.

Маленькая деталь: счётчик `attempts++` теперь должен стоять **после** этих проверок — чтобы неудачный ввод не засчитывался как попытка.

Пустая строка (просто Enter) `scanf("%d")` не смущает: `%d` сам пропускает пробелы, табы и переводы строки, пока не найдёт цифры. Лишние Enter'ы программа молча проглотит и продолжит ждать число.

Буквы и Ctrl+D эта версия уже переживает. Но у неё остались две дыры, и обе я нашёл, подсовывая ей неудобный ввод.

Дыра первая: хвост после числа. Введём `50abc`. `scanf` прочитает `50`, вернёт 1, раунд пройдёт как обычно, а хвост `abc` останется в буфере и всплывёт на **следующем** витке:

```
Твой вариант: 50abc
Меньше.
Твой вариант: Это не число. Введи целое от 1 до 100.
```

Второе приглашение выглядит пустым не потому, что программа сломалась: ответ на него уже лежал в буфере. Игрок ввёл одну строку, а получил ход и ругательство. Честнее отвергнуть всю строку целиком.

Дыра вторая: слишком большое число. `%d` читает только то, что влезает в `int`; что будет с числом побольше, стандарт не определяет вовсе (это неопределённое поведение, см. главу 2: программа вправе сделать что угодно). Маленькая программа, которая делает `scanf("%d", &v)` и печатает результат, на моём маке показывает вот что:

```
in=99999999999999999999   -> rc=1 v=-1
in=2147483648             -> rc=1 v=-2147483648
in=4294967297             -> rc=1 v=1
```

`rc` равен единице — «прочитал успешно», — а в переменной чепуха. Первые две чепухи отсекла бы проверка диапазона 1..100, до которой мы ещё дойдём. А вот третья проскочит: игрок

набрал четыре миллиарда с хвостиком, а игра засчитала ему ход «1». Проверка диапазона тут не спасает.

Обе дыры закрываются двумя приёмами.

1. **Ширина поля: `%9d`.** Число между `%` и `d` ограничивает, сколько символов `scanf` возьмёт на это число. Девять символов — это самое большее 999999999, меньше максимума `int` (2147483647). Значит, переполниться `int` больше не может, неопределённое поведение исчезло. Всё, что длиннее, останется в буфере — и его поймает второй приём.
2. **Смотрим, что стоит сразу за числом.** После удачного `scanf` читаем ещё один символ через `getchar()`. Если там перевод строки (или ввод кончился) — строка была чистым числом. Если что-то другое (`abc` из `50abc` или десятая цифра из `4294967297`) — бракуем всю строку и дочищаем её до `'\n'`.

Есть ещё одна тонкость. Раньше `scanf` писал прямо в `guess`, а `guess` сидит в условии цикла `while (guess != secret)`. Если игрок введёт `45abc`, а секрет 45, то `guess` станет 45, мы строку забракуем — но цикл всё равно закончится, будто игрок угадал. Поэтому читаем в отдельную переменную `value` и переносим её в `guess`, только когда ввод прошёл все проверки.

Полная программа

Собираем всё вместе. Файл — `code/12-guess.c`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    int secret = rand() % 100 + 1;
    int guess = 0;
    int attempts = 0;

    printf("Я загадал число от 1 до 100. Угадай!\n");

    while (guess != secret) {
        printf("Твой вариант: ");

        int value;
```

```
        int rc = scanf("%9d", &value);
        if (rc == EOF) {           /* ввод закончился (Ctrl+D) */
            printf("\nИгра прервана. Я загадывал %d.\n", secret);
            return 0;
        }

        int next = getchar();      /* что стоит сразу за числом */
        while (next == ' ' || next == '\t') {
            next = getchar();      /* пробелы в конце прощаем */
        }
        if (rc != 1 || (next != '\n' && next != EOF)) {
            printf("Не понял. Введи одно целое число от 1 до 100.\n");
            while (next != '\n' && next != EOF) {
                next = getchar();  /* выбрасываем остаток строки */
            }
            continue;
        }

        if (value < 1 || value > 100) {   /* мимо правил игры */
            printf("Число должно быть от 1 до 100.\n");
            continue;
        }

        guess = value;
        attempts++;

        if (guess < secret) {
            printf("Больше.\n");
        } else if (guess > secret) {
            printf("Меньше.\n");
        } else {
            printf("Угадал! Это было число %d.\n", secret);
            printf("Тебе понадобилось попыток: %d.\n", attempts);
        }
    }

    return 0;
}
```

Что здесь нового по сравнению с шагом 8:

- `int value;` и `scanf("%9d", &value)` — читаем не больше девяти символов и не в `guess`, а во временную переменную.
- `int next = getchar();` — символ сразу за числом. Если `scanf` вернул 0 (буквы), это первый символ мусора; если вернул 1 — то, что стоит за цифрами.
- `while (next == ' ' || next == '\t')` — пробелы и табы после числа прощаем: `50` с лишним пробелом в конце — всё ещё нормальный ход.
- `if (rc != 1 || (next != '\n' && next != EOF))` — брак, если число не прочиталось **или** за ним стоит не конец строки. Оператор `||` — «ИЛИ», `&&` — «И» (см. главу 2), скобки группируют «за числом не `'\n'` и не `EOF`» в одно условие.
- внутри — тот же цикл чистки строки, только в нём мы продолжаем с уже прочитанного `next`, а не начинаем с нового `getchar()`: если `next` уже `'\n'`, чистить нечего.
- `if (value < 1 || value > 100)` — вежливость к игроку: если он ввёл число вне диапазона (например, 200 или −5), мы напоминаем правила, а не говорим «Меньше». Условие истинно, если число меньше 1 **или** больше 100.
- `guess = value;` — только теперь, когда ввод признан ходом, он попадает в `guess`.

Обрати внимание, где стоит `continue` и где `attempts++`. Число вне диапазона — такой же «не ход», как и буквы, поэтому бракуем его **до** счётчика, и попытка не тратится. Правило одно на все проверки: сначала отсеиваем всё, что ходом не является, и только потом считаем попытку.

И про границы. Секрет `rand() % 100 + 1` может оказаться и 1, и 100 — оба конца в игре. Проверка `value < 1 || value > 100` их пропускает (`1 < 1` ложно, `100 > 100` тоже ложно), так что назвать их разрешено. Ошибись мы со знаком (`value <= 1`), и секрет 1 стал бы неугадываемым — игра не кончилась бы никогда.

Как это работает

Пройдём по программе сверху вниз.

1. `srand((unsigned int)time(NULL))` засевает генератор текущим временем — один раз на старте. Благодаря этому каждый запуск даёт новую загадку.
2. `rand() % 100 + 1` берёт случайное число и ужимает его до диапазона 1..100. Результат прячем в `secret`.
3. `guess = 0` гарантирует, что цикл зайдёт внутрь хотя бы раз (0 не равно ни одному числу из 1..100).
4. Цикл `while (guess != secret)` крутится, пока игрок не назовёт секрет. Каждый виток:
   - печатаем приглашение;
   - читаем не больше девяти символов числа в `value`: конец ввода → выходим;

- смотрим, что за числом: не число или хвост мусора → ругаемся, дочищаем строку, начинаем виток заново;
- число вне 1..100 → напоминаем правила и тоже начинаем виток заново, не тратя попытку;
- кладём `value` в `guess` и засчитываем попытку;
- сравниваем догадку с секретом и печатаем подсказку;
- на совпадении печатаем поздравление со счётчиком. Тут же условие `guess != secret` становится ложным, и цикл завершается.

5. `return 0` — программа отработала без ошибок.

Почему цикл вообще может закончиться? Потому что единственный способ сделать `guess` равным `secret` — назвать это число, а секрет всегда лежит в 1..100, то есть в том самом диапазоне, который мы разрешаем вводить. Никакой «недостижимой» загадки тут не бывает.

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/12-guess.c -o guess
./guess
```

Пример партии (играем серединой диапазона, как разбирали руками):

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: 50
Меньше.
Твой вариант: 25
Больше.
Твой вариант: 37
Больше.
Твой вариант: 43
Больше.
Твой вариант: 46
Меньше.
Твой вариант: 44
Больше.
Твой вариант: 45
Угадал! Это было число 45.
Тебе понадобилось попыток: 7.
```

Теперь партия «на дурака»: подсунем буквы, число с хвостом, число длиннее `int`, число за верхней границей и отрицательное — и только потом сыграем всерьёз.

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: abc
Не понял. Введи одно целое число от 1 до 100.
Твой вариант: 50abc
Не понял. Введи одно целое число от 1 до 100.
Твой вариант: 4294967297
Не понял. Введи одно целое число от 1 до 100.
Твой вариант: 200
Число должно быть от 1 до 100.
Твой вариант: -5
Число должно быть от 1 до 100.
Твой вариант: 50
Меньше.
Твой вариант: 15
Угадал! Это было число 15.
Тебе понадобилось попыток: 2.
```

Пять браков подряд — и ни падения, ни зависания, ни лишней попытки в счётчике: «понадобилось попыток: 2», то есть 50 и 15. `50abc` больше не превращается в ход плюс ругательство, а четыре миллиарда не притворяются единицей. Считаются только настоящие ходы.

Отдельно проверим Ctrl+D — конец ввода посреди партии:

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: 50
Меньше.
Твой вариант: ^D
Игра прервана. Я загадывал 15.
```

`^D` — так терминал показывает нажатый Ctrl+D. Программа не зависла и не сделала вид, что ты проиграл: честно сказала, что игра прервана, и открыла загадку.

Остались границы. Секрет случайный, так что подходящей партии придётся подождать, но проверить стоит — 1 и 100 должны угадываться наравне со всеми:

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: 50
Больше.
Твой вариант: 100
```

```
Угадал! Это было число 100.
Тебе понадобилось попыток: 2.
```

С единицей то же самое: «Твой вариант: 1» → «Угадал! Это было число 1.». Оба конца диапазона на месте.

Частые ошибки

Забыли `srand`. Если не засеять генератор, `rand()` каждый запуск выдаёт одну и ту же последовательность — компьютер будет загадывать одно и то же число. Всегда зови `srand((unsigned int)time(NULL))` один раз в начале.

Зовут `srand` перед каждым `rand()`. Частая «починка наоборот». Мы это уже видели в шаге 3: пять витков цикла проходят за доли секунды, `time(NULL)` за это время не меняется, и все пять чисел выходят одинаковыми. `srand` — ровно один раз за программу.

Используют `rand()` без диапазона. `rand()` сам по себе возвращает гигантское число. Без `% 100 + 1` секрет окажется где-то в миллиардах, и угадать его нельзя. Не забывай ужимать диапазон.

Промахнулись с `% 100` против `% 101`. `rand() % 100` даёт 0..99. Чтобы получить 1..100, нужно `+ 1`. Если хочешь 0..100 (101 число), тогда `% 101`. Считай границы внимательно.

Забыли `&` в `scanf`. `scanf("%d", guess)` без `&` — почти гарантированное падение (см. главу 2); компилятор с `-Wall` об этом предупреждает, не пропускай его жалобы. Нужно `scanf("%d", &guess)`.

Не проверяют результат `scanf`. Если не ловить случай «введены не цифры», программа зациклится на мусоре во входном буфере, как мы видели в шаге 8. И то же самое случится на конце ввода: запусти наивную версию и сразу нажми Ctrl+D — «Больше» польётся бесконечным потоком. Поэтому проверок две: `rc == EOF` и `rc != 1`. А с числами к ним добавляется третья — что стоит сразу за числом (шаг 8).

Забыли почистить буфер. Проверить `scanf` мало. Если поймать `rc != 1`, напечатать ругательство и просто уйти на следующий виток, буквы так и останутся в буфере — и цикл всё равно зависнет, только теперь с красивым сообщением. Ругательство и `getchar()`-чистка ходят парой.

Верят `scanf` на слово при огромном числе. Голый `scanf("%d")` на числе, которое не влезает в `int`, даёт неопределённое поведение, и на моём маке `4294967297` превращается в 1 при `rc == 1` — мы это разбирали в шаге 8. Проверка диапазона такую чепуху не ловит, потому что 1 — законное значение. Ограничивай ширину (`%9d`) и смотри, что стоит за числом.

Ждут от `rand() % 100` идеальной честности. Остаток от деления чуть-чуть перекашивает вероятности. Посчитаем для моего мака. `rand()` выдаёт числа от 1 до 2147483646 — всего

2147483646 разных значений (шаг 1). Поделим на 100: 2147483646 = 100 × 21474836 + 46. Значит, первые 2147483600 значений раздают каждому остатку 0..99 ровно по 21474836 вариантов, а 46 «лишних» значений (от 2147483601 до 2147483646) дают ещё по одному варианту остаткам 1..46. Я проверил это, прогнав `rand()` по полному кругу: так и есть. Итог: секреты 2..47 выпадают с вероятностью 21474837 / 2147483646 ≈ 0,0100000003, остальные 54 — с вероятностью 21474836 / 2147483646 ≈ 0,0099999998. Разница — один шанс из двух миллиардов с лишним, для игры её не существует. На другой системе число значений `rand()` другое, и «везучими» окажутся другие секреты, но перекос будет того же порядка. Там, где равномерность правда важна, `%` заменяют более аккуратным отбором (лишние значения просто выбрасывают и зовут `rand()` ещё раз).

Упражнения

1. **Другой диапазон.** Сделай, чтобы компьютер загадывал число от 1 до 1000. Подсказка: меняется формула `rand() % ... + 1` и проверка диапазона ввода. Сколько попыток теперь нужно при стратегии «бей в середину»?

   Ответ: `rand() % 1000 + 1` и `value > 1000`. Попыток в худшем случае — 10: 2^9 − 1 = 511 меньше тысячи, а 2^10 − 1 = 1023 уже больше ($\log_2 1001 \approx 9{,}97$, вверх — 10). Десятикратно больший диапазон добавил всего три попытки.

2. **Ограничь число попыток.** Дай игроку не больше 7 попыток. Если он не угадал за 7 ходов — выведи «Ты проиграл, я загадывал N» и заверши игру. Подсказка: добавь в условие цикла проверку `attempts < 7` через `&&`.

   Решение: заголовок цикла становится `while (guess != secret && attempts < 7)`, а после цикла, перед `return 0`, — `if (guess != secret) printf(...)`. Брак ввода попытку по-прежнему не тратит. Семь попыток выбраны не случайно: по расчёту из начала главы игрок, который бьёт в середину, **всегда** успевает. С лимитом 6 уже нет: за шесть догадок покрывается не больше 63 чисел, и при игре серединой 37 загадок из ста требуют седьмой попытки — на них игрок проиграет.

3. **«Горячо/холодно».** Кроме «Больше»/«Меньше» подсказывай, насколько игрок близок. Если разница между догадкой и секретом меньше 5 — печатай «Горячо!», если меньше 15 — «Тепло», иначе — «Холодно». Подсказка: пригодится модуль разности (расстояние между числами без знака: у 42 и 45 оно 3, у 45 и 42 — тоже 3).

   Решение: перед сравнением добавь `int diff = (guess > secret) ? guess - secret : secret - guess;` (тернарный оператор `?:` — «если условие, то первое, иначе второе») и три ветки по `diff`, когда `diff != 0`. Партия с секретом 45: `10` → «Холодно. Больше.», `35` → «Тепло. Больше.», `42` → «Горячо! Больше.», `45` → «Угадал!».

4. **Сыграй ещё раз.** После завершения партии спроси «Сыграем ещё? (1 — да, 0 — нет)» и, если игрок ввёл 1, загадай новое число и начни сначала. Подсказка: оберни всю игру во внешний цикл. Не забудь обнулять `guess` и `attempts` в начале каждой партии, а `srand` оставь **снаружи** цикла — один раз на всю программу.

5. **Компьютер угадывает.** Поменяй роли: теперь **ты** загадываешь число (держишь его в голове), а компьютер угадывает по стратегии «бей в середину». Программа называет число, ты отвечаешь буквой: `b` — «больше», `m` — «меньше», `t` — «точно, угадал». Компьютер ведёт границы `low` и `high`, каждый раз называя `(low + high) / 2`, и сужает диапазон по твоему ответу. Это тот самый бинарный поиск из главы 7, только вживую. Проверь: за сколько ходов компьютер загоняет любое число от 1 до 100?

   Решение:

```c
#include <stdio.h>

int main(void)
{
    int low = 1, high = 100, moves = 0;

    printf("Загадай число от 1 до 100, а я угадаю.\n");
    printf("Отвечай: b - больше, m - меньше, t - угадал.\n");

    while (low <= high) {
        int mid = (low + high) / 2;
        moves++;
        printf("Это %d? ", mid);

        char answer;
        if (scanf(" %c", &answer) != 1) {
            printf("\nВвод закончился.\n");
            return 0;
        }
        if (answer == 't') {
            printf("Ура! Угадал за %d ходов.\n", moves);
            return 0;
        } else if (answer == 'b') {
            low = mid + 1;
        } else if (answer == 'm') {
            high = mid - 1;
```

```
        } else {
            printf("Не понял, ответь b, m или t.\n");
            moves--;              /* этот ход не считаем */
        }
    }
    printf("Так не бывает: ты где-то ошибся в ответах.\n");
    return 0;
}
```

`scanf(" %c", ...)` читает один символ, пропустив пробелы и переводы строки (подробно этот пробел разберём в главе 14). Если ответы противоречат друг другу, границы перехлестнутся (`low > high`), и программа честно скажет, что так не бывает. Я прогнал её автоматически на всех ста загадках: худший случай — 7 ходов (как и обещает $\log_2 101 \approx 6{,}66$, вверх — 7), в среднем — 5,8. Для загаданного 45 диалог такой: 50 `m`, 25 `b`, 37 `b`, 43 `b`, 46 `m`, 44 `b`, 45 `t` — «Ура! Угадал за 7 ходов.»

6. **Счётчик рекордов.** Запоминай лучший результат (наименьшее число попыток) за несколько партий и показывай его перед каждой новой игрой. Подсказка: делается поверх упражнения 4. Заведи снаружи цикла `int best = 0;` (0 — «рекорда ещё нет») и после победы пиши `if (best == 0 || attempts < best) best = attempts;`. Ожидаемо: после партий на 6, 4 и 5 попыток рекорд — 4.

Что мы получили

Мы написали первую полноценную игру и заодно открыли тему, без которой дальше никуда, — **случайные числа**. Теперь ты знаешь, как засеять генератор временем (`srand((unsigned int)time(NULL))`), как получить случайное число (`rand()`) и как загнать его в нужный диапазон (`rand() % 100 + 1`). Ты собрал игровой цикл `while`, который крутится, пока не выполнено условие победы, научился считать попытки и не дал программе сломаться от неправильного ввода — проверяя результат `scanf`, ограничивая ширину числа (`%9d`), глядя на символ за числом и подчищая входной буфер. А ещё увидел, что «случайные» числа `rand` на деле псевдослучайные, что `% 100` даёт крошечный, но настоящий перекос и что за 7 вопросов «бей в середину» угадывает любое число от 1 до 100.

В следующей главе сделаем ещё одну классику — **«камень, ножницы, бумага»**. Компьютер будет выбирать ход случайно (`rand` уже в кармане), а нам предстоит разобрать чуть более хитрую логику сравнения ходов и аккуратно посчитать счёт за несколько раундов.

Глава 13. Игра «Камень, ножницы, бумага»

В прошлой главе компьютер загадывал число, а ты угадывал. Теперь поменяем роли местами: ты и компьютер играете на равных в «Камень, ножницы, бумага». Компьютер делает случайный выбор, ты — свой, и кто-то выигрывает раунд. Сыграем несколько раундов, посчитаем очки и в конце объявим победителя матча.

По дороге познакомимся с новой полезной штукой — **перечислением** (`enum`), научимся аккуратно превращать выбор игрока в число, разберём красивый приём с остатком от деления для определения победителя и сделаем ввод устойчивым к ерунде вроде `hello` или `5`.

Что мы сделаем

К концу главы у нас будет программа `code/13-rps.c`, которая:

- предлагает выбрать камень, ножницы или бумагу (или выйти);
- сама случайно выбирает свой ход;
- говорит, кто выиграл раунд;
- ведёт счёт игрока и компьютера;
- играет столько раундов, сколько захочешь, а по команде выхода печатает итог матча.

Всё, что нужно, у тебя уже есть из прошлых глав: переменные, `if`, циклы, функции (глава 2), массивы и строки (глава 3), а также `rand`/`srand`/`time` для случайных чисел (глава 12). Случайность мы заново разбирать не будем — просто пользуемся.

Разбираем задачу руками

Сначала договоримся о правилах. Их три:

- камень бьёт ножницы (тупит лезвие);
- ножницы бьют бумагу (режут);

- бумага бьёт камень (заворачивает).

Если оба выбрали одно и то же — ничья.

Чтобы компьютер мог «думать», выбор нужно превратить в число. Закодируем ходы так:

- `0` — камень;
- `1` — ножницы;
- `2` — бумага.

Кодировка не случайна. Посмотри, что бьёт что: камень (0) бьёт ножницы (1), ножницы (1) бьют бумагу (2), бумага (2) бьёт камень (0). Каждый ход бьёт **следующий по кругу**: 0→1→2→0. Эта закономерность скоро очень нам пригодится.

Выпишем все девять исходов в таблицу. Слева — ход игрока, сверху — ход компьютера, в клетке — кто выиграл (И = игрок, К = компьютер, Н = ничья):

```
                компьютер
              камень ножницы бумага
игрок    ┌──────────────────────────
камень   │   Н       И        К
ножницы  │   К       Н        И
бумага   │   И       К        Н
```

По диагонали — ничьи (выбрали одинаково). Остальные клетки — кто кого побил по правилам выше. Эту таблицу мы и научим программу заполнять.

Шаг 1. Один раунд, выбор компьютера

Начнём с самого маленького: компьютер делает случайный ход, мы его печатаем. Случайное число от 0 до 2 даёт `rand() % 3` — остаток от деления неотрицательного числа на 3 всегда попадает в диапазон 0..2 (см. главу 12). Честно ли он делит шансы между тремя ходами, проверим в конце главы.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    int computer = rand() % 3;
```

```
    printf("Компьютер выбрал: %d\n", computer);
    return 0;
}
```

Здесь всё знакомо по главе 12:

- `#include <stdlib.h>` — оттуда `rand` и `srand`.
- `#include <time.h>` — оттуда `time`.
- `srand((unsigned int)time(NULL));` — засеваем генератор текущим временем, чтобы от запуска к запуску числа были разными.
- `rand() % 3` — случайное число, остаток от деления на 3, то есть 0, 1 или 2.

Запуск печатает что-то вроде `Компьютер выбрал: 2`. Работает, но человеку «2» ни о чём не говорит. Нужно показывать слово, а не число.

Шаг 2. Превращаем число в слово

Можно было бы каждый раз писать `if (computer == 0) printf("камень")` и так далее. Но это придётся повторять для игрока и для компьютера — дублирование. Лучше сделать **функцию**, которая по числу возвращает название хода.

Функция должна вернуть текст. Текст в Си — это массив символов, а доступ к нему мы передаём через адрес первого символа, то есть через **указатель на `char`** (см. главу 3). Поэтому тип результата — `const char *`:

```
const char *move_name(int m)
{
    if (m == 0) return "камень";
    if (m == 1) return "ножницы";
    return "бумага";
}
```

Разбор того, что здесь нового:

- `const char *` — тип возвращаемого значения: «указатель на символы», то есть строка. Слово `const` означает «только для чтения»: вызывающий не должен менять эти буквы. Это разумно, ведь мы возвращаем **строковый литерал** — текст `"камень"`, зашитый в программу.
- `return "камень";` — возвращаем адрес начала этой строки. Строковый литерал живёт в памяти всё время работы программы, так что возвращать на него указатель безопасно.
- Третий вариант (`return "бумага";`) стоит без `if`: если число не 0 и не 1, остаётся только 2 — бумага.

Теперь вывод понятен человеку:

```
printf("Компьютер выбрал: %s\n", move_name(computer));
```

`%s` печатает строку по указателю, который вернула функция (см. главу 3).

Шаг 3. Даём имена числам — `enum`

Числа 0, 1, 2 работают, но в коде `if (m == 1)` непонятно, что такое «1». Через месяц ты и сам забудешь. В Си для таких случаев есть **перечисление** — `enum` (от англ. *enumeration*, «перечисление»).

`enum` позволяет дать числам понятные имена:

```
enum Move { ROCK, SCISSORS, PAPER };
```

Что здесь происходит:

- `enum Move { ... }` — объявляем новый набор именованных целых констант под общим названием `Move`.
- `ROCK, SCISSORS, PAPER` — имена. По умолчанию `enum` раздаёт значения с нуля по порядку: `ROCK` это 0, `SCISSORS` это 1, `PAPER` это 2. Ровно наша кодировка! Можно задать и явно (`enum Move { ROCK = 0, SCISSORS = 1, PAPER = 2 };`), но и так сходится.

Имена `enum` — это обычные целые числа, просто с человеческим лицом. Можно писать `ROCK` вместо `0` где угодно: в сравнениях, в `switch`, в арифметике. Компилятор подставит число. Перепишем `move_name` понятнее:

```
const char *move_name(int m)
{
    if (m == ROCK)     return "камень";
    if (m == SCISSORS) return "ножницы";
    return "бумага";
}
```

Логика та же, но теперь код читается как фраза: «если ход — камень, верни “камень”». Имена-константы делают программу понятной без комментариев.

Шаг 4. Кто победил — наивный способ

Теперь главное: по двум ходам определить победителя. Самый честный путь — просто перечислить все случаи, как в таблице. Сначала ничья, потом три случая победы игрока, остальное — победа компьютера.

```
int player = ROCK;        /* пока зашьём вручную, ввод будет позже */
int computer = rand() % 3;

if (player == computer) {
    printf("Ничья.\n");
} else if ((player == ROCK     && computer == SCISSORS) ||
           (player == SCISSORS  && computer == PAPER)    ||
           (player == PAPER     && computer == ROCK)) {
    printf("Ты выиграл раунд!\n");
} else {
    printf("Компьютер выиграл раунд.\n");
}
```

Разбор:

- `player == computer` — выбрали одинаково, значит ничья.
- большое условие через `||` («ИЛИ», см. главу 2) перечисляет три выигрышных для игрока расклада прямо по правилам: камень против ножниц, ножницы против бумаги, бумага против камня. Если хоть один верен — победа игрока.
- `else` ловит всё остальное: раз не ничья и не победа игрока, значит выиграл компьютер.

Это абсолютно правильно и понятно. Единственный минус — громоздко: три строки условий, которые легко набрать с опечаткой. Покажу способ короче.

Шаг 5. Красивый приём с остатком

Вернёмся к замеченной закономерности: каждый ход бьёт **следующий по кругу** (0→1→2→0). Посчитаем такое выражение:

```
int result = (computer - player + 3) % 3;
```

Идея: насколько ход компьютера «опережает» ход игрока по кругу. Разберём по кусочкам.

- `computer - player` — разница ходов. Может быть −2, −1, 0, 1 или 2.
- `+ 3` — добавляем 3, чтобы убрать минус. Зачем: в Си остаток от деления берёт знак **делимого** (см. главу 2), поэтому `-1 % 3` равно −1, а `-2 % 3` равно −2. В математике же остаток всегда от 0 до 2: −1 = 3 × (−1) + 2, значит, «математический» остаток от −1 равен 2. После `+ 3` число от −2 до 2 превращается в число от 1 до 5 — точно неотрицательное, и `%` Си уже совпадает с математикой.

- `% 3` — берём остаток от деления на 3. Прибавленная тройка остатка не меняет (3 делится на 3 без остатка: у 2 и у 2 + 3 = 5 остаток один и тот же, 2), зато минусы исчезли. Результат всегда 0, 1 или 2.

Что означают эти три значения? Проверим на каждой паре ходов:

- **0** — ходы равны (`computer == player`): ничья.
- **1** — компьютер на один шаг «впереди» игрока по кругу. Например, игрок камень (0), компьютер ножницы (1): `(1 - 0 + 3) % 3 = 1`. Камень бьёт ножницы — выиграл **игрок**.
- **2** — компьютер на два шага впереди (то же самое, что игрок на один шаг впереди компьютера). Например, игрок ножницы (1), компьютер камень (0): `(0 - 1 + 3) % 3 = 2`. Камень бьёт ножницы — выиграл **компьютер**.

Коротко: `0` — ничья, `1` — победа игрока, `2` — победа компьютера. Одна строка заменила всю таблицу.

Не верь на слово — проверь машиной

Два разобранных примера — это ещё не доказательство. Формулу легко принять на веру и потом полдня искать, почему игра «странно судит». Поэтому сделаем то, что вообще стоит делать с любым хитрым приёмом: **прогоним оба способа по всем девяти парам ходов и сверим ответы**.

Файл — `code/13-rps-check.c`:

```c
#include <stdio.h>

enum Move { ROCK, SCISSORS, PAPER };

const char *move_name(int m)
{
    if (m == ROCK)     return "камень";
    if (m == SCISSORS) return "ножницы";
    return "бумага";
}

/* Наивный способ: 0 - ничья, 1 - игрок, 2 - компьютер. */
int winner_if(int player, int computer)
{
    if (player == computer) {
        return 0;
    }
```

```
    if ((player == ROCK     && computer == SCISSORS) ||
        (player == SCISSORS && computer == PAPER)    ||
        (player == PAPER    && computer == ROCK)) {
        return 1;
    }
    return 2;
}

/* Тот же ответ одной формулой. */
int winner_mod(int player, int computer)
{
    return (computer - player + 3) % 3;
}

int main(void)
{
    const char *who[3] = { "ничья", "игрок", "компьютер" };

    for (int p = 0; p < 3; p++) {
        for (int c = 0; c < 3; c++) {
            int a = winner_if(p, c);
            int b = winner_mod(p, c);
            printf("игрок %d (%s) против %d (%s): "
                   "if=%d формула=%d -> %s%s\n",
                   p, move_name(p), c, move_name(c), a, b, who[b],
                   (a == b) ? "" : "   <-- РАСХОЖДЕНИЕ!");
        }
    }
    return 0;
}
```

Что тут нового:

- **два вложенных цикла** `for` — внешний перебирает ходы игрока (0, 1, 2), внутренний при каждом из них перебирает ходы компьютера. Три на три — ровно девять сочетаний, ни одного не пропустим.
- `const char *who[3]` — массив из трёх строк-названий; `who[b]` берёт название по номеру исхода. Это тот же приём, что и `move_name`, только через массив вместо `if`.

- `(a == b) ? "" : "..."` — **тернарный оператор**: `условие ? одно : другое`. Если условие истинно, всё выражение равно первому значению, иначе — второму. Здесь он подставляет пустую строку, когда способы согласны, и заметное «РАСХОЖДЕНИЕ!», когда нет. Так глазами искать не надо: расхождение само бросится в глаза.
- строка формата разбита на две части подряд (`"игрок..." "if=..."`) — два строковых литерала рядом Си склеивает в один. Удобно, когда строка не влезает в ширину экрана.

Компилируем и запускаем:

```
cc -std=c11 -Wall -Wextra -O2 code/13-rps-check.c -o rps-check
./rps-check
```

```
игрок 0 (камень) против 0 (камень): if=0 формула=0 -> ничья
игрок 0 (камень) против 1 (ножницы): if=1 формула=1 -> игрок
игрок 0 (камень) против 2 (бумага): if=2 формула=2 -> компьютер
игрок 1 (ножницы) против 0 (камень): if=2 формула=2 -> компьютер
игрок 1 (ножницы) против 1 (ножницы): if=0 формула=0 -> ничья
игрок 1 (ножницы) против 2 (бумага): if=1 формула=1 -> игрок
игрок 2 (бумага) против 0 (камень): if=1 формула=1 -> игрок
игрок 2 (бумага) против 1 (ножницы): if=2 формула=2 -> компьютер
игрок 2 (бумага) против 2 (бумага): if=0 формула=0 -> ничья
```

Девять строк, в каждой `if` и `формула` дали одно и то же, ни одного «РАСХОЖДЕНИЕ!». Сверь заодно колонку исходов с таблицей из начала главы — всё сходится: по диагонали ничьи, камень бьёт ножницы, ножницы бьют бумагу, бумага бьёт камень.

Приём работает только потому, что мы заранее **удачно занумеровали** ходы (каждый бьёт следующего по кругу). Поменяй кодировку — и формула сломается. Убедись сам: поменяй в `enum Move` местами `SCISSORS` и `PAPER` и перезапусти проверку — шесть строк из девяти загорятся «РАСХОЖДЕНИЕ!», и врать будет именно формула. Если такой трюк кажется слишком хитрым, оставь явные `if` из шага 4: они длиннее, но понятнее, и это нормальный выбор. Мы возьмём короткую формулу — теперь она подкреплена проверкой, а не обещанием.

Шаг 6. Ввод игрока с проверкой

Игрок должен вводить свой ход. Хочется принимать число (0, 1 или 2) и букву `q` для выхода, и при этом не падать от мусора вроде `hello` или `7`.

Прочитаем ввод как одно «слово» в буфер, а потом разберёмся, что это. Так мы не запутаемся, цифру ввели или букву.

```c
char buf[16];

if (scanf("%15s", buf) != 1) {
    break;              /* ввод кончился (Ctrl+D) — выходим */
}

if (buf[0] == 'q' || buf[0] == 'Q') {
    break;
}

if (buf[0] < '0' || buf[0] > '2' || buf[1] != '\0') {
    printf("Не понял. Введи 0, 1, 2 или q.\n");
    continue;
}

int player = buf[0] - '0';
```

Разбираем построчно:

- `char buf[16];` — массив символов на 16 байт под введённое слово (см. главу 3). С большим запасом.
- `scanf("%15s", buf)` — читаем слово (до пробела) в `buf`. Перед `buf` нет `&`: имя массива само по себе — адрес начала (см. главу 3). `%15s` ограничивает чтение 15 символами, чтобы не переполнить буфер из 16 (последний байт — под `'\0'`). `scanf` возвращает, сколько значений он успешно прочитал; `!= 1` означает «прочитать не удалось». Для `%s` это бывает только в конце ввода (Ctrl+D): «словом» считается любая непустая цепочка символов без пробелов, так что буквы, цифры и знаки `%s` прочитает всегда, а вернуть 0, как `%d` на буквах, не может. Тогда `break` выходит из цикла. Проверять результат `scanf` мы научились в главе 12.
- `buf[0] == 'q' || buf[0] == 'Q'` — первый символ буквы выхода, в любом регистре. Если это `q` — выходим из цикла через `break`.
- большое `if` — проверка корректности. Условие верно (значит ввод плохой), если выполнено хоть что-то:
  - `buf[0] < '0' || buf[0] > '2'` — первый символ не цифра из набора 0,1,2. Символы сравниваем как числа (у `char` есть числовой код, см. главу 2), а цифры в таблице символов идут подряд, поэтому такое сравнение работает.
  - `buf[1] != '\0'` — после первого символа сразу должен стоять конец строки `'\0'`. Если там что-то есть, значит ввели больше одного символа (например `12` или `5x`) — это не наш одиночный ход. Про `'\0'` как конец строки — глава 3.

    - при плохом вводе печатаем подсказку и `continue` — переходим к следующему витку цикла, заново спрашиваем ход.
- `int player = buf[0] - '0';` — превращаем символ-цифру в число. Хитрость в том, что коды символов `'0'`, `'1'`, `'2'` идут подряд. Значит `'0' - '0'` это 0, `'1' - '0'` это 1, `'2' - '0'` это 2. Вычитание кода `'0'` переводит символ цифры в само число. Это стандартный приём, запомни его.

Три честных замечания про эту защиту — чтобы ты знал её границы.

Во-первых, на выход мы смотрим только **первый** символ. Значит, `quit` и даже `qwerty` тоже закончат матч. Для игры это скорее удобно (человек пишет `quit` — программа его понимает), но помни: это осознанная поблажка, а не забытая проверка. Захочешь строгости — допиши `&& buf[1] == '\0'`, как в проверке цифры.

Во-вторых, `%s` читает **слово**, а не строку. Если набрать `0 1` и нажать Enter, программа сыграет два раунда подряд — сначала камнем, потом ножницами, не спрашивая второй раз: как хвост `abc` из `50abc` в первой версии проверки из главы 12, остаток строки просто дожидается следующего витка. Здесь это безвредно: каждое слово — законный ход, и каждый раунд честно засчитан.

В-третьих, `%15s` берёт не больше 15 символов за раз. Если сгоряча набрать двадцать букв, `scanf` отдаст первые 15, а оставшиеся пять подхватит **следующий** вызов — и «Не понял» напечатается дважды на один ввод. Программа не ломается, просто переспрашивает лишний раз. А вот убрать `15` из формата нельзя ни в коем случае: голый `%s` не знает, сколько места в приёмнике, и длинное слово уедет за границу `buf`, затирая соседнюю память (глава 3 предупреждала: следить за размером приёмника — работа программиста, никто её за тебя не сделает). Ошибка тихая и крайне неприятная: программа может ещё долго притворяться рабочей.

Полная программа

Собираем всё вместе: цикл раундов, счёт, итог матча. Это и есть файл `code/13-rps.c`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

enum Move { ROCK, SCISSORS, PAPER };

const char *move_name(int m)
{
    if (m == ROCK)     return "камень";
```

```
    if (m == SCISSORS) return "ножницы";
    return "бумага";
}

int main(void)
{
    srand((unsigned int)time(NULL));

    int player_score = 0;
    int computer_score = 0;
    int rounds = 0;
    char buf[16];

    printf("Камень, ножницы, бумага!\n");
    printf("Выбор: 0 – камень, 1 – ножницы, 2 – бумага, q – выход.\n");

    while (1) {
        printf("\nТвой ход: ");

        if (scanf("%15s", buf) != 1) {
            break;              /* ввод кончился (Ctrl+D) – выходим */
        }

        if (buf[0] == 'q' || buf[0] == 'Q') {
            break;
        }

        if (buf[0] < '0' || buf[0] > '2' || buf[1] != '\0') {
            printf("Не понял. Введи 0, 1, 2 или q.\n");
            continue;
        }

        int player = buf[0] - '0';
        int computer = rand() % 3;

        printf("Ты: %s. Компьютер: %s. ",
               move_name(player), move_name(computer));
```

```
        int result = (computer - player + 3) % 3;

        if (result == 0) {
            printf("Ничья.\n");
        } else if (result == 1) {
            printf("Ты выиграл раунд!\n");
            player_score++;
        } else {
            printf("Компьютер выиграл раунд.\n");
            computer_score++;
        }

        rounds++;
    }

    printf("\n=== Итог ===\n");
    printf("Сыграно раундов: %d\n", rounds);
    printf("Ты: %d, компьютер: %d.\n", player_score, computer_score);

    if (player_score > computer_score) {
        printf("Поздравляю, ты победил в матче!\n");
    } else if (player_score < computer_score) {
        printf("Компьютер оказался сильнее. В другой раз!\n");
    } else {
        printf("Ничья по матчу.\n");
    }

    return 0;
}
```

Как это работает

- В начале `main` заводим три счётчика: `player_score`, `computer_score` и `rounds` (сколько раундов сыграно). Все по нулям.
- `while (1)` — **бесконечный цикл**. Условие `1` — это всегда «истина» (ложь — 0, истина — любое ненулевое, см. главу 2). Сам по себе такой цикл не кончится никогда; выходим из него вручную через `break` — когда игрок ввёл `q` или ввод закончился. `break` мгновенно прекращает цикл и передаёт управление строкам после него.

- Внутри: спрашиваем ход, проверяем ввод (шаг 6), получаем `player`, бросаем кубик за компьютер (`rand() % 3`), печатаем оба хода.
- `result = (computer - player + 3) % 3` — определяем победителя одной формулой (шаг 5). По её значению печатаем итог раунда и при победе увеличиваем нужный счётчик через `++` (см. главу 2).
- `rounds++` в конце витка считает сыгранные раунды (плохой ввод сюда не доходит — там был `continue`).
- После цикла печатаем итог: сколько раундов, какой счёт, и кто победил в матче — сравнением `player_score` и `computer_score`.

Функция `move_name` стоит **выше** `main`, потому что `main` её вызывает, а функцию нужно объявить до вызова (см. главу 2).

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/13-rps.c -o rps
./rps
```

Сыграем матч из трёх раундов: камень, ножницы, бумага — и `q`. Ходы компьютера у тебя будут свои, они случайные:

```
Камень, ножницы, бумага!
Выбор: 0 - камень, 1 - ножницы, 2 - бумага, q - выход.

Твой ход: 0
Ты: камень. Компьютер: камень. Ничья.

Твой ход: 1
Ты: ножницы. Компьютер: бумага. Ты выиграл раунд!

Твой ход: 2
Ты: бумага. Компьютер: ножницы. Компьютер выиграл раунд.

Твой ход: q

=== Итог ===
Сыграно раундов: 3
```

```
Ты: 1, компьютер: 1.
Ничья по матчу.
```

Все три исхода в одной партии: ничья, победа игрока, победа компьютера. Сверь их с таблицей — ножницы против бумаги действительно наши, бумага против ножниц действительно нет.

Теперь защита ввода — подсунем мусор перед нормальным ходом:

```
Камень, ножницы, бумага!
Выбор: 0 – камень, 1 – ножницы, 2 – бумага, q – выход.

Твой ход: 5
Не понял. Введи 0, 1, 2 или q.

Твой ход: hello
Не понял. Введи 0, 1, 2 или q.

Твой ход: 1
Ты: ножницы. Компьютер: бумага. Ты выиграл раунд!

Твой ход: q

=== Итог ===
Сыграно раундов: 1
Ты: 1, компьютер: 0.
Поздравляю, ты победил в матче!
```

`5` и `hello` программа отвергла и переспросила, а `1` приняла и сыграла раунд. Ни падения, ни зацикливания, и в счёт раундов мусор не попал: сыгран ровно один.

Осталось проверить края. Что будет, если выйти сразу, не сыграв ни одного раунда? Или если ввод оборвётся на Ctrl+D? Оба случая дают одно и то же — и это правильно:

```
Камень, ножницы, бумага!
Выбор: 0 – камень, 1 – ножницы, 2 – бумага, q – выход.

Твой ход: q

=== Итог ===
Сыграно раундов: 0
```

```
Ты: 0, компьютер: 0.
Ничья по матчу.
```

Ноль раундов, ноль очков, «ничья по матчу». Программа не объявляет победителя из воздуха — просто честно показывает пустой итог. Ctrl+D вместо `q` даёт тот же результат, оба пути ведут к одному `break`:

```
Камень, ножницы, бумага!
Выбор: 0 — камень, 1 — ножницы, 2 — бумага, q — выход.

Твой ход: ^D
=== Итог ===
Сыграно раундов: 0
Ты: 0, компьютер: 0.
Ничья по матчу.
```

Разница только в одной пустой строке: после `q` ты нажимаешь Enter, и терминал переводит строку сам, а Ctrl+D никакого перевода не даёт — его место занимает `\n` из `printf("\n=== Итог ===\n")`.

Играть удобно и через конвейер (pipe), не набирая ходы руками:

```
printf '0\n1\n2\nq\n' | ./rps
```

Только имей в виду: в конвейере строчки «Твой ход:» останутся без ответа рядом. Эхо набранных символов рисует терминал, а тут никто ничего не набирает — программа читает ходы из трубы молча. Партия пройдёт правильно, но выглядеть будет непривычно.

Честная ли игра: считаем шансы

Против компьютера, который ходит наугад, нет выигрышной стратегии. Проверим это сначала рассуждением, потом машиной.

Вероятность события — это доля случаев, в которых оно происходит, если повторять опыт очень много раз. Монетка падает орлом с вероятностью 1/2: примерно в половине бросков из тысячи.

Пусть ты упрямо ставишь камень. Компьютер выбирает один из трёх ходов, и каждый — с вероятностью 1/3 (три равных варианта, как три грани у «трёхгранного кубика»). Из этих трёх ровно один (ножницы) проигрывает твоему камню, один (камень) даёт ничью и один (бумага) бьёт тебя. Значит, победа, ничья и поражение — по 1/3 каждая. Поставь вместо камня ножницы или бумагу — картина та же: у каждого твоего хода среди трёх ответов компьютера один

проигрывающий, один равный и один выигрывающий. Хоть по кругу ходи, хоть наугад — в долгой игре выйдет примерно поровну.

Рассуждение — ещё не проверка. Сыграем 300 тысяч раундов камнем против `rand() % 3` и посчитаем исходы той же формулой:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    const int rounds = 300000;
    int count[3] = {0, 0, 0};   /* ничьи, победы игрока, компьютера */

    for (int i = 0; i < rounds; i++) {
        int player = 0;                 /* упрямо ставим камень */
        int computer = rand() % 3;
        count[(computer - player + 3) % 3]++;
    }

    printf("ничьих:            %6d (%.4f)\n",
           count[0], (double)count[0] / rounds);
    printf("побед игрока:      %6d (%.4f)\n",
           count[1], (double)count[1] / rounds);
    printf("побед компьютера:  %6d (%.4f)\n",
           count[2], (double)count[2] / rounds);
    return 0;
}
```

Здесь всё знакомо. `count[(computer - player + 3) % 3]++` — формула из шага 5 даёт номер исхода (0, 1 или 2), и он же служит индексом в массиве счётчиков. `(double)count[0] / rounds` — доля исхода: без приведения к `double` деление было бы целочисленным и дало бы 0 (см. главу 2). `%6d` печатает число в поле шириной 6 символов, `%.4f` — дробь с четырьмя знаками после запятой.

Два запуска с разницей в секунду:

```
ничьих:            100291 (0.3343)
побед игрока:       99800 (0.3327)
побед компьютера:   99909 (0.3330)
```

```
ничьих:             99762 (0.3325)
побед игрока:       99996 (0.3333)
побед компьютера:  100242 (0.3341)
```

Все доли около 1/3 = 0,3333, как и предсказывал расчёт. Ровно по 100000 не выходит, и это нормально: случайность всегда даёт разброс. Для 300000 раундов типичное отклонение каждого счётчика от 100000 — около 260 (это корень из 300000 × 1/3 × 2/3), так что отклонения на две-три сотни — обычный шум, а не перекос.

А перекос от `% 3`, о котором мы говорили в главе 12, тут есть? На моём маке — нет вовсе: `rand()` выдаёт 2147483646 разных значений, а это число делится на 3 нацело (2147483646 = 3 × 715827882). Я прогнал `rand()` по полному кругу — каждый остаток 0, 1 и 2 встретился ровно 715827882 раза. На системе, где количество значений на 3 не делится, один-два хода получили бы по одному лишнему варианту из сотен миллионов — разницы не заметить.

Из этого есть полезное следствие для отладки: если за тысячи раундов компьютер выигрывает заметно чаще трети, ищи ошибку в формуле победителя, а не «невезение». А вот на десятке раундов счёт вроде 5:2 — обычное дело, по короткому матчу о честности не судят.

Частые ошибки

Перепутанная кодировка ходов. Формула `(computer - player + 3) % 3` работает только при нумерации 0=камень, 1=ножницы, 2=бумага (каждый бьёт следующего по кругу). Если занумеруешь иначе, формула начнёт врать — мы это видели, переставив в `enum` два имени. Тогда честнее вернуться к явным `if` из шага 4. И заведи себе привычку: любой приём, который работает «магически», проверяй прогоном по всем случаям, как в `13-rps-check.c`. Девять комбинаций перебираются за секунду, а пойманная так ошибка не доживёт до игрока.

Минус в остатке. Без `+ 3` выражение `(computer - player) % 3` для отрицательной разницы даст отрицательный остаток: `-1 % 3` равно −1, `-2 % 3` равно −2 (знак делимого, см. главу 2). Коварство в том, что ломается не всё. Я убрал `+ 3` в `13-rps-check.c` (с веткой `else` на всё, что не 0 и не 1), и расхождение загорелось только в одной строке из девяти: «бумага против камня» — разница −2, программа отдала раунд компьютеру, хотя бумага бьёт камень. Случай «ножницы против камня» (разница −1) случайно попал в `else` и оказался верным. Одна неверная клетка из девяти в живой игре всплывает редко — тем важнее проверка перебором. Тройка гарантирует неотрицательное число перед `%`.

`'1'` это не 1. Символ цифры и само число — разные вещи: код символа `'1'` равен 49, а не 1. Чтобы получить число, вычитаем `'0'`: `buf[0] - '0'`. Забудешь — получишь чушь.

Ход через `%d`. Если читать ход через `scanf("%d", ...)` без проверки результата, то на `hello` программа зациклится: `scanf` не сможет прочитать число и не сдвинет ввод (мы видели это в главе 12). Поэтому мы читаем слово через `%s` и разбираем его сами — так проще не запутаться.

Бесконечный цикл без выхода. В `while (1)` обязательно должен быть `break` (или другой выход), иначе программа повиснет. У нас выходов два — по `q` и по концу ввода, — и второй важнее, чем кажется. Забудешь проверить результат `scanf`, и на Ctrl+D программа закрутится вхолостую: буфер пуст, читать нечего, а цикл всё равно просит ход.

`%s` без ограничения длины. `scanf("%s", buf)` читает столько, сколько введут, и молча вылезает за границу массива. Всегда указывай предел — `%15s` для `char buf[16]`. Число в формате на единицу меньше размера массива: последний байт нужен под `'\0'`.

Счётчик раундов не там, где надо. `rounds++` стоит в самом конце витка, после всех проверок. Поставь его сразу после `printf("Твой ход: ")` — и в итог попадут «раунды», на которых игрок ввёл `hello`. Правило то же, что в главе 12: сначала отсеиваем всё, что ходом не является, и только потом считаем.

Упражнения

1. **До трёх побед.** Останавливай матч автоматически, как только кто-то набрал 3 победы (но не дольше 5 раундов — из-за ничьих до трёх побед можно и не дойти). Подскажу: добавь условие выхода из цикла по `player_score` и `computer_score`.

   Решение: сразу после `rounds++` вставь `if (player_score == 3 || computer_score == 3 || rounds == 5) break;`. Проверка: на ходах `0 1 2 0 1 2 0` матч заканчивается после пятого раунда, седьмой ход уже не читается, итог печатается как обычно.

2. **Пять ходов.** Добавь к игре «ящерицу» и «Спока» — есть такой известный вариант. Правила: камень бьёт ножницы и ящерицу, ножницы — бумагу и ящерицу, ящерица — бумагу и Спока, бумага — камень и Спока, Спок — камень и ножницы. У каждого хода две победы и два поражения. Можно ли обобщить формулу с остатком — или проще задать таблицу исходов массивом? И обязательно переделай `13-rps-check.c` под пять ходов: там будет уже 25 сочетаний, руками их не проверишь.

   Ответ: можно. Занумеруй ходы так, чтобы каждый бил **два следующих** по кругу: 0 — камень, 1 — ножницы, 2 — ящерица, 3 — бумага, 4 — Спок. Тогда `d = (computer - player + 5) % 5`: 0 — ничья, 1 или 2 — победа игрока, 3 или 4 — победа компьютера. Я сверил эту формулу с таблицей `beats[5][5]`, выписанной прямо по правилам: 25 пар, расхождений 0.

Заметь, почему ходов нечётное число: при n ходах у каждого n − 1 соперников, и поделить их поровну на «бью» и «проигрываю» можно, только если n − 1 чётно.

3. **Статистика.** В конце матча печатай, сколько было ничьих и какой ход ты выбирал чаще всего. Заведи счётчики ничьих и трёх ходов (можно массивом из трёх элементов, индекс — номер хода).

   Подсказка к решению: `int draws = 0; int used[3] = {0};`, в раунде — `used[player]++` и `draws++` в ветке ничьей. В конце найди индекс максимума в `used` (как в главе 6) и напечатай `move_name` от него. Ожидаемо: после ходов `0 0 1 q` — «чаще всего: камень (2 раза)». Число ничьих должно совпасть с `rounds - player_score - computer_score` — удобная самопроверка.

4. **Усиленная защита ввода.** Принимай не только цифры, но и первые буквы ходов. Русская буква в UTF-8 занимает два байта (см. главу 3), и `buf[0]` будет лишь первым из них — поэтому проще принимать латинские `r` (rock), `s` (scissors), `p` (paper).

   Подсказка к решению: перед проверкой цифры добавь `if (buf[1] == '\0' && buf[0] == 'r') player = ROCK;` и так далее, а цифровую ветку оставь как есть. Не забудь, что `q` уже занят выходом.

5. **Человеческая подсказка.** Если игрок проиграл раунд, печатай, каким ходом можно было побить ход компьютера.

   Решение: в нашей кодировке каждый ход бьёт следующий по кругу, значит, ход `m` бьёт тот, что стоит **перед** ним: `(m + 2) % 3` (прибавить 2 по кругу из трёх — то же, что отнять 1). В ветку проигрыша добавь `printf("Против хода \"%s\" надо было ставить \"%s\".\n", move_name(computer), move_name((computer + 2) % 3));`. Проверка: на камень советует бумагу, на ножницы — камень, на бумагу — ножницы.

6. **Серия побед.** Считай и показывай самую длинную серию подряд выигранных тобой раундов за матч.

   Подсказка к решению: две переменные — `streak` (текущая серия) и `best_streak`. Победа: `streak++`, и если `streak > best_streak`, обнови рекорд. Ничья или поражение: `streak = 0`. Для раундов «победа, победа, ничья, победа» ответ — 2.

Что мы получили

Мы собрали полноценную интерактивную игру: случайный соперник, проверка ввода, цикл раундов, счёт и итог матча. Попутно появилось перечисление `enum` — способ давать числам понятные имена, разобрали перевод символа-цифры в число через `- '0'` и красивый трюк с остатком от деления, который заменил целую таблицу одной строкой. А ещё посчитали шансы:

против случайного соперника победа, ничья и поражение — по 1/3 при любой твоей стратегии, и 300 тысяч сыгранных машиной раундов это подтвердили.

Но главное — не сам трюк, а то, как мы с ним обошлись. Вместо «поверь, это работает» написали `code/13-rps-check.c`, прогнали оба способа по всем девяти парам ходов и увидели совпадение своими глазами. Такая пятиминутная проверка стоит часа отладки: она ловит ошибку до того, как та превратится в «программа судит как-то странно».

В следующей главе возьмёмся за игру посерьёзнее — **«Виселицу»**. Там понадобится работать со строкой загаданного слова, отмечать угаданные и неугаданные буквы, считать ошибки и рисовать состояние игры. Массивы, строки и `'\0'` из главы 3 наконец развернутся в полную силу.

Глава 14. Игра «Виселица»

Угадал число — пора угадывать слова. «Виселица» (hangman) — это та самая игра, где компьютер загадывает слово, показывает его пустыми прочерками `_ _ _ _`, а ты называешь буквы по одной. Угадал букву — она открывается во всех своих местах. Промахнулся — теряешь жизнь и к виселице пририсовывается ещё одна часть тела. Открыл всё слово — победа; кончились жизни — проигрыш.

Игра простая на вид, но в ней соберётся всё, что ты уже знаешь: массив строк, случайный выбор, посимвольная работа со строкой, цикл, ветвления, флаги. И добавится пара новых приёмов: массив строк `const char *words[]` (сам тип `const char *` мелькнул в главе 13 в `move_name`, теперь разберём его как следует) и хитрый пробел в формате `scanf`, который чистит ввод от лишних переводов строки.

Что мы сделаем

Программу `code/14-hangman.c`, которая:

- держит список слов и выбирает одно случайно;
- показывает его как ряд прочерков, открывая угаданные буквы;
- читает букву с клавиатуры, приводит к нижнему регистру;
- проверяет, есть ли буква в слове; за промах снимает жизнь;
- рисует стадии виселицы ASCII-картинкой;
- объявляет победу, когда открыты все буквы, и поражение, когда кончились жизни.

Сразу важное предупреждение, как и в главе 10: мы считаем **символы**, а в UTF-8 русская буква занимает 2 байта, латинская — 1. Поэтому слова берём **английские** (латиница). Русский текст оставляем только для сообщений — `printf` печатает байты как есть, и они выглядят верно.

Разбираем задачу руками

Возьмём слово `apple`. Что происходит за одну партию?

```
секрет:  a p p l e
экран:   _ _ _ _ _      (пять прочерков — по числу букв)

игрок называет 'p'  -> 'p' есть в слове, открываем ОБА p:
экран:   _ p p _ _

игрок называет 'z'  -> 'z' в слове нет: минус жизнь
игрок называет 'a'  -> открываем: a p p _ _
игрок называет 'l'  -> a p p l _
игрок называет 'e'  -> a p p l e   -> все буквы открыты -> ПОБЕДА
```

Что нам нужно хранить, чтобы это повторить машиной:

1. **само слово** (`secret`);
2. **какие буквы уже названы** — чтобы знать, что открывать и не засчитывать повтор;
3. **сколько ошибок** сделано — для жизней и для картинки виселицы.

Заведём под пункт 2 массив на 26 английских букв: `guessed['a'-'a']`, `guessed['b'-'a']`, … По букве легко получить её номер: `c - 'a'`. Помнишь из главы 2, что `char` — это маленькое целое (код символа)? Значит `'c' - 'a'` равно 2, `'a' - 'a'` равно 0 — готовый индекс.

Дальше строим программу по кусочкам.

Шаг 1. Список слов и случайный выбор

Слова — это набор строк. Удобнее всего хранить их так:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main(void)
{
    const char *words[] = {
        "apple", "banana", "orange", "computer", "program",
        "keyboard", "function", "pointer", "rainbow", "elephant"
    };
    int word_count = (int)(sizeof words / sizeof words[0]);

    srand((unsigned int)time(NULL));
```

```
    const char *secret = words[rand() % word_count];

    printf("Загадано слово из %zu букв.\n", strlen(secret));
    printf("(для проверки: %s)\n", secret);
    return 0;
}
```

Здесь новое — строка `const char *words[]`. Разберём её по косточкам, это важный тип.

- Из главы 3 ты знаешь: строка `"apple"` живёт в памяти как массив символов, а её имя превращается в **адрес первого символа**. То есть каждый литерал в списке — это адрес строки.
- `const char *` — это «**указатель на символ**», то есть переменная, хранящая адрес начала строки. Слово `const` (константа) добавляет обещание: «через этот указатель я строку **менять не буду**». Строковые литералы в Си менять нельзя: попытка — неопределённое поведение (см. главу 2). Я проверил на маке строку `char *s = "apple"; s[0] = 'A';`: собранная с `-O0` программа упала с «Bus error», а с `-O2` молча напечатала `apple`, будто записи не было. Поэтому `const` тут не каприз, а защита: с ним компилятор просто не даст написать в такую строку — будет ошибка сборки.
- `const char *words[]` — массив таких указателей. Получается список адресов: `words[0]` — адрес `"apple"`, `words[1]` — адрес `"banana"`, и так далее. Размер `[]` пустой — компилятор сам посчитает по числу элементов в фигурных скобках.

Дальше:

- `sizeof words / sizeof words[0]` — стандартный приём «сколько элементов в массиве». `sizeof words` — размер всего массива в байтах, `sizeof words[0]` — размер одного элемента (одного указателя). Их частное и есть число слов. Так не придётся вручную писать `10` и ошибаться при добавлении слов. Результат `sizeof` — беззнаковый тип `size_t`, поэтому приводим к `int` через `(int)`, чтобы не было предупреждений при сравнении со счётчиком.
- `srand((unsigned int)time(NULL));` — засеваем генератор случайных чисел текущим временем (см. главу 12): без этого `rand()` каждый запуск выдавал бы одну и ту же «случайность».
- `words[rand() % word_count]` — `rand() % word_count` даёт случайный индекс от 0 до `word_count - 1`, по нему берём слово. Кладём его адрес в `const char *secret`. Перекос от `%` (глава 12) тут ещё меньше, чем там: делитель всего 10.
- `%zu` — спецификатор для `size_t` (то, что возвращает `strlen`). Печатать `size_t` через обычный `%d` — неверный тип; для него есть отдельный `%zu`.

Строку (`для проверки: %s`) мы потом уберём — она просто показывает загаданное слово, чтобы убедиться, что выбор работает.

```
Загадано слово из 6 букв.
(для проверки: orange)
```

Слово выбирается, длину знаем. Едем дальше.

Шаг 2. Показываем слово прочерками

Игрок не должен видеть слово — он видит ряд _. Напишем это отдельной функцией: пройдём по символам и для каждого напечатаем либо саму букву (если уже угадана), либо _.

Пока «угаданных» букв у нас нет, поэтому заведём массив-флаги `guessed` на 26 букв и передадим его в функцию. Все нули — значит, открытых букв нет, и функция нарисует одни прочерки.

```c
int draw_word(const char *word, const int *guessed)
{
    int revealed = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        char ch = word[i];
        if (guessed[ch - 'a']) {
            printf("%c ", ch);
            revealed++;
        } else {
            printf("_ ");
        }
    }
    printf("\n");
    return revealed;
}
```

Разбор функции:

- `int draw_word(const char *word, const int *guessed)` — принимает слово (указатель на строку) и массив флагов. Массив в Си передаётся как указатель на первый элемент (глава 3), поэтому параметр — `const int *guessed`. `const` снова обещает «менять не буду» — рисуем по флагам, но не трогаем их.
- `for (int i = 0; word[i] != '\0'; i++)` — идём по строке, пока не упрёмся в метку конца `'\0'`. Это привычный по главе 10 способ перебрать строку, не зная заранее её длины.
- `char ch = word[i];` — текущий символ.

- `guessed[ch - 'a']` — флаг для этой буквы. `ch - 'a'` — номер буквы (0 для `a`, 25 для `z`). Если флаг ненулевой — букву уже называли, печатаем её через `%c` и пробел, и считаем открытой (`revealed++`).
- иначе печатаем _ и пробел — буква спрятана.
- функция возвращает `revealed` — сколько букв открыто. Это нам пригодится для проверки победы: если открыто столько же, сколько всех букв, — слово отгадано.

Мина под этой функцией

Выражение `guessed[ch - 'a']` верно ровно до тех пор, пока `ch` — строчная латинская буква. Все наши слова такие, так что пока всё хорошо. Но представь: через неделю ты добавляешь в список загадку из двух слов — `"red apple"`. Дойдя до пробела, функция посчитает `' ' - 'a'`, то есть **−65**, и полезет в `guessed[-65]` — за границу массива, в чужую память. Программа не упадёт, ничего не сообщит, просто начнёт вести себя странно: у меня в игре она нарисовала пробел так, будто эта «буква» уже угадана, — в чужой памяти по адресу `guessed[-65]` случайно лежало не ноль.

Это неопределённое поведение, и «странно» тут может быть каким угодно. В маленькой тестовой программе, где слово `"red apple"` было вписано прямо в `main`, clang с `-O2` сам заметил выход за границу и вставил на его место аварийную остановку: программа умерла с «Illegal instruction», не успев ничего показать. А AddressSanitizer из главы 3, наоборот, в игре промолчал: `guessed[-65]` — это 260 байт до начала массива, и туда пришлась чужая, но «законная» память стека, которую санитайзер не охраняет.

Выход за границу массива — самая коварная ошибка в Си (глава 3 предупреждала): чаще всего она не падает, не ругается, просто тихо портит поведение, и даже санитайзер ловит её не всегда. Лечится одной проверкой в начале витка — вставим её сразу после `char ch = word[i];`:

```
        if (ch < 'a' || ch > 'z') {   /* не строчная латиница: */
            printf("%c ", ch);        /* показываем как есть,   */
            revealed++;               /* угадывать её не нужно  */
            continue;
        }
```

Логика простая: всё, что не буква от `a` до `z`, показываем открытым и сразу засчитываем в `revealed`. Пробел (или дефис, или что там ещё окажется в загадке) угадывать не нужно — он и так виден. А засчитать его в `revealed` обязательно: иначе `revealed` никогда не дорос бы до `len`, и слово с пробелом стало бы неугадываемым в принципе.

Четыре строчки, а закрывают сразу две дыры. Такие проверки удобно ставить сразу, пока помнишь, откуда берётся индекс, — потом никто уже не вспомнит, почему тут можно, а тут нельзя. Полный текст функции с этой вставкой ты увидишь в итоговом листинге главы.

Проверим в `main` (заведём пустой `guessed` и нарисуем):

```
int guessed[26] = {0};       /* все нули — ничего не открыто */
draw_word(secret, guessed);
```

`int guessed[26] = {0};` — массив из 26 целых, и `= {0}` заполняет **весь** массив нулями (см. главу 5: хватает одного нуля в фигурных скобках — остальные элементы тоже станут нулями). Вывод для `orange`:

```
_ _ _ _ _ _
```

Шесть прочерков. То, что нужно.

Шаг 3. Читаем букву — и ловим коварный перевод строки

Теперь спросим у игрока букву. Букву читают через `%c` в `scanf`. Напишем наивно:

```
char c;
printf("Твоя буква: ");
scanf("%c", &c);
printf("Ты ввёл: [%c]\n", c);

printf("Ещё буква: ");
scanf("%c", &c);
printf("Ты ввёл: [%c]\n", c);
```

`%c` читает **ровно один символ** (в отличие от `%d`, который пропускает пробелы и читает число). `&c` — адрес переменной, куда записать символ (как в главе 2).

Запускаем, вводим `a`, жмём Enter, потом `b`, Enter:

```
Твоя буква: a
Ты ввёл: [a]
Ещё буква: Ты ввёл: [

]
```

Что за ерунда? Второй раз мы и спросить толком не успели, а в `c` оказался перевод строки. Вот в чём дело: когда ты набрал `a` и нажал Enter, в буфер ввода попало **два** символа — сама буква `a` и невидимый символ перевода строки `'\n'` (от Enter). Первый `scanf` забрал `a`, а `'\n'` остался ждать. Второй `scanf("%c")` тут же схватил этот `'\n'` — вот он и «ввёлся».

Фикс — пробел перед `%c`: `scanf(" %c", &c)`. Пробел в строке формата означает «пропусти сначала любое количество пробельных символов» — пробелы, табы и переводы строки. То есть он съест застрявший `'\n'` (и любые лишние пробелы) и дойдёт до настоящей буквы.

```
char c;
printf("Твоя буква: ");
scanf(" %c", &c);          /* пробел перед %c чистит мусор */
```

Теперь оба ввода читаются как буквы. Запомни этот пробел — он спасает почти всегда, когда читаешь символы по одному.

Приводим к нижнему регистру

Игрок может ввести `A` или `a` — для нас это одна буква. Приведём заглавную к строчной:

```
if (c >= 'A' && c <= 'Z') {
    c = (char)(c - 'A' + 'a');
}
```

- `c >= 'A' && c <= 'Z'` — проверяем, что символ в диапазоне заглавных латинских букв. Сравнивать символы можно как числа — это и есть их коды.
- `c - 'A' + 'a'` — арифметика над кодами: вычли «начало заглавных», прибавили «начало строчных» — получили ту же букву в нижнем регистре (`'B'` → `'b'`). Приведение `(char)` — из-за типов: в Си результат такой арифметики имеет тип `int` (4 байта), а записываем мы его в `char` (1 байт). С флагами книги clang промолчит и без `(char)`, но более строгие режимы предупреждений (например, `-Wconversion` у gcc) на такое сужение ругаются. `(char)` говорит: «сужаю сознательно», значение-то всё равно от `'a'` до `'z'` и в `char` влезает.

И отсечём всё, что не английская буква (цифры, знаки, кириллица):

```
if (c < 'a' || c > 'z') {
    printf("Это не английская буква. Попробуй ещё.\n");
    continue;
}
```

`continue` — пропустить остаток текущего витка цикла и сразу перейти к следующему (см. главу 2; в главе 12 он так же отсеивал плохой ввод). То есть «плохой» ввод не тратит жизнь, мы просто переспрашиваем.

Порядок проверок тут не случайный: **сначала** приводим регистр, **потом** отсекаем не-буквы. Сделай наоборот — и заглавная `A` не пройдёт фильтр `c >= 'a' && c <= 'z'`, потому что код `'A'` (65) меньше кода `'a'` (97). Игрок ввёл нормальную букву, а программа его не поняла.

Проверим этот кусок отдельно

Не будем сразу вставлять всё в игру — соберём маленькую программку, которая только читает буквы и говорит, что о них думает. Так проще увидеть, как ведут себя края.

```c
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 4; i++) {
        char c;
        printf("Твоя буква: ");
        if (scanf(" %c", &c) != 1) {
            printf("\nВвод кончился.\n");
            return 0;
        }
        if (c >= 'A' && c <= 'Z') {
            c = (char)(c - 'A' + 'a');
        }
        if (c < 'a' || c > 'z') {
            printf("Это не английская буква. Попробуй ещё.\n");
            continue;
        }
        printf("Принято: '%c', номер в алфавите: %d\n", c, c - 'a');
    }
    return 0;
}
```

Скормим ей `5`, `B` и русскую `я`:

```
Твоя буква: 5
Это не английская буква. Попробуй ещё.
Твоя буква: B
Принято: 'b', номер в алфавите: 1
Твоя буква: я
Это не английская буква. Попробуй ещё.
Твоя буква: Это не английская буква. Попробуй ещё.
```

Цифру отвергли, заглавную `B` превратили в `b` и посчитали номер 1 — всё как задумано. А вот последние две строки стоит разобрать: ввели мы **одну** русскую букву, а ругательство напечаталось **дважды**.

Причина — та самая UTF-8, о которой предупреждали в начале главы. Русская `я` хранится как два байта, а `%c` читает по одному байту за раз. Первый байт на латинскую букву не похож — «не английская буква». Второй остаётся в буфере и на следующем витке читается сам собой, без

всякого набора с клавиатуры: отсюда и приглашение без ответа рядом, и второе ругательство. Ломаться при этом нечему — оба байта аккуратно отвергнуты, жизнь цела, индекс за границу не улетел. Просто игрок получит два сообщения вместо одного, и теперь ты знаешь, откуда они.

Шаг 4. Запоминаем букву и открываем её

Букву прочитали. Что с ней делать?

Сначала — не повторялась ли она:

```
if (guessed[c - 'a']) {
    printf("Букву '%c' ты уже называл.\n", c);
    continue;
}
guessed[c - 'a'] = 1;        /* отмечаем букву как названную */
```

Если флаг уже стоит — буква была, переспрашиваем (тоже без штрафа). Иначе ставим флаг в 1. С этого момента `draw_word` будет открывать эту букву во всех её местах автоматически — ведь функция рисует по флагам.

Теперь проверим, есть ли буква в слове:

```
int found = 0;
for (int i = 0; i < len; i++) {
    if (secret[i] == c) {
        found = 1;
        break;
    }
}
```

- `found` — флаг «нашли». Заводим как 0.
- идём по слову; как только нашли совпадение, ставим `found = 1` и `break` — выходим из цикла досрочно, дальше искать незачем.
- `len` — длина слова, посчитаем один раз через `strlen` (см. ниже).

Реакция:

```
if (found) {
    printf("Есть такая буква!\n");
} else {
    wrong++;
```

```
    printf("Мимо.\n");
}
```

Промах — `wrong++` (на одну ошибку больше). Попадание — ничего не снимаем.

Шаг 5. Жизни, победа и поражение

Соберём цикл партии. Ошибок разрешим, скажем, шесть — это и будет число стадий виселицы. Заведём константу:

```
#define MAX_WRONG 6
```

`#define` — это **директива препроцессора** (см. главу 5): перед компиляцией текст `MAX_WRONG` везде заменится на `6`. Удобно: захочешь сделать игру легче — поменяешь число в одном месте. Имена таких констант по традиции пишут заглавными.

Главный цикл крутится, пока ошибок меньше предела:

```
int len = (int)strlen(secret);
int wrong = 0;

while (wrong < MAX_WRONG) {
    int revealed = draw_word(secret, guessed);
    if (revealed == len) {
        printf("Победа! Слово: %s\n", secret);
        return 0;
    }
    /* ... спросить букву, обработать (шаги 3–4) ... */
}

printf("Ты проиграл. Слово было: %s\n", secret);
```

Как это работает:

- сначала рисуем слово и узнаём `revealed` — сколько букв открыто;
- если открыто столько же, сколько всего (`revealed == len`), — слово отгадано, печатаем победу и `return 0` (выходим из программы);
- иначе спрашиваем букву и обрабатываем её;
- промахи копят `wrong`; как только `wrong` дорастёт до `MAX_WRONG`, условие `while` станет ложным, цикл закончится — и сработает строка про проигрыш.

Заметь порядок: победу проверяем **в начале** витка, после отрисовки. Поэтому когда ты последней буквой открываешь всё слово, на следующем витке `draw_word` покажет полное слово и тут же объявит победу.

Шаг 6. Рисуем виселицу

Осталось украшение — ASCII-картинка, которая дорастает с каждой ошибкой. Сделаем массив из семи строк: на 0 ошибок, на 1, …, на 6. Индекс — число ошибок.

```
const char *gallows[MAX_WRONG + 1] = {
    "  +---+\n  |   |\n      |\n      |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  O   |\n      |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  O   |\n  |   |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  O   |\n /|   |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  O   |\n /|\\  |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  O   |\n /|\\  |\n /    |\n      |\n=========",
    "  +---+\n  |   |\n  O   |\n /|\\  |\n / \\  |\n      |\n========="
};
```

- это снова массив `const char *` — список строк, как наш список слов.
- `[MAX_WRONG + 1]` — нужно 7 ячеек: от 0 до 6 ошибок включительно.
- внутри строк `\n` — перевод строки: одна строковая константа рисует сразу несколько строк картинки.
- `\\` — это **один** символ обратной косой черты `\`. В строке `\` начинает спецсимвол (`\n`), поэтому сам бэкслеш записывают удвоенным — `\\`. Так рисуются руки и нога человечка.

Печатаем нужную стадию по числу ошибок: `printf("%s\n", gallows[wrong])`. А при проигрыше показываем последнюю, полную: `gallows[MAX_WRONG]`.

Набирать такую картинку в исходнике неудобно, а ошибиться в пробелах проще простого. Поэтому не поленись и посмотри на все семь стадий сразу — маленькой программой:

```
for (int i = 0; i <= MAX_WRONG; i++) {
    printf("--- ошибок: %d ---\n%s\n", i, gallows[i]);
}
```

```
--- ошибок: 0 ---
  +---+
  |   |
      |
      |
      |
=========
```

```
      |
=========
--- ошибок: 1 ---
  +---+
  |   |
  O   |
      |
      |
      |
=========
--- ошибок: 2 ---
  +---+
  |   |
  O   |
  |   |
      |
      |
=========
--- ошибок: 3 ---
  +---+
  |   |
  O   |
 /|   |
      |
      |
=========
--- ошибок: 4 ---
  +---+
  |   |
  O   |
 /|\  |
      |
      |
=========
--- ошибок: 5 ---
  +---+
  |   |
  O   |
```

```
 /|\  |
 /    |
      |
=========
--- ошибок: 6 ---
  +---+
  |   |
  O   |
 /|\  |
 / \  |
      |
=========
```

Виселица стоит с самого начала: на нулевой стадии только столб, перекладина и верёвка. Дальше по детали за ошибку — голова, туловище, левая рука, правая, левая нога, правая. Шесть деталей на шесть ошибок, всё сходится. Если у тебя человечек «съехал» или столб разошёлся, значит, потерян или добавлен пробел: пересчитай их в той строке, а лучше скопируй массив как есть.

Деталей хватает. Складываем программу целиком.

Полная программа

code/14-hangman.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_WRONG 6   /* сколько ошибок позволяем до проигрыша */

/* Рисует слово: открытые буквы — как есть, закрытые — подчёркиванием.
   Возвращает, сколько букв уже открыто. */
int draw_word(const char *word, const int *guessed)
{
    int revealed = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        char ch = word[i];
```

```
        if (ch < 'a' || ch > 'z') {   /* не строчная латиница: */
            printf("%c ", ch);        /* показываем как есть,   */
            revealed++;               /* угадывать её не нужно  */
            continue;
        }
        if (guessed[ch - 'a']) {
            printf("%c ", ch);
            revealed++;
        } else {
            printf("_ ");
        }
    }
    printf("\n");
    return revealed;
}

int main(void)
{
    const char *words[] = {
        "apple", "banana", "orange", "computer", "program",
        "keyboard", "function", "pointer", "rainbow", "elephant"
    };
    int word_count = (int)(sizeof words / sizeof words[0]);

    /* Стадии виселицы: индекс = число ошибок (0..MAX_WRONG). */
    const char *gallows[MAX_WRONG + 1] = {
        "  +---+\n  |   |\n      |\n      |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n      |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n  |   |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|   |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|\\  |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|\\  |\n /    |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|\\  |\n / \\  |\n      |\n========="
    };

    srand((unsigned int)time(NULL));

    const char *secret = words[rand() % word_count];
```

```
    int len = (int)strlen(secret);

    /* guessed[c - 'a'] == 1, если букву уже называли */
    int guessed[26] = {0};
    int wrong = 0;

    printf("Игра \"Виселица\". Угадай слово по буквам!\n");

    while (wrong < MAX_WRONG) {
        printf("\n%s\n\n", gallows[wrong]);

        int revealed = draw_word(secret, guessed);
        if (revealed == len) {
            printf("Победа! Слово: %s\n", secret);
            return 0;
        }

        printf("Ошибок: %d из %d\n", wrong, MAX_WRONG);
        printf("Твоя буква: ");

        char c;
        if (scanf(" %c", &c) != 1) {
            /* ввод кончился (Ctrl+D) — это не проигрыш, а выход */
            printf("\nИгра прервана. Слово было: %s\n", secret);
            return 0;
        }

        if (c >= 'A' && c <= 'Z') {
            c = (char)(c - 'A' + 'a');   /* в нижний регистр */
        }
        if (c < 'a' || c > 'z') {
            printf("Это не английская буква. Попробуй ещё.\n");
            continue;
        }
        if (guessed[c - 'a']) {
            printf("Букву '%c' ты уже называл.\n", c);
            continue;
        }
```

```
        guessed[c - 'a'] = 1;

        int found = 0;
        for (int i = 0; i < len; i++) {
            if (secret[i] == c) {
                found = 1;
                break;
            }
        }
        if (found) {
            printf("Есть такая буква!\n");
        } else {
            wrong++;
            printf("Мимо.\n");
        }
    }

    printf("\n%s\n\n", gallows[MAX_WRONG]);
    printf("Ты проиграл. Слово было: %s\n", secret);
    return 0;
}
```

Пара мест, которые мы ещё не отметили:

- `if (scanf(" %c", &c) != 1) { ... }` — `scanf` возвращает, сколько значений он успешно прочитал. Здесь ждём одно. Если вернулось не 1 — ввод кончился (нажали Ctrl+D или скормили файл, который закончился). Проверять результат `scanf` — хорошая привычка, мы завели её в главах 2 и 12.

  Внутри — `return 0`, а не `break`, и вот почему. Цикл заканчивается ровно в одном случае: игрок исчерпал все шесть ошибок. Поэтому сразу за циклом стоит «Ты проиграл» с полной виселицей. Если по Ctrl+D мы просто выйдем из цикла через `break`, программа объявит проигрыш человеку, у которого могло быть пять открытых букв и ни одной ошибки, и дорисует ему петлю. Это враньё. Прерванная игра — не проигрыш, поэтому выходим отдельным путём и говорим об этом честно.

- `printf("Игра \"Виселица\"...")` — внутри строки кавычку экранируют через `\"`, иначе компилятор решит, что строка тут и закончилась.

Как это работает

Один виток главного цикла:

1. печатаем виселицу по числу ошибок: `gallows[wrong]`;
2. рисуем слово прочерками и узнаём, сколько букв открыто;
3. если открыты все — победа, выходим;
4. показываем счётчик ошибок и просим букву;
5. читаем символ (`" %c"` чистит лишний `'\n'`), приводим к нижнему регистру, отсекаем не-буквы и повторы;
6. ищем букву в слове: попала — радуемся, промах — `wrong++`;
7. возвращаемся к шагу 1.

Когда `wrong` дорастает до `MAX_WRONG`, цикл заканчивается, и печатается проигрыш. Когда `revealed == len` — победа в шаге 3. Массив `guessed` связывает всё вместе: достаточно поставить один флаг, и буква сама «проявляется» при следующей отрисовке во всех своих позициях.

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/14-hangman.c -o hangman
./hangman
```

Удачная партия: выпало слово `banana`, называем `b`, `a`, `n`. Слово выбирается случайно, так что с первого раза может попасться другое — запускай, пока не выпадет нужное:

```
Игра "Виселица". Угадай слово по буквам!


  +---+
  |   |
      |
      |
      |
      |
=========


_ _ _ _ _ _
Ошибок: 0 из 6
Твоя буква: b
```

```
Есть такая буква!

  +---+
  |   |
      |
      |
      |
      |
=========

b _ _ _ _ _
Ошибок: 0 из 6
Твоя буква: a
Есть такая буква!

  +---+
  |   |
      |
      |
      |
      |
=========

b a _ a _ a
Ошибок: 0 из 6
Твоя буква: n
Есть такая буква!

  +---+
  |   |
      |
      |
      |
      |
=========

b a n a n a
Победа! Слово: banana
```

Шесть букв взяты тремя ходами. Мы назвали `a` один раз, а открылись **все три** `a` сразу: массив `guessed` хранит не «какие позиции открыты», а «какие буквы названы», и `draw_word` при каждой отрисовке заново решает судьбу каждой позиции. Победа объявлена не сразу после `n`, а на следующем витке, когда функция нарисовала полное слово и вернула `revealed == 6`. Виселица за партию не изменилась — ошибок не было.

Теперь края. Партия, в которой мы вводим заглавную `A`, потом повтор `a`, а потом выходим по Ctrl+D. Выпало слово `function`:

```
Игра "Виселица". Угадай слово по буквам!

  +---+
  |   |
      |
      |
      |
      |
=========

_ _ _ _ _ _ _ _
Ошибок: 0 из 6
Твоя буква: A
Мимо.

  +---+
  |   |
  O   |
      |
      |
      |
=========

_ _ _ _ _ _ _ _
Ошибок: 1 из 6
Твоя буква: a
Букву 'a' ты уже называл.

  +---+
  |   |
```

```
  O   |
      |
      |
      |
=========


_ _ _ _ _ _ _ _
Ошибок: 1 из 6
Твоя буква: ^D
Игра прервана. Слово было: function
```

Разбирай по шагам, тут сразу три проверки:

- **Регистр.** `A` превратилась в `a`, честно поискалась в `function`, не нашлась — «Мимо», минус жизнь. Появилась голова, счётчик стал «1 из 6». Игроку всё равно, на какой регистр он нажал.
- **Повтор.** Второе `a` уже отмечено в `guessed`, поэтому программа просто напоминает об этом. Смотри на счётчик: он так и остался «1 из 6», виселица не выросла. Жизнь за повтор **не** снимается — иначе игрок терял бы две за одну и ту же ошибку.
- **Ctrl+D.** Никакой петли и никакого «Ты проиграл»: игрок потерял одну жизнь из шести, объявлять его проигравшим было бы враньём. Программа честно говорит «Игра прервана» и открывает слово.

Цифры и русские буквы отсекает та же проверка `c < 'a' || c > 'z'`, которую мы уже гоняли отдельной программкой в шаге 3, — жизнь за них тоже не снимается.

Отдельно стоит убедиться, что игра не жульничает на последнем ходу. Сделаем пять промахов подряд и только потом откроем слово (на этот раз выпало `apple`). Вот конец такой партии:

```
a p p l _
Ошибок: 5 из 6
Твоя буква: e
Есть такая буква!


  +---+
  |   |
  O   |
 /|\  |
 /    |
      |
=========
```

```
a p p l e
Победа! Слово: apple
```

Пять ошибок, последняя жизнь — и победа засчитана. Порядок проверок (рисуем, сверяем `revealed == len`, потом спрашиваем букву) работает как надо.

А вот проигрыш — шестая ошибка. Последний экран программы:

```
  +---+
  |   |
  O   |
 /|\  |
 / \  |
      |
=========

Ты проиграл. Слово было: function
```

Человечек дорисован полностью (6 ошибок), игра честно объявила слово. Заметь, что виселицу тут печатает не цикл, а строка **после** него — цикл-то закончился, `wrong` дорос до `MAX_WRONG`, и внутрь мы больше не заходим.

Кстати, буквы можно скормить и через конвейер (pipe), не набирая их руками — `printf "b\na\nn\n" | ./hangman`. Партия пройдёт так же, только приглашения «Твоя буква:» останутся без ответа рядом: эхо набранного рисует терминал, а из трубы программа читает молча.

Частые ошибки

`scanf("%c")` без пробела. Главная ловушка главы. После ввода в буфере остаётся `'\n'` от Enter, и следующий `%c` читает его вместо буквы. Лечится пробелом: `scanf(" %c", &c)`. Для `%d` и `%s` пробел не нужен — они сами пропускают пробельные символы, а вот `%c` — нет.

Забыл привести регистр. Если не переводить `A` в `a`, то заглавная буква не совпадёт ни с одной строчной буквой слова, и игрок будет честно угадывать, но «промахиваться». Приводи ввод к нижнему регистру.

Повторная буква снимает жизнь. Если не проверять `guessed` до поиска, игрок назовёт ту же неудачную букву дважды и потеряет две жизни за одну ошибку (или будет бесконечно «угадывать» уже открытую). Проверка `if (guessed[c - 'a']) continue;` это закрывает.

Индекс за границей `guessed`. `guessed[c - 'a']` верен только для `c` от `'a'` до `'z'`. Если сначала не отсечь не-буквы, то, скажем, цифра `'5'` даст `'5' - 'a'` — отрицательный индекс и выход

за границы массива (глава 3 предупреждала, чем это грозит). Поэтому проверку диапазона делаем **до** обращения к `guessed` — и во вводе, и внутри `draw_word`. Опаснее всего то, что такой промах обычно **не падает**: программа читает чужую память, принимает её за флаг и просто начинает врать. Как мы видели в шаге 2, даже AddressSanitizer может промолчать.

Порядок «регистр — потом фильтр». Приводить `A` к `a` надо **раньше**, чем отсекать не-буквы. Поменяешь местами — и заглавные буквы отвалятся на фильтре, потому что код `'A'` (65) меньше кода `'a'` (97).

Русская буква даёт два сообщения. Не баг, а следствие UTF-8: одна кириллическая буква — два байта, `%c` читает по байту, значит, «Это не английская буква» напечатается дважды. Жизнь при этом не теряется, ломаться нечему. Просто знай, откуда лишняя строка.

Победу проверяем не там. Если сверять `revealed == len` только после ввода буквы, а не в начале витка, легко получить лишний запрос буквы уже у отгаданного слова. Рисуем — проверяем — потом спрашиваем.

Несколько букв в одной строке. Наберёшь `abc` и нажмёшь Enter — программа не ругнётся, а разберёт это как три отдельных хода: `a`, потом `b`, потом `c`. Так работает `" %c"`, он читает по одному символу и не требует Enter между ними. Иногда удобно, иногда неожиданно — теперь ты предупреждён.

Упражнения

1. Добавь подсчёт **открытых букв** в подсказке: печатай не только «Ошибок: 2 из 6», но и «Открыто: 3 из 5».

   Решение: `draw_word` уже возвращает `revealed`, а длина есть в `len`, так что хватает поменять одну строку: `printf("Ошибок: %d из %d. Открыто: %d из %d\n", wrong, MAX_WRONG, revealed, len);`. Для `apple` после `p` и `a` выйдет «Открыто: 3 из 5» — буква `p` открыла сразу две позиции.

2. Показывай игроку, какие буквы он уже называл (пройди по `guessed` и напечатай все буквы с флагом 1). Это убережёт от повторов.

   Решение: цикл `for (int k = 0; k < 26; k++)` и внутри `if (guessed[k]) printf(" %c", 'a' + k);` — обратное превращение номера в букву. Буквы выйдут в алфавитном порядке, а не в порядке ввода: после `p`, `z`, `a` строка будет «Уже называл: a p z».

3. Сделай уровни сложности: спроси перед игрой число жизней (например, от 4 до 8) и используй его вместо `MAX_WRONG`. Помни, что картинок нужно на одну больше, чем жизней: стадия 0 — пустая виселица.

   Подсказка к решению: заведи `int lives`, прочитай его с проверкой диапазона и пиши `wrong < lives` в условии цикла. Чтобы не рисовать новые картинки, растяни семь имеющихся:

`gallows[wrong * MAX_WRONG / lives]`. При 4 жизнях ошибки 0, 1, 2, 3, 4 покажут стадии 0, 1, 3, 4, 6; при 8 жизнях — 0, 0, 1, 2, 3, 3, 4, 5, 6. В обоих случаях последняя ошибка даёт полного человечка (`lives * MAX_WRONG / lives` равно `MAX_WRONG`), а индекс никогда не выходит за 6.

4. Добавь в список загадку из двух слов — например `"red apple"` — и сыграй партию. Благодаря проверке `ch < 'a' || ch > 'z'` в `draw_word` пробел рисуется как пробел и сразу считается открытым, так что игра должна пройти нормально: на старте ты увидишь `_ _ _ _ _ _ _ _ _`, а победа наступит, когда откроются все восемь букв. Потом убери эту проверку и запусти ещё раз: это выход за границу массива в живом виде. Что именно случится, заранее сказать нельзя — у меня пробел нарисовался «угаданным» (подробности в шаге 2), у тебя программа может и соврать по-другому, и упасть.

5. После партии спроси «Сыграем ещё? (y/n)» и, если `y`, начни заново (вынеси одну партию в функцию `int play_round(void)` и вызывай её в цикле в `main`).

   Подсказка к решению: внутрь `play_round` уходит всё от выбора слова до конца партии, а `guessed` и `wrong` становятся её локальными переменными — и каждый раз заводятся заново, с нулями. Это и есть сброс состояния между партиями. `srand` оставь в `main`, до цикла. Функция может возвращать 1 за победу и 0 за проигрыш — пригодится в следующем упражнении.

6. Подсчёт очков: за победу начисляй очки, зависящие от числа оставшихся жизней, и веди счёт между партиями.

   Подсказка к решению: поверх упражнения 5. Пусть `play_round` возвращает число оставшихся жизней `MAX_WRONG - wrong` при победе и 0 при проигрыше; в `main` копи сумму. Партия без единой ошибки приносит 6 очков, победа на последней жизни — 1.

Что мы получили

Ты собрал полноценную текстовую игру: случайный выбор из списка строк (`const char *words[]`), посимвольная маскировка слова через массив флагов `guessed`, аккуратный ввод одной буквы с чисткой буфера (`scanf(" %c", &c)`), приведение регистра, подсчёт жизней и ASCII-арт, который растёт с ошибками. Попутно разобрали `const char *` (указатель на неизменяемую строку), приём `sizeof массив / sizeof массив[0]` и директиву `#define`.

Главный новый навык — **состояние игры в нескольких переменных** (слово, флаги, счётчик ошибок) и цикл, который это состояние двигает. А ещё мы поймали в собственной функции мину замедленного действия: индекс `ch - 'a'`, который вылетает за границу массива на первом же слове с пробелом. Привыкай смотреть на каждый индекс и спрашивать себя: «а откуда я знаю, что он попадёт в массив?»

В следующей главе шагнём в сторону игрового **поля**: напишем «Крестики-нолики» с двумерным массивом, вложенными циклами и проверкой восьми выигрышных линий. Играть будут двое за одним компьютером, а простой бот останется тебе на упражнения.

Глава 15. Игра «Крестики-нолики»

Пора собрать первую настоящую игру с полем. Крестики-нолики знают все: поле 3×3, двое игроков по очереди ставят X и O, кто первым выстроит три в ряд — выиграл. Если поле заполнилось, а ряда нет — ничья.

Игра простая на вид, но в ней есть всё, из чего состоят игры посложнее: **двумерное поле** в памяти, **печать** этого поля на экран, **ввод хода** с проверкой, **смена игрока** и **проверка условия победы**. Это ровно тот набор, который мы переиспользуем в следующих главах: в сапёре и в змейке.

Главное новое в этой главе — **двумерный массив**. С ним и начнём.

Что мы сделаем

Игру на двоих за одним компьютером. Программа:

- рисует поле 3×3 с номерами клеток-подсказок;
- спрашивает у игрока номер клетки (1–9);
- проверяет, что клетка существует и свободна;
- ставит фигуру, меняет игрока;
- после каждого хода проверяет победу (три в ряд) и ничью;
- объявляет результат и заканчивает партию.

Никаких новых библиотек не понадобится — только `stdio.h`, который ты уже знаешь. Зато разберём двумерный массив, вложенные циклы и аккуратную проверку ввода.

Разбираем задачу руками

Возьми лист бумаги и нарисуй поле. Девять клеток. Пронумеруем их так:

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
```

```
---+---+---
 7 | 8 | 9
```

Игрок X говорит «5» — ставим X в центр. Игрок O говорит «1» — ставим O в левый верхний угол. И так далее. После каждого хода смотрим: не собрал ли кто-то три своих знака в одну линию?

Линий всего восемь:

- три горизонтальные: {1,2,3}, {4,5,6}, {7,8,9};
- три вертикальные: {1,4,7}, {2,5,8}, {3,6,9};
- две диагонали: {1,5,9} и {3,5,7}.

Если в какой-то линии стоят три одинаковых знака — этот игрок победил. Если все девять клеток заняты, а победителя нет — ничья.

Теперь главный вопрос: как хранить поле в памяти?

Двумерный массив — таблица в памяти

Обычный массив — это ряд коробок:

```
int a[5];        // a[0], a[1], a[2], a[3], a[4]
```

Но поле — не ряд, а **таблица**: 3 строки и 3 столбца. Для таблиц в Си есть **двумерный массив** — массив, у которого **два** индекса:

```
char board[3][3];
```

Читается так: «`board` — это 3 строки, в каждой по 3 ячейки типа `char`». Всего 3 × 3 = 9 ячеек. Чтобы добраться до одной, нужны **два** номера:

```
board[row][col]
```

- первый индекс `row` (англ. *row* — строка) — **номер строки**, 0, 1 или 2;
- второй индекс `col` (англ. *column* — столбец) — **номер столбца**, 0, 1 или 2.

Как и у обычного массива, **нумерация с нуля**. Левая верхняя клетка — это `board[0][0]`, правая нижняя — `board[2][2]`. Центр — `board[1][1]`.

Вот вся карта индексов:

```
 board[0][0] | board[0][1] | board[0][2]
-------------+-------------+-------------
 board[1][0] | board[1][1] | board[1][2]
-------------+-------------+-------------
 board[2][0] | board[2][1] | board[2][2]
```

Как это лежит в памяти

В памяти нет никаких «строк» и «столбцов» — там всё подряд, одной лентой. Двумерный массив раскладывается **построчно**: сначала вся строка 0, потом вся строка 1, потом строка 2.

```
 [0][0] [0][1] [0][2] [1][0] [1][1] [1][2] [2][0] [2][1] [2][2]
```

Девять ячеек `char` подряд — это 9 байт. Запись `board[row][col]` — просто удобный способ сказать компьютеру «отсчитай нужную ячейку в этой ленте». Тебе про ленту думать не нужно: пиши два индекса, и всё.

Перебор двумерного массива — цикл в цикле

Один индекс перебирают одним циклом `for`. Два индекса — **двумя вложенными** циклами: внешний идёт по строкам, внутренний по столбцам.

```
for (int row = 0; row < 3; row++) {
    for (int col = 0; col < 3; col++) {
        // здесь доступна клетка board[row][col]
    }
}
```

Как это работает по шагам: внешний цикл ставит `row = 0` и запускает внутренний цикл целиком — `col` пробегает 0, 1, 2. Это вся первая строка. Потом внешний цикл ставит `row = 1`, и внутренний снова крутится 0, 1, 2 — вторая строка. И так до `row = 2`. В сумме внутреннее тело выполнится 3 × 3 = 9 раз, ровно по одному на каждую клетку. Этот приём — основа работы с любым полем.

Шаг 1. Рисуем пустое поле

Начнём с малого: завести поле, заполнить его цифрами-подсказками 1–9 и напечатать. Цифры в пустых клетках удобны — игрок сразу видит, каким числом назвать клетку.

```
#include <stdio.h>

int main(void)
{
    char board[3][3];

    char digit = '1';
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
```

```
            board[row][col] = digit;
            digit++;
        }
    }

    for (int row = 0; row < 3; row++) {
        printf(" %c | %c | %c \n",
               board[row][0], board[row][1], board[row][2]);
    }
    return 0;
}
```

Разбор нового:

- `char board[3][3];` — наш двумерный массив на 9 символов. Тип `char` потому, что в клетке будет либо цифра-подсказка (`'1'`…`'9'`), либо фигура (`'X'` или `'O'`) — а это всё одиночные символы.
- `char digit = '1';` — заводим символ-счётчик и ставим его в `'1'`. Обрати внимание: это **символ** `'1'` в одинарных кавычках, а не число 1. Внутри `char` хранится код символа, и у соседних цифр коды идут подряд: за `'1'` следует `'2'`, за `'2'` — `'3'`. Поэтому…
- `digit++;` — увеличивает символ до следующего. После `'1'` получим `'2'`, и так до `'9'`. Тот же `++`, что и для чисел.
- `board[row][col] = digit;` — кладём текущую цифру в клетку. На первом обороте это `board[0][0] = '1'`, дальше `'2'`, `'3'` и т. д.
- В `printf(" %c | %c | %c \n", ...)` спецификатор `%c` печатает один символ (мы знаем его с главы 2). Три `%c` — три клетки одной строки, между ними рисуем разделитель `|`.

Запуск:

```
 1 | 2 | 3
 4 | 5 | 6
 7 | 8 | 9
```

Поле есть. Но между строками нет горизонтальных линий — выглядит бедно.

Шаг 2. Добавляем разделители и выносим печать в функцию

Печатать поле придётся после **каждого** хода. Копировать цикл печати по всему коду — плохо: длинно и легко ошибиться. Вынесем печать в отдельную функцию `print_board`, а заодно дорисуем горизонтальные линии.

```c
void print_board(char board[3][3])
{
    printf("\n");
    for (int row = 0; row < 3; row++) {
        printf(" %c | %c | %c \n",
               board[row][0], board[row][1], board[row][2]);
        if (row < 2) {
            printf("---+---+---\n");
        }
    }
    printf("\n");
}
```

Разбор:

- `void print_board(char board[3][3])` — функция, которая ничего не возвращает (`void`, см. главу 2), а принимает наше поле. Как передать двумерный массив в функцию? Указываем тип ровно так, как объявляли: `char board[3][3]`. Здесь важен второй размер — длина строки. По нему функция считает, где в ленте памяти начинается строка 1 (через 3 ячейки от начала) и строка 2 (через 6). Первый размер компилятор в параметре молча отбрасывает: `char board[][3]` значило бы то же самое. Мы пишем `[3][3]` просто для наглядности.
  - Тонкость: массив (в том числе двумерный) передаётся в функцию **по адресу**, а не копией (см. главу 3 про указатели). В функцию уходит адрес начала поля, а не девять символов. Значит, функция работает с **тем же самым** полем, что и `main`: если она что-то в нём поменяет, `main` это увидит. Для печати это неважно, а вот позже удобно: не надо копировать поле туда-сюда.
- `if (row < 2)` — линию-разделитель рисуем после строк 0 и 1, но **не** после последней строки 2. Иначе под полем повисла бы лишняя черта.
- `printf("\n")` в начале и в конце — пустые строки сверху и снизу, чтобы поле не липло к остальному тексту.

Теперь `main` короткий: завести поле, заполнить, позвать `print_board`. Запуск даёт аккуратную картинку:

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
```

Шаг 3. Один ход игрока

Теперь научимся ставить фигуру. Игрок вводит число 1–9, а нам нужны два индекса `[row][col]`. Как перевести одно в другое?

Клетки пронумерованы построчно: 1, 2, 3 — первая строка; 4, 5, 6 — вторая; 7, 8, 9 — третья. Вычтем из номера 1, чтобы считать с нуля (`cell - 1` даёт 0…8), и дальше:

- **номер строки** = `(cell - 1) / 3` — целочисленное деление на 3 (помнишь из главы 2: `/` для целых отбрасывает дробь). Для клеток 1–3 даст 0, для 4–6 даст 1, для 7–9 даст 2.
- **номер столбца** = `(cell - 1) % 3` — остаток от деления на 3. Даёт 0, 1, 2 внутри строки.

Проверь руками. Клетка 5: `(5-1)/3 = 4/3 = 1` (строка 1), `(5-1)%3 = 4%3 = 1` (столбец 1) — центр `board[1][1]`. Верно. Клетка 7: `(7-1)/3 = 2`, `(7-1)%3 = 0` — `board[2][0]`, левый нижний угол. Тоже верно.

Наивный ход (пока без проверок):

```
int cell;
scanf("%d", &cell);

int row = (cell - 1) / 3;
int col = (cell - 1) % 3;

board[row][col] = player;     // player — это 'X' или 'O'
```

Работает, но доверять вводу нельзя. Что если игрок наберёт 42? Тогда `row = 41/3 = 13` — и мы запишем фигуру **за границы** массива, в чужую память. Си границы не проверяет (см. главу 3), поэтому проверяем сами.

Шаг 4. Проверяем ввод

Хороший ход обязан пройти три проверки:

1. Введено вообще число? (`scanf` мог не справиться — игрок набрал буквы).
2. Число в диапазоне 1–9?
3. Клетка свободна (там ещё цифра, а не `'X'`/`'O'`)?

```
int cell;
int rc = scanf("%d", &cell);

if (rc == EOF) {                /* ввод закончился (Ctrl+D) */
    printf("\nИгра прервана.\n");
```

```
    return 0;
}
if (rc != 1) {                    /* ввели не число */
    printf("Это не число. Введи цифру от 1 до 9.\n");
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        /* выбрасываем мусорный ввод до конца строки */
    }
    continue;
}
if (cell < 1 || cell > 9) {
    printf("Нет такой клетки. Нужно число от 1 до 9.\n");
    continue;
}

int row = (cell - 1) / 3;
int col = (cell - 1) % 3;

if (board[row][col] == 'X' || board[row][col] == 'O') {
    printf("Клетка %d уже занята. Выбери другую.\n", cell);
    continue;
}
```

Разбор (часть уже знакома по игре «угадай число» из главы 12):

- `int rc = scanf("%d", &cell);` — `scanf` **возвращает**, сколько значений ему удалось прочитать. Мы просили одно число, значит, успех — это `rc == 1`. Ровно так мы проверяли ввод в «Угадай число» (глава 12, шаг 8): без этой проверки буквы вместо цифры зацикливают игру.
- `rc == EOF` — *end of file*, «конец ввода». Бывает, если ввод закончился (например, нажали Ctrl+D или партию подали из файла). Просто выходим.
- `rc != 1` — `scanf` не смог прочитать число (игрок набрал `abc`). Беда в том, что эти буквы **остались** во входном буфере, и следующий `scanf` снова на них споткнётся — получится бесконечный поток ошибок. Поэтому чистим буфер циклом `getchar()` до конца строки. `getchar` читает по одному символу; крутим, пока не съедим перевод строки `'\n'` или не упрёмся в конец ввода.
- `continue;` — пропустить остаток тела цикла и сразу пойти на новую попытку ввода. Игрок ошибся — даём ему повторить ход, не меняя поле и не передавая ход сопернику.

- `board[row][col] == 'X' || board[row][col] == 'O'` — клетка занята, если в ней стоит любая из фигур. Сравниваем символы через == (для одиночных `char` это можно — это не строки, см. главу 3, где строки нельзя сравнивать через ==).

Только пройдя все проверки, ставим фигуру: `board[row][col] = player;`.

Шаг 5. Проверка победы — выносим в функцию

После хода нужно понять, не выиграл ли текущий игрок. Проверять восемь линий прямо в `main` — это каша. Вынесем в функцию `has_won`, которая отвечает на вопрос «занял ли игрок `player` три клетки в ряд?».

```
int has_won(char board[3][3], char player)
{
    for (int row = 0; row < 3; row++) {
        if (board[row][0] == player &&
            board[row][1] == player &&
            board[row][2] == player) {
            return 1;
        }
    }
    for (int col = 0; col < 3; col++) {
        if (board[0][col] == player &&
            board[1][col] == player &&
            board[2][col] == player) {
            return 1;
        }
    }
    if (board[0][0] == player &&
        board[1][1] == player &&
        board[2][2] == player) {
        return 1;
    }
    if (board[0][2] == player &&
        board[1][1] == player &&
        board[2][0] == player) {
        return 1;
    }
```

```
    return 0;
}
```

Разбор:

- `int has_won(...)` — функция возвращает `int`: 1, если игрок выиграл, 0, если нет. Мы помним из главы 2: в Си истина — это любое ненулевое число (обычно 1), ложь — 0. Так что результат можно прямо подставлять в `if`.
- **Три строки.** Цикл по `row`: внутри строки фиксируем номер строки и проверяем все три столбца — `[row][0]`, `[row][1]`, `[row][2]`. Если все три равны `player`, строка собрана — `return 1`.
- **Три столбца.** Зеркально: цикл по `col`, фиксируем столбец и проверяем три строки — `[0][col]`, `[1][col]`, `[2][col]`.
- `&&` — логическое «И» (см. главу 2): победа в линии, только если **все три** клетки совпали с `player`.
- **Главная диагональ** — клетки `[0][0]`, `[1][1]`, `[2][2]` (из левого верхнего угла в правый нижний). **Побочная** — `[0][2]`, `[1][1]`, `[2][0]` (из правого верхнего в левый нижний). У обеих центр общий — `[1][1]`.
- `return 1;` внутри прерывает функцию сразу, как только нашли линию. Если ни одна не сложилась, доходим до `return 0;` в конце.

`return` где угодно завершает функцию и отдаёт значение. Поэтому первое же совпадение даёт ответ, остальное не проверяется.

Шаг 6. Проверка ничьей

Ничья — это когда ходов больше нет, а победителя нет. «Ходов нет» значит, что все девять клеток заняты фигурами. Проверим это функцией:

```
int board_is_full(char board[3][3])
{
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            if (board[row][col] != 'X' && board[row][col] != '0') {
                return 0;
            }
        }
    }
    return 1;
}
```

Логика «от противного»: ищем хотя бы одну **не**занятую клетку. Если в клетке не `'X'` и не `'O'` (там ещё цифра-подсказка) — поле не заполнено, сразу `return 0`. Если оба вложенных цикла отработали и ни одной свободной клетки не нашлось — `return 1`, поле полное.

Порядок проверок в игре важен: **сначала** победа, **потом** ничья. Последним, девятым, ходом игрок может одновременно и заполнить поле, и выстроить линию — это победа, а не ничья. Поэтому в `main` сначала зовём `has_won`, и только если победы нет — `board_is_full`.

Полная программа

Собираем всё вместе. Это `code/15-tic-tac-toe.c`:

```c
#include <stdio.h>

/* Печать поля 3x3 с разделителями. */
void print_board(char board[3][3])
{
    printf("\n");
    for (int row = 0; row < 3; row++) {
        printf(" %c | %c | %c \n",
               board[row][0], board[row][1], board[row][2]);
        if (row < 2) {
            printf("---+---+---\n");
        }
    }
    printf("\n");
}

/* Проверка: занял ли игрок player три клетки в ряд?
   Возвращает 1 (победа) или 0. */
int has_won(char board[3][3], char player)
{
    /* три строки */
    for (int row = 0; row < 3; row++) {
        if (board[row][0] == player &&
            board[row][1] == player &&
            board[row][2] == player) {
            return 1;
        }
```

```
    }
    /* три столбца */
    for (int col = 0; col < 3; col++) {
        if (board[0][col] == player &&
            board[1][col] == player &&
            board[2][col] == player) {
            return 1;
        }
    }
    /* главная диагональ */
    if (board[0][0] == player &&
        board[1][1] == player &&
        board[2][2] == player) {
        return 1;
    }
    /* побочная диагональ */
    if (board[0][2] == player &&
        board[1][1] == player &&
        board[2][0] == player) {
        return 1;
    }
    return 0;
}

/* Все девять клеток заняты фигурами? */
int board_is_full(char board[3][3])
{
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            if (board[row][col] != 'X' && board[row][col] != '0') {
                return 0;
            }
        }
    }
    return 1;
}

int main(void)
```

```
{
    char board[3][3];

    /* Заполняем поле цифрами '1'..'9' — это подсказка номеров клеток. */
    char digit = '1';
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            board[row][col] = digit;
            digit++;
        }
    }

    char player = 'X';   /* первым ходит X */
    int moves = 0;       /* сколько ходов сделано */

    printf("Крестики-нолики. Игроки X и O ходят по очереди.\n");
    printf("Выбирай клетку числом от 1 до 9.\n");

    while (1) {
        print_board(board);
        printf("Ход игрока %c. Клетка (1-9): ", player);

        int cell;
        int rc = scanf("%d", &cell);

        if (rc == EOF) {                /* ввод закончился (Ctrl+D) */
            printf("\nИгра прервана.\n");
            return 0;
        }
        if (rc != 1) {                  /* ввели не число */
            printf("Это не число. Введи цифру от 1 до 9.\n");
            int c;
            while ((c = getchar()) != '\n' && c != EOF) {
                /* выбрасываем мусорный ввод до конца строки */
            }
            continue;
        }
        if (cell < 1 || cell > 9) {
```

```
            printf("Нет такой клетки. Нужно число от 1 до 9.\n");
            continue;
        }

        /* Переводим номер 1..9 в индексы строки и столбца. */
        int row = (cell - 1) / 3;
        int col = (cell - 1) % 3;

        if (board[row][col] == 'X' || board[row][col] == 'O') {
            printf("Клетка %d уже занята. Выбери другую.\n", cell);
            continue;
        }

        board[row][col] = player;        /* ставим фигуру */
        moves++;

        if (has_won(board, player)) {
            print_board(board);
            printf("Победил игрок %c! Ходов: %d.\n", player, moves);
            return 0;
        }
        if (board_is_full(board)) {
            print_board(board);
            printf("Ничья! Поле заполнено.\n");
            return 0;
        }

        /* Меняем игрока: X -> O, O -> X. */
        if (player == 'X') {
            player = 'O';
        } else {
            player = 'X';
        }
    }
}
```

Как это работает

- Три функции стоят **выше** `main` — чтобы к моменту их вызова компилятор их уже знал (про это правило мы говорили в главе 2).
- `while (1)` — **бесконечный** цикл. `1` — это всегда истина, значит, условие никогда не станет ложным само. Это удобно для игрового цикла: один оборот = один ход. Выходим из него не по условию, а изнутри — через `return 0;` (победа, ничья или прерывание ввода). Так часто оформляют игры, где заранее неизвестно, сколько будет ходов.
- Один оборот цикла: показали поле → попросили ход → проверили ввод → поставили фигуру → проверили победу → проверили ничью → сменили игрока → и заново.
- Смена игрока в конце оборота: `'X'` становится `'O'` и наоборот. А если ход был отбракован (`continue`), мы до смены игрока не доходим — ход остаётся за тем же игроком. Это правильно: за чужую ошибку очередь не переходит.
- `moves` считает только **состоявшиеся** ходы (мы увеличиваем его после всех проверок). В конце печатаем, за сколько ходов выиграли.

Проверяем

Скомпилируй и запусти. Компиляция должна пройти без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 15-tic-tac-toe.c -o ttt
./ttt
```

Сыграть можно вручную, набирая клетки и нажимая Enter. Но проверить, что победа и ничья распознаются верно, удобнее «прокруткой» готовой партии через канал ввода. Подадим серию ходов сразу, командой `printf`:

```
printf "1\n4\n2\n5\n3\n" | ./ttt
```

Здесь `\n` — перевод строки (как нажатие Enter после каждого числа), а `|` направляет этот текст программе на вход вместо клавиатуры. Партия: X берёт 1, O берёт 4, X берёт 2, O берёт 5, X берёт 3 — и X собирает всю верхнюю строку. Вывод (конец партии):

```
Ход игрока X. Клетка (1-9):
 X | X | X
---+---+---
 O | O | 6
---+---+---
 7 | 8 | 9

Победил игрок X! Ходов: 5.
```

Победа по строке работает. Проверим **ничью** — заполним всё поле так, чтобы линии ни у кого не сложилось:

```
printf "1\n2\n3\n5\n4\n6\n8\n7\n9\n" | ./ttt
```

Конец партии:

```
Ход игрока X. Клетка (1–9):
 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | X

Ничья! Поле заполнено.
```

А вот победа по **диагонали** (`printf "5\n1\n3\n2\n7\n"` — X занимает 5, 3 и 7, это побочная диагональ):

```
Ход игрока X. Клетка (1–9):
 O | O | X
---+---+---
 4 | X | 6
---+---+---
 X | 8 | 9

Победил игрок X! Ходов: 5.
```

Читается так: X стоит в клетках 3, 5 и 7 — это побочная диагональ, идущая из правого верхнего угла в левый нижний. O успел занять 1 и 2, но линию не собрал. Клетки 4, 6, 8, 9 остались с цифрами-подсказками — до них партия не дошла.

Строка `Ход игрока X. Клетка (1–9):` в конце каждого вывода — это приглашение, которое программа успела напечатать перед последним ходом. Само число не видно: при подаче через `|` ввод берётся из канала, а не с клавиатуры, и терминалу нечего показывать эхом. Если сыграть партию руками, набранные цифры будут стоять сразу после двоеточия.

Проверяем все восемь линий, а не три

Три партии — это приятно, но это не проверка. Победных линий восемь, и любая из них может оказаться забытой в `has_won`. Прогони их все — подряд, одной командой:

```
for m in "1 4 2 5 3" "4 1 5 2 6" "7 1 8 2 9" \
         "1 2 4 3 7" "2 1 5 3 8" "3 1 6 2 9" \
         "1 2 5 3 9" "3 1 5 2 7"; do
    printf '%s\n' $m | ./ttt | tail -1
done
```

Каждая строка тут — это партия, где X забирает одну из восьми линий, а O занимает две посторонние клетки. Вывод:

```
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
Победил игрок X! Ходов: 5.
```

Восемь строк — восемь пойманных линий. Если бы в `has_won` потерялась, скажем, побочная диагональ, восьмая партия закончилась бы не победой, а следующим приглашением к ходу, и это сразу бросилось бы в глаза. Привыкай к такому способу проверки: он занимает минуту, а ловит ровно те ошибки, которые не видны при беглой игре руками.

Победа последним ходом

Мы договаривались, что победу проверяем раньше ничьей. Проверим и это. Партия, где девятый ход и заполняет поле, и достраивает верхнюю строку:

```
printf "1\n5\n2\n6\n4\n7\n8\n9\n3\n" | ./ttt
```

```
Ход игрока X. Клетка (1–9):
 X | X | X
---+---+---
 X | O | O
---+---+---
 O | X | O


Победил игрок X! Ходов: 9.
```

Поле забито до последней клетки, но напечатана победа, а не ничья — порядок проверок в `main` работает как задумано.

Что будет от кривого ввода

Осталось убедиться, что игру нельзя сломать пальцами. Сыграем руками и специально понажимаем не то: повторим занятую клетку, введём ноль, наберём буквы.

```
Ход игрока X. Клетка (1–9): 5

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Ход игрока O. Клетка (1–9): 5
Клетка 5 уже занята. Выбери другую.

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Ход игрока O. Клетка (1–9): 0
Нет такой клетки. Нужно число от 1 до 9.

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Ход игрока O. Клетка (1–9): abc
Это не число. Введи цифру от 1 до 9.

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9
```

```
Ход игрока O. Клетка (1–9): 1

 O | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9

Ход игрока X. Клетка (1–9): ^D
Игра прервана.
```

(`^D` в конце — это Ctrl+D, которым я закончил партию.)

Три разных ошибки — три разных внятных сообщения, и все три раза ход остался за O. Никаких «съеденных» ходов, никаких зависаний на буквах. Обрати внимание на последнюю деталь: после `abc` игра не свалилась в бесконечную ругань — это работает чистка буфера через `getchar`.

Ещё один способ закончить ввод — нажать Ctrl+D (или просто скормить программе пустой файл). Тогда `scanf` вернёт `EOF`:

```
./ttt < /dev/null
```

```
Крестики-нолики. Игроки X и O ходят по очереди.
Выбирай клетку числом от 1 до 9.

 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9

Ход игрока X. Клетка (1–9):
Игра прервана.
```

Все исходы — восемь победных линий, победа последним ходом, ничья и любой мусор на входе — отрабатываются верно.

Сколько всего бывает партий

Восемь партий мы проверили руками. А сколько их вообще бывает? Прикинем сверху. Первый ход — 9 вариантов, второй — 8 (одна клетка уже занята), третий — 7, и так до последнего. Если

бы каждая партия шла все девять ходов, партий было бы $9 \cdot 8 \cdot 7 \cdot \ldots \cdot 1 = 9! = 362\,880$ — это факториал из главы 4. Но многие партии обрываются раньше: кто-то собрал линию на пятом, шестом, седьмом ходу, и дальше игры нет. Значит, настоящее число меньше. Насколько — посчитаем программой.

Идея — перебрать **все** партии рекурсией (глава 9). Функция `play_all` получает поле и того, кто ходит. Она по очереди пробует каждую свободную клетку: ставит туда фигуру, смотрит, чем кончилось, и если партия не кончилась — зовёт саму себя за соперника. Потом **отменяет** ход, возвращает клетке прежнюю цифру и пробует следующую. Функции `has_won` и `board_is_full` берём готовые, из нашей игры.

```
long x_wins, o_wins, draws;   /* счётчики исходов */

/* Перебрать все партии, которые можно доиграть из этой позиции,
   если сейчас ходит player. */
void play_all(char board[3][3], char player)
{
    char next = (player == 'X') ? 'O' : 'X';

    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            if (board[row][col] == 'X' || board[row][col] == 'O') {
                continue;                   /* клетка занята */
            }
            char saved = board[row][col];
            board[row][col] = player;       /* сделали ход */

            if (has_won(board, player)) {
                if (player == 'X') {
                    x_wins++;
                } else {
                    o_wins++;
                }
            } else if (board_is_full(board)) {
                draws++;
            } else {
                play_all(board, next);      /* играем дальше */
            }
```

```
            board[row][col] = saved;        /* отменили ход */
        }
    }
}
```

Разбор:

- `long x_wins, o_wins, draws;` — три глобальных счётчика (глобальные переменные мы видели в главе 9). Глобальные числа в начале программы равны нулю сами. Тип `long` на этом маке — 8 байт (глава 2), печатается через `%ld`.
- `char next = (player == 'X') ? 'O' : 'X';` — тернарный оператор из главы 13: «если ходит X, следующим будет O, иначе X».
- `char saved = board[row][col];` и `board[row][col] = saved;` — запомнили цифру-подсказку, поставили фигуру, а после всех проверок вернули цифру на место. Этот приём «сделал ход — проверил — отменил» называют **перебором с возвратом** (backtracking). Поле передаётся по адресу, поэтому без отмены каждая следующая попытка видела бы фигуры от прошлых.
- База рекурсии — победа или заполненное поле: там мы только увеличиваем счётчик и дальше себя не зовём. Партия длится не больше девяти ходов, поэтому глубина рекурсии тоже не больше девяти.

В `main` заполняем поле цифрами, как в игре, зовём `play_all(board, 'X')` и печатаем:

```
printf("X выиграл: %ld\n", x_wins);
printf("O выиграл: %ld\n", o_wins);
printf("Ничьих:    %ld\n", draws);
printf("Всего партий: %ld\n", x_wins + o_wins + draws);
```

Запуск:

```
X выиграл: 131184
O выиграл: 77904
Ничьих:    46080
Всего партий: 255168
```

Всего 255 168 разных партий — примерно на 30 % меньше, чем 9! = 362 880 (255 168 / 362 880 ≈ 0,70). Партией тут считается последовательность ходов: одинаковые позиции, до которых дошли в разном порядке, считаются разными партиями. Число 255 168 давно известно, и совпадение с ним — ещё одна проверка `has_won`: забудь в ней хоть одну линию, и счётчики разъедутся.

Заодно видно, что ходить первым выгодно: X выигрывает в 131 184 партиях из 255 168 (51,4 %), O — только в 77 904 (30,5 %). Но это доля **партий**, а не вероятность выигрыша. Если оба игрока ставят фигуры наугад, короткая партия выпадает чаще длинной: чтобы случилась конкретная партия из пяти ходов, надо угадать 5 выборов, а из девяти — все 9. Настоящие вероятности посчитаешь в упражнении 7.

Частые ошибки

Путаница «номер клетки» и «индекс». Игрок думает клетками 1–9, а массив — индексами 0–8 по каждому измерению. Если забыть `- 1` в `(cell - 1) / 3`, всё поле «съедет», а клетка 9 вообще улетит за границы. Перевод номера в индексы делай в одном месте и проверяй руками на паре примеров.

Проверка занятости через цифру. Можно подумать: «клетка свободна, если в ней цифра». Но тогда условие пришлось бы писать длинно (`>= '1' && <= '9'`). Надёжнее наоборот: клетка **занята**, если в ней `'X'` или `'O'`. Свободна — во всех остальных случаях. Так не зависишь от того, чем именно помечены пустые клетки.

Грязный буфер ввода. Если игрок вместо числа набрал буквы, `scanf` их не прочитает и **оставит** во входном потоке. Без чистки буфера (`getchar` до конца строки) следующий `scanf` снова на них наткнётся — и игра уйдёт в бесконечную печать «Это не число». Всегда подчищай за неудачным `scanf`.

Сначала ничья, потом победа. Если проверять `board_is_full` раньше `has_won`, то выигрышный девятый ход засчитается как ничья. Победа проверяется первой.

Слишком большое число. Проверка `rc == 1` не ловит число, которое не влезает в `int` (см. главу 2). По стандарту это неопределённое поведение `scanf`, а на этом маке в `cell` попадают младшие 32 бита числа. Набери `4294967301` (это $2^{32} + 5$) — и крестик встанет в клетку 5, будто так и надо. Для учебной игры с этим можно жить; надёжный разбор чисел делают через `strtol`.

Выход за границы при невалидной клетке. Вычислять `row`/`col` и лезть в `board[row][col]` нужно **только после** проверки `cell` на диапазон 1–9. Иначе число 42 уведёт индекс за пределы массива, а Си это не остановит (см. главу 3).

Упражнения

1. **Счёт серий.** Сейчас после партии программа завершается. Оберни игру во внешний цикл: после результата спроси «Сыграем ещё? (y/n)» и, если да, начни новую партию. Веди счёт побед X, побед O и ничьих, печатай его перед каждой партией. *Что должно получиться:* партия `1 4 2 5 3`, ответ `y`, ничья `1 2 3 5 4 6 8 7 9`, ответ `n` — и перед второй

партией программа печатает счёт «X: 1, O: 0, ничьих: 0», а в конце — «X: 1, O: 0, ничьих: 1». Не забудь перед новой партией снова заполнить поле цифрами и обнулить `moves`. Ответ `y/n` удобно читать через `scanf(" %c", &answer)`: пробел перед `%c` съедает перевод строки, оставшийся после номера клетки.

2. **Поле N×N.** Замени `board[3][3]` на `board[N][N]`, где `N` — константа (`#define N 3` вверху файла). Перепиши печать и заполнение через циклы до `N`. Победу пока можешь оставить на 3 в ряд только для `N == 3` — но печать и ввод должны работать при любом `N`. *Подводный камень:* цифры-подсказки кончаются на `'9'`. Следующий `digit++` даст `':'` — это следующий символ в таблице ASCII. Поэтому храни в пустой клетке точку `'.'`, а номер клетки печатай числом: `printf(" %2d ", row * N + col + 1)`. Проверку «клетка свободна» при этом оставь прежней: не `'X'` и не `'O'`. При `N = 4` и крестике в клетке 11 поле выглядит так:

```
  1 |  2 |  3 |  4
----+----+----+----
  5 |  6 |  7 |  8
----+----+----+----
  9 | 10 |  X | 12
----+----+----+----
 13 | 14 | 15 | 16
```

3. **Ввод строкой и столбцом.** Вместо номера 1–9 разреши вводить два числа — номер строки и номер столбца (1–3 каждый). Переведи их в индексы (`row - 1`, `col - 1`) и проверь диапазон обоих. *Что должно получиться:* `scanf("%d %d", &row, &col)` должен вернуть 2; ввод `2 2` ставит фигуру в центр, `1 3` — в правый верхний угол, `3 0` и `4 1` отбиваются сообщением «нет такой клетки». Диапазон проверяй **до** вычитания единицы.

4. **Простой бот.** Сделай так, чтобы за O играл компьютер. Самый наивный бот: пройди поле и поставь O в **первую попавшуюся** свободную клетку. Подключи `rand` (см. главу 12), чтобы бот выбирал свободную клетку случайно — так интереснее. *Что должно получиться:* бот никогда не ходит в занятую клетку и не ругается «клетка занята» — он просто бросает `rand() % 9 + 1` заново, пока не попадёт в свободную. Решение — функция `bot_move` ниже, если заменить первую строку её тела на `int cell;` и убрать следующий за ней `if`.

5. **Подсветка победной линии.** После победы не просто объяви победителя, а определи, какая именно линия собрана, и напечатай её (например, «верхняя строка» или «главная диагональ»). Подсказка: пусть `has_won` возвращает не просто 1, а номер линии. *Что должно получиться:* прогони восемь партий из раздела «Проверяем все восемь линий» — последние строки будут «верхняя строка», «средняя строка», «нижняя строка», «левый столбец», «средний столбец», «правый столбец», «главная диагональ», «побочная диагональ». Решение ниже.

6. **Защита от дурака в боте.** Усложни бота из задачи 4: перед своим ходом пусть он проверит, не может ли соперник выиграть следующим ходом, и закроет эту клетку. *Что должно получиться:* сыграй `1`, потом `2` (если бот не занял 2 или 3 первым ходом) — бот обязан ответить клеткой 3. Решение ниже.

7. **Настоящие вероятности.** Переделай `play_all` так, чтобы она считала не число партий, а **вероятность** каждого исхода, если оба игрока ходят наугад, выбирая любую свободную клетку с равным шансом. Подсказка: передавай в `play_all` третьим параметром `double prob` — вероятность дойти до этой позиции. Если свободных клеток `k`, каждый ход в неё получает `prob / k`. В начале `prob = 1.0`, а вместо `++` к счётчикам прибавляй вероятность. *Ответ:* X выигрывает в 58,5 % случаев, O — в 28,8 %, ничья — 12,7 %. Сумма — 100 %, это хорошая самопроверка.

Решения упражнений 5 и 6

Упражнение 5. `has_won` возвращает номер линии вместо единицы, а названия лежат в массиве строк:

```
/* Какую линию собрал player? 0 — никакую, 1..8 — номер линии:
   1..3 — строки, 4..6 — столбцы, 7 — главная диагональ,
   8 — побочная. */
int has_won(char board[3][3], char player)
{
    for (int row = 0; row < 3; row++) {
        if (board[row][0] == player &&
            board[row][1] == player &&
            board[row][2] == player) {
            return 1 + row;
        }
    }
    for (int col = 0; col < 3; col++) {
        if (board[0][col] == player &&
            board[1][col] == player &&
            board[2][col] == player) {
            return 4 + col;
        }
    }
    if (board[0][0] == player &&
        board[1][1] == player &&
        board[2][2] == player) {
```

```
        return 7;
    }
    if (board[0][2] == player &&
        board[1][1] == player &&
        board[2][0] == player) {
        return 8;
    }
    return 0;
}

const char *line_names[] = {
    "", "верхняя строка", "средняя строка", "нижняя строка",
    "левый столбец", "средний столбец", "правый столбец",
    "главная диагональ", "побочная диагональ"
};
```

В `main` результат запоминаем, чтобы не звать `has_won` дважды:

```
int line = has_won(board, player);
if (line != 0) {
    print_board(board);
    printf("Победил игрок %c! Ходов: %d.\n", player, moves);
    printf("Собрана линия: %s.\n", line_names[line]);
    return 0;
}
```

`line_names[0]` — пустая строка-заглушка: так номер линии совпадает с индексом в массиве. Ноль по-прежнему значит «победы нет», поэтому `if (has_won(...))` в остальном коде продолжает работать.

Упражнение 6 (и заодно 4 — у него нет только первой проверки). Бот пробует каждую свободную клетку за соперника тем же приёмом «сделал ход — проверил — отменил», что и `play_all`:

```
/* Клетка 1..9, ходом в которую who сразу выигрывает, или 0. */
int winning_cell(char board[3][3], char who)
{
    for (int cell = 1; cell <= 9; cell++) {
        int row = (cell - 1) / 3;
        int col = (cell - 1) % 3;
        char old = board[row][col];
        if (old == 'X' || old == 'O') {
```

```
            continue;                   /* клетка занята */
        }
        board[row][col] = who;          /* пробуем ход... */
        int win = has_won(board, who);
        board[row][col] = old;          /* ...и сразу отменяем */
        if (win) {
            return cell;
        }
    }
    return 0;
}

/* Ход компьютера за O: закрыть угрозу X, иначе случайная клетка. */
int bot_move(char board[3][3])
{
    int cell = winning_cell(board, 'X');
    if (cell != 0) {
        return cell;                    /* X грозит — закрываем */
    }
    int row, col;
    do {
        cell = rand() % 9 + 1;
        row = (cell - 1) / 3;
        col = (cell - 1) % 3;
    } while (board[row][col] == 'X' || board[row][col] == 'O');
    return cell;
}
```

В игровом цикле, когда `player == 'O'`, вместо `scanf` берём `cell = bot_move(board)`, печатаем «Компьютер ставит O в клетку N» и дальше ставим фигуру и проверяем победу как обычно. Не забудь `#include <stdlib.h>`, `#include <time.h>` и `srand` в начале `main`. Цикл `do { ... } while` (глава 2) сначала выполняет тело, потом проверяет условие — нам и нужно сначала бросить кубик, а потом смотреть, куда попали. Он точно закончится: бота зовут, только пока поле не заполнено, так что хоть одна свободная клетка есть.

Что мы получили

Ты собрал полноценную игру с полем — и по дороге освоил **двумерный массив**: два индекса `[строка][столбец]`, построчную раскладку в памяти, обход вложенными циклами. Научился переводить номер клетки в пару индексов через `/` и `%`, аккуратно проверять ввод, выносить логику в функции (`print_board`, `has_won`, `board_is_full`) и проверять победу по строкам, столбцам и диагоналям. А рекурсивный перебор с возвратом пересчитал все 255 168 возможных партий и заодно проверил `has_won` на каждой из них.

Двумерное поле плюс вложенные циклы — это каркас почти любой настольной игры. В следующей главе мы возьмём поле побольше и поинтереснее: напишем **сапёр**. Там тоже сетка клеток, но добавятся спрятанные мины, подсчёт мин-соседей вокруг каждой клетки (привет, обход восьми соседей) и рекурсивное «открытие» пустых областей.

Глава 16. Игра «Сапёр»

Помнишь «Сапёра» из старой Windows? Поле из клеток, под некоторыми спрятаны мины, и ты по одной открываешь клетки. В открытой клетке — число: сколько мин стоит вокруг неё. Открыл пустую — рядом раскрывается целый кусок поля. Открыл мину — проиграл.

Это идеальная игра, чтобы собрать вместе всё, что мы уже умеем: **двумерные массивы** (поле — это сетка, см. главу 15), **случайные числа** (мины ставим наугад, `rand`/`srand` — глава 12) и **рекурсию** (автоматическое раскрытие пустой области — глава 9). Ничего нового из языка не понадобится — только новая комбинация знакомых деталей.

Что мы сделаем

Правила нашего «Сапёра»:

- поле фиксированного размера **9×9** клеток;
- на нём случайно расставлены **10 мин**;
- игрок вводит координаты клетки (строку и столбец), чтобы её открыть;
- в открытой клетке показывается **число соседних мин** (от 1 до 8);
- если соседних мин **ноль** — клетка пустая, и вокруг неё автоматически открываются соседи (и так дальше, волной);
- открыл клетку с миной — **проигрыш**, показываем все мины;
- открыл все клетки без мин — **победа**.

Поле в терминале будет выглядеть так (точка — закрытая клетка):

```
  1 2 3 4 5 6 7 8 9
1 . . . . . . . . .
2 . . . . . . . . .
3 . . . . . . . . .
4 . . . . . . . . .
5 . . . . . . . . .
6 . . . . . . . . .
```

```
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
```

К концу главы у тебя будет полноценная играбельная консольная версия.

Разбираем задачу руками

Прежде чем писать код, разложим игру на части на бумаге.

Две карты, наложенные друг на друга

Самое важное решение — как хранить поле. У каждой клетки есть **две независимые** характеристики:

1. **Есть ли тут мина?** Это знает только программа, игрок не видит.
2. **Открыта ли клетка игроком?** Это видно на экране.

Эти два факта не связаны: клетка может быть с миной и закрытой, без мины и открытой, и так далее. Поэтому удобнее всего завести **два двумерных массива** одинакового размера, как две прозрачные плёнки, наложенные друг на друга:

- `mines[H][W]` — где `1`, там мина; где `0`, там пусто;
- `opened[H][W]` — где `1`, там игрок уже открыл клетку; где `0`, клетка ещё закрыта.

Можно было бы обойтись и одним массивом символов с разными состояниями (`'.'`, `'*'`, цифра), но тогда пришлось бы кодировать в одной клетке сразу всё, и логика запуталась бы. Две раздельные карты читаются куда проще: «стоит ли тут мина» и «открыта ли клетка» — два отдельных вопроса, два отдельных массива.

Числа в клетках мы **нигде не храним** — посчитаем их на лету, когда понадобится. Так меньше шансов рассинхронизировать данные.

Сколько мин вокруг клетки

У клетки в середине поля **восемь** соседей: сверху, снизу, слева, справа и четыре по диагонали. Чтобы узнать число в клетке, надо обойти всех восьмерых и посчитать, у скольких стоит мина.

Возьмём кусочек поля. Буква `M` — мина, мы смотрим на центральную клетку `?`:

```
. M .
. ? M
M . .
```

Обходим восемь соседей центральной клетки: сверху-середина — мина, справа — мина, снизу-слева — мина. Остальные пять пустые. Значит, в центральной клетке будет число **3**.

У клетки с краю или в углу соседей меньше — часть «вылезает» за поле. За эти несуществующие клетки нельзя заглядывать: там нет нашего массива, там чужая память. Проверку границ разберём отдельно, это важный момент.

Заливка пустой области

Когда игрок открывает клетку, рядом с которой **ноль** мин, в настоящем «Сапёре» одним кликом раскрывается целая поляна. Почему? Потому что у пустой клетки все соседи безопасны (раз вокруг ноль мин, ни в одном соседе мины нет). Их можно сразу открыть. А если какой-то сосед тоже пустой — открыть и его соседей. И так волной, пока не упрёмся в клетки с числами (они стоят на границе минного поля).

Это в точности **рекурсия** из главы 9: «открыть клетку» сводится к «открыть эту + открыть соседей такой же процедурой». Разберём заливку на маленьком поле 3×3 с одной миной в углу. Цифры — это реальное число соседних мин, точки — пока закрыто:

```
. . .
. . .
. . M
```

Игрок открывает верхний левый угол (клетка 1,1). У неё ноль соседних мин — значит, открываем её и зовём заливку для всех её соседей. Те, что пустые, тоже зовут своих соседей. Доходит до клеток рядом с миной — у них сосед-мина есть, число не ноль, дальше заливка не идёт. В итоге раскроется почти всё поле, а у клеток возле мины проступят числа:

```
      ← пусто (0), пусто, пусто
1 1   ← пусто, рядом с миной, рядом с миной
1 .   ← пусто, рядом с миной, мина (её мы не открываем)
```

Проверь руками любую клетку. У левой средней клетки соседи — верхний ряд, центр и две нижние слева; угловой мины среди них нет, значит, там ноль, и она пустая. А у центра мина в соседях есть — там `1`. Чисел ровно три: у мины в углу всего три соседа.

Ключевой вопрос рекурсии: **где база, на которой она остановится?** У заливки баз целых три:

1. клетка **за пределами поля** — стоп (туда нельзя);
2. клетка **уже открыта** — стоп (иначе закружимся: А зовёт В, В зовёт А, А зовёт В… бесконечно);
3. клетка **не пустая** (рядом есть мины) — открыли её и стоп, дальше волну не пускаем.

Без второй базы заливка зациклится насмерть. Это мы специально увидим ниже на наивной версии.

Теперь, когда план ясен, собираем по кусочкам.

Шаг 1. Поле и его печать

Начнём с самого простого: объявим поле и научимся его рисовать. Размеры и число мин вынесем в именованные константы — так их легко поменять в одном месте.

```c
#include <stdio.h>

#define H 9         /* высота поля (строк)   */
#define W 9         /* ширина поля (столбцов) */
#define MINES 10    /* сколько мин на поле    */

int mines[H][W];   /* 1 — здесь мина, 0 — пусто           */
int opened[H][W];  /* 1 — клетка открыта игроком, 0 — нет */
```

Разбор:

- `#define H 9` — именованная константа, знакомая по решету из главы 5: препроцессор перед компиляцией заменит везде в тексте `H` на `9`. Размер поля задан один раз и читается как имя, а не как магическое число `9` посреди кода. Точки с запятой в конце `#define` **не ставят** — это не инструкция Си, а команда препроцессору: с `;` она подставилась бы в текст вместе с точкой с запятой.
- `int mines[H][W];` — двумерный массив `9×9` целых чисел (см. главу 15). Первый индекс — строка, второй — столбец: `mines[r][c]`.
- Оба массива объявлены **снаружи** функций — это **глобальные** переменные. Глобальные массивы автоматически заполняются нулями при старте программы (локальные — нет, в них мусор). Нам это на руку: поле сразу «чистое» — ни мин, ни открытых клеток. И ещё: к глобальным массивам имеют доступ все функции, не нужно таскать их параметрами.

Теперь функция печати. Сверху нарисуем номера столбцов, слева — номера строк, чтобы игроку было куда целиться:

```c
void print_board(void)
{
    printf("   ");
    for (int c = 0; c < W; c++) {
        printf("%d ", c + 1);        /* шапка со столбцами 1..W */
    }
```

```
    printf("\n");

    for (int r = 0; r < H; r++) {
        printf("%2d ", r + 1);       /* номер строки слева */
        for (int c = 0; c < W; c++) {
            if (opened[r][c] == 0) {
                printf(". ");        /* закрытая клетка */
            } else {
                printf("X ");        /* пока заглушка */
            }
        }
        printf("\n");
    }
}
```

Разбор:

- внешний `for` по `r` идёт по строкам, внутренний по `c` — по столбцам. Это стандартный обход двумерного массива из главы 15: для каждой строки печатаем всю её слева направо, потом `\n`.
- `printf("%2d ", r + 1)` — печатаем номер строки. `%2d` — это `%d` с минимальной шириной 2 (помнишь из главы 2), чтобы однозначные и двузначные числа выровнялись в столбик.
- `c + 1` и `r + 1` — **внутри** массива нумерация с нуля (0..8), а игроку показываем по-человечески с единицы (1..9). Про этот сдвиг — отдельный разговор в главном цикле, это частый источник путаницы.

Пока вместо настоящего содержимого клетки печатаем заглушку `X`. Но поле уже рисуется, и каркас готов.

Шаг 2. Расставляем мины случайно

Нужно поставить ровно `MINES` мин в разные клетки. Напрашивается: выбрать случайную клетку и поставить мину. Сделаем наивно:

```
void place_mines(void)
{
    for (int i = 0; i < MINES; i++) {
        int r = rand() % H;          /* случайная строка  0..H-1 */
        int c = rand() % W;          /* случайный столбец 0..W-1 */
        mines[r][c] = 1;
```

```
    }
}
```

- `rand()` возвращает случайное целое (см. главу 12). `rand() % H` даёт остаток от деления на `H`, то есть число в диапазоне `0..H-1` — как раз допустимый индекс строки. Так же для столбца.
- ставим `1` в выбранную клетку — это мина.

Проблема: `rand()` может **дважды выбрать одну и ту же клетку**. Тогда второй `mines[r][c] = 1` ничего не меняет (там уже `1`), и мин окажется **меньше десяти**. Цикл прокрутится 10 раз, но мин будет, скажем, 9.

Чинить надо так: ставить мину только в **свободную** клетку, а счётчик увеличивать **только когда мину реально поставили**. Поэтому вместо `for` на фиксированное число повторов берём `while` и считаем поставленные мины сами:

```
void place_mines(void)
{
    int placed = 0;
    while (placed < MINES) {
        int r = rand() % H;
        int c = rand() % W;
        if (mines[r][c] == 0) {      /* клетка ещё свободна */
            mines[r][c] = 1;
            placed++;                /* только теперь засчитываем */
        }
    }
}
```

Теперь, если случайно выпала занятая клетка, `if` не сработает, `placed` не вырастет, и `while` просто попробует ещё раз. Цикл крутится, пока не наберётся ровно `MINES` разных мин.

Первый ход не должен убивать

Осталась ещё одна беда, и она не про код, а про честность игры. Мины стоят до того, как игрок сделал первый ход. Поле закрыто целиком, подсказок нет — ткнуть можно только наугад. А мин десять на 81 клетку, то есть шанс наткнуться на мину первым же ходом — 10/81, около 12 %. Примерно каждая восьмая партия заканчивалась бы словом «Бабах» на первом же ходу. Проиграть, не успев ничего решить, — обидно, и игрок тут ни при чём.

В настоящем «Сапёре» этой проблемы нет: первый клик безопасен всегда. Сделаем так же. Приём простой: **не ставить мины заранее**. Ждём первого хода, узнаём, куда игрок ткнул, и уже потом раскидываем мины — в обход этой клетки.

`place_mines` принимает координаты безопасной клетки и пропускает её:

```
void place_mines(int safe_r, int safe_c)
{
    int placed = 0;
    while (placed < MINES) {
        int r = rand() % H;
        int c = rand() % W;
        if (r == safe_r && c == safe_c) {
            continue;                /* сюда игрок сейчас ткнёт */
        }
        if (mines[r][c] == 0) {
            mines[r][c] = 1;
            placed++;
        }
    }
}
```

`continue` внутри `while` работает так же, как в главе 12: бросает текущий виток и уходит на новую проверку условия. Выпала запретная клетка — просто крутим дальше, `placed` не растёт.

А в `main` заводим флажок и ставим мины ровно один раз, между проверкой координат и проверкой «мина ли тут»:

Флаг объявляем в начале `main`, рядом с остальными переменными:

```
int mines_placed = 0;            /* мины ставим после первого хода */
```

А сама постановка мин — уже внутри игрового цикла:

```
if (!mines_placed) {             /* самый первый ход задаёт поле */
    place_mines(row, col);
    mines_placed = 1;
}
```

- `int mines_placed = 0;` — обычная переменная-флаг: 0 значит «ещё не ставили», 1 — «уже».
- `if (!mines_placed)` — `!` переворачивает: условие истинно, пока флаг равен нулю. Внутри ставим мины и тут же поднимаем флаг, чтобы на втором ходу поле не перегенерировалось.

- Порядок важен: сначала убеждаемся, что координаты нормальные (иначе `safe_r`/`safe_c` окажутся мусором), и только потом минируем — до строчки, которая проверяет `mines[row][col] == 1`.

Теперь первый ход гарантированно попадает в клетку без мины, и партия всегда начинается с какой-то информации о поле.

Честно ли раскиданы мины

Заодно проверим, что мины ложатся **равномерно** — у каждой клетки, кроме запретной, одинаковый шанс получить мину. Каждая попытка берёт случайную строку и случайный столбец, то есть любую из 81 клетки с равным шансом. Если попали в запретную или занятую клетку, попытка просто выбрасывается, и следующая снова выбирает из всех клеток заново. Выброшенные попытки никакую клетку не подкручивают, поэтому каждая мина встаёт в любую из ещё свободных клеток с равным шансом. Значит, у каждой из 80 разрешённых клеток шанс оказаться с миной одинаковый: 10 мин на 80 клеток, 10/80 = 0,125.

Машина с этим согласна. Я 800 000 раз расставил мины с запретной клеткой в углу и посчитал, как часто мина попадала в каждую клетку: от 0,1242 до 0,1259 при ожидаемых 0,125, а в запретной — ни разу.

Одна крошечная неровность всё же есть, и она в `rand() % H`. `rand()` выдаёт числа от 0 до `RAND_MAX`, на этом маке это 2 147 483 647 (глава 12), то есть всего 2^{31} = 2 147 483 648 разных значений. На 9 они не делятся нацело: $2^{31} = 9 \cdot 238\,609\,294 + 2$. Два «лишних» значения дают остатки 0 и 1, поэтому строки 0 и 1 выпадают на одно значение из 238 609 294 чаще остальных — разница в миллиардных долях, её не измерить. Но если где-то `RAND_MAX` равен минимально разрешённым 32 767, перекос заметнее: $32\,768 = 9 \cdot 3640 + 8$, и у восьми остатков из девяти будет 3641 вариант против 3640. Для игры это всё равно ничтожно.

Шаг 3. Считаем соседей

Самая аккуратная функция в игре — подсчёт мин вокруг клетки. Соседей восемь, их смещения относительно клетки `(r, c)` такие: строка меняется на −1, 0 или +1, и столбец тоже на −1, 0 или +1. Все сочетания этих сдвигов и дают соседей — плюс саму клетку (сдвиг 0,0), которую считать не надо.

Перебрать все сочетания — это **два вложенных цикла по смещениям** от −1 до 1:

```
int count_neighbors(int r, int c)
{
    int total = 0;
    for (int dr = -1; dr <= 1; dr++) {
```

```
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;            /* саму клетку не считаем */
            }
            int nr = r + dr;         /* строка соседа  */
            int nc = c + dc;         /* столбец соседа */
            if (nr < 0 || nr >= H || nc < 0 || nc >= W) {
                continue;            /* сосед за краем поля */
            }
            if (mines[nr][nc] == 1) {
                total++;
            }
        }
    }
    return total;
}
```

Разберём по частям, тут два тонких места.

- `dr` и `dc` (delta row, delta column) пробегают `-1, 0, 1` каждый. Вместе они дают **девять** комбинаций — восемь соседей плюс саму клетку.
- `if (dr == 0 && dc == 0) continue;` — когда оба смещения нулевые, мы смотрим на саму клетку `(r, c)`, а не на соседа. `continue` — «пропусти оставшееся тело цикла и переходи к следующей итерации» (см. главу 2). Так центральная клетка не учитывается.
- `nr = r + dr`, `nc = c + dc` — координаты соседа.

Теперь **проверка границ** — разберём подробно, потому что без неё программа лезет в чужую память. У клетки на краю поля часть соседей оказывается за пределами массива. Например, для клетки в строке 0 сосед сверху имеет `nr = -1` — такой строки нет. Для клетки в последнем столбце `W-1` сосед справа имеет `nc = W`, а допустимые индексы только `0..W-1`.

Условие

```
if (nr < 0 || nr >= H || nc < 0 || nc >= W)
```

ловит все четыре способа вылезти за поле:

- `nr < 0` — выше верхнего края;
- `nr >= H` — ниже нижнего края (индексы строк только до `H-1`);
- `nc < 0` — левее левого края;
- `nc >= W` — правее правого края.

Если хоть одно верно (`||` — «или»), сосед за полем — `continue`, не трогаем его. Помнишь из главы 3: Си **не проверяет** границы массива сам. Обратись к `mines[-1][5]` — и ты читаешь случайный кусок памяти, получишь мусор или падение. Эта проверка — наша собственная защита, и пропускать её нельзя.

Шаг 4. Открываем клетку и запускаем заливку

Теперь сердце игры — рекурсивная заливка. Сначала напишем её **наивно и неправильно**, чтобы своими глазами увидеть, что бывает без базы.

Наивная идея: открыть клетку, и если она пустая — позвать себя для всех восьми соседей.

```
/* НАИВНАЯ ВЕРСИЯ — С БАГОМ, не используй */
void reveal(int r, int c)
{
    opened[r][c] = 1;                /* открыли клетку */
    if (count_neighbors(r, c) == 0) {
        for (int dr = -1; dr <= 1; dr++) {
            for (int dc = -1; dc <= 1; dc++) {
                reveal(r + dr, c + dc);   /* зовём соседей */
            }
        }
    }
}
```

Что тут не так? Сразу несколько бед:

1. Мы зовём `reveal(r + dr, c + dc)` **без проверки границ**. На краю поля это `reveal(-1, ...)`, и первая же строка `opened[-1][c]` пишет в чужую память.
2. Среди девяти вызовов есть `reveal(r, c)` — клетка зовёт **сама себя**. И вообще: открыли A, пошли к соседу B, B пустой — зовёт соседей, среди них снова A. А раз мы нигде не проверяем «уже открыто», A откроется опять, снова позовёт B, B — A… **бесконечная рекурсия**. Из главы 9 ты помнишь, чем это кончается: стек вызовов переполняется, программа падает с **stack overflow**.

Если такую версию запустить, программа аварийно вылетит. Я звал её на поле с единственной миной, открывая центр, и шелл bash сообщил:

```
Bus error: 10
```

Название ошибки зависит от системы и шелла: тот же обвал может называться `Segmentation fault` (ниже мы его ещё увидим) или `zsh: bus error`. Суть одна — программа полезла в память, которую ей трогать нельзя, и система её убила.

Чиним добавлением **тех самых трёх баз** из разбора руками. База рекурсии — это условие, при котором функция сразу выходит, не углубляясь дальше:

```
void reveal(int r, int c)
{
    if (r < 0 || r >= H || c < 0 || c >= W) {
        return;                     /* база: вышли за поле */
    }
    if (opened[r][c] == 1) {
        return;                     /* база: уже открыто */
    }

    opened[r][c] = 1;               /* открываем эту клетку */

    if (count_neighbors(r, c) != 0) {
        return;                     /* база: рядом есть мины — стоп */
    }

    /* Клетка пустая (0 соседей) — продолжаем заливку на 8 соседей. */
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;
            }
            reveal(r + dr, c + dc);
        }
    }
}
```

Что изменилось и почему теперь всё хорошо:

- **Первая база** — проверка границ. Зашли за поле — сразу `return`, никакого доступа к `opened[-1]...`. Это позволяет звать соседей не глядя: вылетевшие за край вызовы тихо вернутся.
- **Вторая база** — `if (opened[r][c] == 1) return;`. Уже открытую клетку второй раз не трогаем. Вот она и разрывает порочный круг A→B→A: когда заливка вернётся к уже открытой клетке, она остановится. **Это главная база против зацикливания.**

- Только пройдя обе проверки, ставим `opened[r][c] = 1`.
- **Третья база** — `if (count_neighbors(r, c) != 0) return;`. Клетку с числом мы открыли (число станет видно), но волну дальше не пускаем: раз рядом мина, соседей раскрывать опасно.
- И лишь для **пустой** клетки идём по восьми соседям и зовём `reveal` для каждого. Каждый такой вызов либо упрётся в базу, либо откроет ещё кусок. Поле конечное, открытых клеток всё больше, повторно их не трогаем — значит, рано или поздно все вызовы упрутся в базы и заливка закончится. Ровно как очередь из главы 9 докатывается до «я первый».

Сравни с главой 9: там у факториала была одна база (`n <= 1`), здесь их три, но принцип тот же — **каждый вызов либо отвечает сразу (база), либо сводит задачу к меньшей** (открыли клетку — непройденных стало меньше).

Насколько глубоко она уходит

Базы спасли нас от бесконечности. Но есть ещё один вопрос, который надо задавать любой рекурсии: **как глубоко** она успевает залезть, пока не начнёт возвращаться? Из главы 9 ты помнишь: каждый незакрытый вызов занимает место в стеке, а стек не резиновый.

Прикинем худший случай. Заливка идёт, пока встречает пустые клетки, и на каждой уходит вглубь, не возвращаясь. Значит, глубина может дорасти до **числа открытых клеток**. На поле без единой мины волна из угла обойдёт все клетки одной длинной цепочкой — глубина будет равна площади поля.

Для нашего поля это ерунда: 9 × 9 = 81 клетка, глубина не больше 82 вызовов. Стек такое не заметит. А вот если в упражнении 3 ты сделаешь поле настраиваемым и разгонишься, картина меняется. Вот измерения на поле без мин (глубина считалась счётчиком внутри `reveal`, стек по умолчанию 8 МБ):

```
поле 9x9,     открыто 81,     глубина 82
поле 100x100, открыто 10000,  глубина 10000
поле 200x200, открыто 40000,  глубина 40000
поле 350x350, открыто 122500, глубина 122500
поле 400x400: Segmentation fault
```

На `400 × 400` рекурсия упирается в потолок стека — 160 000 вложенных вызовов в него уже не помещаются, и программа падает. Если сужать вилку дальше, на этом маке (сборка с `-O2`) поле `360 × 360` ещё проходит (глубина 129 600), а `370 × 370` уже падает. Значит, один вызов `reveal` съедает около 8 МБ / 130 000 ≈ 64 байт стека. Причём падает не из-за ошибки в логике: логика идеальна, просто её цена — память на каждый вызов.

Запомни это как правило: **рекурсивная заливка масштабируется плохо**. Пока поле маленькое и заранее известно — она самый простой и понятный способ, и мы её оставляем. Как только поле начинает расти, заливку переписывают на цикл со **своей** очередью или стеком (глава 11): там данные лежат в куче, где места несравнимо больше, и глубина перестаёт быть проблемой. Это, кстати, хорошее продолжение упражнения 3.

Шаг 5. Главный цикл: ход, проигрыш, победа

Осталось связать всё в игру. Нам ещё нужно:

- показывать мины при проигрыше — добавим в печать параметр;
- считать, сколько клеток открыто, чтобы поймать победу.

Допишем `print_board` параметром `show_mines` и заменим заглушку `X` на настоящее содержимое. Меняется только тело внутреннего цикла:

```
if (show_mines && mines[r][c] == 1) {
    printf("* ");                /* при проигрыше видны мины */
} else if (opened[r][c] == 0) {
    printf(". ");                /* закрытая клетка */
} else {
    int n = count_neighbors(r, c);
    if (n == 0) {
        printf("  ");            /* открыто и пусто */
    } else {
        printf("%d ", n);        /* открыто, рядом n мин */
    }
}
```

- `show_mines && mines[r][c] == 1` — мину рисуем `*`, но только когда `show_mines` включён (мы делаем это лишь при проигрыше);
- иначе закрытую клетку рисуем точкой;
- иначе клетка открыта: ноль соседей — два пробела (пусто), иначе само число.

Победу ловим так: считаем открытые клетки и сравниваем с числом безопасных (`H*W - MINES`). Открыли все безопасные — победа.

```
int count_opened(void)
{
    int total = 0;
    for (int r = 0; r < H; r++)
        for (int c = 0; c < W; c++)
```

```
            if (opened[r][c] == 1) total++;
    return total;
}
```

Осталось собрать игровой цикл `main` (полный текст — в листинге ниже). Его логика: «нарисовали поле → проверили победу → взяли ход → обработали». Ключевые места:

- `srand((unsigned int)time(NULL))` — встряхиваем генератор от текущего времени, чтобы мины каждый раз ложились по-новому (глава 12). Без `srand` поле было бы одинаковым при каждом запуске.
- `while (1)` — бесконечный цикл (`1` всегда истина); выходим через `break` при победе, проигрыше или конце ввода.
- `scanf("%d %d", &row, &col)` — читаем **два** числа за раз. `scanf` возвращает, сколько значений прочитал. Проверка как в главе 12: `EOF` — ввод кончился (Ctrl+D); `rc != 2` — ввели не два числа, чистим строку через `getchar` и просим заново.
- сначала проверяем, что координаты в человеческом диапазоне: строка `1..H`, столбец `1..W`. Именно **до** вычитания единицы: если игрок наберёт `-2147483648` (это `INT_MIN`, глава 2), то `row - 1` уже не влезет в `int`. Знаковое переполнение — неопределённое поведение, и санитайзер (`-fsanitize=undefined`) на такой ввод ругается `signed integer overflow`. Проверка до вычитания этого не допускает.
- `row--; col--;` — **перевод координат из человеческих в машинные**. Игрок вводит с **1**, массив индексируется с **0**. Забыть этот сдвиг — классическая ошибка: клетка «1 1» превратится в индексы `[1][1]`, всё поле съедет на одну клетку, а последняя строка и последний столбец станут недоступны. Вычли единицу — и `row`, `col` стали правильными индексами.
- дальше ещё две проверки: уже открыто? мина (тогда `print_board(1)` и `break`)? Если всё чисто — `reveal` открывает клетку и запускает заливку.

Полная программа

Файл `code/16-minesweeper.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define H 9         /* высота поля (строк)   */
#define W 9         /* ширина поля (столбцов) */
#define MINES 10    /* сколько мин на поле    */
```

```
int mines[H][W];   /* 1 — здесь мина, 0 — пусто           */
int opened[H][W];  /* 1 — клетка открыта игроком, 0 — нет */

/* Сколько мин стоит вокруг клетки (r, c) — среди 8 соседей. */
int count_neighbors(int r, int c)
{
    int total = 0;
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;            /* саму клетку не считаем */
            }
            int nr = r + dr;         /* строка соседа  */
            int nc = c + dc;         /* столбец соседа */
            if (nr < 0 || nr >= H || nc < 0 || nc >= W) {
                continue;            /* сосед за краем поля */
            }
            if (mines[nr][nc] == 1) {
                total++;
            }
        }
    }
    return total;
}

/* Рекурсивная заливка: открыть клетку (r, c) и, если она пустая,
   открыть всех её соседей, и так дальше. */
void reveal(int r, int c)
{
    if (r < 0 || r >= H || c < 0 || c >= W) {
        return;                      /* база: вышли за поле */
    }
    if (opened[r][c] == 1) {
        return;                      /* база: уже открыто */
    }

    opened[r][c] = 1;                /* открываем эту клетку */
```

```
    if (count_neighbors(r, c) != 0) {
        return;                     /* база: рядом есть мины — стоп */
    }

    /* Клетка пустая (0 соседей) — продолжаем заливку на 8 соседей. */
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;
            }
            reveal(r + dr, c + dc);
        }
    }
}

/* Расставить MINES мин по случайным клеткам без повторов.
   Клетку (safe_r, safe_c) — первый ход игрока — не минируем,
   иначе можно проиграть на первом же ходу, ничего не зная о поле. */
void place_mines(int safe_r, int safe_c)
{
    int placed = 0;
    while (placed < MINES) {
        int r = rand() % H;
        int c = rand() % W;
        if (r == safe_r && c == safe_c) {
            continue;               /* сюда игрок сейчас ткнёт */
        }
        if (mines[r][c] == 0) {     /* клетка ещё свободна */
            mines[r][c] = 1;
            placed++;
        }
    }
}

/* Напечатать поле. Если show_mines != 0 — показать и мины (при проигрыше). */
void print_board(int show_mines)
{
    printf("   ");
```

```
    for (int c = 0; c < W; c++) {
        printf("%d ", c + 1);        /* шапка со столбцами 1..W */
    }
    printf("\n");

    for (int r = 0; r < H; r++) {
        printf("%2d ", r + 1);       /* номер строки слева */
        for (int c = 0; c < W; c++) {
            if (show_mines && mines[r][c] == 1) {
                printf("* ");
            } else if (opened[r][c] == 0) {
                printf(". ");        /* закрытая клетка */
            } else {
                int n = count_neighbors(r, c);
                if (n == 0) {
                    printf("  ");    /* открыто и пусто */
                } else {
                    printf("%d ", n);
                }
            }
        }
        printf("\n");
    }
}

/* Сколько клеток уже открыто. */
int count_opened(void)
{
    int total = 0;
    for (int r = 0; r < H; r++) {
        for (int c = 0; c < W; c++) {
            if (opened[r][c] == 1) {
                total++;
            }
        }
    }
    return total;
}
```

```c
int main(void)
{
    srand((unsigned int)time(NULL));

    int mines_placed = 0;            /* мины ставим после первого хода */
    int safe_cells = H * W - MINES;  /* столько клеток без мин */

    printf("Сапёр %dx%d, мин: %d\n", H, W, MINES);
    printf("Вводи координаты: строка 1..%d и столбец 1..%d.\n", H, W);

    while (1) {
        print_board(0);

        if (count_opened() == safe_cells) {
            printf("Победа! Все безопасные клетки открыты.\n");
            break;
        }

        printf("Ход (строка столбец): ");
        int row, col;
        int rc = scanf("%d %d", &row, &col);
        if (rc == EOF) {
            printf("\nИгра прервана.\n");
            break;
        }
        if (rc != 2) {
            printf("Нужно ввести два числа. Попробуй ещё раз.\n");
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
                /* выбрасываем мусорный ввод до конца строки */
            }
            continue;
        }

        /* Диапазон проверяем ДО вычитания единицы: из INT_MIN
           вычесть 1 нельзя — это переполнение int. */
        if (row < 1 || row > H || col < 1 || col > W) {
```

```
            printf("Координаты вне поля: строка 1..%d, столбец 1..%d.\n",
                   H, W);
            continue;
        }

        row--;                      /* игрок считает с 1, массив — с 0 */
        col--;
        if (opened[row][col] == 1) {
            printf("Эта клетка уже открыта.\n");
            continue;
        }

        if (!mines_placed) {        /* самый первый ход задаёт поле */
            place_mines(row, col);
            mines_placed = 1;
        }

        if (mines[row][col] == 1) {
            print_board(1);         /* показываем все мины */
            printf("Бабах! Ты открыл мину. Игра окончена.\n");
            break;
        }

        reveal(row, col);           /* безопасный ход — заливаем */
    }

    return 0;
}
```

Как это работает

Пройдём один ход целиком.

1. Оба массива, `mines` и `opened`, пока в нулях: мин ещё нет (их поставим после первого хода), не открыто ничего.
2. `print_board(0)` рисует поле из точек — игрок видит закрытую сетку.
3. `count_opened()` вернёт 0, до `safe_cells` (это 81 − 10 = 71) далеко, победы нет.

4. Игрок вводит, скажем, `1 1`. `scanf` прочитал два числа, `rc == 2`. Оба числа в диапазоне `1..9`, и `row--`, `col--` превращают их в индексы `0, 0`.
5. Клетка `(0,0)` не открыта. Ход первый, значит `place_mines(0, 0)` раскидывает 10 мин по всему полю, кроме этой клетки. Мины тут гарантированно нет — зовём `reveal(0, 0)`.
6. Если вокруг `(0,0)` нет мин, заливка раскрывает целую область: `reveal` зовёт соседей, те — своих, пока волна не упрётся в клетки с числами и в края поля. На экране появится пустая поляна, окаймлённая цифрами.
7. Цикл повторяется. Когда `count_opened()` дорастёт до 71, печатается «Победа». Если же игрок ткнёт в мину — печатаем поле с `*` и «Бабах».

Проверяем

Скомпилируй без предупреждений:

```
cc -std=c11 -Wall -Wextra -O2 code/16-minesweeper.c -o minesweeper
```

Запусти `./minesweeper` и поиграй вживую. Вот целая партия из двух ходов — настоящий прогон, у тебя поле будет своё, потому что мины раскладываются случайно. Игрок открыл угол `1 1`, а вторым ходом ткнул наугад в низ поля:

```
Сапёр 9x9, мин: 10
Вводи координаты: строка 1..9 и столбец 1..9.
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . . . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
Ход (строка столбец): 1 1
   1 2 3 4 5 6 7 8 9
 1       1 . . . . .
 2     1 3 . 3 1 3 .
 3     1 . . 1   1 .
 4     1 2 2 1   1 .
 5               1 .
 6       1 1 1   1 .
```

```
 7       1 . 1   1 1
 8 1 1 1 1 . 2 1
 9 . . . . . . 1
Ход (строка столбец): 9 6
   1 2 3 4 5 6 7 8 9
 1       1 * . * . *
 2     1 3 . 3 1 3 .
 3     1 * * 1   1 *
 4     1 2 2 1   1 .
 5               1 .
 6       1 1 1   1 *
 7       1 * 1   1 1
 8 1 1 1 1 . 2 1
 9 . * . . . * 1
Бабах! Ты открыл мину. Игра окончена.
```

Разберём картинку после первого хода — в ней вся суть заливки.

Клетка `(1,1)` оказалась пустой: вокруг неё ни одной мины. Значит, `reveal` открыл её и пошёл по соседям, те — по своим, и волна катилась, пока не упёрлась в клетки с числами. Пробелы на экране — это открытые пустые клетки, целая поляна вдоль левого края и по низу. Числа по её краю — это граница: за ними уже начинается минное поле, и дальше волна не пошла. Точки — то, до чего заливка не добралась.

Проверь глазами любое число, например `3` в клетке `(2,4)`. После второго хода видно, где мины: `(1,5)`, `(3,4)`, `(3,5)`. Все три — соседи `(2,4)`. Сходится.

Второй ход попал в мину `(9,6)`, и `print_board(1)` показал все звёздочки разом. Пересчитай их: `(1,5)`, `(1,7)`, `(1,9)`, `(3,4)`, `(3,5)`, `(3,9)`, `(6,9)`, `(7,5)`, `(9,2)`, `(9,6)` — ровно десять, как и просили в `MINES`. И ни одна не стоит в `(1,1)`, куда игрок ткнул первым ходом: `place_mines` эту клетку обошла.

А вот последний экран выигранной партии — той, где игрок аккуратно вскрыл все безопасные клетки:

```
Ход (строка столбец): 1 9
   1 2 3 4 5 6 7 8 9
 1     1 1 1     1 1
 2     1 . 2 1 2 2 .
 3     1 1 2 . 2 . 3
 4     1 1 3 2 3 2 .
```

```
 5     1 . 2 . 1 1 1
 6     1 1 2 1 2 2 2
 7         1 1 2 . .
 8         1 . 2 2 2
 9         1 1 1
Победа! Все безопасные клетки открыты.
```

Точек осталось ровно десять — это и есть мины. Открывать их не надо и нельзя: победа наступает, когда открыты все **не**минные клетки, то есть 81 − 10 = 71 штука. Программа их и посчитала.

Пробуем сломать игру пальцами

Отдельно проверь, что игру не роняет кривой ввод. Ткни дважды в одну клетку, введи нулевую строку, набери буквы вместо чисел:

```
Сапёр 9x9, мин: 10
Вводи координаты: строка 1..9 и столбец 1..9.
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . . . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
Ход (строка столбец): 5 5
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . 2 . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
```

```
Ход (строка столбец): 5 5
Эта клетка уже открыта.
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . 2 . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
Ход (строка столбец): 0 3
Координаты вне поля: строка 1..9, столбец 1..9.
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . 2 . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
Ход (строка столбец): abc
Нужно ввести два числа. Попробуй ещё раз.
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . 2 . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
Ход (строка столбец): 6 6
   1 2 3 4 5 6 7 8 9
```

```
  1 . * * . . . . .
  2 . * . * . . . .
  3 . . . . . . . .
  4 . * . . . . * .
  5 . . . . 2 . . .
  6 . * . . * * . * .
  7 . . . . . . . .
  8 . . . . . . . .
  9 . . . . . . . .
Бабах! Ты открыл мину. Игра окончена.
```

Каждая ошибка получила своё сообщение, поле не изменилось, ход остался за игроком. Буквы `abc` не отправили игру в бесконечный поток ругани — это работает чистка буфера через `getchar` после неудачного `scanf`.

И заодно эта партия показывает границы нашей защиты первого хода. Игрок ткнул в `5 5`, мины туда не поставили — но соседи-то не запрещены, и клетка сразу показала `2`. Первый ход **безопасен**, но не обязательно открывает поляну. В настоящем «Сапёре» запрещают мины и вокруг первой клетки — это упражнение 4.

Частые ошибки

Выход за границы при подсчёте соседей. Самая частая беда этой игры — забыть проверку `nr < 0 || nr >= H || nc < 0 || nc >= W` в `count_neighbors` (и аналогичную базу в `reveal`). Для клеток у края соседи вылезают за массив, и без проверки ты читаешь чужую память: числа врут, а то и падение. Си границы не сторожит (глава 3) — это твоя забота.

Заливка без базы → зацикливание. Если убрать `if (opened[r][c] == 1) return;`, соседние пустые клетки начнут открывать друг друга по кругу — A зовёт B, B зовёт A, бесконечно. Стек вызовов переполнится, и программа рухнет с `stack overflow` (глава 9). База «уже открыто» обязательна.

Заливка с базой, но на огромном поле. База спасает от бесконечной рекурсии, но не от глубокой. Глубина заливки дорастает до числа открытых клеток: на `9 × 9` это 82 вызова (пустяк), на `400 × 400` без мин — около 160 000, и программа падает с `Segmentation fault`. Увеличиваешь поле — переводи заливку на цикл со своей очередью.

Путаница в нумерации координат. Игрок считает с 1, массив — с 0. Забудешь `row--; col--;` — и поле съедет на клетку, а если ещё и проверку границ писать под индексы `0..8`, ввод `9` вылезет за поле на `mines[9][...]`. Решай это **в одном месте**, сразу после ввода: проверил диапазон `1..9`,

вычел единицу — и дальше везде чистые индексы `0..8`. Обратный порядок (сначала вычесть, потом проверить) ломается на `INT_MIN`: вычитание переполняет `int`.

Мины ставятся с повторами. Если расставлять мины простым `for` на `MINES` повторов без проверки `if (mines[r][c] == 0)`, в одну клетку может попасть две «мины», и их окажется меньше задуманного. Считай **реально поставленные**, как в `place_mines`.

`[H][W]` против `[r][c]`. Лёгко перепутать порядок и написать `mines[c][r]`. Договорись раз и навсегда: первый индекс — строка, второй — столбец, и держись этого во всех функциях.

Упражнения

1. **Счётчики.** Заведи переменную `moves` и при победе печатай, за сколько ходов игрок прошёл поле. А после каждого хода выводи «открыто N из 71» — пусть видно, сколько осталось до конца. Число 71 не вписывай руками, оно уже посчитано в `safe_cells`. *Что должно получиться:* после первого хода из разбора выше (`1 1`, большая поляна) программа печатает «открыто 60 из 71»: на той картинке 21 точка, а 81 − 21 = 60. `moves` увеличивай только за настоящий ход, после всех проверок: ввод `5 5` в уже открытую клетку или `0 3` ходом не считается.

2. **Флажки на мины.** Добавь третий массив `flags[H][W]` и команду пометить клетку флажком (например, ввод `0 r c` — пометить, `1 r c` — открыть). Флажок печатай как `F`. Это помогает игроку помнить, где он подозревает мину. *Что должно получиться:* `0 2 2` рисует `F` во второй строке и втором столбце, повторное `0 2 2` флажок снимает (удобно сделать `flags[r][c] = !flags[r][c]`). Открыть клетку под флажком нельзя — пусть программа так и скажет. А заливка `reveal` должна флажки обходить стороной, иначе она откроет клетку, которую игрок пометил.

3. **Размер и число мин с ввода.** Спроси у игрока `H`, `W` и `MINES` в начале. Подсказка: с фиксированными `#define` так нельзя — понадобятся либо глобальные массивы максимального размера, либо `malloc` (глава 3). Не забудь проверить, что мин меньше, чем клеток. *Что должно получиться:* при `H = 2`, `W = 2`, `MINES = 3` первый ход сразу приносит победу: мины встают во все три клетки, кроме твоей, и безопасных клеток 4 − 3 = 1. А `MINES = 4` на поле 2×2 программа должна отклонить: с запретом на первую клетку мины ставить некуда, и `place_mines` крутилась бы вечно. Строго говоря, мин должно быть **не больше** `H * W - 1`.

4. **Первый ход открывает поляну.** Мы уже сделали первую клетку безопасной. В настоящем «Сапёре» идут дальше: первый ход почти всегда раскрывает целую пустую область. Добейся того же — запрети мины не только в самой клетке, но и во всех восьми её соседях (тогда число соседних мин там будет ноль, и заливка пойдёт). Подсказка: условие в `place_mines` станет проверкой «разница по строке и по столбцу не больше единицы». Не

забудь: мин должно по-прежнему хватить на оставшиеся клетки. *Что должно получиться:* первый ход всегда показывает пустую клетку, и вокруг неё открывается хотя бы квадрат 3×3 (у края поля — 2×3, в углу — 2×2). Решение ниже.

5. **Отметить роковую клетку.** Сейчас при проигрыше все мины выглядят одинаково, и непонятно, на какой именно ты подорвался. Пусть та, в которую ткнул игрок, печатается другим символом — например, `X`. Подсказка: координаты хода у тебя уже есть, передай их в `print_board` рядом с `show_mines`. *Что должно получиться:* в партии из раздела «Проверяем» после хода `9 6` на месте `(9,6)` стоит `X`, а остальные девять мин остаются звёздочками. Для обычной печати передавай координаты `-1, -1` — таких клеток нет, и `X` не появится.

6. **Заливка без рекурсии.** Перепиши `reveal` на цикл. Заведи массив координат «что ещё надо обработать» (это очередь из главы 11): клади туда клетку, потом в цикле бери из очереди по одной, открывай и докладывай её соседей, пока очередь не опустеет. Результат должен быть точно таким же, а глубина стека перестанет расти вместе с полем. Проверь на поле 400 × 400 без мин — там, где рекурсия падала. *Что должно получиться:* поле 400 × 400 без мин открывается целиком (160 000 клеток) без падения. Решение ниже.

Решения упражнений 4 и 6

Упражнение 4. Меняется одна проверка в `place_mines`:

```
if (abs(r - safe_r) <= 1 && abs(c - safe_c) <= 1) {
    continue;          /* клетка первого хода или её сосед */
}
```

`abs` из `<stdlib.h>` — модуль числа: `abs(-1)` равно 1, `abs(2)` равно 2. Строка отличается от строки хода не больше чем на единицу и столбец тоже — значит, это сама клетка хода или один из восьми соседей. Запретных клеток теперь до девяти, поэтому мин должно быть не больше `H * W - 9`: у нас 10 мин на 81 клетку, запас огромный. Я прогнал 100 000 первых ходов в случайные клетки: мины всегда ставятся все десять, первая клетка всегда пустая, и открывается не меньше соседей, чем обещано выше.

Упражнение 6. Клетку отмечаем открытой **в момент, когда кладём её в очередь**. Тогда каждая клетка попадает в очередь не больше одного раза, и очереди хватит `H * W` мест:

```
int queue_r[H * W];   /* очередь клеток: строки */
int queue_c[H * W];   /* ...и столбцы */

void reveal(int r, int c)
{
    int head = 0;     /* откуда берём */
```

```
    int tail = 0;     /* куда кладём  */

    opened[r][c] = 1;
    queue_r[tail] = r;
    queue_c[tail] = c;
    tail++;

    while (head < tail) {
        int cr = queue_r[head];
        int cc = queue_c[head];
        head++;
        if (count_neighbors(cr, cc) != 0) {
            continue;         /* клетка с числом: дальше не идём */
        }
        for (int dr = -1; dr <= 1; dr++) {
            for (int dc = -1; dc <= 1; dc++) {
                int nr = cr + dr;
                int nc = cc + dc;
                if (nr < 0 || nr >= H || nc < 0 || nc >= W) {
                    continue;         /* за краем поля */
                }
                if (opened[nr][nc] == 1) {
                    continue;         /* уже открыта или в очереди */
                }
                opened[nr][nc] = 1;   /* открываем при постановке */
                queue_r[tail] = nr;
                queue_c[tail] = nc;
                tail++;
            }
        }
    }
}
```

- Очередь — два глобальных массива и два номера: `head` показывает, какую клетку брать следующей, `tail` — куда класть новую. Пока `head < tail`, в очереди что-то есть.
- Пару `dr == 0 && dc == 0` отдельно пропускать не нужно: сама клетка уже открыта, и её отсечёт проверка `opened`.

- Массивы глобальные не случайно: на поле 400 × 400 это 2 · 160 000 чисел `int` по 4 байта, около 1,3 МБ. В стеке с его 8 МБ они бы пока поместились, но на поле побольше уже нет, а глобальные массивы живут вне стека.
- Звать `reveal` можно только для закрытой клетки — `main` это уже проверяет.

Проверка: я сравнил эту версию с рекурсивной на 2000 случайных полях 30 × 30 со случайными минами — открытые клетки совпали во всех случаях. Поле 400 × 400 без мин открылось целиком, все 160 000 клеток, и даже 1000 × 1000 (миллион клеток) прошло без падения.

Что мы получили

Мы собрали настоящую игру из готовых кирпичей: **два двумерных массива** как наложенные карты (где мины и что открыто), **случайную расстановку без повторов и без мины под первым ходом**, **подсчёт восьми соседей с проверкой границ** и **рекурсивную заливку с тремя базами**. Ни одной новой конструкции языка — только знакомое из прошлых глав, сложенное в новую форму. Так и пишутся программы посложнее: не из экзотики, а из комбинаций простых, надёжных деталей.

Заодно мы ещё раз прочувствовали рекурсию (глава 9) на живой задаче и увидели, почему база — это не формальность, а единственное, что не даёт программе рухнуть.

В следующей главе возьмёмся за игру в реальном времени — **«Змейку»**. Там поле тоже будет двумерным, но появится новое: считывать клавиши **без нажатия Enter** и перерисовывать экран в цикле, пока змейка ползёт. Для этого познакомимся с режимом терминала через `termios`. Готовь стрелки.

Глава 17. Игра «Змейка»

Вот мы и добрались до финала. «Змейка» — та самая игра из старых телефонов: по полю ползёт змейка, ест еду, растёт и не должна врезаться ни в стену, ни в саму себя. Игра простая на вид, но в ней сходится всё, что мы прошли: двумерное поле, массивы координат, структура состояния, случайная еда, цикл и проверки столкновений.

И тут появляется по-настоящему новое. Раньше наши программы ждали, пока ты нажмёшь клавишу и Enter. Змейка так не умеет: она ползёт **сама**, а клавишу должна ловить **мгновенно**, без Enter и без эха на экране. За это отвечает «сырой» режим терминала через `termios` — главная новая тема главы. Разберём её спокойно и по частям.

Что мы сделаем

К концу главы у тебя будет настоящая игра в терминале:

- поле с рамкой из символов `#`;
- змейка (`0` — голова, `o` — тело), которая ползёт сама;
- управление клавишами W A S D **без нажатия Enter**;
- еда `*`, которая появляется в случайном месте; съел — змейка растёт и счёт увеличивается;
- проигрыш при столкновении со стеной или с собственным телом;
- победа, если змейка заняла всё поле целиком;
- выход по клавише `q` — и терминал, который остаётся рабочим при любом способе закончить игру.

Строить будем как всегда: от простого к сложному. Сначала просто нарисуем поле и неподвижную змейку. Потом заставим её двигаться. Потом научимся ловить клавиши. Потом добавим еду и рост. И в конце — столкновения и проигрыш.

Разбираем задачу руками

Прежде чем писать код, поймём две вещи: как хранить змейку и почему обычного ввода тут не хватит.

Как хранить змейку

Змейка — это цепочка клеток на поле. Каждая клетка — пара чисел: номер столбца `x` и номер строки `y`. Заведём два массива:

```
snake_x:  9  8  7
snake_y:  5  5  5
```

Это змейка из трёх сегментов на одной строке. Элемент с индексом `0` — **голова** (`x=9, y=5`), дальше идёт тело. Длину храним отдельно в переменной `snake_len`.

Как змейка делает шаг? Представь гусеницу: каждый сегмент переползает на место предыдущего, а голова делает шаг в новую клетку. Сдвинули тело — поставили новую голову. Всё.

```
было:   голова(9,5)  тело(8,5)  тело(7,5)
сдвиг:  ?(9,5)        (9,5)      (8,5)      <- каждый занял место соседа
голова: (10,5)        (9,5)      (8,5)      <- голова шагнула вправо
```

Направление движения тоже храним парой чисел `dir_x, dir_y`: вправо — это `(1, 0)`, влево — `(-1, 0)`, вверх — `(0, -1)`, вниз — `(0, 1)`. Обрати внимание: вверх — это **минус** по `y`, потому что строки нумеруются сверху вниз (нулевая строка — самая верхняя).

Почему обычный ввод не подходит

По умолчанию терминал работает в **каноническом режиме** (canonical mode, его ещё зовут «строчным»). Это значит две вещи:

1. Терминал **ждёт Enter**. Что бы ты ни печатал, программа ничего не получит, пока ты не нажмёшь Enter. Вся строка копится в буфере.
2. Терминал **печатает эхо** — показывает на экране каждый набранный символ.

Для обычной программы это удобно: можно стереть опечатку до Enter. Но для игры — катастрофа. Змейка должна реагировать на `d` сразу, а не ждать Enter. И буквы `wasd` не должны замусоривать наше поле.

Значит, нам нужно временно выключить два этих поведения: ожидание Enter и эхо. А ещё сделать чтение **неблокирующим** — чтобы программа не зависала, ожидая клавишу, когда её не нажали. Змейка ведь ползёт и без ввода.

Всё это делается через `termios` — набор настроек терминала в POSIX (стандарт для macOS и Linux). На Windows механизм другой (там для той же задачи берут `_getch` из `<conio.h>`), но мы пишем под macOS/Linux.

Шаг 1. Поле и неподвижная змейка

Начнём с самого простого: нарисуем рамку, змейку и еду — пока без всякого движения. Цель шага — увидеть картинку.

```c
#include <stdio.h>

#define WIDTH   20
#define HEIGHT  10

int main(void)
{
    int snake_x[3] = {9, 8, 7};
    int snake_y[3] = {5, 5, 5};
    int snake_len = 3;

    /* верхняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');

    for (int y = 0; y < HEIGHT; y++) {
        putchar('#');                       /* левая стенка */
        for (int x = 0; x < WIDTH; x++) {
            char ch = ' ';
            for (int i = 0; i < snake_len; i++) {
                if (snake_x[i] == x && snake_y[i] == y) {
                    ch = (i == 0) ? 'O' : 'o';
                    break;
                }
            }
            putchar(ch);
        }
        putchar('#');                       /* правая стенка */
```

```
        putchar('\n');
    }

    /* нижняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');
    return 0;
}
```

Разбор нового:

- `#define WIDTH 20` — именованная константа, как `H` и `W` в сапёре (см. главу 16): препроцессор перед компиляцией заменит везде в тексте `WIDTH` на `20`. Захочешь поле побольше — поменяешь одну строку.
- `WIDTH` и `HEIGHT` — ширина и высота **внутренней** части поля, без рамки. Рамка добавляет по одному символу слева и справа, поэтому в цикле рамки идёт `WIDTH + 2`.
- `putchar('#')` — функция из `stdio.h`, печатает **один символ**. Это как `printf("%c", '#')`, только короче: не нужна строка формата. Аргумент — символ в одинарных кавычках (точнее, его код, тип `int`).
- Внешний цикл по `y` идёт по строкам поля (0..HEIGHT−1), внутренний по `x` — по столбцам (0..WIDTH−1). Для каждой клетки `(x, y)` решаем, что в ней напечатать.
- `char ch = ' ';` — по умолчанию клетка пустая (пробел).
- Внутренний цикл `for (int i = 0; ...)` проверяет: не стоит ли в этой клетке сегмент змейки? Если координаты сегмента `i` совпали с `(x, y)`, рисуем символ.
- `ch = (i == 0) ? 'O' : 'o';` — тернарный оператор из главы 13. Читается так: «если `i == 0`, то значение `'O'`, иначе `'o'`». Голова (индекс 0) — заглавная `O`, тело — строчная `o`.
- `break;` — как только нашли сегмент в клетке, выходим из внутреннего цикла: дальше проверять нет смысла.

Скомпилируй и запусти:

```
######################
#                    #
#                    #
#                    #
#                    #
#                    #
#       ooO          #
```

```
#                    #
#                    #
#                    #
#                    #
######################
```

Поле есть, змейка на месте (голова `O` справа, тело `oo` слева). Но она не двигается. Чиним.

Шаг 2. Заставляем змейку ползти

Чтобы змейка двигалась, нужен **игровой цикл**: бесконечно повторяем «сдвинули змейку → перерисовали поле → подождали чуть-чуть». А чтобы картинка не уезжала вниз простынёй, перед каждой отрисовкой будем **очищать экран**.

Очистка делается специальной командой для терминала — **управляющей последовательностью** (escape sequence):

```
printf("\033[H\033[2J");
```

Разберём по кускам:

- `\033` — это символ Escape (его код 27). Запись `\033` — код в восьмеричной системе. С него начинаются почти все команды управления терминалом. Терминал, увидев Escape и `[`, понимает: дальше не текст для печати, а команда.
- `\033[H` — команда «переместить курсор в левый верхний угол» (Home). Следующая печать пойдёт с самого верха.
- `\033[2J` — команда «стереть весь экран» (J — erase, 2 — целиком).

Вместе `\033[H\033[2J` — «очисти экран и поставь курсор в угол». Это работает в любом нормальном терминале macOS/Linux.

Теперь сдвиг змейки. Как договорились, тело переползает по цепочке, а голова шагает в направлении `dir`:

```
/* сдвигаем тело: каждый сегмент встаёт на место предыдущего */
for (int i = snake_len - 1; i > 0; i--) {
    snake_x[i] = snake_x[i - 1];
    snake_y[i] = snake_y[i - 1];
}
/* голова делает шаг */
snake_x[0] += dir_x;
snake_y[0] += dir_y;
```

Идти надо **с хвоста к голове** (`i` от конца к 1). Если бы мы шли с головы, то затёрли бы координаты, которые ещё нужны соседнему сегменту.

Чтобы между кадрами была видимая пауза, нужна функция `usleep` (*microsecond sleep* — «спать микросекунды») из `<unistd.h>`. Её придумали в BSD-Unix, потом она попала в стандарт POSIX, а в 2008 году её оттуда убрали как устаревшую (на замену есть `nanosleep`). Но и macOS, и Linux её по-прежнему поддерживают, а для игры она проще всех:

```
usleep(150000);   /* 150 000 микросекунд = 0,15 секунды */
```

В одной секунде миллион микросекунд, так что `150000` — это примерно 0,15 секунды между кадрами. Чем меньше число, тем быстрее змейка.

Маленькая деталь сборки. Мы пишем со стандартом `-std=c11`, а `usleep` — функция POSIX, не из чистого C11. Флаг `-std=c11` просит компилятор быть строгим, и часть POSIX-объявлений в заголовках может оказаться спрятана. Поэтому в самом начале файла, **до всех `#include`**, ставим строку:

```
#define _DEFAULT_SOURCE
```

Это «фича-макрос» (feature test macro) — переключатель, которым ты говоришь заголовкам, какой набор объявлений тебе нужен. `#define` без значения просто «включает» имя, само число тут ни при чём.

Разберём точно, что этот макрос делает, потому что в интернете на этот счёт много путаницы:

- На **Linux с glibc** он нужен по-настоящему. Там объявление `usleep` в `<unistd.h>` закрыто условием `#if defined __USE_MISC ...`, а `__USE_MISC` включается как раз через `_DEFAULT_SOURCE`. Без макроса строгий `-std=c11` даст `implicit declaration of function 'usleep'` — предупреждение, а в gcc 14 и новее уже ошибку сборки.
- На **macOS** он не меняет ничего. В тамошнем `<unistd.h>` объявление `usleep` стоит под условием `#if __DARWIN_C_LEVEL >= 199506L`, а оно при обычной сборке истинно, и `-std=c11` его не выключает. Проверить это можно за десять секунд — убери `#define` и собери файл: на маке соберётся молча.

Так зачем оставлять? Чтобы один и тот же файл собирался без предупреждений и на маке, и на Linux. Лишний макрос ничего не ломает, а его отсутствие ломает сборку у половины читателей. Если не хочешь трогать исходник — тот же эффект даёт флаг `-D_DEFAULT_SOURCE` в командной строке.

Соберём шаг 2 целиком — змейка ползёт вправо сама:

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <unistd.h>
```

```c
#define WIDTH   20
#define HEIGHT  10

int main(void)
{
    int snake_x[3] = {9, 8, 7};
    int snake_y[3] = {5, 5, 5};
    int snake_len = 3;
    int dir_x = 1, dir_y = 0;   /* ползём вправо */

    for (int step = 0; step < 8; step++) {   /* пока 8 шагов для пробы */
        printf("\033[H\033[2J");

        /* рисуем поле */
        for (int x = 0; x < WIDTH + 2; x++) putchar('#');
        putchar('\n');
        for (int y = 0; y < HEIGHT; y++) {
            putchar('#');
            for (int x = 0; x < WIDTH; x++) {
                char ch = ' ';
                for (int i = 0; i < snake_len; i++) {
                    if (snake_x[i] == x && snake_y[i] == y) {
                        ch = (i == 0) ? 'O' : 'o';
                        break;
                    }
                }
                putchar(ch);
            }
            putchar('#');
            putchar('\n');
        }
        for (int x = 0; x < WIDTH + 2; x++) putchar('#');
        putchar('\n');

        /* двигаем змейку */
        for (int i = snake_len - 1; i > 0; i--) {
            snake_x[i] = snake_x[i - 1];
```

```
            snake_y[i] = snake_y[i - 1];
        }
        snake_x[0] += dir_x;
        snake_y[0] += dir_y;

        usleep(150000);
    }
    return 0;
}
```

Запусти — увидишь, как змейка проползает вправо восемь шагов и программа завершается. Уже живая картинка! Но управлять ей мы пока не можем, а на стену она просто уедет. Добавим управление.

Шаг 3. Ловим клавиши без Enter — сырой режим

Это сердце главы. Нам нужно:

1. выключить ожидание Enter и эхо (сырой режим);
2. читать клавишу **не блокируясь** — нет нажатия, идём дальше;
3. в конце игры **вернуть** терминал в обычное состояние.

Сначала подключаем заголовки:

```
#include <termios.h>   /* настройки терминала */
#include <unistd.h>    /* read, STDIN_FILENO, usleep, _exit */
#include <signal.h>    /* signal, raise — перехват Ctrl+C и Ctrl+Z */
```

`<termios.h>` даёт структуру `struct termios` (набор флагов терминала) и функции `tcgetattr`/`tcsetattr` (прочитать и записать эти настройки). `STDIN_FILENO` из `<unistd.h>` — это номер стандартного ввода (клавиатуры), равен 0. `<signal.h>` понадобится чуть ниже, когда будем страховаться от Ctrl+C.

Включаем сырой режим

```
static struct termios orig_termios;   /* сюда сохраним исходные настройки */
static int raw_saved;                 /* 1 — orig_termios заполнен */

/* Вернуть терминал в нормальный режим. */
static void disable_raw_mode(void)
{
```

```
    if (raw_saved) {
        tcsetattr(STDIN_FILENO, TCSANOW, &orig_termios);
    }
}

/* Применить игровой режим поверх сохранённых настроек. */
static void apply_raw_mode(void)
{
    struct termios raw = orig_termios;          /* копия для правки */

    raw.c_lflag &= ~(ICANON | ECHO);            /* снять канонику и эхо */
    raw.c_cc[VMIN]  = 0;                        /* read не ждёт символов */
    raw.c_cc[VTIME] = 0;                        /* и не ждёт по таймеру */

    tcsetattr(STDIN_FILENO, TCSANOW, &raw);     /* применить сейчас же */
}

/* Смертельный сигнал (Ctrl+C, kill, закрытие окна): чиним терминал и
   только потом умираем. Сигнал убивает процесс мимо atexit — поэтому
   и перехватываем. */
static void on_fatal_signal(int sig)
{
    disable_raw_mode();
    _exit(128 + sig);    /* 128 + номер сигнала — так принято в шеллах */
}

/* Ctrl+Z: отдать терминал шеллу в нормальном виде и честно уснуть. */
static void on_sigtstp(int sig)
{
    (void)sig;
    disable_raw_mode();
    signal(SIGTSTP, SIG_DFL);   /* вернуть штатное поведение... */
    raise(SIGTSTP);             /* ...и остановиться по-настоящему */
}

/* Нас разбудили командой fg — снова забираем терминал под игру. */
static void on_sigcont(int sig)
{
```

```
    (void)sig;
    signal(SIGTSTP, on_sigtstp);   /* заново повесить наш обработчик */
    apply_raw_mode();
}

/* Включить «сырой» неблокирующий ввод: без ожидания Enter и без эха.
   Возвращает 1 при успехе, 0 — если ввод не терминал (пайп, файл). */
static int enable_raw_mode(void)
{
    if (tcgetattr(STDIN_FILENO, &orig_termios) == -1) {
        return 0;                              /* это не терминал */
    }
    raw_saved = 1;                             /* запомнили как было */
    apply_raw_mode();
    return 1;
}
```

Разбираем по строчкам:

- `static struct termios orig_termios;` — глобальная переменная под исходные настройки. Слово `static` тут значит «видна только в этом файле». `struct termios` — структура с настройками терминала (про структуры мы говорили в главе 11).
- `tcgetattr(STDIN_FILENO, &orig_termios)` — **прочитать** текущие настройки терминала и сложить в `orig_termios`. `&` — адрес: функция запишет настройки в нашу переменную (как `scanf` пишет по адресу). Возвращает `0` при успехе и `-1` при ошибке — и вот это возвращаемое значение проверять надо. Если стандартный ввод не терминал (программу запустили как `./snake < file.txt` или через пайп), у него просто нет настроек терминала: `tcgetattr` отдаст `-1`, и ничего полезного в `orig_termios` не окажется. Без проверки игра пошла бы дальше вслепую: `tcsetattr` на пайпе тоже не сработает, `read` за первый же кадр выгребет всё, что пришло по пайпу, а потом змейка поедет сама, и управлять ею будет нечем.
- `raw_saved` — флажок «в `orig_termios` лежит что-то настоящее». `disable_raw_mode` смотрит на него и не пытается ничего чинить, если чинить нечего.
- `struct termios raw = orig_termios;` — делаем **копию**. Исходник `orig_termios` бережём нетронутым, чтобы в конце восстановить.
- `raw.c_lflag &= ~(ICANON | ECHO);` — самая важная строка. `c_lflag` — поле «локальных флагов» терминала; в нём набор включённых режимов.
  - `ICANON` — флаг канонического режима (ожидание Enter).
  - `ECHO` — флаг эха (печать набранного на экран).

    - `ICANON | ECHO` — операцией `|` (побитовое ИЛИ) объединяем оба флага в один набор битов.
    - `~(...)` — побитовое НЕ: переворачивает биты, получая «всё, кроме этих двух флагов».
    - `&=` — присвоить результат побитового И. Вместе строка означает: «оставить все флаги как есть, но **снять** `ICANON` и `ECHO`». Эта связка `flag &= ~(БИТЫ)` — стандартный способ выключить биты.
- `raw.c_cc[VMIN] = 0; raw.c_cc[VTIME] = 0;` — массив `c_cc` хранит «спецсимволы» и параметры чтения. `VMIN = 0` и `VTIME = 0` вместе означают: «чтение возвращает управление **сразу**, даже если ничего не нажато». Это и есть неблокирующее чтение — без `fcntl`, штатными средствами termios.
- `tcsetattr(STDIN_FILENO, TCSANOW, &raw)` — **записать** новые настройки. `TCSANOW` — «применить немедленно».
- `disable_raw_mode` делает обратное: возвращает сохранённый `orig_termios`. Её **обязательно** вызвать в конце, иначе после игры терминал останется без эха и будет вести себя странно.

Зачем включение разбито на две функции — `apply_raw_mode` и `enable_raw_mode`? Сохранять исходные настройки надо ровно **один раз**, при старте. А вот включать игровой режим придётся ещё и потом: когда игру усыпят по Ctrl+Z и разбудят обратно, терминал надо будет забрать снова. Если бы обе задачи жили в одной функции, второй вызов затёр бы `orig_termios` уже сырыми настройками — и восстанавливать стало бы нечего.

В `main` поэтому проверяем результат и честно отказываемся играть, если клавиатуры нет:

```
if (!enable_raw_mode()) {
    fprintf(stderr, "Змейке нужен настоящий терминал: "
                    "запусти ./snake без пайпа и без < файла.\n");
    return 1;
}
```

`fprintf(stderr, ...)` — тот же `printf`, но пишет в **поток ошибок** `stderr`, а не в обычный вывод. Сообщения о проблемах принято отправлять туда: их видно на экране, но они не попадут в файл, если вывод программы куда-то перенаправили. Проверить это легко:

```
printf 'dddd' | ./snake
```

```
Змейке нужен настоящий терминал: запусти ./snake без пайпа и без < файла.
```

Страховка: терминал надо вернуть в любом случае

«Вызвать в конце» звучит просто, но конец бывает разный. Игра может закончиться нормально — тогда мы дойдём до `disable_raw_mode` сами. А может и не дойти: ты нажмёшь Ctrl+C, программа упадёт, ты закроешь её из другого окна. Терминал при этом останется в сыром режиме, и дальше в нём невозможно работать: печатаешь — букв не видно, жмёшь Enter — ничего.

Проверь сам: запусти змейку и нажми Ctrl+C. Пока в программе нет страховки, ты получишь ровно это (спасает команда `reset`, набранная вслепую). Первая подпорка — `atexit`:

```
atexit(disable_raw_mode);      /* вернуть режим при обычном выходе */
```

`atexit(функция)` из `<stdlib.h>` регистрирует функцию, которую система вызовет **при нормальном завершении программы** — хоть через `return` из `main`, хоть через `exit()`. Аргумент — имя функции без скобок: мы передаём саму функцию, а не результат её вызова. Теперь, даже если мы добавим где-то в середине `exit(1)`, терминал всё равно починится.

Но `atexit` не спасает от **сигналов**. Сигнал — это короткое уведомление, которое ядро посылает процессу; для большинства сигналов поведение по умолчанию — «умереть немедленно», мимо всех зарегистрированных функций. Нас касаются пять:

- `SIGINT` — ты нажал Ctrl+C;
- `SIGTERM` — кто-то сделал `kill` (так же процессы гасит система при выключении);
- `SIGQUIT` — Ctrl+\;
- `SIGHUP` — закрыли окно терминала;
- `SIGTSTP` — ты нажал Ctrl+Z, программу не убивают, а усыпляют.

Первые четыре кончаются смертью, и всем им подойдёт один обработчик:

```
static void on_fatal_signal(int sig)
{
    disable_raw_mode();
    _exit(128 + sig);    /* 128 + номер сигнала — так принято в шеллах */
}
```

- `int sig` — номер пришедшего сигнала; такова обязательная подпись обработчика. Здесь он даже пригодился: подставляем его в код выхода.
- `_exit(...)` из `<unistd.h>` завершает процесс **немедленно**. Почему не обычный `exit`? Внутри обработчика сигнала безопасно вызывать далеко не всё: сигнал может прилететь в любой момент, хоть посреди работы `printf`, и повторный заход в ту же функцию всё сломает. `_exit` — из короткого списка того, что вызывать можно (`tcsetattr` в нём тоже есть — иначе так починить терминал не вышло бы). Обратная сторона: `_exit` не

сбрасывает буфер `stdout`, поэтому последняя недопечатанная строка на экране может оборваться на полуслове. Для аварийного выхода это нормальная цена.

- `128 + sig` — подражаем правилу шеллов: если программу убил сигнал номер `N`, шелл показывает её код завершения (переменная `$?`) как `128 + N`. У `SIGINT` номер 2, значит, после Ctrl+C `echo $?` напечатает 130 — я проверил: Ctrl+C даёт 130, Ctrl+\ — 131, `kill` — 143 (у `SIGTERM` номер 15).

Вешаем его на все четыре сигнала сразу:

```
signal(SIGINT,  on_fatal_signal);   /* Ctrl+C   */
signal(SIGTERM, on_fatal_signal);   /* kill     */
signal(SIGQUIT, on_fatal_signal);   /* Ctrl+\   */
signal(SIGHUP,  on_fatal_signal);   /* окно закрыли */
```

`signal(НОМЕР, функция)` из `<signal.h>` говорит: «когда придёт вот этот сигнал, вызови вот эту функцию вместо поведения по умолчанию».

Ctrl+Z — отдельная история

`SIGTSTP` (Ctrl+Z) не убивает программу, а **усыпляет**: она замирает, а терминал возвращается шеллу, и ты снова в командной строке. Разбудить её потом можно командой `fg`. Проблема ровно та же: пока игра спит, терминал остаётся в сыром режиме — печатаешь и ничего не видишь.

Значит, на засыпание надо вернуть настройки, а на пробуждение — снова их забрать. Пробуждение тоже приходит сигналом, он называется `SIGCONT`:

```
static void on_sigtstp(int sig)
{
    (void)sig;
    disable_raw_mode();
    signal(SIGTSTP, SIG_DFL);   /* вернуть штатное поведение... */
    raise(SIGTSTP);             /* ...и остановиться по-настоящему */
}

static void on_sigcont(int sig)
{
    (void)sig;
    signal(SIGTSTP, on_sigtstp);   /* заново повесить наш обработчик */
    apply_raw_mode();
}
```

- `(void)sig;` — номер сигнала тут не нужен, но параметр убрать нельзя, а `-Wextra` ругнётся на неиспользованный. Эта строка — общепринятый способ сказать компилятору «я знаю про параметр и намеренно его не трогаю».
- Хитрость в `on_sigtstp`. Мы перехватили сигнал — значит, программа **не** уснула, обработчик просто отработал и вернул управление в игру. А уснуть-то надо. Поэтому: чиним терминал, ставим `SIGTSTP` обратно на поведение по умолчанию (`SIG_DFL` — «как было изначально») и `raise(SIGTSTP)` посылаем этот сигнал сами себе. Теперь его никто не перехватывает, и процесс честно засыпает.
- `on_sigcont` — зеркало: после `fg` возвращаем свой обработчик `SIGTSTP` на место (мы же его только что сняли) и снова включаем игровой режим через `apply_raw_mode`. Именно здесь и пригодилось разделение на две функции: `enable_raw_mode` тут звать нельзя, он бы перезаписал `orig_termios`.

Вешаем обе:

```
signal(SIGTSTP, on_sigtstp);        /* Ctrl+Z   */
signal(SIGCONT, on_sigcont);        /* fg       */
```

Полностью герметично это всё равно не получится. Есть сигнал `SIGKILL` (`kill -9`) — его перехватить нельзя в принципе, такова его роль. Если кто-то прибьёт игру через `kill -9`, терминал останется сырым, и спасёт только `reset`. Это не повод не ставить остальные подпорки: они закрывают все случаи, которые случаются в жизни.

Читаем клавишу

```
static int read_key(void)
{
    char c;
    ssize_t n = read(STDIN_FILENO, &c, 1);
    if (n == 1) {
        return (unsigned char)c;
    }
    return -1;
}
```

- `read(STDIN_FILENO, &c, 1)` — POSIX-функция чтения: «прочитать из стандартного ввода в переменную `c` максимум 1 байт». Возвращает, сколько байт реально прочла.
- `ssize_t` — целочисленный тип для результата `read` (может быть и отрицательным при ошибке). Считай его «специальным `int` для размеров».
- Если `n == 1` — клавишу нажали, возвращаем её код. Приведение `(unsigned char)` нужно, чтобы коды не ушли в отрицательные.

- Если нажатия не было, `read` вернёт 0 (мы же сделали неблокирующее чтение), и функция вернёт `-1` — «клавиши нет».

Теперь в `main` обернём всё: включим режим в начале, выключим в конце, а в цикле будем спрашивать `read_key`. Реверс (резкий разворот на 180°) запрещаем — иначе змейка мгновенно врежется в себя:

```
static int dir_x, dir_y;

static void set_dir(int nx, int ny)
{
    if (nx == -dir_x && ny == -dir_y) {
        return;   /* противоположное текущему — игнорируем */
    }
    dir_x = nx;
    dir_y = ny;
}
```

`set_dir` меняет направление, но если новое — точная противоположность текущему (`nx == -dir_x` и `ny == -dir_y`), просто выходит, ничего не меняя.

Выглядит убедительно. И всё-таки эта версия ломается — сейчас увидим, как. Но сначала посмотрим, откуда берётся поломка.

В игровом цикле читаем **все** накопившиеся нажатия за кадр (вдруг ты нажал несколько раз) и применяем их:

```
int key;
while ((key = read_key()) != -1) {
    if (key == 'w' || key == 'W') set_dir(0, -1);
    else if (key == 's' || key == 'S') set_dir(0, 1);
    else if (key == 'a' || key == 'A') set_dir(-1, 0);
    else if (key == 'd' || key == 'D') set_dir(1, 0);
    else if (key == 'q' || key == 'Q') game_over = 1;
}
```

- `while ((key = read_key()) != -1)` — присваиваем результат `read_key()` переменной `key` **и тут же** сравниваем с `-1`. Пока клавиши есть, крутимся. Опустошили буфер — `read_key` вернёт `-1`, цикл закончится. Скобки вокруг `key = read_key()` обязательны.
- Дальше обычный разбор: WASD меняют направление (учитываем и заглавные на случай Caps Lock), `q` — выход.

Где ломается защита от разворота

Вот здесь два куска кода и подставляют друг друга. Цикл выгребает за кадр **все** нажатия подряд, а `set_dir` сравнивает новое направление с `dir_x/dir_y` — то есть с результатом предыдущего нажатия, а не с тем, куда змейка реально ползёт.

Змейка едет вправо, `dir = (1, 0)`. Ты нервно тыкаешь W, а следом A (обе клавиши успевают попасть в один кадр):

1. `set_dir(0, -1)` — вверх. Проверка: `(0,-1)` против `-(1,0) = (-1,0)`. Не совпало, разрешено. Теперь `dir = (0, -1)`.
2. `set_dir(-1, 0)` — влево. Проверка: `(-1,0)` против `-(0,-1) = (0,1)`. Не совпало — тоже разрешено! `dir = (-1, 0)`.

Итог: за один кадр направление перевернулось с «вправо» на «влево» через промежуточное «вверх», и следующий шаг поведёт голову ровно в тот сегмент, который идёт сразу за ней. Мгновенный проигрыш, причём из-за нажатия, которое игрок считает законным.

Проверить это можно, не играя. Я взял функции из итоговой программы (её полный текст — в конце главы; `update` там уже умеет замечать столкновения, мы разберём это в шаге 5), выбросил терминал и написал вместо игрового цикла такой `main`:

```
int main(void)
{
    snake_len = 3;              /* голова в (10,5), тело левее */
    snake_x[0] = 10; snake_y[0] = 5;
    snake_x[1] = 9;  snake_y[1] = 5;
    snake_x[2] = 8;  snake_y[2] = 5;
    dir_x = 1;                  /* ползём вправо */
    dir_y = 0;
    moved_x = dir_x;
    moved_y = dir_y;
    food_x = 0;                 /* еда подальше, в углу */
    food_y = 0;

    /* два нажатия за один кадр: W, потом A */
    set_dir(0, -1);
    set_dir(-1, 0);
    printf("направление после двух нажатий: (%d,%d)\n", dir_x, dir_y);
    update();
    printf("после кадра: голова (%d,%d), game_over=%d\n",
           snake_x[0], snake_y[0], game_over);
```

```
    return 0;
}
```

Запуск со старым `set_dir` (тем, что сравнивает с `dir_x/dir_y`):

```
направление после двух нажатий: (-1,0)
после кадра: голова (10,5), game_over=1
```

Направление стало «влево», следующая клетка головы — `(9,5)`, а там лежит первый сегмент тела. `update` увидел столкновение и закончил игру, не сдвинув голову с `(10,5)`.

Чинится это одной идеей: сравнивать надо не с «накопленным» направлением, а с тем, **куда змейка реально поехала в прошлом кадре**. Заводим для этого вторую пару переменных:

```
static int dir_x, dir_y;       /* куда голова поедет в следующем кадре */
static int moved_x, moved_y;   /* куда она реально поехала в прошлом */

static void set_dir(int nx, int ny)
{
    if (nx == -moved_x && ny == -moved_y) {
        return;   /* противоположное реальному движению — игнорируем */
    }
    dir_x = nx;
    dir_y = ny;
}
```

А `update` в самом начале запоминает, с каким направлением работает этот кадр:

```
static void update(void)
{
    moved_x = dir_x;
    moved_y = dir_y;
    /* ... дальше сдвиг тела и шаг головы, как было ... */
}
```

Теперь второе нажатие в том же кадре меряется от `moved = (1,0)`, и `A` отсекается. Тот же `main` после правки:

```
направление после двух нажатий: (0,-1)
после кадра: голова (10,4), game_over=0
```

Змейка ушла вверх — ровно то, чего игрок и хотел. Не забудь в `main` задать `moved_x = dir_x; moved_y = dir_y;` вместе с начальным направлением, иначе первый же `set_dir` сравнится с парой нулей и пропустит что угодно.

Теперь змейкой можно рулить. Осталась еда, рост и проигрыш.

Шаг 4. Еда, рост и счёт

Еда `*` лежит в случайной клетке. Съел — змейка прибавляет сегмент, а счёт растёт. Заводим переменные `food_x, food_y, score`.

Случайные числа — `rand` из `<stdlib.h>` (глава 12). Кладём еду в свободную клетку:

```
static void place_food(void)
{
    int ok = 0;
    while (!ok) {
        food_x = rand() % WIDTH;
        food_y = rand() % HEIGHT;
        ok = 1;
        for (int i = 0; i < snake_len; i++) {
            if (snake_x[i] == food_x && snake_y[i] == food_y) {
                ok = 0;   /* попали в тело — выбираем заново */
                break;
            }
        }
    }
}
```

- `rand() % WIDTH` — остаток от деления случайного числа на ширину даёт число в диапазоне 0..WIDTH−1, то есть допустимый столбец. Аналогично для строки.
- Цикл `while (!ok)` повторяет выбор, пока еда не окажется в пустой клетке: если случайно попали в тело змейки, ставим `ok = 0` и бросаем кубик заново.

Здесь спрятана мина замедленного действия, и её стоит увидеть заранее. Цикл `while (!ok)` бросает кубик, пока не найдёт свободную клетку. А что, если свободных клеток **не осталось** — змейка выросла и заняла всё поле до последней клетки? Тогда `ok` никогда не станет единицей, и программа зависнет навсегда: экран замрёт, а процессор будет крутить пустой цикл на все сто процентов.

Случай редкий: чтобы до него дожить, надо съесть 197 порций еды подряд, ни разу не врезавшись. Но «редкий» и «невозможный» — разные вещи, а зависшая программа — худший

из возможных исходов. Проверить нехватку места нетрудно: если сегментов уже столько же, сколько клеток на поле, класть еду некуда.

```
static int place_food(void)
{
    if (snake_len >= WIDTH * HEIGHT) {
        food_x = -1;              /* свободных клеток нет: убираем еду */
        food_y = -1;              /* с поля, чтобы не закрыла голову */
        return 0;
    }
    int ok = 0;
    while (!ok) {
        food_x = rand() % WIDTH;
        food_y = rand() % HEIGHT;
        ok = 1;
        for (int i = 0; i < snake_len; i++) {
            if (snake_x[i] == food_x && snake_y[i] == food_y) {
                ok = 0;   /* попали в тело — выбираем заново */
                break;
            }
        }
    }
    return 1;
}
```

Функция стала возвращать `int`: `1` — еду положили, `0` — класть некуда. Тот, кто её зовёт, теперь может отличить одно от другого — и превратить безвыходную ситуацию в победу, а не в зависание. Ниже, в разборе `update`, мы так и сделаем.

Строчки `food_x = -1; food_y = -1;` убирают еду с поля: клеток с координатой −1 нет, и `draw` её нигде не нарисует. Без них на последнем, победном кадре еда осталась бы там, где её только что съели, — то есть под головой. А `draw` проверяет еду раньше змейки и нарисовал бы вместо головы `*`. Эту мелочь я поймал, когда прогнал логику змейки до полной победы на маленьком поле 6×4: без этих строк голова и еда на последнем кадре совпадали.

И ещё о честности жребия. Пока свободных клеток много, цикл почти всегда попадает с первого броска. Каждый бросок выбирает любую из `WIDTH * HEIGHT` = 200 клеток с равным шансом, а неудачные броски просто выбрасываются. Поэтому все свободные клетки получают еду с одинаковой вероятностью — тот же довод, что с минами в сапёре (глава 16). Цена растёт к концу игры: если свободно `k` клеток из 200, в среднем понадобится 200 / `k` бросков. При одной свобод-

ной клетке — около 200 бросков, и в каждом до 199 сравнений с телом: порядка 40 000 простых операций на кадр, для компьютера это мгновение.

Теперь рост. Хитрость в том, что при сдвиге тела последний сегмент (хвост) «теряется» — его координаты затираются. Если змейка съела еду, мы вернём этот хвост обратно, и она станет на сегмент длиннее. Поэтому **перед** сдвигом запоминаем хвост:

```
int tail_x = snake_x[snake_len - 1];
int tail_y = snake_y[snake_len - 1];

/* ...сдвиг тела и шаг головы как раньше... */

if (snake_x[0] == food_x && snake_y[0] == food_y) {
    if (snake_len < MAX_LEN) {
        snake_x[snake_len] = tail_x;   /* отрастили хвост обратно */
        snake_y[snake_len] = tail_y;
        snake_len++;
    }
    score += 10;
    if (!place_food()) {
        game_won = 1;                  /* поле заполнено целиком */
        game_over = 1;
    }
}
```

- `snake_len - 1` — индекс последнего сегмента (помни: нумерация с 0, поэтому последний — не `snake_len`, а на единицу меньше).
- Если голова попала на еду, добавляем новый сегмент в позицию бывшего хвоста: `snake_x[snake_len] = tail_x`, и увеличиваем длину.
- `if (snake_len < MAX_LEN)` — защита от выхода за границы массива: не даём длине превысить размер. `MAX_LEN` — максимум сегментов (все клетки поля).
- `score += 10` — за каждую еду 10 очков.
- `if (!place_food())` — а вот и обещанная развязка. Если положить еду некуда, значит, змейка заняла всё поле. Это не ошибка и не зависание, а самый настоящий выигрыш: поднимаем флаг `game_won` и заканчиваем партию. В конце `main` он превратится в отдельную поздравительную строчку.

Порядок здесь важен: сначала растим змейку, и только потом зовём `place_food`. Тогда новая еда точно не окажется под свежим сегментом — функция проверяет все `snake_len` клеток, включая только что добавленную.

В отрисовку добавляем еду перед проверкой змейки:

```
if (x == food_x && y == food_y) {
    ch = '*';
} else {
    /* ...поиск сегмента змейки, как раньше... */
}
```

Шаг 5. Столкновения и конец игры

Последний штрих — проигрыш. Игра кончается, если голова вышла за стену или наехала на собственное тело. Напрашивается: сделать шаг, а потом посмотреть, куда попали.

```
/* сначала шагнули... */
snake_x[0] += dir_x;
snake_y[0] += dir_y;

/* ...потом проверяем стену */
if (snake_x[0] < 0 || snake_x[0] >= WIDTH ||
    snake_y[0] < 0 || snake_y[0] >= HEIGHT) {
    game_over = 1;
    return;
}
```

Логика верная, игра действительно заканчивается. Но запусти, не трогай клавиши и дай змейке доехать до правой стены — вот последний кадр, который останется на экране:

```
######################
#                    #
#                    #
#             *      #
#                    #
#                    #
#                   oo#
#                    #
#                    #
#                    #
#                    #
######################
Счёт: 0   (WASD — движение, q — выход)
```

```
Игра окончена! Итоговый счёт: 0
```

Тело на месте, а головы нет. И понятно почему: `update` уже сдвинул голову в клетку `x = 20`, которой на поле не существует, а `draw` рисует только клетки `0..19` — голову просто некуда поставить. Игрок видит обезглавленную змейку и недоумевает, обо что он вообще ударился. Порчи памяти тут нет (в массив мы записали своё же поле `snake_x[0]`), но картинка врёт.

Чинится сменой порядка: сначала посчитаем, **куда голова собирается**, проверим это место — и только если оно хорошее, сделаем шаг.

```c
/* куда голова СОБИРАЕТСЯ шагнуть */
int nx = snake_x[0] + dir_x;
int ny = snake_y[0] + dir_y;

/* столкновение со стеной — проверяем ДО шага */
if (nx < 0 || nx >= WIDTH || ny < 0 || ny >= HEIGHT) {
    game_over = 1;
    return;
}

/* столкновение с собственным телом */
for (int i = 1; i < snake_len - 1; i++) {
    if (snake_x[i] == nx && snake_y[i] == ny) {
        game_over = 1;
        return;
    }
}
```

- `nx`, `ny` — «новая x», «новая y». Обычные локальные переменные: посчитали будущую клетку, но в змейку её пока не записали.
- Первое условие: будущая клетка левее 0, правее поля, выше 0 или ниже поля — это удар о стену. Раз шага не было, змейка осталась на поле целиком, и тот же прогон выглядит уже понятно — голова уткнулась в стенку:

```
######################
#                    #
# *                  #
#                    #
#                    #
#                    #
```

```
#                 oo0#
#                    #
#                    #
#                    #
#                    #
######################
Счёт: 0   (WASD — движение, q — выход)

Игра окончена! Итоговый счёт: 0
```

(Еда лежит в разных местах просто потому, что это два разных запуска — `srand` каждый раз даёт новое поле.)

- Цикл по телу начинается с `i = 1`: нулевой сегмент — сама голова, сравнивать её будущую клетку с её же текущей бессмысленно.
- А вот верхняя граница `i < snake_len - 1` — тонкость, которую легко проглядеть. Последний сегмент — хвост, и в этом же кадре он **уползёт** со своей клетки: тело вот-вот сдвинется по цепочке. Значит, наехать на него нельзя — к моменту, когда голова там окажется, клетка уже освободится. Если хвост учитывать, змейка будет ни за что проигрывать в тесных поворотах.
- `game_over = 1` поднимает флаг, `return` обрывает шаг: дальше ни тело двигать, ни еду проверять незачем.

После проверок идёт то, что мы уже разобрали: запомнили хвост, сдвинули тело, поставили голову в `(nx, ny)`, посмотрели на еду.

Главный цикл крутится, пока `!game_over`. Как только флаг поднят (или нажали `q`), выходим, **возвращаем терминал** и печатаем счёт. Собираем всё вместе.

Полная программа

Сохрани это как `code/17-snake.c`.

```c
/* Глава 17. Игра «Змейка» в терминале.
 *
 * Управление: W A S D — вверх/влево/вниз/вправо, q — выход.
 * Компиляция (macOS/Linux):
 *   cc -std=c11 -Wall -Wextra -O2 17-snake.c -o snake
 *
 * Программа интерактивная: запускать в обычном терминале, не в пайпе.
 */
```

```c
/* На Linux (glibc) со строгим -std=c11 объявление usleep спрятано за
   этим макросом. На macOS он ничего не меняет — но пусть стоит, чтобы
   файл собирался одинаково на обеих системах. Ставить ДО #include. */
#define _DEFAULT_SOURCE

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <time.h>
#include <termios.h>
#include <unistd.h>

/* ---- Размеры игрового поля (без учёта рамки) ---- */
#define WIDTH   20
#define HEIGHT  10
#define MAX_LEN (WIDTH * HEIGHT)   /* змейка не длиннее всех клеток поля */

/* ---- Состояние игры ---- */
static int snake_x[MAX_LEN];   /* координаты сегментов: [0] — голова */
static int snake_y[MAX_LEN];
static int snake_len;          /* сколько сегментов сейчас */
static int dir_x, dir_y;       /* куда голова поедет в следующем кадре */
static int moved_x, moved_y;   /* куда она реально поехала в прошлом кадре */
static int food_x, food_y;     /* где лежит еда */
static int score;              /* счёт */
static int game_over;          /* 0 — играем, 1 — конец */
static int game_won;           /* 1 — змейка заняла всё поле */

/* ---- Сырой режим терминала ---- */
static struct termios orig_termios;   /* сюда сохраним исходные настройки */
static int raw_saved;                 /* 1 — orig_termios заполнен */

/* Вернуть терминал в нормальный режим. */
static void disable_raw_mode(void)
{
    if (raw_saved) {
        tcsetattr(STDIN_FILENO, TCSANOW, &orig_termios);
```

```
    }
}

/* Применить игровой режим поверх сохранённых настроек. */
static void apply_raw_mode(void)
{
    struct termios raw = orig_termios;         /* копия для правки */

    raw.c_lflag &= ~(ICANON | ECHO);           /* снять канонику и эхо */
    raw.c_cc[VMIN]  = 0;                       /* read не ждёт символов */
    raw.c_cc[VTIME] = 0;                       /* и не ждёт по таймеру */

    tcsetattr(STDIN_FILENO, TCSANOW, &raw);    /* применить сейчас же */
}

/* Смертельный сигнал (Ctrl+C, kill, закрытие окна): чиним терминал и
   только потом умираем. Сигнал убивает процесс мимо atexit — поэтому
   и перехватываем. */
static void on_fatal_signal(int sig)
{
    disable_raw_mode();
    _exit(128 + sig);    /* 128 + номер сигнала — так принято в шеллах */
}

/* Ctrl+Z: отдать терминал шеллу в нормальном виде и честно уснуть. */
static void on_sigtstp(int sig)
{
    (void)sig;
    disable_raw_mode();
    signal(SIGTSTP, SIG_DFL);   /* вернуть штатное поведение... */
    raise(SIGTSTP);             /* ...и остановиться по-настоящему */
}

/* Нас разбудили командой fg — снова забираем терминал под игру. */
static void on_sigcont(int sig)
{
    (void)sig;
    signal(SIGTSTP, on_sigtstp);   /* заново повесить наш обработчик */
```

```
    apply_raw_mode();
}

/* Включить «сырой» неблокирующий ввод: без ожидания Enter и без эха.
   Возвращает 1 при успехе, 0 — если ввод не терминал (пайп, файл). */
static int enable_raw_mode(void)
{
    if (tcgetattr(STDIN_FILENO, &orig_termios) == -1) {
        return 0;                           /* это не терминал */
    }
    raw_saved = 1;                          /* запомнили как было */
    apply_raw_mode();
    return 1;
}

/* Прочитать одну клавишу. Если её нет — вернуть -1 (не ждём ввода). */
static int read_key(void)
{
    char c;
    ssize_t n = read(STDIN_FILENO, &c, 1);
    if (n == 1) {
        return (unsigned char)c;
    }
    return -1;
}

/* Положить еду в случайную свободную клетку.
   Возвращает 0, если свободных клеток не осталось: змейка заняла всё
   поле. Без этой проверки цикл ниже крутился бы вечно. */
static int place_food(void)
{
    if (snake_len >= WIDTH * HEIGHT) {
        food_x = -1;              /* свободных клеток нет: убираем еду */
        food_y = -1;              /* с поля, чтобы не закрыла голову */
        return 0;
    }
    int ok = 0;
    while (!ok) {
```

```
        food_x = rand() % WIDTH;
        food_y = rand() % HEIGHT;
        ok = 1;
        for (int i = 0; i < snake_len; i++) {
            if (snake_x[i] == food_x && snake_y[i] == food_y) {
                ok = 0;   /* попали в тело — выбираем заново */
                break;
            }
        }
    }
    return 1;
}

/* Нарисовать поле, змейку, еду и счёт. */
static void draw(void)
{
    printf("\033[H\033[2J");   /* курсор в угол + очистка экрана */

    /* верхняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');

    for (int y = 0; y < HEIGHT; y++) {
        putchar('#');                       /* левая стенка */
        for (int x = 0; x < WIDTH; x++) {
            char ch = ' ';
            if (x == food_x && y == food_y) {
                ch = '*';                   /* еда */
            } else {
                for (int i = 0; i < snake_len; i++) {
                    if (snake_x[i] == x && snake_y[i] == y) {
                        ch = (i == 0) ? 'O' : 'o';   /* голова / тело */
                        break;
                    }
                }
            }
```

```
            putchar(ch);
        }
        putchar('#');                       /* правая стенка */
        putchar('\n');
    }

    /* нижняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');

    printf("Счёт: %d   (WASD — движение, q — выход)\n", score);
}

/* Сменить направление, не разрешая разворот на 180 градусов.
   Сравниваем с тем, куда змейка ПОЕХАЛА в прошлом кадре (moved_x/moved_y),
   а не с уже накопленным dir_x/dir_y: иначе два нажатия за один кадр
   (вверх, потом влево, пока ползём вправо) сложились бы в разворот. */
static void set_dir(int nx, int ny)
{
    if (nx == -moved_x && ny == -moved_y) {
        return;   /* противоположное реальному движению — игнорируем */
    }
    dir_x = nx;
    dir_y = ny;
}

/* Один шаг змейки: сдвиг тела, новая голова, столкновения, еда. */
static void update(void)
{
    /* фиксируем направление этого кадра — на него будет смотреть set_dir */
    moved_x = dir_x;
    moved_y = dir_y;

    /* куда голова СОБИРАЕТСЯ шагнуть */
    int nx = snake_x[0] + dir_x;
    int ny = snake_y[0] + dir_y;
```

```
/* столкновение со стеной — проверяем ДО шага, чтобы на последнем
   кадре змейка осталась нарисованной целиком, а не без головы */
if (nx < 0 || nx >= WIDTH || ny < 0 || ny >= HEIGHT) {
    game_over = 1;
    return;
}

/* столкновение с собственным телом. Хвост (последний сегмент) в этом
   кадре уползёт, поэтому его не считаем — вот почему i < snake_len-1. */
for (int i = 1; i < snake_len - 1; i++) {
    if (snake_x[i] == nx && snake_y[i] == ny) {
        game_over = 1;
        return;
    }
}

/* запомним хвост — он пригодится, если змейка вырастет */
int tail_x = snake_x[snake_len - 1];
int tail_y = snake_y[snake_len - 1];

/* сдвигаем тело: каждый сегмент встаёт на место предыдущего */
for (int i = snake_len - 1; i > 0; i--) {
    snake_x[i] = snake_x[i - 1];
    snake_y[i] = snake_y[i - 1];
}

/* голова делает шаг в текущем направлении */
snake_x[0] = nx;
snake_y[0] = ny;

/* съели еду? */
if (nx == food_x && ny == food_y) {
    if (snake_len < MAX_LEN) {
        snake_x[snake_len] = tail_x;   /* отрастили хвост обратно */
        snake_y[snake_len] = tail_y;
        snake_len++;
    }
```

```
        score += 10;
        if (!place_food()) {
            game_won = 1;                   /* поле заполнено целиком */
            game_over = 1;
        }
    }
}

int main(void)
{
    srand((unsigned)time(NULL));   /* разный «случай» при каждом запуске */

    if (!enable_raw_mode()) {      /* переключаем терминал в игровой режим */
        fprintf(stderr, "Змейке нужен настоящий терминал: "
                        "запусти ./snake без пайпа и без < файла.\n");
        return 1;
    }
    atexit(disable_raw_mode);      /* вернуть режим при обычном выходе */
    signal(SIGINT,  on_fatal_signal);   /* Ctrl+C   */
    signal(SIGTERM, on_fatal_signal);   /* kill     */
    signal(SIGQUIT, on_fatal_signal);   /* Ctrl+\   */
    signal(SIGHUP,  on_fatal_signal);   /* окно закрыли */
    signal(SIGTSTP, on_sigtstp);        /* Ctrl+Z   */
    signal(SIGCONT, on_sigcont);        /* fg       */

    /* начальная змейка из 3 сегментов в центре, ползёт вправо */
    snake_len = 3;
    int cx = WIDTH / 2;
    int cy = HEIGHT / 2;
    snake_x[0] = cx;     snake_y[0] = cy;
    snake_x[1] = cx - 1; snake_y[1] = cy;
    snake_x[2] = cx - 2; snake_y[2] = cy;
    dir_x = 1;
    dir_y = 0;
    moved_x = dir_x;
    moved_y = dir_y;
    score = 0;
    game_over = 0;
```

```
    game_won = 0;

    place_food();
    draw();                         /* показываем стартовое поле сразу */

    while (!game_over) {
        /* считываем все накопившиеся нажатия */
        int key;
        while ((key = read_key()) != -1) {
            if (key == 'w' || key == 'W') set_dir(0, -1);
            else if (key == 's' || key == 'S') set_dir(0, 1);
            else if (key == 'a' || key == 'A') set_dir(-1, 0);
            else if (key == 'd' || key == 'D') set_dir(1, 0);
            else if (key == 'q' || key == 'Q') game_over = 1;
        }

        if (game_over) {
            break;              /* нажали q */
        }

        update();
        draw();

        usleep(150000);         /* пауза 0,15 с между кадрами */
    }

    disable_raw_mode();         /* ОБЯЗАТЕЛЬНО вернуть терминал как было */

    if (game_won) {
        printf("\nПоле заполнено целиком — ты прошёл игру!\n");
    }
    printf("\nИгра окончена! Итоговый счёт: %d\n", score);
    return 0;
}
```

Как это работает

Проследим один оборот игрового цикла, пока `game_over` равен 0:

1. **Ввод.** Внутренний `while` вытягивает из буфера все нажатые клавиши. WASD меняют `dir_x/dir_y` через `set_dir` (он сверяется с `moved_x/moved_y` — направлением прошлого кадра — и не пускает разворот), `q` поднимает `game_over`. Нет нажатий — `read_key` сразу отдаёт `-1`, и мы не зависаем.
2. **Шаг.** `update` фиксирует `moved_x/moved_y`, считает будущую клетку головы `(nx, ny)` и проверяет её на стену и на тело — при ударе ставит `game_over` и выходит, ничего не сдвинув. Если клетка хорошая, запоминает хвост, сдвигает тело по цепочке и переносит голову в `(nx, ny)`. Если там лежала еда — отращивает хвост обратно, плюсует счёт и кладёт новую еду.
3. **Отрисовка.** `draw` чистит экран `\033[H\033[2J` и заново рисует рамку, еду `*`, голову `O`, тело `o` и строку счёта.
4. **Пауза.** `usleep(150000)` ждёт 0,15 секунды — иначе кадры мелькали бы слишком быстро, чтобы что-то разглядеть.

Цикл повторяется, и из быстрой смены кадров рождается плавное движение. Как закончили — `disable_raw_mode` возвращает терминал в нормальный вид, и печатается итоговый счёт.

Глобальные переменные (`snake_x`, `dir_x` и прочие) вынесены наружу, чтобы все функции (`update`, `draw`, `place_food`) работали с одним и тем же состоянием, не передавая кучу аргументов. Для маленькой игры в один файл это нормально.

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 17-snake.c -o snake
./snake
```

Команда `-std=c11` задаёт стандарт языка, `-Wall -Wextra` включают все предупреждения (их быть не должно), `-O2` — оптимизацию. Благодаря `#define _DEFAULT_SOURCE` в начале файла никакие лишние флаги для POSIX не нужны — собирается чисто, без предупреждений.

Запусти `./snake` **в обычном окне терминала**. Через пайп змейка играться не будет и честно об этом скажет — мы это специально предусмотрели. Увидишь поле, змейку в центре и еду. Управляй клавишами W, A, S, D — Enter нажимать **не надо**, змейка поворачивает сразу. Наводи голову на `*`: змейка растёт, счёт растёт на 10. Врежешься в стену или в себя — игра закончится и покажет счёт. Выйти в любой момент — клавиша `q`.

Вот настоящий кадр из партии: одна еда уже съедена, змейка ползёт влево (голова `O` слева), следующая порция лежит в правом верхнем углу.

```
####################
#                   *#
#          Oooo      #
#                    #
#                    #
#                    #
#                    #
#                    #
#                    #
#                    #
#                    #
#                    #
####################
Счёт: 10   (WASD — движение, q — выход)
```

Кстати, полезная проверка на внимательность: по картинке всегда можно пересчитать счёт. Стартовая змейка — 3 сегмента, каждая еда даёт `+1` сегмент и `+10` очков. Здесь сегментов четыре (`Oooo`), значит, съедена ровно одна порция, значит, счёт — 10. Если у тебя в игре числа разошлись, ищи ошибку в `update`: скорее всего, счёт растёт не там, где растёт длина.

Проверяем, что терминал возвращается

Самое неприятное в этой главе — испорченный терминал, поэтому проверь все выходы явно, один за другим. После каждого пункта просто набери в терминале любую команду: если буквы видно и Enter работает — всё хорошо.

1. Обычный конец: врежься в стену. Или нажми `q`.
2. Ctrl+C прямо во время игры.
3. Ctrl+Z во время игры — игра замрёт и вернёт тебя в командную строку в нормальном режиме. Набери `fg` — игра проснётся и продолжится ровно с того места, снова забрав терминал под себя.
4. Из **другого** окна терминала: `kill НОМЕР` (номер процесса покажет `ps` или `pgrep snake`). Игра умрёт, терминал останется рабочим.

Единственный случай, который не чинится, — `kill -9`: этот сигнал перехватить нельзя. Если так вышло, набери вслепую `reset` и нажми Enter.

Частые ошибки

Забыл вернуть режим терминала. Если программа завершится (или упадёт) до `disable_raw_mode`, терминал останется в сыром режиме: ты печатаешь, а букв не видно, Enter не срабатывает как

обычно. Это не поломка навсегда — набери вслепую команду `reset` и нажми Enter, терминал починится. Чтобы такого не было, зови `disable_raw_mode` перед выходом и подстрахуйся `atexit` плюс обработчиками сигналов: одного явного вызова в конце `main` мало, потому что до конца `main` можно и не добраться.

Подстраховались только от Ctrl+C. Самая частая половинчатая починка: повесили обработчик на `SIGINT` и успокоились. А программу гасят не только с клавиатуры. Проверь сам — запусти игру, найди её номер (`pgrep snake`) и из другого окна сделай `kill HOMEP`: если `SIGTERM` не перехвачен, терминал останется сырым. То же самое с Ctrl+Z: игра уснёт, а настройки терминала останутся игровыми, и командная строка будет вести себя как сломанная. Закрывай все выходы сразу: `SIGINT`, `SIGTERM`, `SIGQUIT`, `SIGHUP` — на общий обработчик, `SIGTSTP`/`SIGCONT` — на пару «отдал терминал / забрал обратно».

Не проверил, что ввод вообще терминал. `tcgetattr` на пайпе или файле возвращает `-1`, и в структуре не оказывается ничего полезного. Если результат не проверить, игра запустится с вводом, которым нельзя управлять: всё, что пришло по пайпу, прочитается в первом кадре, и змейка уедет в стену сама. Проверяй возвращаемое значение и выходи с понятным сообщением.

Змейка на последнем кадре осталась без головы. Значит, проверка стены стоит **после** шага головы: голова уже уехала за поле, а `draw` рисует только клетки внутри рамки. Считай будущую клетку в отдельные `nx`, `ny`, проверяй их — и шагай только если проверка прошла.

Картинка мерцает или «лесенкой» уезжает вниз. Значит, ты не очищаешь экран перед отрисовкой. Каждый кадр должен начинаться с `printf("\033[H\033[2J")`. Если мерцание сильное, проверь, что не печатаешь лишних пустых строк между кадрами.

Выход за границу массива сегментов. Если растить змейку без проверки `if (snake_len < MAX_LEN)`, при заполнении всего поля длина вылезет за размер массивов `snake_x`/`snake_y` — порча памяти. Граница обязательна.

Поиск свободной клетки крутится вечно. `place_food` бросает кубик, пока не попадёт в пустую клетку. Когда пустых клеток не осталось, этот цикл уже не закончится никогда: программа замирает, а процессор уходит в потолок. Любой цикл «бросай кубик, пока не повезёт» надо сопровождать вопросом: «а бывает ли, что не повезёт никогда?» Если бывает — проверь это условие до цикла и верни отдельный ответ.

Змейка проигрывает в тесном повороте. Если в проверке на укус самой себя дойти циклом до последнего сегмента (`i < snake_len`, а не `i < snake_len - 1`), змейка будет «врезаться» в собственный хвост, который в этом же кадре и так уползёт. Хвост — единственный сегмент, на место которого шагнуть можно.

Змейка мгновенно проигрывает при развороте. Без защиты в `set_dir` нажатие в обратную сторону вгоняет голову в свой второй сегмент — и сразу «конец игры». Поэтому развороты на 180° запрещены. И сверяться при этом надо с `moved_x/moved_y` — направлением, в котором змейка реально сделала прошлый шаг. Если сравнивать с `dir_x/dir_y`, два быстрых нажатия за один кадр (W, потом A на движении вправо) проскочат проверку по очереди и всё равно сложатся в разворот.

Цикл сдвига тела «съедает» змейку. Если сдвигать сегменты от головы к хвосту (а не от хвоста к голове), координаты затрутся раньше времени, и тело схлопнется. Иди в цикле от `snake_len - 1` к `1`.

`implicit declaration of function 'usleep'` при `-std=c11`. Это сообщение ты увидишь на Linux с glibc: там объявление `usleep` спрятано за макросом. Добавь `#define _DEFAULT_SOURCE` в самом начале файла (до всех `#include`) или собирай с флагом `-D_DEFAULT_SOURCE`. На macOS такого предупреждения не будет — там `usleep` объявлен без условий, и файл соберётся даже без макроса. Не удивляйся, если у тебя и товарища на разных системах поведение разное: это не магия, а разные заголовки.

Упражнения

1. **Ускорение.** Сделай так, чтобы с каждой съеденной едой игра шла быстрее: заведи переменную задержки (например, начать с `150000`) и уменьшай её на немного при росте, но не ниже какого-то минимума, чтобы змейка не стала неуправляемой. Передавай это значение в `usleep`. *Что должно получиться:* если начать со `150000` и отнимать `5000` за каждую еду до минимума `60000`, то после 18-й порции ((150 000 − 60 000) / 5000 = 18) скорость перестанет расти: кадр будет идти 0,06 секунды вместо 0,15, то есть в 2,5 раза быстрее.

2. **Пауза.** Добавь клавишу `p` — пауза. По нажатию игра замирает (цикл крутится, но `update` не вызывается и рисуется надпись «ПАУЗА»), повторное нажатие `p` продолжает. *Что должно получиться:* пока пауза, змейка стоит, счёт не меняется, клавиши WASD не должны менять направление (иначе на паузе можно «накопить» разворот). После второго `p` змейка продолжает с той же клетки в прежнюю сторону. Флажок паузы удобно переключать так: `paused = !paused;`.

3. **Стены-телепорты.** Сделай режим, где змейка не разбивается о стену, а появляется с противоположной стороны. Подсказка: вместо проверки на проигрыш заверни будущие координаты по модулю — `nx = (nx + WIDTH) % WIDTH; ny = (ny + HEIGHT) % HEIGHT;`. Прибавка `+ WIDTH` тут не для красоты: в Си остаток от отрицательного числа отрицателен (`-1 % 20` даёт `-1`, а не `19`), и без неё змейка уедет в индекс `-1`. Проверь на всех четырёх стенах. *Что должно получиться:* решение ниже.

4. **Рекорд.** Запоминай лучший счёт за сессию и показывай его рядом с текущим. Усложнение: сохраняй рекорд в файл (`fopen`, `fprintf`, `fscanf`) и читай при старте, чтобы он жил между запусками. *Что должно получиться:* сыграй на 30 очков и выйди — в файле (например, `snake-record.txt`) лежит `30`. Следующий запуск показывает «Рекорд: 30», партия на 20 очков его не трогает, на 40 — переписывает. Если файла ещё нет, `fopen(..., "r")` вернёт `NULL` — это не ошибка, рекорд просто равен нулю.

5. **Две еды.** Добавь второй вид еды, которая даёт больше очков, но живёт на поле ограниченное число кадров и потом исчезает. *Что должно получиться:* например, `$` на 30 очков, живёт 40 кадров (40 · 0,15 = 6 секунд). Она не должна появляться ни на теле змейки, ни на обычной еде, а съеденная `$` тоже удлиняет змейку. Проверка на победу при этом не меняется: сегментов всё так же не больше `WIDTH * HEIGHT`.

6. **Размер поля.** Вынеси `WIDTH` и `HEIGHT` в аргументы командной строки (`int main(int argc, char *argv[])`), чтобы запускать `./snake 30 15`. Подумай, как тогда задать размеры массивов (подсказка: `malloc`). *Что должно получиться:* `./snake 30 15` рисует рамку шириной 32 символа и высотой 17 строк (поле плюс по клетке рамки с каждой стороны), массивы сегментов — на 30 · 15 = 450 элементов. Слишком маленькое поле (скажем, `./snake 2 2`) программа должна отклонить: стартовая змейка из трёх клеток в ряд туда не влезет. Не забудь `free` перед выходом.

Решение упражнения 3

В `update` вместо расчёта `nx`, `ny` и проверки стены пишем:

```
/* стены-телепорты: вышли за край — появились с другой стороны */
int nx = (snake_x[0] + dir_x + WIDTH) % WIDTH;
int ny = (snake_y[0] + dir_y + HEIGHT) % HEIGHT;
```

Проверка на укус самой себя остаётся как была. Разберём числа. Голова в `x = 19` (последний столбец) едет вправо: `(19 + 1 + 20) % 20 = 40 % 20 = 0` — вынырнула в столбце 0. Голова в `x = 0` едет влево: `(0 - 1 + 20) % 20 = 19 % 20 = 19` — последний столбец. Без `+ 20` получилось бы `-1 % 20 = -1`: в Си знак остатка совпадает со знаком делимого, и `%` здесь не математический остаток от деления (тот всегда от 0 до 19), а просто то, что осталось после деления с отбрасыванием дробной части.

Я проверил все четыре стены, подставив голову к краю и сделав один шаг `update`:

```
-1 % 20 = -1, (-1 + 20) % 20 = 19
(19,5) шаг (1,0) -> (0,5), game_over=0
(0,5) шаг (-1,0) -> (19,5), game_over=0
```

```
(10,0) шаг (0,-1) -> (10,9), game_over=0
(10,9) шаг (0,1) -> (10,0), game_over=0
```

Что мы получили

Ты собрал полноценную аркадную игру: поле, самодвижущаяся змейка, мгновенное управление без Enter, случайная еда, рост, счёт и честный проигрыш. По дороге разобрал по-настоящему важную тему — **управление терминалом**: сырой и неблокирующий ввод через `termios`, очистку экрана управляющими последовательностями и игровой цикл с паузой `usleep`.

И отдельно — о том, чему научила именно эта глава: **программа отвечает за то, что она оставит после себя**. Змейка меняет настройки чужого терминала — значит, обязана вернуть их назад, чем бы дело ни кончилось: обычным выходом, Ctrl+C, `kill` или Ctrl+Z. Отсюда и `atexit`, и обработчики сигналов, и проверка `tcgetattr`. Тот же принцип действует всюду, где программа берёт что-то общее: открытый файл, выделенную память, запись в базе.

Это тот самый набор, на котором держатся все текстовые игры — от «Тетриса» до рогаликов.

И на этом наша часть про игры завершается. Оглянись: ты прошёл путь от первого `printf("Привет, мир!")` до интерактивной игры в реальном времени. Между ними — переменные и циклы, массивы и указатели, строки и структуры, рекурсия, сортировки и поиск, двумерные поля, случайные числа и работа с терминалом. Этого фундамента хватит, чтобы браться за свои проекты: придумай игру или инструмент, разбей на маленькие шаги — ровно как мы делали в каждой главе — и собирай по кусочку. Дальше дорогу выбираешь ты.

Глава 18. Что дальше

Если ты дошёл до этой страницы, набирая и запуская программы по дороге, — ты умеешь программировать на Си. Не «слышал про», а умеешь: объявить переменную и понимать, где она лежит, перебрать массив, написать функцию, передать в неё адрес, выделить память и вернуть её, отсортировать, найти бинарным поиском, развернуть список и собрать игру с вводом без Enter.

Но книга кончилась не потому, что Си кончился. Мы брали ровно тот инструмент, который нужен алгоритмам и играм, и намеренно оставили за бортом целые куски ремесла. Эта глава — про них. Не список ссылок «что почитать», а четыре реальных шага, которые можно сделать сегодня вечером: разбить программу на файлы, собрать её через `make`, залезть в неё отладчиком и научиться читать и писать файлы.

Что мы сделаем

- Честно разберём, что ты умеешь и чего пока нет.
- Возьмём одну маленькую программу и разрежем её на `.c` и `.h` — со всеми ошибками, которые при этом вылезают.
- Напишем для неё `Makefile` и увидим, как он пересобирает только то, что изменилось.
- Найдём в этой программе баг отладчиком `lldb`, а не через `printf`.
- Напишем программу, которая пишет и читает текстовый файл (`code/18-files.c`).
- Посмотрим на стандарты C17/C23 и на то, куда идти дальше.

Что ты теперь умеешь

Язык. Типы, `printf`/`scanf`, ветвления, циклы, свои функции (главы 1–2). Массивы, строки как `char`-массивы с `'\0'`, указатели, адресная арифметика, `malloc`/`free` (глава 3). `struct` и `typedef` (глава 11), `enum` (глава 13), указатель на функцию как компаратор для `qsort` (глава 8), `static` и `termios` (глава 17).

Алгоритмы. Цифры числа и переполнение `int` (глава 4), Евклид и решето Эратосфена (глава 5), обход массива (глава 6), линейный и бинарный поиск с честным замером времени (глава 7), три квадратичные сортировки и O-нотация (глава 8), рекурсия и почему наивные числа Фибоначчи катастрофически медленные (глава 9), строковые алгоритмы (глава 10), растущий массив, стек, очередь и односвязный список — руками, на `malloc` (глава 11).

Программы целиком. Шесть игр (главы 12–17): игровой цикл, состояние на двумерной сетке, рекурсивная заливка в «Сапёре», реальное время в «Змейке». В папке `code/` к этой главе накопилось полсотни `.c`-файлов, и каждый собирается одной командой.

Главное, что осталось, — не список тем, а способ работы: разбить задачу на шаги, написать наивную версию, запустить, найти проблему, починить.

Чего ты пока не умеешь

Тоже честно. Ни одного из этих пунктов в книге не было:

- **Проект из нескольких файлов.** Каждая программа книги — один `.c`. Как разложить код по модулям и связать их заголовками — ниже, шаг 1.
- **Система сборки.** Мы каждый раз набирали `cc ... -o ...` руками. На пяти файлах так уже не живут — шаг 2.
- **Отладчик.** Мы искали ошибки глазами и `printf`. Есть инструмент лучше — шаг 3.
- **Файлы.** `fopen`, `fgets`, `fprintf`, `fclose` — ни одному нашему алгоритму они не понадобились, а в жизни нужны постоянно — шаг 4.
- **Деревья и хеш-таблицы.** Стек, очередь и список мы написали, а двоичное дерево поиска и хеш-таблицу — нет. Хеш-таблица с цепочками есть в приложении B (задача 7), но как отдельная тема она не разбиралась.
- **Свежие стандарты.** Мы всю книгу писали на C11. C17 и C23 — в конце главы.
- **Кросс-платформенность.** `termios` из «Змейки» — это POSIX: macOS и Linux. На Windows тот же код не соберётся.

Шаг 1. Разбиваем программу на файлы

Задача-игрушка: посчитать сумму и среднее оценок. Логика подсчёта — отдельно, вывод — отдельно. Создай пустой каталог и в нём два файла.

`stats.c` — только вычисления, никакого `printf`:

```
int sum(const int *a, int n)
{
    int s = 0;
```

```
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}

double average(const int *a, int n)
{
    if (n == 0)
        return 0.0;
    int s = sum(a, n);
    return s / n;
}
```

`main.c` — только ввод-вывод:

```
#include <stdio.h>

int main(void)
{
    int marks[] = {5, 4, 5, 3, 4};
    int n = (int) (sizeof marks / sizeof marks[0]);

    printf("Сумма:   %d\n", sum(marks, n));
    printf("Среднее: %.2f\n", average(marks, n));
    return 0;
}
```

`sizeof marks / sizeof marks[0]` — размер всего массива в байтах, делённый на размер одного элемента, то есть число элементов: 20 / 4 = 5. Работает только там, где `marks` — настоящий массив, а не указатель (каверза 2 приложения Б). Собираем как привыкли — по одному файлу:

```
$ cc -std=c11 -Wall -Wextra main.c -o prog
main.c:8:34: error: call to undeclared function 'sum'; ISO C99 and
later do not support implicit function declarations
[-Wimplicit-function-declaration]
    8 |     printf("Сумма:   %d\n", sum(marks, n));
      |                                  ^
main.c:9:38: error: call to undeclared function 'average'; ISO C99 and
later do not support implicit function declarations
[-Wimplicit-function-declaration]
```

```
    9 |     printf("Среднее: %.2f\n", average(marks, n));
      |                                ^
main.c:9:38: warning: format specifies type 'double' but the argument
has type 'int' [-Wformat]
    9 |     printf("Среднее: %.2f\n", average(marks, n));
      |                       ~~~~     ^~~~~~~~~~~~~~~~
      |                       %.2d
1 warning and 2 errors generated.
```

(Строки с `error:` и `warning:` в терминале длинные и в одну строку — здесь я их перенёс, чтобы влезли в страницу, как договаривались в главе 1.)

Компилятор смотрит на **один файл за раз** и про `stats.c` ничего не знает. Он видит вызов `sum(...)` и не знает ни сколько аргументов, ни какой тип возвращается. Вызывать функцию, которую не объявили, стандарт C99 и новее запрещает — отсюда две ошибки. А третья жалоба, про `%.2f`, — последствие: чтобы продолжить разбор, clang по старому правилу C89 считает, что необъявленная функция возвращает `int`, и видит `int` там, где формат ждёт `double`.

Лечится **объявлением** (прототипом) — это заголовок функции без тела, с точкой с запятой на конце: он ничего не создаёт, он обещает, что такая функция где-то есть. Добавь в начало `main.c`:

```c
int sum(const int *a, int n);
double average(const int *a, int n);
```

```
$ cc -std=c11 -Wall -Wextra main.c -o prog
Undefined symbols for architecture x86_64:
  "_average", referenced from:
      _main in main-408d4b.o
  "_sum", referenced from:
      _main in main-408d4b.o
ld: symbol(s) not found for architecture x86_64
clang: error: linker command failed with exit code 1 (use -v to see invocation)
```

Ошибка сменилась — и это прогресс. Теперь ругается не компилятор, а **компоновщик** (linker) — программа, которая склеивает откомпилированные куски в один исполняемый файл. Он ищет тела функций `sum` и `average` и не находит: мы ему `stats.c` не дали. Подчёркивание перед именем (`_sum`) добавляет сам компилятор (так принято на macOS; на Linux подчёркивания не будет), а `main-408d4b.o` — временный файл, его случайный номер у тебя будет свой. `ld` в предпоследней строке — это и есть компоновщик, так его зовут в системе. Даём оба файла:

```
$ cc -std=c11 -Wall -Wextra main.c stats.c -o prog
$ ./prog
Сумма:   21
Среднее: 4.00
```

Собралось и запустилось. (Среднее неправильное — 4.00 вместо 4.20. Это не опечатка, мы вернёмся к этому на шаге 3.)

Заголовочный файл

Прототипы в `main.c` работают, но это ловушка: поменяешь `sum` в `stats.c` — про копию прототипа в `main.c` никто не вспомнит, а когда файлов станет пять, копий станет пять. Поэтому прототипы выносят в **заголовочный файл** (header) с расширением `.h`, а `#include` вставляет его текст куда попросят. `stats.h`:

```c
#ifndef STATS_H
#define STATS_H

/* Сумма n элементов массива a. */
int sum(const int *a, int n);

/* Среднее арифметическое; при n == 0 возвращает 0.0. */
double average(const int *a, int n);

#endif
```

Три строчки `#ifndef` / `#define` / `#endif` — **страж включения** (include guard). `#ifndef STATS_H` читается как «если имя `STATS_H` ещё не определено». Первый `#include "stats.h"` определяет его и пропускает содержимое дальше; второй и десятый видят, что `STATS_H` уже есть, и выбрасывают весь файл. Без стража при двойном включении получишь «переопределение» на каждый `struct` в заголовке. Имя внутри берут из имени файла, чтобы не совпало с чужим.

В `main.c` вместо двух прототипов пишем `#include "stats.h"`. Угловые скобки `<stdio.h>` означают «ищи среди системных заголовков», кавычки `"stats.h"` — «сначала поищи рядом с моим файлом, потом среди системных»; свои заголовки всегда в кавычках. И — это важно — та же строчка идёт первой **в самом** `stats.c`.

Зачем включать заголовок в файл, где эти функции и написаны? Чтобы компилятор сверил объявление с определением. Поменяй в `stats.c` тип `sum` с `int` на `long`, заголовок не трогай, — и ошибка вылезет сразу:

```
$ cc -std=c11 -Wall -Wextra main.c stats.c -o prog
stats.c:3:6: error: conflicting types for 'sum'
    3 | long sum(const int *a, int n)
      |      ^
./stats.h:5:5: note: previous declaration is here
    5 | int sum(const int *a, int n);
      |     ^
1 error generated.
```

Без `#include "stats.h"` в `stats.c` это разошлось бы молча: `main.c` по-прежнему думает, что `sum` возвращает `int`. Формально это неопределённое поведение (undefined behavior, UB) — стандарт не обещает вообще ничего. С `long` на этой машине случайно печатается верное `21`, и от этого только хуже: ошибка спит. А поменяй тип на `double` — и тот же самый запуск даёт `Сумма: 5` вместо 21. Строка `#include "stats.h"` в `stats.c` превращает такую тихую поломку в ошибку компиляции.

Раздельная компиляция

`cc main.c stats.c -o prog` каждый раз компилирует оба файла заново. Флаг `-c` (compile only) останавливает компилятор сразу после перевода в машинный код: получается **объектный файл** `.o` — кусок программы, ещё не программа. Собрать их в исполняемый файл можно потом, отдельно:

```
$ cc -std=c11 -Wall -Wextra -c main.c
$ cc -std=c11 -Wall -Wextra -c stats.c
$ cc main.o stats.o -o prog
$ ./prog
Сумма:   21
Среднее: 4.00
```

Поменял только `stats.c` — пересобирать `main.o` не нужно. На двух файлах экономия смешная, на двухстах — разница между «пять секунд» и «пять минут». Но следить руками, что устарело, невозможно. За этим и нужен `make`.

Шаг 2. Makefile

`make` — программа, которая читает файл с правилами и пересобирает только устаревшее. Правило выглядит так:

```
цель: от_чего_зависит
```

```
<TAB>команда
```

Отступ перед командой — **настоящий символ табуляции**, не пробелы. Это первая ошибка каждого новичка: `make` скажет `missing separator`. Положи рядом со своими файлами `Makefile` (именно так, с большой буквы, без расширения):

```
CC     = cc
CFLAGS = -std=c11 -Wall -Wextra -g
OBJ    = main.o stats.o

prog: $(OBJ)
    $(CC) $(OBJ) -o prog

main.o: main.c stats.h
    $(CC) $(CFLAGS) -c main.c

stats.o: stats.c stats.h
    $(CC) $(CFLAGS) -c stats.c

clean:
    rm -f prog $(OBJ)
```

Разбор:

- `CC`, `CFLAGS`, `OBJ` — переменные, обращение к ним `$(CC)`, `$(CFLAGS)`. Смысл тот же, что у именованных констант в Си: флаги записаны в одном месте.
- `-g` в `CFLAGS` — «положи в программу отладочную информацию»: имена переменных и номера строк. Без него отладчик покажет голый ассемблер. При разработке держат всегда, на готовой программе убирают.
- `prog: $(OBJ)` — цель `prog` зависит от `main.o` и `stats.o`. Если хоть один из них новее, чем `prog`, — выполняем команду ниже.
- `main.o: main.c stats.h` — вот ради чего всё: `main.o` зависит не только от своего `.c`, но и от **заголовка**. Правишь `stats.h` — пересобираются все, кто его включает.
- `clean` — цель без зависимостей, просто «выполни команду». Файла с именем `clean` нет, поэтому `make clean` работает всегда.

Первая цель в файле — цель по умолчанию, поэтому `make` без аргументов собирает `prog`:

```
$ make
cc -std=c11 -Wall -Wextra -g -c main.c
cc -std=c11 -Wall -Wextra -g -c stats.c
cc main.o stats.o -o prog
```

```
$ ./prog
Сумма:   21
Среднее: 4.00
```

make печатает каждую команду перед запуском — видно, что именно он делает. Теперь запусти второй раз, ничего не меняя:

```
$ make
make: `prog' is up to date.
```

Ни одной команды: всё свежее. (Кавычки вокруг prog разные — так печатает старый GNU Make 3.81, который стоит на macOS; make поновее на Linux пишет 'prog'.) Меняем только stats.c и снова make:

```
$ make
cc -std=c11 -Wall -Wextra -g -c stats.c
cc main.o stats.o -o prog
```

main.c не трогали — main.o не пересобирался. А теперь меняем заголовок stats.h:

```
$ make
cc -std=c11 -Wall -Wextra -g -c main.c
cc -std=c11 -Wall -Wextra -g -c stats.c
cc main.o stats.o -o prog
```

Пересобралось всё — потому что мы честно прописали зависимость от stats.h в обоих правилах. («Меняем» можно и понарошку: команда touch stats.h просто обновляет время изменения файла, а make смотрит именно на время.) А make clean печатает rm -f prog main.o stats.o и убирает мусор.

Этого «Makefile на десять строк» хватает на годы. Дальше бывают шаблонные правила (%.o: %.c), автоматические переменные ($@, $<) и генерация зависимостей флагом -MMD — но это уже когда прижмёт.

Шаг 3. Отладчик: ищем баг в average

Пора разобраться с 4.00. Сумма 21, оценок 5, значит 4.2 — а печатается 4.00. printf виноват? Тип не тот? Вместо гаданий запустим программу под отладчиком.

Отладчик (debugger) умеет остановить программу на нужной строке и показать значения переменных прямо в этот момент. На macOS штатный отладчик — lldb, на Linux — gdb; команды у них разные, идея одна. Программа уже собрана с -g, так что можно начинать. Строки (lldb) — приглашение отладчика, после него набираешь команду. Три самые длинные строки ответа

я перенёс, чтобы влезли в страницу (продолжение начинается с отступа), у тебя каждая из них — одна строка:

```
$ lldb ./prog
(lldb) target create "./prog"
Current executable set to '/Users/me/proj/prog' (x86_64).
(lldb) b stats.c:16
Breakpoint 1: where = prog`average + 46 at stats.c:16:12,
    address = 0x00000001000005de
(lldb) run
Сумма:   21
Process 68496 launched: '/Users/me/proj/prog' (x86_64)
Process 68496 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = breakpoint 1.1
    frame #0: 0x00000001000005de prog`average(a=0x00007ff7bfefe620,
        n=5) at stats.c:16:12
   13       if (n == 0)
   14           return 0.0;
   15       int s = sum(a, n);
-> 16       return s / n;
                  ^
   17   }
Target 0: (prog) stopped.
(lldb) p s
(int) 21
(lldb) p n
(int) 5
(lldb) p s / n
(int) 4
(lldb) p (double) s / n
(double) 4.2000000000000002
(lldb) bt
* thread #1, queue = 'com.apple.main-thread', stop reason = breakpoint 1.1
  * frame #0: 0x00000001000005de prog`average(a=0x00007ff7bfefe620,
        n=5) at stats.c:16:12
    frame #1: 0x000000010000052b prog`main at main.c:10:38
    frame #2: 0x00007ff804aa4b28 dyld`start + 3240
(lldb) c
Среднее: 4.00
```

```
Process 68496 resuming
Process 68496 exited with status = 0 (0x00000000)
(lldb) quit
```

Номер процесса и адреса у тебя будут свои — это нормально. Что тут произошло:

- `b stats.c:16` — `b` от **breakpoint**, точка останова: «останови программу перед выполнением строки 16 файла `stats.c`». Можно и по имени функции: `b average`.
- `run` — запуск. Программа успела напечатать «Сумма: 21» и замерла на нашей строке. (Строка «Сумма» появилась даже раньше, чем отладчик сообщил о запуске: программа и `lldb` пишут в один терминал, и их сообщения перемешиваются.) Отладчик показал кусок исходника и стрелкой `->` отметил строку, которая **ещё не выполнена**.
- `p s` — от **print**: напечатать выражение. Ответ `(int) 21` — отладчик показывает и тип, и значение. Печатать можно не только переменные: `p s / n` даёт `(int) 4`. Вот и баг. Оба операнда — `int`, значит деление целочисленное, дробная часть выброшена **до** того, как результат стал `double` (главы 2 и 4; в чужой функции такое глазами ловится плохо). `p (double) s / n` проверяет гипотезу, не трогая код: `4.2000000000000002`. Хвост `...0002` — не ошибка отладчика: число 4,2 в двоичном виде записывается бесконечной дробью, и `double` хранит ближайшее к нему значение. `printf("%.2f")` его округлит до `4.20`.
- `bt` — от **backtrace**, стек вызовов: кто кого позвал. Читается снизу вверх: система запустила `start`, тот вызвал `main` (строка 10 файла `main.c`), а `main` вызвал `average`. Когда программа падает, `bt` — первое, что надо набрать.
- `c` — от **continue**, «беги дальше»; `quit` — выйти.

Ещё три команды, которые понадобятся сразу: `n` (**next**) — выполнить одну строку и снова остановиться; `s` (**step**) — то же, но зайти внутрь вызванной функции; `finish` — доработать текущую функцию до конца и остановиться у того, кто её вызвал.

Чиним `stats.c`:

```c
double average(const int *a, int n)
{
    if (n == 0)
        return 0.0;
    int s = sum(a, n);
    return (double) s / n;
}
```

`(double) s` — приведение типа: «считай `s` вещественным». Приведение сильнее деления, поэтому выражение читается как `((double) s) / n`, а не `(double) (s / n)` — второе дало бы те же 4.00. Достаточно привести **один** операнд, второй Си повысит сам.

```
$ make
cc -std=c11 -Wall -Wextra -g -c stats.c
cc main.o stats.o -o prog
$ ./prog
Сумма:   21
Среднее: 4.20
```

Обрати внимание: `make` пересобрал только `stats.o`. Всё сходится.

Шаг 4. Файлы: `fopen`, `fgets`, `fclose`

Пока всё, что печатали наши программы, исчезало при закрытии терминала. Файл — это способ сохранить.

Работа с файлом всегда идёт по одной схеме: **открыть — работать — закрыть**. Открывает `fopen` из `<stdio.h>`:

```
FILE *fopen(const char *path, const char *mode);
```

`path` — имя файла, `mode` — строка режима: `"r"` — читать (файла нет → ошибка), `"w"` — писать (файл создаётся, а существующий **обнуляется**), `"a"` — дописывать в конец. Возвращает `FILE *` — указатель на служебную структуру, которую дальше передают всем остальным функциям. Если открыть не удалось, возвращает `NULL`.

Пишут в файл функцией `fprintf` — это `printf`, которому первым аргументом сказали, **куда** печатать; остальное те же `%d`, `%s`, `\n`:

```
FILE *out = fopen("scores.txt", "w");
fprintf(out, "anna 120\n");
fclose(out);
```

(Здесь для краткости нет проверки `fopen` на `NULL` — в итоговой программе ниже она будет, и чуть дальше ты увидишь, почему без неё нельзя.)

`fclose(out)` закрывает файл и, что важнее, дописывает на диск то, что ещё лежало в буфере: `fprintf` сначала копит текст в памяти и отдаёт его системе крупными кусками. Если забыть `fclose`, но нормально выйти из `main`, библиотека закроет файлы сама. А вот если программа упадёт раньше, содержимое буфера пропадёт: я проверил — после `fprintf` и падения файл на диске оказался пустым, 0 байт. Вдобавок открытых файлов у программы может быть не больше, чем разрешает система (на этом маке по умолчанию 256), и в долгой программе незакрытые файлы кончаются.

Читать начнём с готового файла — сделаем его прямо в терминале, чтобы не отвлекаться:

```
$ printf 'anna 120\nboris 95\nvika 143\n' > scores.txt
$ cat scores.txt
anna 120
boris 95
vika 143
```

Наивная читалка, `read1.c`:

```c
#include <stdio.h>

int main(void)
{
    FILE *f = fopen("scores.txt", "r");
    char line[128];

    while (fgets(line, sizeof line, f) != NULL)
        printf("прочитал: %s", line);

    fclose(f);
    return 0;
}
```

`fgets(buf, size, f)` — тот самый `fgets`, которым мы всю книгу читали строки с клавиатуры, только последним аргументом теперь не `stdin`, а наш файл. Читает до перевода строки включительно, но не больше `size - 1` символов, дописывает `'\0'` и возвращает `buf`. Если файл кончился раньше, чем удалось прочитать хоть один символ, или случилась ошибка чтения, возвращает `NULL`.

```
$ cc -std=c11 -Wall -Wextra -O2 read1.c -o read1
$ ./read1
прочитал: anna 120
прочитал: boris 95
прочитал: vika 143
```

Работает. Теперь удали файл (`rm scores.txt`) и запусти снова:

```
$ ./read1
zsh: segmentation fault  ./read1
$ echo $?
139
```

Ни строчки от самой программы, только сообщение оболочки о падении, и код возврата 139 (`$?` — переменная оболочки, в ней код возврата последней команды). Программа упала: 139 — это 128 + 11, а сигнал 11 — `SIGSEGV`, обращение по неверному адресу. Причина: `fopen` вернул `NULL`, и `fgets` полез читать по нулевому адресу. **Результат `fopen` проверяют всегда** — это не паранойя, файла может не быть, к нему может не быть прав, диск может быть переполнен.

Вторая ловушка видна, если вернуть файл на место, добавить проверку `fopen` и напечатать строку в квадратных скобках — `printf("строка: [%s]\n", line)`:

```
$ ./read2
строка: [anna 120
]
строка: [boris 95
]
строка: [vika 143
]
```

`fgets` оставляет в буфере **символ перевода строки**. Если строку дальше сравнивать или склеивать, этот `'\n'` всё испортит. Срезают его так:

```
size_t len = strlen(line);
if (len > 0 && line[len - 1] == '\n')
    line[len - 1] = '\0';
```

Проверка `len > 0` обязательна: без неё на пустом буфере `line[-1]` — выход за границу. Пустым буфер бывает, если строка файла начинается с нулевого байта: `fgets` его прочитает, а `strlen` остановится на нём и вернёт 0. В обычном текстовом файле такого нет, а в чужом — бывает что угодно.

Итоговая программа — `code/18-files.c`. Она пишет файл, читает его обратно, разбирает каждую строку на имя и число и считает сумму:

```
/* 18-files.c — запись и чтение текстового файла.
   Пишем таблицу рекордов, читаем её обратно построчно и считаем сумму.
   Каждый вызов, который может провалиться, проверяем. */
#include <stdio.h>
#include <string.h>

int main(void)
{
    const char *path = "scores.txt";
```

```
    /* --- 1. Пишем файл --- */
    FILE *out = fopen(path, "w");
    if (out == NULL) {
        printf("Не могу открыть %s на запись\n", path);
        return 1;
    }

    fprintf(out, "anna 120\n");
    fprintf(out, "boris 95\n");
    fprintf(out, "vika 143\n");

    if (fclose(out) != 0) {
        printf("Ошибка при закрытии %s\n", path);
        return 1;
    }
    printf("Записал 3 строки в %s\n", path);

    /* --- 2. Читаем файл обратно --- */
    FILE *in = fopen(path, "r");
    if (in == NULL) {
        printf("Не могу открыть %s на чтение\n", path);
        return 1;
    }

    char line[128];
    int players = 0;
    int total = 0;

    while (fgets(line, sizeof line, in) != NULL) {
        size_t len = strlen(line);
        if (len > 0 && line[len - 1] == '\n')
            line[len - 1] = '\0';       /* срезаем перевод строки */

        char name[64];
        int score;
        if (sscanf(line, "%63s %d", name, &score) == 2) {
            players++;
            total += score;
```

```
            printf("%d. %s — очков: %d\n", players, name, score);
        } else {
            printf("не разобрал строку: \"%s\"\n", line);
        }
    }

    if (ferror(in))
        printf("Чтение оборвалось на ошибке\n");
    fclose(in);

    if (players > 0)
        printf("Игроков: %d, всего очков: %d\n", players, total);

    remove(path);                       /* убираем за собой */
    return 0;
}
```

Что нового:

- `sscanf(line, "%63s %d", name, &score)` — `scanf`, который читает не с клавиатуры, а из строки в памяти. Возвращает число успешно разобранных значений, поэтому `== 2` значит «нашлись и имя, и число». `%63s` с числом — ограничитель длины: не больше 63 символов плюс `'\0'` в буфер на 64. Без числа `%s` радостно вылезет за границу массива на длинном имени.
- `ferror(in)` — «чтение закончилось из-за ошибки, а не из-за конца файла?». `fgets` в обоих случаях возвращает `NULL`, и различить их можно только так (или через `feof`).
- `remove(path)` — удалить файл. Здесь он нужен, чтобы программа убирала за собой и её можно было запускать сколько угодно раз. Закомментируй эту строку, если хочешь посмотреть на `scores.txt` своими глазами. И учти: режим `"w"` затирает файл, а `remove` удаляет его — если в текущем каталоге лежит твой собственный `scores.txt` (например, от `read1`), программа его уничтожит.
- `"очков: %d"`, а не `"%d очков"` — чтобы не мучиться с падежами: «120 очков», но «143 очка» и «121 очко».

```
$ cc -std=c11 -Wall -Wextra -O2 code/18-files.c -o files
$ ./files
Записал 3 строки в scores.txt
1. anna — очков: 120
2. boris — очков: 95
```

```
3. vika — очков: 143
Игроков: 3, всего очков: 358
```

Имена здесь латиницей не случайно: если выравнивать колонки по ширине (`%-8s`), ширина считается в **байтах**, а кириллическая буква в UTF-8 занимает два — таблица поедет. Та же история, что с посимвольной обработкой русских слов в главе 10.

Стандарты и платформы, коротко

Всю книгу мы собирали с `-std=c11` — стандартом 2011 года: он давно и всюду поддержан. После него вышли ещё два. **C17** (он же C18) — только правки и уточнения, новых возможностей нет; переключение с C11 на C17 обычно вообще ничего не меняет. **C23** — первый за долгое время стандарт с заметными новинками: `bool`, `true` и `false` без `#include <stdbool.h>`, `nullptr` вместо `NULL`, `constexpr` для настоящих констант, двоичные литералы `0b1011`.

Что понимает твой компилятор, покажет макрос `__STDC_VERSION__` — номер стандарта в виде «год и месяц». Положи в `std.c` обычную программу с единственной строкой в `main`:

```
printf("__STDC_VERSION__ = %ld\n", __STDC_VERSION__);
```

```
$ cc -std=c11 std.c -o std && ./std
__STDC_VERSION__ = 201112
$ cc -std=c17 std.c -o std && ./std
__STDC_VERSION__ = 201710
$ cc -std=c23 std.c -o std && ./std
__STDC_VERSION__ = 202311
```

Пиши на C11, пока не появилась конкретная причина переехать, и проверяй `-std=`, прежде чем копировать чужой свежий код: программа с `nullptr` под `-std=c11` просто не соберётся.

Про платформы. Всё, что мы писали, — чистый Си, кроме «Змейки»: `termios.h`, `tcsetattr`, `usleep` — это POSIX (macOS, Linux и прочие Unix-подобные; `usleep` в новых редакциях POSIX уже заменён на `nanosleep`, но обе системы его по-прежнему держат). На Windows этих заголовков нет, там свой API (`conio.h`, `_kbhit`, `_getch`, `Sleep`). Обычный приём — развилка на этапе компиляции: компилятор сам определяет макросы под текущую систему (`_WIN32` на Windows, `__APPLE__` на macOS, `__linux__` на Linux), а препроцессор выбирает нужный кусок и прячет его за своей функцией вроде `sleep_ms`, чтобы остального кода это не касалось.

Частые ошибки

- **Тело функции в `.h`.** Заголовок вставляется в каждый `.c`, который его включил. Определение функции окажется в двух объектных файлах, и компоновщик скажет `duplicate symbol`. В `.h` — только объявления, `struct`, `#define`.
- **Забыт страж включения.** Работает, пока заголовок включён один раз. Как только он попадёт в программу дважды — посыплются «redefinition».
- **Пробелы вместо табуляции в `Makefile`.** `make` отвечает `missing separator`. Проверь, что редактор не «улучшил» твой TAB.
- **Забыта зависимость от `.h` в `Makefile`.** Правишь заголовок, `make` говорит «up to date», программа собрана из старых `.o` и ведёт себя необъяснимо. Правило: у каждого `.o` в зависимостях перечислены все заголовки, которые он включает.
- **Отладка без `-g`.** `lldb` запустится, но покажет адреса вместо имён и строк. Всегда собирай отладочную версию с `-g`.
- **`fopen` без проверки на `NULL`.** Самая частая ошибка при работе с файлами и, как мы видели, сразу падение.
- **Забытый `fclose`.** Пока программа выходит нормально, библиотека закроет файл за тебя. Но стоит ей упасть — данные из буфера пропадают, файл на диске пустой или обрезанный.
- **`'\n'` в конце строки от `fgets`.** Забудешь срезать — сравнения строк перестанут совпадать, а вывод поедет.

Упражнения

1. Разбей на файлы что-нибудь из книги. Возьми `code/08-bubble.c` или `code/11-stack.c`, вынеси алгоритм в `sort.c`/`sort.h` (или `stack.c`/`stack.h`), оставь в `main.c` только ввод и вывод и собери через `Makefile`. Проверка: вывод новой программы байт в байт совпадает с выводом исходного файла (сравни командой `diff`).
2. Добавь в `Makefile` цель `run`, которая сначала собирает `prog`, а потом его запускает (подсказка: цель `run` должна зависеть от `prog`), и цель `debug`, собирающую с `-fsanitize=address,undefined`. Проверка: после `make clean` команда `make run` печатает три строки сборки, затем `./prog` и результат программы; `make debug` собирает отдельный файл (например, `prog-debug`), который печатает то же самое, а санитайзеры молчат.
3. Возьми программу с багом — например, намеренно испорти границу цикла в бинарном поиске из главы 7 — и найди ошибку **только** отладчиком, без единого `printf`. Поставь точку останова внутри цикла и смотри, как меняются `lo`, `hi`, `mid`. Признак, что нашёл: ты можешь назвать итерацию, на которой `mid` вышел за границу массива или `lo` и `hi` перестали сближаться.

4. Переделай `code/18-files.c` так, чтобы имя файла бралось из командной строки: `int main(int argc, char *argv[])`, файл — `argv[1]`. Не забудь проверить `argc >= 2`: при запуске без аргументов `argc` равен 1, а `argv[1]` — `NULL`. Проверка: `./files` без аргументов печатает подсказку вроде «Использование: ./files файл» и выходит с кодом 1, а `./files my.txt` печатает то же, что в тексте главы.
5. Допиши в «Угадай число» (глава 12) таблицу рекордов: после победы программа дописывает число попыток в `records.txt` режимом `"a"`, а при старте читает файл и печатает лучший результат. Проверка: при самом первом запуске файла ещё нет — `fopen` с `"r"` вернёт `NULL`, и программа должна спокойно сказать «рекордов пока нет», а не упасть.
6. Напиши программу `wc`: читает файл, имя которого дано в командной строке, и печатает число строк, слов и символов. Слова считай так же, как в главе 10, читай посимвольно через `fgetc`. Проверка: на латинском тексте три числа совпадают с системной командой `wc файл` (она считает строки по символам `'\n'`, а «символы» — в байтах, поэтому на кириллице третье число будет вдвое больше букв).

Что мы получили

Четыре вещи, которых в книге не было и которые отделяют «пишу программы в одном файле» от «веду проект»: разделение на модули с заголовками, сборка через `make`, отладчик и работа с файлами. Каждую мы не описали, а сделали руками — с ошибками компоновщика, с `missing separator`, с падением на непроверенном `fopen`.

Куда идти дальше, коротко.

Книги. «Язык программирования C» Кернигана и Ритчи (K&R) — тонкая, строгая, написана авторами языка; читать после этой книги в самый раз, но учти, что второе издание описывает стандарт ANSI C 1989 года. K. N. King, «C Programming: A Modern Approach» (второе издание, 2008) — толстая, подробная, с упражнениями; один из лучших учебников-справочников по языку. Написан под C99, так что почти всё в нём совпадает с тем C11, на котором мы писали.

Задачи. Codeforces (codeforces.com), LeetCode (leetcode.com), Eolymp (eolymp.com, раньше назывался e-olymp) — тысячи задач с автопроверкой. Решать их полезно ровно до тех пор, пока это не заменило собой работу над своими программами.

Свои проекты. Бери то, чем сам будешь пользоваться: свой `grep` или `wc`, разбор CSV-файла, калькулятор с приоритетами операций, генератор лабиринтов, консольный «Тетрис». Хочется игр покрасивее — библиотека **ncurses** возьмёт на себя всё, что мы делали руками через `termios`, и добавит цвета и окна; **SDL** (сейчас актуальная версия — SDL3) и **raylib** дадут настоящее окно, спрайты и звук.

Другие языки. Из Си дорога короткая. **C++** — тот же синтаксис плюс классы, шаблоны и контейнеры: `std::vector` вместо ручного растущего массива из главы 11. **Rust** решает ровно ту проблему, на которой мы спотыкались, — память: компилятор не даст обратиться к уже освобождённой памяти (use-after-free) или освободить её дважды, но за это придётся спорить с ним о времени жизни данных. Утечку, кстати, он не запрещает — её в Rust можно устроить и без хитростей. **Python** — противоположный полюс: медленнее, зато быстро пишется; после Си ты будешь понимать, что там происходит под капотом, и это большое преимущество.

И последнее. Дальше — три приложения с задачами, которые реально дают на собеседованиях С-разработчику: Junior, Middle и Senior. Это не довесок, а хорошая проверка: если ты решишь их сам, значит книга усвоена не на уровне «читал», а на уровне «умею». Начни со следующей короткой главы — она про то, как устроена секция кодинга на собеседовании и как готовиться, чтобы не тратить время зря.

Как готовиться к собеседованию по Си

Дальше идут три приложения с задачами, которые реально дают на собеседованиях C-разработчику: **А** — Junior, **Б** — Middle, **В** — Senior. Прежде чем в них нырять, стоит понять, как устроено само собеседование и что на нём оценивают. Иначе легко готовиться не к тому: вызубрить двадцать алгоритмов и провалиться на вопросе «а что, если массив пустой?».

Эта глава — карта местности. Как проходит секция кодинга, что спрашивают на каждом уровне, в каком порядке проходить задачи, как проверять себя компилятором и санитайзерами и что говорить, когда ответа не знаешь.

Как проходит секция кодинга

Формат почти везде один. Тебе дают задачу на 15–30 минут — вслух или текстом, в редакторе с подсветкой, в Google-документе, а иногда просто на бумаге или на доске. Компилятора обычно нет: код надо написать так, чтобы он собрался «в голове». Интервьюер сидит рядом (или в звонке) и смотрит, как ты работаешь.

Задачи маленькие. Никто не просит написать базу данных — просят развернуть строку, найти второй по величине элемент, развернуть односвязный список, посчитать единичные биты. Всё это есть в приложениях А и Б, и всё это ты умеешь после книги.

Дальше почти всегда идёт вторая часть, короткая: «каверзы». Кусок кода на пять строк и вопрос «что выведет?» или «в чём ошибка?». Их гоняют, потому что за две минуты они показывают то, что задача на код показывает за двадцать.

Что оценивают, кроме правильного ответа

Самое неочевидное для новичка: работающий код — это меньше половины оценки. Смотрят вот на что.

Говоришь ли ты вслух. Молчащий кандидат — чёрный ящик. Интервьюер не видит, думаешь ты или завис, и не может подсказать. Проговаривай: «так, мне нужно пройти строку до конца… длину я знаю… значит, два индекса с двух концов навстречу». Даже неверная мысль вслух лучше тишины.

Уточняешь ли условие. Хороший кандидат до кода задаёт два-три вопроса: «массив отсортирован?», «могут быть дубликаты?», «строку можно менять на месте или нужна копия?», «что вернуть, если элемента нет?». Это не придирки — это то, что в работе называется «разобраться в задаче».

Помнишь ли про краевые случаи. Пустой ввод, ноль, отрицательное число, массив из одного элемента, все элементы равны, переполнение. Идеально — назвать их **до** того, как спросят: «на пустом массиве верну ноль и признак ошибки, потому что максимума тут нет».

Проверяешь ли ввод. `scanf` возвращает число прочитанных значений, `malloc` возвращает `NULL`, `fopen` возвращает `NULL`. Кандидат, который пишет `if (malloc вернул NULL)`, сразу выглядит взрослее.

Помнишь ли про память. Выделил через `malloc` — скажи вслух, где будет `free` и кто за него отвечает. Это главный маркер «человек понимает Си», а не «пишет на Си как на Python».

Как реагируешь на замечание. Тебе покажут ошибку — иногда настоящую, иногда чтобы посмотреть, будешь ли спорить. Правильная реакция: «сейчас проверю… да, действительно, на `n = 1` цикл не выполнится». Спорить, будучи неправым, — худшее, что можно сделать.

Как это звучит на практике, на задаче «найти второй по величине элемент» (приложение А, задача 5):

```
Я:  Уточню: массив может быть пустым? Дубликаты возможны?
Он: Дубликаты возможны, массив непустой.
Я:  Тогда край — массив из одного элемента: второго максимума нет,
    я верну признак ошибки. И дубликаты: в {5, 5, 3} второй по
    величине — это 3 или 5?
Он: Различные значения. Значит, 3.
Я:  Хорошо. Один проход, две переменные: максимум и второй максимум.
    Начальные значения возьму INT_MIN, чтобы не гадать...
```

Половина этого текста — не код. Именно она и решает.

Что спрашивают на каждом уровне

Уровни условны, планка у всех своя, но логика такая.

Junior (приложение А). Проверяют, что ты вообще пишешь код. Синтаксис, циклы, массивы, строки, простые алгоритмы: поменять переменные местами (задача 1), развернуть строку на месте (задача 2), палиндром (задача 3), факториал (задача 4), гласные в строке (задача 6), простое ли число (задача 7), FizzBuzz (задача 11). Плюс каверзы на понимание основ: `sizeof('a')` (каверза 1), `5 / 2` (каверза 2), = вместо == (каверза 5). Блестящих алгоритмов не ждут — ждут аккуратности и того, что ты понимаешь каждую свою строку.

Middle (приложение Б). Главная тема — **указатели и память**. Свои `strlen`, `strcpy`, `strcmp` (задача 1), развернуть односвязный список (задача 2), два указателя разной скорости — середина списка и цикл по Флойду (задача 3), биты: popcount и степень двойки (задача 4), бинарный поиск без ошибки на единицу (задача 5). Каверзы злее и почти все про память: `char *s = "abc"` против `char s[] = "abc"` (каверза 1), `sizeof` массива внутри функции (каверза 2), утечка (каверза 6), use-after-free и двойной `free` (каверза 7), возврат адреса локальной переменной (каверза 8). Проверяют не знание рецептов, а понимание, **что происходит в памяти**.

Senior (приложение В). Тонкости языка плюс инженерное мышление. Быстрая сортировка и сортировка слиянием (задачи 1–2), обобщённые структуры через `void *` (задача 3), таблица обработчиков на указателях на функции (задача 5), кольцевой буфер (задача 6), хеш-таблица с цепочками (задача 7), LRU-кэш (задача 9). Из теории — неопределённое поведение (каверза 1), strict aliasing (каверза 2), `volatile` (каверза 3), две роли `static` (каверза 4), выравнивание и padding (каверза 6), опасные макросы (каверзы 7–8). Здесь мало решить: надо обсудить компромиссы — сложность, память, худший случай, что сломается в продакшене.

План подготовки

Порядок — строго снизу вверх, даже если метишь в Middle или Senior. Приложение А проходится за пару вечеров и освежает базу, а его каверзы всплывают на собеседованиях любого уровня: `sizeof('a')` спрашивают и у сеньоров. Пропустить А и сесть за В — верный способ застрять на пустяке.

Разумный темп — 3–5 задач за подход, не больше. После пятой задачи подряд голова перестаёт замечать краевые случаи, а именно они и нужны. Приложение А — два-три вечера, Б — неделя, В — две-три недели, если разбираться, а не пролистывать.

Решай сам, до решения. Прочитал условие — закрой книгу. Выпиши на бумаге пример руками: вход, ожидаемый выход, один краевой случай. Только потом код. Даже провальная попытка включает голову так, как чтение готового ответа не включит никогда. Подсмотренное решение забывается к утру, выстраданное — остаётся.

Проверяй себя компилятором. Написал — собери, даже если уверен:

```
cc -std=c11 -Wall -Wextra -O2 solution.c -o solution
```

`-Wall -Wextra` ловят половину каверз из приложений сами: и `=` вместо `==` в условии, и лишнюю `;` после `for`, и неинициализированную переменную. Если хоть одно предупреждение — код не готов.

Гоняй санитайзеры. Компилятор видит не всё: выход за границу массива он пропустит. Смотри, что бывает. Вот классическая ошибка на единицу в поиске максимума — файл `maxbug.c`:

```c
#include <stdio.h>
#include <stdlib.h>

int max_of(const int *a, int n)
{
    int m = a[0];
    for (int i = 1; i <= n; i++)      /* баг: <= вместо < */
        if (a[i] > m)
            m = a[i];
    return m;
}

int main(void)
{
    int *a = malloc(4 * sizeof(int));
    if (a == NULL)
        return 1;

    a[0] = 3; a[1] = 9; a[2] = 1; a[3] = 7;
    printf("max = %d\n", max_of(a, 4));

    free(a);
    return 0;
}
```

В массиве четыре элемента, значит индексы 0–3. Цикл с `i <= n` доходит до `a[4]` — читает то, чего нет. Обычная сборка молчит, а программа врёт:

```
$ cc -std=c11 -Wall -Wextra -O2 maxbug.c -o maxbug
$ ./maxbug
max = 2147483647
```

Ни одного предупреждения — и мусор в ответе: 2147483647 — это `INT_MAX`, в массиве такого числа нет. Чтение `a[4]` — неопределённое поведение (UB): стандарт ничего не обещает, и оптимизатор с `-O2` вправе рассуждать так, будто этого чтения не бывает. Собери тот же файл без оптимизации (`-O0`) — и на этой машине он напечатает `max = 9`, верный ответ. Это самый опасный исход: баг есть, а тест проходит. Теперь пересоберём с **санитайзерами**: `-fsanitize=address` подставляет проверки на каждое обращение к памяти, `-fsanitize=undefined` — на неопределённое поведение. `-g` нужен, чтобы в отчёте были номера строк.

```
$ cc -std=c11 -Wall -Wextra -g -fsanitize=address,undefined \
     maxbug.c -o maxbug-san
$ ./maxbug-san
=================================================================
==72744==ERROR: AddressSanitizer: heap-buffer-overflow on address
    0x6020000000e0 at pc 0x000102776989 bp 0x7ff7bd7887e0
    sp 0x7ff7bd7887d8
READ of size 4 at 0x6020000000e0 thread T0
    #0 0x000102776988 in max_of maxbug.c:8
    #1 0x000102776db5 in main maxbug.c:20
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

0x6020000000e0 is located 0 bytes after 16-byte region
    [0x6020000000d0,0x6020000000e0)
allocated by thread T0 here:
    #0 0x000102cd4792 in malloc+0x82
       (libclang_rt.asan_osx_dynamic.dylib:x86_64h+0xa6792)
    #1 0x000102776ad3 in main maxbug.c:15
    #2 0x7ff804aa4b27 in start+0xca7 (dyld:x86_64+0x12b27)

SUMMARY: AddressSanitizer: heap-buffer-overflow maxbug.c:8 in max_of
```

(Адреса и номер процесса у тебя будут свои. Длинные строки отчёта я перенёс с отступом, чтобы влезли в страницу, — в терминале каждая из них одна. Дальше ASan печатает ещё карту теневой памяти — она нужна редко, здесь я её не привожу.)

Читается это так: **heap-buffer-overflow** — вышли за границу блока, выданного `malloc`. `max_of maxbug.c:8` — точное место, восьмая строка, `if (a[i] > m)`. «0 bytes after 16-byte region» — прочитали ровно за концом блока в 16 байт, то есть `a[4]` в массиве из четырёх `int`. Ниже — кто этот блок выделил: `main maxbug.c:15`, наш `malloc`. Ошибка на единицу найдена за секунду.

Переполнение знакового `int` ловит второй санитайзер. Факториал, посчитанный в `int`, лопается уже на 13 — вот `ovf.c`:

```c
#include <stdio.h>

int factorial(int n)
{
    int f = 1;
    for (int i = 2; i <= n; i++)
        f *= i;
    return f;
}

int main(void)
{
    printf("13! = %d\n", factorial(13));
    return 0;
}
```

```
$ cc -std=c11 -Wall -Wextra -g -fsanitize=undefined ovf.c -o ovf-san
$ ./ovf-san
ovf.c:7:11: runtime error: signed integer overflow: 479001600 * 13
    cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior ovf.c:7:11
13! = 1932053504
```

Строка 7 — это `f *= i`, и санитайзер прямо говорит, что 479 001 600 (это 12!) на 13 в `int` не помещается. Программа при этом даже не упала — просто напечатала неправду (настоящее 13! = 6 227 020 800). Именно поэтому в приложении А, задача 4, накопитель объявлен как `long long`. Без санитайзера ты бы этого не заметил, а на собеседовании спросят обязательно.

Правило простое: любое своё решение с указателями или `malloc` прогоняй через `-fsanitize=address,undefin` Если санитайзеры молчат — код можно показывать.

Подводные камни именно в Си

Список того, на чём валятся чаще всего. Рядом — где это разобрано, там и тренируйся.

- **Неопределённое поведение (UB).** Компилятор вправе сделать что угодно, включая «работает у меня на машине». Источники: выход за границы, чтение неинициализированной переменной, изменение строкового литерала, переполнение знакового, два изменения одной переменной в одном выражении. → приложение В, каверза 1; приложение А, каверзы 7 (`i++` и `++i` в одном `printf`) и 8 (неинициализированная переменная).

- **Переполнение знакового `int`.** Границы 32-битного `int` — от −2 147 483 648 до 2 147 483 647, примерно ±2 миллиарда (`INT_MIN`, `INT_MAX` из `<limits.h>`). Проверять надо **до** операции, а не после: `if (x > INT_MAX / 10)`. → приложение А, задача 4 (факториал); приложение В, задача 10 (разворот числа с защитой от переполнения).
- **Знаковость `char`.** Стандарт не говорит, знаковый `char` или беззнаковый, — это решает компилятор: на x86-64 под macOS знаковый, на ARM под Linux беззнаковый. Отсюда беда: байт из UTF-8-строки становится отрицательным, а функции из `<ctype.h>` (`isalpha`, `tolower`) требуют значение либо `EOF`, либо представимое как `unsigned char`. Отсюда приведение `(unsigned char)` перед каждым вызовом. → приложение А, каверзы 3 и 4.
- **`sizeof` массива против указателя.** В своей функции `sizeof arr` — размер массива, а в вызванной функции параметр `int arr[]` — это уже указатель, и `sizeof` даст размер указателя — 8 на 64-битной системе. Длину массива всегда передают отдельным параметром. → приложение Б, каверза 2.
- **Приоритет и порядок операторов.** `*p++` — это не `(*p)++`; `a & 1 == 0` — это не то, что ты думаешь, потому что `==` сильнее `&`; макрос `#define SQR(x) x*x` на `SQR(1+2)` даёт 5. Не помнишь приоритет — ставь скобки. → приложение Б, каверза 4; приложение В, каверзы 7 и 8, а про то, что порядок вычисления аргументов вообще не определён, — каверза 9.
- **Время жизни переменных и `static`.** Локальная переменная умирает на `}`, и возвращать её адрес нельзя. У `static` две совершенно разные роли: внутри функции — «живёт до конца программы», на уровне файла — «не видна из других файлов». → приложение Б, каверза 8; приложение В, каверза 4.
- **Память: утечки, use-after-free, двойной `free`.** На каждый `malloc` ровно один `free`, после `free` присваивай указателю `NULL`, никогда не освобождай дважды. → приложение Б, каверзы 6 и 7.
- **Строки.** `char *s = "abc"` — указатель на неизменяемый литерал, писать в него нельзя; `char s[] = "abc"` — своя копия в массиве, менять можно. `==` сравнивает адреса, а не содержимое, для содержимого — `strcmp`. Место под `'\0'` считать обязательно. → приложение Б, каверза 1.
- **Выравнивание и padding.** `sizeof` структуры часто больше суммы размеров полей: компилятор вставляет пустые байты ради выравнивания. Порядок полей меняет размер структуры. → приложение В, каверза 6.
- **Целочисленное деление и остаток.** `5 / 2` — это 2, а `-7 % 2` — это −1, а не 1: в Си знак остатка совпадает со знаком делимого, это не математический остаток, который всегда от 0 до делителя. Поэтому проверка `n % 2 == 1` для отрицательных ломается; пиши `n % 2 != 0`. → приложение А, каверзы 2 и 9.

Что говорить, если не знаешь

Не знать — нормально, никто не держит в голове весь стандарт. Плохо не это, а враньё и молчание. Работает такой порядок.

Скажи честно и сразу. «Точно не помню» — нормальный ответ, он экономит время обоим. «Наверное, оно как-то так работает» — плохой: интервьюер видит, что ты угадываешь.

Покажи, что рядом ты знаешь. «Про strict aliasing я знаю только, что нельзя читать `int` через `float *`, а почему именно — не разбирался. Зато точно знаю, что безопасный способ — `memcpy`». Это ответ уровня «человек в теме, но конкретная деталь не изучена».

Рассуждай вслух от того, что знаешь. «`volatile` я не применял. Но раз компилятор оптимизирует чтения переменной, а тут значение может поменяться снаружи, — видимо, `volatile` запрещает такую оптимизацию. Правильно?» Ход мысли ценится почти так же, как ответ.

Спроси и запиши. «Расскажете, как правильно?» — совершенно нормальная просьба, и она превращает провал в маленький плюс: человек учится прямо на собеседовании.

Не притворяйся, что решал. Ложь про опыт вскрывается одним уточняющим вопросом, и после этого не верят уже ничему.

И то же самое про задачу, которая не идёт. Скажи, где застрял: «Я вижу решение за O(n^2) — два вложенных цикла. Чувствую, что можно за O(n), но пока не вижу как. Напишу сначала квадратичное?» Почти всегда интервьюер скажет «пиши» или подкинет подсказку. Рабочее медленное решение сильно лучше пустого экрана.

Что дальше

Дальше — три приложения по уровням. Проходи их подряд: **А**, потом **Б**, потом **В**. У каждой задачи есть условие, идея, решение и блок «что спросят дальше» — те самые дожимающие вопросы, к которым и надо готовиться. Файлы с рабочими решениями лежат в `code/19-*.c`, `code/20-*.c` и `code/21-*.c`; собирай их так же, как всё в книге:

```
cc -std=c11 -Wall -Wextra -O2 code/20-05-binsearch.c -o binsearch
./binsearch
```

Не пытайся выучить всё за ночь. Начинай с приложения А — даже если метишь выше.

Приложение A. Задачи с собеседований: Junior

Книга закончилась, а собеседования только начинаются. Это приложение — сборник задач, которые реально дают на собеседовании C-разработчику уровня **Junior** (младший, начинающий). Здесь нет ничего, чего ты не проходил: всё опирается на главы 1–18 — числа, массивы, строки, указатели, циклы, функции. Новой теории не будет. Будет другое — взгляд на знакомые приёмы глазами интервьюера.

На собесе мало написать работающий код. Спрашивающий смотрит, **как ты думаешь**: учёл ли краевые случаи, помнишь ли про переполнение, понимаешь ли, что делает каждая строка. Поэтому к каждой задаче я добавил блок «что спросят дальше» — те самые уточняющие вопросы, которыми тебя будут «дожимать».

Как пользоваться

Каждая задача оформлена так:

- **Условие** — как её формулируют вслух на собесе.
- **Идея** — как к ней подступиться, без кода.
- **Решение** — рабочий код.
- **Разбор** — почему так, а не иначе, какая сложность, где края.
- **Что спросят дальше** — продолжения и подвохи.

Главный совет: **сначала реши сам**, на бумаге или в редакторе, и только потом смотри решение. Подсмотренное решение забывается к утру. Решённое самому — остаётся. Даже если решил криво — сравни с моим вариантом, и разница тебе многое объяснит.

Приложение делится на три части:

1. **Задачи 1–8** — у каждой есть готовый файл `code/19-0N-*.c`, номер задачи совпадает с номером файла. Собирается и запускается так:

```
cc -std=c11 -Wall -Wextra -O2 code/19-07-prime.c -o prime
./prime
```

2. **Задачи 9–11** — три классики, которые решаются в несколько строк; отдельных файлов под них нет, весь код прямо в тексте.
3. **Каверзы** — короткие фрагменты «что выведет / в чём ошибка». Многие из них намеренно содержат ошибку или неопределённое поведение, поэтому в `code/` они не вынесены: в файлах книги лежит только код, который **можно** запускать.

Все выводы программ в этом приложении получены реальным запуском на Apple clang 21 (x86-64, macOS). Где результат зависит от платформы или от компилятора, я это отдельно оговариваю.

Часть 1. Задачи с готовыми решениями

Задача 1. Поменять местами две переменные

Условие. Есть две переменные `a` и `b`. Поменяй их значения местами. После обмена в `a` должно лежать то, что было в `b`, и наоборот.

Идея. Наивная попытка «в лоб» не работает:

```
a = b;   /* старое значение a потеряно навсегда */
b = a;   /* сюда попадёт b, а не старое a */
```

Первая же строка затирает `a`. Нужна **временная переменная**, чтобы запомнить значение перед затиранием.

Решение (`code/19-01-swap.c`):

```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 9;

    printf("До:    a = %d, b = %d\n", a, b);

    int temp = a;   /* запомнили a */
```

```
    a = b;          /* в a положили b */
    b = temp;       /* в b положили старое a */

    printf("После: a = %d, b = %d\n", a, b);
    return 0;
}
```

Запуск:

```
До:    a = 5, b = 9
После: a = 9, b = 5
```

Разбор. `temp` хранит копию `a`, пока мы перезаписываем `a` значением `b`. Потом достаём из `temp` старое `a` и кладём в `b`. Три присваивания, ни одного условия, ни одного цикла — O(1) по времени и O(1) по памяти (одна лишняя переменная). Работает для любых значений, включая `INT_MIN` и `INT_MAX`: мы ничего не складываем и не вычитаем, переполниться нечему.

Что спросят дальше. «А можно без временной переменной?» Да, два способа.

Через арифметику:

```
a = a + b;
b = a - b;   /* = старое a */
a = a - b;   /* = старое b */
```

Через XOR (исключающее «или», оператор `^`):

```
a = a ^ b;
b = a ^ b;
a = a ^ b;
```

Проверим XOR на наших числах. 5 в двоичном виде — `0101`, 9 — `1001`. `^` ставит 1 там, где биты **разные**:

```
a = a ^ b:  0101 ^ 1001 = 1100   (a = 12, «смесь» a и b)
b = a ^ b:  1100 ^ 1001 = 0101   (b = 5  — старое a)
a = a ^ b:  1100 ^ 0101 = 1001   (a = 9  — старое b)
```

Работает, потому что `x ^ y ^ y == x`: повторный XOR с тем же числом «снимает» его обратно. Арифметика устроена так же: в `a` лежит сумма 14, `14 - 9 = 5`, `14 - 5 = 9`. Оба фрагмента я собрал и запустил: `a = 9, b = 5`.

Оба выглядят эффектно, и интервьюер любит про них спросить. Но честный ответ — **в реальном коде так делать не надо**. Почему:

- Арифметический способ **переполняется**: если `a + b` вылезет за пределы `int`, это не «завернётся в минус», а неопределённое поведение — переполнение знакового типа стандарт не описывает вообще (см. главу 4 про переполнение; подробный разбор со всеми источниками UB — в Приложении В, каверза 1). Пример: `a = INT_MAX`, `b = 1` — санитайзер `-fsanitize=undefined` (см. главу «Как готовиться к собеседованию») ловит здесь три переполнения подряд. С `unsigned` этот трюк как раз корректен: беззнаковая арифметика по стандарту идёт по модулю 2^N и не переполняется.
- Оба трюка ломаются, если `a` и `b` — **одна и та же переменная** (один адрес). Так бывает в функции обмена через указатели: `swap(&x, &x)`, например когда сортировка меняет элемент сам с собой. XOR: `*a ^= *b` превращает `x` в `x ^ x`, то есть в 0, и дальше из нуля старое значение уже не достать. Сложение: `*a = x + x`, потом `*b = 2x - 2x` = 0 — тоже ноль. Я проверил на `x = 42`: оба варианта дают 0, а обмен через `temp` оставляет 42. Одинаковые **значения** в разных переменных (`a = 42`, `b = 42`) при этом не страшны — ломает именно общий адрес.
- XOR работает только с целыми: для `double` строка `a = a ^ b` просто не соберётся (`invalid operands to binary expression`).
- Оба варианта **нечитаемы** и не быстрее: современный компилятор и так оптимально реализует обмен через `temp` — на уровне ассемблера это часто просто две пересылки через регистры, а иногда компилятор вообще обходится без обмена, запомнив, в каком регистре что лежит.

Правильный вывод, который хочет услышать грамотный интервьюер: «знаю про XOR-трюк, но в продакшене пишу через `temp` — понятнее и без подвохов».

Второе продолжение: «сделай так, чтобы обмен работал из функции». Это уже задача про указатели, она разобрана в Приложении Б (задача 9).

Задача 2. Развернуть строку на месте

Условие. Дана строка. Переверни её **на месте** — без создания второго массива.

Идея. Два индекса: один на начало (`left`), другой на конец (`right`). Меняем символы местами и идём навстречу, пока они не встретятся посередине.

Проверим руками на слове `abcde` (пять символов, индексы 0…4):

```
шаг 0:  a b c d e     left=0, right=4  →  меняем a и e
шаг 1:  e b c d a     left=1, right=3  →  меняем b и d
шаг 2:  e d c b a     left=2, right=2  →  left < right ложно, стоп
```

Средний символ `c` остался на месте — так и должно быть.

Решение (`code/19-02-reverse-string.c`):

```c
#include <stdio.h>
#include <string.h>

/* Переворачивает строку на месте, без второго массива. */
void reverse(char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        char tmp = s[left];
        s[left] = s[right];
        s[right] = tmp;
        left++;
        right--;
    }
}

int main(void)
{
    char text[] = "Hello, world";
    reverse(text);
    printf("%s\n", text);
    return 0;
}
```

Запуск:

```
dlrow ,olleH
```

Разбор. `strlen(s)` — длина строки без учёта завершающего нуля `'\0'` (см. главу 10). Возвращает она `size_t` — беззнаковый тип; мы приводим результат к `int`, потому что `right` должен уметь стать отрицательным. Если бы `right` был `size_t`, то для **пустой** строки `strlen(s) - 1` дало бы не −1, а огромное число (беззнаковое вычитание заворачивается), и цикл полез бы за границы массива. Это типичная ловушка со знаковостью, о ней стоит сказать вслух.

Внутри цикла — обычный обмен через `tmp` (как в задаче 1). Условие `left < right` останавливает нас на середине: средний символ при нечётной длине трогать не нужно.

Сложность. O(n) по времени: `strlen` один раз проходит строку целиком, а потом делаем $\lfloor n/2 \rfloor$ обменов (n/2 с округлением вниз: для `abcde` при n = 5 — два обмена). O(1) по дополнительной памяти — новый буфер не выделяли.

Краевые случаи. Пустая строка "": `strlen` даёт 0, `right` = −1, условие `0 < -1` ложно, цикл не выполняется — правильно. Строка из одного символа: `left = right = 0`, тоже ноль итераций. Строка из двух: ровно один обмен.

Что спросят дальше.

- «Почему `char text[] = "..."`, а не `char *text = "..."`?» Менять строковый литерал стандарт запрещает — это неопределённое поведение, а на практике литералы обычно лежат в памяти только для чтения, и программа падает. Массив `char text[]` — твоя изменяемая копия литерала, её менять можно. Подробный разбор с двумя разными сборками одной программы — в Приложении Б, каверза 1.
- «А если в строке кириллица?» Развернётся мусор: в UTF-8 русская буква занимает два байта, и посимвольный разворот перемешает половинки букв. Наш `reverse` разворачивает **байты**, а не буквы. Для UTF-8 нужен разбор по символам — это честный и правильный ответ.
- «Сделай без `strlen`.» Найди конец сам: `int right = 0;`, потом `while (s[right] != '\0') right++;` и `right--`. Ровно то же самое, что делает `strlen`.

Задача 3. Проверка палиндрома

Условие. Палиндром читается одинаково слева направо и справа налево (`level`, `radar`). Проверь, палиндром ли данная строка.

Идея. Та же пара индексов с краёв, но теперь не меняем символы, а **сравниваем**. Нашли несовпадение — сразу не палиндром, дальше можно не смотреть.

Решение (`code/19-03-palindrome.c`):

```c
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если строка — палиндром, иначе 0. */
int is_palindrome(const char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        if (s[left] != s[right]) {
            return 0;   /* нашли несовпадение — это не палиндром */
        }
        left++;
        right--;
```

```
    }
    return 1;
}

int main(void)
{
    const char *words[] = {"level", "radar", "hello", "noon"};
    for (int i = 0; i < 4; i++) {
        if (is_palindrome(words[i])) {
            printf("%s — палиндром\n", words[i]);
        } else {
            printf("%s — нет\n", words[i]);
        }
    }
    return 0;
}
```

Запуск:

```
level — палиндром
radar — палиндром
hello — нет
noon — палиндром
```

Разбор. Параметр `const char *s` — обещание «эту строку я менять не буду, только читаю». `words` — массив указателей на строковые литералы, и объявлен он как `const char *words[]`: менять литералы нельзя, и мы честно фиксируем это в типе.

Отсюда следствие, которое любят проверять. Убери `const` у параметра — и вызов `is_palindrome(words[i])` перестанет собираться молча:

```
pal.c:23:27: warning: passing 'const char *' to parameter of type
'char *' discards qualifiers
[-Wincompatible-pointer-types-discards-qualifiers]
   23 |         if (is_palindrome(words[i])) {
      |                           ^~~~~~~~
pal.c:5:25: note: passing argument to parameter 's' here
    5 | int is_palindrome(char *s)
      |                         ^
1 warning generated.
```

(Длинную первую строку я перенёс, чтобы она влезла в страницу; имя файла у тебя будет своё.)

Правило одностороннее: `char *` передать туда, где ждут `const char *`, можно, а обратно — нет. Поэтому `const` на входном параметре тебя не ограничивает, а наоборот — расширяет круг того, что функции можно скормить.

`return 0` при первом же несовпадении — это **ранний выход**: нет смысла проверять дальше, ответ уже ясен.

Сложность. Худший случай (строка действительно палиндром) — $\lfloor n/2 \rfloor$ сравнений, то есть O(n). Лучший случай — одно сравнение, если первый и последний символы разные. Плюс один проход `strlen` в начале — он тоже O(n). Памяти O(1).

Краевые случаи. Пустая строка и строка из одного символа считаются палиндромами — цикл не выполняется, возвращается 1. Это общепринятая трактовка, но на собесе её лучше проговорить вслух: «пустую строку считаю палиндромом, если надо иначе — скажи».

Что спросят дальше.

- «Сделай регистронезависимо: `Level` тоже палиндром.» Сравнивай не `s[left]` и `s[right]`, а их версии в одном регистре (приём приведения к нижнему регистру — в задаче 6 про гласные).
- «А если игнорировать пробелы и знаки: `A man a plan a canal Panama`?» Тогда индексы нужно «перепрыгивать» через не-буквы:

  ```
  while (left < right && !isalpha((unsigned char)s[left]))  left++;
  while (left < right && !isalpha((unsigned char)s[right])) right--;
  ```

  Эти две строки ставятся в начало тела цикла, перед сравнением; `isalpha` («это буква?») живёт в `<ctype.h>`. Приведение к `unsigned char` тут обязательно — почему, разбираем в каверзе 4.
- «Проверь палиндром рекурсивно.» Функция сравнивает края и вызывает себя для внутренней части. Работает, но глубина рекурсии — около n/2 вызовов, то есть O(n) памяти стека (см. главу 9), — итеративно лучше.

Задача 4. Факториал итеративно

Условие. Посчитай факториал числа `n` циклом (без рекурсии). `n! = 1·2·…·n`, при этом `0! = 1`.

Идея. Накопитель `fact = 1`, домножаем на 2, 3, …, n. Стартовое значение — **единица**, а не ноль (умножение на 0 всё обнулит). И сразу помним про переполнение из главы 4.

Решение (`code/19-04-factorial.c`):

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }
    if (n > 20) {
        printf("Слишком большое: long long переполнится при n > 20.\n");
        return 1;
    }

    long long fact = 1;   /* 0! = 1 и 1! = 1 — стартуем с единицы */
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %lld\n", n, fact);
    return 0;
}
```

Запуск:

```
Факториал какого числа? 13
13! = 6227020800
```

Разбор. Тип накопителя — `long long` (на этой машине 8 байт), потому что `int` лопается уже на 13!: 6 227 020 800 больше, чем `INT_MAX` = 2 147 483 647. Печатаем через `%lld` — свой спецификатор для `long long`; напишешь `%d`, и `printf` возьмёт из аргументов не то количество байт, получится мусор (а компилятор с `-Wall` заругается).

Цикл идёт с 2, потому что умножать на 1 бессмысленно. Для `n = 0` и `n = 1` он не выполнится ни разу, и `fact` останется 1 — верный ответ без всякого отдельного `if`.

`scanf("%d", &n)` возвращает **количество успешно прочитанных значений**. Мы сравниваем его с 1: если пользователь ввёл `abc`, `scanf` вернёт 0, и мы честно скажем «это не число», а не станем считать факториал мусора. Пропущенная проверка `scanf` — одна из самых частых претензий на собеседовании.

Сложность. O(n) умножений, O(1) памяти. Для `n ≤ 20` это не больше девятнадцати умножений (от 2 до 20).

Что спросят дальше.

- «До какого `n` это корректно?» До 20: 20! = 2 432 902 008 176 640 000 ещё помещается в `long long` (предел 9 223 372 036 854 775 807), а 21! — уже нет. Поэтому мы заранее отсекаем `n > 20`. Грамотный кандидат **сам** проговаривает границу типа, не дожидаясь вопроса.
- «А если бы отсечки не было?» Получили бы переполнение знакового типа — это UB, а не «просто неверное число». Программа имеет право напечатать что угодно.
- «А если ввести 2147483648?» Проверь сам: программа ответит «Факториал определён только для n >= 0.». Число не влезло в `int`, а для `scanf` с `%d` такой ввод — неопределённое поведение (стандарт C11, раздел 7.21.6.2); на этой машине в `n` оказалось отрицательное число. Проверка «`scanf` вернул 1» от этого не спасает. Надёжный способ — читать строку и разбирать её самому с контролем переполнения, как в Приложении В, задача 4.
- «А рекурсией?» Можно (см. главу 9), но итеративно — без риска переполнить стек вызовов на больших `n` и обычно быстрее.
- «А факториал 100?» В `long long` не влезет никогда. Нужна «длинная арифметика»: хранить число как массив цифр и умножать столбиком.

Задача 5. Второй по величине элемент массива

Условие. Найди второй по величине элемент массива за **один** проход (не сортируя).

Идея. Держим две переменные: `max` (наибольший) и `second` (второй). Для каждого числа: если оно больше `max` — старый `max` становится `second`, а само число новым `max`. Иначе, если оно меньше `max`, но больше `second` — обновляем `second`. Дубликаты максимума вторым считать не должны.

Решение (`code/19-05-secondmax.c`):

```c
#include <stdio.h>
#include <limits.h>

int main(void)
```

```
{
    int a[] = {3, 9, 9, 1, 7, 9, 4};
    int n = (int)(sizeof(a) / sizeof(a[0]));

    int max = INT_MIN;     /* самое большое */
    int second = INT_MIN;  /* второе по величине */

    for (int i = 0; i < n; i++) {
        if (a[i] > max) {
            second = max;  /* бывший максимум становится вторым */
            max = a[i];
        } else if (a[i] > second && a[i] < max) {
            second = a[i]; /* не максимум, но больше второго */
        }
    }

    if (second == INT_MIN) {
        printf("Второго по величине нет.\n");
    } else {
        printf("Максимум: %d, второй: %d\n", max, second);
    }
    return 0;
}
```

Запуск:

```
Максимум: 9, второй: 7
```

Разбор. `INT_MIN` из `<limits.h>` — самое маленькое значение `int` (на этой машине −2147483648). Стартуем с него, чтобы любой реальный элемент оказался больше.

Пройдём массив `{3, 9, 9, 1, 7, 9, 4}` руками:

```
элемент  max      second   что произошло
   3     3        INT_MIN  первый больше max
   9     9        3        новый максимум, старый ушёл во второй
   9     9        3        не больше max и не меньше max — мимо обеих веток
   1     9        3        меньше second — мимо
   7     9        7        меньше max, больше second — обновили
   9     9        7        снова дубликат максимума — мимо
   4     9        7        меньше second — мимо
```

Максимум 9 встречается трижды, но вторым выводится 7, а не 9 — условие `a[i] < max` отсекает дубликаты максимума. Это именно тот случай, который любят подсунуть.

Сложность. Один проход — O(n) сравнений, O(1) памяти. Через сортировку было бы O(n log n), и интервьюер обязательно спросит, почему ты не отсортировал: потому что нам нужен один элемент, а не порядок.

Что спросят дальше.

- «Что если в массиве все элементы одинаковы (`{5,5,5}`)?» Тогда второго по величине нет — наш код напечатает «Второго по величине нет.», потому что `second` так и остался `INT_MIN`. Уточни у интервьюера, считать ли дубликат вторым: трактовки бывают разные, и важно проговорить выбор.
- «А если в массиве честно лежит `INT_MIN`?» Тогда наш признак «второго нет» ложно сработает на массиве `{5, INT_MIN}`: второй там есть, это `INT_MIN`, а программа скажет «нет». Лечится флагом `int have_second = 0;` вместо сравнения с `INT_MIN`. Хороший ответ, который показывает, что ты видишь дырку в собственном решении.
- «А массив из одного элемента?» Максимум найдётся, второго не будет — код напечатает «Второго по величине нет.», и это верно.
- «А пустой массив?» Тоже «Второго нет», но и `max` останется `INT_MIN`, хотя максимума у пустого массива нет вообще. В функции, которая возвращает результат, на `n < 2` сразу возвращают признак ошибки.
- «А k-й по величине?» Обобщение — там уже нужна частичная сортировка, куча (heap) или алгоритм быстрого выбора (quickselect). Это уже уровень Middle и выше.

Задача 6. Подсчёт гласных в строке

Условие. Посчитай, сколько гласных букв в строке (английские a, e, i, o, u в любом регистре).

Идея. Идём по строке посимвольно до `'\0'`. Для каждого символа проверяем, не гласная ли он. Чтобы не плодить проверки для верхнего и нижнего регистра, приводим символ к нижнему.

Решение (`code/19-06-vowels.c`):

```c
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если c — английская гласная (любого регистра). */
int is_vowel(char c)
{
    char lower = c;
    if (c >= 'A' && c <= 'Z') {
```

```
        lower = (char)(c - 'A' + 'a');   /* привели к нижнему регистру */
    }
    return lower == 'a' || lower == 'e' || lower == 'i'
        || lower == 'o' || lower == 'u';
}

int main(void)
{
    const char *s = "Programming In C";
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (is_vowel(s[i])) {
            count++;
        }
    }
    printf("В строке \"%s\" гласных: %d\n", s, count);
    return 0;
}
```

Запуск:

```
В строке "Programming In C" гласных: 4
```

Считаем руками: `o`, `a`, `i` в `Programming` и `I` в `In` — четыре. Буква `C` не гласная, пробелы тоже.

Разбор. Трюк `c - 'A' + 'a'` переводит заглавную букву в строчную. Символы в C — это коды (см. главу 2): буквы `A..Z` идут в ASCII подряд, и `a..z` тоже подряд. Разница между заглавной и строчной одной буквы постоянна и равна `'a' - 'A'` = 97 − 65 = 32, поэтому сдвиг работает для всех букв разом. (Строго говоря, стандарт Си не обещает ASCII и не обещает, что буквы идут подряд, — это обещают только для цифр `'0'..'9'`. Но на всех машинах, где ты будешь это запускать, кодировка совместима с ASCII.) Приведение `(char)` — явная пометка «сужаю обратно в `char`, так задумано»: арифметика в Си поднимает `char` до `int`, и результат выражения — `int`. clang 21 здесь молчит даже с `-Wconversion`, но другие компиляторы на такое неявное сужение могут предупредить.

Условие цикла `s[i] != '\0'` — канонический способ пройти строку до завершающего нуля. Как только упёрлись в него, строка кончилась.

Сложность. O(n) — один проход, по одному сравнению на символ. Памяти O(1).

Что спросят дальше.

- «А `<ctype.h>`?» В стандартной библиотеке есть `tolower()` — он делает ровно наш сдвиг и уважает локаль. Но у него важная тонкость: аргумент надо приводить к `unsigned char`, иначе байт с кодом больше 127 в знаковом `char` превратится в отрицательное число, и получишь UB. Правильно так: `char lower = (char)tolower((unsigned char)c);`. Почему — разбираем в каверзе 4. На собесе ценят, когда ты знаешь про готовое, но умеешь и руками, и помнишь про эту ловушку.
- «Учитывай ‘y’.» Расширение списка — проверка внимательности: в английском `y` считается гласной не всегда, и стоит уточнить условие.
- «А русские гласные?» Не через `char`: в UTF-8 русская буква — два байта, посимвольная проверка сломается. Нужен разбор UTF-8 или широкие символы `wchar_t`.

Задача 7. Проверка на простое число

Условие. Дано число `n`. Определи, простое ли оно (делится только на 1 и на себя; 0, 1 и отрицательные — не простые).

Идея. Перебираем возможные делители от 2. Нашли хоть один — число составное. Ключевая оптимизация: проверять делители только **до корня** из `n`, потому что делители идут парами (`n = d · (n/d)`), и меньший из пары всегда не больше $\sqrt{n}$: если бы оба множителя были больше $\sqrt{n}$, их произведение было бы больше n. Для 36 пары такие: $2\cdot18$, $3\cdot12$, $4\cdot9$, $6\cdot6$ — и дальше они просто повторяются зеркально ($9\cdot4$, $12\cdot3$, $18\cdot2$). Значит, если до $\sqrt{n}$ делителя нет, то его нет и после.

Решение (`code/19-07-prime.c`):

```c
#include <stdio.h>

/* Возвращает 1, если n — простое число, иначе 0. */
int is_prime(int n)
{
    if (n < 2) {
        return 0;             /* 0, 1 и отрицательные — не простые */
    }
    for (int d = 2; d <= n / d; d++) {   /* то же, что d * d <= n */
        if (n % d == 0) {
            return 0;         /* нашли делитель — составное */
        }
    }
    return 1;
}
```

```c
int main(void)
{
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое\n", n);
    } else {
        printf("%d — не простое\n", n);
    }
    return 0;
}
```

Запуск:

```
Введи число: 17
17 — простое
```

Разбор. Условие цикла `d <= n / d` — это и есть «до корня»: оно верно ровно тогда, когда `d * d <= n`, то есть когда d не больше √n. Проверим на 36: при `d = 6` имеем `36 / 6 = 6`, и `6 <= 6` — ещё проверяем; при `d = 7` уже `36 / 7 = 5` (деление нацело), `7 <= 5` ложно — стоп, и `7 * 7 = 49` тоже больше 36. Для простого 999 983 цикл проверит делители от 2 до 999 — это 998 делений вместо почти миллиона при переборе до `n`.

Почему не написать проще, `d * d <= n`? Так пишут почти все, и это ловушка: при `n` рядом с `INT_MAX` квадрат `d * d` сам вылезает за пределы `int`. Разбор этой ловушки — ниже, в «что спросят дальше».

Случаи `n < 2` отсекаем сразу — иначе единица ошибочно сошла бы за простую: цикл `d <= 1 / 2` при `d = 2` не выполнится ни разу, и функция вернула бы 1.

Сложность. O(√n) делений в худшем случае (когда `n` простое и цикл доходит до конца). Для составных чисел обычно гораздо быстрее: у половины чисел делитель 2 находится на первой же итерации.

Что спросят дальше.

- «Почему не `d <= sqrt(n)`?» Чтобы не звать дробную функцию `sqrt`: не нужен `<math.h>` (а на Linux часто ещё и ключ `-lm` при сборке), нет перевода в `double` и обратно. Для `int` корень на обычной машине посчитается точно, но для `long long` больше 2^53 (около 9·10^15) `double` уже не хранит все целые подряд, и граница цикла может сдвинуться на единицу. Целочисленное сравнение точное всегда.
- «А почему не `d * d <= n`?» Потому что `d * d` может переполниться. 46340^2 = 2 147 395 600 ещё помещается в `int`, а 46341^2 = 2 147 488 281 уже больше `INT_MAX` = 2 147 483 647. Для составных чисел цикл обрывается раньше, но если `n` простое и больше 2 147 395 600, цикл дойдёт до `d = 46341`, и умножение даст переполнение — UB. Такое число есть, и очень знакомое: сам `INT_MAX`, 2 147 483 647, — простое. Я проверил: версия с `d * d <= n`, собранная с `-O2`, на нём отвечает «не простое» — неправда. `d <= n / d` не умножает вообще и переполниться не может. Назвать эту ловушку самому — сильный ход.
- «Ускорь ещё.» Проверять 2 отдельно, а дальше только нечётные делители (`d = 3; d += 2`) — вдвое меньше итераций.
- «Все простые до N?» Это уже **решето Эратосфена** (см. главу 5) — частый следующий шаг: O(N log log N) вместо N отдельных проверок, каждая до O(√N), то есть до O(N√N) в сумме.

Задача 8. Цифры числа: сколько их и какая сумма

Условие. Дано целое число. Посчитай, сколько в нём цифр (123 → 3) и чему равна их сумма (456 → 4+5+6 = 15).

Идея. Обе задачи решаются одним циклом. Делим число на 10 нацело, пока оно не станет нулём: каждое деление откусывает одну цифру справа (см. главу 4). Заодно на каждом шаге берём остаток `n % 10` — это и есть откушенная цифра.

Разберём 456 руками:

```
n = 456   digit = 456 % 10 = 6   sum = 6    count = 1   n = 45
n = 45    digit = 45  % 10 = 5   sum = 11   count = 2   n = 4
n = 4     digit = 4   % 10 = 4   sum = 15   count = 3   n = 0
n = 0  →  цикл кончился
```

Решение (`code/19-08-digits.c`):

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
```

```
        printf("Это не число.\n");
        return 1;
    }

    long long m = n;   /* в long long модуль INT_MIN помещается */
    if (m < 0) {
        m = -m;        /* знак на количество и сумму цифр не влияет */
    }

    int count = 0;     /* сколько цифр */
    int sum = 0;       /* сумма цифр */

    if (m == 0) {
        count = 1;     /* у нуля одна цифра */
    }
    while (m > 0) {
        int digit = (int)(m % 10);
        sum += digit;
        count++;
        m /= 10;
    }

    printf("Цифр: %d, сумма цифр: %d\n", count, sum);
    return 0;
}
```

Запуск:

```
Введи целое число: 90834
Цифр: 5, сумма цифр: 24
```

Проверяем: 9+0+8+3+4 = 24, цифр действительно пять.

Разбор. Считаем мы не в самом `n`, а в его копии `m` типа `long long` — зачем, объясню в «что спросят дальше», это главный подвох задачи. `m % 10` — последняя цифра, `m /= 10` (то же, что `m = m / 10`) — отбрасываем её. `digit` вычисляем **один раз** и кладём в переменную, чтобы не писать `m % 10` второй раз (см. главу 4, шаг 3). `(int)` перед `m % 10` честно говорит: остаток от 0 до 9, в `int` он помещается. `count` и `sum` стартуют с 0: накопитель суммы всегда начинается с нейтрального элемента, для сложения это ноль.

Отдельный случай — само число **0**: цикл `while (m > 0)` для него не выполнится ни разу, и `count` остался бы нулём, а ответ должен быть 1 (одна цифра). Поэтому ноль обрабатываем вручную. С суммой всё в порядке и без этого: сумма цифр нуля равна нулю.

Сложность. Число итераций равно количеству цифр. Для $n \geq 1$ цифр ровно $\lfloor \log_{10} n \rfloor + 1$ (логарифм по основанию 10, округлённый вниз, плюс один): у 90834 логарифм примерно 4,96, вниз — 4, плюс один — 5 цифр. Отсюда сложность O(log n). Для `int` это максимум 10 шагов: у 2 147 483 647 десять цифр.

Что спросят дальше.

- «А отрицательные?» У −123 три цифры, минус не считается, поэтому мы берём модуль. И вот подвох: напиши просто `if (n < 0) n = -n;` в `int` — и для `INT_MIN` получишь UB, потому что у самого маленького `int` нет положительной пары (диапазон несимметричен: от −2 147 483 648 до 2 147 483 647, и 2 147 483 648 в `int` не влезает). Я проверил такую версию: с `-O2` она на −2147483648 случайно печатает верное `Цифр: 10, сумма цифр: 47`, а собранная с `-fsanitize=undefined` — ругается на отрицание и печатает `Цифр: 0, сумма цифр: 0`. Один и тот же код, два разных ответа — типичное лицо UB. Поэтому модуль мы берём в `long long`: там 2 147 483 648 помещается с запасом. Другой путь — вообще не брать модуль и работать с отрицательными остатками (`-123 % 10` даёт −3, см. каверзу 9). На Junior это редко спрашивают, но упомянуть — большой плюс.
- «Цифровой корень: складывай сумму цифр, пока не останется одна цифра (9875 → 29 → 11 → 2).» Это цикл поверх нашего цикла — частое продолжение. Бонус: цифровой корень равен `1 + (n - 1) % 9` для положительных `n`, и это интервьюеру нравится. Проверим: 9874 = 9 · 1097 + 1, остаток 1, плюс один — 2. Работает, потому что число и сумма его цифр дают одинаковый остаток при делении на 9 (10 = 9 + 1, поэтому каждый разряд «весит» по модулю 9 ровно единицу).
- «Переверни число: 123 → 321.» Тот же цикл, но вместо суммы копим `rev = rev * 10 + digit`. С защитой от переполнения — см. Приложение В, задача 10.

Часть 2. Три классики без отдельного файла

Эти три задачи решаются в несколько строк, и отдельных файлов в `code/` под них нет — весь код прямо здесь. Но спрашивают их не реже остальных.

Задача 9. Проверить чётность числа

Условие. Дано целое число. Определи, чётное оно или нечётное.

Идея. Школьный способ — остаток от деления: `n % 2 == 0` (см. главу 4). Но на собесе часто хотят увидеть **битовый** приём: у чётного числа младший бит равен 0, у нечётного — 1. Проверить младший бит можно операцией `&` (побитовое «И») с единицей.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int n = 7;

    if (n % 2 != 0) {
        printf("%d — нечётное\n", n);
    } else {
        printf("%d — чётное\n", n);
    }
    return 0;
}
```

Запуск:

```
7 — нечётное
```

Разбор. Почему я написал `n % 2 != 0`, а не более привычное `n % 2 == 1`? Потому что для **отрицательных** нечётных `n % 2` даёт не +1, а −1 (см. каверзу 9), и проверка `== 1` молча провалится: −7 будет объявлено чётным. Сравнение с нулём такой ошибки не допускает — от знака оно не зависит.

Битовый вариант выглядит так:

```c
if (n & 1) {
    printf("%d — нечётное\n", n);
}
```

`n & 1` сравнивает биты `n` и числа 1 по позициям. У единицы выставлен только самый младший бит. Поэтому `n & 1` оставляет от `n` ровно этот младший бит: 1 у нечётных, 0 у чётных. В `if` ненулевое — это истина (см. главу 2).

Что спросят дальше. «Чем `n & 1` лучше `n % 2`?» Для положительных чисел — ничем существенным: оба корректны. Проверка `n % 2 != 0` компилятор с `-O2` превращает ровно в ту же команду, что и `n & 1`, — я смотрел ассемблер (`cc -O2 -S`), там одна инструкция `andl $1`. Сам по себе оста-

ток `n % 2` для знакового `n` стоит на пару команд дороже: компилятору приходится поправлять знак. А вот для отрицательных есть тонкость, о которой хорошо бы сказать честно.

Стандарт C11 разрешает три представления знаковых чисел: дополнительный код, обратный код и «знак-величина». Побитовые операции работают с битами представления, поэтому результат `n & 1` для отрицательного `n` формально **implementation-defined** — зависит от платформы. В обратном коде, например, −7 записывается как `...11111000`, и `-7 & 1` дал бы 0 — «чётное». На любой реальной машине представление дополнительное (а с C23 оно и вовсе единственное разрешённое), и `-7 & 1` даёт 1. Но правильный ответ на собеседовании звучит так: «на нашей платформе — 1, потому что дополнительный код; переносимо и без оговорок — `n % 2 != 0`».

Задача 10. Максимум и минимум массива за один проход

Условие. Дан массив целых. Найди наибольший и наименьший элементы за один проход.

Идея. Заводим `max` и `min`, инициализируем их **первым элементом** массива (не нулём! см. подвох ниже) и идём по остальным, обновляя.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int a[] = {4, -2, 9, 1, 7, -8, 3};
    int n = (int)(sizeof(a) / sizeof(a[0]));

    int max = a[0];
    int min = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
        if (a[i] < min) {
            min = a[i];
        }
    }

    printf("min = %d, max = %d\n", min, max);
    return 0;
}
```

Запуск:

```
min = -8, max = 9
```

Разбор. `sizeof(a) / sizeof(a[0])` — это размер всего массива в байтах, делённый на размер одного элемента, то есть **число элементов**. Здесь 28 / 4 = 7. Приём работает, только пока `a` — настоящий массив, а не указатель (см. подвох ниже).

Цикл стартует с `i = 1`, потому что нулевой элемент уже лежит и в `max`, и в `min` — сравнивать его с самим собой незачем.

Сложность. O(n) времени, два сравнения на каждый элемент, кроме первого, то есть 2(n − 1) сравнений — примерно 2n. O(1) памяти.

Что спросят дальше.

- «Почему инициализируешь первым элементом, а не нулём?» Если поставить `max = 0`, а все числа в массиве отрицательные — максимумом ошибочно останется 0, которого в массиве нет. То же с `min` и массивом из одних положительных. Безопасно стартовать значением **из самого массива**.
- «А если массив пустой?» `a[0]` уже выход за границу — UB. Нужна проверка `if (n == 0) return ...;` в самом начале. Пустой ввод — первое, о чём спрашивают.
- «Сделай за 1.5n сравнений.» Есть приём: брать элементы **парами**, сначала сравнивать их между собой, и меньший сравнивать с `min`, а больший с `max`. Получается 3 сравнения на 2 элемента вместо 4, то есть около 3n/2 = 1.5n сравнений на весь массив.
- «А если передать массив в функцию и там взять `sizeof`?» Не сработает! В функции параметр-массив превращается в **указатель**, и `sizeof` вернёт размер указателя (8 байт на этой машине), а не массива. Поэтому длину всегда передают **отдельным параметром**: `void f(int *a, int n)`. Классический подвох — полный разбор с реальным выводом и предупреждением компилятора в Приложении Б, каверза 2.

Задача 11. FizzBuzz

Условие. Числа от 1 до 100. Кратные 3 заменяй на `Fizz`, кратные 5 — на `Buzz`, кратные и 3, и 5 — на `FizzBuzz`, остальные печатай как есть.

Идея. Вся соль — в **порядке** проверок. Самое строгое условие (кратно 15) должно идти первым, иначе число 15 поймается веткой «кратно 3» и выдаст `Fizz` вместо `FizzBuzz` (полный разбор — в главе 4).

Решение (готовый файл — `code/04-fizzbuzz.c` из главы 4):

```
#include <stdio.h>
```

```c
int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

Первые пятнадцать строк вывода:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Разбор. В цепочке `if / else if` срабатывает **первая** подходящая ветка, остальные даже не проверяются. Поэтому «кратно 15» (то же, что «кратно и 3, и 5») ставим раньше «кратно 3» и «кратно 5». Поменяй порядок — и на 15-й строке вместо `FizzBuzz` вылезет `Fizz`. Это ровно та ошибка, ради которой задачу и дают.

Сложность. O(n) — одна итерация на число, внутри константа операций.

Что спросят дальше. «Сделай без `% 15`, только через `% 3` и `% 5`.» Собери строку из флагов:

```
int by3 = (i % 3 == 0);
int by5 = (i % 5 == 0);
if (by3) printf("Fizz");
if (by5) printf("Buzz");
if (!by3 && !by5) printf("%d", i);
printf("\n");
```

Этот вариант легко расширяется (добавить «кратно 7 → Bazz» — две новые строки, `int by7 = ...` и `if (by7) printf("Bazz");`, и `&& !by7` в последнем условии; тогда 105 = 3 · 5 · 7 напечатается как `FizzBuzzBazz`), и интервьюер часто просит именно его: проверить, что ты не зубрил, а понимаешь структуру. Обрати внимание — `printf` без `\n` внутри и один общий перевод строки в конце: так `Fizz` и `Buzz` склеиваются в `FizzBuzz` сами собой. Я сверил: на числах от 1 до 100 обе версии печатают одно и то же, байт в байт.

Часть 3. Каверзы: «что выведет / в чём ошибка»

Здесь короткие фрагменты. На собесе их показывают и спрашивают: «что напечатает?» или «найди ошибку». Цель — не код написать, а проверить, насколько хорошо ты знаешь язык в деталях. Сначала подумай сам, потом читай ответ.

Все фрагменты я запускал, завернув каждый в обычную программу: `#include <stdio.h>`, `#include <limits.h>`, пустая строка, `int main(void)`, `{`, сам фрагмент, `return 0;`, `}`. Файлы назывались `k1.c`, `k2.c` и так далее по номеру каверзы — отсюда имена и номера строк в сообщениях компилятора ниже. Длинные строки сообщений я перенёс, чтобы влезли в страницу.

Важная общая мысль: у части таких вопросов **нет одного правильного ответа**. Если фрагмент содержит неопределённое поведение (UB) или опирается на свойства платформы, ответ «выведет вот столько» просто неверен. Правильный ответ — назвать это своим именем. Ниже я отмечаю каждый такой случай.

Каверза 1. `sizeof('a')` — сколько это?

```
printf("%zu %zu\n", sizeof(int), sizeof('a'));
```

Вопрос. Что выведет? Подсказка-ловушка: `'a'` — это же один символ, значит 1 байт?

Ответ. На этой машине выведет:

```
4 4
```

Сюрприз в том, что в языке **C** символьный литерал `'a'` имеет тип не `char`, а **`int`**! Поэтому `sizeof('a')` равен `sizeof(int)`.

Важно: это особенность именно C. В C++ `'a'` — это `char`, и там `sizeof('a')` равно 1. На собесе по C правильный ответ — «столько же, сколько `sizeof(int)`».

И вторая половина ответа: число 4 — это **не** гарантия стандарта. Стандарт требует только, чтобы `int` вмещал хотя бы числа от −32767 до 32767 (то есть имел минимум 16 бит) и чтобы его диапазон был не уже, чем у `short`. На современных 32- и 64-битных системах `int` — 4 байта, на 8- и 16-битных микроконтроллерах бывает 2 байта. Единственный размер, зафиксированный намертво, — `sizeof(char)`, он **всегда** 1: char и есть единица измерения для `sizeof`.

`%zu` — спецификатор для типа `size_t`, который возвращает `sizeof` (беззнаковый тип для размеров). Печатать его через `%d` формально неверно: `size_t` на 64-битной системе шире `int`, и `printf` возьмёт из аргументов не то количество байт.

Каверза 2. Целочисленное деление `5 / 2`

```c
printf("%d\n", 5 / 2);
double x = 5 / 2;
printf("%f\n", x);
```

Вопрос. Что в обеих строках?

Ответ. Реальный запуск:

```
2
2.000000
```

Первая строка: деление двух **целых** идёт нацело, дробная часть отбрасывается (см. главу 2), а не округляется. `5 / 2` это 2, а не 2.5 и не 3. Кстати, `7 / 2` тоже 3, а не 4 — отбрасывание, не округление.

Вторая печатает `2.000000`, а **не** 2.5, и это главный подвох. Хотя `x` объявлен как `double`, выражение `5 / 2` считается **до** записи в `x`: оба операнда целые → целочисленное деление → 2 → и уже 2 превращается в `2.0`. Тип переменной слева не влияет на то, как вычисляется правая часть. Это общее правило Си, и оно ловит очень многих.

Чтобы получить 2.5, дробным должен быть хотя бы один операнд: `5.0 / 2`, или `(double)5 / 2`, или `5 / 2.0`. Достаточно одного — второй операнд подтянется к `double` автоматически.

Каверза 3. `printf("%d", c)` для `char`

```
char c = 'A';
printf("%d\n", c);
```

Вопрос. Что выведет?

Ответ.

```
65
```

Символ в C — это маленькое целое, его **код** (см. главу 2). Код буквы `A` в таблице ASCII равен 65; стандарт Си ASCII не требует, но на всех привычных машинах кодировка с ним совпадает. Спецификатор `%d` просит напечатать число, поэтому печатается не сама буква, а её код. Хочешь увидеть букву `A` — используй `%c`.

И наоборот полезно помнить: `printf("%c\n", 66)` напечатает `B` — число истолкуется как код символа. На этой двойственности (символ = число) держатся трюки вроде перевода регистра из задачи 6 и перевода символа-цифры в число через `c - '0'`.

Каверза 4. `char` — знаковый или беззнаковый?

```
char c = (char)200;
printf("%d\n", c);
```

Вопрос. Что выведет — 200 или что-то другое?

Ответ. Правильный ответ: **зависит от платформы**, стандарт этого не фиксирует. Тип `char` в Си — не то же самое, что `signed char` или `unsigned char`; это **третий**, отдельный тип, и знаковый он или нет, решает компилятор. Стандарт называет это implementation-defined поведением.

На этой машине (clang, x86-64, macOS) `char` знаковый, и вывод такой:

```
-56
```

Откуда −56? Байт `200` в двоичном виде — `11001000`. В знаковый `char` (от −128 до 127) число 200 не помещается, и что получится при таком преобразовании, стандарт тоже оставляет на усмотрение компилятора. Все распространённые компиляторы просто берут те же 8 бит и читают их в дополнительном коде: старший бит весит не +128, а −128, поэтому −128 + 64 + 8 = −56. То же самое проще: 200 − 256 = −56. Проверить, какой `char` у тебя, можно через `<limits.h>`: у нас `CHAR_MIN = -128`, `CHAR_MAX = 127` — значит, знаковый. На ARM под Linux или на PowerPC ты увидел бы `CHAR_MIN = 0` и вывод `200`. (Не путай с Mac на процессорах Apple: это тоже ARM, но Apple сделала `char` знаковым, как на x86.)

Почему это важно на практике. Все функции из `<ctype.h>` — `isalpha`, `isdigit`, `tolower`, `toupper` — принимают `int` и требуют, чтобы значение было либо `EOF`, либо представимым как `unsigned char`. Передашь туда знаковый `char` с кодом больше 127 (например, байт из UTF-8-строки) — получишь отрицательное число, выход за таблицу и UB:

```
char c = s[i];
if (isalpha(c)) { ... }              /* ОПАСНО: c может быть < 0 */
if (isalpha((unsigned char)c)) { ... }   /* правильно */
```

То же самое с `strcmp`: стандарт требует сравнивать символы **как `unsigned char`**, иначе строки с русскими буквами сравнивались бы не в том порядке. Именно поэтому в Приложении Б в самодельной `my_strcmp` стоит приведение к `unsigned char`.

Хочешь однозначности — пиши `signed char` или `unsigned char` явно. Голый `char` используй только для текста.

Каверза 5. = вместо == в условии

```
int n = 5;
if (n = 0) {
    printf("ноль\n");
} else {
    printf("не ноль\n");
}
printf("n = %d\n", n);
```

Вопрос. Что выведет?

Ответ. Реальный запуск:

```
не ноль
n = 0
```

Здесь стоит один `=` (присваивание), а не `==` (сравнение). Поэтому `n = 0` сначала **кладёт** 0 в `n`, а потом результатом всего выражения становится само присвоенное значение — 0. Ноль в условии — это «ложь», поэтому срабатывает ветка `else`. Заодно `n` теперь равно 0, хотя мы вроде бы только «проверяли» — вторая строка вывода это подтверждает.

Если бы было `if (n = 5)`, условие стало бы истинным (5 — ненулевое), и такие баги особенно коварны: код «как будто работает».

Спасает компилятор. Собери с `-Wall` — и он сам покажет пальцем:

```
k5.c:7:11: warning: using the result of an assignment as a condition
without parentheses [-Wparentheses]
    7 |     if (n = 0) {
      |         ~~^~~
k5.c:7:11: note: place parentheses around the assignment to silence
this warning
    7 |     if (n = 0) {
      |           ^
      |         (    )
k5.c:7:11: note: use '==' to turn this assignment into an equality
comparison
    7 |     if (n = 0) {
      |           ^
      |           ==
1 warning generated.
```

(Первая подсказка — «поставь лишние скобки, если присваивание задумано», вторая — «или ты хотел ==».)

Защитный приём — «йода-условие»: писать `if (0 == n)`; тогда опечатку `0 = n` компилятор отвергнет как ошибку (константе присвоить нельзя). Приём спорный — читается непривычно, — но на собеседовании про него приятно знать.

Каверза 6. Лишняя `;` после `for(...)`

```c
int sum = 0;
for (int i = 1; i <= 5; i++);
{
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Найди ошибку. Что не так?

Ответ. Лишняя точка с запятой сразу после `for(...)`. Эта `;` — **пустое тело цикла**. То есть цикл крутится 5 раз и ничего не делает. А блок в фигурных скобках ниже — это уже **обычный блок**, не связанный с циклом; он выполнится **один раз**.

Хуже того: этот фрагмент вообще не скомпилируется — `i` объявлена внутри `for` и за его пределами не видна. Вот что реально говорит clang:

```
k6.c:9:16: error: use of undeclared identifier 'i'
    9 |         sum += i;
      |                ^
k6.c:7:33: warning: for loop has empty body [-Wempty-body]
    7 |     for (int i = 1; i <= 5; i++);
      |                                 ^
k6.c:7:33: note: put the semicolon on a separate line to silence this
warning
1 warning and 1 error generated.
```

Обрати внимание: компилятор выдал и ошибку, и предупреждение про пустое тело — то есть увидел обе проблемы разом.

Если бы `i` была объявлена снаружи цикла (`int i;` и `for (i = 1; ...)`), код собрался бы — с тем же предупреждением про пустое тело — и напечатал `6`: цикл ничего не сделал, а блок один раз прибавил к нулю значение `i`, оставшееся после цикла (6). Логика при этом всё равно сломана: ждали 1+2+3+4+5 = 15. Я проверил и этот вариант — вывод действительно `6`.

Мораль: точка с запятой сразу после `for`, `while` или `if` — почти всегда ошибка. Настоящий пустой цикл (например, `while (*p++);`) пишут либо с `{}`, либо с `;` на отдельной строке — именно это и советует clang.

Каверза 7. `printf("%d %d", i++, ++i)`

```c
int i = 1;
printf("%d %d\n", i++, ++i);
```

Вопрос. Что выведет?

Ответ. Честный ответ: **неизвестно**, «правильного» числа тут нет. Это **неопределённое поведение** (undefined behavior, UB). В одном выражении переменная `i` меняется дважды (`i++` и `++i`) без точки следования между изменениями. Стандарт C не определяет ни порядок вычисления аргументов `printf`, ни момент применения `++`, — а модификация одного объекта дважды без упорядочивания прямо объявлена неопределённым поведением.

Компилятор про это знает и предупреждает:

```
k7.c:7:24: warning: multiple unsequenced modifications to 'i'
[-Wunsequenced]
    7 |     printf("%d %d\n", i++, ++i);
      |                        ^   ~~
1 warning generated.
```

Если запустить, на этой машине печатается `1 3` — и с `-O0`, и с `-O2`. Но это **не ответ на вопрос**. Другой компилятор, другая версия того же компилятора или другие флаги имеют полное право выдать `2 3`, `1 2` или вообще что угодно: программа некорректна, и обсуждать её вывод бессмысленно.

Если на собесе спрашивают «что выведет» — правильный ответ не «2 3» и не «1 3», а именно: «это UB, результат не определён, так писать нельзя». Интервьюер проверяет, узнаёшь ли ты опасную конструкцию, а не считаешь ли в уме «как у меня на ноутбуке». А если он настаивает, что ответ один-единственный, — покажи ему `-Wunsequenced`.

Лечится разнесением по строкам:

```
int a = i++;   /* теперь порядок однозначен */
int b = ++i;
printf("%d %d\n", a, b);
```

Между полными выражениями (то есть между `;`) точка следования есть, и всё определено: `a` получит 1, `b` — 3.

Каверза 8. Неинициализированная переменная

```
int sum;
for (int i = 1; i <= 5; i++) {
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Что выведет?

Ответ. Заранее **неизвестно**, и это снова UB, а не «какое-то число». `sum` объявлена, но не инициализирована — в локальной переменной лежит мусор (что осталось в этой ячейке стека). К мусору прибавляется 1+2+3+4+5 = 15.

Вот три запуска подряд одной и той же программы, собранной с `-O2`:

```
-1302150376
-1227496680
-1154252008
```

Числа разные при каждом запуске. Собранная с `-O0` та же программа у меня три раза подряд напечатала одно и то же `1250286487` — тоже мусор, просто стабильный. Ни то ни другое не 15, и ни на что из этого нельзя полагаться. Может «повезти» и вывестись 15, если в ячейке был ноль, — и это худший исход: баг спрячется и вылезет у пользователя.

Компилятор ловит это сам:

```
k8.c:8:9: warning: variable 'sum' is uninitialized when used here
[-Wuninitialized]
    8 |         sum += i;
      |         ^~~
k8.c:6:12: note: initialize the variable 'sum' to silence this warning
    6 |     int sum;
      |            ^
      |             = 0
1 warning generated.
```

Фикс — один символ: `int sum = 0;`. Любой накопитель перед циклом обнуляй (сумма — в 0, произведение — в 1). И не игнорируй предупреждения: `-Wall` тут буквально дописал за тебя правильный код.

Отдельно запомни: **глобальные** и `static`-переменные обнуляются автоматически, а локальные — нет. Отсюда и путаница у новичков: «у меня же в прошлый раз работало».

Каверза 9. Остаток от отрицательного числа

```c
printf("%d %d\n", -7 / 2, -7 % 2);
```

Вопрос. Что выведет?

Ответ.

```
-3 -1
```

В современном C (стандарт C99 и новее) деление целых округляется **к нулю**: `-7 / 2` это −3, а не −4. Раньше, в C89, направление округления было отдано на откуп компилятору, — но это уже история.

Остаток подчиняется тождеству `(a / b) * b + a % b == a`. Подставим: `(-3) * 2 + x == -7`, значит `x == -1`. Отсюда `-7 % 2` равен −1, а не +1. Проще запомнить так: **знак остатка совпадает со знаком делимого** (если остаток не ноль). Обрати внимание: это не тот остаток, что в школьной математике. Там $-7 = 2 \cdot (-4) + 1$, остаток 1 — он всегда от 0 до делителя. Оператор `%` в Си — не «математический остаток», а «то, что осталось после деления с отбрасыванием дробной части».

Практический вывод: проверять нечётность через `n % 2 == 1` **опасно** — для отрицательных нечётных получится −1, и проверка провалится. Пиши `n % 2 != 0`. Это прямая связь с задачей 9 про чётность.

Ещё одно применение того же правила — разворот числа в Приложении В (задача 10): там `-123 % 10` даёт −3, и знак сохраняется сам собой, без единого `if`.

Ошибки, на которых сыплются в этих задачах

Собрал в одном месте то, что чаще всего ломается именно в задачах этого уровня. Пробеги глазами перед собеседованием.

- **Накопитель без начального значения.** `int sum;` вместо `int sum = 0;` — и в сумме мусор (каверза 8). Для произведения начальное значение 1, не 0.
- **Старт `max`/`min` с нуля.** На массиве из одних отрицательных максимум окажется нулём, которого в массиве нет. Стартуй элементом массива (задача 10).
- **Забытая проверка `scanf`.** `scanf("%d", &n)` возвращает число прочитанных значений. Не проверил — работаешь с мусором в `n`.
- **`sizeof` массива внутри функции.** Даёт размер указателя, а не массива. Длину передавай параметром.
- **`n % 2 == 1` для отрицательных.** Даёт −1, проверка проваливается (каверза 9).
- **Пустая строка и `strlen(s) - 1` в `size_t`.** Беззнаковое вычитание заворачивается в огромное число, и цикл уходит за границы. Держи индексы в `int` или проверяй пустоту отдельно.
- **`-n` для `INT_MIN`.** Переполнение знакового, то есть UB. Всплывает в «возьми модуль» (задача 8).
- **`ctype.h` без `(unsigned char)`.** UB на байтах с кодом больше 127 (каверза 4).
- **Одна переменная меняется дважды в одном выражении.** UB (каверза 7). Разнеси по строкам.
- **Точка с запятой после `for`/`while`/`if`.** Пустое тело, логика ломается молча (каверза 6).

Тренировочные задачи

Решай без подглядывания в разборы. Каждая — это чуть изменённое условие из тех, что уже разобраны: интервьюеры именно так и «дожимают».

1. Перепиши задачу 3 (палиндром) так, чтобы регистр не имел значения и игнорировались все не-буквы. Проверь на строке `"A man, a plan, a canal: Panama"` — должно получиться «палиндром». Ещё проверки: `"Level"` и `"No 'x' in Nixon"` — палиндромы, `"hello"` и `"ab!"` — нет, пустая строка и `"!!"` — палиндромы (букв нет, сравнивать нечего). Подсказка: сравнивай `tolower((unsigned char)s[left])` и `tolower((unsigned char)s[right])`.

2. Допиши задачу 5 (второй максимум) так, чтобы она правильно работала на массиве `{5, INT_MIN}`. Подсказка: замени сравнение с `INT_MIN` на отдельный флаг. Ожидаемо: для `{5, INT_MIN}` и для `{INT_MIN, 5}` — `Максимум: 5, второй: -2147483648`; для `{5, 5, 5}` и для `{42}` — по-прежнему «Второго по величине нет.»; для массива из задачи — `Максимум: 9, второй: 7`. Учти, что флаг нужен и для максимума: у второго максимума стартового значения «меньше всех» больше нет.
3. Реализуй счётчик цифр из задачи 8 для `long long` и убедись, что для `LLONG_MIN` программа не падает и не выдаёт мусор. Подвох: у `long long` нет типа шире, куда можно убежать от `-LLONG_MIN`, так что модуль не бери, а работай с отрицательными остатками (`n % 10` для отрицательного `n` даёт цифру со знаком минус — возьми её модуль, он от 0 до 9) и цикл веди до `n != 0`. Ожидаемо: для `-9223372036854775808` — 19 цифр, сумма 89; для `9223372036854775807` — 19 цифр, сумма 88; для 0 — 1 цифра, сумма 0.
4. Напиши функцию `int count_words(const char *s)`, которая считает слова, разделённые пробелами. Проверь на строках `""`, `"   "`, `"one"`, `"  dva   slova  "`. Ожидаемо: 0, 0, 1, 2. Подсказка: слово начинается там, где после пробела (или в самом начале строки) идёт не пробел, — как в главе 10.
5. Напиши «обратный FizzBuzz»: программа получает строку `Fizz`, `Buzz`, `FizzBuzz` или число и печатает наименьшее число от 1 до 100, которое ей соответствует. Ожидаемо: `Fizz` → 3, `Buzz` → 5, `FizzBuzz` → 15, `7` → 7. А для `9` ответа нет: число 9 FizzBuzz печатает как `Fizz`, так что строку `9` не выдаёт никогда, — программа должна так и сказать.
6. Возьми задачу 7 (простое число) и верни «наивное» условие `d * d <= n` вместо `d <= n / d`. Собери с `-fsanitize=undefined` и введи 2147483647. Ожидаемо: санитайзер сообщит `signed integer overflow: 46341 * 46341 cannot be represented in type 'int'`, а обычная сборка с `-O2` ответит «не простое», хотя 2147483647 простое. Для всех `n` до 200 000 обе версии отвечают одинаково — ошибка живёт только у самого края диапазона `int`, и поэтому её так трудно поймать тестами.

Мини-резюме

Что стоит унести из этого приложения — не сами решения (их в интернете тысячи), а привычки, которые отличают подготовленного кандидата:

- **Краевые случаи проговаривай вслух**: пустой массив, один элемент, ноль, отрицательное, дубликаты, переполнение. Интервьюер часто ставит плюс именно за то, что ты сам про них вспомнил.
- **Помни про типы и переполнение.** `int` лопается на ~2 млрд; символьный литерал — это `int` в C; деление целых идёт нацело; знаковость `char` зависит от платформы. Эти четыре факта спрашивают чаще всего.

- **Знай про опасные приёмы, но не используй их без нужды.** XOR-своп, арифметический обмен — рассказать можешь, в код кладёшь `temp`.
- **UB называй своим именем.** Увидел `i++` и `++i` в одном выражении, неинициализированную переменную, изменение строкового литерала, переполнение знакового — отвечай «это неопределённое поведение», а не угадывай результат. И отдельно называй implementation-defined: размер `int`, знаковость `char` — тут ответ есть, но он свой на каждой платформе.
- **Компилируй с `-Wall -Wextra`.** Больше половины каверз из третьей части компилятор ловит сам — мы это только что видели по его сообщениям.

Куда дальше

Это был уровень Junior — задачи, где главное показать, что основы языка сидят крепко, а голова замечает краевые случаи. В **Приложении Б (Middle)** мы поднимем планку: связные списки и их развороты, работа с указателями всерьёз, бинарный поиск без ошибок на единицу, битовые приёмы, аккуратная работа с памятью под присмотром санитайзеров, и каверзы потяжелее — про строковые литералы, распад массива в указатель, `const`, утечки и время жизни объектов. Если задачи отсюда даются легко и без подглядывания — ты готов двигаться дальше.

Приложение Б. Задачи с собеседований: Middle

В Приложении А мы размялись на простых задачах уровня Junior. Теперь — ступень выше. Это вопросы, которые задают на собеседовании, когда хотят проверить, что ты понимаешь Си **по-настоящему**, а не просто умеешь печатать `printf`. Здесь всё крутится вокруг пяти тем, на которых ломается большинство:

- **указатели** — кто на что смотрит и кто кого меняет;
- **память** — где живёт строка, когда `malloc`, когда `free`;
- **строки** — массив `char` с `'\0'` на конце (см. главу 3);
- **биты** — числа как набор нулей и единиц;
- **связные списки** — узлы и стрелки `next` (см. главу 11).

Как пользоваться этим приложением

Не читай решение сразу. Прочитай условие, закрой книгу и попробуй сам — хоть на бумаге, хоть в редакторе. На собеседовании ценят не зазубренный код, а ход мысли. Поэтому у каждой задачи есть блок «идея» (как рассуждать) и «доп. вопросы» — то, что интервьюер спросит следом, чтобы понять, не списал ли ты.

Задачи идут от простого к сложному. Сначала — **задачи на код**. У задач 1–8 номер совпадает с номером файла в `code/`: задача 5 — это `code/20-05-binsearch.c`, и так далее. Каждый файл собирается без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 code/20-05-binsearch.c -o binsearch
./binsearch
```

У задачи 9 отдельного файла нет — она в пять строк, код прямо в тексте.

Потом идут **каверзы**: коварные вопросы про язык, где правильный ответ не «что выведет», а «почему так, и можно ли на это полагаться». Часть из них содержит намеренные ошибки

и неопределённое поведение, поэтому в `code/` их нет: в файлах книги лежит только тот код, который можно запускать.

Все выводы и все сообщения компилятора в этом приложении получены реальным запуском: Apple clang 21, x86-64, macOS. Где результат зависит от платформы или от флагов сборки — я это отдельно оговариваю.

Поехали.

Часть 1. Задачи на код

Задача 1. Свои `strlen`, `strcpy`, `strcmp`

Условие. Реализуй три функции стандартной библиотеки `<string.h>` своими руками, без её вызова: длину строки, копирование, сравнение.

Идея. Строка в Си — это массив `char`, заканчивающийся нулевым символом `'\0'` (см. главу 3). Значит, любой обход строки — это «иди по символам, пока не упрёшься в `'\0'`». На этом держатся все три функции.

Решение (`code/20-01-mystr.c`):

```c
#include <stdio.h>

/* Своя длина строки: считаем символы до '\0'. */
size_t my_strlen(const char *s)
{
    size_t n = 0;
    while (s[n] != '\0') {  /* пока не дошли до конца строки */
        n++;
    }
    return n;
}

/* Своё копирование: льём src в dst посимвольно, вместе с '\0'. */
char *my_strcpy(char *dst, const char *src)
{
    char *start = dst;          /* запомнили начало, чтобы вернуть */
    while ((*dst = *src) != '\0') {
        dst++;
        src++;
    }
```

```
    return start;
}

/* Своё сравнение: 0 если равны, иначе разница первых отличий. */
int my_strcmp(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}
```

В файле после этих трёх функций идёт `main`, который их гоняет; я его не привожу, чтобы не отвлекаться, — открой файл и посмотри.

Объяснение.

- `size_t` — специальный беззнаковый тип для размеров и длин. Именно его возвращает настоящая `strlen`. Печатают `size_t` через `%zu`. Стандарт объявляет его сразу в нескольких заголовках, среди них `<stddef.h>`, `<string.h>` и `<stdio.h>`, поэтому одного `#include <stdio.h>` здесь хватает.
- `const char *s` — указатель на строку, которую мы **только читаем**. Слово `const` обещает «я не буду менять то, на что смотрит `s`». Это хороший тон: компилятор подстрахует, и тому, кто вызывает функцию, видно, что аргумент в безопасности. И `const` работает в одну сторону: передать `char *` туда, где ждут `const char *`, можно свободно, а наоборот — нет, clang предупредит: `passing 'const char *' to parameter of type 'char *' discards qualifiers`. Поэтому `const` на входном параметре расширяет круг того, что тебе можно передать, а не сужает.
- В `my_strcpy` трюк в строке `while ((*dst = *src) != '\0')`. Сначала `*dst = *src` копирует один символ **и** возвращает его значение (присваивание в Си — это выражение, а не только действие). Это значение тут же сравниваем с `'\0'`. Получается: скопировали символ; если это был конец строки — выходим, но `'\0'` уже скопирован, что нам и нужно. Скобки вокруг `*dst = *src` обязательны: у `!=` приоритет выше, чем у `=`, и без скобок вышло бы `*dst = (*src != '\0')`, то есть в приёмник попали бы единицы и ноль вместо букв.
- `my_strcpy` возвращает `start` — указатель на начало приёмника. Так делает и стандартная `strcpy`: удобно писать `printf("%s", my_strcpy(buf, "abc"))`.
- В `my_strcmp` идём, пока символы совпадают и строка не кончилась. На первом различии (или на общем конце) выходим и возвращаем разность кодов символов. Если строки равны, оба `*a` и `*b` — это `'\0'`, разность 0.

- Приведение к `unsigned char` в `my_strcmp` не украшательство: `char` бывает знаковым (на этой машине он именно такой — см. Приложение А, каверза 4), и байт с кодом 200 стал бы −56. Тогда строка с русской буквой оказалась бы «меньше» строки с латинской. Стандарт требует сравнивать именно как `unsigned char`, и мы это требование выполняем.

Запуск:

```
my_strlen("Hello") = 5
my_strcpy -> skopiruem
my_strcmp(abc, abc) = 0
my_strcmp(abc, abd) = -1
my_strcmp(abd, abc) = 1
```

−1 получилось как `'c' - 'd'` = 99 − 100. Число могло бы быть и другим — важен только знак.

Сложность. O(n) — это запись «время растёт не быстрее, чем пропорционально n» (подробно — в главе 8). Здесь n — длина строки: `my_strlen` и `my_strcpy` делают ровно n шагов цикла, `my_strcmp` — не больше длины более короткой строки. Дополнительной памяти O(1), то есть несколько переменных, сколько бы длинной ни была строка.

Доп. вопросы интервьюера.

- *Чем опасна `strcpy`?* Она не знает размер приёмника `dst` и пишет, пока в `src` не кончатся символы. Если `dst` короче — затрёт чужую память (переполнение буфера). Это классический источник уязвимостей. Безопаснее `snprintf` или `strncpy` — но у `strncpy` своя ловушка: если источник не влез, она **не** дописывает `'\0'`.
- *Почему `strcmp` возвращает не просто 1/−1, а разность?* Стандарт гарантирует только знак результата (меньше/равно/больше нуля), не само число. Полагайся на знак, не на конкретную величину: разные реализации вернут разное.
- *Что будет, если `dst` и `src` в `strcpy` перекрываются?* UB. Для перекрывающихся областей есть `memmove`, а `memcpy` и `strcpy` этого не допускают.

Задача 2. Развернуть односвязный список

Условие. Дан односвязный список `1 -> 2 -> 3 -> ... -> NULL`. Разверни его, чтобы стало `... -> 3 -> 2 -> 1 -> NULL`. На месте, без выделения нового списка.

Идея. У каждого узла есть стрелка `next`, смотрящая вперёд. Развернуть список — значит у каждого узла **перенаправить стрелку назад**. Идём по списку и на каждом шаге разворачиваем одну стрелку. Но если просто написать `cur->next = prev`, мы потеряем доступ к хвосту — поэтому хвост надо запомнить **до** разворота. Нужны три указателя: `prev` (узел позади), `cur` (текущий), `next` (запасённый хвост).

Нарисуем. Было:

```
prev      cur
 |         |
 ▼         ▼
NULL    [1|·] ──► [2|·] ──► [3|·] ──► NULL
```

Запомнили `next = cur->next` (это `[2]`), развернули стрелку `cur` на `prev`:

```
        prev   next
          |     |
          ▼     ▼
NULL ◄── [1|·] [2|·] ──► [3|·] ──► NULL
```

Сдвинули `prev` и `cur` на шаг вперёд (`prev = cur; cur = next`) и повторяем. Когда `cur` станет `NULL`, `prev` будет указывать на бывший последний узел — это новая голова.

Решение (`code/20-02-reverse-list.c`, привожу ядро):

```c
struct Node {
    int value;
    struct Node *next;
};

/* Разворот списка через три указателя: prev, cur, next. */
struct Node *reverse(struct Node *head)
{
    struct Node *prev = NULL;       /* позади ничего нет */
    struct Node *cur = head;        /* бегунок по списку */
    while (cur != NULL) {
        struct Node *next = cur->next;  /* запомнили хвост */
        cur->next = prev;               /* развернули стрелку назад */
        prev = cur;                     /* prev сдвинулся вперёд */
        cur = next;                     /* cur сдвинулся вперёд */
    }
    return prev;                    /* prev — новая голова */
}
```

Объяснение. Порядок четырёх строк внутри цикла — священен.

1. `next = cur->next` — сохранили, куда идти дальше, пока стрелка цела.
2. `cur->next = prev` — развернули стрелку текущего узла назад.
3. `prev = cur` — `prev` догнал `cur`.

4. `cur = next` — `cur` шагнул на сохранённый хвост.

Поменяешь местами хоть две — либо потеряешь хвост, либо зациклишь узел сам на себя. Например, если поставить `cur->next = prev` перед сохранением `next`, то `cur->next` уже перезаписан и хвост списка недостижим: остальные узлы утекут.

Функция возвращает `prev`, потому что после цикла `cur` уже `NULL`, а `prev` стоит на последнем обработанном узле — он и есть новая голова. В `main` обязательно перезаписываем голову: `head = reverse(head)`. Забудешь — и старая `head` будет указывать на узел, у которого `next` теперь `NULL`: список «схлопнется» в один элемент, а остальные утекут.

`->` — это сокращение: `cur->next` то же самое, что `(*cur).next`. Сначала разыменовать указатель, потом взять поле.

Сложность. Один проход — O(n) по времени, O(1) по памяти: три указателя, новый список не выделяем, `malloc` ни разу не зовём.

Краевые случаи. Пустой список (`head == NULL`): цикл не выполнится, вернём `NULL` — верно. Список из одного узла: одна итерация, `prev` станет этим узлом, его `next` станет `NULL` — тоже верно.

Запуск:

```
До:    5 -> 4 -> 3 -> 2 -> 1 -> NULL
После: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
```

Список собран через `push_front` в цикле `for (i = 1; i <= 5; i++)`, поэтому «до» он идёт от 5 к 1 — вставка в начало переворачивает порядок сама по себе.

Доп. вопросы.

- *Как развернуть рекурсивно?* Можно (см. главу 9 про рекурсию), но тогда расход памяти O(n) на стек вызовов, и на длинном списке возможно переполнение стека. Итеративная версия лучше.
- *Как развернуть только кусок списка, с позиции m по n?* Тот же приём, но нужно запомнить узел перед `m` и узел `m` (он станет хвостом перевёрнутого куска), а после разворота сшить три части.
- *А если список двусвязный?* Там достаточно у каждого узла поменять местами `prev` и `next` и переставить голову с хвостом.

Задача 3. Два указателя: середина списка, цикл и его начало

Все три части этой задачи решает один и тот же приём — **два указателя разной скорости**, — и все они живут в одном файле `code/20-03-floyd.c`. Спрашивают их обычно подряд: сначала середину, потом «а теперь найди цикл», потом «а где он начинается».

3а. Середина списка

Условие. Найди средний узел односвязного списка за один проход, не зная заранее его длину.

Идея. Самый прямой способ — пройти список дважды: сначала посчитать длину, потом дойти до середины. Но можно за **один** проход трюком «два указателя разной скорости». Медленный `slow` шагает по одному узлу, быстрый `fast` — по два. Когда `fast` добежит до конца, `slow` будет ровно посередине: он прошёл вдвое меньше.

Решение:

```c
/* Середина списка: медленный шагает на 1, быстрый на 2. */
struct Node *middle(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          /* +1 */
        fast = fast->next->next;    /* +2 */
    }
    return slow;
}
```

Объяснение. Главное здесь — условие цикла `fast != NULL && fast->next != NULL`. Быстрый прыгает через два узла, и перед прыжком надо убедиться, что **оба** узла существуют: и сам `fast`, и `fast->next`. Иначе `fast->next->next` разыменует `NULL` — падение.

Порядок проверок важен: `&&` вычисляет левую часть первой, и если `fast` уже `NULL`, до `fast->next` дело не дойдёт. Это **укороченное вычисление** (short-circuit, см. главу 2): правая часть `&&` не вычисляется, если левая уже дала «ложь». Переставь проверки местами — и получишь разыменование `NULL` на списке чётной длины.

Проследим на списке из 7 узлов (`1 2 3 4 5 6 7`):

```
старт:  slow=1  fast=1
шаг 1:  slow=2  fast=3
шаг 2:  slow=3  fast=5
шаг 3:  slow=4  fast=7   → fast->next == NULL, цикл кончился
```

`slow` стоит на четвёртом узле — это и есть середина семи элементов.

Сложность. O(n) времени, O(1) памяти. Посчитаем точнее: для 7 узлов быстрый сделал 3 прыжка по 2 узла (6 переходов по `next`), медленный — 3 перехода, всего 9. Вариант «сначала посчитай длину, потом дойди до середины» делает 6 переходов на подсчёт и 3 на второй проход —

тоже 9. Так что по числу переходов выигрыша нет, оба около `1.5·n`. Трюк ценен другим: он работает за **один** проход, и его можно применить там, где второго прохода не будет (например, список достраивается на лету). На собесе скажи об этом честно — интервьюер оценит, что ты не повторяешь миф «один проход всегда быстрее двух».

Доп. вопросы.

- *А если узлов чётное число?* Тогда `slow` остановится на «правом из двух центральных»: для `1 2 3 4 5 6` это 4. Хочешь левый (3) — крути цикл, пока `fast->next != NULL && fast->next->next != NULL`. Я прогнал оба варианта на списках длиной от 1 до 8: на нечётных длинах они совпадают, на чётных дают правый и левый центр. Только не забудь отдельно проверить пустой список: в этой версии `fast->next` вычисляется сразу, и при `head == NULL` это разыменование `NULL`.
- *А пустой список?* В нашей версии `slow` останется `NULL`, вызывающий должен это проверить перед `->value`.

3б. Цикл в списке (черепаха и заяц, алгоритм Флойда)

Условие. Список может быть «закольцован»: последний узел указывает не на `NULL`, а на один из предыдущих. Обычный обход по такому списку зациклится навечно. Определи, есть ли в списке цикл.

Идея. Та же пара указателей. Заяц (`fast`) бежит вдвое быстрее черепахи (`slow`). Если список заканчивается `NULL` — заяц добежит до конца, цикла нет. Если есть кольцо — заяц никогда не выйдет, он начнёт бегать по кругу и рано или поздно **догонит** черепаху сзади. Как на круговой беговой дорожке: быстрый бегун обязательно настигнет медленного. Момент `slow == fast` и означает цикл.

Решение:

```c
/* Обнаружение цикла: черепаха и заяц (Флойд). 1 если цикл есть. */
int has_cycle(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {         /* заяц догнал черепаху */
            return 1;
        }
    }
```

```
    return 0;                       /* быстрый упёрся в NULL — цикла нет */
}
```

Объяснение. Сравниваем именно **указатели** (`slow == fast`), то есть адреса узлов, а не их значения: важно, что оба бегунка попали в один и тот же узел. Два разных узла могут хранить одинаковое число — это ничего не значит.

Почему они обязательно встретятся. Введём два числа. Пусть μ («мю») — сколько шагов от головы до первого узла кольца, а λ («лямбда») — длина кольца в узлах. В нашем примере ниже список `1 2 3 4 5 6 7`, и узел 7 смотрит на узел 3: μ = 2 (от 1 до 3 два шага), λ = 5 (узлы 3, 4, 5, 6, 7).

Через μ шагов черепаха входит в кольцо. Заяц к этому моменту уже внутри и стоит где-то на круге. Будем мерить, на сколько узлов заяц **отстаёт** от черепахи, если идти по кольцу вперёд: это число от 0 до λ − 1. За один шаг черепаха уходит на 1 узел, а заяц — на 2, то есть отставание уменьшается **ровно на 1**. Число, которое каждый шаг уменьшается на 1, не может перепрыгнуть через 0: за не больше чем λ − 1 шагов оно станет нулём, и указатели совпадут. Значит, встреча случится не позже, чем через μ + λ шагов от старта, — по времени это O(n), где n — число узлов. Я проверил это программой на всех списках до 40 узлов, для каждого возможного места закольцовки: встреча всегда наступала не позже шага μ + λ.

Проверка `slow == fast` стоит **после** сдвигов. Поставь её до — и цикл вернёт 1 сразу на старте, когда оба указателя ещё на голове.

3в. Где начинается цикл

Условие. Список закольцован. Найди узел, с которого начинается кольцо (в нашем примере — узел 3).

Идея. Вторая фаза того же алгоритма: после встречи поставь один указатель обратно в голову, второй оставь на месте встречи и двигай оба **по одному** узлу. Место, где они совпадут, — начало кольца.

Решение:

```
/* Начало цикла (вторая фаза Флойда) или NULL, если цикла нет. */
struct Node *cycle_start(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
```

```
        fast = fast->next->next;
        if (slow == fast) {          /* фаза 1: встретились внутри кольца */
            struct Node *p = head;   /* фаза 2: второй бегунок — в голову */
            while (p != slow) {      /* оба шагают по одному узлу */
                p = p->next;
                slow = slow->next;
            }
            return p;                /* встретились ровно в начале кольца */
        }
    }
    return NULL;
}
```

Почему это работает. Пусть до встречи черепаха сделала k шагов. Заяц за то же время сделал 2k шагов, то есть на k шагов больше. Оба стоят в одном узле, значит, эти лишние k шагов заяц потратил на целые круги по кольцу: k делится на λ без остатка.

Теперь посмотри на черепаху: она стоит в узле, до которого от головы k шагов. Сделаем ей ещё μ шагов — получится k + μ шагов от головы. Первые μ из них приводят к началу кольца, а оставшиеся k — это целое число кругов, после которых возвращаешься туда же. Значит, через μ шагов черепаха будет **в начале кольца**. Указатель `p`, стартовавший из головы, тоже через μ шагов окажется в начале кольца — по определению μ. Раньше они совпасть не могут: пока `p` не дошёл до кольца, он снаружи, а черепаха внутри. Вот они и встречаются ровно на первом узле кольца.

Проверим на нашем примере: μ = 2, λ = 5. Первая фаза: черепаха делает шаги 1→2→3→4→5→6, заяц 1→3→5→7→4→6 — встреча в узле 6 после k = 5 шагов (и 5 делится на λ = 5). Вторая фаза: `p` идёт 1→2→3, черепаха 6→7→3 — через 2 шага (это μ) оба в узле 3. Верно.

В программе мы сначала проверяем нормальный список (цикла нет), потом вручную закольцовываем его (последний узел направляем на третий) и проверяем снова. Полный вывод файла `20-03-floyd.c`:

```
Середина списка из 7: 4
Цикл есть? 0
После закольцовки цикл есть? 1
Цикл начинается в узле 3
```

Важная деталь из кода: перед `free` цикл надо **разорвать** (`last->next = NULL`). Иначе `free_list` после узла 7 перейдёт на уже освобождённый узел 3: сначала прочитает его поле `next` (use-after-free), потом освободит его второй раз (double free) — оба бага разобраны в каверзе 7.

Сложность. O(n) по времени (обе фазы — не больше μ + λ шагов каждая), O(1) по памяти. Альтернатива — хранить посещённые узлы в хеш-таблице: тоже O(n) времени в среднем, но O(n) памяти. На собесе стоит назвать оба варианта и объяснить, почему выбираешь Флойда.

Доп. вопросы.

- *Как узнать длину цикла?* После встречи оставь один указатель на месте и гони второй по кольцу по одному узлу, считая шаги до возврата. В нашем примере насчитаешь 5.
- *А если список длинный, но цикла нет — не зря ли мы гоняем зайца?* Нет: заяц дойдёт до `NULL` за n/2 прыжков, это те же O(n).

Задача 4. Биты: popcount и степень двойки

Два вопроса, которые идут парой и держатся на одном и том же приёме. Файл — `code/20-04-popcount.c`.

4a. Подсчёт единичных битов (popcount)

Условие. Дано целое число. Посчитай, сколько единиц в его двоичной записи. Например, 7 это `111` — три единицы; 8 это `1000` — одна.

Идея. Наивно: смотрим младший бит (`n & 1`), прибавляем его к счётчику, сдвигаем число вправо (`n >> 1`), повторяем, пока число не обнулится. Это делает столько шагов, сколько цифр в двоичной записи числа — до старшей единицы включительно. У `1024` (`10000000000`) это 11 шагов, у `0xFFFFFFFF` — все 32. Красивее — **приём Брайана Кернигана**: операция `n &= n - 1` за один раз **гасит самый младший единичный бит**. Тогда число итераций равно числу единиц, а не общему числу битов.

Почему `n & (n - 1)` убирает младшую единицу? Вычитание единицы переворачивает младший единичный бит в 0, а все нули правее него — в единицы. Например, `n = 1100`, `n - 1 = 1011`. Их `&` даёт `1000` — младшая единица исчезла, старшие целы:

```
  1100   n
  1011   n - 1
& ————
  1000   n & (n - 1)
```

Решение:

```c
/* Наивный popcount: проверяем младший бит и сдвигаем вправо. */
int popcount_naive(unsigned int n)
{
    int count = 0;
```

```
    while (n != 0) {
        count += n & 1u;    /* младший бит: 0 или 1 */
        n >>= 1;            /* сдвинули число на один бит вправо */
    }
    return count;
}

/* Приём Брайана Кернигана: n &= n - 1 гасит младший единичный бит. */
int popcount_kernighan(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        n &= n - 1;         /* убрали одну единицу */
        count++;            /* итераций ровно столько, сколько единиц */
    }
    return count;
}
```

Объяснение.

- Аргумент — `unsigned int` (беззнаковое). Битовые трюки и сдвиги надёжнее всего делать на беззнаковых типах. У знаковых есть две ямы (стандарт C11, раздел 6.5.7): сдвиг вправо отрицательного числа — implementation-defined, то есть каждый компилятор сам решает, тянуть ли знаковый бит, и описывает это в документации; а сдвиг **влево** отрицательного числа — UB. С `unsigned` таких вопросов нет: всё определено, а выход за границу честно заворачивается по модулю 2 в степени N, где N — число битов типа. У `unsigned int` на этом маке N = 32 (стандарт гарантирует только «не меньше 16»), то есть по модулю 2^{32} = 4 294 967 296.
- `n & 1u` — побитовое И с единицей оставляет только младший бит: результат 0 или 1. Суффикс `u` у литерала `1u` подчёркивает, что работаем с беззнаковым, и убирает лишние вопросы о приведении типов.
- `n >>= 1` — то же, что `n = n >> 1`: сдвиг всех битов на одну позицию вправо, как деление на 2 без остатка.
- `n &= n - 1` — то же, что `n = n & (n - 1)`: гасим младшую единицу.

Запуск (файл гоняет обе функции и заодно проверку из пункта 4б):

```
    0: бит=0 (Керниган=0), степень_двойки=0
    1: бит=1 (Керниган=1), степень_двойки=1
    2: бит=1 (Керниган=1), степень_двойки=1
```

```
    7: бит=3 (Керниган=3), степень_двойки=0
    8: бит=1 (Керниган=1), степень_двойки=1
  255: бит=8 (Керниган=8), степень_двойки=0
 1024: бит=1 (Керниган=1), степень_двойки=1
```

Обе функции дают одно число, но версия Кернигана для числа `1024` (это `10000000000`) сделает **один** виток цикла, а наивная — одиннадцать.

Сложность. Наивная делает столько итераций, сколько двоичных цифр в числе (до старшей единицы): от 0 для нуля до 32 для чисел с выставленным старшим битом. Керниган делает столько итераций, сколько в числе единиц: тоже от 0 до 32, но никогда не больше наивной. На «разреженных» числах вроде `1024` разница заметна (1 против 11), на `0xFFFFFFFF` обе делают 32 шага.

Доп. вопросы.

- *Есть ли это в железе?* Да, у многих процессоров есть инструкция popcount; в GCC/Clang к ней даёт доступ встроенная функция `__builtin_popcount` (это расширение компилятора, не стандартный Си). Я посмотрел, во что clang на этом маке превращает её с `-O2`: без дополнительных флагов — в десяток сдвигов и масок, а с флагом `-mpopcnt` — в одну инструкцию `popcntl`. Работает она в обоих случаях.
- *Как посчитать единицы в `unsigned long long`?* Тот же код, только параметр типа `unsigned long long` и литерал `1ull`. Счётчик `int` спокойно вмещает 64.

4б. Степень ли двойки

Условие. Проверь, является ли число степенью двойки (1, 2, 4, 8, 16, ...).

Идея. У степени двойки в двоичной записи **ровно один** единичный бит: `1`, `10`, `100`, `1000`. А мы только что видели, что `n & (n - 1)` гасит младшую единицу. Если единица была одна-единственная — после гашения останется 0. Значит, проверка: `n != 0 && (n & (n - 1)) == 0`.

Решение:

```c
/* Степень двойки: ровно один единичный бит, и число положительное. */
int is_power_of_two(unsigned int n)
{
    return n != 0 && (n & (n - 1)) == 0;
}
```

Объяснение. Условие `n != 0` обязательно. Без него ноль прошёл бы проверку: `0 - 1` для беззнакового заворачивается в самое большое значение типа, `UINT_MAX` (на этом маке `0xFFFFFFFF`),

и `0 & 0xFFFFFFFF` равно 0, так что `is_power_of_two(0)` ошибочно вернуло бы «истина». Ноль не степень двойки, поэтому отсекаем его явно.

Результат функции — значение логического выражения: в Си это честные 1 (истина) или 0 (ложь), поэтому можно смело писать `return` без всякого `if`.

Сложность. O(1) — несколько операций, без циклов.

Доп. вопросы.

- *Почему не считать popcount и не сравнивать с 1?* Можно, но это цикл вместо трёх операций.
- *А для знакового `int`?* Тогда сначала отсекай `n <= 0`. Главная ловушка — `INT_MIN` (−2147483648): в дополнительном коде это ровно один выставленный бит, старший, и «беззнаковый» трюк объявил бы его степенью двойки. А в знаковом `int` вычисление `n - 1` для `INT_MIN` — переполнение, то есть UB. Условие `n > 0 && (n & (n - 1)) == 0` закрывает обе дыры: до `n - 1` дело доходит только для положительных.
- *Как округлить до ближайшей степени двойки вверх?* Серия сдвигов и `|`, либо инструкция подсчёта ведущих нулей (`__builtin_clz`).

Задача 5. Корректный бинарный поиск

Условие. Реализуй бинарный поиск в отсортированном массиве. Главная ловушка — ошибки «на единицу» (off-by-one): неверная граница цикла или неверный сдвиг `lo`/`hi` ведут к зацикливанию или пропуску элемента.

Идея. Держим отрезок `[lo, hi]`, где может находиться искомое. Смотрим середину. Угадали — готово. Если в середине меньше искомого — всё левее бесполезно, двигаем `lo`. Если больше — двигаем `hi`. Отрезок сжимается, пока не найдём или пока он не опустеет.

Решение (`code/20-05-binsearch.c`, ядро):

```
/* Бинарный поиск в отсортированном массиве.
   Возвращает индекс target или -1, если не найдено. */
int binary_search(const int *a, int n, int target)
{
    int lo = 0;
    int hi = n - 1;             /* оба конца ВКЛЮЧИТЕЛЬНО */
    while (lo <= hi) {          /* пока отрезок непустой */
        int mid = lo + (hi - lo) / 2;   /* середина без переполнения */
        if (a[mid] == target) {
            return mid;
        } else if (a[mid] < target) {
```

```
            lo = mid + 1;       /* искомое правее */
        } else {
            hi = mid - 1;       /* искомое левее */
        }
    }
    return -1;
}
```

Объяснение. Три места, где обычно ошибаются, и как мы их обходим:

- **Границы включительные.** `hi = n - 1`, и цикл идёт, пока `lo <= hi` (не <). Когда `lo == hi`, отрезок ещё содержит один элемент — его нужно проверить. Напишешь `lo < hi` — пропустишь этот последний элемент, и поиск будет иногда врать «не найдено».
- **Сдвиг с +1/−1.** После проверки середины сам `mid` уже исключён, поэтому новые границы — `mid + 1` и `mid - 1`, а не `mid`. Если сдвигать на `mid`, то на отрезке из двух элементов `mid` всё время равен `lo`, границы не двигаются и цикл виснет навсегда.
- **Середина без переполнения.** Пишем `lo + (hi - lo) / 2`, а не `(lo + hi) / 2`. На больших индексах сумма `lo + hi` может переполнить `int` — а это UB, и обычно результат становится отрицательным, что даёт `a[-1]` и выход за границы. Разность `hi - lo` неотрицательна и безопасна. Классический баг: он много лет жил в `Arrays.binarySearch` стандартной библиотеки Java, пока его не исправили в 2006 году.

Пустой массив обрабатывается сам собой: `n = 0` даёт `hi = -1`, условие `0 <= -1` ложно, вернём −1.

Запуск (массив `{-7, -3, 0, 2, 5, 8, 11, 17, 42}`):

```
   5 найдено на индексе 4
  42 найдено на индексе 8
  -7 найдено на индексе 0
 100 не найдено
   0 найдено на индексе 2
```

Обрати внимание на границы: находятся и первый элемент (−7), и последний (42) — это ровно те две позиции, которые чаще всего теряются при off-by-one. А `100`, которого в массиве нет, честно даёт «не найдено», а не индекс наугад.

Сложность. O(log n): каждый шаг вдвое урезает отрезок. Точнее, в худшем случае цикл делает $\lfloor \log_2 n \rfloor + 1$ итераций (формулу и логарифм мы разбирали в главе 7). Для миллиона элементов: $2^{19} = 524\,288 \le 1\,000\,000 < 2^{20}$, значит $\lfloor \log_2 n \rfloor = 19$ и итераций не больше 20. Для миллиарда: $2^{29} \le 10^9 < 2^{30}$, итераций не больше 30. Память O(1).

Доп. вопросы.

- *Что вернуть, если в массиве есть дубликаты искомого?* Базовая версия вернёт «какой-то» из них. Чтобы найти первый или последний, цикл меняют так, чтобы он не останавливался на совпадении, а продолжал сужать сторону.
- *А если массив не отсортирован?* Алгоритм тихо вернёт мусор. Бинарный поиск не проверяет отсортированность — это предусловие, о котором надо сказать вслух.
- *Есть ли готовая функция?* Есть `bsearch` в `<stdlib.h>` — принимает компаратор, как `qsort` (см. главу 8).

Задача 6. Слияние двух отсортированных массивов

Условие. Даны два отсортированных массива. Слей их в один отсортированный. Это сердце сортировки слиянием (merge sort).

Идея. Идём по обоим массивам сразу, двумя индексами. Сравниваем текущие элементы и в результат кладём **меньший**, продвигая его индекс. Так — пока один из массивов не кончится. Остаток второго дописываем как есть: он уже отсортирован.

Решение (`code/20-06-merge.c`, ядро):

```c
/* Слияние двух отсортированных массивов a и b в результат out.
   out должен вмещать na + nb элементов. */
void merge(const int *a, int na, const int *b, int nb, int *out)
{
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {       /* пока есть в обоих */
        if (a[i] <= b[j]) {
            out[k++] = a[i++];       /* меньший идёт первым */
        } else {
            out[k++] = b[j++];
        }
    }
    while (i < na) {                 /* хвост a, если остался */
        out[k++] = a[i++];
    }
    while (j < nb) {                 /* хвост b, если остался */
        out[k++] = b[j++];
    }
}
```

Объяснение. Запись `out[k++] = a[i++]` делает три вещи разом: кладёт `a[i]` в `out[k]`, увеличивает `i` и увеличивает `k`. Постфиксный `++` сначала отдаёт старое значение, а потом увеличивает переменную — поэтому индексы в самом присваивании ещё старые (подробный разбор всех четырёх форм `++` с указателями — в каверзе 4).

Сравнение `a[i] <= b[j]` именно «меньше или равно», а не «меньше». Это делает слияние **устойчивым** (стабильным): при равенстве берём элемент из первого массива, и равные сохраняют исходный относительный порядок. Для голых чисел разницы не видно, но если бы мы сливали записи с полями (фамилия + возраст), стабильность стала бы заметна сразу.

Два хвостовых цикла дописывают то, что осталось. Сработает ровно один из них: после первого цикла хотя бы один массив исчерпан.

Ответственность за размер `out` — на вызывающем: там должно быть место под `na + nb` элементов. В `main` массив `out[11]` именно такой (5 + 6 = 11). Если ошибиться на единицу, получишь запись за границу массива — тихое UB, которое поймает только санитайзер.

Запуск (`{1,3,5,7,9}` и `{2,4,6,8,10,12}`):

```
1 2 3 4 5 6 7 8 9 10 12
```

Сложность. O(na + nb) по времени — каждый элемент копируем ровно один раз, а сравнений не больше na + nb − 1: каждое сравнение отправляет в `out` один элемент, а последний элемент всегда уходит без сравнения. Память — под результат, O(na + nb).

Можно ли слить на месте, без второго массива, и всё равно за линейное время? Теоретически да: такие алгоритмы есть (первый опубликовал Кронрод в 1969 году), но они настолько сложны и у них такие большие постоянные множители, что на собеседовании достаточно сказать «существуют, но обычно берут буфер».

Доп. вопросы.

- *Как слить «на месте» в конец первого массива (классическая задача merge sorted array)?* Идти **с конца** обоих массивов, заполняя результат справа налево — тогда не затрёшь ещё не прочитанные элементы.
- *А если массивов k штук?* Тогда берут кучу (min-heap) из k голов: O(N log k) вместо O(N · k).

Задача 7. Удалить дубликаты из отсортированного массива на месте

Условие. Дан отсортированный массив, например `[1, 1, 2, 2, 2, 3, 5, 5, 8]`. Убери повторы **на месте**, не выделяя второй массив. Верни новую длину; уникальные элементы должны оказаться в начале.

Идея. Раз массив отсортирован, все одинаковые элементы лежат рядом. Заведём два индекса (приём «два указателя» в одном массиве): `r` (reader) бежит по всем элементам, `w` (writer) показы-

вает, куда положить следующий уникальный. Если `a[r]` отличается от последнего записанного `a[w-1]` — это новый уникальный, копируем его в `a[w]` и двигаем `w`.

Решение (`code/20-07-dedup.c`, ядро):

```c
/* Удаление дубликатов из ОТСОРТИРОВАННОГО массива на месте.
   Возвращает новую длину; уникальные элементы — в начале a. */
int dedup_sorted(int *a, int n)
{
    if (n == 0) {
        return 0;
    }
    int w = 1;                  /* w — куда писать следующий уникальный */
    for (int r = 1; r < n; r++) {   /* r — что читаем */
        if (a[r] != a[w - 1]) {     /* отличается от последнего записанного */
            a[w] = a[r];
            w++;
        }
    }
    return w;
}
```

Объяснение. Первый элемент `a[0]` уникален всегда, поэтому стартуем с `w = 1` и `r = 1`.

Сравниваем именно с `a[w - 1]` — последним, что мы уже оставили, а не с соседом `a[r - 1]`. На серии `1 1 1 2` разницы нет, но общая привычка сравнивать с «последним записанным» переносится на задачи, где допускают не более k повторов, и там `a[r-1]` уже не годится.

Проверка `n == 0` нужна, иначе `a[w - 1]` при пустом массиве полез бы в `a[-1]` — выход за границу и UB.

Прогоним `{1, 1, 2, 2, 2, 3, 5, 5, 8}` руками:

```
r=1  a[1]=1 == a[0]=1   пропуск          w=1
r=2  a[2]=2 != a[0]=1   a[1]=2           w=2
r=3  a[3]=2 == a[1]=2   пропуск          w=2
r=4  a[4]=2 == a[1]=2   пропуск          w=2
r=5  a[5]=3 != a[1]=2   a[2]=3           w=3
r=6  a[6]=5 != a[2]=3   a[3]=5           w=4
r=7  a[7]=5 == a[3]=5   пропуск          w=4
r=8  a[8]=8 != a[3]=5   a[4]=8           w=5
```

Запуск:

```
Уникальных: 5
1 2 3 5 8
```

Сложность. O(n) времени, O(1) дополнительной памяти — записи идут в тот же массив, и `w` никогда не обгоняет `r`, поэтому мы не затираем непрочитанное.

Доп. вопросы.

- *Почему важно, что массив отсортирован?* Только тогда дубликаты рядом и хватает сравнения с предыдущим уникальным. Для неотсортированного понадобится множество (хеш) — O(n) памяти, либо сначала сортировка за O(n log n).
- *Что в хвосте массива после удаления?* Старый мусор за позицией `w`. Мы его не трогаем — вызывающий пользуется только первыми `w` элементами. Если это важно, хвост можно занулить.
- *А если разрешено оставлять не более двух повторов?* Тогда сравнивай с `a[w - 2]` и стартуй с `w = 2` и `r = 2`, а массив из двух и меньше элементов возвращай как есть (`if (n <= 2) return n;`). На `{1, 1, 1, 2, 2, 2, 3}` получится 5 элементов: `1 1 2 2 3`.

Задача 8. Число, встречающееся нечётное число раз (XOR)

Условие. В массиве все числа встречаются **парами**, и только одно — нечётное число раз. Найди его. Желательно за один проход и без лишней памяти.

Идея. Спасает операция XOR (исключающее ИЛИ, `^`). Она сравнивает числа бит за битом: в результате стоит 1 там, где биты **разные**, и 0 там, где одинаковые. Например, `6 ^ 3`: `110 ^ 011 = 101`, то есть 5. У XOR два золотых свойства: `x ^ x == 0` (у числа с самим собой все биты одинаковые — одни нули) и `x ^ 0 == x` (ноль ничего не меняет). Проксорим **все** элементы подряд. Парные погасят друг друга и дадут 0, а одиночка останется — это и есть ответ. Порядок неважен: XOR коммутативен и ассоциативен, поэтому пары «найдут друг друга» независимо от того, где они лежат.

Решение (`code/20-08-oddtimes.c`, ядро):

```c
/* Число, встречающееся нечётное число раз.
   XOR всех элементов: парные гасят друг друга, остаётся одиночка. */
int find_odd(const int *a, int n)
{
    int x = 0;
    for (int i = 0; i < n; i++) {
        x ^= a[i];      /* a ^ a == 0, x ^ 0 == x */
    }
```

```
    return x;
}
```

Объяснение. `x ^= a[i]` — то же, что `x = x ^ a[i]`. Стартуем с `x = 0`, потому что 0 — нейтральный элемент для XOR (как 0 для сложения и 1 для умножения). После прохода все, кто шёл парами, взаимно уничтожились, в `x` лежит число с непарным количеством.

XOR даёт 1 в тех битах, где операнды различаются. Поэтому `x ^ x` — это все нули, а `x ^ 0` оставляет `x` как есть.

Мелочь, о которой спросят: XOR на **знаковых** `int` работает с битами представления числа. Здесь это безопасно: на всех современных машинах отрицательные числа хранятся в дополнительном коде (в стандарте C23 это стало обязательным), и XOR двух настоящих `int` всегда даёт настоящий `int`. Переполнения у XOR не бывает в принципе: он не переносит единицы из разряда в разряд, как сложение.

Запуск (в массиве `4` встречается 3 раза, остальные парами):

```
Нечётное число раз встречается: 4
```

Сложность. O(n) времени, O(1) памяти — одна переменная. Наивный подход «для каждого посчитать вхождения» — $O(n^2)$, подход через хеш — O(n) времени и O(n) памяти. XOR выигрывает у обоих.

Доп. вопросы.

- *А если два числа непарные?* Тогда общий XOR даст их «смесь». Их можно разделить: взять любой выставленный бит в этой смеси (например, младший), разбить массив на две группы по этому биту и проксорить каждую отдельно. Младший бит достают выражением `x & -x`, но считай его в `unsigned`: `(unsigned)x & -(unsigned)x`. В знаковом `int` при `x == INT_MIN` вычисление `-x` — переполнение, то есть UB. Решение — в ответах к тренировочной задаче 5.
- *А если числа встречаются по три раза, кроме одного?* XOR уже не спасёт — там считают биты по модулю 3.
- *Где XOR ещё полезен?* Обмен двух чисел без временной переменной, простое шифрование, контрольные суммы, RAID-массивы.

Задача 9. Поменять местами два числа через указатели

Отдельного файла у этой задачи нет — она в пять строк и целиком здесь.

Условие. Напиши функцию `swap`, которая меняет местами значения двух переменных. Так, чтобы изменения были видны **снаружи** функции.

Идея. В Си аргументы передаются по значению — функция получает копии (см. главу 3). Поменяв копии, оригиналы не тронешь. Чтобы дотянуться до самих переменных, передаём их **адреса** и работаем через указатели.

Решение:

```c
#include <stdio.h>

void swap(int *a, int *b)   /* принимаем адреса двух переменных */
{
    int t = *a;             /* запомнили значение по a */
    *a = *b;                /* в a положили значение по b */
    *b = t;                 /* в b положили запомненное */
}

int main(void)
{
    int x = 5, y = 9;
    printf("до:    x=%d y=%d\n", x, y);
    swap(&x, &y);           /* передаём АДРЕСА x и y */
    printf("после: x=%d y=%d\n", x, y);
    return 0;
}
```

Запуск:

```
до:    x=5 y=9
после: x=9 y=5
```

Объяснение. `int *a` — указатель: `a` хранит адрес чужой переменной. `*a` — значение по этому адресу, то есть сама переменная вызывающего. Меняя `*a` и `*b`, мы меняем `x` и `y` в `main`. При вызове пишем `&x` и `&y` — берём адреса.

Именно поэтому `scanf` тоже хочет `&`: ему нужен адрес, чтобы записать в твою переменную. Забытая `&` в `scanf` — это передача **значения** там, где ждали адрес. Это UB: на практике `scanf` истолкует мусорное число как адрес и полезет писать по случайному месту в памяти. С `-Wall` clang ловит это ещё при сборке: `format specifies type 'int *' but the argument has type 'int'`.

Доп. вопросы.

- *Почему версия `void swap(int a, int b)` не работает?* Она меняет местами копии; снаружи всё остаётся как было. Компилятор не ругнётся — код формально корректен, просто бесполезен.
- *А как поменять местами два указателя?* Тогда нужен указатель на указатель: `void swap_ptr(int **a, int **b)`.
- *Можно ли без временной переменной, через XOR?* Можно: `*a ^= *b; *b ^= *a; *a ^= *b;`. Но есть подвох: если `a` и `b` указывают на **одну** переменную, она обнулится (я проверил: `xswap(&x, &x)` при `x = 9` оставляет 0) — первый же `*a ^= *b` превращает её в `x ^ x`, то есть в ноль. Версия с временной `t` надёжнее и читается лучше — на собеседовании предпочитают её.

Часть 2. Каверзы

Эти вопросы не про «реши задачу», а про «понимаешь ли ты язык». Любимая территория собеседующих: ответ короткий, но за ним стоит понимание памяти, указателей и неопределённого поведения. Решений в `code/` для них нет — некоторые специально содержат ошибку, и класть такое в файлы книги как «образец» нельзя.

Каверза 1. `char *s = "abc"` против `char s[] = "abc"`

```c
char *s1 = "abc";       /* указатель на строковый ЛИТЕРАЛ */
char  s2[] = "abc";     /* МАССИВ-копия, лежит у тебя */

s2[0] = 'A';            /* ок: меняем свой массив, выйдет "Abc" */
s1[0] = 'A';            /* НЕЛЬЗЯ: запись в литерал — UB */
```

В чём разница. `s2` — это **массив**: компилятор выделил тебе личные 4 байта (`a`, `b`, `c`, `\0`) и скопировал туда содержимое литерала. Твой массив — меняй на здоровье.

`s1` — это **указатель** на строковый литерал `"abc"`. Стандарт C11 (раздел 6.4.5) говорит коротко: попытка изменить строковый литерал — **неопределённое поведение** (undefined behavior, UB), то есть стандарт не обещает про исход вообще ничего. Где именно лежит литерал, стандарт не диктует; на macOS и Linux компилятор кладёт его в область памяти только для чтения.

И вот это стоит увидеть своими глазами, потому что «не обещает ничего» звучит абстрактно. Вот программа целиком:

```c
#include <stdio.h>


int main(void)
```

```
{
    char *s1 = "abc";
    char s2[] = "abc";

    s2[0] = 'A';
    printf("s2 = %s, sizeof s2 = %zu\n", s2, sizeof s2);
    printf("сейчас попробуем записать в литерал...\n");
    fflush(stdout);         /* вытолкнуть текст до возможного падения */
    s1[0] = 'A';
    printf("s1 = %s\n", s1);
    return 0;
}
```

`fflush(stdout)` заставляет `printf` сразу отдать накопленный текст в терминал: если программа упадёт, строка уже будет на экране. Соберём этот файл двумя способами, а код выхода покажем командой `echo $?` сразу после запуска.

Без оптимизации (`-O0`):

```
s2 = Abc, sizeof s2 = 4
сейчас попробуем записать в литерал...
[код выхода: 138]
```

Программа умерла на строке `s1[0] = 'A'`. Код 138 — это 128 плюс номер сигнала 10, то есть `SIGBUS` (на macOS у него номер 10): запись в память, которая открыта только на чтение. Ожидаемое падение.

Тот же файл с `-O2`:

```
s2 = Abc, sizeof s2 = 4
сейчас попробуем записать в литерал...
s1 = abc
[код выхода: 0]
```

Ничего не упало. И — приглядись — `s1` остался `abc`, хотя мы вроде бы записали туда `'A'`. Оптимизатор рассуждает так: «запись в литерал — UB, значит, по стандарту такого случиться не может, значит, эту строку можно считать несуществующей». Он её просто выбросил.

Вот это и есть самое опасное лицо UB. Не падение — падение как раз подарок, ты сразу видишь проблему. Опасно, когда программа с UB **молча делает не то**, и поведение меняется от флагов сборки: отладил на `-O0`, выкатил на `-O2` — и получил другой продукт. Полагаться нель-

зя ни на один из двух исходов. Отвечая на собесе, так и говори: «это UB; я видел и падение, и молчаливое игнорирование записи — зависит от уровня оптимизации».

Правило. Хочешь менять строку — заводи массив (`char s[]`). Указатель на литерал годится только для чтения; чтобы не забыть, объявляй его `const char *s1 = "abc";` — тогда компилятор сам запретит запись ещё на этапе сборки, и до UB дело не дойдёт.

Ещё одна разница, про которую спрашивают. `sizeof s2` — это 4 (размер массива), а `sizeof s1` — 8 (размер указателя на этом 64-битном маке). И `strlen` у обоих 3. Не путай размер хранилища с длиной строки.

Каверза 2. `sizeof` массива внутри функции

```c
#include <stdio.h>

void f(int arr[])                /* это int *arr в маскировке! */
{
    printf("внутри функции sizeof arr = %zu\n", sizeof arr);
}

int main(void)
{
    int a[10];
    printf("в main sizeof a = %zu\n", sizeof a);
    f(a);
    return 0;
}
```

В чём подвох. В месте объявления `sizeof a` знает настоящий размер массива — 40 байт (10 чисел по 4 байта; `int` здесь 4-байтный). Но при передаче в функцию массив **распадается** (decays) в указатель на первый элемент. Параметр `int arr[]` — это синтаксическая маскировка под `int *arr`; даже число в скобках (`int arr[10]`) компилятор проигнорирует. Поэтому внутри функции `sizeof arr` — это размер **указателя**, 8 байт на 64-битной системе.

Проверка на живой программе даёт ровно это:

```
в main sizeof a = 40
внутри функции sizeof arr = 8
```

Причём компилятор про такую ошибку знает и сам предупреждает — ещё один довод всегда собирать с `-Wall`:

```
warning: sizeof on array function parameter will return size of
'int *' instead of 'int[]' [-Wsizeof-array-argument]
    5 |     printf("внутри функции sizeof arr = %zu\n", sizeof arr);
      |                                                        ^
note: declared here
    3 | void f(int arr[])
      |            ^
```

Следствие. Внутри функции по самому массиву его длину не узнать. Поэтому длину **всегда передают отдельным параметром**: `f(int *arr, int n)`. Так устроены все наши функции в этом приложении. Это же объясняет, почему строкам нужна `strlen`: длину массива `char` иначе не вычислить, спасает только метка `'\0'`.

Что ещё спросят. «А если параметр объявить как `int (*arr)[10]` — указатель на массив из 10 элементов?» Тогда `sizeof *arr` даст честные 40. Приём рабочий, но редкий: он жёстко фиксирует длину.

Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)`

```c
int arr[] = {10, 20, 30};
printf("%d\n", arr[1]);     /* 20 */
printf("%d\n", *(arr + 1)); /* 20 — то же самое */
printf("%d\n", 1[arr]);     /* 20 — и это тоже!? */
```

Запуск подтверждает — все три строки печатают одно:

```
20
20
20
```

Объяснение. По определению языка `arr[i]` — это **ровно** `*(arr + i)`: «возьми адрес начала, прибавь `i`, разыменуй». Сложение коммутативно: `arr + i` и `i + arr` — одно и то же. Значит, `*(arr + i)` равно `*(i + arr)`, а это по тому же правилу записывается как `i[arr]`. Поэтому `1[arr]` — легальный (хоть и дикий) способ написать `arr[1]`.

`arr + 1` — это **арифметика указателей**: «следующий элемент», то есть адрес сдвигается не на 1 байт, а на размер одного элемента (`sizeof(int)`, здесь 4). Компилятор сам умножает на размер типа. Если бы это был массив `double`, шаг был бы 8 байт — но в коде это выглядит одинаково.

Зачем это знать. На собеседовании этим проверяют, понимаешь ли ты, что индексация — синтаксический сахар над арифметикой указателей. Писать `1[arr]` в реальном коде, разумеется, нельзя — это запутывает.

Отсюда же следует ответ на другой частый вопрос: почему индексы в Си начинаются с нуля? Потому что `arr[0]` — это `*(arr + 0)`, то есть сам адрес начала, без сдвига. Ноль тут не соглашение, а арифметика.

Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++`

Четыре похожих записи, которые делают **разное**. Разберём по очереди. Пусть:

```
int b[] = {10, 20, 30};
int *p = b;             /* p смотрит на b[0] (значение 10) */
```

- **`++*p`** — это `++(*p)`. Разыменуй `p` (получим `b[0]`) и сделай префиксный инкремент значения. Результат: `b[0]` стало 11, `p` не двигался. Выражение даёт 11.
- **`(*p)++`** — постфиксный инкремент **значения**, на которое смотрит `p`. `b[0]` станет 11, но выражение вернёт **старое** значение 10. `p` на месте.
- **`*p++`** — здесь приоритет у постфиксного `++`: это `*(p++)`. Сначала берётся текущий `p` (адрес `b[0]`), потом `p` сдвигается на следующий элемент. Звёздочка разыменует **старый** адрес. Итог: выражение даёт 10 (значение `b[0]`), а `p` теперь смотрит на `b[1]`. Значения в массиве не менялись. Эта запись — рабочая лошадка при проходе по строке: `while (*p++)`.
- **`*++p`** — сначала префиксный `++p` (сдвигаем `p` на `b[1]`), потом разыменование. Выражение даёт 20, `p` смотрит на `b[1]`.

Все четыре, запущенные по очереди (каждый раз с чистым массивом `{10, 20, 30}` и `p`, возвращённым на начало):

```
++*p   -> выражение 11, b[0]=11, p смотрит на b[0]
(*p)++ -> выражение 10, b[0]=11, p смотрит на b[0]
*p++   -> выражение 10, b[0]=10, p смотрит на b[1]
*++p   -> выражение 20, b[0]=10, p смотрит на b[1]
```

Сравни первые две строки: обе меняют `b[0]` на 11, но `++*p` отдаёт наружу новое значение, а `(*p)++` — старое. И сравни вторую пару: там массив не тронут вовсе, двигается только указатель.

Как не путаться. Запомни приоритеты: постфиксный `++` цепляется к `p` сильнее, чем `*`. Поэтому `*p++` — это «разыменуй, а указатель сдвинь», а не «сдвинь значение». Если хочешь менять **значение** — ставь скобки: `(*p)++`. На собеседовании именно скобки показывают, что ты понимаешь, что делаешь.

И отдельное предупреждение: в одном выражении такие штуки не смешивают. `*p++ = *q++` — нормальная идиома, потому что каждая переменная меняется один раз. А вот `p[i++] = i++` — уже UB (см. Приложение А, каверза 7).

Каверза 5. `const char *p` против `char * const p`

```
char a = 'a', b = 'b';

const char *p1 = &a;    /* нельзя менять *p1, можно менять p1 */
/* *p1 = 'x';  -- ошибка компиляции */
p1 = &b;                /* ок: указатель переставили */

char * const p2 = &a;   /* можно менять *p2, нельзя менять p2 */
*p2 = 'x';              /* ок: значение поменяли (a стало 'x') */
/* p2 = &b;  -- ошибка компиляции */
```

Как читать. Правило простое: читай объявление **справа налево от имени**.

- `const char *p1` → «`p1` — указатель на `const char`». Под запретом **то, на что смотрим** (`*p1`). Сам указатель свободно переставляется.
- `char * const p2` → «`p2` — `const`-указатель на `char`». Под запретом **сам указатель** (его нельзя перенаправить). А значение по нему менять можно.

Если раскомментировать обе запрещённые строки, компилятор скажет ровно то, чего мы ждём:

```
error: read-only variable is not assignable
    6 |     *p1 = 'x';
      |     ~~~ ^
error: cannot assign to variable 'p2' with const-qualified type
'char *const'
   11 |     p2 = &b;
      |     ~~ ^
note: variable 'p2' declared const here
```

Заметь: это **ошибки**, а не предупреждения. `const` — не совет компилятору, а запрет.

Зачем. `const char *` — самая частая форма для параметров-строк, которые функция только читает (как в нашей `my_strlen`). Такой параметр сразу говорит вызывающему: «твою строку не испорчу». А бывает и `const char * const p` — нельзя ни то, ни другое.

И важная тонкость: `const` защищает от **случайной** правки, а не от злонамеренной. Приведением типа `(char *)p1` его можно снять, и компилятор смолчит. Но если сам объект был объявлен как `const` (например, `const char c = 'a';`), запись в него через такой указатель — UB по стандарту (C11, 6.7.3). А если объект вдобавок лежит в памяти только для чтения, запись упадёт, как в каверзе 1.

Каверза 6. Найди баг: утечка памяти

```c
#include <stdio.h>
#include <stdlib.h>

char *make_greeting(const char *name)
{
    char *buf = malloc(64);
    if (buf == NULL) return NULL;
    snprintf(buf, 64, "Привет, %s!", name);
    return buf;
}

int main(void)
{
    printf("%s\n", make_greeting("Аня"));   /* БАГ: утечка */
    return 0;
}
```

Где баг. `make_greeting` выделяет память через `malloc` и возвращает указатель. Но в `main` мы передаём этот указатель сразу в `printf` и **теряем** его — переменной, которая хранила бы адрес, нет. Освободить такой блок уже нечем: `free` вызвать не от чего. Это **утечка памяти**: блок занят до конца программы.

Второй, менее заметный баг: `make_greeting` может вернуть `NULL` (если памяти не хватило), а `printf("%s", NULL)` — это UB. Проверять надо и результат `malloc` внутри функции, и результат самой функции снаружи.

Как чинить. Сохрани адрес, проверь, попользуйся, освободи:

```c
char *msg = make_greeting("Аня");
if (msg != NULL) {
    printf("%s\n", msg);
    free(msg);              /* на каждый malloc — свой free */
}
```

Правило из главы 3 в полный рост: на каждый `malloc` однажды должен прийтись ровно один `free`. Если функция **возвращает** выделенную память, это надо оговаривать в её документации или прямо в имени (`..._alloc`, `..._new`, `..._dup`) — иначе вызывающий не догадается, что теперь он в ответе за освобождение. Ровно так устроена `strdup`.

Как ловить. Утечку не видно ни глазом, ни по выводу программы — только инструментами:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address prog.c -o prog
```

На Linux адресный санитайзер сам сообщит про leak в конце работы. На macOS с процессором Intel поиск утечек в санитайзере не работает — я проверил, программа под ним молча печатает приветствие и выходит с кодом 0. Зато на macOS есть системная утилита `leaks`. Собери программу без санитайзера и запусти так: `leaks -atExit -- ./prog`. В конце её отчёта будет строка:

```
Process 59853: 1 leak for 80 total leaked bytes.
```

Одна утечка. Байтов 80, а не 64: `leaks` считает размер блока, который аллокатор реально отвёл под запрос, а он бывает чуть больше запрошенного. Номер процесса у тебя будет другим.

Каверза 7. Найди баг: use-after-free и двойной free

```c
int *p = malloc(sizeof(int));
*p = 42;
free(p);                /* блок вернули аллокатору */

printf("%d\n", *p);     /* БАГ 1: use-after-free — читаем мёртвое */
free(p);                /* БАГ 2: double free — освобождаем дважды */
```

Что не так. После `free(p)` блок больше не наш: `free` отдаёт его аллокатору (той части библиотеки, что обслуживает `malloc`), и тот вправе выдать его следующему `malloc`. Операционной системе память при этом обычно не возвращается. Указатель `p` всё ещё хранит старый адрес (он стал «висячим», dangling), но трогать память по нему нельзя:

- `*p` после `free` — **use-after-free**: читаем (или пишем) освобождённую память. UB: может показать старое значение, мусор или уронить программу. Хуже всего первый вариант — аллокатор ещё не успел отдать блок кому-то другому, всё «работает», и баг ждёт своего часа.
- второй `free(p)` — **double free**: освобождение уже освобождённого блока. Тоже UB, частая причина падений и уязвимостей: аллокатор ведёт свои служебные структуры прямо рядом с блоком, и повторное освобождение их портит.

Как чинить. Не обращайся к памяти после `free`, а указатель сразу обнуляй:

```c
free(p);
p = NULL;       /* теперь p честно «никуда»; free(NULL) безопасен */
```

Привычка `p = NULL` после `free` лечит оба бага разом: повторный `free(NULL)` по стандарту не делает ничего (C11, 7.22.3.3), а случайное `*p` по `NULL` — тоже UB, но на macOS и Linux оно по-

чти всегда сразу роняет программу, а не портит данные втихую. Вот что показал запуск кода выше без санитайзера: `printf` вывел мусор, каждый раз разный (например, `-1899806656`, но не 42!), а второй `free` уронил программу с кодом 134 — это 128 + 6, сигнал `SIGABRT`: аллокатор сам заметил двойное освобождение и прервал работу.

Оба бага мгновенно ловятся адресным санитайзером — он показывает и место освобождения, и место повторного обращения:

```
cc -std=c11 -g -fsanitize=address prog.c -o prog && ./prog
```

Первая же строка отчёта называет баг по имени:

```
ERROR: AddressSanitizer: heap-use-after-free on address ...
READ of size 4 at ... thread T0
```

Санитайзер останавливает программу на первом баге, поэтому до двойного `free` дело уже не доходит. Убери строку с `printf` — и он так же поймает `attempting double-free`.

Именно поэтому все программы этого приложения прогнаны под `-fsanitize=address,undefined` — и молчат.

Каверза 8. Возврат адреса локальной переменной

```c
#include <stdio.h>

int *broken(void)
{
    int x = 42;
    return &x;          /* БАГ: x умрёт по выходе из функции */
}

int main(void)
{
    int *p = broken();
    printf("%d\n", *p); /* читаем мёртвую память — UB */
    return 0;
}
```

Где баг. `x` — локальная переменная, она живёт на стеке только пока работает `broken`. После `return` её память освобождается — точнее, объявляется свободной: следующий же вызов любой функции положит туда свои данные. Адрес `&x` становится висячим. Обращение `*p` в `main` — неопределённое поведение.

Хорошая новость: этот баг компилятор ловит сам, если собирать с `-Wall`:

```
warning: address of stack memory associated with local variable 'x'
returned [-Wreturn-stack-address]
    6 |     return &x;
      |            ^
```

А вот чем оборачивается игнор предупреждения. Три запуска подряд одного и того же бинарника, собранного с `-O2`:

```
-1114688744
-1126284520
-1159945448
```

Не 42, не ноль и каждый раз разное: в этой ячейке стека лежат остатки служебных данных, а адреса от запуска к запуску сдвигает рандомизация адресного пространства. У тебя числа будут другими.

А теперь забавная деталь: соберём тот же файл с `-O0` (без оптимизации) — и три запуска честно печатают `42`, `42`, `42`. Значение просто не успели затереть. Это ровно та ловушка, о которой речь: UB иногда «работает», и именно поэтому на него нельзя опираться. Полагаться нельзя ни на 42, ни на мусор.

Адресный санитайзер по умолчанию этот баг не ловит. Его надо попросить переменной окружения:

```
cc -std=c11 -g -fsanitize=address prog.c -o prog
ASAN_OPTIONS=detect_stack_use_after_return=1 ./prog
```

Тогда он останавливает программу с отчётом `ERROR: AddressSanitizer: stack-use-after-return`.

Как чинить. Есть три законных способа:

1. вернуть **значение**, а не адрес (`int broken(void)` с `return x;`) — самый простой и обычно правильный;
2. выделить память динамически (`malloc`) и отдать ответственность за `free` вызывающему — как в каверзе 6;
3. принять место для результата от вызывающего параметром: `void fill(int *out) { *out = 42; }` — так устроены многие функции стандартной библиотеки: `scanf` пишет по переданным адресам, `strtol` через второй параметр сообщает, где кончилось число.

Возвращать адрес `static`-переменной технически можно — она живёт до конца программы, — но это общая память на все вызовы: два вызова подряд затрут результат друг друга, а в многопоточной программе будет гонка.

Ошибки, на которых сыплются в этих задачах

- **Забыть `head = reverse(head)`.** Функция вернула новую голову, а её никто не принял — список потерян.
- **Переставить проверки в `fast != NULL && fast->next != NULL`.** Порядок в && имеет значение: сначала `fast != NULL`, потом `fast->next != NULL`, и только потом прыжок `fast->next->next`.
- **`(lo + hi) / 2` в бинарном поиске.** Переполнение `int` на больших индексах — UB; на практике обычно получается отрицательный индекс.
- **`lo < hi` вместо `lo <= hi`.** Теряется последний элемент отрезка.
- **Битовые операции на знаковых типах.** Сдвиг влево отрицательного — UB; сдвиг вправо — implementation-defined. Бери `unsigned`.
- **Забыть, что массив в функции — это указатель.** `sizeof` соврёт, длину надо передавать отдельно.
- **Писать в строковый литерал.** UB. У нас на `-O0` программа упала, а на `-O2` молча проигнорировала запись; другой компилятор вправе сделать третье.
- **Потерять адрес из `malloc`.** Утечка, которую не видно по выводу.
- **Не обнулить указатель после `free`.** Use-after-free и double free ждут за углом.
- **Вернуть адрес локальной переменной.** Компилятор предупредит — прочитай предупреждение.

Тренировочные задачи

1. Допиши `my_strncpy(char *dst, const char *src, size_t n)` — копирует не больше `n` символов. Реализуй две версии: как в стандарте (может не дописать `'\0'`) и «безопасную» (всегда дописывает). Объясни, чем вторая лучше.
2. Напиши `struct Node *reverse_between(struct Node *head, int m, int n)` — разворот куска списка с позиции `m` по `n` (позиции считаем с 1). Проверь на краях: `m == 1`, `n == длина`, `m == n`.
3. Реализуй поиск **первого** элемента, который не меньше искомого, в отсортированном массиве с дубликатами (его называют lower bound). Проверь на массиве из одних одинаковых чисел.
4. Переделай `merge` из задачи 6 так, чтобы он сливал `a` и `b` в конец массива `a` (места хватает), идя с конца. Ни одного лишнего буфера.
5. Найди в массиве **два** числа, встречающихся нечётное число раз (остальные встречаются чётное число раз), за O(n) времени и O(1) памяти. Подсказка: `x & -x` даёт младший выставленный бит — считай его в `unsigned`.

6. Возьми программу из каверзы 8 и почини её тремя способами из разбора. Каждый вариант собери с `-Wall` и убедись, что предупреждение исчезло.

Ответы к тренировочным задачам

Сначала попробуй сам. Все решения ниже собраны с `-std=c11 -Wall -Wextra -O2` без предупреждений, прогнаны под `-fsanitize=address,undefined`, а те, где есть `malloc`, ещё и через `leaks` — чисто.

Задача 1.

```c
#include <stdio.h>
#include <string.h>

/* Как в стандарте: не больше n символов; если src длиннее,
   '\0' НЕ дописывается. Остаток до n заполняется нулями. */
char *my_strncpy(char *dst, const char *src, size_t n)
{
    size_t i = 0;
    for (; i < n && src[i] != '\0'; i++) {
        dst[i] = src[i];
    }
    for (; i < n; i++) {
        dst[i] = '\0';
    }
    return dst;
}

/* «Безопасная»: dst вмещает size байт, '\0' есть всегда. */
char *safe_strncpy(char *dst, const char *src, size_t size)
{
    if (size == 0) {
        return dst;
    }
    size_t i = 0;
    for (; i + 1 < size && src[i] != '\0'; i++) {
        dst[i] = src[i];
    }
    dst[i] = '\0';
```

```
    return dst;
}

int main(void)
{
    char a[4];
    my_strncpy(a, "Hello", sizeof a);
    printf("my_strncpy:   %.4s (a[3] = '%c')\n", a, a[3]);

    char b[4];
    safe_strncpy(b, "Hello", sizeof b);
    printf("safe_strncpy: %s (длина %zu)\n", b, strlen(b));
    return 0;
}
```

```
my_strncpy:   Hell (a[3] = 'l')
safe_strncpy: Hel (длина 3)
```

Стандартная версия честно скопировала 4 символа `Hell` и **не** поставила `'\0'`: в `a[3]` лежит `'l'`. Печатать такой массив через `%s` нельзя — `printf` побежит дальше за границу искать ноль. Поэтому здесь `%.4s`: точность `.4` велит напечатать не больше 4 символов. Безопасная версия оставила место под `'\0'` и отрезала строку до `Hel` — зато результат всегда корректная строка. Второй цикл в `my_strncpy` дописывает нули до `n`: так ведёт себя и стандартная `strncpy`, если источник короче.

Задача 2.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;
    struct Node *next;
};

/* Развернуть узлы с позиции m по n (нумерация с 1, 1 <= m <= n). */
struct Node *reverse_between(struct Node *head, int m, int n)
{
    struct Node dummy = {0, head};   /* фиктивный узел перед головой */
    struct Node *before = &dummy;
```

```
    for (int i = 1; i < m; i++) {
        before = before->next;       /* узел перед позицией m */
    }
    struct Node *first = before->next;   /* станет хвостом куска */
    struct Node *prev = NULL;
    struct Node *cur = first;
    for (int i = m; i <= n; i++) {       /* обычный разворот куска */
        struct Node *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    before->next = prev;     /* начало куска — бывший его конец */
    first->next = cur;       /* хвост куска смотрит на остаток */
    return dummy.next;
}

struct Node *build(int len)
{
    struct Node *head = NULL;
    for (int i = len; i >= 1; i--) {
        struct Node *x = malloc(sizeof *x);
        if (x == NULL) {
            exit(1);
        }
        x->value = i;
        x->next = head;
        head = x;
    }
    return head;
}

void print_and_free(struct Node *head)
{
    while (head != NULL) {
        struct Node *next = head->next;
        printf("%d ", head->value);
        free(head);
```

```
        head = next;
    }
    printf("\n");
}

int main(void)
{
    print_and_free(reverse_between(build(5), 2, 4));
    print_and_free(reverse_between(build(5), 1, 5));
    print_and_free(reverse_between(build(5), 1, 2));
    print_and_free(reverse_between(build(5), 3, 3));
    return 0;
}
```

```
1 4 3 2 5
5 4 3 2 1
2 1 3 4 5
1 2 3 4 5
```

Фиктивный узел `dummy` стоит перед головой, и благодаря ему случай `m == 1` ничем не отличается от остальных: «узел перед куском» есть всегда. После разворота `prev` указывает на бывший конец куска (он стал началом), `first` — на бывшее начало (оно стало концом), а `cur` — на первый узел после куска. Два последних присваивания сшивают три части. Четыре строки вывода — это `m=2, n=4`, весь список, `m=1, n=2` и `m == n` (ничего не меняется).

Задача 3.

```
#include <stdio.h>

/* Первый индекс i, где a[i] >= target; n, если такого нет. */
int lower_bound(const int *a, int n, int target)
{
    int lo = 0;
    int hi = n;                 /* полуинтервал [lo, hi) */
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) {
            lo = mid + 1;       /* ответ правее mid */
        } else {
            hi = mid;           /* mid может быть ответом */
```

```
        }
    }
    return lo;
}

int main(void)
{
    int a[] = {1, 2, 2, 2, 5, 7};
    int same[] = {4, 4, 4, 4};
    printf("%d %d %d %d\n", lower_bound(a, 6, 2), lower_bound(a, 6, 3),
           lower_bound(a, 6, 0), lower_bound(a, 6, 9));
    printf("%d\n", lower_bound(same, 4, 4));
    return 0;
}
```

```
1 4 0 6
0
```

Здесь другая схема границ, чем в бинарном поиске из задачи 5: полуинтервал `[lo, hi)`, где `hi` не входит. Поэтому `hi = n`, условие `lo < hi` и сдвиг `hi = mid` без `−1` — `mid` сам может оказаться ответом. Цикл кончается, когда `lo == hi`, и это единственный кандидат. В массиве `{1, 2, 2, 2, 5, 7}` первая двойка стоит на индексе 1; для 3 ответ 4 (первый элемент ≥ 3 — это 5); для 0 — индекс 0; для 9 — `n = 6`, «такого нет». На массиве из одних четвёрок ответ 0, а обычный бинарный поиск из задачи 5 вернул бы первый попавшийся индекс — 1.

Задача 4.

```
#include <stdio.h>

/* a вмещает na + nb элементов, заняты первые na. Сливаем с конца. */
void merge_into_a(int *a, int na, const int *b, int nb)
{
    int i = na - 1;          /* последний из a */
    int j = nb - 1;          /* последний из b */
    int k = na + nb - 1;     /* куда пишем */
    while (j >= 0) {         /* пока в b что-то осталось */
        if (i >= 0 && a[i] > b[j]) {
            a[k--] = a[i--];
        } else {
            a[k--] = b[j--];
```

```
        }
    }
}

int main(void)
{
    int a[11] = {1, 3, 5, 7, 9};
    int b[] = {2, 4, 6, 8, 10, 12};
    merge_into_a(a, 5, b, 6);
    for (int i = 0; i < 11; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

```
1 2 3 4 5 6 7 8 9 10 12
```

Пишем в `a` с конца: позиция `k` всегда правее или равна `i`, поэтому мы не затираем ещё не прочитанные элементы `a`. Цикл идёт, пока не кончится `b`: если раньше кончится `a`, остаток `b` допишется той же веткой `else`, а если раньше кончится `b` — оставшиеся элементы `a` уже стоят на своих местах.

Задача 5.

```
#include <stdio.h>

/* Два числа, встречающихся нечётное число раз; остальные — чётное. */
void find_two_odd(const int *a, int n, int *x, int *y)
{
    unsigned all = 0;
    for (int i = 0; i < n; i++) {
        all ^= (unsigned)a[i];          /* all = x ^ y */
    }
    unsigned bit = all & -all;          /* младшая единица all */
    unsigned g1 = 0, g2 = 0;
    for (int i = 0; i < n; i++) {
        if ((unsigned)a[i] & bit) {
            g1 ^= (unsigned)a[i];
        } else {
```

```
            g2 ^= (unsigned)a[i];
        }
    }
    *x = (int)g1;
    *y = (int)g2;
}

int main(void)
{
    int a[] = {4, 7, 1, 4, 9, 1, 9, 9, 9, 3};
    int x, y;
    find_two_odd(a, 10, &x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

```
7 3
```

В массиве `{4, 7, 1, 4, 9, 1, 9, 9, 9, 3}` пары — 4, 1 и 9 (девятка четыре раза), одиночки — 7 и 3. Первый проход даёт `all = 7 ^ 3`: `111 ^ 011 = 100`, то есть 4. Раз бит 4 выставлен, у 7 и 3 он разный. Второй проход делит числа на две группы по этому биту: 7 и 4 (бит есть), 3, 1 и 9 (бита нет). В каждой группе пары снова гасятся, и остаётся по одиночке. Всё считаем в `unsigned`: там `~all` определено всегда (это 2^{32} `- all` на этом маке), а в знаковом `int` при `INT_MIN` было бы переполнение. Проходов два, но время всё равно O(n), а памяти — три переменные.

Задача 6.

```
#include <stdio.h>
#include <stdlib.h>

int fixed_value(void)           /* способ 1: вернуть значение */
{
    int x = 42;
    return x;
}

int *fixed_heap(void)           /* способ 2: память из malloc */
{
    int *p = malloc(sizeof *p);
    if (p != NULL) {
```

```
        *p = 42;
    }
    return p;                   /* free — забота вызывающего */
}

void fixed_out(int *out)        /* способ 3: место даёт вызывающий */
{
    *out = 42;
}

int main(void)
{
    printf("%d\n", fixed_value());

    int *p = fixed_heap();
    if (p != NULL) {
        printf("%d\n", *p);
        free(p);
    }

    int v;
    fixed_out(&v);
    printf("%d\n", v);
    return 0;
}
```

```
42
42
42
```

Все три способа собираются без `-Wreturn-stack-address`. Во втором способе за `free` отвечает вызывающий — это надо оговаривать в комментарии к функции, как мы и сделали.

Мини-резюме

Что должно остаться в голове после этого приложения:

- **Строки** — это `char`-массивы с `'\0'`; любая своя строковая функция идёт по символам до нуля. Литерал `"abc"` менять нельзя, сравнивать символы надо как `unsigned char`.

- **Списки** — три указателя разворачивают, два указателя разной скорости находят середину, ловят цикл и находят его начало (Флойд). Следи за порядком присваиваний и не разыменовывай `NULL`.
- **Биты** — `n & (n-1)` гасит младшую единицу (popcount Кернигана, проверка степени двойки); XOR гасит пары (одиночка за O(1) памяти). Работай на `unsigned`.
- **Поиск и слияние** — бинарный поиск держи на включительных границах `[lo, hi]` с `lo + (hi - lo) / 2`; слияние — два индекса плюс хвосты.
- **Память** — на каждый `malloc` свой `free`; после `free` ставь `p = NULL`; не возвращай адрес локальной переменной; помни про decay массива в указатель и `const`-квалификаторы.

И главное, что проверяют на Middle: ты не зубришь алгоритмы, а понимаешь, **что происходит в памяти**. Поэтому на собеседовании проговаривай вслух, кто на что указывает и кто кого освобождает, — это ценится больше готового ответа.

Что дальше

Дальше — **Приложение В: Senior**. Там задачи злее: quicksort и merge sort с разбором худших случаев, обобщённые структуры через `void *` и указатели на функции, хеш-таблица с цепочками, кольцевой буфер, LRU-кэш, свои `atoi`/`itoa` с защитой от переполнения. А в теоретической части — strict aliasing, `volatile`, две роли `static`, выравнивание и padding в структурах, ловушки макросов, порядок вычисления аргументов и endianness. Если этот раздел дался без больших мучений — ты готов идти выше.

Приложение B. Задачи с собеседований: Senior

Это третий, самый трудный сборник задач с собеседований. Если в «Junior» проверяли, помнишь ли ты синтаксис, а в «Middle» — умеешь ли собрать алгоритм, то здесь смотрят глубже: понимаешь ли ты, **как Си устроен внутри**. Память, указатели на указатели, `void *`, выравнивание структур, неопределённое поведение (UB), тонкости препроцессора — то, на чём спотыкаются даже люди с опытом.

Задачи идут от простого к сложному и делятся на два блока:

- **на код** — написать рабочую программу: быстрые сортировки, обобщённые структуры через `void *`, хеш-таблица, кольцевой буфер, LRU-кэш и другие классические задачи;
- **каверзы и теория** — короткие вопросы, где надо не написать код, а объяснить, **почему так**: что выведет хитрый `sizeof`, чем опасен макрос `SQR(x)`, что такое strict aliasing.

Всё, что нужно, ты уже прошёл в главах 1–18: указатели и память (главы 3, 11), рекурсия (глава 9), сортировки и `qsort` (глава 8), структуры и связные списки (глава 11). Здесь мы доведём это до уровня, который ждут от senior-разработчика на Си.

Совет тот же, что и раньше: сначала реши задачу сам на бумаге и в редакторе, и только потом сверяйся с разбором. Все программы лежат в `code/21-*.c` и собираются без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 code/21-01-quicksort.c -o q && ./q
```

Номер задачи здесь совпадает с номером файла: задача 7 — это `code/21-07-hashtable.c`. Все программы, где есть `malloc`, прогнаны под `-fsanitize=address,undefined` и проверены на утечки — санитайзеры молчат. Каверзы во второй части в `code/` не вынесены: часть из них содержит намеренное UB, и такому не место в файлах книги.

Все выводы программ и все сообщения компилятора получены реальным запуском: Apple clang 21, x86-64, macOS. Где ответ зависит от платформы или от флагов сборки — я это отдельно оговариваю, потому что «одного правильного числа» там просто нет.

В конце приложения — разбор типовых ошибок и семь задач на самостоятельную отработку, а после них — ответы с проверенным кодом.

Часть 1. Задачи на код

Задача 1. Быстрая сортировка (quicksort)

Условие. Реализуй быструю сортировку массива `int` на месте (без дополнительного массива). Объясни выбор опорного элемента и сложность.

Идея. В главе 8 мы разобрали идею словами: выбираем **опорный** (pivot) элемент, переставляем массив так, чтобы слева оказались все меньшие опорного, а справа — большие. После такой перестановки опорный стоит на своём окончательном месте. Дальше рекурсивно сортируем левую и правую части.

Главная деталь — **разбиение** (partition). Возьмём схему Ломуто: опорным берём последний элемент `a[hi]`. Ведём границу `i` — индекс последнего «малого» элемента. Идём `j` от начала: как только встретили элемент меньше опорного, сдвигаем границу и меняем его туда. В конце ставим опорный сразу за границей.

Решение (`code/21-01-quicksort.c`):

```c
#include <stdio.h>

/* Обмен двух int через указатели (см. главу 3). */
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Разбиение Ломуто: опорный — последний элемент a[hi].
   Расставляем меньшие опорного слева, большие — справа,
   и возвращаем итоговую позицию опорного. */
int partition(int a[], int lo, int hi)
{
    int pivot = a[hi];          /* опорный (pivot) */
    int i = lo - 1;             /* граница «меньших опорного» */

    for (int j = lo; j < hi; j++) {
```

```
        if (a[j] < pivot) {    /* нашли элемент меньше опорного */
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);   /* опорный встаёт на своё место */
    return i + 1;
}

/* Быстрая сортировка: разбили относительно опорного и
   рекурсивно сортируем левую и правую части. */
void quicksort(int a[], int lo, int hi)
{
    if (lo >= hi) {            /* 0 или 1 элемент — уже отсортировано */
        return;
    }
    int p = partition(a, lo, hi);
    quicksort(a, lo, p - 1);
    quicksort(a, p + 1, hi);
}
```

В `main` зовём `quicksort(a, 0, n - 1)` — границы **включительно**, поэтому `hi = n - 1`, а не `n`. Массив там `{5, 2, 9, 1, 5, 6, 3, 8, 7, 0}`, и запуск даёт:

```
0 1 2 3 5 5 6 7 8 9
```

Две пятёрки на месте — дубликаты сортировка переживает нормально, хотя порядок именно этих двух элементов между собой не гарантирован (quicksort нестабильна, об этом ниже).

Объяснение.

- `swap(int *a, int *b)` принимает **адреса** переменных и меняет их содержимое через разыменование `*a`, `*b` (см. главу 3). Передай мы числа по значению — обмен случился бы только внутри функции.
- `int i = lo - 1;` — граница начинается «до» отрезка: малых элементов пока нет. **Инвариант** — это утверждение, которое остаётся верным перед каждым витком цикла. Здесь он такой: всё в `a[lo..i]` меньше опорного, всё в `a[i+1..j-1]` — больше либо равно. На старте обе части пустые, и утверждение верно; каждый виток его сохраняет; значит, после цикла оно верно для всего отрезка `a[lo..hi-1]`.
- цикл `for (j = lo; j < hi; j++)` доходит только до `hi - 1`, сам опорный `a[hi]` не трогаем — его поставим в конце.

- `swap(&a[i + 1], &a[hi])` — после цикла `i+1` это первая «большая» ячейка; меняем её с опорным, и тот оказывается ровно между меньшими и большими. Возвращаем `i + 1` — его финальную позицию.

Сложность: лучший и средний случай. Одно разбиение отрезка из `m` элементов делает ровно `m – 1` сравнений: каждый элемент, кроме опорного, один раз сравнивается с опорным. Если опорный всякий раз делит отрезок пополам, получается дерево вызовов высотой около $\log_2 n$, и на каждом его уровне в сумме меньше `n` сравнений — итого O(n log n). Это **лучший** случай.

Средний случай — это не «опорный всегда попадает в середину», а среднее по всем возможным порядкам входа (или по случайному выбору опорного). Опорный может попасть куда угодно, но в среднем части выходят достаточно ровными: даже если бы каждое разбиение делило отрезок в пропорции 1 к 9, глубина была бы около логарифма `n` по основанию 10/9 — это примерно $6.6 \cdot \log_2 n$, то есть тоже O(log n). Точный подсчёт даёт в среднем примерно $2n \cdot \ln n \approx 1.39 \cdot n \cdot \log_2 n$ сравнений — это O(n log n), всего на 39% больше идеала.

Худший случай. Если опорный каждый раз оказывается минимумом или максимумом, одна часть получается пустой, а другая — из `m – 1` элементов. Тогда сравнений `(n–1) + (n–2) + … + 1 = n(n–1)/2`, это $O(n^2)$, а глубина рекурсии дорастает до `n – 1`. Наша версия (опорный — последний элемент) попадает в худший случай на трёх очень жизненных входах: уже отсортированном, отсортированном наоборот и **из одних одинаковых** чисел (условие `a[j] < pivot` для равных ложно, и все они уходят в одну часть).

Это не теория — я замерил на этом маке (`–O2`):

```
1 000 000 случайных чисел      0.07 с
  100 000 из одних семёрок     2.3 с
  100 000 по убыванию          3.1 с
  100 000 уже отсортированных  4.6 с
  200 000 уже отсортированных  падение, код выхода 139
```

Отсортированные 100 тысяч сортируются в 60 с лишним раз дольше, чем случайный миллион. А на 200 тысячах глубина рекурсии — 200 тысяч вызовов, каждый со своим кадром на стеке, и 8 МБ стека кончаются: `SIGSEGV` (139 = 128 + 11). Рекурсию и переполнение стека мы разбирали в главе 9.

Лечат это выбором опорного поумнее:

- **случайный опорный** — поменять `a[hi]` с ‘a[lo + rand() % (hi - lo
  - 1)]перед разбиением. Тогда никакой вход не бывает плохим «всегда»: O(n²) случается только при крайне невезучих случайных числах, а в среднем — O(n log n) на любом входе. Одна оговорка: стандарт обещаетRAND_MAXне меньше 32767, и там, где он

такой маленький (так в Windows), на отрезке длиннее 32768 выражение выберет не каждый индекс. На macOSRAND_MAX‘ = 2147483647.

- **медиана трёх** — берут медиану (среднее по величине) из `a[lo]`, `a[mid]`, `a[hi]`. Почти бесплатно и спасает на отсортированном и обратном входе, но не от всего: массив из одних одинаковых чисел с разбиением Ломуто всё равно даст $O(n^2)$, а для медианы трёх существуют специально подобранные «убийственные» входы.

Против одинаковых элементов помогает другое разбиение: схема Хоара или «трёхпутевое» разбиение (меньше / равно / больше опорного).

Память. Quicksort сортирует на месте, дополнительная память — только стек рекурсии: O(log n) в среднем и O(n) в худшем случае, что мы и увидели. Гарантировать O(log n) можно приёмом из тренировочной задачи 2: рекурсивно спускаться только в **меньшую** часть, а большую обрабатывать циклом.

Доп. вопросы.

- *Чем схема Хоара (два бегунка навстречу) лучше Ломуто?* Она делает в среднем примерно втрое меньше обменов, а массив из одинаковых чисел делит пополам, а не на «пусто и всё остальное».
- *Почему quicksort нестабильна?* **Устойчивая** (стабильная) сортировка сохраняет исходный порядок равных элементов. Quicksort переставляет элементы обменами через весь отрезок, и равные могут перепрыгнуть друг через друга. Самый маленький пример для нашей версии: две равные записи `1a, 1b`. Опорный — `1b`, `1a < 1b` ложно, и финальный `swap` ставит `1b` на место `1a`: получается `1b, 1a`.
- *Как сделать quicksort нерекурсивной?* Завести свой стек пар `(lo, hi)` — решение в ответе к тренировочной задаче 2.

Задача 2. Сортировка слиянием (merge sort)

Условие. Реализуй сортировку слиянием. В чём её плюс перед quicksort?

Идея. Делим массив пополам, рекурсивно сортируем каждую половину, потом **сливаем** две отсортированные половины в одну. Слияние — главный шаг: идём двумя бегунками по головам половин и каждый раз забираем меньший. В отличие от quicksort, merge sort даёт **гарантированные** O(n log n) даже в худшем случае и **стабильна**, но требует доп. памяти под временный буфер.

Решение (`code/21-02-mergesort.c`):

```
/* Слияние двух отсортированных половин a[lo..mid] и a[mid+1..hi]
   через временный буфер tmp. Берём меньший из голов и пишем по
   порядку — отсюда стабильность сортировки. */
```

```
void merge(int a[], int tmp[], int lo, int mid, int hi)
{
    int i = lo;          /* бегунок по левой половине */
    int j = mid + 1;     /* бегунок по правой половине */
    int k = lo;          /* куда пишем в tmp */

    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) {     /* <= , а не <  — это и даёт стабильность */
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
        }
    }
    while (i <= mid) {          /* хвост левой половины */
        tmp[k++] = a[i++];
    }
    while (j <= hi) {           /* хвост правой половины */
        tmp[k++] = a[j++];
    }
    for (int t = lo; t <= hi; t++) {   /* копируем обратно в a */
        a[t] = tmp[t];
    }
}

/* Сортировка слиянием: делим пополам, сортируем половины, сливаем. */
void merge_sort(int a[], int tmp[], int lo, int hi)
{
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;   /* без переполнения lo + hi */
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid + 1, hi);
    merge(a, tmp, lo, mid, hi);
}
```

На том же массиве `{5, 2, 9, 1, 5, 6, 3, 8, 7, 0}` результат совпадает с quicksort — так и должно быть, отсортированный порядок один:

```
0 1 2 3 5 5 6 7 8 9
```

Объяснение.

- `tmp` — **один** временный буфер на весь алгоритм, выделяем его в `main` через `malloc((size_t)n * sizeof(int))` и передаём внутрь. Так не выделяем память на каждом слиянии.
- `int mid = lo + (hi - lo) / 2;` — почему не `(lo + hi) / 2`? Сумма `lo + hi` может **переполнить** `int` на огромных индексах. Запись `lo + (hi - lo)/2` даёт ту же середину, но без риска переполнения. Это классический приём (та же ловушка живёт и в бинарном поиске, см. главу 7 и Приложение Б, задача 5).
- `a[i] <= a[j]` — именно `<=`, а не `<`: при равенстве берём элемент из **левой** половины, то есть тот, что стоял в массиве раньше, и равные сохраняют исходный порядок. Это и есть **стабильность** (устойчивость, см. главу 8). Я проверил это отдельной программой: отсортировал 2000 случайных массивов из записей «ключ + исходный номер» с ключами от 0 до 9 — ни разу равные ключи не поменялись местами. С `<` вместо `<=` сортировка осталась бы правильной, но перестала бы быть стабильной.
- три `while` подряд: первый сливает, пока живы обе половины; два следующих дописывают остаток той половины, что ещё не кончилась.

Сложность и память. Глубина рекурсии — $\lceil \log_2 n \rceil$: каждый уровень делит отрезок пополам. На каждом уровне все слияния вместе трогают каждый элемент один раз, это O(n). Итого O(n log n) **всегда** — и в лучшем, и в худшем случае, потому что деление не зависит от данных. Сравнений не больше $n \cdot \lceil \log_2 n \rceil$. Память: буфер `tmp` на `n` элементов, то есть O(n), плюс O(log n) на стек рекурсии.

Quicksort или merge sort? Обычно говорят «quicksort быстрее на практике: сортирует на месте, лучше дружит с кэшем». Я проверил наши две версии на этом маке (`-O2`, случайные `int`):

```
              quicksort   merge sort
1 000 000       0.07 с      0.06 с
10 000 000      0.93 с      0.84 с
```

Наша простая merge sort даже чуть обогнала наш простой quicksort на Ломуто. Слова «quicksort быстрее» относятся к хорошо отлаженным реализациям (схема Хоара, вставки на коротких кусках и т. п.), а не к любому коду. Зато разница в худшем случае огромная: отсортированные 100 000 чисел quicksort разбирал 4.6 с, а merge sort — 0.003 с. Merge sort всегда O(n log n) и стабилен, но тратит O(n) дополнительной памяти. Для связных списков merge sort ещё удобнее: слияние не требует доступа по индексу, а буфер вообще не нужен — узлы просто перецепляются.

Доп. вопросы.

- *Сколько памяти под `tmp`?* O(n). *Можно ли слить две половины на месте за O(n)?* Алгоритмы такие есть, но они сложные и медленные на практике; в реальных библиотеках берут буфер.
- *Что такое «восходящий» (bottom-up) merge sort?* Без рекурсии: сначала сливаем соседние пары отрезков длины 1, потом длины 2, 4, 8 и так далее, пока отрезок не станет длиной во весь массив. Сложность та же, стек не нужен.

Задача 3. Обобщённый стек через `void *`

Условие. Напиши стек (или динамический массив), который умеет хранить элементы **любого** типа — `int`, `double`, структуру, — не переписывая код под каждый тип.

Идея. Шаблонов, как в C++, в Си нет. Универсальность строят на `void *` («указатель на что угодно») и на знании **размера элемента в байтах**. Стек хранит сырой байтовый буфер и `elem_size`. `push` **копирует** `elem_size` байт из переданного адреса в буфер, `pop` — обратно. Сам стек не знает, что за тип внутри: он просто двигает байты через `memcpy`.

Решение (`code/21-03-generic-stack.c`):

```c
typedef struct {
    char *data;          /* буфер из байтов */
    size_t elem_size;    /* размер одного элемента в байтах */
    size_t count;        /* сколько элементов лежит */
    size_t capacity;     /* сколько элементов влезает */
} Stack;

/* Готовим стек под элементы размера elem_size. 1 — успех, 0 — ошибка. */
int stack_init(Stack *s, size_t elem_size)
{
    if (elem_size == 0 || elem_size > SIZE_MAX / 4) {
        s->data = NULL;              /* такой размер элемента не годится */
        return 0;
    }
    s->elem_size = elem_size;
    s->count = 0;
    s->capacity = 4;
    s->data = malloc(s->capacity * elem_size);
    return s->data != NULL;
}

/* Кладём элемент: копируем elem_size байт по адресу elem на вершину. */
```

```
int stack_push(Stack *s, const void *elem)
{
    if (s->count == s->capacity) {            /* места нет — растём */
        if (s->capacity > SIZE_MAX / 2 / s->elem_size) {
            return 0;          /* new_cap * elem_size не влезет в size_t */
        }
        size_t new_cap = s->capacity * 2;
        char *bigger = realloc(s->data, new_cap * s->elem_size);
        if (bigger == NULL) {
            return 0;
        }
        s->data = bigger;
        s->capacity = new_cap;
    }
    memcpy(s->data + s->count * s->elem_size, elem, s->elem_size);
    s->count++;
    return 1;
}

/* Снимаем верхний элемент в out (если out != NULL). 0 — стек пуст. */
int stack_pop(Stack *s, void *out)
{
    if (s->count == 0) {
        return 0;
    }
    s->count--;
    if (out != NULL) {
        memcpy(out, s->data + s->count * s->elem_size, s->elem_size);
    }
    return 1;
}
```

Использование одинаково для разных типов (в этом кратком наброске результаты `stack_init` и `stack_push` не проверяются; в файле они проверены):

```
Stack si; stack_init(&si, sizeof(int));
int x = 10; stack_push(&si, &x);
```

```
Stack sd; stack_init(&sd, sizeof(double));
double d = 3.14; stack_push(&sd, &d);
```

В `main` файла в целочисленный стек кладут 10, 20, 30, 40, 50, а в вещественный — 3.14 и 2.71, потом обе стопки вычерпывают до дна. Запуск:

```
int:    50 40 30 20 10
double: 2.71 3.14
```

Порядок обратный порядку вставки — это и есть стек (LIFO, последним пришёл — первым ушёл). Обрати внимание: код самого стека между двумя блоками не изменился ни на строку, поменялся только `elem_size`.

Объяснение.

- буфер — `char *`, потому что `sizeof(char)` по определению равен 1; адресная арифметика `data + count * elem_size` ходит по буферу побайтно и попадает ровно в начало нужного элемента. По `void *` так шагать **нельзя**: стандарт разрешает `+` и `-` только указателям на типы с известным размером, а у `void` размера нет. GCC и clang разрешают это как своё расширение и по умолчанию молчат; с флагом `-pedantic` clang честно пишет `arithmetic on a pointer to void is a GNU extension`.
- `const void *elem` в `push` — «адрес элемента, тип неизвестен, менять не буду». Вызывающий передаёт `&x`. Внутри `memcpy` копирует `elem_size` байт — ровно один элемент.
- `pop` пишет результат по адресу `out` (если он не `NULL`) — тем же `memcpy`. Так мы «возвращаем» значение неизвестного типа: через выходной указатель, а не через `return`.
- `realloc` с удвоением вместимости — амортизированная вставка за O(1) (см. растущий массив в главе 11): отдельная вставка иногда стоит O(n) на копирование, но в сумме n вставок стоят O(n). Результат кладём во **временный** `bigger` и проверяем на `NULL`, иначе при неудаче потеряем старый блок.
- проверка `s->capacity > SIZE_MAX / 2 / s->elem_size` — защита от переполнения при умножении. `size_t` беззнаковый, и если `new_cap * elem_size` не влезет, произведение молча завернётся по модулю и станет маленьким: `realloc` выделит крошечный блок, а мы запишем за его границу. Делением такую проверку можно сделать, ни разу не переполнившись. `SIZE_MAX` — самое большое значение `size_t`, оно объявлено в `<stdint.h>`. По той же причине `stack_init` отказывается от нулевого и гигантского `elem_size`: на ноль делить нельзя.

Плата за обобщённость. Нет проверки типов: положил `int`, достал как `double` — компилятор смолчит, а ты получишь мусор. И `memcpy` с размером, известным только во время работы, может оказаться медленнее прямого присваивания. Зато один код работает со всем.

Доп. вопросы.

- *Почему буфер `char *`, а не `void *`?* Арифметика указателей, см. выше.
- *Как сделать обобщённую функцию печати?* Передать ещё указатель на функцию `void print(const void *)` — см. задачу 5 и тренировочную задачу 3.
- *Чем такой стек похож на `qsort`?* Тем же приёмом: `qsort` тоже получает `void *` на начало массива и размер элемента и двигает байты, не зная типа. Разница в том, что ей ещё нужен компаратор — указатель на функцию, которая знает тип и умеет сравнить два элемента.

Задача 4. Свои `atoi` и `itoa` с обработкой краёв

Условие. Напиши `my_atoi` (строка → `int`) и `my_itoa` (`int` → строка). Корректно обработай знак, пробелы, мусор и **переполнение**, включая `INT_MIN`.

Идея. В `atoi` копим число пошагово: `result = result*10 + цифра`. Опасность — переполнение `int`. Поэтому копим в `long long` (стандарт гарантирует ему не меньше 64 бит, а `int` на всех распространённых платформах 32-битный) и после каждой цифры проверяем выход за `INT_MAX`/`INT_MIN`. В `itoa` главная каверза — `INT_MIN`: его модуль `-INT_MIN` в `int` **не помещается** (диапазон несимметричен: от −2147483648 до 2147483647). Спасает работа в `long long`.

Почему именно `long long`, а не `long`? Стандарт обещает про `long` только «не меньше 32 бит». На 64-битных macOS и Linux он 8 байт, а на 64-битной Windows — 4, и там промежуточное `result * 10` переполнилось бы прямо внутри «защиты от переполнения». `long long` по стандарту не меньше 64 бит везде — вот его и берём.

Решение (`code/21-04-atoi-itoa.c`):

```
#include <limits.h>

/* Строка -> int. Обрабатываем пробелы, знак и переполнение.
   Возвращает 1 при успехе (результат в *out), 0 при ошибке
   (нет цифр или число не влезает в int). */
int my_atoi(const char *s, int *out)
{
    while (*s == ' ' || *s == '\t') {   /* пропускаем ведущие пробелы */
        s++;
    }

    int sign = 1;
    if (*s == '+' || *s == '-') {
        if (*s == '-') {
            sign = -1;
```

```
        }
        s++;
    }

    if (*s < '0' || *s > '9') {          /* ни одной цифры — это не число */
        return 0;
    }

    long long result = 0;                /* копим в long long: он шире int */
    while (*s >= '0' && *s <= '9') {
        result = result * 10 + (*s - '0');
        if (sign == 1 && result > INT_MAX) {       /* вышли за +предел */
            return 0;
        }
        if (sign == -1 && -result < INT_MIN) {     /* вышли за -предел */
            return 0;
        }
        s++;
    }

    *out = (int)(sign * result);
    return 1;
}

/* int -> строка. buf должен быть не меньше 12 байт:
   знак + 10 цифр + завершающий '\0'. */
void my_itoa(int n, char *buf)
{
    char tmp[12];
    int i = 0;
    int neg = 0;

    /* Берём значение в long long: -INT_MIN в int не влезает. */
    long long value = n;
    if (value < 0) {
        neg = 1;
        value = -value;
    }
```

```
    do {                            /* цифры выходят задом наперёд */
        tmp[i++] = (char)('0' + value % 10);
        value /= 10;
    } while (value > 0);

    int j = 0;
    if (neg) {
        buf[j++] = '-';
    }
    while (i > 0) {                         /* разворачиваем в buf */
        buf[j++] = tmp[--i];
    }
    buf[j] = '\0';
}
```

Вывод программы целиком:

```
my_atoi("42") = 42
my_atoi("  -17") = -17
my_atoi("+5") = 5
my_atoi("2147483647") = 2147483647
my_atoi("2147483648") = ошибка
my_atoi("-2147483648") = -2147483648
my_atoi("abc") = ошибка
my_atoi("-2147483649") = ошибка
---
my_itoa(0) = "0"
my_itoa(42) = "42"
my_itoa(-17) = "-17"
my_itoa(2147483647) = "2147483647"
my_itoa(-2147483648) = "-2147483648"
```

Обрати внимание на две соседние строки: `2147483647` разбирается, а `2147483648` — уже нет. Это ровно граница `int`, и проверка на переполнение срабатывает на единицу за ней, а не «примерно там». Симметрично внизу: `-2147483648` разбирается (это минимальный `int`), `-2147483649` — ошибка.

Объяснение.

- `*s - '0'` — перевод символа-цифры в число. Цифры в таблице ASCII идут подряд: `'7' - '0' == 7` (см. главу 10).
- проверки переполнения **после каждой цифры**: для плюса `result` превысил `INT_MAX`, для минуса `-result` ушёл ниже `INT_MIN`. Важно, что проверка стоит внутри цикла, а не после него. Пока проверка не сработала, `result` не больше 2147483648 (это модуль `INT_MIN`), и следующая цифра даст не больше `2147483648 * 10 + 9 = 21474836489` — это легко влезает в `long long` (его потолок около $9.2 \cdot 10^{18}$). Значит, само промежуточное вычисление никогда не переполнится, сколько бы цифр ни было во входе: строку из 23 девяток функция честно отвергает.
- в `my_itoa` цикл `do { } while` обязателен: иначе для `n == 0` мы не выведем ни одной цифры. `do/while` гарантирует хотя бы один проход.
- `value = -value` для отрицательного — безопасно только потому, что `value` это `long long`. Сделай мы `int value = n; value = -value;` при `n == INT_MIN` получили бы **UB** (переполнение знакового): у −2147483648 нет положительной пары в `int`.
- размеры буферов: у 32-битного `int` не больше 10 цифр. В `tmp` лежат только цифры, так что 12 байт там с запасом. В `buf` идут знак, 10 цифр и `'\0'` — ровно 12 байт, поэтому `buf` должен быть не меньше
  12.

Что функция считает числом. Мы пропускаем только пробелы и табы (стандартная `atoi` пропускает всё, что `isspace` считает пробельным, включая `'\n'`). Разбор останавливается на первой не-цифре, и хвост молча игнорируется — как у `atoi`. Я проверил края: `""` и `"-"` — ошибка (нет цифр), `"12abc"` — 12, `"-0"` — 0, строка `"00000000000000000000042"` — 42 (ведущие нули не раздувают `result`), `"\n5"` — ошибка. Если хвост должен считаться ошибкой, проверь после цикла, что `*s == '\0'`.

Доп. вопросы.

- *Почему `abs(INT_MIN)` — это UB?* Результат, 2147483648, не помещается в `int`, а стандарт (C11, 7.22.6.1) прямо называет такой случай неопределённым поведением.
- *Как `my_itoa` обобщить на любую систему счисления?* Делить не на 10, а на `base`, а цифры брать из строки `"0123456789abcdef"` — решение в ответе к тренировочной задаче 4.
- *Что вернёт стандартный `atoi("99999999999")`?* Неизвестно: если значение не помещается в `int`, у `atoi` UB (C11, 7.22.1). Поэтому в реальном коде берут `strtol`: при выходе за пределы `long` она возвращает `LONG_MAX` или `LONG_MIN` и ставит `errno = ERANGE`. Но на macOS и Linux `long` 64-битный, и `"99999999999"` в него влезает без ошибки — проверку на границы `int` всё равно придётся сделать самому.

Задача 5. Таблица обработчиков на указателях на функции

Условие. Сделай диспетчер: по символу операции (+, −, ∗) вызывается нужная функция. Заодно покажи компаратор для `qsort`.

Идея. Функцию в Си можно передать как аргумент и хранить в переменной — через **указатель на функцию**. Связку «ключ → функция» складываем в массив структур (таблицу). Вместо длинного `switch` пробегаем таблицу и зовём найденный обработчик. Это тот же приём, что делает `qsort` универсальной: ей передают указатель на компаратор.

Решение (`code/21-05-funcptr.c`):

```c
#include <stdio.h>
#include <stdlib.h>

/* Три простые операции — все одной формы int(int,int). */
int op_add(int a, int b) { return a + b; }
int op_sub(int a, int b) { return a - b; }
int op_mul(int a, int b) { return a * b; }

/* Таблица обработчиков: символ операции -> указатель на функцию. */
typedef struct {
    char symbol;
    int (*fn)(int, int);   /* указатель на функцию (int, int) -> int */
} Handler;

/* Компаратор для qsort: сортировка по УБЫВАНИЮ, без вычитания
   (вычитание a-b может переполнить int, см. главу 8). */
int cmp_desc(const void *pa, const void *pb)
{
    int a = *(const int *)pa;
    int b = *(const int *)pb;
    return (a < b) - (a > b);   /* +1 если a<b (a идёт позже), -1 если a>b */
}

int main(void)
{
    Handler table[] = {
        {'+', op_add},
        {'-', op_sub},
        {'*', op_mul},
```

```
    };
    int n = (int)(sizeof table / sizeof table[0]);

    int x = 6, y = 4;
    char ops[] = {'+', '-', '*'};
    for (int k = 0; k < 3; k++) {
        char op = ops[k];
        for (int i = 0; i < n; i++) {
            if (table[i].symbol == op) {           /* нашли обработчик */
                int r = table[i].fn(x, y);         /* вызов через указатель */
                printf("%d %c %d = %d\n", x, op, y, r);
                break;
            }
        }
    }

    printf("---\n");

    int a[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int m = (int)(sizeof a / sizeof a[0]);
    qsort(a, (size_t)m, sizeof a[0], cmp_desc);
    for (int i = 0; i < m; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

Запуск:

```
6 + 4 = 10
6 - 4 = 2
6 * 4 = 24
---
9 6 5 4 3 2 1 1
```

Сверху — три вызова через таблицу, снизу — тот же массив, отсортированный по убыванию силами `qsort` с нашим компаратором.

Объяснение.

- `int (*fn)(int, int);` — самая пугающая запись в Си. Читай изнутри: `fn` — это указатель (`*fn`), на функцию, принимающую `(int, int)` и возвращающую `int`. **Скобки вокруг `*fn` обязательны**: без них `int *fn(int, int)` означало бы «функция, возвращающая `int *`» — совсем другое.
- имя функции `op_add` в выражении само превращается в указатель на неё (так записано в стандарте, C11 6.3.2.1), поэтому `{'+', op_add}` записывает указатель. Амперсанд не нужен (хотя `&op_add` тоже допустимо и означает то же).
- `sizeof table / sizeof table[0]` — размер всей таблицы в байтах, делённый на размер одной записи, то есть число записей (здесь 3).
- `table[i].fn(x, y)` — вызов через указатель. Можно писать и `(*table[i].fn)(x, y)` — это эквивалентно.
- `cmp_desc` возвращает `(a < b) - (a > b)`. Сравнение в Си даёт 1 или 0, поэтому при `a = 3, b = 9` выйдет `1 - 0 = 1`, при `a = 9, b = 3` — `0 - 1 = -1`, при равных — `0 - 0 = 0`. `qsort` ставит `a` раньше `b`, когда компаратор вернул отрицательное число, — значит, большее идёт раньше, это и есть убывание. И никакого опасного `a - b`, которое может переполнить `int` на больших числах (см. ловушку в главе 8).

Где это применяют по-настоящему. Таблицы команд в интерпретаторах, обработчики событий, колбэки (callbacks), state-машины, плагины. Указатель на функцию — основа любого «подключаемого» поведения в Си.

Доп. вопросы.

- *Что такое `typedef int (*BinOp)(int, int);`?* Короткое имя для типа «указатель на функцию `(int, int) -> int`». После него поле структуры пишется просто `BinOp fn;`, а параметр — `BinOp op`.
- *Чем `qsort_r` удобнее обычной `qsort`?* Она передаёт в компаратор ещё один указатель — «контекст», например направление сортировки, — без глобальных переменных. Но это **не** стандартный Си: `qsort_r` есть в BSD/macOS и в glibc, причём с разным порядком аргументов, а в стандарте C11 её нет (там есть необязательная `qsort_s` из приложения K, которую macOS и glibc не поддерживают).
- *Можно ли сделать массив указателей на функции без структуры?* Да: `int (*ops[])(int, int) = {op_add, op_sub};`, а вызов — `ops[0](6, 4)`.

Задача 6. Кольцевой буфер (ring buffer)

Условие. Реализуй кольцевой буфер фиксированного размера: запись в хвост, чтение из головы, переполнение и опустошение — корректно.

Идея. Это очередь (FIFO) на кольцевом буфере — ты уже встречал её в главе 11, здесь она оформлена аккуратной структурой. Храним массив, индекс головы `head` и количество `count`.

Хвост вычисляем: `(head + count) % CAP`. Остаток от деления `%` заворачивает индекс с конца массива в начало — массив словно свёрнут в кольцо. Пуст при `count == 0`, полон при `count == CAP`.

Решение (`code/21-06-ringbuf.c`):

```c
#define CAP 4   /* размер кольца */

/* Кольцевой буфер на массиве фиксированного размера.
   Храним голову и количество; хвост вычисляем. */
typedef struct {
    int data[CAP];
    int head;     /* индекс первого элемента */
    int count;    /* сколько элементов лежит */
} Ring;

/* Кладём в хвост. 0 — буфер полон. */
int ring_push(Ring *r, int value)
{
    if (r->count == CAP) {
        return 0;
    }
    int tail = (r->head + r->count) % CAP;   /* первое свободное место */
    r->data[tail] = value;
    r->count++;
    return 1;
}

/* Забираем из головы в *out. 0 — буфер пуст. */
int ring_pop(Ring *r, int *out)
{
    if (r->count == 0) {
        return 0;
    }
    *out = r->data[r->head];
    r->head = (r->head + 1) % CAP;           /* голова заворачивается */
    r->count--;
    return 1;
}
```

Объяснение.

- почему хранить `count`, а не индекс `tail`? Если держать `head` и `tail`, возникает путаница: при `head == tail` буфер пуст **или** полон — состояния неразличимы. С `count` всё однозначно.
- `(head + count) % CAP` — позиция за последним элементом, она же первое свободное место. Когда сумма дорастает до `CAP`, `%` сбрасывает её в 0.
- `head = (head + 1) % CAP` — после чтения голова шагает вперёд и тоже заворачивается.
- никаких `malloc` — буфер на массиве фиксированного размера внутри самой структуры, освобождать нечего.
- `%` здесь безопасен: `head` и `count` никогда не бывают отрицательными, а для неотрицательных чисел остаток `%` в Си совпадает с математическим (для отрицательных — нет, см. задачу 10).

Вывод целиком:

```
push 1: ok
push 2: ok
push 3: ok
push 4: ok
push 5: полон
push 6: полон
pop 1
pop 2
pop 3
pop 4
pop 7
pop 8
```

Читается он так. Первые четыре `push` заполнили кольцо (`CAP` равен 4), пятый и шестой уже не влезли. Дальше два `pop` забрали `1` и `2` — и в освободившиеся ячейки легли `7` и `8`, завернувшись в начало массива. Финальный `while` вычерпывает остаток: сначала «старые» `3` и `4`, и только потом `7` и `8`. Порядок FIFO соблюдён, хотя физически в массиве `7` и `8` лежат раньше `3` и `4`.

Зачем нужно. Кольцевые буферы — это потоки аудио/видео, буфер клавиатуры, логи фиксированного размера, обмен данными между потоком- производителем и потоком-потребителем. Размер заранее известен, память не скачет.

Доп. вопросы.

- *Чем плох вариант с двумя индексами без `count`?* Неразличимы «пусто» и «полно»; лечат «потерей одной ячейки» (буфер считается полным, когда `(tail + 1) % CAP == head`, как в главе 11) или отдельным флагом.
- *Почему `CAP` — степень двойки иногда быстрее?* Для неотрицательного `x` остаток `x % 4` равен `x & 3`: это младшие два бита. Побитовое И дешевле деления. С константой `CAP` компилятор делает такую замену сам, особенно если индексы беззнаковые.
- *Как сделать буфер растущим?* Решение — в ответе к тренировочной задаче 6.

Задача 7. Хеш-таблица «строка → счётчик» с цепочками

Условие. Посчитай, сколько раз встретилось каждое слово. Реализуй хеш-таблицу с разрешением коллизий **цепочками**.

Идея. Хеш-таблица — массив «корзин». Функция-хеш превращает строку в число, остаток от деления на число корзин даёт индекс. Разные строки могут попасть в одну корзину — это **коллизия**. Решаем цепочками: каждая корзина это связный список записей, в ней лежащих. Поиск: вычислили корзину, прошли её цепочку, сравнили ключи `strcmp`.

Решение (`code/21-07-hashtable.c`):

```c
#define NBUCKETS 16   /* число корзин (степень двойки — удобно) */

/* Запись в цепочке: ключ-строка, счётчик и указатель на следующую
   запись в той же корзине (разрешение коллизий цепочками). */
typedef struct Entry {
    char *key;
    int count;
    struct Entry *next;
} Entry;

typedef struct {
    Entry *buckets[NBUCKETS];
} HashTable;

/* Хеш строки djb2 — простой и хороший для коротких ключей. */
unsigned long hash_str(const char *s)
{
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*s++) != 0) {
        h = h * 33u + (unsigned long)c;
```

```
    }
    return h;
}

void table_init(HashTable *t)
{
    for (int i = 0; i < NBUCKETS; i++) {
        t->buckets[i] = NULL;
    }
}

/* Увеличить счётчик слова на 1 (или добавить новое). 0 — нет памяти. */
int table_inc(HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % NBUCKETS;

    /* проходим цепочку этой корзины — нет ли уже такого ключа */
    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {
            e->count++;
            return 1;
        }
    }

    /* нового ключа нет — создаём запись и вешаем в начало цепочки */
    Entry *e = malloc(sizeof(Entry));
    if (e == NULL) {
        return 0;
    }
    e->key = malloc(strlen(key) + 1);   /* +1 под '\0' */
    if (e->key == NULL) {
        free(e);
        return 0;
    }
    strcpy(e->key, key);
    e->count = 1;
    e->next = t->buckets[idx];
    t->buckets[idx] = e;
```

```
    return 1;
}
```

В `main` мы прогоняем список слов `{"cat", "dog", "cat", "bird", "dog", "cat"}` и потом спрашиваем счётчики, в том числе у слова, которого в таблице нет:

```
cat  = 3
dog  = 2
bird = 1
fish = 0
```

Ноль у `fish` — это не «встретилось ноль раз», а «не нашли». Такой интерфейс удобен для счётчиков, но для таблицы «ключ → значение» он бы не годился: там надо отличать «нет ключа» от «значение равно нулю», и функция должна возвращать признак успеха отдельно (как `lru_get` в задаче 9).

Объяснение.

- `hash_str` (алгоритм djb2 Дэниела Бернстайна): начинаем с 5381, на каждый символ `h = h*33 + c`. Множитель 33 и старт 5381 подобраны опытным путём — на обычных словах функция неплохо разбрасывает ключи по корзинам. Тип `unsigned long`, чтобы переполнение было определённым: для беззнаковых это не UB, а честный остаток по модулю 2^{64} (у `unsigned long` на этом маке 64 бита; см. каверзу 1 про UB).
- `(unsigned char)*s` — важная деталь: `char` может быть знаковым (на этом маке так и есть), и байты с кодом больше 127 — например, буквы кириллицы в UTF-8 — превратились бы в отрицательные числа. Хеш всё равно считался бы, но **другой**, чем на платформе с беззнаковым `char`: я проверил на букве «ж» — 5866955 с приведением и 5858251 без него. Одна и та же строка на разных машинах попадала бы в разные корзины. Приведение к `unsigned char` делает хеш одинаковым везде.
- коллизии: при `strcmp` мы **сравниваем сами ключи**, а не только хеши. Два разных слова могут попасть в одну корзину, и `strcmp` их различит. Это не выдумка: в нашем примере `dog` (хеш 193489663) и `fish` (хеш 6385224719) оба дают остаток 15 от деления на 16. Поиск `fish` приходит в корзину 15, находит там `dog`, `strcmp` говорит «не то» — и функция честно возвращает 0.
- **владение памятью**: ключ копируем своим `malloc(strlen(key)+1)` + `strcpy`. Нельзя просто сохранить переданный указатель — строка вызывающего может измениться или исчезнуть. На каждый `malloc` есть `free` в `table_free` (сначала `free(e->key)`, потом `free(e)`).
- вставка в **начало** цепочки — O(1), порядок внутри корзины неважен.

Про сложность. Главное число хеш-таблицы — **коэффициент загрузки** (load factor) $\alpha = n / m$, где n — сколько записей лежит, m — сколько корзин. Это средняя длина цепочки. У нас после шести слов записей три (`cat`, `dog`, `bird`), корзин 16: $\alpha = 3/16 \approx 0.19$.

- **В среднем** (если хеш разбрасывает ключи равномерно) поиск проходит цепочку средней длины α, то есть стоит $O(1 + \alpha)$. Пока α ограничено константой — скажем, не больше 1, — это $O(1)$.
- **В худшем случае** все ключи попадают в одну корзину, цепочка длиной n, и поиск — $O(n)$. От этого не спасает никакое число корзин: это вопрос хеш-функции и входных данных (злоумышленник может нарочно подобрать ключи с одинаковым хешем).
- **Наша таблица** держит 16 корзин навсегда. Значит, при n словах $\alpha = n/16$ растёт вместе с n, и даже средний поиск — $O(n/16)$, то есть $O(n)$. Для счётчика в несколько сотен слов это не страшно, но «O(1)» у этого кода честно не выполняется.

Лечат **рехешированием**: когда α превысит порог (обычно от 0.75 до 1–2), число корзин удваивают и перекладывают все записи. Одно рехеширование стоит $O(n)$, но случается всё реже, и в сумме вставка выходит **амортизированно** $O(1)$ — тот же расчёт, что у растущего массива в главе 11. Не путай: «в среднем» — про случайность входа, «амортизированно» — про сумму по длинной серии операций.

Доп. вопросы.

- *Чем цепочки отличаются от открытой адресации?* При открытой адресации записи лежат прямо в массиве, а при коллизии ищут соседнюю свободную ячейку (линейное пробирование — следующую, квадратичное — через 1, 4, 9… шагов). Плюсы: нет `malloc` на каждую запись, данные лежат подряд и хорошо дружат с кэшем. Минусы: α должен оставаться меньше 1, производительность резко падает при заполнении, а удаление требует особых «надгробий» на месте удалённых записей.
- *Когда делать рехеш?* Когда α перешёл выбранный порог — см. выше; реализация — в ответе к тренировочной задаче 5.
- *Почему число корзин берут простым или степенью двойки?* Остаток от деления на простое число зависит от всех битов хеша, и это сглаживает слабую хеш-функцию. Степень двойки позволяет заменить `%` на быстрое `&`, но тогда в индекс идут только младшие биты хеша, и хеш должен хорошо перемешивать именно их.

Задача 8. Точка пересечения двух односвязных списков

Условие. Даны два односвязных списка, которые в какой-то момент сливаются в общий хвост. Найди **узел слияния** (по адресу, не по значению). Если не пересекаются — `NULL`.

Идея. Наивно — для каждого узла A проверять весь B: $O(n \cdot m)$. Умнее: если списки сливаются, у них **общий хвост одинаковой длины**. Значит, разница только в длине «головных» частей. По-

считаем длины, проматаем длинный список вперёд на разницу — и теперь оба узла на равном расстоянии до конца. Шагаем параллельно: первый узел, где **адреса совпадут**, и есть точка пересечения. Это O(n + m) времени и O(1) памяти.

Решение (`code/21-08-list-intersection.c`):

```c
typedef struct Node {
    int value;
    struct Node *next;
} Node;

int length(Node *head)
{
    int len = 0;
    for (Node *p = head; p != NULL; p = p->next) {
        len++;
    }
    return len;
}

/* Узел пересечения двух списков или NULL.
   Идея: выравниваем старты. Длинный список проматываем вперёд на
   разницу длин — теперь до конца обоим идти поровну. Дальше шагаем
   параллельно: где адреса совпадут, там и общий узел. */
Node *intersection(Node *a, Node *b)
{
    int la = length(a);
    int lb = length(b);

    while (la > lb) { a = a->next; la--; }
    while (lb > la) { b = b->next; lb--; }

    while (a != b) {        /* сравниваем АДРЕСА узлов, не значения */
        a = a->next;
        b = b->next;
    }
    return a;               /* общий узел или NULL */
}
```

В `main` мы строим общий хвост `7 -> 8 -> 9 -> NULL`, вешаем на него список A (`1 -> 2 -> 3 ->` `хвост`) и список B (`4 -> 5 -> хвост`), а потом ищем точку слияния:

```
Пересечение в узле со значением 7
```

Длины 6 и 5, разница 1: указатель по списку A делает один холостой шаг, и дальше оба бегут синхронно — совпадение случается ровно на первом узле общего хвоста.

Объяснение.

- ключевая мысль — сравнивать `a != b`, то есть **указатели**, а не `a->value != b->value`. Пересечение списков означает, что с какого-то узла они физически идут по **одной и той же** памяти. Значения могут совпадать и случайно, а адрес узла уникален.
- после выравнивания обоим до конца идти поровну. Если есть общий хвост, бегунки войдут в него синхронно и встретятся на узле слияния. Если пересечения нет, оба одновременно дойдут до `NULL` — и `a == b == NULL`, цикл закончится, вернём `NULL`. Один и тот же код покрывает оба случая.
- освобождение тут с подвохом: общий хвост нельзя `free` дважды. Сначала чистим уникальные части A и B **до** общего узла `tail`, потом — сам хвост один раз. Двойной `free` — это UB (см. блок ниже).

Доп. вопросы.

- *А если в списках может быть цикл?* Тогда `length` зациклится. Сначала проверь оба списка алгоритмом Флойда — он и начало цикла находит, всё это разобрано в Приложении Б, задача 3.
- *Как решить без подсчёта длин?* Два указателя: дойдя до конца своего списка, каждый перепрыгивает на начало чужого. Оба пройдут одинаковый путь `la + lb` и встретятся на пересечении (или вместе придут в `NULL`).
- *Почему сравнение по значению дало бы неверный ответ?* Представь A = `1 -> 7 -> 8 -> 9` и B = `5 -> 7 -> 8 -> 9`, где у каждого списка свои узлы: значения совпадут на 7, хотя списки вообще не пересекаются.

Задача 9. Набросок LRU-кэша

Условие. Реализуй кэш на N записей с вытеснением **давно не использованных** (LRU — Least Recently Used). `get` и `put` — в среднем за O(1).

Идея. Нужны две вещи сразу: быстрый **поиск по ключу** и быстрое определение, **кто давно не использовался**. Поиск даёт хеш-таблица. Порядок использования — **двусвязный список**: голова это самый свежий, хвост — самый старый кандидат на вылет. Каждый узел живёт одновременно в обеих структурах. `get`/`put` находят узел через хеш и переставляют его в голову списка. При переполнении удаляем хвост — и из списка, и из хеша.

Ключевой код (`code/21-09-lru.c`; функции `lru_init`, `list_unlink`, `list_push_front`, `hash_find` и `lru_free` — в файле, они короткие и прямолинейные):

```
typedef struct Node {
    int key;
    int value;
    struct Node *prev;    /* соседи в списке свежести */
    struct Node *next;
    struct Node *hnext;   /* следующий в хеш-корзине */
} Node;

typedef struct {
    int capacity;         /* сколько записей помещается */
    int size;             /* сколько записей сейчас */
    Node *head;           /* самый свежий */
    Node *tail;           /* самый старый */
    Node **buckets;       /* массив корзин: их столько же, сколько мест */
    unsigned nbuckets;
} LRU;

/* убрать узел из его хеш-цепочки */
static void hash_remove(LRU *c, Node *n)
{
    Node **pp = &c->buckets[bucket_of(c, n->key)];
    while (*pp != n) {
        pp = &(*pp)->hnext;
    }
    *pp = n->hnext;
}

/* get: 1 и значение в *out, если ключ есть. Обращение освежает запись. */
int lru_get(LRU *c, int key, int *out)
{
    Node *n = hash_find(c, key);
    if (n == NULL) {
        return 0;
    }
    list_unlink(c, n);
    list_push_front(c, n);   /* подняли наверх как самый свежий */
```

```
    *out = n->value;
    return 1;
}

/* put: 1 — записано, 0 — не хватило памяти (кэш тогда не меняется). */
int lru_put(LRU *c, int key, int value)
{
    Node *n = hash_find(c, key);
    if (n != NULL) {                /* ключ уже есть — обновим и освежим */
        n->value = value;
        list_unlink(c, n);
        list_push_front(c, n);
        return 1;
    }

    n = malloc(sizeof(Node));       /* сначала память, потом вытеснение */
    if (n == NULL) {
        return 0;
    }

    if (c->size == c->capacity) {   /* кэш полон — вытесняем хвост */
        Node *old = c->tail;
        list_unlink(c, old);
        hash_remove(c, old);
        free(old);
        c->size--;
    }

    n->key = key;
    n->value = value;
    unsigned b = bucket_of(c, key);
    n->hnext = c->buckets[b];       /* в начало хеш-цепочки */
    c->buckets[b] = n;
    list_push_front(c, n);          /* и в голову списка свежести */
    c->size++;
    return 1;
}
```

Объяснение.

- узел в **двух** структурах: `prev/next` — порядок свежести, `hnext` — цепочка хеша. Двусвязный список позволяет вынуть узел из середины за O(1): соседи известны сразу (у односвязного пришлось бы искать предыдущий проходом по списку).
- `list_unlink` + `list_push_front` — «освежить»: при каждом обращении узел переезжает в голову. Значит, у хвоста всегда самый давний доступ.
- в `hash_remove` интересна строка `Node **pp = &c->buckets[...]` — **указатель на указатель**. Он позволяет удалить узел из односвязной хеш-цепочки без отдельной проверки «это голова или середина»: `pp` всегда указывает на то поле (`hnext` предыдущего узла или сам `buckets[b]`), которое надо перенаправить. Классический senior-приём.
- `lru_put` сначала выделяет память под новый узел и только потом вытесняет старый. Если `malloc` не удался, кэш остаётся нетронутым, а функция возвращает 0. В обратном порядке мы бы выкинули запись и ничего не положили взамен.
- `lru_init` отказывается от вместимости меньше 1: при `capacity == 0` условие «кэш полон» было бы верно сразу, а хвоста ещё нет, и `list_unlink(c, NULL)` разыменовал бы `NULL`.

Правда ли тут O(1)? Честно разберём по частям. Операции со списком (`list_unlink`, `list_push_front`) — O(1) всегда: несколько присваиваний. Поиск `hash_find` и удаление `hash_remove` проходят одну хеш-цепочку, и их цена — длина цепочки. Корзин мы заводим **столько же, сколько мест в кэше**, поэтому записей никогда не больше, чем корзин: коэффициент загрузки $\alpha \le 1$ (см. задачу 7). Значит, **в среднем** цепочка короткая, и `get`/`put` — O(1) в среднем. В **худшем** случае (все ключи дают один и тот же остаток `key % nbuckets`, например кратны вместимости) цепочка одна на всех, и операции стоят O(N), где N — вместимость кэша.

Частая ошибка на собесе — завести фиксированное число корзин, скажем 16, как в задаче 7. Тогда на кэше в миллион записей средняя цепочка была бы длиной 1 000 000 / 16 = 62 500, и обещанное O(1) превратилось бы в O(N). Поэтому корзины выделяются в `lru_init` под размер кэша.

Я прогнал кэш против простой «эталонной» реализации (массив и перебор): вместимости от 1 до 9, по 200 случайных серий из 200 операций `get`/`put` с ключами от −11 до 11, всего 360 тысяч операций под `-fsanitize=address,undefined`. Результаты совпали везде, санитайзеры молчали.

Вывод программы подтверждает логику: после `get 1` ключ 1 стал свежим, и `put 3` вытеснил именно 2 (его давно не трогали), а 1 уцелел:

```
get 1 = 100 (теперь 1 свежий)
get 2 = промах (вытеснен)
get 1 = 100
get 3 = 300
```

Доп. вопросы.

- *Почему именно **дву**связный список, а не односвязный?* Удаление узла из середины за O(1), см. выше.
- *Чем LRU отличается от LFU и FIFO?* LRU выкидывает то, к чему дольше всего не обращались; LFU (Least Frequently Used) — то, к чему обращались реже всего; FIFO — то, что раньше всех положили, даже если им пользуются постоянно.
- *Как сделать кэш потокобезопасным?* Самое простое — один мьютекс вокруг каждой операции. Заметь: даже `get` меняет список, поэтому «читателей» тут тоже надо запирать.

Задача 10. Разворот числа с защитой от переполнения

Условие. Развернуть цифры знакового `int` (`123 → 321`, `-456 → -654`). Если перевёрнутое число не помещается в `int` — сообщить об ошибке, а не выдать мусор.

Идея. Снимаем последнюю цифру `n % 10`, отбрасываем её `n /= 10`, дописываем в результат `result = result*10 + цифра`. Ловушка та же, что в `atoi`: `result*10` может переполнить `int`. В `atoi` мы спаслись более широким `long long`, но на собеседовании эту задачу часто дают с условием «более широких типов нет, работай в `int`». Поэтому проверяем **до** умножения: не выйдет ли `result*10 + цифра` за границу.

Решение (`code/21-10-reverse-int.c`):

```c
#include <limits.h>

int reverse_int(int n, int *out)
{
    int result = 0;
    while (n != 0) {
        int digit = n % 10;   /* знак сохраняется: -123 % 10 == -3 (C99) */
        n /= 10;

        /* Проверяем ДО умножения: влезет ли result*10 + digit.
           Для положительной части сравниваем с INT_MAX, для
           отрицательной — с INT_MIN. */
        if (result > INT_MAX / 10 ||
            (result == INT_MAX / 10 && digit > INT_MAX % 10)) {
            return 0;
        }
        if (result < INT_MIN / 10 ||
            (result == INT_MIN / 10 && digit < INT_MIN % 10)) {
```

```
            return 0;
        }
        result = result * 10 + digit;
    }
    *out = result;
    return 1;
}
```

Запуск на наборе из семи чисел:

```
reverse(123) = 321
reverse(-456) = -654
reverse(120) = 21
reverse(0) = 0
reverse(1534236469) = переполнение
reverse(1000000003) = переполнение
reverse(-2147483648) = переполнение
```

Три случая, ради которых задачу и дают. `120` разворачивается в `21`, а не в `021` — ведущий ноль просто исчезает, потому что мы работаем с числом, а не со строкой. `1000000003` перевернулось бы в `3000000001` — это больше `INT_MAX`. А `INT_MIN` не разворачивается: `-8463847412` не влезает в 32-битный `int`.

Объяснение.

- работаем **без** перевода в `unsigned` или `long` — прямо в `int`. Поэтому `result` хранит уже со знаком: для отрицательного `n` и `digit`, и `result` отрицательны. Так работает деление в Си начиная с C99: частное округляется **к нулю** (`-123 / 10 == -12`), а остаток подбирается так, чтобы выполнялось `a == (a / b) * b + a % b`, — то есть `-123 % 10 == -3` и знак остатка совпадает со знаком делимого. Это **не** математический остаток: в математике остаток от деления −123 на 10 равен 7 (−123 = −13·10 + 7). Нам как раз удобен сишный вариант: цифры отрицательного числа выходят отрицательными, и `result` растёт вниз, к `INT_MIN`, без единого взятия модуля.
- проверка «влезет ли `result*10 + digit`» сделана **арифметикой без переполнения**. Подставим числа: `INT_MAX / 10` = 214748364, `INT_MAX % 10` = 7. Если `result` больше 214748364, то `result*10` не меньше 2147483650 — это больше `INT_MAX`, мимо. Если `result` ровно 214748364, влезет только цифра не больше 7 (2147483647 — это `INT_MAX`). Внизу симметрично: `INT_MIN / 10` = −214748364, `INT_MIN % 10` = −8. Так мы ни разу не выходим за границы `int` в самих вычислениях.
- `1534236469` развернётся в `9646324351` — это больше `INT_MAX` (2147483647), поэтому функция честно вернёт «переполнение», а не мусор.

Доп. вопросы.

- *Почему опасно «развернуть, потом проверить»?* Переполнение знакового — это UB ещё **до** проверки, и компилятор вправе выкинуть саму проверку (пример — в каверзе 1).
- *Как развернуть строку на месте?* Два указателя навстречу, см. главу 10.
- *Чем отличается `%` для отрицательных в C89 и C99?* В C89, если хоть один операнд отрицательный, направление округления частного (к нулю или вниз) выбирала реализация, а знак остатка шёл за ним. C99 и C11 закрепили округление к нулю.

Часть 2. Каверзы и теория

Здесь код не пишут — объясняют. Эти вопросы отделяют тех, кто «знает синтаксис», от тех, кто понимает Си. Отвечай по существу и с примером.

Каверза 1. Что такое UB и где оно прячется

Неопределённое поведение (undefined behavior, UB) — это ситуации, для которых стандарт Си **не ставит никаких требований** (так и сказано в определении, C11 3.4.3). Компилятор вправе делать что угодно: выдать ожидаемый результат, выдать мусор, уронить программу или (самое коварное) «оптимизировать» твой код, считая, что UB невозможно. Программа с UB не «иногда работает» — она **некорректна**, даже если сегодня выводит правильное.

Запомни главное правило: **на UB полагаться нельзя**. Поведение может меняться от компилятора, флагов оптимизации, версии и даже соседних строк кода. Частые источники:

```c
/* 1. Переполнение знакового int — UB */
int x = INT_MAX;
x = x + 1;              /* НЕ «станет INT_MIN», а UB */

/* 2. Разыменование NULL */
int *p = NULL;
int y = *p;             /* падение или хуже */

/* 3. Выход за границы массива */
int a[3];
a[3] = 5;               /* записали мимо — UB */

/* 4. Чтение неинициализированной локальной переменной */
int z;
printf("%d", z);       /* значение не определено — UB */
```

```c
/* 5. Изменение строкового литерала */
char *s = "hello";
s[0] = 'H';            /* UB; на macOS — падение, см. Приложение Б */

/* 6. Использование памяти после free */
int *q = malloc(sizeof *q);
free(q);
*q = 7;                /* «use after free» — UB */
```

Про пункт 4 есть тонкость для самых въедливых интервьюеров: стандарт (C11, 6.3.2.1) называет UB чтение неинициализированной локальной переменной, у которой ни разу не брали адрес, — ровно наш `z`. Если адрес брали, значение просто «неопределённое», но полагаться на него всё равно нельзя.

Почему переполнение **знакового** — это UB, а беззнакового — нет? Для `unsigned` стандарт прямо говорит: считаем по модулю 2^n, где n — число битов типа (заворачивается). А для знаковых C11 разрешал три разных представления отрицательных чисел (дополнительный код, обратный код и «знак + модуль»), и результат переполнения на них разный, — поэтому его оставили неопределённым. В C23 дополнительный код стал обязательным, но переполнение так и осталось UB: компиляторы слишком активно этим пользуются для оптимизаций. Например, оптимизатор вправе считать выражение `x + 1 > x` заведомо истинным: раз переполнения «не бывает», сравнение всегда верно. Ровно поэтому самодельные проверки вида «сложу, а потом посмотрю, не стало ли меньше» не работают — компилятор их вырезает.

Вот это надо увидеть. Программа:

```c
#include <stdio.h>
#include <limits.h>

int bigger(int x)
{
    return x + 1 > x;    /* «x + 1 больше x?» */
}

int main(void)
{
    volatile int v = INT_MAX;  /* volatile: не даём подставить число */
    printf("%d\n", bigger(v));
    return 0;
}
```

`volatile` (подробно — в каверзе 3) здесь только для того, чтобы компилятор не вычислил всё заранее, ещё при сборке. Запуск:

```
-O0:  0
-O2:  1
```

Без оптимизации процессор честно сложил, `INT_MAX + 1` завернулось в отрицательное число, и сравнение дало 0. С `-O2` clang заменил всю функцию на `return 1;` — я посмотрел ассемблер, там ровно одна команда `movl $1, %eax`. Один и тот же код, два разных ответа.

Спорить, какой из них «правильный», бесполезно: правильного нет. Зато есть инструмент, который скажет правду. Соберём ту же программу с `-fsanitize=undefined`:

```
ovf2.c:6:14: runtime error: signed integer overflow: 2147483647 + 1
cannot be represented in type 'int'
SUMMARY: UndefinedBehaviorSanitizer: undefined-behavior ovf2.c:6:14
```

Санитайзер не гадает «что напечатается» — он прямо говорит: здесь UB. Это и есть правильный ответ на такой вопрос на собеседовании.

Лечение: не выходить за границы, инициализировать переменные, проверять указатели на `NULL`, для изменяемых строк заводить `char s[] = "hello";` (массив, а не указатель на литерал), не трогать память после `free`. Под рукой держи санитайзеры: `cc -fsanitize=address,undefined` ловит многие UB на лету — и именно так проверены все программы этого приложения.

И запомни разницу в терминах, за неё цепляются:

- **undefined behavior** — стандарт не обещает ничего: переполнение знакового, выход за границы, use-after-free. Ответа нет.
- **unspecified behavior** — стандарт даёт несколько вариантов и не говорит, какой выберут: порядок вычисления аргументов функции. Программа корректна, но результат может отличаться.
- **implementation-defined** — вариант тоже свой у каждой платформы, но компилятор обязан его задокументировать: размер `int`, знаковость `char`, сдвиг вправо отрицательного числа. Ответ есть — он просто не один на всех.

Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать

Отдельный, неочевидный вид UB. Правило **strict aliasing** («строгое правило псевдонимов», C11 6.5, пункт 7) гласит: к объекту можно обращаться только через его собственный тип (с точностью до `signed`/`unsigned` и `const`) или через символьный тип. Прочитать или записать `float`

через `int *` — UB. Поэтому компилятор считает, что `int *` и `float *` никогда не указывают на одно место, и оптимизирует, опираясь на это.

```
float f = 1.0f;
int *p = (int *)&f;    /* смотрим на float как на int */
*p = 0;                /* нарушение strict aliasing — UB */
```

Так часто пытаются «достать биты» числа. Что из этого выходит, видно на маленькой программе:

```
#include <stdio.h>

int f(int *ip, float *fp)
{
    *ip = 1;
    *fp = 0.0f;      /* компилятор считает: ip и fp — разная память */
    return *ip;
}

int main(void)
{
    int x;
    printf("%d\n", f(&x, (float *)&x));
    return 0;
}
```

Оба указателя смотрят на один и тот же `x`. Запись `0.0f` кладёт туда четыре нулевых байта, значит, `*ip` после неё должен быть 0. Запуск:

```
-O0:  0
-O2:  1
```

С `-O2` clang рассудил: «`fp` указывает на `float`, `ip` — на `int`, по правилу они не пересекаются, значит, `*ip` всё ещё 1», — и вернул единицу, не перечитывая память. Ни одного предупреждения при этом не было, даже с `-Wall -Wextra`.

Правильный способ посмотреть на биты — `memcpy` (или `union`, это C11 тоже разрешает):

```
int bits;
memcpy(&bits, &f, sizeof bits);   /* законно: memcpy работает с байтами */
```

Размеры тут должны совпадать: на этом маке и `int`, и `float` — по 4 байта. Надёжнее взять `uint32_t` из `<stdint.h>`, у него размер зафиксирован.

Исключение из правила — символьные типы (`char`, `signed char`, `unsigned char`): через них **можно** смотреть на байты чего угодно. Обобщённый стек из задачи 3 ни разу не обращается к элементу через чужой тип: он двигает байты `memcpy` и шагает по буферу через `char *` — поэтому с ним всё законно.

Каверза 3. `volatile` — зачем он нужен

`volatile` говорит компилятору: «эта переменная может измениться **в любой момент** способом, которого ты не видишь, — не кэшируй её, не выкидывай обращения, читай каждый раз заново».

Где это критично:

```
volatile int *reg = (volatile int *)0x40000000;  /* аппаратный регистр */
while (*reg == 0) { }   /* ждём, пока железо изменит регистр */
```

Без `volatile` компилятор видит, что внутри цикла `*reg` никто не меняет, и вправе прочитать его один раз. Я проверил на обычной глобальной переменной `int flag;` и цикле `while (flag == 0) { }`: clang с `-O2` выкинул цикл **целиком** — функция сразу возвращается. Стандарт это разрешает (C11 6.8.5, пункт 6): цикл без ввода-вывода, без `volatile` и без атомарных операций компилятор вправе считать завершающимся. С `volatile int flag` в ассемблере честный цикл: чтение памяти, сравнение с нулём, переход назад. Ровно это `volatile` и обещает: каждое обращение в коде — настоящее обращение к памяти.

Второй классический случай — переменная, меняемая в обработчике сигнала:

```
volatile sig_atomic_t stop = 0;   /* флаг «пора остановиться» */
```

Чего `volatile` **не** делает: он **не** обеспечивает потокобезопасность и атомарность. Для многопоточности нужны `_Atomic` / мьютексы, а не `volatile`. Это частая путаница на собеседовании.

Каверза 4. Две роли `static`

Слово `static` в Си означает **разные вещи** в зависимости от места.

1. Внутри функции — переменная получает **статический срок жизни**: она существует всё время работы программы (а не создаётся заново при каждом вызове) и сохраняет значение между вызовами:

```
int next_id(void)
{
    static int id = 0;   /* создаётся один раз, помнит значение */
    return ++id;
```

```
}
/* вызовы вернут 1, 2, 3, ... */
```

Три вызова подряд в `main` печатают `1 2 3` — проверено. Инициализатор `= 0` срабатывает **один раз**, причём ещё до запуска `main`, а не при первом вызове функции (C11 6.2.4, пункт 3). Поэтому он обязан быть константой: `static int id = f();` в Си не соберётся. Память — не на стеке, а в статической области.

2. На уровне файла (у глобальной переменной или функции) — `static` ограничивает видимость **этим файлом** (внутренняя компоновка). Из других `.c`-файлов к ней не достучаться:

```
static int counter;          /* виден только в этом файле */
static void helper(void) { } /* приватная функция модуля */
```

Это способ скрыть детали реализации — аналог `private`. Без `static` глобальные имена видны всей программе, и легко получить конфликт имён при сборке из нескольких файлов.

Каверза 5. Stack vs heap и причины segfault

Стандарт Си слов «стек» и «куча» не знает — он говорит о сроках жизни объектов: автоматическом (локальные переменные), статическом, выделенном (`malloc`). Но на всех обычных платформах это устроено так, и на собесе говорят именно этими словами. Новичку важны две области:

- **Стек** (stack) — здесь живут локальные переменные и кадры вызовов функций. Выделяется и освобождается **автоматически**, быстро, но размер ограничен (обычно несколько МБ). Бесконечная рекурсия его переполняет — **stack overflow** (см. главу 9).
- **Куча** (heap) — память от `malloc`/`calloc`/`realloc`. Большая, живёт, пока сам не вызовешь `free`. За ней следишь **ты**: забыл `free` — утечка; освободил дважды — UB.

```
int arr[1000000];     /* на стеке — рискуешь переполнить стек */
int *arr = malloc(1000000 * sizeof(int));  /* на куче — надёжнее */
```

Цифры не абстрактные. На этой машине лимит стека главного потока — 8192 КБ (проверяется командой `ulimit -s`), то есть 8 388 608 байт. Я проверил программу, которая первой строкой `main` печатает «вошли в main», потом заполняет локальный массив и печатает его последний элемент:

```
int arr[1000000]   4 000 000 байт    работает
int arr[2000000]   8 000 000 байт    работает (впритык)
int arr[4000000]  16 000 000 байт    падает, код выхода 139
```

139 — это 128 плюс сигнал 11, `SIGSEGV`. Причём на 4 миллионах программа не успела напечатать даже «вошли в main»: падает она не на обращении к элементу, а на самом входе в функцию, когда резервируется место под локальные переменные. Отсюда правило: всё, что больше десятков килобайт, — на куче.

Segmentation fault («сегфолт») — аварийное завершение при обращении к памяти, которая тебе не принадлежит. Типичные причины:

- разыменование `NULL` или «висячего» указателя (после `free`);
- выход далеко за границы массива;
- запись в строковый литерал (на macOS это приходит как `SIGBUS`, код 138, — см. Приложение Б, каверза 1);
- переполнение стека глубокой рекурсией.

Сегфолт — это ещё везение: программа хотя бы падает сразу. Хуже, когда UB тихо портит данные.

Каверза 6. Выравнивание и padding: что выведет `sizeof`

```
struct A {
    char  c;    /* 1 байт */
    int   i;    /* 4 байта */
    char  d;    /* 1 байт */
};
printf("%zu\n", sizeof(struct A));   /* НЕ 6, а обычно 12 */
```

Проверим на этой машине. Заодно попросим компилятор показать смещения полей — их даёт макрос `offsetof` из `<stddef.h>`:

```
printf("sizeof(struct A) = %zu\n", sizeof(struct A));
printf("offsetof c=%zu i=%zu d=%zu\n",
       offsetof(struct A, c),
       offsetof(struct A, i),
       offsetof(struct A, d));
printf("_Alignof(int) = %zu\n", _Alignof(int));
```

```
sizeof(struct A) = 12
offsetof c=0 i=4 d=8
_Alignof(int) = 4
```

Почему не `1 + 4 + 1 = 6`? Процессор читает данные эффективнее, когда они лежат по «круглым» адресам: `int` (4 байта) должен начинаться с адреса, кратного 4. Это **выравнивание** (alignment). Чтобы его соблюсти, компилятор вставляет невидимые **байты-заполнители** (padding):

```
[ c ][pad][pad][pad][  i (4)  ][ d ][pad][pad][pad]
  0    1    2    3    4 5 6 7    8    9   10  11
```

- после `c` (адрес 0) — 3 байта padding, чтобы `i` встал на адрес 4;
- `d` на адресе 8;
- в конце ещё 3 байта, чтобы **весь размер** структуры был кратен выравниванию самого «требовательного» поля (`int`, 4). Это нужно, чтобы в массиве структур каждая следующая тоже была выровнена.

Итого 12. А если переставить поля по убыванию размера — padding почти исчезнет:

```
struct B {
    int   i;    /* 4 */
    char  c;    /* 1 */
    char  d;    /* 1 */
};              /* sizeof == 8: i, c, d, и 2 байта хвоста */
```

И это тоже проверено запуском:

```
sizeof(struct B) = 8
```

Правило: **группируй поля от больших к меньшим** — структура займёт меньше. Здесь мы сэкономили треть размера, просто переставив поля; на массиве из миллиона структур это 4 МБ.

Важная оговорка: числа 12 и 8 — не гарантия стандарта, а implementation-defined поведение. Стандарт (C11 6.7.2.1) лишь требует, чтобы поля шли в памяти в порядке объявления, чтобы перед первым полем padding не было (его смещение всегда 0) и чтобы выравнивание соблюдалось; сколько именно padding вставить между полями и в конце, решает ABI платформы — свод правил, по которым компиляторы одной системы раскладывают данные. Узнать выравнивание типа можно через `_Alignof(int)` (здесь 4), а смещение поля — через `offsetof(struct A, i)` из `<stddef.h>`. Это и есть правильный ответ на собесе: «на x86-64 будет 12, а вообще — считается по правилам выравнивания, вот они».

Что спросят дальше. «Как убрать padding?» Нестандартными расширениями: `#pragma pack(1)` или `__attribute__((packed))`. Но платить придётся скоростью — процессор читает невыровненные данные медленнее, а на некоторых архитектурах это вообще падение. Упаковку применяют, когда структура ложится в бинарный формат файла или сетевого пакета, где раскладка задана извне.

Каверза 7. Опасный макрос `#define SQR(x) x*x`

```
#define SQR(x) x*x
```

Препроцессор делает **текстовую подстановку**, не понимая арифметики. Отсюда два бага.

Баг 1 — приоритет операций. `SQR(a + b)` развернётся в `a + b*a + b`, то есть `a + (b*a) + b`, а вовсе не $(a+b)^2$:

```
int r = SQR(a + b);   /* a + b*a + b — НЕ то, что хотели */
```

Подставим числа: при `a = 2`, `b = 3` мы ждём $(2+3)^2 = 25$, а получаем `2 + 3*2 + 3 = 11`. Проверка запуском:

```
SQR(a+b) = 11, a + b*a + b = 11
```

Ошибка на 14 — и ни одного предупреждения от компилятора: с точки зрения языка выражение совершенно законно.

Лечится скобками вокруг **каждого аргумента и всего выражения**:

```
#define SQR(x) ((x) * (x))
```

Теперь `SQR(a + b)` → `((a + b) * (a + b))` — верно.

Баг 2 — двойное вычисление аргумента. Даже с правильными скобками `SQR(x)` подставляет `x` **дважды**. Если аргумент имеет побочный эффект, он сработает два раза:

```
int i = 3;
int r = SQR(i++);     /* ((i++) * (i++)) — i++ два раза */
```

Это вдобавок UB: `i` меняется дважды без упорядочивания (см. каверзу 9), поэтому даже «вырастет дважды» обещать нельзя. clang с `-Wall` замечает и это, сквозь макрос: `multiple unsequenced modifications to 'i' [-Wunsequenced]`. От этого скобки не спасают — лекарство только одно: **не передавай в макросы выражения с побочными эффектами**, а лучше используй `static inline`-функцию:

```
static inline int sqr(int x) { return x * x; }
```

Функция вычислит аргумент один раз и уважает типы. В современном Си макрос для такого уже почти не нужен.

Каверза 8. Идиома `do { ... } while(0)` в макросах

Как написать макрос из нескольких инструкций, чтобы он вёл себя как одна? Наивно:

```c
#define SWAP(a, b) { int t = a; a = b; b = t; }
```

Сломается в `if`-без-скобок из-за лишней `;`:

```c
if (x) SWAP(p, q); else foo();   /* ; после } разрывает if/else — ошибка */
```

Это не теория — компилятор действительно отказывается это собирать:

```
error: expected expression
    if (x) SWAP(p, q); else foo();
                       ^
```

Макрос развернулся в блок `{ ... }`, а точка с запятой после него стала **отдельным пустым оператором**. Получилось, что у `if` уже два оператора, и `else` повис в воздухе.

Канонический приём — обернуть тело в `do { ... } while(0)`:

```c
#define SWAP(a, b) do { int t = (a); (a) = (b); (b) = t; } while (0)
```

Теперь `SWAP(p, q);` — это один корректный оператор с точкой с запятой на конце, и он спокойно живёт в `if/else`. `while(0)` гарантирует ровно один проход, а компилятор с оптимизацией выкинет фиктивный цикл. Это стандартная идиома многострочных макросов в Си.

Каверза 9. Порядок вычисления аргументов не определён

```c
int i = 1;
printf("%d %d\n", i++, i++);   /* что напечатается? */
```

Ответ: **неизвестно**, и «правильного числа» тут не существует. Тут наложились два разных сорта неопределённости, и на собесе полезно различать их.

Первый: порядок вычисления аргументов функции стандарт не задаёт вовсе — слева направо, справа налево или вперемешку. Это **unspecified behavior**: сам по себе он программу не ломает, просто результат может отличаться от компилятора к компилятору.

Второй, куда серьёзнее: два изменения одной переменной, не упорядоченные между собой, — это **undefined behavior** (C11 6.5, пункт 2). Вычисления аргументов функции друг относительно друга не упорядочены, так что два `i++` в одном вызове — ровно этот случай. В старых стандартах то же правило формулировали через «точки следования». Программа некорректна целиком, и обсуждать её вывод бессмысленно.

Компилятор это видит и говорит прямо:

```
warning: multiple unsequenced modifications to 'i' [-Wunsequenced]
    printf("%d %d\n", i++, i++);
                      ^    ~~
```

Если всё-таки запустить, clang на этой машине печатает `1 2` — и с `-O0`, и с `-O2`. Но это не ответ: другой компилятор имеет полное право выдать `2 1`, `2 2` или что угодно ещё. Правильный ответ на собеседовании — «так писать нельзя, это UB», и в подтверждение — `-Wunsequenced`.

То же относится к выражениям вроде:

```
a[i] = i++;          /* UB: и читаем, и меняем i без упорядочивания */
foo(i++, i++);       /* UB */
x = f() + g();       /* порядок вызовов f и g не задан */
```

Последняя строка — не UB: сами вызовы не перемешиваются, просто неизвестно, какой будет первым (unspecified). Если `f()` и `g()` имеют побочные эффекты и зависят от порядка — результат непредсказуем. Вывод: **не меняй одну переменную несколько раз в одном выражении** и не полагайся на порядок вычисления аргументов. Разнеси по разным строкам.

Каверза 10. Endianness — порядок байтов

Многобайтовое число (`int` — 4 байта) можно хранить в памяти двумя способами:

- **little-endian** — младший байт первым (x86; ARM умеет оба порядка, но почти всегда работает в этом; значит, и большинство ПК и телефонов);
- **big-endian** — старший байт первым (некоторые сетевые протоколы, старые процессоры; «сетевой порядок» — big-endian).

Проверить порядок на своей машине можно так:

```
int x = 1;
unsigned char *p = (unsigned char *)&x;   /* смотрим на байты числа */
if (p[0] == 1) {
    printf("little-endian\n");   /* младший байт (1) лежит первым */
} else {
    printf("big-endian\n");
}
```

Число `1` в памяти это байты `01 00 00 00` (little) или `00 00 00 01` (big). Читаем первый байт через `unsigned char *` — это **разрешено** strict aliasing (см. каверзу 2). Если там `1` — машина little-endian.

На этой машине (x86-64) программа, которая заодно печатает все четыре байта, выдаёт:

```
little-endian
байты числа 1: 01 00 00 00
```

Единица лежит первой — младший байт впереди. Это и есть little-endian. Тот же код на big-endian-машине напечатал бы `00 00 00 01`, и никакой «правильный» ответ тут не существует: порядок байтов — свойство платформы, а не языка.

Когда это важно: чтение бинарных файлов и сетевых пакетов, обмен данными между разными машинами. Поэтому в сети числа гоняют в фиксированном порядке через `htons`/`htonl` (host-to-network) и обратно `ntohs`/`ntohl`.

Ошибки, на которых сыплются в этих задачах

Всё перечисленное — из настоящих решений, а не выдумано для примера.

- **Опорный элемент = последний.** На уже отсортированном входе quicksort скатывается в $O(n^2)$, а на 200 000 элементов переполняет стек (мы это видели). Случайный опорный или медиана трёх обязательны, если вход может быть отсортирован; от массива одинаковых чисел спасает только другое разбиение (Хоара или трёхпутевое).
- **`(lo + hi) / 2` вместо `lo + (hi - lo) / 2`.** Переполнение знакового `int` — UB, а не «отрицательный индекс».
- **Арифметика на `void *`.** Стандартом не определена; для побайтового хода бери `char *` (он же единственный тип, которым законно смотреть на чужие байты — см. каверзу 2).
- **`realloc` прямо в поле структуры.** `s->data = realloc(s->data, …)` теряет старый блок при неудаче. Клади результат во временную переменную и проверяй.
- **`long` вместо `long long` в защите от переполнения.** На платформах, где `long` — 4 байта, «защита» сама переполняется.
- **`char` без приведения к `unsigned char` в хеш-функции.** Байты с кодом больше 127 становятся отрицательными там, где `char` знаковый, и одна и та же строка получает разный хеш на разных платформах.
- **Хранение чужого указателя как ключа.** Хеш-таблица обязана копировать ключ себе (`malloc` + `strcpy`), иначе он протухнет.
- **Два индекса `head`/`tail` в кольцевом буфере без счётчика.** Состояния «пусто» и «полно» становятся неразличимы.
- **Односвязный список в LRU.** Удаление узла из середины превращается в O(n), и обещанное O(1) исчезает.
- **Фиксированное число корзин.** Хеш-таблица на 16 корзин без рехеширования — это O(n) на поиск, как только записей становится много. Обещать «O(1)» можно, только пока коэффициент загрузки ограничен.
- **`free` общего хвоста дважды** при пересечении списков — double free.

- **Макрос без скобок вокруг аргументов** и с побочными эффектами внутри.
- **`volatile` вместо синхронизации.** Атомарности он не даёт.

Тренировочные задачи

1. Перепиши `partition` из задачи 1 по схеме Хоара (два бегунка навстречу) и добавь выбор опорного «медиана трёх». Проверь на уже отсортированном массиве из 100 000 элементов, что глубина рекурсии не взлетела.
2. Сделай quicksort нерекурсивной: заведи явный стек пар `(lo, hi)` и всегда клади в него **бóльшую** половину, а меньшую обрабатывай циклом. Так глубина стека остаётся O(log n) даже в худшем случае.
3. Добавь в обобщённый стек из задачи 3 функцию `void stack_foreach(const Stack *s, void (*fn)(const void *))` — печать элементов через переданный обработчик. Проверь на `int` и на `double`.
4. Обобщи `my_itoa` на любое основание от 2 до 16: `void my_itoa_base(int n, int base, char *buf)`. Не забудь про `INT_MIN` и про буквы `a–f`.
5. Допиши в хеш-таблицу из задачи 7 рехеширование: когда число записей превысит число корзин вдвое, удвой массив корзин и перераспредели записи. Убедись под `-fsanitize=address`, что ничего не потерялось и не утекло.
6. Сделай кольцевой буфер динамическим: вместо `int data[CAP]` — буфер от `malloc` с функцией `ring_grow`. Аккуратно перенеси элементы — при завороте они лежат в массиве двумя кусками.
7. Возьми `struct A` из каверзы 6 и напиши программу, которая печатает `sizeof` и `offsetof` для всех полей. Переставляй поля и смотри, как меняется размер. Найди перестановку с минимальным padding.

Ответы к тренировочным задачам

Все решения собраны с `-std=c11 -Wall -Wextra -O2` без предупреждений и прогнаны под `-fsanitize=address,undefined`; те, где есть `malloc`, ещё и через `leaks` — чисто. Выводы ниже — реальные.

Задача 1. Хоар и медиана трёх:

```
#include <stdio.h>
#include <stdlib.h>

static int depth, max_depth;    /* для замера глубины рекурсии */
```

```c
static void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Медиана трёх: после неё a[lo] <= a[mid] <= a[hi]. */
static int median_of_three(int a[], int lo, int hi)
{
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < a[lo]) swap(&a[mid], &a[lo]);
    if (a[hi] < a[lo]) swap(&a[hi], &a[lo]);
    if (a[hi] < a[mid]) swap(&a[hi], &a[mid]);
    return a[mid];
}

/* Разбиение Хоара: после него a[lo..j] <= pivot <= a[j+1..hi]. */
static int partition_hoare(int a[], int lo, int hi)
{
    int pivot = median_of_three(a, lo, hi);
    int i = lo - 1;
    int j = hi + 1;
    for (;;) {
        do { i++; } while (a[i] < pivot);   /* слева ищем >= pivot */
        do { j--; } while (a[j] > pivot);   /* справа ищем <= pivot */
        if (i >= j) {
            return j;
        }
        swap(&a[i], &a[j]);
    }
}

static void quicksort_hoare(int a[], int lo, int hi)
{
    if (lo >= hi) {
        return;
    }
```

```
    depth++;
    if (depth > max_depth) max_depth = depth;
    int p = partition_hoare(a, lo, hi);
    quicksort_hoare(a, lo, p);          /* p входит в левую часть */
    quicksort_hoare(a, p + 1, hi);
    depth--;
}

static void test(const char *name, int *a, int n)
{
    max_depth = 0;
    quicksort_hoare(a, 0, n - 1);
    for (int i = 1; i < n; i++) {
        if (a[i - 1] > a[i]) {
            printf("%s: НЕ отсортировано!\n", name);
            return;
        }
    }
    printf("%s: глубина %d\n", name, max_depth);
}

int main(void)
{
    int n = 100000;
    int *a = malloc((size_t)n * sizeof *a);
    if (a == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) a[i] = i;
    test("отсортирован", a, n);
    for (int i = 0; i < n; i++) a[i] = n - i;
    test("по убыванию", a, n);
    for (int i = 0; i < n; i++) a[i] = 7;
    test("одни семёрки", a, n);
    srand(1);
    for (int i = 0; i < n; i++) a[i] = rand();
    test("случайный", a, n);
    free(a);
```

```
    return 0;
}
```

```
отсортирован: глубина 17
по убыванию: глубина 17
одни семёрки: глубина 17
случайный: глубина 32
```

Вместо 100 000 уровней (и падения) — 17: это $\lceil \log_2 100\,000 \rceil$, идеальное деление пополам. Медиана трёх на отсортированном входе всегда выбирает настоящую середину. Одни семёрки тоже делятся пополам: бегунки Хоара останавливаются на **равных** опорному (`<` и `>` в циклах `do`) и меняют их местами, так что встречаются посередине. Обрати внимание на два отличия от Ломуто: опорный не встаёт на своё место, поэтому левая часть — `lo..p` **включая** `p`; и циклы `do { } while` не выбегают за границы — после медианы трёх `a[lo] <= pivot <= a[hi]`, и эти элементы остановят бегунков.

Задача 2. Quicksort без рекурсии:

```
#include <stdio.h>
#include <stdlib.h>

static void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

static int partition(int a[], int lo, int hi)   /* Ломуто, задача 1 */
{
    int pivot = a[hi];
    int i = lo - 1;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);
    return i + 1;
```

```c
}

/* Без рекурсии: бо́льшую часть — в стек, меньшую — сразу дальше. */
static int quicksort_iter(int a[], int n)
{
    int stack[64][2];           /* 64 пар хватит: глубина <= log2(n) */
    int top = 0, max_top = 0;
    stack[top][0] = 0;
    stack[top][1] = n - 1;
    top++;
    while (top > 0) {
        top--;
        int lo = stack[top][0];
        int hi = stack[top][1];
        while (lo < hi) {
            int p = partition(a, lo, hi);
            if (p - lo < hi - p) {          /* левая часть меньше */
                stack[top][0] = p + 1;      /* правую — в стек */
                stack[top][1] = hi;
                hi = p - 1;                 /* левую — продолжаем */
            } else {
                stack[top][0] = lo;         /* левую — в стек */
                stack[top][1] = p - 1;
                lo = p + 1;                 /* правую — продолжаем */
            }
            top++;
            if (top > max_top) max_top = top;
        }
    }
    return max_top;
}

int main(void)
{
    int n = 100000;
    int *a = malloc((size_t)n * sizeof *a);
    if (a == NULL) {
        return 1;
```

```
    }
    for (int i = 0; i < n; i++) a[i] = i;
    printf("отсортирован: стек %d\n", quicksort_iter(a, n));
    srand(1);
    for (int i = 0; i < n; i++) a[i] = rand();
    printf("случайный: стек %d\n", quicksort_iter(a, n));
    for (int i = 1; i < n; i++) {
        if (a[i - 1] > a[i]) {
            printf("НЕ отсортировано\n");
        }
    }
    free(a);
    return 0;
}
```

```
отсортирован: стек 1
случайный: стек 10
```

Почему глубина стека не больше $\log_2 n$: мы всегда продолжаем с **меньшей** частью, а она не больше половины текущего отрезка. Каждая лежащая в стеке пара «сторожит» отрезок, который начался в предыдущем, вдвое большем, — вдвое делить 100 000 можно не больше 17 раз. Поэтому 64 пар хватит на любой `int`-массив. Время на отсортированном входе осталось $O(n^2)$ — у меня 2.95 с на 100 000 (со старым разбиением Ломуто иначе и быть не может), зато программа больше не падает.

Задача 3. Добавляем к стеку из задачи 3 (его функции без изменений):

```
/* Обойти стек от дна к вершине, вызывая fn для каждого элемента. */
void stack_foreach(const Stack *s, void (*fn)(const void *))
{
    for (size_t i = 0; i < s->count; i++) {
        fn(s->data + i * s->elem_size);   /* адрес i-го элемента */
    }
}

static void print_int(const void *p)
{
    printf("%d ", *(const int *)p);
}
```

```c
static void print_double(const void *p)
{
    printf("%.2f ", *(const double *)p);
}

int main(void)
{
    Stack si, sd;
    if (!stack_init(&si, sizeof(int))) {
        return 1;
    }
    if (!stack_init(&sd, sizeof(double))) {
        stack_free(&si);
        return 1;
    }
    for (int i = 1; i <= 6; i++) {
        double d = i / 2.0;
        if (!stack_push(&si, &i) || !stack_push(&sd, &d)) {
            printf("Не хватило памяти\n");
            break;
        }
    }
    stack_foreach(&si, print_int);
    printf("\n");
    stack_foreach(&sd, print_double);
    printf("\n");
    stack_free(&si);
    stack_free(&sd);
    return 0;
}
```

```
1 2 3 4 5 6
0.50 1.00 1.50 2.00 2.50 3.00
```

`stack_foreach` не знает тип элемента — она передаёт обработчику **адрес** i-го элемента, а приводит его к нужному типу уже обработчик: `*(const int *)p`.

Задача 4.

```c
#include <stdio.h>
#include <limits.h>

/* int -> строка в системе счисления base (2..16).
   buf: не меньше 34 байт (знак + 32 двоичные цифры + '\0'). */
void my_itoa_base(int n, int base, char *buf)
{
    const char *digits = "0123456789abcdef";
    char tmp[33];
    int i = 0;
    int neg = 0;

    if (base < 2 || base > 16) {
        buf[0] = '\0';                  /* такое основание не умеем */
        return;
    }
    long long value = n;                /* -INT_MIN в int не влезает */
    if (value < 0) {
        neg = 1;
        value = -value;
    }
    do {
        tmp[i++] = digits[value % base];
        value /= base;
    } while (value > 0);

    int j = 0;
    if (neg) {
        buf[j++] = '-';
    }
    while (i > 0) {
        buf[j++] = tmp[--i];
    }
    buf[j] = '\0';
}

int main(void)
{
```

```
    char buf[34];
    int vals[] = {0, 255, -255, INT_MAX, INT_MIN};
    int bases[] = {2, 8, 10, 16};
    for (int v = 0; v < 5; v++) {
        for (int b = 0; b < 4; b++) {
            my_itoa_base(vals[v], bases[b], buf);
            printf("%s ", buf);
        }
        printf("\n");
    }
    return 0;
}
```

```
0 0 0 0
11111111 377 255 ff
-11111111 -377 -255 -ff
11111111111111111111111111111111 17777777777 2147483647 7fffffff
-10000000000000000000000000000000 -20000000000 -2147483648 -80000000
```

Самая длинная запись — `INT_MIN` в двоичной: знак и 32 цифры, плюс `'\0'` — 34 байта. Отрицательные числа здесь печатаются со знаком минус, как математическая запись, а не как биты дополнительного кода.

Задача 5. Хеш-таблица с рехешированием:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Entry {
    char *key;
    int count;
    struct Entry *next;
} Entry;

typedef struct {
    Entry **buckets;
    size_t nbuckets;    /* число корзин, степень двойки */
    size_t size;        /* число записей */
} HashTable;
```

```
unsigned long hash_str(const char *s)        /* djb2, как в задаче 7 */
{
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*s++) != 0) {
        h = h * 33u + (unsigned long)c;
    }
    return h;
}

static Entry **alloc_buckets(size_t n)
{
    Entry **b = malloc(n * sizeof *b);
    if (b != NULL) {
        for (size_t i = 0; i < n; i++) {
            b[i] = NULL;
        }
    }
    return b;
}

int table_init(HashTable *t)
{
    t->nbuckets = 4;
    t->size = 0;
    t->buckets = alloc_buckets(t->nbuckets);
    return t->buckets != NULL;
}

/* Удвоить корзины и переложить записи. Узлы не копируем —
   только перецепляем. 0 — нет памяти (таблица остаётся старой). */
static int rehash(HashTable *t)
{
    size_t nn = t->nbuckets * 2;
    Entry **nb = alloc_buckets(nn);
    if (nb == NULL) {
        return 0;
```

```
    }
    for (size_t i = 0; i < t->nbuckets; i++) {
        Entry *e = t->buckets[i];
        while (e != NULL) {
            Entry *next = e->next;
            size_t idx = hash_str(e->key) % nn;
            e->next = nb[idx];
            nb[idx] = e;
            e = next;
        }
    }
    free(t->buckets);
    t->buckets = nb;
    t->nbuckets = nn;
    return 1;
}

int table_inc(HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % t->nbuckets;
    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {
            e->count++;
            return 1;
        }
    }
    if (t->size + 1 > 2 * t->nbuckets) {    /* α превысит 2 — растём */
        if (!rehash(t)) {
            return 0;
        }
        idx = hash_str(key) % t->nbuckets;  /* корзина поменялась */
    }
    Entry *e = malloc(sizeof(Entry));
    if (e == NULL) {
        return 0;
    }
    e->key = malloc(strlen(key) + 1);
    if (e->key == NULL) {
```

```
        free(e);
        return 0;
    }
    strcpy(e->key, key);
    e->count = 1;
    e->next = t->buckets[idx];
    t->buckets[idx] = e;
    t->size++;
    return 1;
}

int table_get(const HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % t->nbuckets;
    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {
            return e->count;
        }
    }
    return 0;
}

void table_free(HashTable *t)
{
    for (size_t i = 0; i < t->nbuckets; i++) {
        Entry *e = t->buckets[i];
        while (e != NULL) {
            Entry *next = e->next;
            free(e->key);
            free(e);
            e = next;
        }
    }
    free(t->buckets);
    t->buckets = NULL;
}

int main(void)
```

```
{
    HashTable t;
    if (!table_init(&t)) {
        return 1;
    }
    char word[16];
    for (int round = 0; round < 3; round++) {   /* 1000 слов по 3 раза */
        for (int i = 0; i < 1000; i++) {
            snprintf(word, sizeof word, "w%d", i);
            if (!table_inc(&t, word)) {
                printf("Не хватило памяти\n");
                table_free(&t);
                return 1;
            }
        }
    }
    int bad = 0;
    for (int i = 0; i < 1000; i++) {
        snprintf(word, sizeof word, "w%d", i);
        if (table_get(&t, word) != 3) {
            bad++;
        }
    }
    printf("записей %zu, корзин %zu, с ошибкой %d\n",
           t.size, t.nbuckets, bad);
    table_free(&t);
    return 0;
}
```

```
записей 1000, корзин 512, с ошибкой 0
```

Стартуем с 4 корзин; каждый раз, когда записей стало бы больше удвоенного числа корзин, число корзин удваивается: 4, 8, 16, … 512. При рехешировании записи не копируются — узлы просто перецепляются в новые корзины, поэтому `malloc` нужен один, на новый массив корзин. После рехеша индекс новой записи пересчитываем: корзин стало больше. Все 1000 слов нашлись с правильным счётчиком 3.

Задача 6. Растущее кольцо:

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    int cap;      /* сколько мест */
    int head;
    int count;
} Ring;

int ring_init(Ring *r, int cap)
{
    r->data = malloc((size_t)cap * sizeof *r->data);
    r->cap = cap;
    r->head = 0;
    r->count = 0;
    return r->data != NULL;
}

/* Вдвое больше места. Элементы переносим по порядку, от головы:
   при завороте они лежат двумя кусками, а в новом буфере — подряд. */
static int ring_grow(Ring *r)
{
    int ncap = r->cap * 2;
    int *nd = malloc((size_t)ncap * sizeof *nd);
    if (nd == NULL) {
        return 0;
    }
    for (int i = 0; i < r->count; i++) {
        nd[i] = r->data[(r->head + i) % r->cap];
    }
    free(r->data);
    r->data = nd;
    r->cap = ncap;
    r->head = 0;
    return 1;
}
```

```
int ring_push(Ring *r, int value)
{
    if (r->count == r->cap && !ring_grow(r)) {
        return 0;                       /* полон и расти некуда */
    }
    r->data[(r->head + r->count) % r->cap] = value;
    r->count++;
    return 1;
}

int ring_pop(Ring *r, int *out)
{
    if (r->count == 0) {
        return 0;
    }
    *out = r->data[r->head];
    r->head = (r->head + 1) % r->cap;
    r->count--;
    return 1;
}

int main(void)
{
    Ring r;
    int x;
    if (!ring_init(&r, 4)) {
        return 1;
    }
    for (int v = 1; v <= 4; v++) ring_push(&r, v);
    ring_pop(&r, &x);
    ring_pop(&r, &x);                   /* забрали 1 и 2 */
    for (int v = 5; v <= 9; v++) {      /* 5, 6 — заворот; 7 — рост */
        if (!ring_push(&r, v)) {
            printf("Не хватило памяти\n");
        }
    }
    printf("мест %d:", r.cap);
    while (ring_pop(&r, &x)) {
```

```
        printf(" %d", x);
    }
    printf("\n");
    free(r.data);
    return 0;
}
```

```
мест 8: 3 4 5 6 7 8 9
```

После двух `pop` голова стоит на индексе 2, `5` и `6` завернулись в индексы 0 и 1 — элементы лежат двумя кусками: `[5 6 3 4]`. На `7` кольцо полное, и `ring_grow` переносит элементы **в порядке очереди** (`(head + i) % cap`), так что в новом буфере они лежат подряд `[3 4 5 6 …]`, и `head` снова 0. Скопировать старый массив «как есть» было бы ошибкой: заворот сломал бы порядок.

Задача 7.

```
#include <stdio.h>
#include <stddef.h>

struct A1 { char c; int i; char d; };
struct A2 { char c; char d; int i; };
struct A3 { int i; char c; char d; };

int main(void)
{
    printf("c,i,d: sizeof %zu, c=%zu i=%zu d=%zu\n", sizeof(struct A1),
           offsetof(struct A1, c), offsetof(struct A1, i),
           offsetof(struct A1, d));
    printf("c,d,i: sizeof %zu, c=%zu d=%zu i=%zu\n", sizeof(struct A2),
           offsetof(struct A2, c), offsetof(struct A2, d),
           offsetof(struct A2, i));
    printf("i,c,d: sizeof %zu, i=%zu c=%zu d=%zu\n", sizeof(struct A3),
           offsetof(struct A3, i), offsetof(struct A3, c),
           offsetof(struct A3, d));
    return 0;
}
```

```
c,i,d: sizeof 12, c=0 i=4 d=8
c,d,i: sizeof 8, c=0 d=1 i=4
i,c,d: sizeof 8, i=0 c=4 d=5
```

Минимум — 8 байт, и его дают обе раскладки, где два `char` стоят рядом: 6 байт полезных данных плюс 2 байта padding (в первом случае между `d` и `i`, во втором — в хвосте). Меньше 8 не выйдет: размер обязан быть кратен выравниванию `int`, то есть 4, а 6 байт данных в 4 не влезают. Остальные три перестановки (`d,c,i`, `d,i,c`, `i,d,c`) зеркальны этим трём.

Итог практикума

Ты прошёл три сборника задач с собеседований — Junior, Middle и Senior — и это достойный финал всей книги. Пробежимся по тому, что осело.

На уровне **кода** ты теперь умеешь не просто «сортировать массив», а реализовать quicksort и merge sort с пониманием их сложности, худших случаев и выбора опорного. Ты собрал **обобщённые** структуры через `void *` и размер элемента, написал хеш-таблицу с разрешением коллизий цепочками, кольцевой буфер, нашёл пересечение списков за O(n+m) и набросал LRU-кэш на связке «двусвязный список + хеш». Это уже не учебные игрушки, а кирпичики, из которых сложены настоящие программы.

На уровне **понимания языка** ты разобрался с тем, что отделяет крепкого Си-разработчика от новичка: что такое UB и почему на него нельзя полагаться, чем опасны strict aliasing и переполнение знакового, зачем нужны `volatile` и две роли `static`, как устроены стек и куча и откуда берётся segfault, почему `sizeof` хитрой структуры больше суммы полей (выравнивание и padding), как не подорваться на макросе `SQR(x)` и зачем макросам идиома `do { } while(0)`, и почему порядок вычисления аргументов и endianness — вещи, о которых нельзя забывать.

Напутствие

Если ты дочитал до этих строк и хотя бы половину задач решил сам — поздравляю, ты уже не новичок. Ты прошёл путь от первого `printf` (глава 1) до LRU-кэша и тонкостей UB. Си не прощает небрежности, и именно поэтому он так многому учит: каждый указатель, каждый `malloc`, каждый байт выравнивания заставляют **понимать**, а не угадывать.

Что делать дальше:

- **пиши код каждый день** — алгоритмы забываются без практики; бери задачи, реализуй, ломай, чини;
- **читай чужой Си** — исходники SQLite, ядра Linux, Redis; там живут лучшие образцы того, как пишут на этом языке;
- **гоняй санитайзеры и предупреждения** — `-Wall -Wextra`, `-fsanitize=address,undefined`, valgrind: пусть инструменты ловят UB за тебя;
- **не бойся низкого уровня** — теперь ты понимаешь, что под капотом, и любой язык повыше будет казаться проще.

Спасибо, что прошёл эту книгу до конца. Главного ты уже добился — научился думать как программист, который **знает, что происходит в памяти**. Остальное — дело практики. Удачи на собеседованиях и в реальных проектах.